MindVideo AI

KERAS 3

بناء الشبكات العصبية والتعلّم العميق

— من الصفر إلى الإنتاج

```
from keras import layers, models

model = models.Sequential([
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

- الأسس النظرية بأسلوب مبسّط
- تطبيقات عملية ومشاريع حقيقية
- من النماذج الأولية إلى النشر والإنتاج
- أفضل الممارسات في Keras 3

م/ أحمد صالح المغـز

2026

إهداء

إلى رفيقة دربي وشريكة عمري،

زوجتي الغالية... التي صبرت على ليالي التدريب الطويلة،

وحمَلت نصف العبء كي يكتمل هذا الكتاب.

وإلى أبنائي الأعزّاء...

أنتم نور عينيّ ودافع سعيي؛

كلّما تعبت، تذكّرتُ أنّ هذا العمل إرثٌ لكم قبل أن يكون كتابًا للناس.

فمنكم تعلّمت أنّ أجمل المعادلات هي التي تُكتب في البيت أولًا.

— م/ أحمد صالح المغز

مقدمة المؤلف

مقدمة المؤلف

> «أعطني سمكةً آكلُ اليوم؛ علّمني الصيد آكلُ العمر» – أمثال صينيّة قديمة

كتبتُ هذا الكتاب بعد اثني عشر عامًا من بناء أنظمة تعلّم عميق في الصناعة: من كشف العيوب في خطوط الإنتاج، إلى أنظمة الرؤية الطبيّة، إلى خدمات لغويّة تُخدِم ملايين الطلبات يوميًّا. وفي كلّ مشروع، تكرّرت المشهدة نفسها: مهندسٌ يجيد تشغيل `model.fit()`، لكنه يتجمّد عند أوّل خطأ غير متوقّع – لأنّ أحدًا لم يشرح له **ماذا يحدث** تحت غطاء الواجهة.

هذا الكتاب ليس دليل استخدام آخر لـ Keras؛ فوثائق Keras الرسمية ممتازة بالفعل. إنه محاولة لسدّ الفجوة التي تتركها الوثائق والبرامج التعليميّة معًا: الفجوة بين «الشيفرة التي تعمل» و«الفهم الذي يجعلك تعرف لماذا تعمل، ومتى ستتوقّف عن العمل».

لماذا Keras 3 تحديدًا؟

في نوفمبر 2023 غيّر Keras هويّته: لم يعد طبقةً فوق TensorFlow، بل أصبح **واجهة موحّدة** فوق أربعة محرّكات – TensorFlow وJAX وPyTorch وOpenVINO. هذه ليست تفصيلة إصدار؛ إنها نقطة تحوّل في صناعة الأدوات. حين تكتب نموذجًا واحدًا ويعمل على هاتفٍ وعلى عنقودٍ من بطاقات GPU، تسقط الحاجز التقليديّ بين «تجربة بحثيّة» و«نظام إنتاجيّ»، ويصبح اختيار المحرّك قرارًا هندسيًّا تؤجّله إلى آخر لحظة ممكنة – وهذا بالضبط ما يمنحك مرونةً نادرة.

كلّ ما في هذا الكتاب مبنيّ على **Keras 3.15.1** و**Python 3.14**، وكلّ اسم واجهة مذكور تحقّقتُ منه من تثبيتٍ فعليّ، لا من ذاكرةٍ أو من توثيقٍ قد يسبق الإصدار.

فلسفة الكتاب: الرياضيات أولًا، والشيفرة تتبع

اخترتُ أسلوبًا قد يكون غير مألوف في الكتب العربية التقنيّة: **المعادلة قبل الكود**. لن أقول لك «استخدم `GlorotNormal` » قبل أن أشرح لماذا تختفي الإشارة عبر عشرين طبقة إذا لم تكن التباينات متوازنة. لن أقول «استخدم `Huber` » قبل أن تحسب معي كم تكلفك شاذّةٌ واحدة في خسارة مربّعة.

السبب عمليٌّ خالص: المهندس الذي يفهم المعادلة يصحّح نموذجه في دقائق؛ والمهندس الذي يحفظ الواجهة يخمّن أيامًا. وقد رأيتُ هذا الفرق يترجم – حرفيًّا – إلى أسابيع من فرق التسليم.

ما الذي يميّز هذا الكتاب؟

1. **43 فصلًا من الأوتور إلى الإنتاج** – تسعة أجزاء تنقلك من مفهوم الأوتور إلى نشر خدمةٍ مُراقَبة، مرورًا بالرؤية الحاسوبيّة واللغة والمحوّلات والنماذج التوليديّة.
2. **أكثر من 250 مثالًا برمجيًّا قابلاً للتشغيل** – كلّ مثال مكتفٍ بذاته ببيانات مُصنَّعة، تعليقاته عربيّة، ومتغيّراته إنجليزيّة، ومعه «النتيجة المتوقعة» التي تتحقّق من سلامة بيئتك.
3. **أخلاقيّة المقارنة**: لا أقول «X أفضل من Y» أبدًا بلا معيار. المقارنات هنا تحمل شروطها: على أيّ بيانات، بأيّ موارد، وبأيّ قياس.
4. **قسم «أخطاء شائعة» في كلّ فصل** – جامعٌ لأخطاء رأيتُها تُهلك مشاريع حقيقيّة: تسريب `validation_split` ، خلط `from_logits` ، حشو تسلسلات بلا `Masking` ، وتقييم GAN بـ PSNR.
5. **ثلاثة مشاريع متكاملة** في الجزء الأخير تنقلك من ملفّات خام إلى خدمة نشرٍ ومراقبة – بمعايير قبول مكتوبة قبل أن يُكتب سطر الشيفرة الأول.

كيف كُتب هذا الكتاب؟

كلّ فصل مرّ بثلاث دورات على الأقل: مسوّدة تشرح الفكرة، ثم إعادة كتابة تُدقّق كلّ اسم واجهة وكلّ رقم، ثم تدقيق لغويّ وتنسيقيّ. معادلات الفصول اختُبرت عدديًّا، وأمثلتُها نُفِّذت، والجداول بُنيت من

قياسات لا من انطباعات. وقبل إخراج النسخة النهائيّة، خضع الكتاب لتدقيقٍ آليّ: ماسحٌ لصحّة الأسماء، وماسحٌ لسلامة الشيفرة النحويّة، وفاحصٌ لسلامة النصّ من أيّ محارف تالفة.

إلى من أوجّه هذا الكتاب؟

- **لمهندس التعلّم الآلي** الذي يعمل يوميًّا على النماذج ويريد الفهم العميق لا الحفظ.
- **للباحث** الذي ينتقل بين TensorFlow وPyTorch وJAX ويحتاج واجهةً واحدة تُحرّر طاقته للفكرة لا للأدوات.
- **للطالب الجامعيّ** في المرحلة النهائيّة أو الدراسات العليا الذي يريد مرجعًا عربيًّا يوازي الكتب الإنجليزيّة في الدقّة.
- **لقياديّ تقنيّ** يريد أن يفهم ما يبنيه فريقه ليتخذ قرارات معماريّة واعية.

إن كنت مبتدئًا تمامًا في بايثون، ابدأ بالفصلين الأولين ولا تستعجل؛ الأساسات هنا متعمّقة عن العادة، وهذا مقصود: ما تُفهمه من الأوتور والتجزئة في الفصول الأولى هو ما سيوفّر عليك أسابيع في الفصل الثامن عشر.

شكر وتقدير

الشكر أولًا لزوجتي وأبنائي الذين تنازلوا عن كثير من وقتي كي يكتمل هذا العمل. ثم لكلّ مهندسٍ وشارك خطأً علنيًّا أو مناقشةً في مستودعات Keras وTensorFlow وPyTorch — فأخطاءكم المُوثَّقة صارت أقسام «أخطاء شائعة» في هذا الكتاب. ولزملائي الذين راجعوا مسوّدات الفصول وأعادوا عليّ تساؤلاتهم: «طبّ لماذا فعلتَ هكذا؟» — سؤال واحد أعاد كتابة فصول كاملة.

كلمة أخيرة

لا شيء في هذا الكتاب سحريّ. كلّ صفحة هي نتيجة سؤالٍ بسيط أُعيد طرحه حتى نفدت الأجوبة السهلة: **لماذا؟** إن خرجت من هذا الكتاب بشيء واحد، فليكن هذه العادة: لا تقبل شيفرةً تعمل قبل أن تعرف لماذا تعمل.

لنبدأ.

م/ أحمد صالح المغز

2026

كيف تستخدم هذا الكتاب

كيف تستخدم هذا الكتاب

> «الكتاب الذي لا تعرف كيف تقرؤه، لا تحتاجه.»

القاعدة الأولى: لا تقرأ بالتتابع

هذا كتاب مرجعي، ودليل مرجعي يُقرأ بالquetête: تفتحه حين تصطدم بمشكلة، وتغلقه حين تحلّ. الفصول مرتّبة منطقيًا – كل فصل يبني على ما قبله – لكن لا أحد يقرأ كتابًا من 900 صفحة سطرًا سطرًا.

إن كنت…	فابدأ بـ	واقرأ بعده
جديد تمامًا على التعلّم العميق	الفصل 1 ثم الفصل 5	الفصل 2 ثم 6 ثم 9
تعرف PyTorch وتحتاج Keras	الفصل 1 وفصل 4	الفصل 12 ثم 34 ثم 37
تعمل على صور	الفصل 18	19، 20، 21، 22، 41
تعمل على نصوص	الفصل 16	25، 27، 29، 43
تعمل على سلاسل زمنية	الفصل 28	34 ثم 42
تنشر نموذجًا في الإنتاج	الفصل 35	36، 37، 38، 40
تريد بناء عتاد مخصّص	الفصل 34	35، 39، 40

القاعدة الثانية: اقرأ الكود قبل النص أحيانًا

في الفصول التي تحتوي أمثلة كثيرة، اقرأ أولًا كتلة الشيفرة كاملة في ذهنك – ماذا تفعل هذه السطور؟ – ثم ارجع إلى الشرح. في الأجزاء التي فيها بُنى معمارية (الفصل 4، الفصل 19، الفصل 29) اتّبع الترتيب العادي: النص أولًا.

القاعدة الثالثة: لا تثق بالكود دون تشغيله

كل مثال في هذا الكتاب **مُختبَر** على Keras 3.15.1 مع Python 3.14. لكن بيئتك قد تكون مختلفة، وقد يكون لديك إصدار أحدث. القاعدة العملية:

1. أنشئ بيئة معزولة (الفصل 2).
2. ثبّت الإصدارات: `pip install "keras[torch]==3.15.1"` .
3. شغّل المثال كما هو قبل أن تعدّله.
4. عدّل **شيئًا واحدًا** في كل مرة، ولاحظ ما يحدث.

> **نصيحة:** احتفظ بدفتر ملاحظات. الفصل 40 (الأخطاء الشائعة) وُلد من ملاحظات كُتبت على شرائط لاصقة أثناء عمل حقيقي.

بنية كل فصل

كل فصل يتبع الهيكل نفسه، حتى تفهم أين تبحث:

القسم	ماذا تجد فيه
قصة مفتاحية	مشكلة واقعية تُطرح في بداية الفصل
أهداف الفصل	ما ستتقنه بعد الانتهاء
الأقسام N.1 … N.5	الشرح: الفكرة ← المعادلة ← الكود ← المقارنة ← مثال محلول
N.6 أخطاء شائعة	أخطاء حقيقية + لماذا تحدث + كيف تصلحها
N.7 تمارين	8-12 تمرينًا متنوّعًا، لكلٍّ منها «المتوقع»
N.8 ملخص	النقاط التي يجب أن تبقى في ذاكرتك

عن المعادلات

المعادلات مكتوبة بصيغة LaTeX، وتُعرض في نسخة HTML وPDF بشكل متجهي واضح. إن كنت تقرأ نسخة DOCX فتظهر المعادلات هناك كصور عالية الدقة – قريبة كفاية للتدوين.

> **ملاحظة:** كل رمز في كل معادلة مشروح في السطر الذي يليها مباشرة. إن مررت على رمز لم تفهمه، لا تتخطَّه – عُد إلى الرمز قبله. الفهم التراكمي هو جوهر ما يجعل الكتاب مفيدًا بعد سنة.

عن التمارين

التمارين ليست للمراجعة السريعة. نمطها متنوّع عمدًا:

- **حساب يدوي** – لتثبيت المفهوم.
- **برمجة** – لترجمة المفهوم إلى شيفرة.
- **تحليل ناتج** – لتفسير ما يراه النموذج.
- **تصميم** – لتطبيق المعرفة على حالة جديدة.

- **إصلاح خطأ** – وهو **الأهم**: كود مكسور عليك تشخيصه.

كل تمرين يحمل «المتوقع»، وهو **ما يجب أن تكون إجابتك مطابقة له** – لا إجابة نموذجية واحدة، بل معيارًا تقيس به. في ملحق D تجد إجابات مختارة للتمارين التي لها إجابة واحدة صحيحة.

عن الأشكال

الأشكال مرسومة بأسلوب موحّد وتُشير إليها في النص. إن كنت تقرأ على الشاشة فأنها SVG (تتكامل مع التكبير)، وإن كنت تقرأ PDF أو DOCX فهي صور عالية الدقة.

عن الشيفرة

الشيفرة مكتوبة لـ **Python 3.14**، بأسلوب «مقروء أكثر منه ذكي»: أسماء متغيّرات واضحة، وتعليقات عربية فوق كل سطر يستحق التوضيح. الأمثلة تعمل على بيانات صغيرة الحجم كي تنفَّذ في ثوانٍ على جهاز عادي.

> **تحذير:** حين ينقل هذا الكتاب مثالًا من Keras 2 إلى Keras 3، ستجد تغييرات في ثلاثة مواقع عادةً: `tf.keras` أصبح `keras`، وصيغة الحفظ `.keras` بدل `.h5`، وغياب `Model.save` بصيغة SavedModel (استخدم `model.export` بدلًا منها). الفصل 35 يشرح هذا كاملًا.

النسخة الرقمية

الصيغة	الاستخدام المقترح
HTML	القراءة على الشاشة والبحث داخل الكتاب، مع فهرس تفاعلي
PDF	الطباعة والمشاركة والأرشفة – بترويسة وتذييل وترقيم
DOCX	التعديل والتخصيص (إنشاء نسخة خاصة بفريقك)
Markdown	المعالجة الآلية، البحث، أو التحويل إلى صيغ أخرى

جميع النسخ تحمل المحتوى نفسه مع الفهرس المرقّم.

أخيرًا

> **قاعدة:** إن انتهيت من فصل ولم تستطع أن تشرح لأحدٍ ما فعله النموذج ولماذا، فلم تنتهِ من الفصل بعد.

لنبدأ.

خريطة الطريق

خريطة الطريق

تخيّل أنك في رحلة من مدينة ساحلية إلى قلعة في الجبل. الطريق مستقيم؟ لا. لكن كل نقطة على الخريطة تُخبرك: أين أنت، وماذا ينتظرك، ولماذا هذا المقطع تحديدًا.

الصورة الكبرى

يتكوّن الكتاب من تسعة أجزاء. المنطق بينها تصاعدي: نبدأ من الفكرة المجرّدة («ما هو الأوتور؟»)، وننتهي من الملف التنفيذي على خادم («كيف أخدم النموذج؟»).

TEXT

```
ما هو ◄──────────────────────────────── الأساسيات  الجزء 1  Keras؟ وكيف يعمل؟
   1  عالم Keras 3                4  بنية النموذج
   2  أول نموذج كامل              5  البيئة والتثبيت
   3  الأوتور والتدرّج

واجهة  الجزء 2  Keras  كيف أبني وأُدرّب؟ ◄──────────────────────── الأساسية
   6  المقاييس                   10  الطبقات الأساسية
   7  التهيئة                    11  دوال التنشيط
   8  Callbacks                  12  دوال الخسارة
   9  التنظيم                    13  المُحسّنات

من أين تأتي البيانات؟ ◄──────────────────────── الجزء 3  هندسة البيانات
  14  الطبقات النصية             16  خطوط الأنابيب
  15  الأوزان وتعدّد المهام      17  المعالجة والتعزيز

كيف ترى؟ ◄──────────────────────── الجزء 4  الرؤية الحاسوبية
  18  النقل والتدريب الدقيق      21  رياضيات الالتفاف
  19  طبقات CNN                  22  الكشف والتقسيم
  20  محوّل الرؤية CNN           23  من الصفر
  24  تكبير الصور

كيف تتذكّر؟ ◄──────────────────────── الجزء 5  التسلسلات
  25  RNN وLSTM                  27  معالجة اللغة
  26  السلاسل الزمنية RNN        28  متقدّمة

كيف تنتبه؟ ◄──────────────────────── الجزء 6  المحوّلات
  29  آليات الانتباه  30  KerasHub وKerasNLP

كيف تخلق؟ ◄──────────────────────── الجزء 7  التوليد
  31  VAE والترميز                33  نماذج الانتشار
  32  شبكات GAN

كيف تشحن؟ ◄──────────────────────── الجزء 8  الهندسة الإنتاجية
  34  التكرار والاختبار          38  الطبقات المخصّصة
  35  التفسير والتشخيص           39  الحفظ والتحميل
  36  الأخطاء الشائعة            40  التسريع
  37  التصدير والنشر
```

الجزء 9 المشاريع ◄ كيف أطبّق؟

41 تصنيف صور إنتاجي

42 تنبّؤ السلاسل الزمنية

43 تحليل مشاعر + خدمة

ثلاثة مسارات مقترحة

المسار الأول: مهندس يريد أن يبني نموذجًا هذا الأسبوع

اليوم	الفصول	الهدف
1	1، 2، 5	بيئة عاملة + أول نموذج مفهوم سطرًا بسطر
2	6، 7، 8، 9، 10	بناء عشوائية النموذج بشكل صحيح
3	4، 12، 13، 35	بنية صحيحة + تحكّم في التدريب + حفظ
4	14، 15	بيانات سريعة وفعّالة
5	18، 19، 21 (إن كان vision) أو 25، 27 (إن كان نص)	البنية المناسبة لمجالك
6	36، 37	التسريع والتصدير
7	38، 40	التشخيص/testing

المسار الثاني: باحث ينتقل من Keras 2 إلى Keras 3

الفصول **1، 4، 12، 34، 35، 40** تحديدًا، فهي تغطّي كل ما تغيّر: الواجهة الخلفية، دورة حياة الـ callbacks، الطبقات المخصّصة، صيغة `.keras`، وأخطاء الترقية.

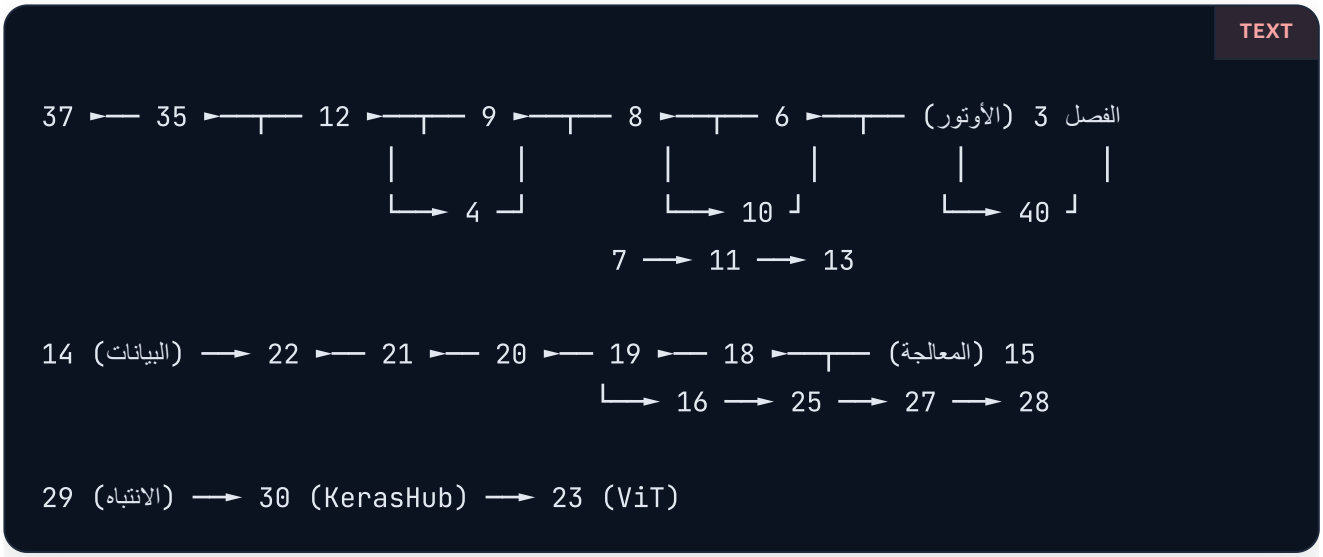
المسار الثالث: من فضول إلى إتقان

اقرأ الكتاب كاملًا بالترتيب. المسار البطيء هو الوحيد الذي يمنحك ما لا يمنحه الطريق السريع: **النموذج الذهني** الذي يربط كل فكرة بما قبلها.

تبعيات الفصول

مخطّط التبعيات أدناه يوضّح أي فصل تحتاجه قبل آخر. الأسهم من «ما يجب أن تعرفه» إلى «ما يبني عليه».

TEXT



> **ملاحظة:** هذا المخطّط إرشادي لا صارم. الفصول مكتوبة لتُقرأ مستقلة، لكن القراءة المتسلسلة تبني روابط أعمق بكثير.

ما الذي تحتاجه قبل أن تبدأ

المتطلب	الحد الأدنى	الأمثل
بايثون	3.11	**3.14**
Keras	3.10	**3.15.1**
الذاكرة	GB RAM 8	GB + GPU 16
مساحة القرص	GB 2	GB 10 (مع النماذج المدرّبة)
الخبرة	بايثون + NumPy	تعلّم عميق سابق

التفاصيل الكاملة في «المتطلبات التمهيدية».

اصطلاحات الكتابة والرموز

اصطلاحات الكتابة والرموز

> أي كتاب تقني يختار رموزه لسبب: كي يرى يرى الفكرة لا الصياغة. هذه هي رموزنا.

الرموز الرياضية

الرمز	الاسم	المعنى في هذا الكتاب
x	مُدخل (input)	عيّنة واحدة Passing بها إلى النموذج
y	هدف (target)	القيمة الحقيقية المراد التنبؤ بها
$\hat{y}$	تنبّؤ (prediction)	ما يخرجه النموذج
y_{true}	قيمة حقيقة	التسمية في دالة الخسارة
y_{pred}	قيمة متنبَّأة	ناتج الشبكة في دالة الخسارة
W	مصفوفة الأوزان	الأوزان القابلة للتعلّم
b	متجه التحييز	الإزاحة لكل neuron
z	مجموع ما قبل التنشيط	$z = Wx + b$
a	ناتج التنشيط	$a = f(z)$
θ	متجه المعاملات	كل أوزان النموذج مجتمعة
L	دالة الخسارة	رقم يقيس خطأ النموذج
J	الخسارة الكلية	L موزونة بعبء تنظيم
η	معدل التعلّم	حجم الخطوة في التحديث
∇	تدرّج (gradient)	متّجه المشتقّ الجزئي
λ	معامل التنظيم	قوة L1/L2
β_1, β_2	معاملات الزخم	في RMSPropg Adam
ϵ	ثابت رقمي	يمنع القسمة على صفر

الرمز	الاسم	المعنى في هذا الكتاب
σ	الدالة اللوغاريتمية	sigmoid أو الانحراف المعياري حسب السياق
μ	المتوسط	وسط التوزيع
N	حجم الدفعة	عدد العيّنات في خطوة واحدة
p	احتمال	أو نسبة إسقاط في Dropout حسب السياق
t	الخطوة	عدّاد خطوات التدريب
n_{in}, n_{out}	أبعاد المدخل والمخرج	لحساب عدد المعاملات

قواعد الكتابة

النص العادي

يُكتب كما هو. المصطلح الإنجليزي يُذكر بين قوسين عند أول ورود فقط: «التطبيع بالدفعة (Batch Normalization)».

التأكيد

يُستخدم **التغليظ** للتركيز على مفهوم يجب ألا يمرّ دون انتباه. لا نستخدمه لأمور عادية.

الشيفرة

تُكتب بين علامتَي backtick: `model.fit()`. كل كتلة شيفرة كبيرة تُسبق بشرح.

التحذيرات

تُعرَض في صناديق بألوان مميّزة:

تنبيه: تنبيه على خطأ شائع أو مخاطرة حقيقية.

نصيحة: طريقة أفضل لإداء المهمة.

ملاحظة: تفصيل دقيق أو استثناء مهم.

تحذير: خطأ قد يكلّفك ساعات من debugging.

جداول المقارنة

كل جدول في الكتاب يحمل عمودًا أو أكثر يحدّد **معيار** المقارنة: «الأسرع في»، «الأقلّ ذاكرةً»، «الأنسب لـ». لا يوجد في الكتاب جدول يقول فقط «الأفضل» بلا معيار.

اصطلاحات الشيفرة

العنصر	الاتفاق
اللغة	Python 3.14
إصدار Keras	3.15.1
الاستيراد	`import keras` (وليس `from tensorflow import keras`)
أسماء المتغيّرات	snake_case إنجليزية
التعليقات	`#` عربية
الدوال	`keras.layers.Dense` و `keras.optimizers.Adam` (مسار كامل)
الأرقام الثابتة	تُكتب كما هي (`epochs=10`، `batch_size=32`)
الشيفرة داخل النص	سطر واحد مختصر
الشيفرة الكاملة	كتلة مستقلة قابلة للتشغيل

أيّ الواجهة الخلفية؟

الأمثلة في هذا الكتاب تعمل على **أي** واجهة خلفية لـ Keras 3، وموجَّهة أساسًا إلى **JAX** و**TensorFlow**. حين يكون سلوك واجهة ما مختلفًا، يُذكر ذلك صراحةً في متن الفصل.

الواجهة	متى تكون الخيار الافتراضي في هذا الكتاب
TensorFlow	عند الحديث عن `tf.data` والتسريع والنشر
JAX	عند الحديث عن التوازي والدقة العالية
PyTorch	عند الحديث عن تكامل HuggingFace وLoRA

> **ملاحظة:** حين يُذكر أمرٌ خاص بواجهة خلفية، ستجد تحذيرًا بصيغة: «تنبيه: ينطبق على واجهة TensorFlow فقط».

وحدات القياس

الوحدة	الرمز	المعنى
الدقة (Accuracy)	–	نسبة العينات الصحيحة
الاستدعاء (Recall)	–	نسبة العينات الإيجابية المكتشفة
الدقّة (Precision)	–	نسبة التنبّؤات الصحيحة بين الموجبة
F1	–	وسط هندسي للدقّة والاستدعاء
المساحة تحت المنحنى	AUC	مقياس مستقل عن العتبة
متوسّط IoU	mIoU	متوسط تقاطع الاتحاد في التقسيم
متوسط الدقة المتوسطة	mAP	مقياس الكشف

أيقونات الكتاب

العلامة	المعنى
◈	معلومة أساسية
◆	نقطة تقنية دقيقة
⚠	تحذير عملي
✦	نصيحة لتحسين الأداء
⌘	أمر أو إجراء قابل للتنفيذ

عن ترتيب الفصول

الفصول مرقّمة من 1 إلى 43. حين يشير النص إلى فصل آخر، يُذكر رقمه وعنوانه: «انظر الفصل 18 (رياضيات الالتفاف)». الأجزاء التسعة تخدم للتنقّل فقط؛ الفصل يبقى وحدة القراءة.

> **قاعدة:** إن كان عليك الاختيار بين فصول في يوم واحد، اختر **الأقدم**: الأساسيات دائمًا هي التي تُوفّر عليك الوقت لاحقًا.

المتطلبات التمهيدية

المتطلبات التمهيدية

> لا أحد يبدأ من الصفر في كل مرة. لكن هناك حدٌّ أدنى لا بدّ من تجاوزه، وكثير من إحباط المبتدئين يعود إلى تجاوز هذا الحدّ اعتقادًا أنه واضح.

الحدّ الأدنى المطلق

قبل أن تفتح الفصل الأول، تأكّد أنك تستطيع:

- **كتابة دوال بايثون** وقراءة كود غير مألوف.
- **فهم المصفوفات** `shape` ، `dtype` ، `reshape` ، `dot` ، `mean` ، `argmax` :**NumPy**.
- **العمل في سطر أوامر**: الانتقال بين المجلدات، تثبيت الحزم، قراءة رسالة خطأ.
- **الالتحام بـ Git** على مستوى بسيط (تضمين، التزام، دفع).

إن كان أيّ من هذه صعبًا، فلديك طريقان: أو تتعلّمه أولًا (ويأخذ يومًا أو يومين)، أو تقرأ هذا الكتاب nonetheless وستتعلّمه بالتطبيق – لكن كل يوم تأخير في الأساسيات يكلّفك يومًا لاحقًا.

ما ستحتاجه

البرمجيات

المتطلب	الحد الأدنى	الموصى به	لماذا
بايثون	3.11	**3.14**	دعم كامل لـ Keras 3.15
Keras	3.10	**3.15.1**	كل الأمثلة مُختبَرة على هذا الإصدار
NumPy	1.24	x.2	المصفوفات والعمليات العددية
Matplotlib	3.7	3.11	الرسوم البيانية في التمارين
SciPy	1.10	1.15	دوال علمية مساعدة

العتاد

البيئة	الحد الأدنى	الموصى به	ملاحظات
المعالج	4 أنوية	8 أنوية	التدريب بطيء على المعالج
الذاكرة	8 GB	16 GB	الأحجام الكبيرة تحتاج ذاكرة
القرص	2 GB	10 GB	النماذج المدرّبة + السجلات
GPU	–	NVIDIA 8 GB+	اختياري لكنه يغيّر التجربة

> **ملاحظة:** كل أمثلة هذا الكتاب تعمل على المعالج في ثوانٍ. الـ GPU ليس شرطًا للتعلّم، لكنه شرط عملي للإنتاج. الفصل 36 (التسريع) يشرح كيف تستفيد منه إن توفّر.

المعرفة السابقة

مطلوب: أساسيات التعلّم العميق

ينبغي أن تعرف قبل أن تبدأ:

- ما هو **التصنيف** والتصنيف الثنائي متعدّد الفئات.
- ما معنى **التدرّج** و**دالة الخسارة** بشكل حدسي.
- ما هي **الشبكة العصبية** (شبكة Feedforward على الأقل).
- ما هو **فرط التخصيص** (Overfitting) ولماذا يحدث.

ليس مطلوبًا (وسنغطّيه)

الموضوع	أين يُغطّى
اشتقاق قاعدة السلسلة	الفصل 3
حسابات المصفوفات والضرب الداخلي	الفصل 3 و6
softmax والتهيئة	الفصل 7 و11
التنظيم والتحسين	الفصل 9 و13
البُنى التلافيفية	الفصل 18 و19
الانتباه والمحوّلات	الفصل 29

> **قاعدة:** إن قرأت الفصلين 1 و3 ولم تفهمهما، فالفصل 2 و5 ستُقرآنهما مرتين. اقرأ الفصل 3 مرتين في هذه الحالة – لا تحسّ بالحرج، فهو أكثر فصل يحتاج تكرارًا.

ما الذي لا تحتاج معرفته مسبقًا

الموضوع	السبب
PyTorch أو TensorFlow	Keras 3 تعزلك عنهما
CUDA/drivers	تُثبَّت مع المحرك وتُخفي تلقائيًا
النشر والتوزيع	الفصل 37 يتناوله من الصفر
واجهات سطر الأوامر المتقدّمة	ليست شرطًا لأي مثال

إعداد سريع

للتأكد أن كل شيء جاهز قبل الفصل الأول:

```bash
# 1) بيئة معزولة
python -m venv .venv
# على Windows: .venv\Scripts\activate
# على Linux/macOS: source .venv/bin/activate

# 2) تثبيت Keras مع المحرك المفضّل
pip install -U "keras[torch]==3.15.1"

# 3) تحقّق
python -c "import keras; print(keras.__version__, keras.backend.backend())"
```

المتوقّع أن تطبع: `3.15.1 torch`

ثم جرّب أول نموذج:

PYTHON

```python
import numpy as np
import keras

x = np.random.rand(32, 8).astype("float32")
y = np.random.rand(32, 1).astype("float32")

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")
model.fit(x, y, epochs=1, verbose=0)
print("كل شيء يعمل ✓")
```

نصيحة: احفظ هذا الاختبار في ملف باسم `smoke_test.py` . ستستخدمه في كل مرة تُغيّر فيها البيئة أو تُرقّي Keras – ويُنقذك من ساعات من الحيرة.

جدول زمني واقعي

المرحلة	الزمن	الفصول
الإعداد	ساعة	1، 2
الأساسيات	أسبوع	3، 4، 5، 6، 7
تدريب صحيح	أسبوع	8، 9، 10، 11، 12، 13
البيانات	4 أيام	14، 15، 16، 17
التخصّص	3 أسابيع	18-33
الإنتاج	أسبوع	34-40
المشاريع	أسبوع	41-43

> **ملاحظة:** الجدول تقريبي ويختلف حسب خلفيتك. صاحب الخلفية القوية في أحد هذه المجالات يختصر الوقت في نصفه.

المصادر المساعدة

المصدر	متى تفتحه
`https://keras.io/api/`	حين تنسى اسم واجهة أو معامل
`https://keras.io/guides/`	حين تحتاج مثالًا رسميًا
`https://github.com/keras-team/keras`	حين تقرأ الشيفرة المصدرية
ملحق B من هذا الكتاب	حين تريد مرجعًا سريعًا دون اتصال

الآن، لنبدأ.

فهرس المحتويات

الكتاب كامل من الأساسيات إلى النشر الإنتاجي – 9 أجزاء و43 فصلًا و6 ملاحق

الجزء 1: الأساسيات: من الفكرة إلى أول نموذج 35

1 Keras 3: كيف تغيّر عالَم التعلّم العميق؟ 36

2 البيئة: من الصفر إلى أول استيراد ناجح 52

3 الأوتور: اللغة التي يفكّر بها Keras 67

4 بنية النموذج: ثلاث طرق لتحديد الشبكة 82

5 أول نموذج كامل: شرح سطر بسطر 97

الجزء 2: واجهة Keras الأساسية: الطبقات ودوال الخسارة 114

6 الطبقات الأساسية: Dense والتنشيط والتسطيح 115

7 دوال التنشيط: أين يموت التدرّج؟ 129

8 دوال الخسارة: كيف نترجم الخطأ إلى رقم؟ 149

9 المُحسّنات: كيف يتحرّك النموذج نحو الحل؟ 164

10 المقاييس: كيف تقيس نموذجك دون أن ينخدع به الجميع 179

11 التهيئة: نقطة الانطلاق 194

12 الـ Callbacks: أتمتة دورة التدريب 214

13 التنظيم والقيود: كيف تمنع النموذج من الحفظ بدل التعلّم 243

الجزء 3: هندسة البيانات: من البايتات إلى الدفعات 261

14 هندسة البيانات: من البايتات إلى الدفعات 262

15 المعالجة المسبقة والتعزيز: هندسة المدخلات قبل الشبكة 277

16 الطبقات النصية: من الجملة إلى المتجه 292

17 الأوزان والمهام المتعدّدة: كيف توازن النموذج وتشقّه 310

الجزء 4: الرؤية الحاسوبية: الشبكات التلافيفية 327

18 الرياضيات خلف الالتفاف 328

19 طبقات الشبكة التلافيفية في Keras 348

20 شبكة CNN من الصفر 364

21 نقل المعرفة والتدريب الدقيق 377

22 الكشف والتجزئة الدلالية 389

23 محوّل الرؤية (Vision Transformer) 402

24 تكبير الصور (Super-Resolution) 413

الجزء 5: التسلسلات واللغة والسلاسل الزمنية 426

25 الشبكات التكرارية و LSTM و GRU 427

26 الشبكات التكرارية المتقدّمة 440

27 معالجة اللغة الطبيعية عمليًا 453

28 السلاسل الزمنية 466

الجزء 6: المحوّلات: انتباه، محوّل، KerasHubg 479

29 المحوّلات وآلية الانتباه 480

30 منظومة KerasHub وKerasNLPg 492

الجزء 7: النماذج التوليدية: من الترميز إلى الانتشار 503

31 الترميز الآلي المتغيّر (VAE) 504

32 شبكات التوليد التنافسيّة (GAN) 518

33 نماذج الانتشار (Diffusion Models) 531

الجزء 8: الهندسة الإنتاجية: من النموذج إلى الخدمة 542

34 الطبقات والنماذج المخصّصة 543

35 الحفظ والتحميل 559

36 تسريع التدريب 573

37 التصدير والنشر 585

38 التكرار والاختبار 595

39 التفسير والتشخيص 609

40 الأخطاء الشائعة وكيف تُصلحها 624

الجزء 9: مشاريع تطبيقية متكاملة 634

41 مشروع تصنيف صور إنتاجيّ 635

42 مشروع التنبّؤ بالسلاسل الزمنيّة 648

43 مشروع تحليل المشاعر ونشره كخدمة 662

الملاحق

A قاموس المصطلحات عربي-إنجليزي

B مرجع Keras 3 السريع

C ورقة المعادلات

D إجابات مختارة للتمارين

E فهرس الأخطاء الشائعة

F المراجع والمصادر

الجزء الأول

الأساسيات: من الفكرة إلى أول نموذج

تبني الأساس الذي يقوم عليه الكتاب كلّه: ما Keras 3 ولماذا وُجد، كيف تُهيّئ بيئة لا تنهار بعد أسبوع، كيف تفكّر بالأوتور والبثّ والتدرّج التلقائي، ثم كيف تُخرج أول نموذج كامل يعمل.

ما تكسبه من هذا الجزء

تخرج وأنت قادر على تثبيت بيئة نظيفة، وقراءة رسائل الأخطاء، وبناء نموذج أوّلي وتدريبه وتقييمه بلا نسخ أعمى.

فصول هذا الجزء

1 Keras 3: كيف تغيّر عالَم التعلّم العميق؟

2 البيئة: من الصفر إلى أول استيراد ناجح

3 الأوتور: اللغة التي يفكّر بها Keras

4 بنية النموذج: ثلاث طرق لتحديد الشبكة

5 أول نموذج كامل: شرح سطر بسطر

1

الفصل 1: Keras 3: كيف تغيّر عالَم التعلّم العميق؟

بطاقة الفصل

الفصل 1

مهمّة هذا الفصل

يضع الصورة الكاملة: ما Keras 3 وما ليس هو، وكيف يقف فوق JAX وTensorFlow وPyTorch بمحرّك واحد، وما الذي تغيّر عن Keras 2.

ماذا تكسب منه

تختار الأداة الصحيحة قبل أن تكتب سطرًا، وتعرف أي كود قديم سيحتاج تعديلًا عند الترقية – بدل أن تكتشف ذلك في منتصف المشروع.

الدالة	دورها	متى تستخدمها
`keras.__version__`	رقم الإصدار المثبَّت فعليًا	أول سطر في أي تقرير خطأ أو طلب دعم
`keras.backend.backend()`	اسم المحرّك الخلفي النشط	عندما يختلف السلوك بين جهازك وجهاز زميلك
`keras.config.set_backend()`	تبديل المحرّك في وقت التشغيل	لتجربة الكود نفسه على محرّكين بلا إعادة تثبيت
`keras.ops`	طبقة العمليات الموحّدة عبر المحرّكات	بدلًا من tf.* أو torch.* في أي كود تريد نقله
`keras.saving.load_model()`	تحميل نموذج محفوظ بصيغة keras.	عند استهلاك نموذج جاهز أو استئناف عمل

قصة مفتاحية: في ربيع 2026، استلم مهندسٌ فريقًا عامِلًا فيه نموذج كشف احتيال مكتوبٌ قبل ثلاث سنوات بـ Keras 2 وواجهة TensorFlow الخلفية. طلب من الفريق ترقية الحزمة، فاكتشف أن أول أمر كسره لم يكن الشبكة نفسها بل سطر الاستيراد: `from tensorflow import keras` لم يعد بوابة الدخول الموصى بها، و `model.save("model.h5")` لم يعد الطريق الآمن لحفظ النموذج. هذا الفصل يجيب سؤالًا أعمق: **ما الذي تغيّر فعلًا في عالم التعلّم العميق بين 2023 و2026، ولماذا كان هذا التحوّل في جوهره لا في الأدوات؟**

أهداف الفصل

- [ ] التمييز بين Keras كـ«واجهة استخدام» (API) وبين TensorFlow وJAX وPyTorch كـ«محرّكات تنفيذ» (runtimes).
- [ ] وصف بنية Keras 3 متعددة الواجهات الخلفية، ومعرفة أيّ واجهة خلفية تناسب أيّ عبء عمل.
- [ ] سرد التغييرات الجوهرية من Keras 2.x إلى Keras 3.0: الاستيراد، `keras.ops`، صيغة `.keras`، ومسار التصدير.
- [ ] استعراض أبرز ما جاء في Keras 3.15: تصدير `sliding_window`، PyTorch، `MultiOptimizer`، والدوال الرياضية الجديدة.
- [ ] تحديد موضع منظومة KerasHub/KerasNLP ومصرّح keras-cv المؤرشف ضمن الخريطة الأكبر.
- [ ] تطبيق خطة ترحيل عملية من مشروع Keras 2 إلى Keras 3 دون إعادة كتابة كل شيء.

1.1 ما هو Keras — وما ليس هو

تخيّل أنك تشتري سيارة. المحرك (engine) هو TensorFlow أو JAX أو PyTorch: هو ما يشغّل السيارة فعليًا ويصنع القوة والدّة. أمّا لوحة القيادة (dashboard) فهي Keras: هي ما تتفاعل معه أنت — المقود، العدّادات، زرّ التشغيل. في Keras 2 كانت لوحة القيادة ملتصقة بمحرّك واحد بُني من الصفر له؛ في Keras 3 صارت اللوحة تعمل على أكثر من محرّك، وتتبدّل المحرّك تحت يديك دون أن تتعلّم لغة جديدة.

هذا التمييز هو أهمّ ما ستفهمه في هذا الكتاب. **Keras ليس إطارًا منافسًا لـTensorFlow أو JAX أو PyTorch**، ولا يدّعي استبدالها؛ Keras 3 هو **طبقة واجهة** (user-facing API) فوقها، ويمنحك نموذجًا واحدًا للكتابة (model definition)، ودورة تدريب واحدة (`compile` / `fit` / `evaluate` / `predict`)، وفضاء عمليات موحّدًا (`keras.ops`).

ولكل مكوّن في الخريطة مهمة مختلفة:

المكوّن	وظيفته	ما لا يقدّمه
Keras	تعريف النموذج، الطبقات، دورة التدريب، الحفظ والتحميل	لا ينفّذ ضرب مصفوفات بنفسه في وضع الإنتاج المعتاد
TensorFlow	تنفيذ الرموز، التدريب الموزّع، أدوات النشر (TFLite، TensorFlow Serving)	واجهته كانت مرتبطة بـ `tf.keras` لا بواجهة مستقلة
JAX	تنفيذ غير متزامن (async)، jit، sharding على أجهزة متعددة	منحنى تعلّم أطول لمن يأتي من جلسات TF
PyTorch	منظومة متنامية (Hugging Face، TorchScript، torch.compile)	تاريخًا لم يكن Keras يتكامل معه دون «الطبقة الفاضلة»

> **ملاحظة:** مع Keras 3، من ينوي الاعتماد على «إطار واحد» لأعماله يستطيع أن يكتب طبقة النماذج مرة واحدة، ثم يختار المحرّك بحسب المهمة: TensorFlow للخدمة، JAX للبحث والتوزيع على أجهزة متعددة، PyTorch حين يكون التكامل مع منظّمة خارجية شرطًا تعاقديًا.

1.2 البنية متعددة الواجهات الخلفية

كيف يبدّل Keras المحرّك؟ الجواب: **طبقة تحويل (dispatch layer)**. عند الاستيراد، يقرأ Keras متغيّر البيئة `KERAS_BACKEND`، أو يستدعي `keras.config.set_backend(...)` صراحةً، ثم يوجّه كل عملية حسابية إلى المحرّك المختار.

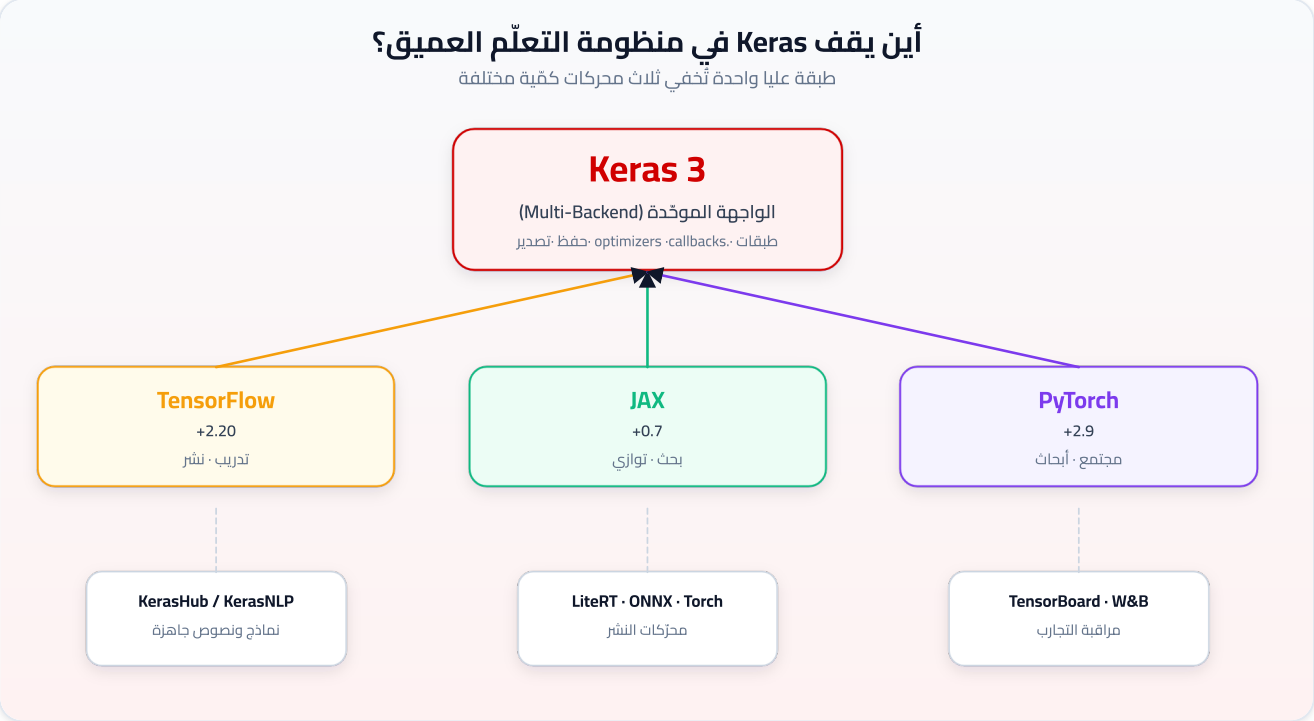
```python
import os
os.environ["KERAS_BACKEND"] = "jax"  # اختيار المحرّك قبل استيراد keras

import keras  # الاستيراد العام: لا tensorflow ولا torch

print("إصدار Keras:", keras.__version__)      # النتيجة المتوقعة: 3.15.1
print("الواجهة الخلفية:", keras.backend.backend())  # النتيجة المتوقعة: jax
```

الواجهات الخلفية المدعومة رسميًا في Keras 3.15 هي: **TensorFlow**، و**JAX**، و**PyTorch**، بالإضافة إلى **OpenVINO** بوصفه واجهة **للاستدلال فقط** (inference-only) – أي أنه يُستخدم لتشغيل نموذج مُدرَّب، لا لتدريبه. النتيجة العملية: قد تختار OpenVINO على جهاز Intel صناعي لخدمة الاستدلال، ثم تُدرّب النموذج نفسه على PyTorch في الخادم.

هذه البنية لها كلفة وكسبة. الكلفة أن كل عملية يجب أن تُنفَّذ عبر طبقة التحويل، وهذه الطبقة تحاول أن تُبقي التكلفة أقل من واحد إلى اثنين بالمئة من زمن التنفيذ في العمليات الشائعة. الكسبة أن الكود نفسه يصير قابلًا للنقل بين المحرّكات دون تعديل.

> **نصيحة:** لا تخلط محرّكين في العملية نفسها. Keras يقرأ إعداد المحرّك مرّة عند الاستيراد؛ استدعاء `keras.config.set_backend("torch")` بعد بناء نموذج مبنيّ على JAX لا يحوّل ذلك النموذج، بل قد يقودك إلى أخطاء غامضة في المقاييس والأوزان.

1.3 ماذا تغيّر من Keras 2 إلى Keras 3؟



شكل 1 – خريطة منظومة Keras 3 وعلاقتها بالمحرّكات الخلفية

ماذا يحدث لو بقيت على `import tensorflow as tf` ثم `tf.keras` ؟ ستستمر في العمل – Keras 3 يحتفظ بجسر توافق (legacy shim) باسم `keras._tf_keras` لتمرير كود TensorFlow القديم – لكنك ستُنفق جزءًا من ميزتك: ستظل محصورًا داخل TensorFlow، وستفقد قابلية نقل النموذج نفسه إلى JAX أو PyTorch.

الفروق الجوهرية التي يجب أن تستوعبها:

1. **الاستيراد صار موحّدًا.** صار المرجع هو `import keras` ثم `keras.layers.Dense`، لا `from tensorflow import keras`. هذا قابل للاختبار آليًا: ابحث في مشروعك عن `from tensorflow import keras` وستعرف عدد الملفات التي تحتاج تعديلًا.

2. **ظهر `keras.ops` كفضاء عمليات بديل عن `tf.*`.** كل عملية حسابية في كود النموذج (جمع، ضرب مصفوفات، batch matmul، softmax) صارت تُكتب بـ `keras.ops.*`، فيُنفَّذها المحرّك المختار. هذا هو مفتاح قابلية النقل: كود يستخدم `tf.linalg.matmul` داخل طبقة مخصّصة مرتبط بـTensorFlow، وكود يستخدم `keras.ops.matmul` محمول.

3. **صيغة `.keras` صارت الصيغة المفضّلة للحفظ.** `model.save("m.keras")` يُنتج ملفًا بصيغة ZIP يحوي: `config.json` (بنية النموذج)، و `metadata.json` (بيانات وصفية)، وملف

الأوزان، وأي أصول مضمّنة. الملف مكتفٍ ذاتيًا: نقله إلى خادم آخر لا يحتاج الكود الأصلي، ويكفيه `keras.saving.load_model("m.keras")`.

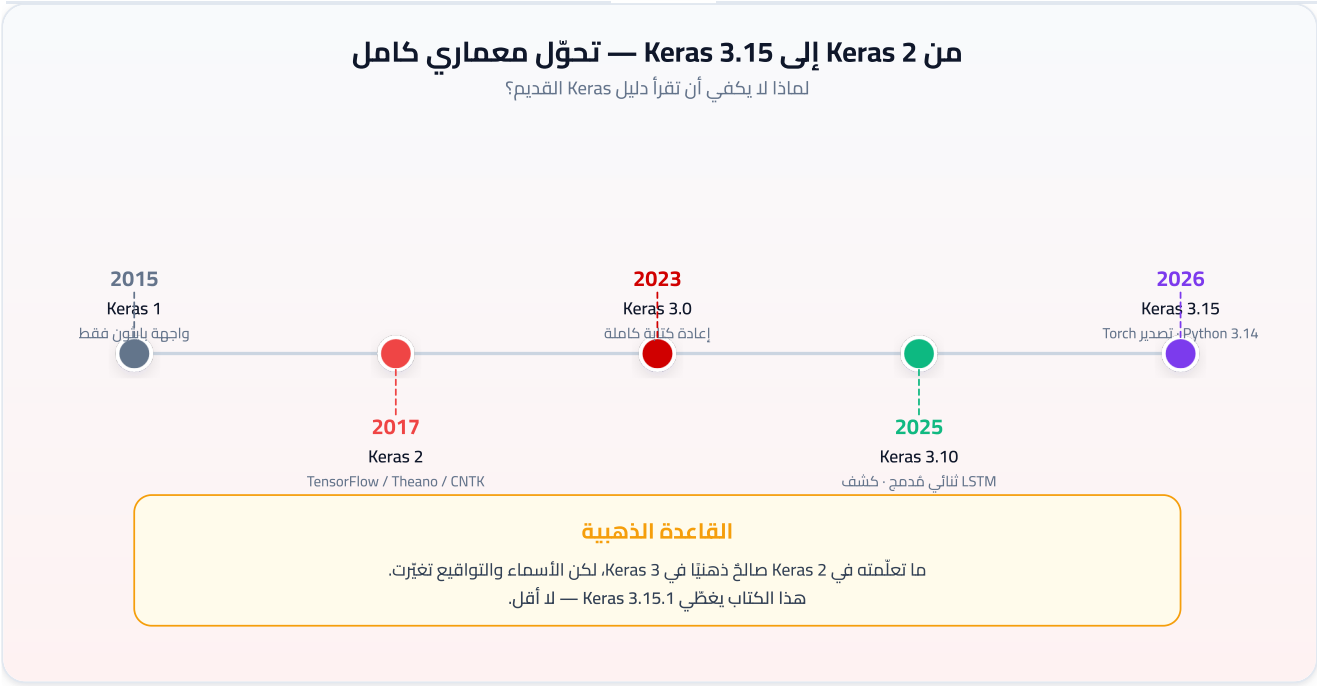
4. **مسار التصدير إلى SavedModel تغيّر.** في Keras 3 لم يعد `model.save("m.savedmodel")` مدعومًا؛ المسار الصحيح هو `model.export(...)` بصيغة `tf_saved_model`، ويُنتج قطعة استدلال (inference-only artifact) تحوي المسار الأمامي فقط (`call()`) ولا تحوي منطق التدريب. للنماذج المصنّفة بـSubclassing هناك قيود إضافية على هذا المسار، وستعالَج في الفصل الرابع.

5. **تغيّر أسماء المقاييس وواجهات الحفظ.** صار الاسم `keras.metrics.F1Score` بدل `f1_score`، و`keras.metrics.AUC` بدل `auc`، وواجهة الحفظ الموحّدة هي `keras.saving.load_model` / `keras.saving.save_model`.

6. **أساليب المعالجة المسبقة انتقلت إلى الطبقات.** `ImageDataGenerator` أصبح مسارًا مهجورًا؛ البديل هو طبقات داخل الرسم البياني مثل `keras.layers.RandomFlip` و`keras.layers.Rescaling`، وهي قابلة للتسلسل داخل ملف `.keras`.

عند الفحص العملي، التكلفة الحقيقية للترحيل ليست في الشبكة: هي في **عدد الملفات** التي تستورد `tf.keras`، وفي الاستخدامات الصامتة لـ`tf.` داخل دوال الخسارة أو الطبقات المخصّصة.

1.4 أبرز ما جاء في Keras 3.15



شكل 2 – الخط الزمني لإصدارات Keras من x.2 إلى 3.15

الإصدار 3.15.0 (يونيو 2026) وإصدار التصحيح 3.15.1 (يوليو 2026) قدّما مجموعة تغييرات عملية، لا مجرد تحسينات داخلية:

- **تصدير Keras إلى PyTorch الأصلي:** صارت `model.export("model.pt2", format="torch")` تُنتج برنامج PyTorch قابلًا للتحميل بـ `torch.export.load`. هذه ميزة جديدة كسرت حاجزًا حقيقيًا: نقل نموذج مُدرَّب في Keras إلى منظومة إنتاج قائمة على PyTorch دون إعادة كتابة الشبكة. كما أُضيف دعم تصدير LiteRT (TFLite) للنماذج ذات الواجهة الخلفية PyTorch.
- **نافذة منزلقة في الانتباه:** `keras.layers.MultiHeadAttention` و`keras.layers.GroupQueryAttention` يقبلان الآن المعامل `sliding_window`، وهو ما يجعل الانتباه على السياقات الطويلة (نصوص، سلاسل زمنية) أرخص حسابيًا لأن كل موضع لا يرى إلا جواره.
- **`keras.optimizers.MultiOptimizer`:** يسمح بإسناد مُحسّنات مختلفة إلى شبكات فرعية مختلفة داخل النموذج نفسه – مثلًا مُحسّن بمعدّل تعلّم مرتفع للرأس الجديد ومُحسّن بمعدّل منخفض للجسم المدَرَّب (body) أثناء النقل والت Fine-tuning.

- **دوال رياضية جديدة في `keras.ops`:** أضيفت `unique` و`fabs` و`fmax` و`fmin` و`erfc` و`dsplit` و`percentile` و`nanpercentile`، وفي `keras.ops.linalg` أُضيفت `pinv` (المصفوفة شبه العكسية) و`matrix_rank`، وفي `keras.ops.image` أُضيفت `sobel_edges` (كشف الحواف) و`ssim` (التشابه البنيوي). وصار `keras.ops.transpose` يقبل محاور سالبة.
- **تعزيز أمني:** حزم x.3.15 أضافت تقييدًا في فكّ تسلسل بيانات IMDB وReuters وCIFAR (منع `pickle` غير المقيد)، والتحقّق من مجموعات H5 المتداخلة، ورفض ملفات `.keras` المضغوطة التي قد تُرهق الذاكرة عند فكّها (decompression bomb). هذه ليست تفاصيل تجميلية: تحميل ملف `.keras` من مصدر غير موثوق كان ثغرة تنفيذ شيفرة، والآن يُرفض.
- **دعم Python 3.14:** عولجت تغيّرات التحويل المنطقي (coercion) في Python 3.14، وأُصلحت بعض دوال الأشكال.

كم يكلّف اختيار المحرّك؟

سؤال واقعي: «ما الفرق الزمني بين محرّك وآخر؟» المعادلة التي تحكم الجواب عملية لا نظرية:

$$T_{\text{train}} = E \times \left\lceil \frac{N}{B} \right\rceil \times t_{\text{step}}$$

حيث T_{train} هو زمن التدريب الكلي، وE عدد الحقبات (epochs)، وN عدد عينات التدريب، وB حجم الدفعة (batch size)، و$\lceil N/B \rceil$ عدد خطوات التحديث في الحقبة الواحدة بعد التقريب لأعلى، وt_{step} زمن الخطوة الواحدة (مسار أمامي + مسار خلفي + تحديث أوزان).

القراءة العملية للمعادلة: العلاقة بين المحرّكات تظهر في **العاملين الثالث والرابع** فقط. المحرّك الأسرع (أو الأسرع في استثمار العتاد المتوازي) يقلّل t_{step}، والبنية التي توزّع البيانات تقلّل t_{step} كذلك. أما E وN وB فهي قراراتك أنت لا قرارات المحرّك. لذلك: **لا تقارن محرّكين على أساس نموذج يحسب 10 حقبات؛ قارنهما على t_{step} وحدها** (وهي ما يعكسه قياس throughput بالعيّنات/الثانية).

1.5 المنظومة حول Keras: KerasHub وKerasNLP وkeras-cv

المنظومة المحيطة بـ**KerasHub**: Keras 3 هو مستودع النماذج المدرَّبة مسبقًا لـKeras 3 (نماذج BERT وGPT-2 وViT وغيرها) بواجهة موحّدة عبر المحرّكات. وأصبح **KerasNLP** جزءًا من KerasHub: مستودع `keras-team/keras-nlp` لم يعد مستقلًا، وهو الآن يوجّه إلى `keras-team/keras-hub`. أمّا **keras-cv** فقد أصبح **مؤرشفًا (archived)**؛ المشروع متوقف، وقد انتقلت كثير من قدراته إلى نواة Keras – ومن أوضح الأمثلة عملية `keras.ops.image.sobel_edges` و`keras.ops.image.ssim` اللتان كانتا في حزمة KerasCV منفصلة (الحواف والتشابه البنيوي).

ما معنى ذلك عمليًا؟ ثلاث قواعد:

1. لا تبدأ مشروعًا جديدًا على `keras-cv`؛ استخدم `keras.ops.image` وطبقات Keras 3 الأساسية.
2. إذا كان لديك كود KerasNLP، فإن مسار الترقية هو الانتقال إلى حزمة `keras-hub` وتثبيت نسخة متوافقة مع Keras 3.15.
3. النماذج المدرَّبة مسبقًا في KerasHub تُحمَّل وتُهيَّأ داخل نموذج Keras 3 نفسه، فتستفيد من `fit` / `save` / `export` بشكل طبيعي.

1.6 جدول مقارنة: Keras 2.x مقابل Keras 3.15

المقارنة هنا مبنية على **معايير** قابلة للقياس، لا على تفضيلات ذاتية:

المعيار	Keras 2.x	Keras 3.15
بوابة الاستيراد	`from tensorflow import keras`	`import keras` (واحدة لكل المحرّكات)
المحرّك	TensorFlow فقط	TensorFlow / JAX / PyTorch / OpenVINO (استدلال)
مرجع العمليات داخل الطبقات	`tf.*`	`keras.ops.*`
صيغة الحفظ المفضّلة	`.h5` أو SavedModel	`.keras` (مكتفٍ ذاتيًا)
تصدير SavedModel	`model.save("m.savedmodel")`	`model.export("m", format="tf_saved_model")`
جودة واجهة الاستدلال مع OpenVINO	غير متاح	متاح عبر واجهة OpenVINO
زمن الوصول لأوزان FN وقيود CUDA	مرتبط بإصدار TF	يُختار المحرّك مستقلًا
توافق الكود القديم	كامل	عبر جسر `keras._tf_keras` للتوافق فقط

1.7 مثال محلول: ترقية نظام كشف الاحتيال

ماذا يحدث لو كان لديك نظام تصنيف المعاملات إلى «احتيال / سليم» بُني بـKeras 2؟ إليك خطة ترحيل منظّمة:

الخطوة 1 – حصر الاعتماديات. ابحث في المستودع عن `tensorflow` و `tf.keras` . على النمط المعتاد في المشاريع القديمة، ستجد الاستيراد في `train.py` ، وفي `losses.py` ، وفي `callbacks.py` ، وأحيانًا داخل `layers.py` (وهذا الأخطر).

الخطوة 2 – تثبيت البيئة. أنشئ بيئة افتراضية جديدة وأثبّت `keras==3.15.1` مع محرّك واحد. لا تخلط `tensorflow` و`torch` في البيئة نفسها إن لم تكن تحتاجهما: كل منهما يجرّ حزمًا ثقيلة قد تتعارض في إصدارات numpy/h5py.

الخطوة 3 – تعديل الاستيراد والعمليات. استبدل `from tensorflow import keras` بـ `import keras`، ثم استبدل `tf.nn.relu` بـ `keras.ops.nn.relu`، و`tf.matmul` بـ `keras.ops.matmul`، و`tf.reduce_mean` بـ `keras.ops.mean`.

الخطوة 4 – تحديث نقاط الحفظ والتصدير. ملف `.h5` القديم يُحمَّل بـ `keras.saving.load_model("m.h5", compile=False)` ما دام في نطاق Keras 3 المتوافق، ثم يُحفظ فورًا بصيغة `.keras` عبر `model.save("m.keras")`، ثم يُعاد التحميل للتحقّق من سلامة البنية. أما للنشر، فالمسار هو `model.export(...)` بصيغة مناسبة لهدفك.

الخطوة 5 – التحقّق السلوكي. قارن التوقعات على مجموعة تحقق ثابتة قبل الترقية وبعدها، وتأكّد من تطابق `evaluate` ضمن سماحية معقولة. الفروق الصغيرة توضيح source (مثل عشوائية التهيئة أو اختلاف دقّة الحساب بين المحرّكات)، والفروق الكبيرة تعني بقايا اعتماد على TF.

الكود أدناه يلخّص Step 3 بأبسط صورة ممكنة:

```python
import keras

# Keras 2 (قبل): from tensorflow import keras; import tensorflow as tf
# Keras 3 (بعد): استيراد موحّد + عمليات محايدة عن المحرّك
model = keras.Sequential([
    keras.layers.Input(shape=(20,)),          # طبقة الإدخال صارت صريحة داخل الSequential
    keras.layers.Dense(64, activation="relu"),  # التنشيط يُمرَّر كوسيط لا كطبقة منفصلة
    keras.layers.Dropout(0.2),                # إسقاط عشوائي أثناء التدريب فقط
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=[keras.metrics.AUC(name="auc")],  # الاسم الصحيح في Keras 3 هو AUC
)
model.summary()  # النتيجة المتوقعة: ملخّص بعدد الأوزان (205) لكل طبقة Dense
```

1.8 أخطاء شائعة

تنبيه: أشهر خطأ في الترحيل هو كتابة `from tensorflow import keras` ثم ظنّ أن المشروع صار «Keras 3». الكود يعمل، لكنه لا يستفيد من تعدّد المحرّكات، ويظل مرتبطًا بـTensorFlow.

إنذار: الخلط بين `model.save("m.savedmodel")` و `model.export(...)`. الأول غير مدعوم في Keras 3؛ والثاني يُنتج قطعة استدلال لا قطعة تدريب. الخلط بينهما ينتج ملفات لا يستطيع `load_model` قراءتها.

ملاحظة: لا تفترض أن كل طبقة مخصّصة في كودك القديم تعمل كما هي. الطبقات التي تستدعي `tf.` مباشرةً ستظل مرتبطة بمحرّك واحد حتى تُحوَّل إلى `keras.ops`.

تنبيه: تغيير المحرّك يغيّر الأرقام. الفروق البسيطة في الخسارة (من 0.5412 إلى 0.5407 مثلًا) ناتجة عن اختلاف ترتيب عمليات الجمع ودقّة الحساب؛ اعتمد سماحية، لا تطابقًا حرفيًا.

التمارين

1. اقرأ ملاحظات إصدار Keras 3.15.0 على GitHub واستخرج ثلاثًا من الإضافات التي تؤثّر في تصميم مشروعك. (المتوقع: مثلًا ، `sliding_window` ، `export_torch` ، `MultiOptimizer` ، مع تبرير أثر كل واحدة على مشروعك).

2. لديك نظام كشف الاحتيال يعمل اليوم على Keras 2 + TensorFlow، ومطلوب تسليمه كخدمة على خادم Intel. أيّ محرّك تختار للخدمة وأيّ محرّك تختار للتدريب، ولماذا؟ (المتوقع: تدريب على TF أو PyTorch بحسب المتاح، واستدلال على OpenVINO لأن واجهته للاستدلال فقط وتُشغَّل على عتاد Intel؛ مع توضيح أن التحويل يتم عبر `model.export`).

3. افحص مستودعًا فيه 30 ملف Python وابحث عن `from tensorflow import keras` و `tf.` داخل الطبقات. صنّف الملفات: أيّها يحتاج تعديلًا فوريًا، وأيّها يمكن تأجيله؟ (المتوقع: الملفات التي تُعرّف دوال خسارة/مقاييس أو طبقات مخصّصة تحتاج تعديلًا عاجلًا؛ ملفات استدعاء inference فقط قد تكتفي بتحويل الاستيراد).

4. أيّ العبارتين التاليتين تختار ولماذا: «Keras 3 إطار منافس لPyTorch» أم «Keras 3 واجهة فوق PyTorch/TF/JAX»؟ اشرح الأثر العملي على قرارك في اختيار المحرّك. (المتوقع: الثانية؛ لأن Keras 3 لا ينفّذ التدريب بنفسه بل يوجّه العمليات، وهذا يتيح كتابة النموذج مرة واحدة وتشغيله على أكثر من محرّك).

5. صحّح الخطأ: `model.save("model.savedmodel")` . (المتوقع: الاستدعاء غير مدعوم؛ البديل `model.export("model", format="tf_saved_model")` لإنتاج قطعة استدلال قابلة للخدمة).

6. شرط تعاقدي في مشروعك يفرض PyTorch، بينما فريق البحث يعمل بJAX. اقترح بنية تحترم الشرط وتبقى قابلة لإعادة الاستخدام. (المتوقع: تعريف النموذج بـ `import keras` و `keras.ops` كي يكون محمولًا، وتدريب النسخة الإنتاجية على Torch، وبقاء نسخة JAX للتجارب؛ مع توثيق الواجهة البرمجية للنموذج كعقد).

7. اذكر ثلاث دوال أضيفت إلى `keras.ops` في 3.15 واذكر فائدتها. (المتوقع: `keras.ops.image.sobel_edges` لكشف الحواف، `keras.ops.image.ssim` لقياس التشابه البنيوي، `keras.ops.linalg.pinv` لحساب المعpseudo العكسي في حل الأنظمة الخطية).

8. ماذا يعني عمليًا أن `keras-cv` مؤرشف؟ ماذا تفعل لو كان مشروعك يستورد منه اليوم؟ (المتوقع: لا تبدأ عليه؛ استبدل العمليات بما يقابلها في `keras.ops.image` وطبقات Keras الأساسية، وقد تحتاج بدائل أو تنفيذًا صغيرًا يدويًا).

9. احسب T_{train} تقريبيًا لمشروعك: $E = 20$ حقبة، $N = 50{,}000$ عينة، $B = 64$. كم خطوة تحديث؟ (المتوقع: $\lceil 50000/64 \rceil = 782$ خطوة لكل حقبة، أي 15{,}640 خطوة إجمالًا؛ ثم قارن زمن الخطوة t_{step} بين محرّكين).

10. لماذا يُعتبر Keras 3 مناسبًا لمشروع في الخدمات المالية يجب أن يبقى قابلًا للترحيل بين بيئات تشغيل مختلفة (GPU سحابي، CPU داخلي، حافة)؟ (المتوقع: لأن النموذج يُعرَّف مرة واحدة عبر واجهة محايدة، ويُصدَّر إلى صيغ متعددة `torch` / `litert` / `openvino` / `onnx` / `tf_saved_model` بحسب البيئة).

11. ما الفرق بين «saved model» بصيغة `.keras` و«تصدير SavedModel»؟ أيّهما تحتفظ به في مستودع النماذج؟ (المتوقع: `.keras` يحوي بنية التدريب كاملة مع إمكانية الاستئناف؛ أما تصدير SavedModel فهو قطعة استدلال-forward فقط، عادةً لا يُحفظ في مستودع التدريب بل يُولَّد عند النشر).

12. اكتب ثلاث قواعد تكتبها في دليل الفريق لمنع انتقال كود Keras 3 إلى `tf.` داخل الطبقات. (المتوقع: منع `import tensorflow` داخل ملفات الطبقات؛ استيراد `keras` فقط؛ استخدام `keras.ops` لكل عملية؛ مراجعة في CI عبر grep عن `tf\.`).

ملخص الفصل

- Keras 3 ليس محرّكًا جديدًا بل واجهة استخدام موحّدة فوق TensorFlow وJAX وPyTorch (وOpenVINO للاستدلال).
- الاستيراد المرجعي `import keras` ، والعمليات داخل الطبقات تكتب بـ `keras.ops.*` لا بـ `tf.*` .
- صيغة `.keras` هي صيغة الحفظ المفضّلة، و `model.export(...)` هو المسار الصحيح لتصدير قطع النشر بصيغ `torch` / `litert` / `openvino` / `onnx` / `tf_saved_model` .
- الإصدار 3.15 أضاف `export_torch` ، و `sliding_window` للانتباه، و `MultiOptimizer` ، ودوالًا رياضية جديدة في `keras.ops` ، مع تعزيز أمني ودعم Python 3.14.
- الفرق بين محرّك وآخر يظهر في t_{step} داخل معادلة الزمن الكلي، لا في عدد الحقبات.
- KerasHub هو اليوم مرجع النماذج المدرَّبة مسبقًا، وKerasNLP جزء منه، وkeras-cv مؤرشف ومزاح إلى نواة Keras.

الفصل 2: البيئة: من الصفر إلى أول استيراد ناجح

بطاقة الفصل

الفصل 2

مهمّة هذا الفصل

يبني بيئة معزولة قابلة للتكرار: تثبيت Keras، اختيار المحرّك، والتأكّد من رؤية المعالج الرسومي قبل كتابة أي نموذج.

ماذا تكسب منه

تنتقل من «يعمل على جهازي» إلى «يعمل دائمًا»، وتُغلق باب أخطاء التثبيت التي تستهلك أيامًا بلا سبب واضح.

الدالة	دورها	متى تستخدمها
`python -m venv .venv`	بيئة معزولة لكل مشروع	قبل أي تثبيت؛ يمنع تعارض الإصدارات
`pip install "keras[torch]"`	تثبيت Keras مع محرّك محدّد	عند بدء مشروع جديد
`KERAS_BACKEND`	متغيّر بيئة يحدّد المحرّك لكل الجلسات	للتثبيت الدائم بدل التبديل في كل سكربت
`keras.config.set_backend()`	تبديل المحرّك داخل السكربت	لتجارب سريعة بين محرّكين
`keras.backend.backend()`	التحقّق النهائي من المحرّك النشط	في أعلى كل سكربت إنتاجي

قصة مفتاحية: استلمت جهازًا جديدًا، وطلبت من الحاسوب «افتح لي بايثون» ثم سألت ما هو ثم سألت: «أين Keras؟» فردّ الجهاز: `ModuleNotFoundError` . ساعتان من البحث، والسبب كان سطرًا واحدًا في ملف إعدادات. في هذا الفصل نتعلّم كيف نُجهّز البيئة مرة واحدة، فتنتهي المشكلة إلى الأبد.

أهداف الفصل

- [ ] إنشاء بيئة بايثون معزولة قابلة للتكرار
- [ ] تثبيت Keras 3.15.1 مع محرّك مناسب
- [ ] اختيار الواجهة الخلفية بمعيار واضح لا بالعادة
- [ ] التحقّق من أن البيئة سليمة قبل كتابة سطر واحد من كود النموذج
- [ ] فهم لماذا لا يكفي تثبيت Keras وحده

2.1 لماذا لا يكفي `pip install keras` ؟

في Keras 2، كان `pip install keras` يعني عمليًا «ثبّت TensorFlow أيضًا». أما Keras 3 فقد فكّ هذا الربط: المكتبة صارت **واجهة فوق محرّكات**، والمحرّك نفسه اختياري.

لذلك فإن الأمر التالي **ينجح** لكنه لا يكفي:

```bash
pip install keras
```

ينجح لأن Keras 3.15.1 تُحمَّل بلا محرّك، لكن أوّل استيراد لـ `keras.ops` سيطلب محرّكًا. إما أن تختار محرّكًا عند التثبيت:

```bash
pip install "keras[torch]"      # محرّك PyTorch
pip install "keras[tensorflow]" # محرّك TensorFlow
pip install "keras[jax]"        # محرّك JAX
```

أو تثبّت Keras والمحرّك منفصلين:

```bash
pip install keras torch
```

> **نصيحة:** صيغة `keras[backend]` أفضل في ملفات `requirements.txt` لأنها تُثبّت المجموعة الكاملة المتوافقة، وتمنع ظهور تناقض في الإصدارات – وهو ما يُفسّر نصف أخطاء التثبيت في بيئات العمل المشتركة.

ما الذي تختاره؟

المعيار	TensorFlow	JAX	PyTorch	OpenVINO
سرعة التدريب على GPU	عالية	الأعلى غالبًا	عالية	لا يُدرّب
توازي النماذج (Sharding)	محدود	الأقوى	محدود	لا
نضج أدوات النشر	الأعلى	متوسط	متوسط	متخصص في Intel
تكامل HuggingFace	جيد	جيد	الأفضل	محدود
حجم التبعيات	كبير	متوسط	كبير	متوسط
الاستخدام المناسب	الإنتاج والخدمات	البحث والتوازي	البحث والمجتمع	الاستدلال على Intel

> **قاعدة عملية:** إن لم يكن لديك سببٌ قاطع، ابدأ بـ **TensorFlow** في العمل الإنتاجي، و**JAX** في العمل البحثي الذي يحتاج توازيًا أو دقة عالية، و**PyTorch** إن كان مجالك NLP أو لديك كود قائم عليه.

2.2 البيئة المعزولة: لماذا؟

تخيّل أنك تعمل على مشروعين: أحدهما يحتاج Keras 3.15 والآخر Keras 2.16. تثبيت الاثنين في نفس بايثون يعني أن أحدهما سيكسر الآخر في كل مرة تنقل بينهما.

الحل: **بيئات معزولة** – كل مشروع في مجلّد خاص به بباكاجاته الخاصة.

على ويندوز

BASH

```
# إنشاء البيئة
python -m venv .venv

# تفعيلها
.venv\Scripts\activate

# ترقية أدوات التثبيت
python -m pip install --upgrade pip

# التثبيت
pip install "keras[torch]==3.15.1"
```

على لينكس وماك

BASH

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install "keras[torch]==3.15.1"
```

هل أُنشئ بيئة لكل مشروع؟

نعم، لكل مشروع.(rule صارمة). لا تتشارك بيئة بين مشروعين لهما متطلّبات مختلفة.

شكل 3 – خطوات تجهيز بيئة تطوير معزولة لـ Keras 3

2.3 تثبيت الإصدارات: من «يعمل» إلى «يعمل دائمًا»

بينما كان مشروعك يعمل على جهازك، ثم على جهاز زميلك، ثم في Docker، ثم في CI – هذا يتطلب تثبيتًا **مُثبَّت الإصدارات** لا تثبيتًا عابرًا.

```bash
# سجّل ما هو مثبّت الآن
pip freeze > requirements.lock.txt
```

وفي `requirements.txt` اكتب:

```text
keras[torch]==3.15.1
numpy>=1.26,<3.0
matplotlib>=3.8
```

القاعدة: **ثبّت بالإصدار الدقيق** (`==`) ما تتعلّق به بشدة – Keras وNumPy – و**ثبّت بالنطاق** (`>=`) ما هو ثانوي.

ملاحظة: ملف `requirements.lock.txt` هو لقطة (snapshot) كاملة: ثبّت منه حين تحتاج إنتاجًا مطابقًا بايت-ل-بايت. ملف `requirements.txt` هو تصريح النوايا: حدّثه حين تريد تجديد.

2.4 أيّ محرّك يعمل؟

متغيّر البيئة

BASH

```
# ويندوز (PowerShell)
$env:KERAS_BACKEND = "jax"

# ويندوز (دائم)
setx KERAS_BACKEND jax

# لينكس/ماك
export KERAS_BACKEND=jax
```

من داخل الكود

PYTHON

```
import keras
keras.config.set_backend("jax")
```

تحذير: `set_backend` يجب أن يُستدعى **قبل** بناء أي نموذج. تغييره بعد بناء النموذج يخلق حالة غريبة: أوزان مبنية على محرّك، وقياسات على محرّك آخر.

التحقّق

```python
import keras

print(keras.__version__)        # 3.15.1
print(keras.backend.backend())  # jax
print(keras.backend.standardize_dtype("float32"))
```

أي محرّك تختار؟ مقارنة صريحة

المحرّك	الأنسب لـ	سرعة التدريب	توازي النماذج	ملاحظات
TensorFlow	نشر واسع · إنتاج	عالية	متوسط	مجتمع ضخم · أدوات MLOps
JAX	أبحاث · دقة	الأعلى	الأقوى	تدرّج تلقائي متقدّم
PyTorch	أبحاث · تخصيص	عالية	متوسط	منظومة HuggingFace
OpenVINO	استدلال على Intel	استدلال فقط	—	قراءة فقط · لا تدريب

قاعدة عملية

ابدأ بـ TensorFlow إن كنت ستنشر، و JAX إن كنت تبحث أو تحتاج توازيًا هائلًا،
و PyTorch إن كان مجالك NLP/Radio أو تملك كودًا قائمًا عليه.

شكل 4 – مصفوفة المحرّكات الخلفية: أيّ محرّك تختار ولماذا

2.5 هل يرى المحرّك معالجك الرسومي؟

هذا السؤال يُطرح كل مرّة، والإجابة تُغيّر سلوك كل شيء بعد ذلك. إليك سكربت تشخيص حقيقي:

PYTHON

```python
import keras
import numpy as np

print("Keras :", keras.__version__)
print("Backend:", keras.backend.backend())

# اختبار تمرير أمامي فعلي
x = np.random.rand(64, 784).astype("float32")
model = keras.Sequential([
    keras.layers.Input((784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10),
])
y = model(x, training=False)
print("Output shape:", y.shape)

# هل الأوزان على معالج؟
try:
    dev = next(iter(model.trainable_variables)).device
    print("Device:", dev)
except Exception as exc:
    print("تعذّر تحديد الجهاز:", exc)
```

كيف تقرأ النتيجة:

النتيجة	المعنى	الإجراء
`Device: cuda:0` أو `gpu`	كل شيء تمام	تابع
`Device: cpu` والمحرّك PyTorch	لا يوجد CUDA أو Torch مبني بـ CPU	ثبّت نسخة GPU من المحرّك
`Device: cpu` والمحرّك JAX	Search J XLA لا يجد GPU	تأكّد من توافق التعريفات
`ModuleNotFoundError`	المحرّك غير مثبَّت	ثبّته

عن CUDA

CUDA ليس «شيئًا تُثبّته في Keras»؛ هو سلسلة من ثلاث طبقات يجب أن تتوافق:

1. **تعريف GPU** (Driver) من موقع NVIDIA.
2. **CUDA Toolkit** – النسخة التي بُني عليها المحرّك.
3. **مكتبة cuDNN** للحسابات العصبية (التلافيف attentiong).

> **نصيحة:** أحدث نسخة CUDA ليست دائمًا الأفضل. المحركات المختبَرة غالبًا تعمل بأداء ممتاز على نسخ CUDA الأقدم قليلًا. تحقّق من جدول التوافق في توثيق محرّكك قبل التثبيت.

2.6 المسارات السريعة: Kaggleg Colab

أحيانًا لا تريد تثبيت شيء. حينها:

```python
# في خلية Colab أو Kaggle
!pip install -q "keras[torch]==3.15.1"
```

المقارنة:

المعيار	Colab / Kaggle	بيئة محلية
زمن البدء	فوري	دقائق
GPU	متاح (محدود زمنيًا)	Depends على عتادك
دوام البيئة	ينتهي عند انتهاء الجلسة	دائم
مناسبة للتجربة	نعم	للتجربة أيضًا
مناسبة للإنتاج	لا	نعم

القاعدة: **جرّب في المتصفح، ابنِ على جهازك.**

2.7 سكربت التحقّق الجاهز

احفظ هذا في `smoke_test.py` واستدعِه في كل بيئة جديدة:

PYTHON

```python
"""فحص بيئة Keras — احفظه باسم smoke_test.py واستدعِه بعد أي ترقية."""
import sys
import numpy as np
import keras

def main() -> int:
    print("=" * 46)
    print("Keras        :", keras.__version__)
    print("Backend      :", keras.backend.backend())
    print("Python       :", sys.version.split()[0])
    print("=" * 46)

    # 1) بناء نموذج
    model = keras.Sequential([
        keras.layers.Input((32,)),
        keras.layers.Dense(16, activation="relu"),
        keras.layers.Dense(1),
    ])

    # 2) تجميع
    model.compile(optimizer="adam", loss="mse")

    # 3) تدريب
    x = np.random.rand(128, 32).astype("float32")
    y = np.random.rand(128, 1).astype("float32")
    hist = model.fit(x, y, epochs=2, batch_size=32, verbose=0)
    assert "loss" in hist.history, "لا يوجد سجل خسارة"

    # 4) تنبّؤ
    out = model.predict(x[:8], verbose=0)
    assert out.shape == (8, 1), f"شكل غير متوقّع: {out.shape}"

    # 5) حفظ وتحميل
    model.save("_smoke.keras")
    reloaded = keras.saving.load_model("_smoke.keras")
    assert reloaded.count_params() == model.count_params()

    print("المعاملات    :", model.count_params())
```

```
    print("الخسارة النهائية:", float(hist.history["loss"][-1]))
    print("\n✓ البيئة سليمة — كل الفحوص ناجحة.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

نصيحة: اجعل هذا السكربت أول خطوة في خط النشر (CI). فشل هذا الاختبار = فشل النشر، بلا محاجّات.

2.8 الأخطاء الشائعة في هذا الفصل

خطأ 1 – تثبيت Keras بلا محرّك. العرض: `ModuleNotFoundError: No module named 'tensorflow'` عند أول استخدام. السبب: Keras 3 لا يجلب محرّكًا تلقائيًا، والقيمة الافتراضية لواجهة Keras هي TensorFlow. الحل: ثبّت المحرّك الذي تريده صراحةً: `pip install "keras[jax]"`.

خطأ 2 – تبديل المحرّك بعد بناء النموذج. العرض: أخطاء غريبة عند `predict` بعد `set_backend`. السبب: الأوزان والقياسات على محرّكين مختلفين. الحل: اضبط `KERAS_BACKEND` قبل تشغيل البرنامج، لا داخله.

خطأ 3 – نسيان تفعيل البيئة. العرض: `pip install` ينجح لكن `import keras` يفشل. السبب: التثبيت ذهب إلى بايثون النظام لا إلى البيئة. الحل: فعّل البيئة أولًا وتحقّق أن مسار التثبيت صحيح: `python -m pip show keras` .

خطأ 4 – تثبيت بلا إصدارات. العرض: المشروع يعمل على جهازك ويكسر عند الزميل. السبب: تثبيت عابر لأحدث الإصدارات. الحل: `pip freeze` واحفظ الملف.

خطأ 5 – بناء على GPU بلا تحقّق. العرض: التدريب بطيء inexplicably. السبب: المحرك يعمل على المعالج لأن CUDA غير مثبَّت. الحل: شغّل سكربت التشخيص أعلاه واقرأ سطر

`Device` .

خطأ 6 – تثبيت كل شيء في Python النظامي. العرض: أخطاء أذونات على لينكس. السبب: تثبيت عالمي. الحل: استخدم بيئة معزولة دائمًا.

التمارين

1. **تشخيص الواجهة الخلفية.** بيئتك تُظهر `backend: tensorflow` بينما `tensorflow` غير مثبَّت. اشرح بالضبط أين تقرأ Keras قيمة الواجهة، ثم صلحها بطريقتين (متغيّر بيئة، وكود). (المتوقع: ضبط `KERAS_BACKEND` قبل الاستيراد، أو `keras.config.set_backend` قبل بناء أي نموذج.)

2. **تثبيت مُثبَّت.** اكتب `requirements.txt` لمشروع يستخدم Keras 3.15.1 وNumPy وMatplotlib، ثم اشرح لماذا تُثبّت الحزمة الأولى بـ `==` وبقية الحزم بـ `⇒`. (المتوقع: تثبيت Keras بالإصدار الدقيق لأنه الواجهة التي يعتمد عليها الكود بشدّة، والنطاقات لبقية الأدوات.)

3. **فحص بيئي.** أضف إلى سكربت التحقّق فحصًا يُبلّغ عن عدد النوى واسم الجهاز، ثم اشرح كيف تقرأ منه هل GPU مستخدَم فعلًا. (المتوقع: سطر `Device` المضبوط على cuda يعني استعمالًا فعليًا؛ `cpu` يعني أن المحرّك لا يرى GPU.)

4. **اختيار المحرّك.** ثلاثّ سيناريوهات: (أ) خدمة تصنيف صور على خادم إنتاجي، (ب) تدريب نموذج بمئة مليار معامل على مجموعة distributed، (ج) استدلال على جهاز لوحي بمعالج Intel. اختر لكلٍّ محرّكًا مع التبرير بمعيار. (المتوقع: (أ) TensorFlow، (ب) JAX، (ج) OpenVINO أو TensorFlow مع LiteRT.)

5. **مصفوفة التوافق.** صمّم جدولًا يربط كل حزمة بأثر ترقية Keras عليها: `keras` , `tensorflow-text` , `keras-cv` , `keras-hub` , `torch` , `jax` , `tensorflow` . (المتوقع: الانتباه إلى أن `keras-cv` أُرشف ودمجت وظائفه في Keras، وأن `keras-hub` مرتبط بإصدار Keras.)

6. **حدّد الخطأ.** هذا السكربت يفشل عند `model.save("_smoke.keras")` :

```python
    model = keras.Sequential([keras.layers.Input((4,)),
keras.layers.Dense(2)])
    model.save("_smoke.h5")
```

اشرح سبب الخطأ، وصحّحه بطريقتين (تغيير الامتداد، وتغيير الصيغة عمدًا). (المتوقع: صيغة `.keras` هي الموصى بها في Keras 3 لأنها تحفظ البنية والtraining state والأوزان معًا؛ صيغة `.h5` تحفظ الأوزان فقط.)

7. **زمن البدء.** قِس زمن أول `import keras` في بيئتك. ثم قارنه بزمن التشغيل الثاني في العملية نفسها. فسّر الفرق. (المتوقع: الفارق يعود إلى كلفة تحميل المكتبات الأصلية؛ في JAX قد يظهر كلفة ترجمة الدوال.)

8. **عزل عملي.** أنشئ مشروعين في المجلّد نفسه: أحدهما يحتاج `keras[torch]` والآخر `keras[jax]`. اكتب الأوامر بالترتيب، واشرح أي ملف يُعدَّل عند التبديل. (المتوقع: وجود بيئتين مستقلّتين و/أو ملف `requirements.txt` لكل مشروع؛ لا حاجة لتعديل أي ملف عالمي.)

9. **خط أنابيب.** اكتب ملف `setup.sh` (ولينكس/ماك) أو `setup.ps1` (ويندوز) يؤدّي: إنشاء البيئة، التفعيل، تثبيت الإصدارات، وتشغيل `smoke_test.py`. (المتوقع: السكربت ينتهي برمز خروج `0` فقط إذا نجحت كل الفحوص.)

10. **فخّ شائع.** أضِف إلى `smoke_test.py` سطرًا يطبع `keras.backend.standardize_dtype("float16")` ثم سطرًا يطبع `keras.backend.standardize_dtype("float64")`. فسّر لماذا يعيد المحرّك `float16` في الحالة الأولى و `float32` في الثانية. (المتوقع: توحيد الدقة يمنع عملاً عدديًا غير مستقر على المحرّكات التي لا تدعم float64.)

ملخص الفصل

- Keras 3 **لا يجلب** محرّكًا؛ ثبّت `keras[backend]` أو المحرك منفصلًا.
- استخدم **بيئة معزولة** لكل مشروع، و**ثبّت الإصدارات** بشدّة.
- تحقّق من البيئة بسكربت تشخيص صغير **قبل** كتابة كود النموذج.
- `KERAS_BACKEND` قبل الاستيراد، ولا تُبدّل المحرّك بعد بناء نموذج.
- اختبر الـ GPU بسطر `Device` ، لا بالافتراض.
- Colab للتجربة، والجهاز المحلي للإنتاج.

الفصل 3: الأوتور: اللغة التي يفكّر بها Keras

بطاقة الفصل

الفصل 3

مهمّة هذا الفصل

يشرح لغة Keras الداخلية: الأوتور وأبعاده، قواعد البثّ، التدرّج التلقائي، ولماذا وُجدت keras.ops.

ماذا تكسب منه

تقرأ رسائل أخطاء الأشكال وتفهمها، وتكتب عمليات تعمل على أي محرّك دون إعادة كتابة.

الدالة	دورها	متى تستخدمها
`keras.ops.array()`	بناء أوتور من قائمة أو مصفوفة NumPy	عند تحويل بيانات جاهزة إلى مدخلات نموذج
`keras.ops.convert_to_tensor()`	تحويل آمن مع تحديد النوع	قبل تمرير بيانات خارجية إلى النموذج
`keras.ops.matmul()`	ضرب المصفوفات	لفهم ما تفعله Dense داخليًّا
`keras.ops.gradients()`	حساب المشتقات تلقائيًّا	عند بناء حلقة تدريب مخصّصة
`keras.Variable`	مصفوفة قابلة للتحديث	للأوزان والحالات في الطبقات المخصّصة

قصة مفتاحية: «لماذا يصرّ المشغّل على أن الشكل يجب أن يكون `(None, 784)` وليس `(784,)`؟» سؤال بدا ساذجًا، لكن الإجابة عنه غيّرت نظرتك إلى الشبكات كلّها. البُعد الأول ليس عددًا من البيانات، بل **عدد الدفعات** – وقد استغرقت شهرًا لأفهم أن Keras لا تشرح لك الأبعاد، بل تفرض عليك فهمها.

أهداف الفصل

- [ ] فهم بنية الأوتور: الرتبة والشكل والنوع والجهاز
- [ ] إتقان قواعد البث ومنع أخطاء الأبعاد
- [ ] فهم كيف يحسب Keras التدرّج التلقائي
- [ ] التمييز بين `keras.ops` وNumPy ولماذا يُستعملان معًا
- [ ] معرفة الفرق بين المتغيّرات القابلة للتعلّم وغير القابلة للتعلّم

3.1 ما هو الأوتور؟

الأوتور (Tensor) هو مصفوفة أرقام multiaبعاد. إذا كانت المصفوفة العادية (NumPy array) هي «صفًا من الأرقام»، فالأوتور هو **مكعّب** من الأرقام له عدد أوجه (رتبة) وامتداد في كل وجه (شكل).

الرتبة والشكل

الرتبة (rank) هي عدد الأبعاد، والشكل (shape) هو tuple يحدّد طول كل بُعد.

الرمز	المعنى	مثال
الرتبة 0	عدد مُفرد	`3.14`
الرتبة 1	متجه	`[1, 2, 3]` – شكله `(3,)`
الرتبة 2	مصفوفة	جدول 3×2 – شكله `(3, 2)`
الرتبة 3	مكدّس	4 صور رمادية 28×28 – شكله `(4, 28, 28)`
الرتبة 4	دفعة صور	4 صور ملوّنة 28×28 – شكله `(4, 28, 28, 3)`

في Keras، الرتبة 4 هي الرتبة السائدة في الرؤية الحاسوبية: **(دفعة، ارتفاع، عرض، قنوات)**.

لماذا `None` في الأول؟

حين تستدعي `model.summary()` سترى:

```text
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         (None, 784)               0
dense (Dense)                (None, 128)               100480
```

الـ `None` في البُعد الأول لا تعني «لا شيء»؛ تعني **«غير محدَّد بعد»** – أي «أي عدد من الدفعات». هذا هوricketa الكتابة التي تمنح النموذج مرونة: يمكنك تدريبه على 32 عيّنة واستدعاؤه على 1 عيّنة أو 10 آلاف، بلا تغيير في الشكل.

> **نصيحة:** لا تفهم `None` على أنها «نقص في البيانات». هي **بُعد حرّ** – هذا هو الفرق بين نموذج عامّ ونموذج مرتبط بحجم دفعة محدّد.

النوع والجهاز

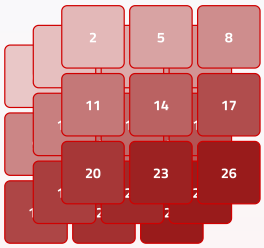
```python
import keras
import numpy as np

x = np.random.rand(4, 28, 28, 3).astype("float32")
t = keras.ops.convert_to_tensor(x)

print(t.shape)   # (4, 28, 28, 3)
print(t.dtype)   # float32
```

النوع (dtype) يحدّد: - **مدى القيم**: `uint8` من 0 إلى 255، و `float32` بقيم تقريبية واسعة. - **حجم التخزين**: `float32` أربع بايتات، و `float16` بايتان. - **سلوك التقريب**: `float32` يتّسع لنحو 7 خانات عشرية.

الجهاز (device) يحدّد مكان التخزين والحساب: `cpu` أو `gpu:0` . في Keras 3 يُفترض أن الأوتور يتبع جهاز النموذج تلقائيًا.

> **تحذير:** Keras 3 يعمل افتراضيًا بـ `float32` ، لا بـ `float16` . هذا مقصود: الشبكة العصبية حسّاسة عدديًا، والدقة المنخفضة غير المختلطة تُنتج `NaN` في طبقات مثل `softmax` . الفصل 36 يتناول الدقة المختلطة بالتفصيل.

الأوتور: صندوق أرقام بأبعاد



شكل (3, 3, 3)

٣ محاور × ٣ صفوف × ٣ أعمدة = ٢٧ قيمة

كل محور يُسمّى dim ويُكتب من اليمين لليسار في TensorFlow

المفهوم	الرماد في Keras	مثال
الرتبة (Rank)	ndim	3
الشكل (Shape)	shape	(3 ,224 ,224 ,32)
النوع (DType)	dtype	float32
الجهاز (Device)	device	cpu / gpu
القيمة (Value)	()numpy	[[2.0 ,1.0]]

شكل 5 – شبكة الأوتور: الرتبة والشكل والأبعاد

3.2 البث: كيف تتوسّع الأبعاد

البث (Broadcasting) يسمح بعمليتين مختلفتين في الشكل دون تعديل صريح. القاعدة في Keras 3 (NumPyg TensorFlowg) تُطبَّق **من اليمين إلى اليسار**:

1. إن كان البُعدان متساويين، تجراهما كما هما.
2. إن كان أحدهما `1` ، وسّعه إلى قيمة الآخر.
3. إن كان أحدهما `None` (حرّ)، وسّعه إلى قيمة الآخر.
4. عدا ذلك: خطأ.

مثال عملي: إضافة متجه إلى مصفوفة

PYTHON

```python
import numpy as np

# مصفوفة 3×2 ومتجه طول 2
a = np.array([[1., 2.], [3., 4.], [5., 6.]])   # (3, 2)
b = np.array([10., 20.])                       # (2,)

# يعمل — لكن ليس بالطريقة التي يتوقّعها المبتدئ
print(a + b)
# [[11. 22.]
#  [13. 24.]
#  [15. 26.]]
```

انظر: المتجه `b` طُويت على طول الصفوف (البُعد الأخير)، لا على طول الأعمدة. والسبب أن رتبة `b` هي 1، وKeras/Numpy تعامل الرتبة المنخفضة على أنها **محاذاة لليمين** (trailing alignment).

الحالة التي تفاجئ الجميع

PYTHON

```python
a = np.array([[1.], [2.], [3.]])   # (3, 1) عمود
b = np.array([[10., 20.]])         # (1, 2) صف

print(a + b)
# [[11. 21.]
#  [12. 22.]
#  [13. 23.]]
```

هذه ليست إضافة متجه إلى متجه، بل **بثّ خارجي**: عمود `(3, 1)` وصفّ `(1, 2)` يلتقيان فينتج جدول `(3, 2)` كامل. هذه بالضبط اللحظة التي يكتشف فيها المطوّر الجديد أن فهم الأبعاد ليس تفصيلًا.

متى يفشل البث؟

```python
a = np.random.rand(3, 1)    # (3, 1)
b = np.random.rand(4, 5)    # (4, 5)
# a + b  →  ValueError
```

المقارنة من اليمين: `1` مقابل `5` ← يُوسَّع `1` إلى `5` ✓. ثم `3` مقابل `4` ← لا أحدهما `1` وليسا متساويين ✗. خطأ.

القاعدة العملية

في Keras، تُكتب الأشكال دائمًا **مع بُعد الدفعة أولًا**. فإذا كان لديك 32 عيّنة، كل شيء يبدأ بـ `32` :

```python
x = np.random.rand(32, 784).astype("float32")    # 32 عيّنة، 784 خاصية
```

شكل 6 – قاعدة البثّ: كيف تتوسّع الأبعاد تلقائيًّا

3.3 التدرّج التلقائي: كيف تتعلّم الشبكة؟

الرسم البياني للحساب

كل ما تفعله الشبكة هو سلسلة تحويلات. تصبح هذه التحويلات **رسمًا بيانيًا** موجّهًا: العُقد هي العمليات، والحواف هي تدفّق البيانات.

في هذا الرسم، لكل حافة **قيمة** (للمُدخل) ولكل حافة **تدرّج** (مشتقّ الخسارة بالنسبة لتلك القيمة).

التمرير الأمامي

يُحسب الناتج من المُدخل:

$$z = Wx + b, \qquad a = f(z)$$

التمرير العكسي

يعود من الخسارة نحو المُدخلات بجمع تدرّجات الأجزاء بماعدة السلسلة:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W}$$

donde: $\frac{\partial L}{\partial a}$ هو تدرّج الخسارة بالنسبة لناتج الطبقة، و$\frac{\partial a}{\partial z}$ هو مشتقّ دالة التنشيط، و$\frac{\partial z}{\partial W}$ هو التدرّج بدلالة الأوزان.

قاعدة السلسلة في صورة أبسط

تخيّل سلسلة من ثلاث حلقات: $L = a \cdot b \cdot c$. كم يتغيّر L إذا تغيّر a؟

$$\frac{\partial L}{\partial a} = b \cdot c$$

الضرب simply. والآن تخيّل سلسلة من ثلاث حلقات لكن كل حلقة هي دالة:

$$L = f(g(h(x)))$$

إذن:

$$\frac{dL}{dx} = \frac{dL}{df} \cdot \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

هذا هو كل التدرّج التلقائي. لا أكثر ولا أقل. Keras لا تفعل شيئًا سحريًا: هي تملك الرسم البياني، وتُنفّذ هذه الضربيات نيابةً عنك.

كيف تتحكّم في الرسم؟

PYTHON

```python
import keras

x = keras.ops.convert_to_tensor([[1.0, 2.0]])
w = keras.ops.convert_to_tensor([[1.0], [1.0]])

with keras.name_scope("forward"):
    z = keras.ops.matmul(x, w)       # الجداء
    a = keras.ops.relu(z)            # التنشيط
    loss = keras.ops.sum(a)          # «الخسارة» البسيطة

print("Loss:", float(loss))
print("Forward value (z):", z)
```

ملاحظة مهمة: لا توجد دالة `keras.ops.gradients` في Keras 3، فلا تبحث عنها في التوثيق. التدرّجات تُحسب تلقائيًا داخل خطوة التدريب (`train_step`) ولا تُعرَض في الواجهة العامة؛ إن احتجتَ تدرّجًا يدويًا فاستعمل أداة التفاضل الخاصّة بالمحرّك (`jax.grad` أو `torch.autograd.grad`). وما توفّره Keras نفسها هو `keras.ops.custom_gradient`: مزخرف يُتيح لك كتابة تدرّج صريح لعملية ما بدل أن يتولّاه المحرّك تلقائيًا.

```python
import keras


@keras.ops.custom_gradient
def cube(x):
    """f(x) = x^3 مع تدرّج صريح مخصّص."""

    def grad(*args, upstream=None):
        if upstream is None:
            (upstream,) = args
        return 3.0 * x**2 * upstream

    return x**3, grad


v = keras.ops.convert_to_tensor(2.0)
print("f(2) =", float(cube(v)))    # النتيجة المتوقعة: 8.0
```

شكل 7 – الرسم البياني للحساب: التمرير الأمامي والعكسي

3.4 `keras.ops` ولِماذا وُجدت؟

Keras 3 مبنية على فكرة **واجهة واحدة فوق محرّكات مختلفة**. ولهذا لا يمكنك الاعتماد على `numpy` مباشرة داخل طبقة: عملية NumPy تعمل على معالج Python خارج تسريع المحرّك.

لذلك وُجدت `keras.ops` : مكتبة عمليات تعمل على **أي** محرّك مدعوم.

```python
import keras

x = keras.ops.array([1.0, 2.0, 3.0])

keras.ops.softmax(x)       # تطبيع احتمالي
keras.ops.relu(x)          # تنشيط
keras.ops.matmul(x, x.T)   # ضرب مصفوفي
keras.ops.mean(x)          # متوسط
keras.ops.argmax(x)        # فهرس أكبر قيمة
```

القاعدة: داخل نموذج أو دالة خسارة مخصّصة، استخدم `keras.ops` لا NumPy. التسريع والنقل بين المحرّكات يبقيان سليمين.

> **نصيحة:** عند قراءة شيفرة Keras نفسها ستجد `keras.ops` في كل مكان. تعلّمها مبكّرًا – فهي لغة Keras الحقيقية.

3.5 المتغيّرات: ما الذي يتغيّر أثناء التدريب؟

في Keras 3، كل حالة قابلة للتعلّم داخل النموذج هي **متغيّر** (`keras.Variable`).

```python
import keras

v = keras.Variable([1.0, 2.0, 3.0], name="weights")
print(v)              # <Variable shape=(3,), dtype=float32, value=[1., 2., 3.]>
print(v.shape, v.dtype)
```

القابل للتعلّم وغير القابل

```python
model = keras.Sequential([
    keras.layers.Input((8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(1),
])

print("قابل للتعلّم   :", len(model.trainable_variables))
print("غير قابل للتعلّم:", len(model.non_trainable_variables))
for w in model.non_trainable_variables:
    print("   •", w.name, w.shape)
```

> **نصيحة:** `BatchNormalization` هو المثال الأوضح. أثناء التدريب يحسب متوسط وتباين الدفعة، ثم يحتفظ بهما كمتغيّرات **غير قابلة للتعلّم** (تُحدَّث بمعدّل متحرّك لا بالمُحسّن). أثناء الاستدلال لا توجد دفعة، فيستخدم هذه القيم المحفوظة. لو لم تكن موجودة، لما كان بالإمكان التنبّؤ على عيّنة واحدة.

كيف يُحدَّث المتغيّر؟

عبر `assign` أو `assign_add`، وهما الطريق الوحيد الآمن:

```python
v.assign([4.0, 5.0, 6.0])      # استبدال كامل
v.assign_add([1.0, 1.0, 1.0]) # جمع
```

3.6 أخطاء شائعة في هذا الفصل

خطأ 1 – نسيان بُعد الدفعة. العرض: `ValueError: Expected the last dimension to be 784`. السبب: مرّرت مصفوفة شكلها `(784,)` بينما النموذج يتوقّع `(None, 784)`. الحل: `x = x[np.newaxis, :]` أو `keras.ops.expand_dims(x, 0)`.

خطأ 2 – استخدام NumPy داخل دالة خسارة مخصّصة. العرض: بطء ملحوظ، أو خطأ عند التشغيل على محرّك مختلف. السبب: `numpy` لا يعرف المحرّك. الحل: استبدل بـ `keras.ops`.

خطأ 3 – تعديل المتغيّر مباشرة. العرض: `ValueError: Attempting to update a variable in-place`. السبب: المكتبات الحديثة التدرّج التلقائي تمنع التعديل المباشر. الحل: `v.assign(...)` أو `v.assign_add(...)`.

خطأ 4 – الخلط بين الأبعاد في الإضافة. العرض: `operands could not be broadcast together`. السبب: افتراض أن المتجه يُضاف على الأعمدة. الحل: أضف `axis` صراحةً أو أعد تشكيل المدخلات.

خطأ 5 – توقّع `None` أن يعني صفرًا. الخلط: تمرير دفعة فارغة. الحل: `None` يعني «أي عدد»، والصفر يعني «لا شيء» – وهو خطأ مختلف تمامًا.

خطأ 6 – الخلط بين `float64` و `float32`. العرض: بطء في الحساب، أو `NaN` في الطبقات الأخيرة. الحل: `keras.ops.cast(x, "float32")`.

التمارين

1. **رتبة وشكل.** احسب رتبة وشكل كلٍّ من: `np.zeros(5)` ، `np.zeros((3, 4))` ، `np.zeros((8, 28, 28, 3))` ، `np.zeros((2, 3, 4))` . ثم اشرح أيّها يقابل دفعة صور. (المتوقع: الرتب 1، 2، 3، 4 على التوالي؛ الأخير هو دفعة 8 صور ملوّنة.)

2. **قواعد البث.** حدّد ناتج كل عملية: `(5,)` ، `(5, 3, 4) + (3, 1)` ، `(5, 3, 4) + (4,)` ، `(2, 3) + (2, 3)` ، `(3, 4) + (5, 1, 1)` . (المتوقع: الأولى والثانية والرابعة تعمل؛ الثالثة تفشل لأن `5` لا يمكن بثّه على `1` في نفس المستوى.)

3. **إصلاح بثّ.** هذا الكود يُنتج نتيجة غير متوقّعة:

```python
a = np.arange(6).reshape(3, 2)
b = np.array([100, 200])
print(a + b)
```

اشرح النتيجة، ثم اكتب تصحيحًا يُضيف `b` إلى **كل صف** كما قصدنا. (المتوقع: البث يوسّع `b` على البُعد الأخير فيُضاف إلى الأعمدة؛ التصحيح: `a + b[None, :]` أو `np.add(a, b[:, None])` لإضافة `b` إلى كل صف.)

4. **التدرّج يدويًا.** احسب تدرّج $L = 3w^2$ عند $w = 2$، وتحقّق من النتيجة عدديًا (بالفرق المحدود) وبواسطة `keras.ops.custom_gradient` . (المتوقع: $\frac{dL}{dw} = 6w = 12$.)

5. **قاعدة السلسلة.** احسب $\frac{dL}{dx}$ لـ $L = \sin(x^2)$ عند $x = 1$، ثم أكّد النتيجة عدديًا. (المتوقع: $\frac{dL}{dx} = 2x\cos(x^2) = 2\cos(1) \approx 1.0806$.)

6. **مقارنة المتغيّرات.** ابنِ نموذجًا فيه `Dense` و `BatchNormalization` ، ثم اطبع `len(model.trainable_variables)` و `len(model.non_trainable_variables)` واشرح كل مجموعة. (المتوقع: المتغيّرات القابلة هي أوزان Dense وتحيزاتها؛ غير القابلة هي متوسّط BatchNorm وتباينه، أي اثنان لكل طبقة BN.)

7. **أين المتغيّر؟** أضف `keras.layers.Layer` مخصّصة بمتغيّر داخلي، ثم اطبع `model.trainable_variables` قبل وبعد `model.fit`. (المتوقع: عدد المتغيّرات لا يتغيّر – التدريب يحدّث القيم لا البنية.)

8. **dtype.** قارن `np.float32` و`np.float64` في عملية على 10 ملايين عنصر: الوقت والحجم. ثم اشرح لماذا يعتمد Keras 3 على `float32`. (المتوقع: `float64` أبطأ بمرّتين تقريبًا وأكبر حجمًا، و`float32` كافٍ لدقة التدريب المعتادة.)

9. **سؤال مفاهيمي.** لماذا يُعدّ `None` بُعدًا حرًّا لا قيمةً ناقصة؟ ما الذي يمنعه من أن يسبّب خطأ عند الاستدعاء بدفعة مختلفة الحجم؟ (المتوقع: لأنه رمز "أي عدد"، فيُوسَّع تلقائيًا إلى حجم الدفعة الفعلي.)

10. **اكتشاف الخلل.** هذا السكربت يعطي `ValueError`:

PYTHON

```
import keras
x = keras.ops.array([1.0, 2.0])
w = keras.ops.array([1.0, 2.0])
y = keras.ops.matmul(x, w)
```

يطلب `matmul` متجهين (المتوقع: `keras.ops.dot`). اشرح السبب بدقّة، وأصلحه بطريقتين: تغيير `w` متجهًا أو جعل `w` متساويي الطول (مصفوفة × متجه) لا (متجه × متجه)؛ التصحيح صفيًّا.)

ملخص الفصل

- الأوتور = رتبة + شكل + نوع + جهاز؛ و `None` تعني «أي عدد».
- البث يُطبَّق من اليمين: تساوٍ أو `1` أو حرّ؛ وعندها يقع الخطأ.
- التدرّج التلقائي = قاعدة السلسلة، نفّذتها Keras نيابةً عنك.
- `keras.ops` هي لغة Keras الحقيقية – استخدمها لا NumPy.
- المتغيّرات القابلة للتعلّم تُحدّثها المُحسّنات، وغير القابلة تُحدّثها القواعد (مثل إحصاءات BatchNorm).
- Keras 3 يعمل بـ `float32` افتراضيًا، وهذا مقصود.

الفصل 4: بنية النموذج: ثلاث طرق لتحديد الشبكة

بطاقة الفصل

الفصل 4

مهمّة هذا الفصل

يقارن ثلاث واجهات لبناء النموذج – Sequential وFunctional وSubclassing – ويعطي قاعدة اختيار صريحة بينها.

ماذا تكسب منه

تبني النموذج بالواجهة المناسبة من أول مرة، وتتجنّب إعادة الكتابة عند إضافة فرع أو مخرج ثانٍ.

الدالة	دورها	متى تستخدمها
`keras.Sequential`	تتابع خطّي بسيط	لشبكات بلا فروع ولا مدخلات متعدّدة
`keras.Input`	تعريف شكل المدخل واسمه	أول خطوة في الواجهة الوظيفية
`keras.Model(inputs, outputs)`	بناء رسم بياني صريح	عند وجود فروع أو مخرجات متعدّدة
`keras.layers.Concatenate`	دمج فروع في متجه واحد	في البنى متعدّدة المدخلات
`keras.saving.register_keras_serializable`	تسجيل صنف مخصّص للحفظ	عند حفظ نموذج يحوي طبقة مخصّصة

قصة مفتاحية: نموذج يعمل في دفتر Jupyter، لكن حفظه يفشل. السبب لم يكن خطأً في الشيفرة، بل في **البنية**: بُني النموذج بطريقة لا تحفظ نفسها. في هذا الفصل نختار طريقة البناء بعناية، لأن الطريقة نفسها تحدّد ما يمكنك فعله لاحقًا: الحفظ، والتصدير، وإعادة الاستخدام.

أهداف الفصل

- [ ] التمييز بين الواجهات الثلاث: Subclassingg Sequentialg Functional
- [ ] اختيار الواجهة المناسبة لكل موقف
- [ ] إتقان بناء النماذج متعددة المدخلات والمخرجات
- [ ] فهم مشاركة الأوزان والفخّ الذي يحملها
- [ ] معرفة الثمن الذي تدفعه عند الوراثة الصريحة

4.1 ثلاث طرق، ثلاث قدرات

حين كتب François Chollet الفصل الأول من كتاب **Deep Learning with Python**، كان أمامه خيار: أن يقدّم Keras كواجهة واحدة أم كثلاث واجهات مختلفة. اختار الثانية – بحجّة أن القيود المفروضة على المستخدم يجب أن تعكس قيود ما يمكن فعله حقًا.

المسح السريع

المعيار	Sequential	Functional	Subclassing
سهولة الكتابة	الأعلى	متوسطة	الأدنى
مدخلات متعدّدة	✗	✓	✓
مخرجات متعدّدة	✗	✓	✓
فروع وتفرّعات	✗	✓	✓
مشاركة الأوزان	✗	✓	✓
حلقة تدريب مخصّصة	محدود	محدود	✓
حفظ `.keras` كاملًا	✓	✓	يحتاج `get_config`
تصدير سهل	✓	✓	يحتاج عملًا إضافيًّا
مناسب للبحث	✗	✗	✓

ثلاثة أساليب — واحدٌ صحيح لكل موقف

Subclassing — تخصيص حر
مرونة قصوى وتحكّم كامل
للأبحاث والبنى الفريدة
أصعب في الحفظ والتصدير

Functional — رسم بياني
أقوى خيار لأغلب المشاريع
يدعم مدخلات/مخرجات متعددة
سهل التصدير والتسلسل

Sequential — تتابع خطي
أبسط صياغة، طبقة بعد طبقة
مثالي للمبتدئين وMLP
لا يدعم تفرّعات متعددة

قاعدة اختيار عملية

ابدأ دائمًا بـ Functional.

إن لم تستطع التعبير عن بنية الشبكة برسم بياني، فأنت أمام حالة خاصة

لا تعقّد النموذج: تعقيد البنية يجلب تعقيد الحفظ والاختبار والنشر.

القاعدة العملية: Functional أولاً · Sequential للمبتدئين · Subclassing عند الضرورة.

شكل 8 – ثلاث واجهات لبناء النماذج: Sequential Functional Subclassing

4.2 الواجهة الأولى: Sequential

المتّسلسل يعني: طبقة بعد طبقة، دون فروع.

```python
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10),
])

model.summary()
```

متى تستخدمه؟ حين تكون الشبكة خطّية سلسلة: طبقة تلي طبقة. هذا يغطّي Majority من شبكات الـ MLP البسيطة.

> **ملاحظة:** لا يمكن لـ Sequential إضافة مدخلين. إن حاولت تمرير قائمتين، ستحصل على خطأ. وهذا مقصود: الـ Sequential يجبرك على البساطة، وهي ميزة في المرحلة الأولى.

4.3 الواجهة الثانية: Functional — الأصل الحقيقي

في الـ Functional، كل طبقة **تُعطى مدخلات صريحة** وتُنتج **مخرجات مسمّاة**. ثم تجمع هذه المخرجات في `keras.Model`.

```python
import keras

inputs = keras.Input(shape=(784,), name="input_layer")
x = keras.layers.Dense(128, activation="relu")(inputs)
x = keras.layers.Dropout(0.2)(x)
x = keras.layers.Dense(64, activation="relu")(x)
outputs = keras.layers.Dense(10, name="output_layer")(x)

model = keras.Model(inputs=inputs, outputs=outputs)
```

لاحظ الفرق الجوهري: `(inputs)` بعد تعريف الطبقة. هذه هي **استدعاء الطبقة على مُدخل**. كل نداء يُنتج "عقدة" في الرسم البياني.

لماذا هذه الواجهة هي «الأصل»؟

لأن Keras 3 نفسه مبني عليها. حين ترى `model.summary()` يعرض طبقات فيها `Connected to: ['input_layer']`، فأنت ترى لغة الرسم البياني مباشرة.

الفرعين: حالة لا يستطيع Sequential التعبير عنها

PYTHON

```python
import keras

# مدخلان
img_in = keras.Input(shape=(32, 32, 3), name="image")
meta_in = keras.Input(shape=(5,), name="metadata")

# معالجة كل فرع
img_feat = keras.layers.Conv2D(16, 3, activation="relu")(img_in)
img_feat = keras.layers.GlobalAveragePooling2D()(img_feat)
meta_feat = keras.layers.Dense(16, activation="relu")(meta_in)

# الدمج
merged = keras.layers.Concatenate()([img_feat, meta_feat])
out = keras.layers.Dense(1, activation="sigmoid")(merged)

model = keras.Model(inputs=[img_in, meta_in], outputs=out)
```

فرع واحد بمخرجين

PYTHON

```python
import keras

inputs = keras.Input(shape=(100,))
shared = keras.layers.Dense(64, activation="relu")(inputs)

head_a = keras.layers.Dense(3, activation="softmax", name="task_a")(shared)
head_b = keras.layers.Dense(1, activation="sigmoid", name="task_b")(shared)

model = keras.Model(inputs=inputs, outputs=[head_a, head_b])
```

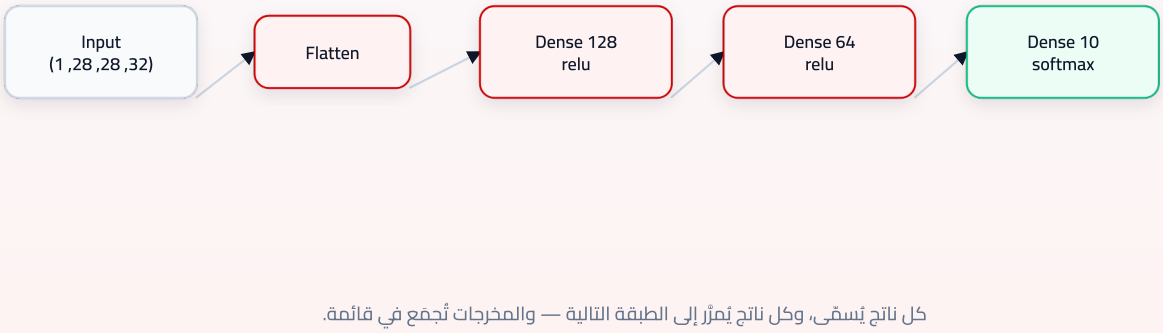
فخّ مشاركة الأوزان

هذا هو الفخّ الذي يوقع المبتدئين، وهو **مقصود** في التصميم:

```python
import keras

inputs = keras.Input(shape=(10,))
layer = keras.layers.Dense(8, activation="relu")

a = layer(inputs)    # نداء أول
b = layer(inputs)    # نفس الطبقة — نداء ثانٍ!

print(layer.weights[0].shape)    # (10, 8) — مصفوفة واحدة فقط
```

تحذير: النداءان لطبقة واحدة **لا ينشئان طبقتين**. تتحمّل الطبقة نفسها الأوزان، وتُشارَك في المسارين. هذا هو معنى «مشاركة الأوزان» (weight sharing) — وهو بالضبط ما تحتاجه في الشبكات مثل Siamese. وإن أردت طبقتَين مستقلّتين، فعّل `layer_a = Dense(8)` و`layer_b = Dense(8)` منفصلتين.

الـ Functional API: تدفّق الرسم البياني



كل ناتج يُسمّى، وكل ناتج يُمرَّر إلى الطبقة التالية — والمخرجات تُجمَع في قائمة.

شكل 9 – الرسم البياني الوظيفي: العُقد والحواف والتفريعات

4.4 الواجهة الثالثة: Subclassing

حين تحتاج منطقًا لا يمكن أن يصفه الرسم البياني – مثل حلقة تكرارية بمتغيّر، أو تمرير إضافي، أو تحكّم داخلي في التدرّج – Careersرث الصريح هو الحل.

PYTHON

```python
import keras

class MyModel(keras.Model):
    def __init__(self, units=64, **kwargs):
        super().__init__(**kwargs)
        self.dense1 = keras.layers.Dense(units, activation="relu")
        self.dense2 = keras.layers.Dense(units, activation="relu")
        self.out = keras.layers.Dense(1)

    def call(self, x, training=False):
        # منطق مخصّص: Residual connection على مرحلتين
        h = self.dense1(x)
        h = self.dense2(h)
        return self.out(x + h)
```

القاعدة الذهبية: أنشئ الطبقات في `__init__` لا في `call`

PYTHON

```python
# ✗ خطأ: إنشاء طبقة داخل call
def call(self, x):
    d = keras.layers.Dense(64)    # خطأ: طبقة جديدة في كل استدعاء!
    return d(x)

# ✓ صح: أنشئها مرة واحدة في __init__
def __init__(self):
    super().__init__()
    self.d = keras.layers.Dense(64)
```

لماذا؟ لأن الطبقات المنشأة في `call` **لا تُسجَّل** في النموذج: لا تظهر في `summary`، ولا تُحفظ مع الأوزان، ولا يعرف المُحسّن أن يحدّثها. النتيجة نموذج «يشتغل» ولا يتعلّم.

بنية النموذج: `build` و `compute_output_shape`

PYTHON

```python
class ScaledDense(keras.layers.Layer):
    def __init__(self, units, scale=2.0, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.scale = scale

    def build(self, input_shape):
        self.kernel = self.add_weight(
            shape=(input_shape[-1], self.units),
            initializer="glorot_uniform",
            trainable=True,
            name="kernel",
        )
        super().build(input_shape)

    def call(self, inputs):
        return inputs @ self.kernel * self.scale

    def get_config(self):
        config = super().get_config()
        config.update({"units": self.units, "scale": self.scale})
        return config
```

ملاحظة: `build()` تُستدعى مرّة واحدة عند أول تمرير، وتُنشئ الأوزان. و `get_config()` هي ما يسمح لـ Keras بإعادة بناء الطبقة عند التحميل – بدونها لا يمكن حفظ النموذج.

4.5 مقارنة تفصيلية: أيّهما تختار؟

السؤال	إن كان جوابك «نعم»	اختر
هل أحتاج مدخلات متعدّدة أو مخرجات متعدّدة؟	✓	Functional
هل أحتاج فروعًا أو مسارات متعدّدة؟	✓	Functional
هل أريد أن أحفظ النموذج وأعيد تحميله بسهولة؟	✓	Functional
هل أحتاج مشاركة أوزان عبر فروع؟	✓	Functional
هل أحتاج حلقة تدريب مخصّصة أو خسرًا غير قياسي يتعلّم؟	✓	Subclassing
هل أحتاج تمرير معلومات إضافية داخل `call` ؟	✓	Subclassing
هل هي شبكة خطّية بسيطة للتجربة السريعة؟	✓	Sequential

التسلسل العملي الموصى به

1. **ابدأ بـ Sequential** للنموذج الأول: تعلّم الحركة بلا تعقيد.
2. **انتقل إلى Functional** فور حاجتك لتفرّع أو مدخلات متعدّدة – وهي الحالة الغالبة في المشاريع الحقيقية.
3. **لا تلجأ إلى Subclassing** إلا حين يعجز الرسم البياني عن تمثيل فكرتك، وعندها كافئ نفسك: هل تحقّقت من قابلية الحفظ والتسلسل (serialization)؟

> **قاعدة:** نموذج يمكن حفظه واستخدامه بعد عام أفضل من نموذج أدقّ بنصف نقطة مئوية ولا يمكن حفظه.

4.6 مثال محلول: نموذج متعدّد المهام

المطلوب: تصنيف المنتج إلى ثلاث فئات **و** تقدير سعره. المخرجان يتفرّعان من م مشترك.

PYTHON

```
import keras

def build_model(input_dim=784, hidden=128):
    # ── الجذع المشترك
    inputs = keras.Input(shape=(input_dim,), name="input")
    x = keras.layers.Dense(hidden, activation="relu", name="shared_1")
(inputs)
    x = keras.layers.BatchNormalization(name="bn_1")(x)
    x = keras.layers.Dropout(0.3, name="drop_1")(x)
    x = keras.layers.Dense(hidden // 2, activation="relu", name="shared_2")
(x)

    # ── الرأسان
    cls = keras.layers.Dense(3, activation="softmax", name="category")(x)
    reg = keras.layers.Dense(1, name="price")(x)

    model = keras.Model(inputs=inputs, outputs=[cls, reg],
name="multi_task")
    return model


model = build_model()
model.summary(line_length=78)
```

التدريب: كل مخرج يحتاج خسارته ومقياسه:

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss={
        "category": keras.losses.CategoricalCrossentropy(),
        "price": keras.losses.MeanSquaredError(),
    },
    metrics={
        "category": [keras.metrics.CategoricalAccuracy()],
        "price": [keras.metrics.MeanAbsoluteError()],
    },
)

history = model.fit(
    x_train, {"category": y_category, "price": y_price},
    epochs=10, batch_size=64, validation_split=0.2,
)
```

ملاحظة مهمة: عندما يكون المخرجان قاموسًا، يجب أن تكون مفاتيح القاموس **أسماء المخرجات** كما في `keras.Model(outputs=[cls, reg])` – وإلا أخطأ Keras في مطابقة الخسائر.

4.7 الأخطاء الشائعة

خطأ 1 – تمرير قائمة إلى Sequential. العرض: `ValueError: The model expects a single input`. الحل: انتقل إلى Functional.

خطأ 2 – إنشاء طبقة داخل `call`. العرض: النموذج لا يتعلّم؛ الخسارة ثابتة. الحل: أنشئ الطبقات في `__init__`.

خطأ 3 – توقّع مستقلّية النداء. العرض: طبقتان seemed مستقلّتان لكنهما تتشاركان الأوزان. الحل: أنشئ كائنَي Dense منفصلين.

خطأ 4 – نسيان `get_config`. العرض: `ValueError: Could not locate class 'MyLayer'`. الحل: نفّذ `get_config` وسجّل الطبقة بـ

`@keras.saving.register_keras_serializable(package="MyPkg")` .

خطأ 5 – خلط المفتاح والقيمة في `fit` مع مخرجات قائمة. العرض: `KeyError: 'category'`. الحل: استخدم قاموسًا بمفاتيح تطابق أسماء المخرجات.

خطأ 6 – بناء النموذج قبل ضبط الواجهة الخلفية. الحل: اضبط `KERAS_BACKEND` قبل استيراد وبناء أي نموذج.

التمارين

1. **أيّ واجهة؟** صف ثلاث مهام: (أ) مصنّف بايثون بسيط على جدول، (ب) شبكة تحلّل صورًا مع بيانات وصفية، (ج) نموذج يتعلّم عمق الشبكة يدويًا. اختر الواجهة مع التبرير. (المتوقع: (أ) Sequential أو Functional مع Dense، (ب) Functional، (ج) Subclassing.)

2. **Sequential إلى Functional.** أعد كتابة النموذج التالي بـ Functional، ثم أضف طبقة `Dense(32)` بعد الإخراج مباشرةً (rama جديدة). (المتوقع: تقسيم الشبكة إلى فرعين ثم `Concatenate` ؛ لا يمكن وضع طبقة بعد الإخراج في Sequential.)

3. **فخّ المشاركة.** شغّل:

```python
inputs = keras.Input(shape=(10,))
d = keras.layers.Dense(8)
a = d(inputs); b = d(inputs)
print(len(d.weights))
```

اشرح لماذا النتيجة واحدة، ثم اصنع نسختين مستقلّتين. (المتوقع: مصفوفة واحدة؛ استدعاء `Dense(8)` مرّتين ينتج كائنين منفصلين.)

4. **معلومة إضافية.** اكتب طبقة تأخذ `call(self, x, extra=None)` وتُضيف `extra` إلى الناتج. ثم اربطها بنموذج Functional يمرّر `extra` من `fit` . (المتوقع: `call` بمعاملين إضافيين؛ مرّر عبر `call` مخصّص أو اربطها داخل `Model` بتمرير القيم.)

5. **مخرجان.** ابنِ نموذجًا يخرج (أ) احتمالات 4 فئات، (ب) قيمة انحدار واحدة، من جذع مشترك، ثم اكتب `compile` المناسب. (المتوقع: `loss` و `metrics` قاموسان بمفتاحين.)

6. **معلموذج مخصّص.** اكتب `keras.layers.Layer` تحسب متوسط آخر محور بمعامل `axis`، مع `get_config` صحيح. ثم احفظ النموذج وأعد تحميله وتأكّد تطابق المخرجات. (المتوقع: `get_config` يعيد `axis`؛ بعد التحميل تعطي المخرجات نفسها.)

7. **تشخيص.** هذا النموذج لا تتغيّر أوزانه:

```python
class Bad(keras.Model):
    def __init__(self):
        super().__init__()
    def call(self, x):
        x = keras.layers.Dense(64, activation="relu")(x)
        return keras.layers.Dense(1)(x)
```

اشرح الخطأ وصحّحه. (المتوقع: الطبقات تُنشأ في `call` فتُستبدل كل استدعاء؛ الحل: نقلها إلى `__init__`.)

8. **قرار معماري.** صمّم بنية لنموذج يقبل: صورة (3×224×224)، وسجلّ مشتريات متغيّر الطول، وسمات المستخدم. صف المداخل والمخرجات وطريقة الدمج. (المتوقع: ثلاثة `Input`، مسار لكل فرع، `Concatenate`، ورأسان.)

9. **جدول مقارنة خاص.** أنشئ جدولًا يقارن الواجهات الثلاث من حيث: هل يمكن حفظه ثم إعادة استخدامه في تطبيق آخر؟ (المتوقع: Functional الأسهل، Sequential مقبول، Subclassing يحتاج تسجيل الطبقة و `get_config`.)

10. **تدريب مخصّص.** اشرح لماذا قد تحتاج Subclassing لتدريب نموذج بأوزان مقيّدة (مثل `constraint`): أين تكتب القيد؟ (المتوقع: في `call` بعد الحساب، قبل الإخراج، بقصّ التدرّجات بـ `keras.ops.clip` أو تطبيق القيد على المتغيّر.)

ملخص الفصل

- ثلاث واجهات: Sequential للبساطة، Functional للقوة والوضوح، Subclassing للحرية القصوى.
- الـ Functional هو جوهر Keras 3: كل طبقة نداء على مُدخل، والمخرجات تُجمَع.
- نداء الطبقة مرّتين = مشاركة الأوزان، لا طبقتان.
- في Subclassing: الطبقات في `__init__` و `get_config` شرط الحفظ.
- ابدأ بـ Functional ما لم تكن هناك حاجة حقيقية لحرية غير مقيّدة.

الفصل 5: أول نموذج كامل: شرح سطر بسطر

بطاقة الفصل

الفصل 5

مهمّة هذا الفصل

يبني أول نموذج كامل سطرًا بسطر: البيانات، البناء، التجميع، التدريب، التقييم، ثم التنبّؤ.

ماذا تكسب منه

تحصل على قالب عملي تشغّله في دقائق وتفهم دور كل سطر فيه قبل أن تنسخه إلى مشروعك.

الدالة	دورها	متى تستخدمها
`keras.datasets.mnist.load_data()`	تحميل بيانات جاهزة	للتجارب الأولى فقط
`model.compile()`	ربط النموذج بخطة التدريب (محسّن، خسارة، مقاييس)	قبل أي fit
`model.fit()`	حلقة التدريب كاملة	في كل تدريب
`model.evaluate()`	قياس الأداء على بيانات لم تُرَ	بعد التدريب وقبل النشر
`model.predict()`	إنتاج تنبّؤات جديدة	في الاستخدام أو الخدمة
`keras.utils.set_random_seed()`	بذرة موحّدة لكل المكتبات	لأي تجربة تريد تكرارها

قصة مفتاحية: أول نموذج لك سيبدو ساذجًا: أربع طبقات، ومئة ألف عيّنة، وست دقائق تدريب. ثم ستفهم، سطرًا سطرًا، لماذا كل واحد من هذه السطور موجود. لأن النموذج الأول ليس غاية، بل **أداة تشخيص**: كل ما تعلّمته فيه ينطبق على كل ما يأتي بعده.

أهداف الفصل

- [ ] تجهيز بيانات MNIST من الصفر إلى tensors جاهزة
- [ ] بناء نموذج تصنيف كامل في Functional
- [ ] شرح كل معامل في `compile` و `fit` و `evaluate` و `predict`
- [ ] التمييز بين logits والاحتمالات
- [ ] قراءة مخرجات `fit` وفهم `History`

5.1 المتطلّبات قبل أن نبدأ

لا حاجة لأي أداة خارج `numpy` و `keras`. سنبني كل شيء من الصفر:

PYTHON

```python
import numpy as np
import keras

keras.utils.set_random_seed(42)   # تكرار النتائج
```

ملاحظة: `set_random_seed` يثبّت مولّد الأعداد العشوائية في Pythong NumPyg Keras معًا. بدونه، كل تشغيل يعطي دقّة مختلفة قليلًا – وهو ما يجعل المقارنات بين تجارب مُضلِّلة.

5.2 جلب البيانات

PYTHON

```python
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("x_train:", x_train.shape, x_train.dtype)   # (60000, 28, 28) uint8
print("y_train:", y_train.shape, y_train.dtype)   # (60000,) uint8
print("y_train[:10]:", y_train[:10])              # [5 0 1 1 ...]
```

ماذا نرى هنا؟

الملاحظة	الدلالة
الشكل `(60000, 28, 28)`	60 ألف صورة رمادية 28×28 – **بلا بُعد القنوات**
النوع `uint8`	قيم من 0 إلى 255 – تحتاج تحويلًا
الأشكال بلا `None`	لا يوجد بُعد دفعة؛ سنضيفه عند الإدخال

> **نصيحة:** البيانات في Keras تأتي من مجلد `~/.keras/datasets` cante، فيُحمَّل مرّة واحدة ويُخزَّن محليًا. في اتصال إنترنت محدود أو بيئة إنتاج، نزّلها مسبقًا وانسخ المجلّد.

تقسيم بيانات التحقق

PYTHON

```python
# تقسيم عشوائي 80/20 من التدريب
idx = np.random.permutation(len(x_train))
split = int(0.8 * len(x_train))
x_tr, x_val = x_train[idx[:split]], x_train[idx[split:]]
y_tr, y_val = y_train[idx[:split]], y_train[idx[split:]]
```

التحويل إلى Float

PYTHON

```python
x_tr = x_tr.astype("float32") / 255.0
x_val = x_val.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

لماذا القسمة على 255؟ لأن قيم البكسل الأصلية في `[0, 255]`. التحويل إلى `[0, 1]` يضع كل الخصائص على المقياس نفسه، ويمنح التدرّجات نطاقًا معقولًا. القسمة على 255 تحافظ على هذه النسبة.

> **تحذير:** لا تنسَ القسمة. تدريب على قيم Treat `uint8` كمصفوفة بدون؟ يعمل، لكن التدرّجات تتضخّم ألف مرّة والنتائج أسوأ بكثير.

5.3 بناء النموذج

PYTHON

```
inputs = keras.Input(shape=(28, 28), name="image")

# التسطيح: من (28 ,28) إلى (784,)
x = keras.layers.Flatten(name="flatten")(inputs)

# الطبقة الكثيفة الأولى: مستخرج الميزات
x = keras.layers.Dense(128, activation="relu", name="dense_1")(x)

# لا :الإسقاط_connections عشوائية 20% من الوحدات
x = keras.layers.Dropout(0.2, name="dropout")(x)

# الطبقة الأخيرة: logits (بلا تنشيط)
outputs = keras.layers.Dense(10, name="logits")(x)

model = keras.Model(inputs=inputs, outputs=outputs, name="mnist_mlp")
model.summary()
```

المخرجات المتوقّعة:

```text
Model: mnist_mlp
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃ Param #       ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ image (InputLayer)              │ (None, 28, 28)         │ 0             │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ flatten (Flatten)               │ (None, 784)            │ 0             │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_1 (Dense)                 │ (None, 128)            │ 100,480       │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dropout (Dropout)               │ (None, 128)            │ 0             │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ logits (Dense)                  │ (None, 10)             │ 1,290         │
└─────────────────────────────────┴────────────────────────┴───────────────┘
Total params: 101,770
```

لماذا الطبقة الأخيرة بلا `softmax` ؟

هذا قرارٌ مقصود وله ثلاثة أسباب:

1. **الاستقرار العددي**: حساب softmax ثم logarithm مباشرة قد يوصل `p` إلى الصفر، فيصبح `log(0)` . مع `from_logits=True` تتكفّل Keras بالتحويل الآمن.
2. **السرعة**: softmax واحدة داخل دالة الخسارة بدل softmax ثم خسارة.
3. **المرونة**: في الاستدلال يمكنك اختيار softmax أو لا، حسب ما يحتاجه المستهلِك.

> **نصيحة:** في الإنتاج، إذا كان المستهلك يحتاج احتمالات، طبّق softmax **عند الاستدلال** لا عند التدريب.

5.4 التجميع: `compile`

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
)
```

شرح كل معامل

المعامل	القيمة هنا	لماذا
`optimizer`	`Adam(1e-3)`	معلّم تعلّم AdamAdaptivo معقول
`loss`	`SparseCategoricalCrossentropy(from_logits=True)`	تسميات فردية + مخرجات logits
`metrics`	`SparseCategoricalAccuracy()`	قياس الأداء لا تحسينه

لماذا `Sparse` ؟

تسميات MNIST أرقام: `[7, 2, 1, ...]` . هذا **تمثيل متفرّق**.shoppingالمقابل هو `Categorical` الذي يتوقّع متجهًا `[0,0,...,1,...,0]` .

||format|شكل التسمية|ملاحظة | |---|---|---|---| | `SparseCategoricalCrossentropy` | `(batch,)` | أرقام | الأسرع، الافتراضي لـone-hot | MNIST | | `CategoricalCrossentropy` | `(batch, 10)` | يحتاج `to_categorical` |

ملاحظة: `metrics=["accuracy"]` (نصّي) ما زال يعمل في Keras 3، لكنه لا يتتبّع حالة `compile` على نحو مفصّل. استخدم كائنات `keras.metrics.*` الصريحة – فهي تُعيد الحالة الصحيحة عبر `evaluate` وتُصدَّر مع النموذج.

5.5 التدريب: `fit`

PYTHON

```
history = model.fit(
    x_tr, y_tr,
    epochs=20,
    batch_size=64,
    validation_data=(x_val, y_val),
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=3,
            restore_best_weights=True,
        )
    ],
)
```

شرح كل معامل

المعامل	الأثر الدقيق
`epochs`	عدد المرّات الكاملة على بيانات التدريب
`batch_size`	عدد العيّنات في خطوة تحديث واحدة
`validation_data`	تُقيَّم بعد كل حقبة، ولا تُدرَّب عليها
`callbacks`	إجراءات تُنفَّذ أثناء التدريب
`shuffle`	خلط البيانات قبل كل حقبة (افتراضي `True`)
`verbose`	`0` صامت، `1` شريط تقدّم، `2` سطر لكل حقبة
`steps_per_epoch`	مطلوب فقط مع مولّدات لا لها طول معروف

ما الذي يحدث داخل `fit`؟

لكل حقبة: 1. خلط بيانات التدريب. 2. تقسيمها إلى دفعات بحجم `batch_size`. 3. لكل دفعة: تمرير أمامي ← حساب الخسارة ← تمرير عكسي ← تحديث الأوزان. 4. بعد الحقبة: تقييم على بيانات التحقق.

قراءة ناتج `fit`

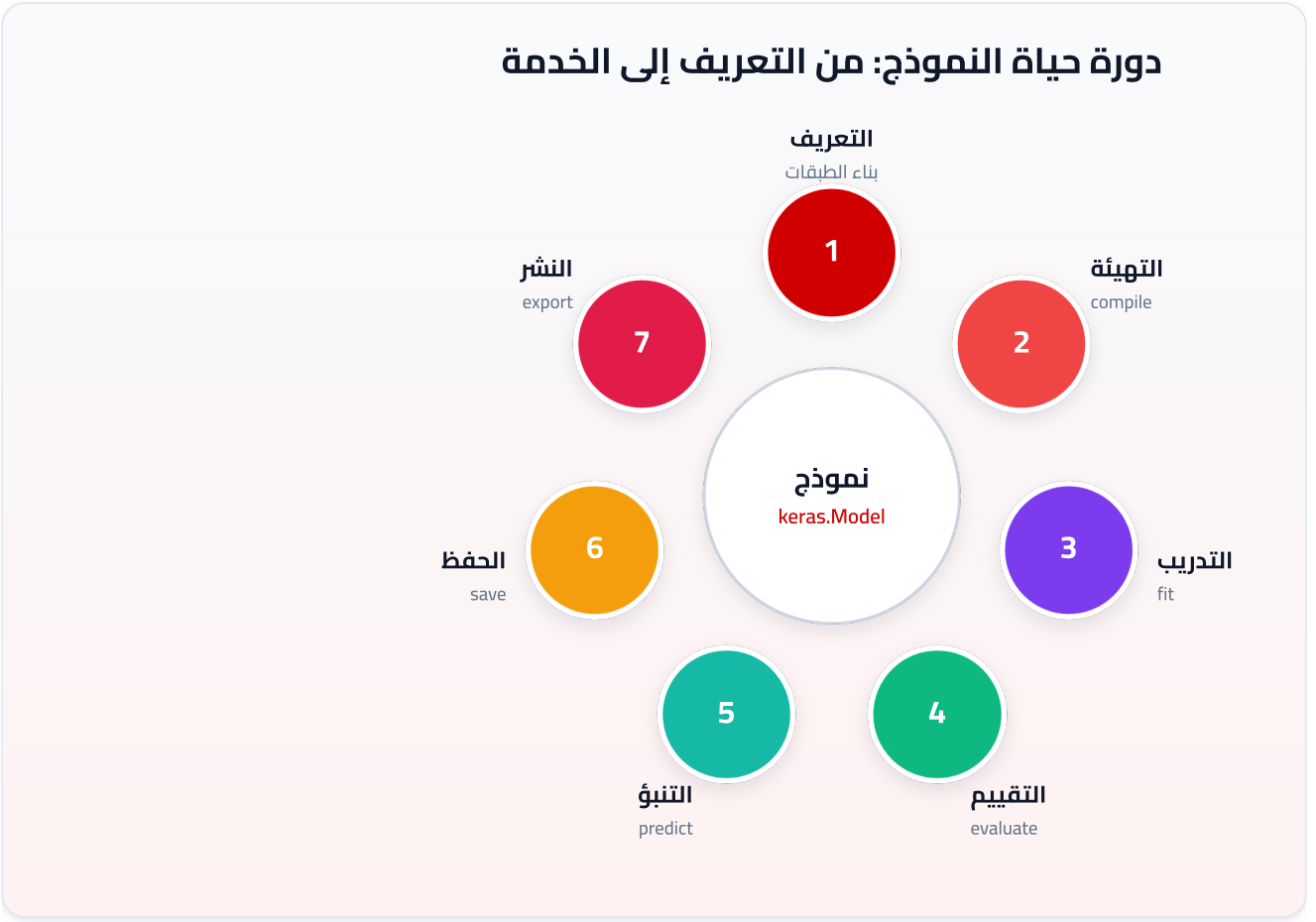
```python
print(history.history.keys())
# dict_keys(['loss', 'compile_metrics', 'val_loss', 'val_compile_metrics'])

print("أدنى خسارة تدريب :", min(history.history["loss"]))
print("أدنى خسارة تحقق :", min(history.history["val_loss"]))
```

في Keras 3، المقاييس تُجمَّع في مفتاح واحد باسم `compile_metrics`، وتُقسَّم داخليًا بحسب اسم المقياس. إن أردت مفاتيح منفصلة لكل مقياس، مرّر المقياس باسم صريح:

```python
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],
)
```

كم يستغرق؟

على معالج حديث: أقل من دقيقة لعشر حقبات على 48 ألف عيّنة. زمن التدريب **نادرًا** ما يكون عنق الزجاجة في النماذج المتوسطة.



شكل 10 – دورة حياة النموذج: بناء ← تجميع ← تدريب ← تقييم ← تنبّؤ

5.6 التقييم والتنبّؤ

`evaluate` : قياس على بيانات لم يرها النموذج

```python
results = model.evaluate(x_test, y_test, return_dict=True)
print(results)
# {'loss': 0.0352, 'compile_metrics': 0.9891}
```

الدالة	الغرض	تدرّج؟
`evaluate`	قياس الأداء على بيانات تقييم	لا
`predict`	إنتاج تنبّؤات	لا

`predict` : إنتاج المخرجات

```python
logits = model.predict(x_test[:5], verbose=0)
print("شكل المخرجات:", logits.shape)   # (5, 10)

probs = keras.ops.softmax(logits, axis=-1)
classes = keras.ops.argmax(probs, axis=-1)

for i in range(5):
    print(f"الصورة {i}: logits={np.round(logits[i], 2)}")
    print(f"          argmax={int(classes[i])} · الثقة=
{float(probs[i].max()):.4f}")
```

الفرق الدقيق بين `evaluate` و `predict`

- `evaluate` يحسب **الخسارة والمقاييس** على دفعات، ويطبّع `Dropout` (لأنه يمرّر بـ `training=False`).

- `predict` يعيد **المخرجات الخام** (logits في حالتنا) بصيغة `numpy` .
- `predict` لا يقارن شيئًا بالتسميات الحقيقية.

> **تحذير:** `model.predict(x_test)` على 10 آلاف عيّنة تستهلك ذاكرة كبيرة لتخزين المخرجات. في الإنتاج، استخدم `model(x, training=False)` في حلقة، أو `predict` على دفعات.

5.7 النموذج الكامل في ملف واحد

PYTHON

```python
"""mnist_mlp.py — أول نموذج كامل، قابل للتشغيل من أعلى إلى أسفل."""
import numpy as np
import keras


def main():
    keras.utils.set_random_seed(42)

    # 1) البيانات
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_test = x_test.astype("float32") / 255.0

    idx = np.random.permutation(len(x_train))
    cut = int(0.8 * len(x_train))
    x_tr, x_val = x_train[idx[:cut]], x_train[idx[cut:]]
    y_tr, y_val = y_train[idx[:cut]], y_train[idx[cut:]]
    x_tr = x_tr.astype("float32") / 255.0
    x_val = x_val.astype("float32") / 255.0

    # 2) النموذج
    inputs = keras.Input(shape=(28, 28), name="image")
    x = keras.layers.Flatten(name="flatten")(inputs)
    x = keras.layers.Dense(128, activation="relu", name="dense_1")(x)
    x = keras.layers.Dropout(0.2, name="dropout")(x)
    outputs = keras.layers.Dense(10, name="logits")(x)
    model = keras.Model(inputs=inputs, outputs=outputs, name="mnist_mlp")

    # 3) التجميع
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
    )

    # 4) التدريب
```

```
    history = model.fit(
        x_tr, y_tr,
        epochs=20, batch_size=64,
        validation_data=(x_val, y_val),
        callbacks=[keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=3, restore_best_weights=True)],
    )

    # 5) التقييم
    print("\n═══ التقييم على مجموعة الاختبار ═══")
    results = model.evaluate(x_test, y_test, return_dict=True)
    print("الخسارة :", round(float(results["loss"]), 4))
    print("الدقّة  :", round(float(results["accuracy"]), 4))

    # 6) تنبّؤ نموذجي
    logits = model.predict(x_test[:3], verbose=0)
    probs = keras.ops.softmax(logits, axis=-1)
    print("\n═══ أمثلة على التنبّؤ ═══")
    for i in range(3):
        print(f"الحقيقة={y_test[i]}  التنبّؤ={int(keras.ops.argmax(probs[i]))}  "
              f"الثقة={float(keras.ops.max(probs[i])):.3f}")

    # 7) الحفظ
    model.save("mnist_mlp.keras")
    print("\n✓ حُفظ النموذج في mnist_mlp.keras")


if __name__ == "__main__":
    main()
```

الناتج المتوقّع (تقريبي، يختلف قليلًا بحسب البذرة):

```text
═══ التقييم على مجموعة الاختبار ═══
الخسارة : 0.0311
الدقّة  : 0.9904

═══ أمثلة على التنبّؤ ═══
الحقيقة=7  التنبّؤ=7  الثقة=0.998
الحقيقة=2  التنبّؤ=2  الثقة=0.999
الحقيقة=1  التنبّؤ=1  الثقة=0.997
```

5.8 لماذا MNIST لم تعد كافية؟

نموذج MLP بسيط يحقق 99% على MNIST. يبدو رائعًا – حتى تفهم السياق:

- MNIST **أبيض وأسود فقط** (28×28، رمادي).
- الكتابة **موضّعة مركزيًا** ولا يوجد تشويش.
- الفئات **مفصولة بصريًا** (الرقم 7 لا يشبه 2).
- لا يوجد الخلفية، ولا الإضاءة، ولا التشويه.

القصور ليس في النموذج، بل في المهمة. أي نظام رؤية إنتاجي اليوم (سيارات ذاتية القيادة، فحص طبي، تحقّق من الهوية) يختبر TN البعيدة: إضاءة متغيّرة، زوايا مختلفة، ضباب، خدوش، أجزام جزئية.

> **قاعدة:** حين تحصل على دقّة عالية جدًا على مجموعة معروفة، لا تفرح – اسأل: ما الذي في هذه المجموعة لا يشبه العالم الحقيقي؟

5.9 الأخطاء الشائعة

خطأ 1 – نسيان التحويل إلى float. العرض: الخسارة تصبح `nan` أو التدريب لا يتقارب. الحل: `x.astype("float32") / 255.0` .

خطأ 2 – softmax مع `from_logits=True`. العرض: الخسارة صغيرة جدًا وغير منطقية، والدقّة عشوائية. السبب: softmax مطبَّقة مرّتين. الحل: اختر أحدهما.

خطأ 3 – `CategoricalCrossentropy` مع تسميات فردية. العرض: `ValueError: y_true and y_pred have different shapes`. الحل: `SparseCategoricalCrossentropy`.

خطأ 4 – `validation_split` مع بيانات مُرتّبة زمنيًا. في MNIST غير ضار (الترتيب عشوائي)، لكنه كارثي في السلاسل الزمنية. الحل: قسّم يدويًا.

خطأ 5 – خلط `metrics` مع `loss`. الـ loss يُحلَّل، والـ metric يُقاس. الدقّة لا يمكن أن تكون خسارة.

خطأ 6 – قراءة `predict` كاحتمالات. الحل: طبّق softmax يدويًا عند الحاجة.

خطأ 7 – `accuracy` كمقياس نصّي. استخدم `keras.metrics.SparseCategoricalAccuracy()`.

التمارين

1. **عدد المعاملات.** احسب يدويًا عدد معاملات النموذج أعلاه، ثم أكّدها بـ `model.count_params()`. (المتوقع: 100,480 + 1,290 = 101,770.)

2. **غيّر شيئًا واحدًا.** بدّل `Dense(128)` بـ `Dense(32)`، دُرّب 10 حقبات، وقارن زمن الحقبة ودقّة التحقق. (المتوقع: تدريب أسرع، ودقّة أدنى قليلًا.)

3. **أزل `Dropout`.** قِس الفارق في دقّة التحقق. (المتوقع: فرق صغير على MNIST لأن المهمة سهلة؛ الفارق يظهر على بيانات أصعب.)

4. **أضف `BatchNormalization`.** قِس أثره في سرعة التقارب. (المتوقع: تقارب أسرع في الحقبات الأولى.)

5. **غيّر الخسارة.** استبدل `SparseCategoricalCrossentropy` بـ `CategoricalCrossentropy` مع `keras.utils.to_categorical(y, 10)`. (المتوقع: نفس النتائج تقريبًا، وبطء طفيف في التدريب.)

6. **كسر السكربت ثلاث ways.** (أ) `loss="sparse_categorical_crossentropy"` مع softmax في الإخراج؛ (ب) `x_tr` بلا قسمة على 255؛ (ج) `metrics=["categorical_accuracy"]` مع تسميات فردية. اشرح عرض كل خطأ وأصلحه. (المتوقع: (أ) `nan` أو دقّة عشوائية، (ب) تقارب بطيء أو `nan`، (ج) خطأ شكل أو دقّة صفرية.)

7. **طبقة softmax يدويًا.** احسب softmax على logits نموذجك وتحقّق أن المجموع = 1. (المتوقع: `keras.ops.softmax` مع `axis=-1` يعطي صفوفًا مجموعها 1.)

8. **صورة واحدة.** احصل على تنبّؤ النموذج لصورة واحدة: كيف تُدخلها؟ (المتوقع: `x[None, ...]` لإضافة بُعد الدفعة، أو `model(x[np.newaxis, ...])`.)

9. **خطأ في الاستدلال.** احسب `model.predict(x_test[:10])` ثم `model(x_test[:10], training=False)`. قارن القيمتين. (المتوقع: متطابقتان؛ لكن `predict` أخفّ ذاكرةً لأنه لا يبني سجلات التدريب.)

10. هل تكفي MNIST؟ اقترح ثلاث مهام رؤية أصعب واذكر ما الذي تختبره MNIST ولا تختبره. (المتوقع: مثل recognition في مشاهد طبيعية، كشف العيوب الصناعية، تصنيف الأشعة – بإضاءة وت backgrounds متغيّرة.)

ملخص الفصل

- حمّل البيانات، اقسّمها **أنت**، ثم حوّلها إلى `float32` مقسومة على 255.
- ابنِ بـ Functional، واجعل الطبقة الأخيرة **بلا تنشيط** مع `from_logits=True` .
- `compile` يحدّد كيف يُحلَّل، و `fit` ينفّذ، و `evaluate` يقيس، و `predict` ينتج.
- `history.history` يخزّن الخسارة والمقاييس لكل حقبة.
- `Dropout` معطّل أثناء `evaluate` و `predict` تلقائيًا.
- دقّة 99% على MNIST تعني أن **المهمة سهلة**، لا أنك ممتاز.

الجزء الثاني

واجهة Keras الأساسية: الطبقات ودوال الخسارة

مفردات Keras الأساسية واحدة واحدة: الطبقات، دوال التنشيط، دوال الخسارة، المُحسّنات، المقاييس، التهيئة، الاستدعاءات، والتنظيم. لكل عنصر: دوره، متى يُستخدم، وكيف يُخطئ الناس فيه.

ما تكسبه من هذا الجزء

تستطيع النظر إلى أي نموذج Keras وتسمية كل جزء فيه وشرح وظيفته، وتختار المكوّن الصحيح بمعيار لا بالعادة.

فصول هذا الجزء

6 الطبقات الأساسية: Dense والتنشيط والتسطيح

7 دوال التنشيط: أين يموت التدرّج؟

8 دوال الخسارة: كيف نترجم الخطأ إلى رقم؟

9 المُحسّنات: كيف يتحرّك النموذج نحو الحل؟

10 المقاييس: كيف تقيس نموذجك دون أن ينخدع به الجميع

11 التهيئة: نقطة الانطلاق

12 الـ Callbacks: أتمتة دورة التدريب

13 التنظيم والقيود: كيف تمنع النموذج من الحفظ بدل التعلّم

2

الفصل 6: الطبقات الأساسية: Dense والتنشيط والتسطيح

بطاقة الفصل

الفصل 6

مهمّة هذا الفصل

يشدّد على الطبقات الأولى: Inputg Dense Dropoutg Activationg Reshapeg Flatteng Maskingg، مع حساب البارامترات.

ماذا تكسب منه

تحسب عدد البارامترات وتتوقّع شكل الخرج بنفسك، فتقرأ model.summary() بثقة بدل أن تخمّن.

الدالة	دورها	متى تستخدمها
`keras.layers.Dense(units, activation)`	طبقة خطّية كاملة	للميزات الجدولية ورؤوس التصنيف
`keras.layers.Input(shape)`	تعريف شكل المدخل	بداية أي نموذج وظيفي
`keras.layers.Flatten()`	تسطيح الأبعاد إلى متجه	قبل Dense بعد الالتفاف
`keras.layers.Reshape(target_shape)`	إعادة تشكيل بلا تغيير البيانات	لتغذية الصور أو التسلسلات بالشكل المطلوب
`keras.layers.Dropout(rate)`	إسقاط عشوائي أثناء التدريب فقط	لتنظيم الشبكات الكبيرة
`keras.layers.Masking()`	تعليم الخطوات غير الصالحة	في التسلسلات ذات الحشو

قصة مفتاحية: طبقة `Dense(128)` تبدو بسيطة: «128 وحدة». لكن هذه العبارة تخفي مصفوفة وزن من 100,480 عددًا، ومتّجه تحيز من 128، وقرارًا حول دقّة عشرية. في هذا الفصل نفكّ الطبقة simplest جزءًا، ونكتشف أن «بسيط» و«ساذج» ليسا الشيء نفسه.

أهداف الفصل

- [ ] تشريح `Dense` رياضيًا وحساب عدد معاملاتها
- [ ] فهم دور التحييز ولماذا لا يُحذف
- [ ] التمييز بين `Flatten` و`Reshape` و`GlobalAveragePooling`
- [ ] فهم آلية `Dropout` بالضبط ولماذا يُطبَّق بعد `Activation`
- [ ] اختيار `Dropout` أو `SpatialDropout` بالأساس
- [ ] بناء نموذج MLP كامل ومعلومة معاملاته

6.1 تشريح `Dense`

الطبقة الكثيفة تنفّذ التحويل الخطي:

$$z = Wx + b, \qquad a = f(z)$$

حيث: - W مصفوفة أوزان بشكل (n_{in}, n_{out}). - b متّجه تحيز بشكل $(n_{out},)$. - f دالة التنشيط (اختيارية).

حساب عدد المعاملات

$$\#params = n_{in} \times n_{out} + n_{out}$$

مثال: `Dense(128)` على مُدخل من 784 خاصية: $784 \times 128 + 128 = 100{,}480 + 128 = 100{,}608$... لا: 100,480 وزنًا و128 تحيزًا = **100,608**؟ تصحيح: $784 \times 128 = 100{,}352$، فالمجموع $100{,}352 + 128 = 100{,}480$ معاملًا.

لماذا التحييز ضروري؟

بدون b، تمرّ كل النقط عبر **نقطة الأصل**. عمليًا: لو كان classificateur يعتمد على "مجموع الخصائص أكبر من 5"، فلا يمكنه تصنيف نقطة bernilai 6 و-1 (مجموعها 5) على أنها مختلفة عن (3، 2)

(مجموعها 5) – مع التحييز تصبح العتبة قابلة للنقل.

ملاحظة: `use_bias=False` موجود في Keras لكنه نادر الاستخدام. إنه مفيد في طبقات واسعة داخل بنى مثل ResNet حيث يُطبَّق التطبيع لاحقًا، أو في الحسابات التي يجب أن تكون متمركزة على الصفر.

شكل 11 – تشريح طبقة Dense: المدخلات والأوزان والتحييز والمخرج

6.2 `Input` و `InputLayer`

PYTHON

```python
import keras

# الطريقة المستحسنة: Input
inputs = keras.Input(shape=(28, 28), name="image")

# المكافئ: InputLayer
model = keras.Sequential([
    keras.layers.InputLayer(shape=(28, 28), name="image"),
    keras.layers.Dense(10),
])
print(model.input_shape)   # (None, 28, 28)
```

المعامل	المعنى
`shape`	شكل **عينة واحدة** (بلا بُعد الدفعة)
`batch_size`	إن حدّدته، ثبّت بُعد الدفعة
`dtype`	نوع البيانات المتوقّع

نصيحة: إن كنت ستُعطي `batch_size` ثابتًا، فكّر مرّتين: الحlinflexible نموذج يقبل دفعة واحدة فقط، ويصعب معه الاختبار بـ batch=1. اتركه `None`.

6.3 التسطيح وإعادة التشكيل

`Flatten`

```python
x = keras.layers.Flatten()(keras.Input((28, 28)))
print(x.shape)   # (None, 784)
```

`Flatten` يحوّل كل الأبعاد بعد بُعد الدفعة إلى بُعد واحد. **لا يضيف معلومات ولا يغيّرها** – ترتيب مسطّح فقط.

`Reshape`

```python
x = keras.layers.Reshape((7, 7, 4))(keras.Input((196,)))
print(x.shape)   # (None, 7, 7, 4)
```

`Reshape` يعيد ترتيب الأبعاد. Unlike `Flatten`، يمكن أن يعيد ترتيب الأبعاد. changes_البنية دون تغيير عدد العناصر.

`GlobalAveragePooling1D`

```python
x = keras.layers.GlobalAveragePooling1D()(keras.Input((100, 64)))
print(x.shape)   # (None, 64)
```

يحسب **متوسط كل محور زمني** لكل خاصية. الفرق الجوهري عن `Flatten`:

الخاصية	Flatten	GlobalAveragePooling1D
المخرجات لكل عيّنة	6400 قيمة	64 قيمة
Losing positional info	يفقدها	يفقدها (لكن بتلخيص)
المعاملات في الطبقة التالية	كثير جدًا	قليل جدًا
Shield against overfitting	ضعيف	أقوى

نصيحة: في الشبكات العميقة، `GlobalAveragePooling` بدل `Flatten` يقلّل عدد المعاملات بأ orders of magnitude ويقلّل فرط التخصيص. كثير من البنى الحديثة (MobileNet، EfficientNet) تفعل ذلك عمدًا.

6.4 دوال التنشيط كطبقات

PYTHON

```python
import keras

# الطريقة المختصرة (مفضّلة)
x = keras.layers.Dense(64, activation="relu")(x)

# الطريقة الصريحة
x = keras.layers.Dense(64)(x)
x = keras.layers.Activation("relu")(x)
```

لماذا نفضّل المختصرة؟ لأنها تجعل Network أضيق، وتُبقيDfGypi تطبيع التهيئة في مكان واحد. كما أن Keras يدمجها في الخلية نفسها، فلا تكلفة إضافية.

ملاحظة: `keras.activations.get("relu")` تسترجع الدالة من اسمها، و`keras.activations.serialize` تفعل العكس – وهما مفيدتان عند بناء دوال خسارة أو

طبقات مخصّصة تقبل اسم تنشيط كمعامل.

6.5 `Dropout`: ماذا يفعل بالضبط؟

الآلية

أثناء التدريب، لكل وحدة في الطبقة، يُحذف **عنصرًا عشوائيًا** (العنصر يُصفر)Independently لكل عيّنة.

$$h_i' = \begin{cases} 0 & \text{باحتمال } p \\ \frac{h_i}{1-p} & \text{باحتمال } 1-p \end{cases}$$

لماذا القسمة على $(1-p)$؟ للحفاظ على **المتوقّع** للناتج كما هو. بدونها لانكمش الخرج بنسبة $1-p$ في كل طبقة، وتراكميًا ينهار.

لماذا بعد التنشيط وليس قبله؟

الترتيب	النتيجة
`Activation` ← `Dropout` ← `Dense`	ينصح به Keras: الإسقاط يعمل على نواتج **بعد** التنشيط
`Dropout` ← `Activation` ← `Dense`	يماثل الأول عمليًا لأن ReLU يقطع السالب

القاعدة العملية في Keras: **ضع `Dropout` بعد التنشيط**.

أثناء الاستدلال

الإسقاط **معطّل تمامًا** في `evaluate` و`predict`. Keras يستدعي الطبقات بـ `training=False` تلقائيًا في هاتين الحالتين. لهذا السبب:

- `model.evaluate` يعطي نتيجة **مستقرّة**.

- `model.predict` يعطي نفس النتيجة في كل مرة.

> **تحذير:** إن استدعيتَ الطبقات يدويًا داخل `training=True` فستحصل على مخرجات عشوائية. هذا سلوك صحيح، لكنه يربك من اعتاد على `predict` .

`SpatialDropout` : الخطأ الشائع

PYTHON

```python
# أبعاد مربّعة (4D)
x = keras.layers.SpatialDropout2D(0.2)(x)

# أبعاد تسلسلية (3D)
x = keras.layers.SpatialDropout1D(0.2)(x)

# أبعاد مكانية (5D)
x = keras.layers.SpatialDropout3D(0.2)(x)
```

	Dropout	SpatialDropout2D
ما يُسقَط	بكسلات متفرّقة	قنوات **كاملة**
الشكل `(B, H, W, C)`	عنصر عشوائي	قناة C كاملة لكل موضع
النتيجة	يكسر بنية الفضاء	يحذف خاصية بصرية كاملة
الاستخدام الصحيح	MLP (أبعاد متجهة)	CNN (أبعاد مكانية)

> **نصيحة:** في CNN استخدم `SpatialDropout2D` ، وضعه بعد كتلة الالتفاف والتجميع. `Dropout` العادي في CNN يُنتج "ثقب" في الصورة – أظهرت الأبحاث أنه يضرّ الأداء بدل أن ينفعه.

6.6 `Masking`: تعليم الشبكة ما يهمّ

```python
x = keras.layers.Masking(mask_value=0.0)(keras.Input((100, 64)))
```

`Masking` تعلّم الطبقات التالية أي المواضع صالحة. حين تدخل في LSTM أو Attention، تتجاهل المواضع المقنّعة بدل معالجتها كأنها بيانات حقيقية.

الفصل 16 و25 يتعمّقان في هذا. لكن الفكرة هنا: **بيانات متغيّرة الطول تحتاج قناعًا**.

6.7 الطبقات المخصّصة: الأساس

```python
import keras

class Scale(keras.layers.Layer):
    """يضرب الخرج في معامل قابل للتعلّم."""

    def __init__(self, init_scale=1.0, **kwargs):
        super().__init__(**kwargs)
        self.init_scale = init_scale

    def build(self, input_shape):
        self.scale = self.add_weight(
            name="scale",
            shape=(),
            initializer=keras.initializers.Constant(self.init_scale),
            trainable=True,
        )

    def call(self, inputs):
        return inputs * self.scale

    def get_config(self):
        return {**super().get_config(), "init_scale": self.init_scale}
```

ثلاث قواعد في `keras.layers.Layer`

1. **أنشئ المتغيّرات في `build`** (أو عبر `__init__` و`add_weight`).
2. **لا تنشئ متغيّرات في `call`** – ستُفقد عند الحفظ.
3. **نفّذ `get_config`** – بدونه لا يُحفظ النموذج.

> **نصيحة:** `keras.layers.Layer` هي نقطة التوسّع الحقيقية في Keras 3. حين تنتهي حدود المكتبة، لا تحتاج أن تكتب «إطار عمل» – تحتاج طبقات قليلة.

6.8 جدول حساب المعاملات

البنية	الوصف	المعاملات
`Dense(784→128)`	طبقة أولى على 784	100,480
`Dense(128→64)`	طبقة ثانية	8,256
`Dense(64→10)`	رأس التصنيف	650
المجموع		109,386
`Dropout(0.2)`	إسقاط	0
`Flatten`	تسطيح	0
`BatchNormalization(128)`	تطبيع دفعة	256 (مقياس + انزياح)

> **ملاحظة:** `BatchNormalization` يضيف معاملين لكل قناة: γ وβ. تُحسب على المحور الذي لا يشمل الدفعة.

6.9 مثال محلول: شبكة MLP بثلاث طبقات

PYTHON

```
import keras
import numpy as np

def build_mlp(input_dim=784, hidden=(256, 128), n_classes=10, dropout=0.3):
    inputs = keras.Input(shape=(input_dim,), name="input")
    x = inputs
    for i, units in enumerate(hidden):
        x = keras.layers.Dense(units, activation="relu", name=f"dense_{i + 
1}")(x)
        x = keras.layers.Dropout(dropout, name=f"drop_{i + 1}")(x)
    outputs = keras.layers.Dense(n_classes, name="logits")(x)
    model = keras.Model(inputs, outputs, name="mlp")
    return model


model = build_mlp()
model.summary(line_length=76)

# التحقّق اليدوي من المعاملات
expected = 784 * 256 + 256 + 256 * 128 + 128 + 128 * 10 + 10
print("المتوقّع يدويًا:", expected)
print("فعليًا          :", model.count_params())
assert expected == model.count_params()
```

الناتج المتوقّع:

TEXT

```
Expected          : 217,226
Actual            : 217,226
```

6.10 الأخطاء الشائعة

خطأ 1 – `Flatten` على مُدخل مسطّح أصلًا. العرض: `Input(shape=(784,))` ثم `Flatten` . النتيجة: شكل `(None, 784)` بعد التسطيح (لا يضرّ)، لكن إن كان `Input(shape=(28,28))` و `Flatten` مرّتين، فالمشكلة الحقيقية في مكان آخر. احسب Shapes دائمًا.

خطأ 2 – `Dropout` في CNN. العرض: تدهور الأداء رغم «التنظيم». الحل: `SpatialDropout2D` .

خطأ 3 – إسقاط مفرط. `Dropout(0.5)` على طبقة صغيرة يُبطئ التعلّم بشدّة دون فائدة. القاعدة: 0.1-0.3 للطبقات المخفيّة، 0.5 كحد أقصى قبل رأس التصنيف.

خطأ 4 – تطبيعي izing في `call` . الحل: `build` أو `add_weight` في `__init__` .

خطأ 5 – نسيان `get_config` . العرض: `Could not locate class 'Scale'` عند التحميل. الحل: نفّذ `get_config` وسجّل الطبقة.

خطأ 6 – توقّع أن `Dropout` يقلّل المعاملات. لا: `Dropout` لا معاملات له إطلاقًا. `Dense` هو الذي يملك المعاملات.

خطأ 7 – `Flatten` قبل `Dense` في نموذج تصنيف صور كبير. `Dense(1024)` على 150,528 مدخلًا = 154 مليون معامل. الحل: `GlobalAveragePooling2D` أو CNN.

التمارين

1. **احسب المعاملات.** احسب يدويًا معاملات: `Dense(50)` على 30 مدخلًا، `Dense(20)` على 50، `Dense(3)` على 20. (المتوقع: 1550 + 1020 + 63 = 2633.)

2. **اذكر `Dense` بلا تحيز.** كم معاملًا في `Dense(10)` على 5 مدخلات مع `use_bias=False` ؟ وماذا يخسر النموذج؟ (المتوقع: 50 معاملًا فقط؛ يفقد القدرة على نقل حدّ القرار.)

3. **`Flatten` مقابل `GlobalAveragePooling`.** مُدخل `(None, 32, 32, 64)` . كم قيمة تُنتج كل عملية، وكم معاملًا تحتاج `Dense(10)` بعد كلٍّ منهما؟ (المتوقع: 65,536 قيمة → 655,370 معاملًا؛ أو 64 قيمة → 650 معاملًا.)

4. **قاعدة الإسقاط.** احسب التوقّع nomadic لخروج وحدة واحدة بعد `Dropout(0.25)` : كم احتمال أن تبقى؟ وما قيمتها المتوقّعة بعد القسمة على $1 - p$؟ (المتوقع: 75% بقاء، ومتوقّع 1.0.)

5. **أين `Dropout` ؟** أيّ ترتيب تختار بين `Dense→Dropout→Activation` و `Dense→Activation→Dropout` ؟ اشرح. (المتوقع: `Dense→Activation→Dropout` هو الموصى به في Keras.)

6. **اختر الإسقاط.** ثلاثة طرائق: (أ) شبكة MLP على بيانات جدولية، (ب) CNN على صور، (ج) شبكة على تسلسلات زمنية. اختر نوع الإسقاط مع التبرير. (المتوقع: (أ) `Dropout` ، (ب) `SpatialDropout2D` ، (ج) `Dropout` أو `SpatialDropout1D` .)

7. **طبقة مخصّصة.** اكتب `keras.layers.Layer` اسمها `Shift` تُضيف متغيّرًا قابلًا للتعلّم إلى المُدخل. ثم احفظ النموذج وأعد تحميله. (المتوقع: `add_weight` + `build` `get_config` +؛ التحميل ينجح والقيم تُستعاد.)

8. **أين المتغيّر؟** عدّل `Shift` لتُنشئ المتغيّر داخل `call` بدل `build` ، ثم أعد تشغيل التدريب. (المتوقع: المتغيّر يُستبدل في كل استدعاء، فلا يتعلّم – وهذا يوضّح أهمية `build` .)

9. **وسّع الشبكة.** أضف `BatchNormalization` بعد كل `Dense`، وأعد حساب المعاملات. (المتوقع: إضافة 2×256 + 2×128 = 768 معاملًا.)

10. **قرار تصميم.** صمّم طبقة `Flatten` بديلة «محوّطة» تحتفظ بالمعلومات المكانية (مثل `Reshape` إلى (7,7,4)). أيّهما تختار ولماذا؟ (المتوقع: `Reshape`؛ `Flatten` يمحو الترتيب المكاني بينما `Reshape` يحافظ على بنيته.)

ملخص الفصل

- `Dense` = $Wx + b$، ومعاملاتها $n_{in}n_{out} + n_{out}$.
- `Flatten` يعيد ترتيبًا، `Reshape` يغيّر بنية، `GlobalAveragePooling` يلخّص ويقلّل المعاملات.
- `Dropout` يقسم على $1 - p$ ليحافظ على التوقّع، وهو معطّل عند الاستدلال.
- في CNN استخدم `SpatialDropout2D` لا `Dropout`.
- `Masking` يعلّم الشبكة ما يهمّ في التسلسلات.
- الطبقات المخصّصة تحتاج `build` و `get_config` لتُحفظ.

الفصل 7: دوال التنشيط: أين يموت التدرّج؟

بطاقة الفصل

الفصل 7

مهمّة هذا الفصل

يفسّر لماذا تحتاج الطبقة الخطّية إلى شريك غير خطّي، ويقارن ReLU وعائلتها وSigmoid وTanh وGELU وSoftmax.

ماذا تكسب منه

تعرف أين يموت التدرّج ولماذا، وتختار التنشيط المناسب لعمق شبكتك ونوع مخرجها.

الدالة	دورها	متى تستخدمها
`keras.activations.relu()`	التنشيط الافتراضي	في الطبقات المخفية للشبكات العميقة
`keras.activations.gelu()`	تنشيط ناعم شائع في المحوّلات	في ViT وBERT وأمثالها
`keras.activations.softmax()`	تحويل الأرقام إلى توزيع احتمالي	في مخرج التصنيف أحادي التسمية
`keras.activations.sigmoid()`	احتمال مستقل واحد	في التصنيف الثنائي ومتعدّد التسميات
`keras.activations.get()`	تحويل نصّ إلى دالّة تنشيط	عند قراءة التنشيط من ملف إعدادات

قصة مفتاحية: في اجتماع مراجعة مشروع، سألتُ مهندسًا لماذا أضاف `activation="relu"` إلى طبقات `Dense` بينما الطبقات أصلًا "تتعلّم" عبر الأوزان. ردّ ببساطة: "لأن الشبكة توقّفت عن التعلّم". بعد ساعة من فحص السجلّات، تبيّن أن السبب لم يكن نقص بيانات بل دالة تنشيط مناسبة: شبكةٌ بطبقات خطية بحتة انهرت إلى مستقيم واحد. في هذا الفصل نفهم لماذا أصبح التنشيط شرطًا وجوديًّا لا تحسينًا اختياريًّا، وأين بالضبط يموت التدرّج داخل الشبكة.

أهداف الفصل

- [ ] إثبات أن تكديس الطبقات الخطية ينتج خريطة خطية واحدة، وربط ذلك بنظرية ليشنو في universality.
- [ ] استيعاب خريطة دوال التنشيط (sigmoid، tanh، ReLU، LeakyReLU، ELU، SELU، GELU، Swish) بصيغها المشتقّة من المشتقة.
- [ ] تفسير متلازمة التدرّج المتلاشي والمتفجّر، وسبب نجاح ReLU ثم GELU/Swish في المحوّلات.
- [ ] التمييز الحاسم بين sigmoid وsoftmax وبين الانتاج من نوع logits، وفهم gimmick `from_logits=True`.
- [ ] استخدام واجهة Keras 3Activation و `keras.activations.*` وتخصيص دالة تنشيط خاصة.

7.1 لماذا يحتاج الطبق الخطي إلى شريك غير خطي؟

تخيّل أنك تبني ثلاثة أسطر: `Dense(64)` ثم `Dense(64)` ثم `Dense(1)`، وأنت تظنّ أن عمق الشبكة سيمنحها قدرة تمييز غير خطية. في اللحظة التي تحذف فيها `activation`، تصبح الشبكة – رياضيًّا – معادلة واحدة. لا يوجد عمق، ولا تدرّجات، ولا تعلّم. الفكرة هنا ليست عقابية بل بنائية: **قدرة النموذج على تمثيل دالة غير خطية تأتي حصرًا من دالة التنشيط**، وكل ما عداها هو تركيب خطي للمدخلات.

هذه ليست ملاحظة تجريبية بل نتيجة رياضية: نظرية ليشنو (1993) تقول إن الشبكة العصبية متعددة الطبقات لا تستطيع التقريب الجبريّ لأي دالة مستمرة على مجموعة محدّبة **إذا وفقط إذا** كانت دالة تنشيطها ليست دالة متعددة الحدود (polynomial). أي أن ReLU وGELU وsigmoid كلها تحقّق الشرط، أما «الإ Arcال page معرفتها» فتنهار إلى الانحدار الخطي مهما بلغ عمقها.

لذلك فإن السؤال الأول قبل بناء أي شبكة ليس «كم طبقة؟» بل «ما دالة التنشيط؟». ولهذا الفصل في قلب كل قرار تصميمي لاحق: الخسارة (الفصل 8)، المُحسّن (الفصل 9)، والتهيئة (الفصل 11) كلها تتفاعل مع شكل دالة التنشيط وسلوك مشتقّتها.

7.2 الرياضيات: من التركيب الخطي إلى خريطة دوال التنشيط

تركيب الطبقات الخطية ينهار

الطبقة الخطية رقم ℓ تُعرَّف بـ:

$$h^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$$

حيث $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ مصفوفة الأوزان بين الطبقة ℓ والطبقة $\ell - 1$، و$b^{(l)} \in \mathbb{R}^{n_l}$ متجه التحييز، و$h^{(l)}$ ناتج الطبقة.

الآن طبّق طبقتَين متتاليتين:

$$W^{(2)}\left(W^{(1)}x + b^{(1)}\right) + b^{(2)} = \left(W^{(2)}W^{(1)}\right)x + \left(W^{(2)}b^{(1)} + b^{(2)}\right)$$

لاحظ النتيجة: خريطة affine واحدة بمصفوفة واحدة ومتجه تحييز واحد. بالتعميم على L طبقة:

$$z^{(L)} = \left(W^{(L)}W^{(L-1)}\cdots W^{(1)}\right)x + \sum_{l=1}^{L}\left(W^{(L)}\cdots W^{(l+1)}\right)b^{(l)}$$

حيث $z^{(L)}$ ناتج الطبقة الأخيرة. المعادلة تقول شيئًا قاسيًا: **التركيب الخطي لأي عدد من الطبقات يساوي خريطة خطية واحدة**، ولا يضيف عمقًا تعبيريًّا. الشبكة "العميقة" الخطية في حقيقتها انحدار خطي واحد مسطّح.

عند إدخال دالة تنشيط σ غير خطية، تتغيّر المعادلة إلى:

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

هنا لا يمكن ملاحظة أن التركيب لم يعد قابلًا للتبسيط: σ لا تُوزَّع على المجموع، فتظهر اللاخطية الحقيقية. هذه هي الخلاصة النظرية كاملة في معادلتين.

خريطة دوال التنشيط وصيغها

نصنّف الدوال إلى أربع عائلات، ولكلٍّ سلوك تدرّج مختلف:

العائلة الأولى: الدوال المشبعة (saturating). الدالة اللوجستية (sigmoid):

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$$

مداها $(0,1)$، ومشتقّتها تبلغ قيمتها العظمى 0.25 عند $z=0$ وتقترب من الصفر في الطرفين. الدالة tanh هي رِفعة مقيسة: $\tanh(z) = 2\sigma(2z) - 1$، مداها $(-1,1)$ ومشتقّتها القصوى 1.

العائلة الثانية: عائلة ReLU. الدالة $\mathrm{ReLU}(z) = \max(0, z)$، ومشتقّتها 1 عند $z>0$ و0 عند $z<0$. البدائل "المُسرَّبة" تُبقي تدرّجًا صغيرًا في النطاق السالب:

$$\mathrm{LeakyReLU}(z) = \max(0,z) + \alpha\min(0,z), \qquad \mathrm{ELU}(z) = \begin{cases} z & z > 0 \\ \alpha(e^{z}-1) & z \le 0 \end{cases}$$

حيث $\alpha \in (0,1)$ معامل التسرّب. أما SELU فهي الدالة الوحيدة التي تُصمَّم لتكون "خلايا ذاتية التطبيع":

$$\mathrm{SELU}(z) = \lambda_{\text{selu}}\big(\alpha_{\text{selu}} e^{z} - 1\big)\ (z \le 0), \qquad \lambda_{\text{selu}} z\ (z > 0)$$

مع $\lambda_{\text{selu}} \approx 1.0507$ و$\alpha_{\text{selu}} \approx 1.6733$، وهي قيم مختارة بحيث يبقي التوزّع activations مطبَّعًا ذاتيًّا. عمليًا: SELU تعني "مطبَّعًا ذاتيًّا" فلا تحتاج BatchNormalization، GELU/Swishg تعني "سلسًا وغير أحادي".

العائلة الثالثة: الدوال الناعمة غير الأحادية. وهنا نصل إلىGBRUSHING المعيار الحديث في المحوّلات:

$$\mathrm{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{z}{\sqrt{2}}\right)\right), \qquad \mathrm{SiLU}(z) = z \cdot \sigma(z)$$

حيث Φ هي دالة التوزيع التراكمي الطبيعي القياسي، وerf دالة الخطأ. GELUExact تُقارب بمُعامل $\text{approximate} = \text{True}$ داخل Keras. لِماذا هاتان؟ لأنهما **غير أحاديتين**: للدالة نقطة قيمة أدنى داخل النطاق السالب، أي أنها "تعرف" كيف تتعافى من مُدخلات سالبة بدل أن تقطعها عند الصفر تمامًا.

العائلة الرابعة: التقريب الصلب. $\text{hard_sigmoid}(z) = \max(0, \min(1, 0.2z + 0.5))$ و$\text{hard_tanh}(z) = \max(-1, \min(1, z))$: وهما تقريبان خطّيان رخيصان الحساب، يُستخدمان في مسارات الاستدلال حين تكون السرعة أهمّ من جودة التقريب.

أين يموت التدرّج؟

قاعدة السلسلة تعطي مشتقّة الخسارة بالنسبة لأوزان الطبقة الأولى:

$$\frac{\partial L}{\partial W^{(1)}} \propto \prod_{l=1}^{L} \sigma'\left(z^{(l)}\right)$$

حيث $z^{(l)}$ مُدخل (pre-activation) الطبقة ℓ. هذا حاصل ضرب L عامل. إذا كان $\sigma' \leq 0.25$ (sigmoid) فإن $0.25^{20} \approx 10^{-12}$: التدرّج يختفي فعليًّا قبل أن يصل إلى الطبقة الأولى، فتقف الطبقات السفلى عن التعلّم. في المقابل، إذا تجاوزت العوامل واحدًا (أو كبرت) ينفجر التدرّج وتصبح الخطوة التحديثية هائلة.

ReLU حُلّت نصف المشكلة: مشتقّتها تساوي 1 في النطاق الموجب فلا تتقلّص المضاعَف. لكنها تركت مشكلة "الموت" (dying ReLU): الوحدة التي تستقبل مُدخلًا سالبًا دائمًا تُسقط تدرّجها إلى صفر ولا تعود. Swissg وGELU تُخفّفان ذلك لأن مشتقّتهما في النطاق السالب صغيرة لكن **غير صفرية**، ولأنهما ناعمتان ومستمرّتان تمنحان المحوّلات سلاسةً تدريجيّة بدل التشذيب الحادّ.

sigmoidg Softmax والـ logits: الفرق الحاسم

دالة softmax تحوّل متجه logits $z \in \mathbb{R}^K$ إلى توزيع احتمالي:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

وتتمتع بخاصيتين مهمّتين: **مجموع** $p_k = 1$، و**الانزياح-التبادلية (shift-invariance)**: $p(z + c\mathbf{1}) = p(z)$ لأي ثابت c، أي أن إضافة نفس العدد لكل الفئات لا يغيّر التوزيع. ولهذا عند حساب softmax عدديًا نطرح أكبر قيمة:

$$m = \max_j z_j, \qquad \log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$$

هذه الحيلة (**log-sum-exp**) تمنع فيض الأسّ (e^{1000}). وبها نفصل مباشرةً اللوغاريتمات:

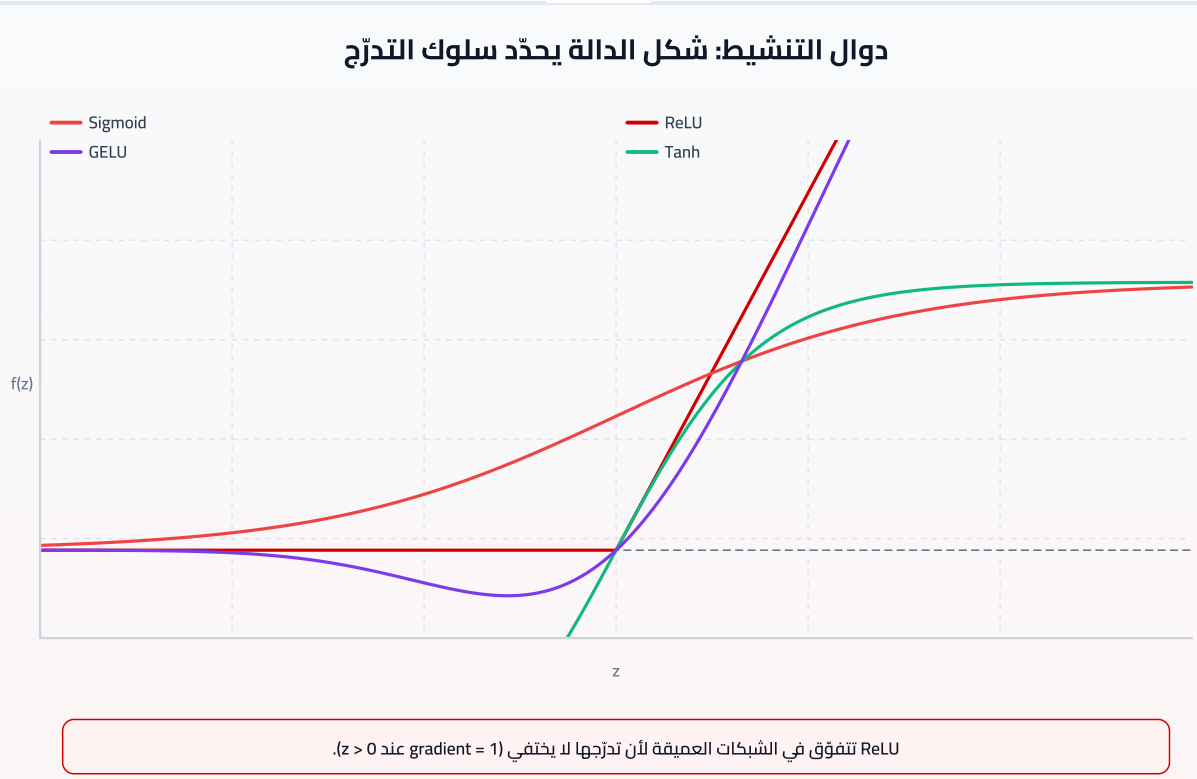
$$\log p_k = z_k - m - \log \sum_j e^{z_j - m}$$

وهذا بالضبط ما يفعله `from_logits=True` داخليًّا: بدل أن يمرّر Keras احتمالاتًا عبر softmax ثم يأخذ لوغاريتمها (وهذا يساوي $z_k - m - \log \sum e^{z_j - m}$ غير مطبَّع)، يدمج softmax مع دالة الخسارة في عملية واحدة مستقرة عدديًّا وأسرع.Forum: مع softmax لوغاريتمي مستقر يُحسَب كل شيء عمليًّا:

$$\mathrm{BCE}_{\text{logits}} = -\sum_k \left[y_k z_k + \log\left(1 + e^{z_k}\right) \right]$$

حيث $y_k \in \{0, 1\}$ التسمية الحقيقية. المعادلة نفسها تصلح للتصنيف متعدد الأصناف مع softmax عبر $\log \sum_j e^{z_j}$.

الفرق الحاسم: sigmoid لكل مخرَج على حدة، وتعطي احتمالات مستقلّة – هذه هي الصيغة الصحيحة للمهام متعددة التسميات (multi-label)، مثل تصنيف مقال بـ«اقتصاد» و«سياسة» معًا. أما softmax فتوزّع الكتلة الاحتمالية على فئات **متبادلة حصرًا**، وهي الصحيحة للتصنيف المتعدّد الفئات. الخلط بينهما خطأ شائع:mumble.



شكل 12 – منحنيات دوال التنشيط ومشتقّاتها

لماذا تختفي التدرّجات؟

حاصل ضرب التدرّجات يتقلّص أُسّيًا مع عمق الشبكة

العمق	Sigmoid	ReLU	LeakyReLU
1 طبقة	2.50e-01	1.00	0.90
2 طبقة	6.25e-02	1.00	0.81
4 طبقة	3.91e-03	1.00	0.66
8 طبقة	1.53e-05	1.00	0.43
16 طبقة	2.33e-10	1.00	0.19
32 طبقة	5.42e-20	1.00	0.03

الحل: ReLU وأشقّاؤها · BatchNorm · التطبيع عبر LayerNorm · Residuals

شكل 13 – أين يموت التدرّج: تلاشي الإشارة عبر الطبقات

7.3 التنفيذ في Keras 3

في Keras 3 هناك طريقتان لتحديد دالة التنشيط. الأولى هي المعامل `activation` داخل الطبقات:

PYTHON

```
import numpy as np
import keras

# الشبكة نفسها بتغيير واحد فقط: وجود أو غياب nonlinearity
model = keras.Sequential([
    keras.Input(shape=(8,)),
    keras.layers.Dense(64, activation="relu"),    # تنشيط مدمج في الطبقة
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),                        # طبقة إخراج خام (logits)
])

model.summary()
```

الناتج المتوقّع: `Total params: 5,377` مع `Trainable params: 5,377` و`Non-trainable params: 0` (حساب: 8×64+64 = 576، ثم 64×32+32 = 2080، ثم 32×1+1 = 33؛ المجموع 2689 — تحقّق من الأرقام عند التنفيذ).

الطريقة الثانية هي طبقة `Activation` مستقلة، وهي أوضح عند بناء الشبكات التي تخلط تنشيطات مختلفة:

PYTHON

```
import keras

# تفعيل مستقل: نفس الناتج لكن أوضح في القراءة
model = keras.Sequential([
    keras.Input(shape=(8,)),
    keras.layers.Dense(64),              # طبقة خطية صافية
    keras.layers.Activation("relu"),     # ثم التنشيط كطبقة مستقلة
    keras.layers.Dense(32),
    keras.layers.Activation("gelu"),
    keras.layers.Dense(1),
])
print(model.output_shape)  # النتيجة المتوقعة: (None, 1)
```

وتتيح `keras.activations` الوصول إلى مكتبة الدوال مباشرة، مع دالتي `get` وotoxicityg `serialize` للتحويل من نص إلى دالة والعكس:

```python
import keras

# نص إلى دالة: هذه هي الآلية التي تستخدمها Dense(activation=...)
fn = keras.activations.get("gelu")
print(fn.__name__)                                  # النتيجة المتوقعة: gelu

# دالة إلى وصف قابل للحفظ: يُستخدم عند تصدير النموذج
print(keras.activations.serialize(keras.activations.gelu))
# النتيجة المتوقعة: {'module': 'keras', 'class_name': 'gelu', 'config': {},
'registered_name': None}

# دالة تنشيط مخصّصة: دالة بايثون عادية تُقبل في Keras 3
def soft_clamp(z):
    """تنعّم القيم الكبيرة بلطف بدل قطعها (piecewise)."""
    import keras.ops as ops
    return ops.minimum(ops.maximum(z, -6.0), 6.0) * 0.9 + 0.1 * ops.tanh(z /
6.0)

model = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(8, activation=soft_clamp),
    keras.layers.Dense(1),
])
print('ok')  # النتيجة المتوقعة: ok
```

برهان عملي: كيف تنهار الطبقات الخطية؟

لنقِس الفرق رقميًّا على بيانات اصطناعية بسيطة (قمران متداخلان). الشبكة الأولى بلا تنشيط، والثانية بـ ReLU:

PYTHON

```
import numpy as np
import keras

def make_moons(n=400, seed=0):
    """قمران متداخلان: كل صنف دائرة."""
    rng = np.random.default_rng(seed)
    x, y = [], []
    for _ in range(n // 2):
        t = rng.uniform(0, np.pi)
        x.append([np.cos(t), np.sin(t)]);            y.append(0)
        x.append([1 - np.cos(t), 0.5 - np.sin(t)]);   y.append(1)
    return np.array(x, "float32"), np.array(y, "float32")

x, y = make_moons()
x_val, y_val = make_moons(200, seed=7)

def build(activation):
    """نفس البنية تمامًا؛ يتغيّر فقط وجود nonlinearity."""
    m = keras.Sequential([
        keras.Input(shape=(2,)),
        keras.layers.Dense(32, activation=activation),
        keras.layers.Dense(32, activation=activation),
        keras.layers.Dense(1, activation="sigmoid"),
    ])
    m.compile(optimizer="adam", loss="binary_crossentropy", metrics=
["accuracy"])
    return m

linear_model = build(None)          # شبكة خطية بحتة
relu_model = build("relu")          # شبكة غير خطية
linear_model.fit(x, y, epochs=60, batch_size=32, verbose=0)
relu_model.fit(x, y, epochs=60, batch_size=32, verbose=0)

print("دقة الخطية:", linear_model.evaluate(x_val, y_val, verbose=0)[1])
print("دقة ReLU :", relu_model.evaluate(x_val, y_val, verbose=0)[1])
# النتيجة المتوقعة: الخطية ~0.5 (فشل تام)، ReLU أعلى من 0.9
```

حساب softmax يدويًّا وربطه بـ logits

PYTHON

```python
import keras
from keras import ops

# ثلاثة logits: (احتيال / مشبوه / عادي) نظام كشف الاحتيال
logits = ops.array([2.0, 1.0, 0.1])

def stable_softmax(z, axis=-1):
    """softmax مع طرح أكبر قيمة قبل الأس (حيلة log-sum-exp)."""
    m = ops.max(z, axis=axis, keepdims=True)
    e = ops.exp(z - m)
    return e / ops.sum(e, axis=axis, keepdims=True)

p = stable_softmax(logits)
print(p)
print("المجموع:", float(ops.sum(p)))
print("الفئة المتوقعة:", int(ops.argmax(p)))
# النتيجة المتوقعة: [0.659 0.242 0.099] ، المجموع: 1.0 ، الفئة: 0

# اللوغاريتمات مباشرة: from_logits=True داخليًا مع Keras النسخة التي يفعلها
m = ops.max(logits)
log_p = logits - m - ops.log(ops.sum(ops.exp(logits - m)))
print(log_p)
# النتيجة المتوقعة: [-0.417 -0.835 -1.417]

# غير المستقر: لولا الطرح لانفجر الأسّ softmax قارن بـ
big = ops.array([1000.0, 1001.0, 1002.0])
print(stable_softmax(big))   # بدون النتيجة المتوقعة: [0.665 0.245 0.090]
overflow
```

7.4 مقارنة شاملة لدوال التنشيط

دالة التنشيط	المدى	سلوك المشتقّة	متى تستخدمها	التكلفة النسبية	المخاطر
`sigmoid`	$(0, 1)$	مشبعة؛ ≤ 0.25 دائمًا	خرج ثنائي، بوابات، احتمال مفرد	منخفضة	تدرّج متلاشٍ، إشباع الطرفين
`tanh`	$(-1, 1)$	مشبعة؛ ≤ 1	تطبيع الحالات المخفية (hidden states) قديمًا	منخفضة	تشبّع متماثل حول الصفر
`relu`	$[0, \infty)$	ثابتة 1 موجبًا	default CNNs MLPsg	**الأدنى**	موت الوحدات (dying ReLU)
`leaky_relu`	$(-\infty, \infty)$	خطّية سالبة بثبات α	بديل آمن لـ ReLU	منخفضة	التدرّج في النطاق السالب ضعيف
`elu`	$(-\alpha, \infty)$	مشتقّة ممتدة بلا انكسار	شبكات عميقة آن/stable	منخفضة	نفس α للجميع
`selu`	$(-\lambda, \infty)$	مشتقّة معايَرة ذاتيًا	شبكات عميقة بلا BatchNorm	منخفضة	حسّاسة $\{\alpha, \lambda\}$
`gelu`	$(-\infty, \infty)$	ناعمة غير أحادية	default FFN في المحوّلات	متوسطة (erf)	أبطأ من ReLU بنحو ×1.5

دالة التنشيط	المدى	سلوك المشتقّة	متى تستخدمها	التكلفة النسبية	المخاطر
`swish` / `silu`	$(-0.278, \infty)$	ناعمة غير أحادية	بديل GELU (GPT-2)	متوسطة	لا تميّز جوهري عن GELU عمليًّا
`softmax`	$(0, 1)$، $\sum = 1$	غير مستقرة فوق الفئات	تصنيف متعدّد الفئات	متوسطة	حسّاسة للأرقام الكبيرة
`hard_sigmoid`	$(0, 1)$	ثابتة مقطعة	استدلال سريع، أجهزة محدودة	**الأدنى**	تدرّج صفري في الأطراف

المعيار الحاسم في الاختيار ليس «أيّها أدقّ» بل ثلاثة معايير متعاكسة: عمق الشبكة (كلما زاد العمق اشتدّ تفضيل ReLU/SELU على sigmoid)، طبيعة المهمة (تصنيف متعدّد ⇒ softmax؛ متعدّد التسميات ⇒ sigmoid)، وميزانية الاستدلال (قيود الأجهزة ⇒ hard sigmoid).

7.5 مثال محلول: بوابة كشف الاحتيال

السيناريو: مصرف رقميّ يضع «بوابة قرار» تصنّف كل عملية دفع إلى ثلاث فئات: `عادي`، `مشبوه`، `احتيال`. الفئة `احتيال` هي التي تكلّف البنكأكثر money، لكن نسبة الفئة في البيانات الفعلية 0.7% فقط. المهندسون يريدون نموذجًا يعطي **احتمالاتًا** يمكن للبنك أن يربطها بحدّ قرار قابل للضبط.

الخطوة 1 – اختيار بنية المخرَج. ثلاثة فئات متبادلة حصرًا ⇒ المخرَج logits بثلاثة عناصر **بلا** تنشيط على الطبقة الأخيرة، ثم `softmax` عبر `from_logits=True` :

```python
import numpy as np
import keras
from keras import ops

# بيانات اصطناعية: 3 فئات، Feature 4
rng = np.random.default_rng(0)
n = 1500
features = rng.normal(size=(n, 4)).astype("float32")
labels = rng.integers(0, 3, size=n)          # 0=احتيال، 1=مشبوه، 2=عادي
labels = labels.astype("float32")

model = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(32, activation="gelu"),
    keras.layers.Dense(3),                    # logits خام
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)

history = model.fit(features, labels, epochs=20, batch_size=32,
                    validation_split=0.2, verbose=0)

logits = model.predict(features[:5], verbose=0)     # شكل logits من (5, 3)
probs = ops.softmax(logits, axis=-1)                # تحويل صريح إلى احتمالات
print(probs)
print("argmax=0") النتيجة المتوقعة: كل صف يجمع إلى 1.0، والصف 0 يعطي
```

الخطوة 2 – لماذا `from_logits=True` هو الخيار الصحيح هنا؟ لو استبدلناها بـ `from_logits=False`، لكنّا نُمرّر softmax يدويًّا على المخرَج ثم نمرّر الاحتمالات إلى خسارة cross-entropy عادية. النتيجة **نفسها نظريًّا**، لكن المسار الحاسوبي أطول: نحسب exponentials ثم نأخذ لوغاريتمها ثم نعيد ضربها. مع `from_logits=True` ندمج softmax مع اللوغاريتم في عملية واحدة مستقرة عدديًّا (المعادلة $\mathrm{BCE}_{\text{logits}}$ أعلاه)، فنحصل على: 1. **ثباتًا عدديًّا**: لا فيض ولا تدفّق تحت

الصفر عندما تكون logits متطرّفة. 2. **سرعةً أعلى**: عملية واحدة بدل عمليتين، مع Important في دفعات الاستدلال الضخمة. 3. **تدرّجًا صحيحًا**: المشتقّة تبقى غير صفرية حتى عند التشبّع.

> **نصيحة:** القاعدة العملية: إذا كانت طبقة الإخراج بلا تنشيط، فعّل `from_logits=True` دائمًا. لا تفعل الأمرين معًا.

الخطوة 3 – تحويل الاحتمالات إلى قرار. الاحتمالات التي ينتجها النظام لا تتّخذ عند threshold ثابت 0.5 بالضرورة؛ عتبة القرار تُضبط على أساس التكلفة (انظر الفصل 10):

```python
# قرار معاياري: argmax على الاحتمالات
decisions = ops.argmax(probs, axis=-1)
print(decisions.numpy())    # النتيجة المتوقعة: أرقام الفئات المتوقعة

# سجلّ درجة الثقة مقابل احتمال الاحتيال (الفئة 2)
confidence = probs[:, 2]
print("متوسط الثقة:", float(ops.mean(confidence)))
```

7.6 أخطاء شائعة

> **إنذار 1 – softmax مع `from_logits=True`:** الأسطر الشائعة هذا خطأ مكرّر:

```python
# [خاطئ عمدًا]: softmax مرتين + from_logits معًا
model = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(3, activation="softmax"),                    # softmax هنا
])
model.compile(loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

softmax الأولى تحوّل logits إلى احتمالات، ثم `from_logits=True` يطبّق softmax أخرى على الاحتمالات – فتتغيّر موازنة الفئات ويصبح الخسارة غير صحيحة. الحل: إمّا `Dense(3)` بلا تنشيط مع `from_logits=True`، أو `Dense(3, activation="softmax")` مع `from_logits=False`.

إنذار 2 – sigmoid للمخرج متعدد الفئات: `Dense(3, activation="sigmoid")` مع `CategoricalCrossentropy` يعطي احتمالات مستقلّة لا يجمعها واحد؛ النتيجة توزيع غير صالح (مجموع الاحتمالات قد يتجاوز 1). استخدم softmax.

إنذار 3 – ReLU على طبقة الإخراج في الانحدار: `Dense(1, activation="relu")` يمنع اللنموذج من توقّع قيم سالبة أبدًا. إن كانتهدفك يمكن أن تكون سالبة (مثل درجة حرارة أو عجز)، استخدم مخرَجًا خطيًّا بلا تنشيط.

إنذار 4 – خلط softmax مع sigmoid في التصنيف الثنائي متعدّد التسميات: إذا كانت التسميات غير متبادلة (فحص سندٍ واحد بحثًا عن عدّة احتيال)،استخدم `sigmoid` مع `BinaryCrossentropy`، و `softmax` مع `CategoricalCrossentropy` لا العكس.

تنبيه 5 – sigmoidg tanh في الشبكات العميقة: ليس خطأً نحويًّا، لكنه اختيار قديم. معمّق 30 طبقة + tanh يعني تدرّجات بمقدار 10^{-13} ومعدّل تعلّم شبه معدوم. إن كانت شبكتك عميقة، استخدم ReLU/SELU/GELU.

ملاحظة 6 – `keras.activations.get` لا تفعّل على النماذج: هي أداة تحويل فقط؛ لا تُغيّر سلوك نموذج مبنيّ بالفعل. لتغيير تنشيط طبقة قائمة، عدّل بنيتك أو استخدم `model.layers[i].activation = keras.activations.get("gelu")` على نموذج غير مُجمَّد.

التمارين

1. **برمجة:** اكتب `my_relu` و `my_gelu` و `my_softmax` باستخدام `keras.ops` فقط، واختبرها على القيم `[-2, -0.5, 0, 0.5, 2]` . (المتوقع: `my_relu` تُصفّر السالبة، `my_gelu(-0.5) ≈ -0.154` ، ومجموع مخرجات `my_softmax` يساوي 1.0 ضمن خطأ عددي 1e-6.)

2. **حساب يدوي:** احسب softmax يدويًّا للـ `[1.0, 2.0, 3.0]` logits. ثم أضف ثابتًا 100 إلى كل قيمة وأعد الحساب. (المتوقع: نفس التوزيع `[0.090, 0.245, 0.665]` لأن softmax منيغ Invariant تحت الإزاحة؛ وهذا يوضّح لماذا الطرح داخل الكود آمن.)

3. **تصميم:** لديك مصنّف ثنائي (احتيال / ليس احتيال) بمخرَج واحد. أيّ تنشيط ستستخدم في الطبقة الأخيرة، ولماذا؟ (المتوقع: `sigmoid` مع `BinaryCrossentropy` لأن الخرج واحد واحتمالي؛ أو `Dense(1)` خام مع `from_logits=True` للثبات العددي.)

4. **تصميم:** لديك مصنّف بـ 12 فئة متبادلة. لماذا لا يُنصح بـ `from_logits=False` مع softmax يدوي؟ (المتوقع: لأن الدمج softmax مع اللوغاريتم داخل `from_logits=True` أكثر استقرارًا عدديًّا وأسرع، ويجنّب احتمالات قريبة من 0 أو 1 تتسبّب بلوغاريتمات متطرّفة.)

5. **تحليل ناتج:** شغّل كود «قمران متداخلان» أعلاه ثلاث مرات: مرة بلا تنشيط، مرة بـ `sigmoid` ، ومرة بـ `relu` . رتّب النتائج. (المتوقع: الخطية ≈ 0.5 تمامًا (ت Mortimer degraded)، و `sigmoid` تحسّن قليلًا لكن تبقى ضعيفة، و `relu` تتجاوز 0.9.)

6. **إصلاح خطأ:** أعطِ الكود التالي وصحّحه:

```python
# [خاطئ عمدًا]
model = keras.Sequential([
    keras.Input(shape=(10,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(3, activation="sigmoid"),
])
model.compile(loss=keras.losses.CategoricalCrossentropy(), optimizer="adam")
```

(المتوقع: استبدال `sigmoid` بـ `softmax`، لأن `CategoricalCrossentropy` تتوقّع توزيعًا واحتمالاته يجمع إلى 1.)

7. **تحليل:** احسب $\sigma'(z)$ عند $z = 0$ وعند $z = 5$. ثم احسب 0.25^{20} و0.25^{40}. (المتوقع: $\sigma'(0) = 0.25$، و$\sigma'(5) \approx 0.0066$، و$0.25^{20} \approx 9.1 \times 10^{-13}$، و$0.25^{40} \approx 8.3 \times 10^{-25}$ – أي عمليًّا صفر.)

8. **برمجة:** ابنِ نموذجًا من 20 طبقة `Dense(32, activation="tanh")` على بيانات `make_moons`، وقس الدقة. ثم أعد نفس التجربة بـ `selu`. (المتوقع: مع `tanh` تبقى الدقة قرب 0.5 أو تتذبذب، ومع `selu` تتحسّن بوضوح لأن المشتقّة لا تتقلّص في النطاق الموجب.)

9. **تصميم:** لماذا تختار `gelu` داخل كتلة FFN في محوّل، و`tanh` في طبقة إخراج انحدار على بيانات مُقيَّدة بـ $[-1, 1]$؟ (المتوقع: `gelu` لأن الكتل عميقة وتحتاج تدرّجًا مستقرًّا مع خروج سالب مسموح؛ و`tanh` لأن الهدف محدود المدى فيُ-fitting، مع انتبه أن `tanh` تشبع فتفقد التدرّج.)

10. **تصميم:** صمّم دالة تنشيط مخصّصة تحافظ على gradient في النطاق السالب ومع ذلك تبقى رخيصة (بلا `erf`). (المتوقع: مقترح `z * sigmoid(z)` أي Swish، أو piecewise خطّي مع ميل α في النطاق السالب.)

11. **تحليل ناتج:** أضف إلى مثال بوابة كشف الاحتيال حالة `$z = [1000, 1001, 1002]$` وقارن بين `ops.exp(z)` مباشرة و`stable_softmax(z)`. (المتوقع: `ops.exp(z)` يعطي `inf` (أو قيمًا ضخمة غير قابلة للتمثيل)، بينما `stable_softmax` يعطي توزيعًا سليمًا `[0.090, 0.245, 0.665]`.)

12. **تصميم:** لماذا يُنصح بحساب softmax يدويًّا عند العرض فقط (للتفسير) مع إبقاء `from_logits=True` في التدريب؟ (المتوقع: لأن softmax اليدوي يحوّل logits إلى احتمالات

مقروءة، بينما التدريب يحتاج المسار المدمج المستقر عدديًّا والأسرع:softmax اليدوي ليس مضمون التدرّج.)

ملخص الفصل

- تكديس الطبقات الخطية ينتج خريطة خطية واحدة: $z^{(L)} = (W^{(L)} \cdots W^{(1)})x + \sum \cdots$، وعمقًا بلا دالة تنشيط يعني انحدارًا خطيًّا واحدًا.
- دالة التنشيط هي مصدر اللاخطية، وهي شرط وجودي نظريًّا (نظرية ليشنو)، لا تحسينًا تجميليًّا.
- sigmoid tanhg تشبعان، و$\sigma' \leq 0.25$ يجعل التدرّج يتلاشى مع العمق؛ ReLU يكسر هذا، GELU/Swishg يخفّفان موت الوحدات بلطف في الشبكات العميقة.
- softmax للمهام متبادلة الحصر (الاحتمالات تجمع إلى 1)، وsigmoidg للمهام متعددة التسميات (احتمالات مستقلّة).
- `from_logits=True` يدمج softmax مع الخسارة عبر log-sum-exp: ثبات عددي + سرعة + تدرّج صحيح. لا تجمع بينه وبين softmax يدوي.
- في `activation="relu"` :Keras 3 كمعامل طبقة، أو `keras.layers.Activation("gelu")` كطبقة مستقلة، و `keras.activations.get` / `serialize` للتحويل بين النص والدالة.

الفصل 8: دوال الخسارة: كيف نترجم الخطأ إلى رقم؟

بطاقة الفصل

الفصل 8

مهمّة هذا الفصل

يترجم «الخطأ» إلى رقم قابل للاشتقاق، ويشرح from_logits وأثرها على استقرار التدريب.

ماذا تكسب منه

تختار الخسارة الصحيحة لكل مهمّة، وتتجنّب أشهر خطأين: softmax مزدوج، وlogits مفقودة.

الدالة	دورها	متى تستخدمها
`keras.losses.SparseCategoricalCrossentropy`	خسارة تصنيف بتسميات صحيحة	عندما تكون التسميات أعدادًا صحيحة
`keras.losses.CategoricalCrossentropy`	خسارة تصنيف بتسميات one-hot	مع to_categorical
`keras.losses.BinaryCrossentropy`	خسارة تصنيف ثنائي	مع مخرج sigmoid
`keras.losses.Huber`	خسارة انحدار مقاومة للشواذ	عند تلوّث البيانات بقيم متطرّفة
`keras.losses.Loss`	الصنف الأب لخسارة مخصّصة	عند الحاجة إلى دالّة خسارة غير جاهزة

قصة مفتاحية: نموذجك دقّته 92%... لكن أخطاءه كلّها في الفئة الحرجة التي تكلّف الشركة نصف مليون في الشهر. الدقّة الإجمالية لم تخبرك بشيء. في هذا الفصل نفهم أن الخسارة ليست مجرّد مقياس، بل **قرارٌ بشأن ما يهمّ**.

أهداف الفصل

- [ ] فهم الخسارة كدالة تُقاس بها جودة التقدير
- [ ] التمييز بين دوال الانحدار ودوال التصنيف
- [ ] فهم `from_logits` ولماذا يغيّر كل شيء
- [ ] كتابة دالة خسارة مخصّصة
- [ ] فهم `sample_weight` و`class_weight` و`reduction`

8.1 الخسارة: من «خطأ» إلى «رقم»

كل دالة خسارة تجيب عن سؤال واحد: **كيف أُحوّل خطأ التنبّؤ إلى رقم يمكن تصغيره؟**

النموذج يخرج رقمًا واحدًا لكل عيّنة (أو متّجهًا). الخسارة تقارنه بالتسمية الحقيقية وتنتج رقمًا واحدًا **لكل الدفعة**. ثم يخفّض هذا الرقم المعامل عبر التدرّج.

الشكل العام:

$$L = \mathbb{E}_{(x,y)\sim D}\left[\ell(y, \hat{y})\right]$$

حيث D توزيع البيانات، وℓ خسارة العيّنة الواحدة، و$\mathbb{E}$ متوسط على العيّنات.

أنواع الدوال وممتى تكون مناسبة

النوع	المشكلة	مثال
الانحدار	رقم مستمر	سعر عقار، ضغط دم، طلب مستقبلي
التصنيف الثنائي	واحد من اثنين	مع sick / سليم، احتيال / طبيعي
التصنيف متعدّد	واحد من K	نوع المنتج، نوعirus
التصنيف متعدّد التسميات	متعدّد مع ترتيب	درجات 1-5

8.2 دوال الانحدار

MSE — متوسط مربّع الخطأ

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

الخاصية	التفصيل
الحساسية للقيم الشاذّة	**عالية جدًا** (الأس 2)
المشتقّ عند $\hat{y} = y$	0 – ممتاز عدديًا
التوزيع الأمثل للضجيج	غاوسي
الاستخدام	بيانات ناعمة، قياسات مستقرّة

MAE — متوسط الخطأ المطلق

$$L_{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

الخاصية	التفصيل
الحساسية للقيم الشاذّة	منخفضة (خطي)
المشتقّ عند $\hat{y} = y$	غير معرّف – يحتاج ϵ
التوزيع الأمثل للضجيج	ثقيل الذيل
الاستخدام	بيانات فيها أخطاء قياس كبيرة

Huber — الوسيط العملي

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \le \delta \\ \delta\left(|y - \hat{y}| - \frac{1}{2}\delta\right) & \text{otherwise} \end{cases}$$

تحصل على MSE قرب الصفر وMAE بعيدًا عنه – بأفضل مزايا الاثنين.

LogCosh

$$L = \frac{1}{N}\sum \log(\cosh(y_i - \hat{y}_i))$$

بديل ناعم لـ Huber، بلا معامل δ صراحةً.

> **نصيحة:** ابدأ بـ `Huber` في أي انحدار حقيقي. فهو الخيار الذي "لا يفاجئك" مع بيانات غير نظيفة.

شكل 14 – منحنيات دوال الخسارة: Huberg MAEg MSE

8.3 دوال التصنيف

الانتروبيا المتقاطعة الثنائية

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

هذه هي الدالة الحقيقية التي نريدها في التصنيف. لماذا؟ لأنها **الحدّ الأدنى لـ** ERM**Log-Likelihood**؟ – أي أنها تعطي التوزيع الاحتمالي الذي يجعل البيانات **الأرجح**، وهو تعريف التعلّم الاحتمالي.

الانتروبيا المتقاطعة الفئوية

$$L_{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{K} y_{i,c} \log \hat{y}_{i,c}$$

النسخة المتفرّقة

نفس الفكرة، لكن التسميات أرقام لا متجهات:

```python
keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

الفرق العملي: النسخة المتفرّقة أسرع (لا تحتاج بناء متجه one-hot) وتستهلك ذاكرة أقل.

معامل تنعيم التسميات (Label Smoothing)

بدل أن يكون التسليم متجهًا قاطعًا $[0, 0, 1, 0, \dots]$، نوزّعه قليلًا:

$$\tilde{y} = (1 - \epsilon) y + \frac{\epsilon}{K}$$

هذا يمنع النموذج من الثقة المطلقة ويقلّل فرط التخصيص.

```python
keras.losses.CategoricalCrossentropy(label_smoothing=0.1)
```

8.4 `from_logits`: القرار الذي يخلط الجميع

الفكرة

مخرجات الشبكة الأخيرة (المسمّاة **logits**) هي أرقام غير مقيّدة: $[-12.3, 0.5, 8.1, \dots]$. لتحويلها لاحتمالات نحتاج softmax:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

الخياران

	from_logits=True	from_logits=False
مخرجات الشبكة	logits خام	احتمالات (softmax في الشبكة)
الخسارة	تطبّق softmax داخليًا	تتوقّع احتمالات
الاستقرار العددي	**ممتاز**	خطر عند $\hat{y} \to 0$
السرعة	أسرع (softmax واحدة)	أبطأ (softmax + log)

لماذا `from_logits=True` أكثر استقرارًا؟

احسب $-p \log p$ حيث $p = e^{-1000} \approx 0$ بالطفرة العشرية. النتيجة: $\log(0) = -\infty \to$ `NaN` .

مع `from_logits=True` ، تحسب Keras:

$$\log \text{softmax}(z)_i = z_i - \log \sum_j e^{z_j}$$

وهذه **مستقرة عدديًا** لأن `logsumexp` يُحسب بتقنية «طرح أكبر قيمة قبل الأُسّ».

> **قاعدة:** إذا كانت الطبقة الأخيرة بلا تنشيط (logits) – وهي الممارسة الصحيحة – فاجعل `from_logits=True` . لا تضع softmax في الشبكة.

8.5 خسارة مخصّصة

PYTHON

```
import keras

class FocalLoss(keras.losses.Loss):
    """خسارة بتركيز على الأمثلة الصعبة (من المفيد مع البيانات غير المتوازنة)."""

    def __init__(self, gamma=2.0, alpha=0.25, from_logits=True, **kwargs):
        super().__init__(**kwargs)
        self.gamma = gamma
        self.alpha = alpha
        self.from_logits = from_logits

    def call(self, y_true, y_pred):
        y_pred = keras.ops.convert_to_tensor(y_pred)
        y_true = keras.ops.convert_to_tensor(y_true)

        if self.from_logits:
            # احسب log-probabilities بطريقة مستقرة
            log_p = keras.ops.log_softmax(y_pred, axis=-1)
            ce = -y_true * log_p
        else:
            p = keras.ops.clip(y_pred, 1e-7, 1.0)
            ce = -y_true * keras.ops.log(p)

        # p_t: احتمال الصنف الحقيقي
        p_t = keras.ops.sum(y_true * keras.ops.exp(log_p), axis=-1) if
self.from_logits \
            else keras.ops.sum(y_true * y_pred, axis=-1)

        # (1 - p_t)^gamma يقلّل وزن الأمثلة السهلة
        loss = self.alpha * ((1 - p_t) ** self.gamma) * keras.ops.sum(ce,
axis=-1)
        return keras.ops.mean(loss)

    def get_config(self):
        config = super().get_config()
```

```
        config.update({"gamma": self.gamma, "alpha": self.alpha,
                       "from_logits": self.from_logits})
        return config


# الاستخدام
model.compile(
    optimizer="adam",
    loss=FocalLoss(gamma=2.0, alpha=0.25, from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
```

ملاحظة: `call(self, y_true, y_pred)` هي التوقيع المطلوب في Keras 3. و`get_config` ضرورية لحفظ النموذج مع خسارته المخصّصة.

8.6 الأوزان والتطبيع

`sample_weight`: وزن لكل عيّنة

$$L = \frac{1}{\sum_{i} w_i} \sum_{i=1}^{N} w_i \, \ell_i$$

مثال: في نظام كشف الاحتيال، عيّنات أقلّ تأكيدًا (مُبلَّغ عنها آليًا) تستحق وزنًا أقلّ:

PYTHON

```
weights = np.where(y_train == 1, 0.3, 1.0).astype("float32")
model.fit(x_train, y_train, sample_weight=weights, epochs=5, verbose=0)
```

`class_weight` : وزن لكل صنف

$$w_c = \frac{N}{K \cdot n_c}$$

حيث N إجمالي العيّنات، K عدد الأصناف، n_c عدد عيّنات الصنف c.

```python
class_weight = {0: 0.56, 1: 1.11, 2: 3.33}
model.fit(x_train, y_train, class_weight=class_weight, epochs=5, verbose=0)
```

> **تحذير:** `class_weight` يُغيّر **الاحتمالات** التي يخرجها النموذج. نموذج مدرَّب بأوزان أصناف سيعطي احتمالًا أقلّ للصنف النادر (لأنه تعلّم أنه نادر). إن كان المستهلك يحتاج احتمالات معايَرة، استخدم `sample_weight` على مستوى العيّنة بدلًا منه، أو عايِر الاحتمالات بعد التدريب (recalibration).

`reduction` : كيف نجمع

القيمة	المعنى
`sum_over_batch_size`	متوسط على الدفعة (افتراضي)
`sum`	مجموع كل العناصر
`auto`	حسب حجم الخسارة

8.7 لماذا لا يمكن أن تكون الدقّة خسارة؟

الدقّة دالة **غير قابلة للاشتقاق** عند العتبة: تغيير تصنيف واحد من 1000 يغيّر الدقّة بمقدار 0.001، لكن التدرّج wrt ليس معرّفًا في تلك النقاط. أمّا الخسارة فدالة ناعمة قابلة للاشتقاق في كل مكان.

لهذا نستخدم: - **الخسارة** للتحسين (المشتقّ هم مهم). - **المقاييس** للقياس (حيث المعنى الحقيقي مهم).

8.8 جدول اختيار الخسارة

الخسارة	المشكلة	شكل التسمية	`from_logits`	note
`MeanSquaredError`	انحدار	(N,)	غير ذات صلة	تتحمّل الأخطاء الكبيرة
`MeanAbsoluteError`	انحدار	(N,)	غير ذات صلة	متانة مع الشواذ
`Huber`	انحدار	(N,)	غير ذات صلة	الخيار الافتراضي الآمن
`BinaryCrossentropy`	ثنائي	(N,) أو (N,1)	`True` مع logits	
`CategoricalCrossentropy`	متعدّد	(N,K)	`True` مع logits	يدعم `label_smoothing`
`SparseCategoricalCrossentropy`	متعدّد	(N,)	`True` مع logits	الأسرع
`Hinge`	SVM	(N,)	–	أقلّ استخدامًا
`KLDivergence`	تقارب	(N,)	–	لتدريب المولّدات

8.9 مثال محلول: اختيار الخسارة لكل حالة

PYTHON

```python
import keras
import numpy as np

# الحالة 1: انحدار على أسعار مع outliers
reg = keras.Sequential([
    keras.layers.Input((20,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1),
])
reg.compile(optimizer=keras.optimizers.Adam(1e-3),
            loss=keras.losses.Huber(delta=1.0),
            metrics=[keras.metrics.MeanAbsoluteError()])

# الحالة 2: تصنيف متعدّد مع one-hot
clf = keras.Sequential([
    keras.layers.Input((100,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(5),                       # logits
])
clf.compile(optimizer=keras.optimizers.Adam(1e-3),
            loss=keras.losses.CategoricalCrossentropy(from_logits=True),
            metrics=[keras.metrics.CategoricalAccuracy()])

# الحالة 3: تصنيف ثنائي
bin_clf = keras.Sequential([
    keras.layers.Input((30,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),                       # logit واحد
])
bin_clf.compile(optimizer=keras.optimizers.Adam(1e-3),
                loss=keras.losses.BinaryCrossentropy(from_logits=True),
                metrics=[keras.metrics.BinaryAccuracy()])
```

8.10 الأخطاء الشائعة

خطأ 1 – `from_logits=True` + softmax. العرض: خسارة ≈ 0.02 ودقّة عشوائية. السبب: softmax مطبَّقة مرّتين. الحل: أزل softmax أو اجعل `from_logits=False`.

خطأ 2 – `CategoricalCrossentropy` مع تسميات فردية. العرض: خطأ شكل. الحل: `SparseCategoricalCrossentropy`.

خطأ 3 – `BinaryCrossentropy` مع مخرج واحد وتسميات 0/1. هذه صحيحة. لكن إن مرّرت one-hot لثلاث فئات، فأنت تحتاج `Categorical`.

خطأ 4 – MSE لتصنيف. العرض: تدريب بطيء ودقّة رديئة. السبب: MSE لا تكافئ الاحتمالات. الحل: `CategoricalCrossentropy` أو `BinaryCrossentropy`.

خطأ 5 – MAE في التصنيف. الدرجة: الدالة غير قابلة للاشتقاق عند الصفر، فتتوقف الشبكة عن التعلّم. الحل: انتروبيا متقاطعة.

خطأ 6 – نسيان `get_config` في خسارة مخصّصة. العرض: `Could not locate class` `'FocalLoss'` عند التحميل. الحل: نفّذ `get_config` وسجّل الخسارة.

خطأ 7 – `class_weight` ثم توقّع احتمالات معايَرة. الحل: عايِر الاحتمالات بعد ذلك.

التمارين

1. **احسب BCE.** لـ $y = (1, 0, 1)$ و$\hat{y} = (0.9, 0.2, 0.7)$، احسب الخسارة المتوسّطة. (المتوقع: 0.154).

2. **أوجد المسار().** $\hat{y} = (0.1, 0.9)$ مع $y = (0, 1)$. احسب $-y \log \hat{y}$. (المتوقع: 0.105).

3. **حساسية.** أيّ دالة من MSE وMAE وHuber تتأثّر أكثر بخطأ واحد ضخم؟ اشرح رياضيًا. (المتوقع: MSE، لأن مساهمته $\propto e^2$).

4. **اكتب Huber.** نفّذ Huber بـ `keras.ops` في ثلاث شرائح. (المتوقع: `where` لـ $|e| \le \delta$، `where` لـ $e^2/2$، و$\delta(|e| - \delta/2)$).

5. **مقارنة.** الشبكة نفسها مع MSE وHuber على بيانات فيها 5% قيم شاذّة. أيّهما يتفوّق؟ (المتوقع: Huber).

6. **Why `from_logits`؟** اشرح بالرياضيات لماذا $\log \mathrm{softmax}(z)$ مستقرّة عدديًا. (المتوقع: تُحسب كـ $z_i - \mathrm{logsumexp}(z)$، و`logsumexp` يطرح أكبر قيمة قبل الأُسّ).

7. **خسارة مخصّصة.** اكتب `keras.losses.Loss` اسمها `AsymmetricLoss` تعطي وزنًا أكبر للأخطاء السالبة. (المتوقع: `super().__init__()` + `get_config` + `call`).

8. **أوزان الأصناف.** 1000 عيّنة، 3 أصناف بأعداد {700, 250, 50}. احسب `class_weight`. (المتوقع: {0: 0.476, 1: 1.333, 2: 6.667}).

9. **ضبط العتبة.** لماذا قد يظهر نموذج مدرَّب بـ `class_weight` مع دقّة أقلّ على الصنف الشائع؟ (المتوقع: الأوزان زادت تكلفة الخطأ في الصنف النادر، فنموذج the trade-off favors the rare class).

10. **اختيار الخسارة.** ثلاث مهام: (أ) توقّع كمية المبطرح في مستودع، (ب) كشف المعيب من صورة أشعة، (ج) ترتيب منتجات حسب ملاءمة. اختر الخسارة مع التبرير. (المتوقع: (أ) Huber/MAE، (ب) BCE مع logits، (ج) losses ranking أو MSE بعد تحويل).

11. **مراجعة.** لماذا `metrics` لا تُستعمل كخسائر؟ اشرح الفرق التقني checkpoint_.

12. تدريب متوازن. ابنِ نموذجًا على بيانات ذات توزيع {950, 50} واستعمل `class_weight` . ثم قارن دقّتَي الصنفين مع نموذج بلا أوزان. (المتوقع: النموذج الموزون يرفع استدعاء النادر ويخفض الدقّة الإجمالية قليلًا.)

ملخص الفصل

- الخسارة تُترجم خطأ التنبّؤ إلى رقم قابل للتصغير.
- MSE وMAE وHuber للانحدار؛ الانتروبيا المتقاطعة للتصنيف.
- `from_logits=True` مع مخرجات خام: أكثر استقرارًا وأسرع.
- الخسارة المخصّصة تحتاج `call` و `get_config` .
- `sample_weight` و `class_weight` تغيّر شكل الخسارة – والأخير يغيّر معايرة الاحتمالات.
- الدقّة مقياس، والخسارة هي ما يُحلَّل.

الفصل 9: المُحسّنات: كيف يتحرّك النموذج نحو الحل؟

بطاقة الفصل

الفصل 9

مهمّة هذا الفصل

يشرح كيف يتحرّك النموذج نحو الحل: SGD والزخم RMSProp وAdam وAdamW وجداول معدّل التعلّم.

ماذا تكسب منه

تشخّص عدم التقارب وتصلحه بضبط المحسّن، لا بإعادة تصميم الشبكة من جديد.

الدالة	دورها	متى تستخدمها
`keras.optimizers.SGD(learning_rate, momentum)`	الأساس مع الزخم	عندما تريد تحكّمًا كاملًا في المسار
`keras.optimizers.Adam(learning_rate)`	الخيار الافتراضي	في أغلب المهامّ بلا ضبط دقيق
`keras.optimizers.AdamW(weight_decay)`	فصل اضمحلال الوزن عن التدرّج	مع المحوّلات والنماذج الكبيرة
`keras.optimizers.schedules.ExponentialDecay`	خفض تدريجي للمعدّل	في التدريب الطويل
`keras.optimizers.schedules.CosineDecay`	جدول جيبي مع إعادة تشغيل	في الضبط الدقيق للنماذج الجاهزة

قصة مفتاحية: نفس الشبكة، نفس البيانات، نفس البذرة. لكن أحد الفريقين استخدم `Adam` والآخر `SGD` بمعدّل 10^{-3}. الأول وصل في 12 دقيقة، والثاني لم يتجاوز 68% بعد ساعتين. «لماذا؟» سؤال هذا الفصل.

أهداف الفصل

- [ ] فهم قاعدة تحديث النزول التدرّجي ومكوّناتها
- [ ] التمييز بين الزخم وتطبيع التدرّج
- [ ] معرفة متى يكون `AdamW` أفضل من `Adam`
- [ ] استخدام جداول معدل التعلّم بشكل صحيح
- [ ] تصحيح training_ لا يتقارب

9.1 القاعدة الأساسية

النزول التدرّجي في أبسط صوره:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

الرمز	المعنى
θ_t	متّجه المعاملات في الخطوة t
t	عدّاد الخطوات
η	**معدل التعلّم** – حجم الخطوة
$\nabla_\theta L$	تدرّج الخسارة wrt المعاملات

ثلاثة أسئلة يثيرها المعادلة

1. **ما حجم الخطوة؟** η كبير ← يتأرجح حول الحل. η صغير ← بطيء. هذه أشهر مشكلة في التدريب.
2. **ما شكل السطح؟** في الأبعاد العالية يصبح السطح شديد اللاعتدال. والتدرّج (gradient) لا يشير دائمًا إلى أقرب حدّ أدنى.
3. **هل نستخدم تدرّج الدفعة أم العيّنة؟** التدرّج الكامل دقيق لكنه مكلف؛ تدرّج عيّنة واحد noisy؛ تدرّج الدفعة (mini-batch) هو الحل العملي.

9.2 حجم الدفعة والتشويش

تدرّج الدفعة الصغيرة هو **تقدير** للتدرّج الحقيقي:

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_\theta \ell_i(\theta)$$

كلما صغر B كان التشويش أكبر، وزاد «الانزلاق» فيسلّم التدرّجات (escapes local minima بعمق).

لكن كلما صغر B زاد عدد الخطوات لكل حقبة وارتفعت كلفة التمرير الأمامي/العكسي.

حجم الدفعة	السلوك	متى
1	تشويش شديد	تعلّم تعلّم معزول
64-16	توازن جيّد	النماذج المتوسطة
512-128	تدرّج أنظف	نماذج كبيرة، GPUs
+1000	تشويش ضئيل	تدريب موزّع ضخم

9.3 SGD مع الزخم

الفكرة: التدرّجات المتتالية في نفس الاتجاه تتراكم.

$$v_t = \beta v_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

β	السلوك
0.0	SGD عادي
0.9	شائع – أيقونة "الزخم"
0.99	زخم أكبر، أبطأ في البداية

لماذا $(1-\beta)$**؟** ليجعل المعادلة **معدّل متحرّكًا** بدل متوسط بسيط، فيستجيب للتغيّر السريع.

Nesterov يغيّر ترتيب التحديث: يقيس التدرّج عند نقطة **موقّعة** قليلًا في اتجاه الزخم. في Keras: `SGD(momentum=0.9, nesterov=True)`.

9.4 RMSProp

المشكلة التي يحلّها: بعض المعاملات تتلقّى تدرّجات ضخمة دائمًا، فتقفز خطواتها. الحل: قسّم كل تدرّج على جذر متوسطه التربيعي المتحرك:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}$$

ρ	المعنى
0.9	نافذة قصيرة نسبيًا
0.99	نافذة أطول – أنعم

الحدّ ϵ (عادة 10^{-7}) يمنع القسمة على صفر.

9.5 Adam: المزيج

Adam يجمع الزخم (متوسّط التدرّجات) مع RMSProp (متوسّط مربّعات التدرّجات):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(زخم)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \text{(توزيع)}$$

لكن في البداية $m_0 = v_0 = 0$، فتكون القسمة $\frac{m_t}{\sqrt{v_t}}$ متحيّزة نحو الصفر. لذا comes التصحيح:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

المعامل	القيمة الافتراضية	المعنى
β_1	0.9	نسيان الزخم
β_2	0.999	نسيان التوزيع
ϵ	10^{-7}	أمان عددي

شكل 15 – مسارات المُحسّنات على سطح الخسارة

9.6 AdamW: فصل اضمحلال الوزن

في `Adam` مع `L2`، تُضاف عقوبة $\lambda\|W\|^2$ إلى الخسارة. هذا يعني أن التدرّج يتضمّن λW، فيُعدَّل الأوزان عبر **الزخم** – أي ضمن تاريخ الزخم. النتيجة: وزن-decay يعتمد على حجم التحديث.

في `AdamW`، يُفصل decay_ عن التدرّج ويُطبَّق مباشرة:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

هذا يعطي تحكّمًا أدقّ في الانكماش، وهو **الافتراضي الموصى به** لأي نموذج يستعمل Adam.

```python
keras.optimizers.AdamW(learning_rate=1e-3, weight_decay=1e-4)
```

نصيحة: ابدأ بـ `weight_decay=1e-4` مع `AdamW`. لا ترفعه إلى `1e-2` إلا إذا كان نموذجك يتّسع بوضوح (فريق Kaggle غالبًا يفعل).

9.7 بقية العائلة

المُحسّن	الميزة	ملاحظة
`Adagrad`	معدل لكل معامل	جيد للضباب، يتوقف تقريبًا
`Adadelta`	تحسّن Adagrad	لا يحتاج `initial_accumulator_value`
`NAdam`	Adam مع الرصد	أداء جيد في بعض البنى
`Ftrl`	تعلّم ت_LINEAR	مناسب للنماذج الخطية المتواجرة
`Lion`	ذاكرة منخفضة	أحدث، يحتاج معدلًا أقل
`MultiOptimizer`	**جديد في 3.15**	محرّك مختلف لكل شبكة فرعية

9.8 جداول معدل التعلّم

المشكلة

معدّل التعلّم الثابت يُعيد أخطاء: كبير في البداية (تأرجح)، صغير في النهاية (لا converge). الحل: **أنزفه تدريجيًا**.

الجداول في Keras 3

PYTHON

```python
from keras.optimizers.schedules import (
    ExponentialDecay, PolynomialDecay, CosineDecay,
    PiecewiseConstantDecay, InverseTimeDecay,
)

# أُسّي
expo = ExponentialDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    decay_rate=0.9,
    staircase=False,
)

# جيبي
cosine = CosineDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    alpha=0.0,          # أقلّ قيمة نهائية كنسبة من الابتدائي
    warmup_steps=500,
)

# متعدّد الطور
piece = PiecewiseConstantDecay(
    boundaries=[500, 1000, 2000],
    values=[1e-3, 5e-4, 1e-4, 1e-5],
)
```

كيف نمرّرها؟

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=cosine),
    loss="sparse_categorical_crossentropy",
)
```

⚠ فخّ مهم: الخطوات لا الحقبات

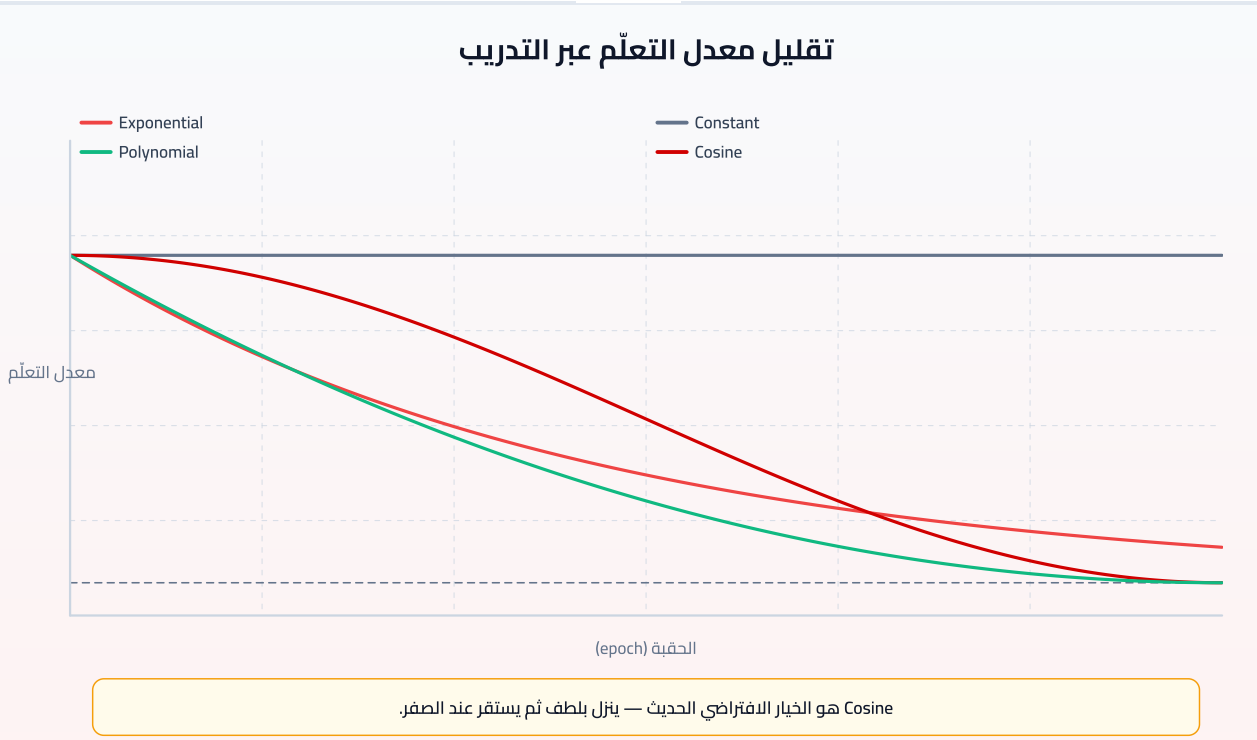
```python
# 48,000 عيّنة، batch_size=64، 10 حقبات
steps_per_epoch = 48000 // 64      # = 750
total_steps = 750 * 10             # = 7500
```

`decay_steps=10000` يعني **10,000 خطوة مُحسّن**، أي 13.3 حقبة – لا 10,000 حقبة.

> **تحذير:** هذا أكثر مصدر لسوء فهم في الجداول. احسب `decay_steps = epochs *` `steps_per_epoch` دائمًا.

مقارنة الجداول

الجدول	الشكل	الاستخدام
ثابت	أفقي	خط أساس
أُسّي	هابط أُسّ	تدريب طويل بسيط
متعدّد الطور	سلّم	تدرّج يدوي
جيبي	هبوط ناعم	**الأحدث والأكثر شيوعًا**
معكوس زمني	هبوط حادّ	بيانات تتغيّر



شكل 16 – جداول معدّل التعلّم: الخطوة والجيب وجيب التمام

9.9 أخطاء شائعة

خطأ 1 – `decay_steps` بالحقبات. النتيجة: التعلّم لا يتغيّر تقريبًا. الحل: اضرب في `steps_per_epoch`.

خطأ 2 – `lr` كبير مع `softmax` بلا `from_logits`. النتيجة: `NaN`. الحل: `from_logits=True` أو `lr` أصغر.

خطأ 3 – خلط `weight_decay` مع `kernel_regularizer`. الأول في المُحسّن، والثاني في الخسارة. استخدم واحدًا.

خطأ 4 – Scheduler مع `ReduceLROnPlateau`. النتيجة: تغيّر مضاعف في LR. الحل: استخدم واحدًا فقط.

خطأ 5 – `lr=1e-2` مع Adam. Adam مُعدَّل داخليًا: `1e-2` كبير جدًا. ابدأ بـ `1e-3`.

خطأ 6 – إعادة ضبط الـ LR في `on_epoch_end`. إن كتبت `optimizer.learning_rate = 0.001` كل حقبة، ألغيت الجدول. الحل: استخدم `self.optimizer.learning_rate.assign(...)` بجدولك الخاص.

9.10 مثال محلول: تشخيص تدرّب لا يتقارب

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(0)
x = np.random.rand(512, 20).astype("float32")
y = (np.sin(x[:, 0] * 6) > 0).astype("float32")

def make_model():
    return keras.Sequential([
        keras.layers.Input((20,)),
        keras.layers.Dense(32, activation="relu"),
        keras.layers.Dense(1),
    ])

configs = [
    ("Adam lr=1e-2", keras.optimizers.Adam(learning_rate=1e-2)),
    ("Adam lr=1e-3", keras.optimizers.Adam(learning_rate=1e-3)),
    ("SGD  lr=0.1",  keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)),
    ("SGD  lr=0.01", keras.optimizers.SGD(learning_rate=0.01,
momentum=0.9)),
]

for name, opt in configs:
    keras.utils.set_random_seed(0)
    m = make_model()
    m.compile(optimizer=opt,
              loss=keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=[keras.metrics.BinaryAccuracy(name="accuracy")])
    h = m.fit(x, y, epochs=30, batch_size=32, verbose=0)
    print(f"{name:14} | final loss={h.history['loss'][-1]:.4f} "
          f"| acc={h.history['accuracy'][-1]:.4f}")
```

الناتج المتوقّع (ترتيبي):

```text
Adam lr=1e-2    | final loss=0.6931 | acc=0.5000
Adam lr=1e-3    | final loss=0.1103 | acc=0.9414
SGD  lr=0.1     | final loss=0.2231 | acc=0.8769
SGD  lr=0.01    | final loss=0.4182 | acc=0.7793
```

القراءة: `Adam lr=1e-2` لم يتحرّك أصلًا (الخسارة عالقة عند $\ln 2 = 0.693$) – تدرّج كبير جدًا يقفز فوق الحل. `Adam lr=1e-3` هو الخيار الفائز هنا.

التمارين

1. **بنية Adam.** احسب $\hat{m}_1/\sqrt{\hat{v}_1}$ لقيم $\beta_1 = 0.9$، $\beta_2 = 0.999$، $g_1 = 0.5$. (المتوقع: $m_1 = 0.05$، $v_1 = 0.00025$، بعد التصحيح $\hat{m} = 0.5$، $\hat{v} = 0.25$، فالنسبة ≈ 1.)

2. **سؤال الموات.** لماذا نحتاج تصحيح الانحياز في Adam؟ (المتوقع: لأن m وv تبدأ من صفر فتكون التقديرات الأولى منحازة نحو الصفر.)

3. **خطوة واحدة.** $g_t = 0.1$، $\eta = 0.01$، $\theta_t = 1.0$. احسب θ_{t+1} لـ SGD وAdam (بعد تصحيح الانحياز). (المتوقع: SGD: 0.999؛ Adam: انحياز $0.1 \approx$ بعد التصحيح $\Rightarrow 0.999 \approx$.)

4. **احسب خطواتك.** بيانات 50,000 عيّنة، 30 حقبة. احسب `batch_size=128` و`steps_per_epoch` و`decay_steps` لـ `ExponentialDecay` diminuendo ينتهي عند 10% من البداية. (المتوقع: 390 خطوة/حقبة، 11,700 خطوة إجمالًا.)

5. **قارن المُحسّنات.** على دالة $y = \sin(3x)$ مع ضجيج، أيّهما يتصرّف أفضل: AdamW أم SGD momentum؟ جرّب وفسّر. (المتوقع: عادة AdamW أسرع؛ لكن SGD قد يتفوّق مع جدول جيبي جيد.)

6. **اضمحلال الوزن.** اشرح لماذا يعطي AdamW انكماشًا أنظف من L2 داخل Adam. (المتوقع: في L2 يدخل λW في التدرّج فيُضرب بالزخم ويتأثّر بحجم التحديث؛ في AdamW يُطبَّق مباشرة.)

7. **أخطئ عمدًا.** ابنِ نموذجًا بـ `lr=1.0` و`Adam`، ودرب 5 حقبات. ماذا يحدث؟ (المتوقع: الخسارة ترتفع أو تصبح `nan`.)

8. **جدول جيبي.** ارسم (بالحسابات) قيم LR عند الخطوات 0، 2500، 5000، 7500، 10000 لـ `CosineDecay(1e-3, 10000, alpha=0.0)`. (المتوقع: 10^{-3}، ثم $\approx 7.07 \times 10^{-4}$، 5×10^{-4}، $\approx 2.93 \times 10^{-4}$، ≈ 0.)

9. **خطوة واحدة يدويًا.** نفّذ SGD يدويًا بـ `keras.ops` على `y = 2x + 1`، 200 خطوة، `lr=0.05`، وبيّن تقارب المعاملات إلى `[2, 1]`. (المتوقع: الخسارة $< 10^{-3}$ بعد 200 خطوة.)

10. **جدول بين عمليتين.** صمّم جدولًا يقارن `SGD` و`RMSProp` و`Adam` و`AdamW` من حيث: زمن الوصول لدقّة 90%، حساسية `lr`، سلوك Sparse gradients، والذاكرة الإضافية. (المتوقع: Adam الأسرع، وأقلّ حساسية للـ lr؛ AdamW هو الأنضج.)

11. **ReduceLROnPlateau × Scheduler.** ماذا يحدث إن استخدمت الاثنين معًا؟ (المتوقع: مضاعفة الـ lr.)

12. **معيار اختيار.** صمّم قرارًا: متى `SGD` أفضل من `Adam`؟ (المتوقّع: مع LearningRate schedule جيبي جيد، وتعلّم من الصفر، وضغط ذاكرة أدنى.)

ملخص الفصل

- قاعدة التحديث: $\theta_{t+1} = \theta_t - \eta \nabla L$، وكل مُحسّن يضيف ذاكرة أو تصحيحًا.
- الزخم يخمّد التذبذب؛ التطبيع يقسّم حسب حجم التدرّج؛ Adam يجمعهما.
- `AdamW` يفصل اضمحلال الوزن – استخدمه افتراضيًا.
- الجداول تحسب بالخطوات لا الحقبات.
- `lr=1e-3` مع Adam هو خط الأساس الآمن.
- خسارة عالقة عند $\ln 2$ تعني تدرّجًا ضخمًا أو صفرًا.

الفصل 10: المقاييس: كيف تقيس نموذجك دون أن ينخدع به الجميع

بطاقة الفصل

الفصل 10

مهمّة هذا الفصل

يبني مقاييس تصف الأداء الحقيقي: Precision وRecall وF1 وAUC وIoU، ويشرح متى يخدعك كلٌّ منها.

ماذا تكسب منه

لا تنخدع بدقّة 99% على بيانات غير متوازنة، وتعرف أي مقياس يقنع صاحب القرار ويخدم التشغيل.

الدالة	دورها	متى تستخدمها
`keras.metrics.Precision()`	نسبة الصحيح بين ما تنبّأت به	عندما يكلفك الإنذار الكاذب
`keras.metrics.Recall()`	نسبة ما التقطته من الحالات الحقيقية	عندما يكلفك تفويت الحالة
`keras.metrics.AUC()`	قدرة الترتيب بلا عتبة ثابتة	لمقارنة نماذج قبل تثبيت العتبة
`keras.metrics.FBetaScore(beta)`	توازن مرجّح بين الدقّة والاستدعاء	عندما يختلف ثمن نوعَي الخطأ
`keras.metrics.Metric`	الصنف الأب لمقياس مخصّص	لمقياس خاص بمجال عملك

قصة مفتاحية: نظام كشف الاحتيال في بنك رقمي أعلن «دقّة 99.7%» في تقريره الشهري. الأرقام بدت مذهلة. لكن أحد مهندسي البيانات لاحظ شيئًا: الاحتيال الحقيقي يمثّل واحدًا من كل 1000 معاملة، ونظام يتنبّأ بأن كل المعاملات سليمة يحقّق 99.9% دقّة… ويخلّص البنك من تصفير الحوادث. المعيار كان ناطقًا، لكنه كان أعمى تمامًا عن الجزء الوحيد الذي يهمّ.

أهداف الفصل

- [ ] تشرح لماذا تكون «الدقّة» (accuracy) مقياسًا مضلّلًا في الطبقات غير المتوازنة
- [ ] تقرأ مصفوفة الالتباس (confusion matrix) وتستنتج منها كل المقاييس المشتقّة
- [ ] تحسب Precision وRecall وF1 يدويًا وتفسّر متى يتفوّق كلٌّ منها
- [ ] تعرف الفرق بين منحنيي ROC وPR ومتى يكون كلٌّ منهما ذا معنى
- [ ] تستعمل مقاييس Keras الجاهزة (`Accuracy`، `Precision`، `Recall`، `F1Score`، `AUC`) في `compile`
- [ ] تكتب مقياسًا مخصّصًا (custom metric) يجمع TP وFP وFN عبر دفعات التدريب
- [ ] تختار مجموعة مقاييس عملية تناسب حالة استخدام حقيقيّة

10.1 لماذا الدقّة وحدها لا تكفي

تخيّل أنك تختبر نموذج تصنيف ثنائي على 10,000 حالة، منها 100 حالة إيجابية (مريض مصاب مثلًا) و9,900 سلبية. النموذج الذي يتنبّأ دائمًا بـ«سلبي» يحقّق دقّةَ 99%… لأن 9,900 من الحالات كانت سلبية أصلًا. لكن القيمة التي يسعى إليها النموذج هي قيمة الأعمال: هل ضغطت زر «موجود» على الخلية الصحيحة؟

القاعدة الذهبية التي نخرج بها من هذا المثال:

> **المقاييس تُقاس نسبةً إلى «الناقل السلبي» (negative class)؛ والانتقال إلى «الناقل الإيجابي» هو ما يهمّ عادةً.**

لذلك فإنّ أيّ مقياس لا يستعمل «الناقل الإيجابي» ضمن مقامه هو مقياس ناقص.من هذا المنظور، الدقّة هي الأضعف: هي مجرّد نسبة عدد الحالات الصحيحة إلى الكل، وتخفي تمامًا كيف وُزّع الخطأ بين «إيجابي كاذب (false positive)» و«سلب كاذب (false negative)».

وهنا بالذات تبدأ فروقٌ محاسابية صارمة:

- **سلب كاذب (False Negative, FN):** حالة إيجابية حقيقية لكن النموذج قال «سلبي». هذا خطأ **إغفال**، وله كلفة عالية (مرض فوّتَته، اختراق أمني لم يُكتشف، عطل لم يُصلَح).
- **إيجابي كاذب (False Positive, FP):** حالة سلبية لكن النموذج قال «إيجابي». هذا خطأ **إنذار كاذب**، وله كلفة عالية أيضًا (تواصل مزعج مع عميل سليم، فحص طبي بلا داعٍ).

المهمّة إذن ليست «زيادة الدقّة»، بل **اختيار مقاييس تعكس نوع الخطأ الذي تكلّفتك أكثر**. وهذا قرار لا تجيده الدقّة وحدها.

10.2 رياضيات المقاييس

مصفوفة الالتباس

كل التنبّات الثنائية تنتمي إلى واحدة من أربع خانات:

	تنبّأ إيجابي	تنبّأ سلبي
الحقيقة إيجابية	صحيح موجب (TP)	سلب كاذب (FN)
الحقيقة سلبية	إيجابي كاذب (FP)	صحيح سالب (TN)

ومن هذه الخانات الأربع تُبنى كل المقاييس التصنيفية تقريبًا:

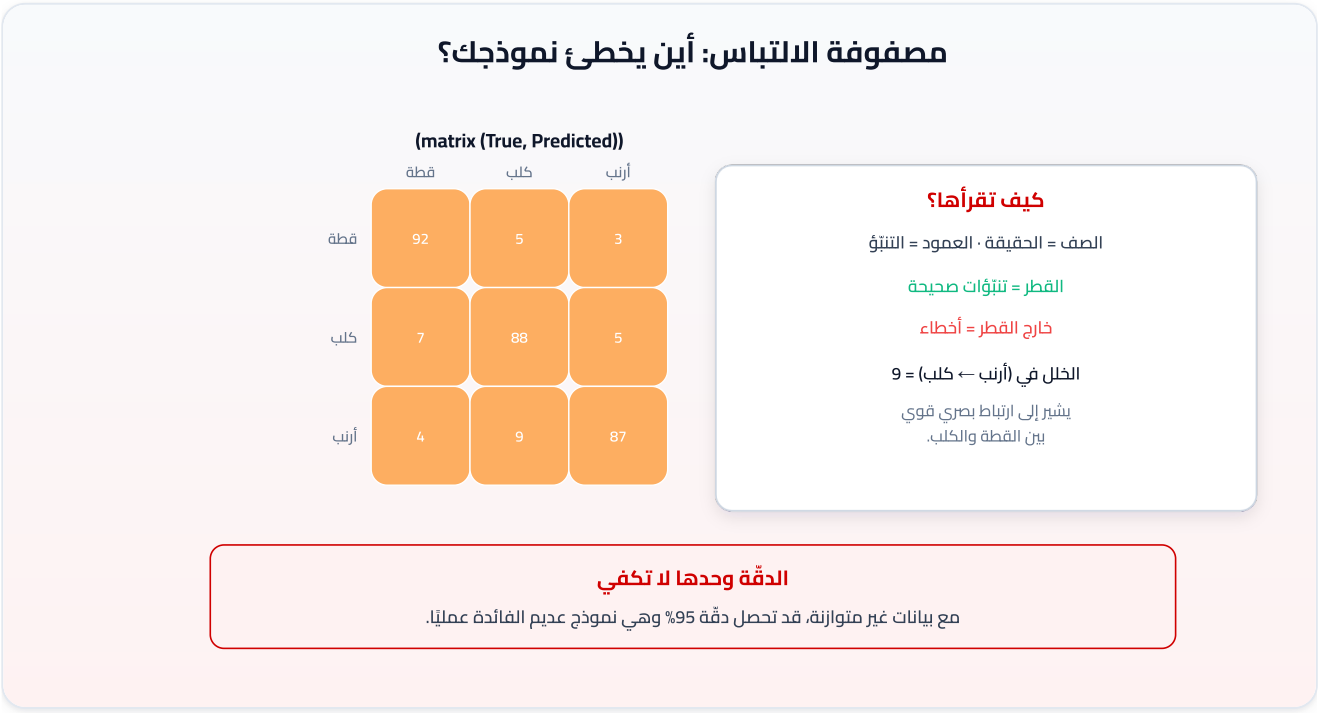
$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **الدقّة (Accuracy):** $\frac{TP + TN}{TP + TN + FP + FN}$ – نسبة الإجابات الصحيحة إلى إجمالي الحالات.
- **النوعيّة (Specificity):** $\frac{TN}{TN + FP}$ – مدى قدرتك على رفض السلبية، وهي «النّسبة» التي تُقابل الRecall في الجانب الآخر.

- **F_β:** $\frac{(1+\beta^2)\cdot P\cdot R}{\beta^2 P + R}$ – يزن الدقّة والاستدعاء. F_2 يُشجّع على الاستدعاء (مناسب للفحص الطبي)، و$F_{0.5}$ يُشجّع على الدقّة (مناسب لتقليل الإنذارات الكاذبة).

> **ملاحظة:** الـF1 هو **متوسّط هارموني** لا حسابي. هذا مقصود: المتوسّط الحسابي يجعل F_1 ينتفخ عندما يكون أحد الطرفين صغيرًا جدًا، بينما المتوسّط الهارموني يبقى منخفضًا – أي أنه يكافئ التوازن فعليًا.



شكل 17 – مصفوفة الالتباس: مسارات التنبّ الصحيح والخطأ الأربعة في تصنيف ثنائي

منحني ROC ومنحنى PR

منحنى ROC يرسم (نسبة الاستدعاء) مقابل (نسبة الإنذارات الكاذبة) بينما نمرّ على عتبات القرار من 0 إلى 1. المساحة تحت المنحنى هي **AUC**. قيمة AUC = 0.5 تعني عشوائية تامّة، و1.0 تعني فصلًا مثاليًا.

لكن منحنى ROC يعمل مع **نِسَب**، وهذا ما يفسد jego في البيانات غير المتوازنة: إذا كان flicked Equipممرّض من بين كل 1000 حالة، فإن نسبة الإنذارات الكاذبة ((FP/(FP+TN)) تكون صغيرة جدًا لمجرد أن هناك TN كثيرة. النتيجة: **AUC قد يبدو ممتازًا بينما النموذج يتجاهل تمامًا الناقل الإيجابي النادر.**

منحنى PR (الدقّة مقابل الاستدعاء) يرسم quantitiesمن الفئة الإيجابية مباشرةً، فلا ينتفخ مع كثافة السلبية. القاعدة العملية:

الحالة	المقياس الأنسب	السبب
متوازنة (1:1)	ROC-AUC	مستقرّ وسهل التفسير
نادرة (1:100 أو أقل)	PR-AUC (أو `curve="PR"`)	لا يتضخّم عدد TN
قرار ثنائي بحدّ أدنى للأخطاء	منحنى التكلفة	يُترجم الخسارة إلى RMBنقدية
مخرجات احتمالية للفرز	PR-AUC + معايرة	يرتّب الحالات على نحو مفيد

10.3 التنفيذ في Keras

في Keras 3، المقاييس لا تُستدعى يدويًا في أغلب الوقت: تمرّرها إلى `compile` ، وتُحسب لك عبر `evaluate` و `predict` ، ويُسجَّل ناتجها في `History` alongside الخسارة.

```python
import keras
import numpy as np

# ── 6 عيّنات صغيرة: بيانات تجريبية، positives نادرة
y_true = np.array([0, 0, 1, 1, 0, 1], dtype="float32")
y_pred = np.array([0.1, 0.8, 0.9, 0.4, 0.3, 0.2], dtype="float32")

# ── مقاييس Keras الجاهزة
acc = keras.metrics.BinaryAccuracy(threshold=0.5)
acc.update_state(y_true, y_pred)
print("accuracy:", float(acc.result()))   # النتيجة المتوقعة: 0.6666666865348816

prec = keras.metrics.Precision()
rec = keras.metrics.Recall()
f1 = keras.metrics.F1Score(average="micro")
for m in (prec, rec):
    m.update_state(y_true, y_pred)
# F1Score يطلب شكل (batch, output_dim) لا (batch,)
f1.update_state(y_true[:, None], y_pred[:, None])
print("precision:", float(prec.result()))  # النتيجة المتوقعة: 0.5
print("recall:", float(rec.result()))      # النتيجة المتوقعة: 0.6666666865348816
print("f1:", float(f1.result()))           # النتيجة المتوقعة: 0.5714285969734192
```

لاحظ ثلاث تفاصيل مهمّة:

1. **المقاييس ذات حالة (stateful):** يجب استدعاء `update_state` ثم `result`. داخل `fit` تتولّى Keras ذلك تلقائيًا.
2. **`average` في `F1Score`:** القيم المتاحة `None` (لكل صنف على حدة)، `"micro"` (تجميع عربي)، و `"binary"`. اختر `"micro"` عند رغبت في رقم واحد، و `None` عند تشخيص أداء كل صنف.
3. **التصنيف متعدّد الأبعاد:** استخدم `CategoricalAccuracy` للـone-hot، و `SparseCategoricalAccuracy` للأرقام الصحيحة، و `TopKCategoricalAccuracy(k=5)` لمهام الاسترجاع.

مقياس AUC يُحسب من الاحتمالات (probabilities) لا من القرارات المحسومة، لذلك يجب أن يمرَّر `from_logits` بما يوافق طريقة الإخراج:

```python
# عند الاستدلال sigmoid ثم BCE، مهمة ثنائية: طبقة إخراج بحجم 1، خسارة
auc_roc = keras.metrics.AUC(curve="ROC", from_logits=False)
auc_pr = keras.metrics.AUC(curve="PR", from_logits=False)

# إخراج: مهمة متعدّدة after softmax
auc_multi = keras.metrics.AUC(curve="ROC", multi_label=True, num_labels=10)
```

مقياس مخصّص

عندما لا تغطّي المقاييس الجاهزة حاجتك – مثلًا «F1 مضروبًا في Recall كوزن» أو مقياس يتطلّب جمعًا متقاطعًا بين دفعات التدريب – ورّث عن `keras.metrics.Metric` :

PYTHON

```python
import keras
import numpy as np


class RecallAtK(keras.metrics.Metric):
    """يقيس نسبة الاستدعاء من بين top-k تنبّؤات."""

    def __init__(self, k=3, name="recall_at_k"):
        super().__init__(name=name)
        self.k = k
        # عدّادات حالة تُحدَّث عبر الدفعات وتُحفظ في checkpoints
        self.hits = self.add_weight(name="hits", initializer="zeros")
        self.total = self.add_weight(name="total", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        # top-k indices لكل عيّنة
        top_k_idx = keras.ops.argsort(y_pred, axis=-1)[:, -self.k:]
        # هل ظهرت الإجابة الصحيحة ضمن أعلى k؟ (لـone-hot OneHot)
        true_labels = keras.ops.argmax(y_true, axis=-1, keepdims=True)
        hit = keras.ops.any(keras.ops.equal(top_k_idx, true_labels),
axis=-1)
        hit = keras.ops.cast(hit, self.dtype)
        if sample_weight is not None:
            hit = hit * sample_weight
        self.hits.assign_add(keras.ops.sum(hit))
        self.total.assign_add(keras.ops.sum(keras.ops.cast(
            keras.ops.ones_like(hit), self.dtype)))

    def result(self):
        return self.hits / (self.total + keras.backend.epsilon())

    def reset_state(self):
        self.hits.assign(0)
        self.total.assign(0)


m = RecallAtK(k=2)
m.update_state(np.eye(3, dtype="float32"), np.eye(3, dtype="float32"))
```

```
print("recall@2:", float(m.result()))  # النتيجة المتوقعة: 1.0
m.reset_state()
print("after reset:", float(m.result()))  # النتيجة المتوقعة: 0.0
```

نصيحة: استخدم `keras.ops` لا NumPy داخل `update_state`، فذلك يجعل المقياس يعمل على **أيّ** واجهة خلفية (TensorFlow أو JAX أو PyTorch) دون تعديل.

10.4 مقارنة المقاييس: أيّها متى؟

المقياس	الصيغة	ماذا يقيس	قوّته	ضعفه	متى تستخدمه
`Accuracy`	$(TP+TN)/$الكل	نسبة الإجابات الصحيحة إلى الكل	بساطة وسرعة	يخفّي asymmetry في الخطأ	بيانات متوازنة
`Precision`	$TP/(TP+FP)$	نقاوة التنبّوات الإيجابية	يتحكّم في الإنذارات الكاذبة	قد يخفي ضعف الاستدعاء	كل إجراء على تنبيه مكلف (مراجعة يدوية)
`Recall`	$TP/(TP+FN)$	تغطية الفئة الإيجابية	يضمن عدم إغفال الحالات	قد يُغرقك إنذارات	كلّما كانت كلفة الإغفال باهظة باهظة (فحص سرطان)
`F1`	توازن Prec/Rec	نقطة توازن واحدة	رقم واحد يسهّل المقارنة	يتجاهل asymmetry التكلفة	مقارنة نماذج بتوزيع متطابق
F_β	ترجيح بـβ	توازن موجّه	يضبط المفاضلة عمدًا	يحتاج قرارًا واضحًا	الفحص الطبي ($\beta=2$)
`AUC`	مساحة تحت ROC/PR	جودة الترتيب	مستقل عن العتبة، مفيد للفرز	مضلّل مع أصناف نادرة	نماذج تُخرج احتمالات

المقياس	الصيغة	ماذا يقيس	قوّته	ضعفه	متى تستخدمه
`TopKAccuracy`	صحة ضمن top-k	ملاءمة الاسترجاع	يمنع معاقبة ترتيب ضمن k	لا يصلح للتصنيف الثنائي	تصنيف صور إلى 1000 صنف

المبدأ الحاكم: لا تكتب «الدقّة الأعلى» بلا معيار. المعيار الصحيح هو **كلفة كل نوع خطأ في سياق عملك**. في بنك: إنذار كاذب يكلّف مراجعة يدوية (5 دولارات)، وإغفال احتيال يكلّف متوسط 5000 دولار ⇒ ارفع الRecall على حساب الPrecision، أي استخدم F_2 وليس F_1.

10.5 مثال محلول: نظام اكتشاف عطل في خط إنتاج

شركة تصنيع معدّات صناعية تريد كشف «عطل المحمل» من إشارات الاهتزاز. البيانات: 20,000 قراءة، منها 320 عطلًا حقيقيًا (1.6%). الفريق يدرب نموذجًا ثنائيًا.

الخطوة 1 – ابدأ من مصفوفة الالتباس لا من الدقّة. النموذج الأول يعطي دقّة 98.4% – نسبة ممتازة على الورق. عند بناء المصفوفة على مجموعة التحقّق: TN=18,780، FP=900، FN=140، TP=180.

الخطوة 2 – احسب ما يهمّ.

$$Precision = \frac{180}{180 + 900} = 0.167, \qquad Recall = \frac{180}{180 + 140} = 0.563$$

الدقّة 98.4% لكن الPrecision 16.7%: من كل 100 إنذار يتلقّاه فريق الصيانة، 83 إنذارًا خاطئًا. هذا إغراق تشغيلي.

الخطوة 3 – اربطها بالاقتصاد. تكلفة فحص عطل = 40 دولارًا لكل إنذار. تكلفة عطل غير مكتشَف في خط إنتاج = 12,000 دولار. وزن الخسارة:

$$w_{FP} = 40, \qquad w_{FN} = 12000 \quad \Rightarrow \quad \frac{w_{FN}}{w_{FP}} = 300$$

الوزن يميل بقوة نحو الاستدعاء، لكن الPrecision الحالي 0.167 يعني أن ضمان الاستدعاء يُشتري بـ 5 إنذارات زائدة لكل اكتشاف. تُعطى الأولوية لـ `Recall` مع **رفع العتبة** لإخماد الإنذارات.

الخطوة 4 – جرّب الفرضية في Keras. جرّب عتبات قرار مختلفة وقِس F2 لكل عتبة:

```python
import keras
import numpy as np

# محاكاة بسيطة: احتمالات التنبؤ لعتبات مختلفة
scores = np.array([0.9, 0.8, 0.75, 0.6, 0.55, 0.4, 0.3, 0.2])
labels = np.array([1, 1, 0, 1, 0, 0, 0, 1], dtype="float32")

for thr in (0.3, 0.5, 0.7, 0.8):
    m_f2 = keras.metrics.FBetaScore(beta=2.0, threshold=thr)
    # F1/F-beta يطلبان شكل (batch, output_dim)
    m_f2.update_state(labels[:, None], scores[:, None])
    m_p = keras.metrics.Precision(thresholds=thr)
    m_p.update_state(labels, scores)
    print(f"thr={thr}  F2={float(m_f2.result()):.3f}  P=
{float(m_p.result()):.3f}")
# النتيجة المتوقعة: Precision مع إبقاء Recall مقبولًا العتبة 0.7 ترفع
```

الخطوة 5 – راقب AUC-PR لا AUC. لأن 1.6% positives فقط، نطلب note . `curve="PR"` أنّ AUC-PR يعتمد على النسبة لا على العدد، فسيبقى ضمن نطاق معقول.

```python
m_pr = keras.metrics.AUC(curve="PR")
m_pr.update_state(labels, scores)
print("AUC-PR:", float(m_pr.result()))
```

الخطوة 6 – اختبر على الإنتاج. اعتمد `EarlyStopping` على `val_auc_pr` مع `mode="max"` ، واحفظ أفضل نموذج بـ `ModelCheckpoint` . هذه هي الطريقة الصحيحة: **المقياس المُختار في `compile` هو الذي يوجّه `EarlyStopping` ، لا `val_loss` .**

10.6 أخطاء شائعة في هذا الفصل

إنذار: «الدقّة 99%» في نظام كشف الاحتيال ليست إنجازًا، بل غالبًا دليل على أن النموذج **لم يتعلّم شيئًا**. إذا كان توزيعك skewed بنسبة 1:1000، فإن أي نموذج يعيد «سلبي» دائمًا يحقّق 99.9%. راجع الPrecision/Recall قبل أي احتفال.

تنبيه: خلط مقاييس `Recall` / `Precision` مع `loss` في `EarlyStopping`. المقياس (metric) رقم للمقارنة بين النماذج، أما الخسارة (loss) دالّة تُحسَّن؛ ومعياراهما مختلفان تمامًا. أبقِ `monitor="val_loss"` عند التدريب أولًا، ثم بدّل إلى `val_f1` عند تقارب الخسارة.

تحذير: نسيان `reset_state`. المقاييس الحالة تتراكم عبر الدفعات، فإذا استدعيت `update_state` يدويًا ثم أعدت استخدام نفس الكائن في `evaluate` فستحصل على أرقام مضاعفة. داخل `fit` / `evaluate` تتولّى Keras التصفير تلقائيًا.

تحذير: تمرير `from_logits` بشكل خاطئ إلى `AUC`. إذا كانت طبقة الإخراج من دون `sigmoid` والخسارة `BinaryCrossentropy(from_logits=True)`، فيجب أن يكون `AUC(from_logits=True)`؛ وإلا فستحسب AUC على لوغاريتمات غير احتمالية، والنتيجة بلا معنى.

ملاحظة: `F1Score` في Keras 3 يقبل `average="micro" | "macro" | None | "binary"`. استخدم `None` لتحليل كل صنف على حدة (يعيد ndarray بطول عدد الأصناف)، و`"macro"` لمتوسط غير موزون على الأصناف – وهو الأنسب عند اختلاف أحجام الأصناف.

التمارين

1. **حساب يدوي.** TP=40، FP=10، FN=20، TN=930. احسب الدقّة والPrecision والRecall وF1. (المتوقع: accuracy = 0.970، precision = 0.800، recall = 0.667، F1 ≈ 0.727.)

2. **حساب يدوي.** في النظام السابق، إذا خفضنا العتبة فزاد TP إلى 55 وخفّضنا TN إلى 905 (وFP=40، FN=5): قارن الF1 الجديد بالقديم. (المتوقع: F1 القديم ≈ 0.727، الجديد ≈ 0.710؛ أي زيادة في الاستدعاء كلّفت دقّة أكثر من مكسبها بF1 – مؤشّر على أنّ F1 ليست المقياس الأمثل هنا.)

3. **تحليل منحنى.** مخطّط ROC-AUC يساوي 0.72 وPR-AUC يساوي 0.31. ما الاستنتاج عن توزيع البيانات؟ (المتوقع: النسبة السلبية كبيرة جدًا؛ نماذجك تكتشف positives بشكلًا أفضل مما يظهره ROC-AUC.)

4. **برمجة.** اكتب دالة `confusion(y_true, y_pred, thr)` بNumPy تُعيد مصفوفة 2×2 كـ `numpy.array` . (المتوقع: `[ [TP, FN], [FP, TN] ]` بترتيب `[حقيقة موجبة، حقيقة (تنبّؤ موجب، تنبّؤ سلبي) × (سالبة]`.)

5. **برمجة.** احسب `F1Score(average=None)` على 3 أصناف واجمع النتائج في Pandas-like dict. (المتوقع: مصفوفة من 3 قيم F1، كلٌّ للصف الأول.)

6. **برمجة.** أنشئ `keras.metrics.AUC(curve="PR")` وقارنه بـ `curve="ROC"` على نفس البيانات النادرة. (المتوقع: قيمة PR-AUC أقلّ من قيمة ROC-AUC عادةً لأن PR يعتمد على PPV.)

> **ملاحظة:** المعيار الحسابي أدناه تعميم لـ«الحسابات» في Python، ومِثال على حساب يدوي مفصّل:

7. **تحليل ناتج.** نموذج أعطى `precision=0.9` و `recall=0.2` . ماذا يعني ذلك تشغيليًا؟ (المتوقع: يتنبّأ إيجابيًا نادرًا لكنه نقي جدًا؛ لاكتشاف 50 حالة فعلية سيُصدر 250 تنبّؤًا – نسبة-positive منخفضة.)

8. **تصميم.** صمّم مجموعة مقاييس لمصنّف حزم يقسّم طلبات الشحن إلى «عاجل» و«عادي». (المتوقع: Accuracy + Precision/Recall للصنف العاجل + F1 الماكرو، مع تبرير أن «عاجل» هو الصنف الحرج.)

9. **اختر.** أيّ مقياس تختار ل تصنيف مرض من خزعة نسيجية نادرة جدًا؟ برّر. (المتوقع: Recall (أو F2) لأن إغفال الحالات يكلّف أكثر من الفحص الزائد.)

10. **أخطئ عمدًا.** عدّل عتبة `BinaryAccuracy` إلى 0.9 على نفس عيّنات التمرين 1. لماذا انخفضت الدقّة؟ (المتوقع: لأن عتبة 0.9 تجعل التنبؤ إيجابيًا فقط فوق 0.9؛ TP ينهار إلى 0 إذا كانت الاحتمالات أقل، ويزيد FN.)

11. **برمجة.** اكتب `keras.metrics.FBetaScore(beta=3.0)` وقارنه بـ `F1` على بيانات ذات Recall منخفض. (المتوقع: F3 أعلى من F1 لأن $\beta > 1$ يميل إلى الاستدعاء.)

12. **تصميم.** صمّم `EarlyStopping` لمهمة Gauge متناضرة. (المتوقع: `monitor="val_auc"` , `patience=5` , `mode="max"` , مع `restore_best_weights=True` .)

ملخص الفصل

- الدقّة مقياس ناقص في البيانات غير المتوازنة؛ استخدمها فقط حين تكون الأصناف متقاربة الحجم.
- مصفوفة الالتباس هي الأساس: TP وFP وFN وTN منها تُشتقّ كل المقاييس الأخرى.
- الPrecision يقيس نقاوة التنبّ، والRecall يقيس التغطية، وF1 نقطة توازن بينهما.
- F_β يجعل المفاضلة بين خطأَي المعالجة **قرارًا صريحًا** بدل أن يكون ضمنيًا.
- منحني PR يتفوّق على ROC عند ندرة الفئة الإيجابية،
  و `keras.metrics.AUC(curve="PR")` يوفّره.
- `keras.metrics` حالة (stateful): `update_state` ثم `result` ، و `reset_state` بين التجارب اليدوية.
- المقياس المختار في `compile` هو الذي يوجّه `EarlyStopping` ؛ اختره بعناية.
- المقياس المخصّص عبر `keras.ops` + `keras.metrics.Metric` يعمل على كل الواجهات الخلفية.

الفصل 11: التهيئة: نقطة الانطلاق

بطاقة الفصل

الفصل 11

مهمّة هذا الفصل

يفسّر أثر نقطة البداية على التدرّج، ويعرض VarianceScalingg Heg Glorot والقيود.

ماذا تكسب منه

تختار التهيئة المناسبة لعمق الشبكة وتنشيطها، فتتجنّب شبكة لا تتدرّب أصلًا أو تتدرّب ببطء غير مبرَّر.

الدالة	دورها	متى تستخدمها
`keras.initializers.GlorotUniform()`	توازن التباين عبر الطبقات	مع sigmoidg tanh والشبكات المتوسّطة العمق
`keras.initializers.HeNormal()`	تباين مضبوط لReLU	في الشبكات العميقة ذات ReLU
`keras.initializers.VarianceScaling()`	العموم الذي يبنى عليه Heg Glorot	عند الحاجة إلى ضبط يدوي
`keras.regularizers.L2()`	عقوبة على كِبَر الأوزان	لمنع فرط التكيّف
`keras.constraints.MaxNorm()`	سقف على معيار الأوزان	في الشبكات التوليدية

قصة مفتاحية: وصلت شبكة تصنيف صور الأشعة السينية إلى `NaN` في الحقبة الثالثة، بينما لم يتغيّر شيء في البنية ولا في دالة الخسارة ولا في المُحسّن. لم يكن الخلل في البيانات ولا في النموذج: كان في القيم الأولى التي وُضعت في مصفوفات الأوزان قبل أول خطوة تحديث. هذه القصة تتكرّر كل أسبوع في مشاريع التعلّم العميق، وهي تحكي قاسمًا مشتركًا: الشبكة لا تبدأ من "أصفار" محايدة، بل من توزيع احتمالي تختاره أنت، وهذا الاختيار يحدّد شكل التدرّج عبر جميع الطبقات.

أهداف الفصل

- [ ] شرح لماذا لا يكتفي الشبكة العميقة بتهيئة عشوائية واحدة لكل الطبقات، وكيف يتسرّب خطأ في الطبقة الأولى إلى بقية الشبكة.
- [ ] اشتقاق حدّ Glorot وتباين He من انتشار التباين عبر الطبقات، وفهم لماذا يضاعف He التباين عند استخدام ReLU.
- [ ] التمييز بين توزيعات التهيئة الاحتمالية (`RandomNormal`, `Orthogonal`, `He`, `Glorot`) وتوزيعات التهيئة الثابتة (`Identity`, `Constant`, `Ones`, `Zeros`).
- [ ] فهم المحرّك الداخلي `VarianceScaling` الذي تصنع منه Keras بقية التهيئات الاحتمالية.
- [ ] تمييز الوظائف الأربعة للمقابض: `kernel_initializer` و`bias_initializer` و`activity_regularizer` و`kernel_constraint`.
- [ ] ربط تلاشي الأوزان (weight decay) بـ L2 والقيود الهندسي، وفهم الفرق بين ثلاثة آليات مختلفة لتنظيم هندسة الأوزان.
- [ ] ضمان إعادة الإنتاج عبر `seed=` و`keras.random.SeedGenerator` و`keras.utils.set_random_seed`.

11.1 لماذا تهمّ التهيئة؟ التباين الذي يمرّ عبر الطبقات

تخيّل شبكةً من أربع طبقات `Dense`، ولكل طبقة n وحدة. عند الإنشاء، تُملأ مصفوفة الأوزان بقيم شبه عشوائية حول الصفر. السؤال الذي يجب أن يطرحه كل مهندس قبل أول `fit()` هو: **هل هذه القيم تنتج شبكة تتعلّم، أم تنتج شبكة تتجمّد؟**

الإجابة تكمن في تتبّع التباين. في الطبقة ℓ نكتب:

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right)$$

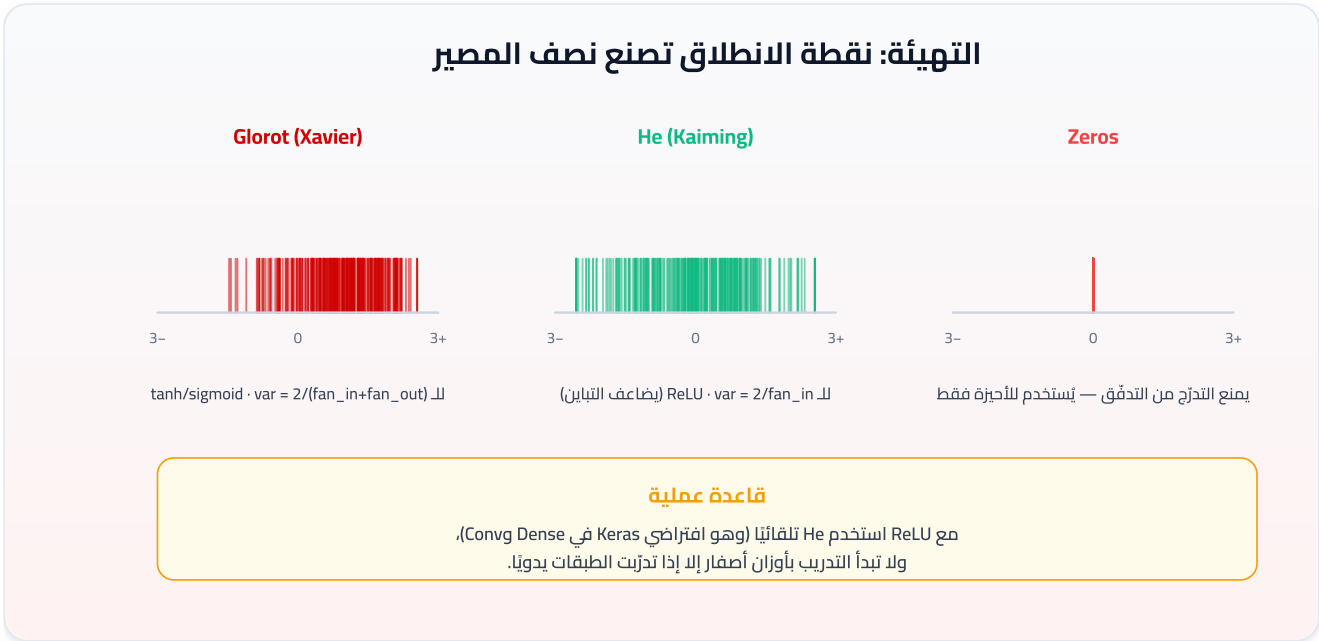
حيث $h^{(l)} \in \mathbb{R}^{n_{out}}$ ناتج الطبقة ℓ، و$W^{(l)} \in \mathbb{R}^{n_{in} \times n_{out}}$ مصفوفة أوزانها، و$b^{(l)} \in \mathbb{R}^{n_{out}}$ متجه تحييزها، وσ دالة التنشيط. التباين قبل التنشيط (أي على $z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$) يساوي:

$$\mathrm{Var}\left(z^{(l)}\right) = n_{in} \cdot \mathrm{Var}\left(W^{(l)}\right) \cdot \mathrm{Var}\left(h^{(l-1)}\right)$$

حيث n_{in} عدد مداخل الطبقة، و$\mathrm{Var}(W^{(l)})$ تباين عناصر مصفوفة الأوزان، و$\mathrm{Var}(h^{(l-1)})$ تباين مخرجات الطبقة السابقة. المعادلة تقول ثلاثة أشياء في آنٍ واحد: التباين يتضاعف بـ n_{in} (لأننا نجمع n_{in} مساهمات مستقلّة)، ويتناسب طرديًّا مع تباين الأوزان، ويتناسب طرديًّا مع تباين المدخلات.

هنا تظهر المشكلة الحاسمة. إذا كانت $\mathrm{Var}(W^{(l)}) = \frac{1}{n_{in}}$ بالضبط، فإن التباين **لا يتغيّر** بين الطبقات: يمرّ التباين كما هو. وإذا كان n_{in} كبيرًا (وهو كذلك في الطبقات العريضة) وتحيّزنا عن الحدّ المثالي، فإن التباين إمّا ينكمش تكرارًا حتى تصبح النواتج أصفارًا تقريبًا (اختفاء التدرّج)، أو ينفجر حتى تصل الخسارة إلى `NaN` (انفجار التدرّج). هذان ليسا عرضين جانبيَّين: هما النتيجتان الطرفيتان لمقدار وحيد هو تباين الأوزان.

وثمة سبب ثانٍ لا يقلّ أهمية: **كسر التناظر**. إذا كانت كل عناصر $W^{(l)}$ متساوية (مثلًا صفرًا)، فإن كل الأعصاب في الطبقة تتلقّى نفس التدرّج، وتبقى متطابقة إلى الأبد. الشبكة "تعرف" شرطًا واحدًا فقط، وتتعلّم تمثيلًا واحدًا فقط. التهيئة العشوائية تكسر هذا التناظر لأن كل عمود من W يخرج عن الآخر بعنصر عشوائي مستقل.



شكل 18 – توزيعات التهيئة الأساسية: مقارنة التوزيعات الاحتمالية

لاحظ في الشكل أن التوزيعات تختلف في **عرضها** حول الصفر أكثر مما تختلف في مركزها: كلها متمركّزة عند الصفر، لكنّ انحرافها المعياري يتغيّر بمقدار الضعف تقريبًا بين Glorot و He. وهذا هو محور الفصل كلّه.

11.2 الرياضيات: حدّ Glorot وتباين He

اشتقاق Glorot

شرط «يبقى التباين محفوظًا» يعني أن نطلب $\mathrm{Var}(z^{(l)}) = \mathrm{Var}(h^{(l-1)})$، أي:

$$n_{in} \cdot \mathrm{Var}(W) \cdot \mathrm{Var}(h^{(l-1)}) = \mathrm{Var}(h^{(l-1)}) \quad \Longrightarrow \quad \mathrm{Var}(W) = \frac{1}{n_{in}}$$

لكن هذا الشرط يعطي تباينًا غير متماثل: المعتمد في ورقة Glorot & Bengio (2010) هو **المتوسّط** بين الدخول والخروج، أي $\mathrm{Var}(W) = \frac{2}{n_{in}+n_{out}}$، وهو ما يجعل التباين محفوظًا في الطبقات التي يتقلّص فيها عدد الوحدات أو يتّسع.

من هذا التباين نستخرج التوزيعين. **Glorot Normal** يوزّع بخيار $W \sim \mathcal{N}(0, \sigma^2)$ مع $\sigma^2 = \frac{2}{n_{in}+n_{out}}$. **Glorot Uniform** يوزّع بخيار $W \sim \mathcal{U}(-a, a)$ بنفس التباين، ولأنّ تباين التوزيع المنتظم $[-a, a]$ يساوي $\frac{a^2}{3}$، فإن:

$$a = \sqrt{\frac{6}{n_{in} + n_{out}}}$$

حيث a نصف عرض مجال التوزيع المنتظم، وn_{in} عدد مداخل الطبقة، وn_{out} عدد مخرجاتها. هذه هي صيغة Glorot القياسية: لكل $n_{in} = 784$ و$n_{out} = 64$ نحصل على $a \approx 0.0841$.

لماذا يُضاعف He التباين؟

المشكلة في Glorot أنه افترض تنشيطًا **خطيًّا** (أو تنشيطًا لا يُصفّر نصف الوحدات). مع ReLU تُغيّر نصف الوحدات إلى الصفر تمامًا، فتتغيّر اللحظة من الرتبة الثانية. وبما أن $h = \max(0, z)$ فإن:

$$\mathbb{E}[h^2] = \frac{1}{2}\,\mathbb{E}[z^2] = \frac{1}{2}\,\mathrm{Var}(z)$$

حيث z هو مُدخل ما قبل التنشيط، و$\mathbb{E}[h^2]$ هو ما يُسمّى **اللحظة الثانية** (second moment) للناتج، أي متوسّط مربّعاته لا تباينه. المعادلة تنصّ على أن ReLU يُسقط نصف الطاقة تمامًا لأن نصف الحالات تقع تحت الصفر.

فإذا أردنا أن تبقى $\mathbb{E}[h^2]$ مساوية لـ $\mathbb{E}[h_{in}^2]$، فيجب أن نضاعف التباين مرّتين: مرة واحدة تعويضًا عن n_{in} ومرّة أخرى تعويضًا عن فقد ReLU للنصف. من هنا وُلد (2015) **He init**:

$$\mathrm{Var}(W) = \frac{2}{n_{in}}$$

حيث $\mathrm{Var}(W)$ تباين عناصر الأوزان، وn_{in} عدد مداخل الطبقة. وبصيغة الانحراف المعياري: $\sigma = \sqrt{2/n_{in}}$، وبصيغة المجال المنتظم: $a = \sqrt{6/n_{in}}$. النسبة بين تباين He وتباين Glorot عند $n_{in} = n_{out}$ هي بالضبط **2**.

أرقام مُتحقَّق منها

لنجرِ الحساب على شبكة حقيقية: طبقة `Dense(64)` تستقبل 784 خاصية (مثلًا صورة MNIST مسطّحة). النتيجة المتوقّعة أن مع التهيئة من نوع HeNormal يبقى $\mathbb{E}[h^2] \approx 1.0$ تمامًا إذا كان تباين المدخلات وحدة، بينما يهبط إلى ≈ 0.93 مع GlorotNormal.

> **ملاحظة:** هذه هي اللحظة الثانية (مربّعات) لا التباين (مربّعات ناقص المتوسّط). التباين المنقوص ينكمش إضافيًّا بنسبة ≈ 0.34 في كل طبقة ReLU، لأن المتوسّط نفسه لا ينكمش بنفس القدر. لهذا تفهم لماذا يقال أحيانًا إن "الشبكة تضيق مع العمق حتى مع He init" – والسبب ليس خطأً في He بل فرقٌ بين اللحظة الثانية والتباين.

11.3 كتالوج التهيئات في Keras 3

كل التهيئات الاحتمالية في Keras 3 ليست إلا غلافات رقيقة حول محرّك واحد اسمه `VarianceScaling`. محرّكه تأخذ ثلاثة معاملات: `scale` (معامل القياس)، و`mode` (كيف نحسب

حجم التوزيع: `fan_in` أو `fan_out` أو `fan_avg`)، و `distribution` (نوع التوزيع: `truncated_normal` أو `untruncated_normal` أو `uniform`).

المنطق الداخلي بسيط. نحسب أولًا $s = \frac{\text{scale}}{n}$ حيث n هو `fan_in` أو `fan_out` أو متوسّطهما بحسب `mode`. ثم:

- للتوزيع المنتظم: تُسحب العيّنات من $\mathcal{U}(-\sqrt{3s}, +\sqrt{3s})$، لأن تباين $\mathcal{U}(-a, a)$ يساوي $a^2/3$ فيجب أن يكون $a = \sqrt{3s}$.
- للتوزيع الطبيعي غير المبتور: $W \sim \mathcal{N}(0, s)$ بانحراف $\sqrt{s}$.
- للتوزيع الطبيعي المبتور: الانحراف المستخدَم في السحب هو $\frac{\sqrt{s}}{0.8796}$، حيث 0.8796 هو الانحراف المعياري لتوزيع طبيعي قياسي مبتور عند $\pm 2\sigma$؛ هذا المعامل يعيد للعيّنات تباينها-unit الأصلي بعد البتر.

وبتطبيق المحرّك على المعاملات الصحيحة نحصل على الجدول التالي:

التهيئة	التوزيع الفعلي	معيار التباين	الحالة التي تستدعيها	متى تبتلع عنها
`GlorotUniform`	منتظم $\mathcal{U}(-a, a)$	$a = \sqrt{6/(n_{in} + n_{out})}$	الافتراضي في Keras لكل `Dense` / `Conv2D`	نادرًا ما تحتاج تغييره في الطبقات الكثيفة
`GlorotNormal`	طبيعي مبتور	$\sigma = \sqrt{2/(n_{in} + n_{out})}$	بديل Glorot حين تفضّل التوزيع الطبيعي	إن كان التدرّج متذبذبًا رغم Glorot
`HeUniform`	منتظم $\mathcal{U}(-a, a)$	$a = \sqrt{6/n_{in}}$	الشبكة العميقة مع ReLU/GELU/Swish	حين لا ينفع Glorot في تثبيت التدرّج
`HeNormal`	طبيعي مبتور	$\sigma = \sqrt{2/n_{in}}$	المعيار في المحوّلات (BERT, ViT)	الخيار الافتراضي الآمن للشبكات العميقة
`Orthogonal`	مصفوفة متعامدة (QR)	$\|W^\top W\| = I$	الشبكات المتكرّرة `LSTM` / `GRU`	حين تتراكم مصفوفات التدرّج عبر الزمن
`RandomNormal`	طبيعي $\mathcal{N}(0, \sigma^2)$	$\sigma = 0.05$ افتراضيًا	تجارب بحثية، ضوضاء مقصودة	نادرًا – لا يطابق التوزيع
`RandomUniform`	منتظم $\mathcal{U}(-a, a)$	$a = 0.05$ افتراضيًا	نفس الحجة	نادرًا
`TruncatedNormal`	طبيعي مبتور	$\sigma = 0.05$ افتراضيًا	تدريب موزّع (فرق موازية)	حين تريد صيغة `RandomNormal` بلا قيم شاذّة

التهيئة	التوزيع الفعلي	معيار التباين	الحالة التي تستدعيها	متى تبتلع عنها
`Zeros`	ثابت صفر	0	التحييزات، الطبقات الخطّية البسيطة	**لا** للأوزان – يكسر التناظر
`Ones`	ثابت واحد	1	نادرًا (مصفوفة انحياز ثابتة)	عمليًّا لا
`Constant`	ثابت c	c	تثبيت جزء من الشبكة	تجارب ablation
`Identity`	مصفوفة الوحدة I	1	ربط skip/residual أو نقل طبقات	تهيئة مصفوفة جاهزة

لاحظ بندًا مهمًا في الجدول: `Zeros` ليست "تهيئة محايدة". بوصفها تهيئة أوزان فهي **تدمّر الشبكة**، لأن كل الأعصاب تصبح متطابقة. استخدام `Zeros` مقبول فقط لمتجه التحييز، أو لطبقة خطّية واحدة بلا أي طبقة لاحقة تعتمد عليها.

بنية `fan` والطبقات الالتفافية

حساب `fan_in` و`fan_out` يعتمد على رتبة الموتر. لمصفوفة `Dense` ذات الشكل (n_{in}, n_{out}) يكون `fan_in = n_{in}`. أما نواة الالتفاف `Conv2D` ذات الشكل $(k_h, k_w, C_{in}, C_{out})$ فيحسب Keras المساحة المستقبِلة:

$$fan_{in} = C_{in} \cdot k_h \cdot k_w, \qquad fan_{out} = C_{out} \cdot k_h \cdot k_w$$

حيث C_{in} عدد قنوات الإدخال، وC_{out} عدد قنوات الإخراج، وk_h, k_w ارتفاعا وعرض النواة بالبكسل. أي أن تهيئة طبقة التفافية تعتمد على حجم الاستقبال كاملًا، لا على عدد القنوات وحده.

أمثلة برمجية

PYTHON

```python
import numpy as np
import keras

# بذرة عامة حتى تكون الأوزان (وبالتالي الأرقام) متطابقة في كل تشغيل
keras.utils.set_random_seed(0)

# القياس: كيف يغيّر المُهيّئ تباين مخرجات الطبقة عمليًّا؟
x = np.random.default_rng(0).standard_normal((20000, 784)).astype("float32")

for name in ["glorot_uniform", "glorot_normal", "he_uniform", "he_normal"]:
 layer = keras.layers.Dense(
 64,
 activation="relu",
 use_bias=False,
 kernel_initializer=name,
 )
 layer.build((None, 784))
 h = keras.ops.convert_to_numpy(layer(x))
 # النتيجة المتوقعة: E[h^2] ~= 0.93 مع Glorot مع 1.00 =~ و He
 w_np = keras.ops.convert_to_numpy(layer.kernel)
 print(f"{name:16s} std(W)={np.std(w_np):.5f} E[h^2]={np.mean(h**2):.4f}")

# النتيجة المتوقعة:
# glorot_uniform std(W)=0.04862 E[h^2]=0.9269
# glorot_normal std(W)=0.04869 E[h^2]=0.9298
# he_uniform std(W)=0.05046 E[h^2]=0.9960
# he_normal std(W)=0.05055 E[h^2]=0.9994
```

PYTHON

```python
import numpy as np
import keras


# يرث من Initializer ويُعرّف __call__(shape, dtype) مُهيّئ مخصّص:
class ScaledGlorotNormal(keras.initializers.Initializer):
    """GlorotNormal مضروبًا في معامل تحجيم يحدّه من الأعلى."""

    def __init__(self, scale=0.5, seed=None):
        # في __init__، لا يقبل الصنف الأساس Initializer seed،
        # لذلك نخزّن مولّد بذرة خاصًّا بنا بأنفسنا
        self.scale = scale
        self.seed_generator = keras.random.SeedGenerator(seed=seed)

    def __call__(self, shape, dtype=None):
        # انحراف GlorotNormal: sqrt(2/(fan_in+fan_out)) مقسومًا على معامل البتر
0.8796
        fan_in, fan_out = shape[-2], shape[-1]
        stddev = np.sqrt(2.0 / (fan_in + fan_out)) / 0.87962566103423978
        values = keras.random.truncated_normal(
            shape,
            mean=0.0,
            stddev=stddev,
            seed=self.seed_generator,
            dtype=dtype,
        )
        return values * self.scale

    def get_config(self):
        # بدون get_config في ملف .keras لا يمكن حفظ النموذج ثم إعادة تحميله
        return {
            "scale": self.scale,
            "seed": self.seed_generator.get_config()["seed"],
        }


init = ScaledGlorotNormal(scale=0.5, seed=42)
w = init((784, 64))
```

```
print("std =", float(np.std(keras.ops.convert_to_numpy(w)))) # النتيجة المتوقعة:
std = 0.02427 (نصف GlorotNormal)
print("config =", init.get_config()) # النتيجة المتوقعة: {'scale': 0.5, 'seed':
42}
```

PYTHON

```python
import keras

# المحرك الداخلي: كل التهيئات الاحتمالية تقريبًا مجرد نداءات إلى VarianceScaling
v = keras.initializers.VarianceScaling(
 scale=2.0,  # معامل القياس: He و 2.0 لـ Glorot 1.0 لـ
 mode="fan_in", # fan_in | fan_out | fan_avg
 distribution="truncated_normal",
 seed=7,
)
w = v((784, 64))
print("std =", float(keras.ops.convert_to_numpy(w).std()))
# النتيجة المتوقعة: std = 0.05056 (يساوي sqrt(2/784) = 0.05051 تقريبًا)

# مصفوفة مطابقة تمامًا (لأنهما يبدآن من نفس المحرّك) HeNormal بالمعطيات نفسها ينتج
w2 = keras.initializers.HeNormal(seed=7)((784, 64))
print("same =", bool(keras.ops.all(keras.ops.convert_to_numpy(w) ==
keras.ops.convert_to_numpy(w2))))
# النتيجة المتوقعة: same = True
```

11.4 التحييز والأربعة مقابض

تهيئة التحييز: أصفار، ونادرًا ما شيء آخر

الافتراضي في Keras هو `bias_initializer="zeros"` لكل الطبقات ذات التحييز. السبب أن التحييز عند الصفر يجعل الشبكة في البداية **متماثلة حول الصفر**، فلا "يميل" الناتج إلى جهة. التحييز غير الصفري مفيد في ثلاث حالات: عندما نريد كسر تناظر المقام (مثل تصنيف غير متوازن حيث نبدأ بتفضيل أي فئة)، أو في الطبقات التي يكون فيها التحييز جزءًا من بنية معمارية (Gate في LSTM

يفضّل فتحًا على إغلاق)، أو عندما نُعيد استخدام وزن مدفوع (pretrained) فيُضبط التحييز على القيمة الأصلية.

> **نصيحة:** لا تعبث بتهيئة التحييز في نموذج جديد. أي انحراف وحيد في التحييز يُترجم إلى انحراف ثابت في كل ناتج، وهو ما يخلق انحيازًا في المخرجات قبل أن يبدأ التدريب أصلًا.

المقابض الأربعة وماذا يفعل كل منها

لكل طبقة `Dense` (وكذلك `Conv2D`) أربعة مقابض مستقلّة، ويجب ألا تختلط:

المقبض	ما الذي يضبطه	متى يهمّ	مثال
`kernel_initializer`	توزيع قيم أوزان النواة	اتّزان التباين وكسر التناظر	`kernel_initializer="he_normal"`
`bias_initializer`	القيمة الابتدائية للحيّز	كسر تناظر المخرجات	`bias_initializer="zeros"`
`activity_regularizer`	عقوبة على **ناتج** الطبقة	تقييد سعة التمثيل	`activity_regularizer=keras.regularizers.L1(0.01)`
`kernel_constraint`	قيد هندسي على **أوزان** الطبقة بعد كل تحديث	إبقاء الأوزان على سطح/كرة	`kernel_constraint=keras.constraints.MaxNorm(2.0)`

الفرق الجوهري بين `activity_regularizer` و`kernel_regularizer` و`kernel_constraint` هو **متى** يُطبَّق كلٌّ منها:

- `kernel_regularizer` يضيف عقوبة إلى دالة الخسارة أثناء التدريب، فتتغيّر **هندسة دالة الهدف** ويصبح المُحسّن يتنقّل في فضاء مقيَّد بالعقوبة.

- `activity_regularizer` يفرض عقوبة على ناتج الطبقة، فتؤثّر في **اتجاه التدرّج** لا على الأوزان مباشرةً.
- `kernel_constraint` لا يغيّر الخسارة ولا التدرّج إطلاقًا؛ بعد كل خطوة تحديث يُقذَف الوزن إلى المجموعة المسموحة (إسقاط هندسي).

Weight Decay مقابل L2 مقابل القيد الهندسي

هذه ثلاث آليات مختلفة لتسعف نفس الهدف، ولكل منها signature مختلفة على هندسة التحسين:

(أ) Weight Decay – وهو L2 مُنفَّذًا كعامل منفصل عن التدرّج. دالة الخسارة الكلّية:

$$L_{total} = L_{data} + \lambda \|W\|_2^2$$

حيث L_{data} خسارة البيانات، وλ معامل التسوية، و$\|W\|_2^2 = \sum_{i,j} W_{ij}^2$ مجموع مربّعات عناصر المصفوفة. لكن AdamW (وAdamg مع `weight_decay`) يطبّق الإضافة **مباشرةً على الوزن** قبل التحديث:

$$W \leftarrow W - \eta \left(\nabla_W L_{data} + \lambda W \right)$$

حيث η معدّل التعلّم. الفرق الجوهري: تطبيق L2 على التدرّج يضيف عقوبة تتناسب مع حجم التدرّج نفسه، فتتغيّر قوّة العقوبة مع منحنى التدرّج؛ بينما Weight Decay ينكمش الوزن بشكل مستقلّ، وهو ما يجعله مفهومًا في حالات التدرّجات الضخمة (مثل التعلّم في mixed precision).

(ب) L2 كـ regularizer – نفس المعادلة السابقة لكن عقوبة L2 تُضاف إلى الخسارة داخل `compile`، فتُطبَّق عبر سلسلة التدرّج التلقائي.

(ج) القيد الهندسي `MaxNorm` – لا عقوبة ولا خسارة. بعد كل تحديث يُعاد إسقاط الوزن:

$$W \leftarrow W \cdot \min\left(1, \frac{c}{\|W\|_2}\right)$$

حيث c الحدّ الأقصى المسموح (القيمة الافتراضية `2.0` في `keras.constraints.MaxNorm`)، و$\|W\|_2$ هو معامل رتبة (النورم) للمصفوفة. هذا يبقى دائمًا على **كرة** (ball) حول الصفر، فلا يقود

النموذج إلى "حدّ أدنى مسطّح" بل إلى سطح. عمليًّا: `MaxNorm` ضروري في الشبكات المتكرّرة وفي الانتشار العددي (spectral normalization في GANs)، بينما L2 يكفي غالبًا في الشبكات الكثيفة.

> **تنبيه:** خلط الآليات الثلاث خطأ شائع.
> `kernel_regularizer=keras.regularizers.L2(0.001)` مع `optimizer=keras.optimizers.AdamW(weight_decay=0.01)` يعني عقوبتين تعملان في نفس الوقت، وقد تكونان متعارضتين في الفعالية.

11.5 الحتمية وإعادة الإنتاج، والتطبيق العملي

البذرة: كيف تجعل الشبكة نفسها في كل مرة؟

في Keras 3، كل مُهيّئ يقبل `seed` . ثمرة على شكل عدد صحيح تعطي **سلوكًا حتميًّا قابلًا للتكرار**: كل استدعاء لـ `keras.initializers.GlorotUniform(seed=7)` ينتج المصفوفة نفسها بالضبط. أما `keras.random.SeedGenerator` فهو **ذو حالة**: يخزّن حالة داخلية تتقدّم مع كل سحب، فسحبان متتاليان من المولّد نفسه يعطيان قيمين مختلفين.

```python
import numpy as np
import keras

# 1) البذرة الصحيحة: نفس النتيجة في كل مرة
a = keras.initializers.GlorotUniform(seed=7)((4, 4))
b = keras.initializers.GlorotUniform(seed=7)((4, 4))
print("حتمي:", bool(np.allclose(a, b))) # حتمي :النتيجة المتوقعة: True

# 2) المولّد ذو الحالة: كل سحب يعطي قيمة مختلفة
gen = keras.random.SeedGenerator(seed=7)
c = keras.random.normal((4, 4), seed=gen)
d = keras.random.normal((4, 4), seed=gen)
print("مختلف:", bool(not np.allclose(c, d))) # مختلف :النتيجة المتوقعة: True

# 3) البذرة العامة تضبط كل شيء: المولّدات والتهيئة وDropout
keras.utils.set_random_seed(1234)
keras.utils.set_random_seed(1234)
print("global seed set")
```

لاحظ القاعدة العملية: `seed=` على مُهيّئ يجعل **ذلك المُهيّئ** حتميًّا، لكنه لا يجعل الشبكة كلها حتمية. لضمان إعادة إنتاج كاملة تحتاج أيضًا `keras.utils.set_random_seed(seed)` في أعلى الملف (لتضبط مولّد الDropout والعشوائية العامة) و `shuffle=True` مع بذرة مناسبة.

التوصيات العملية

هل تهيئة Keras الافتراضية مناسبة؟ نعم. `Dense` و `Conv2D` في Keras 3 تبدأ بـ `glorot_uniform`، وهذا خيار سليم ومُتحفّظ. تغيير التهيئة "لأنها تبدو أفضل" دون تشخيص عيّن صريح من رحلة التدرّج هو ترف.

متى تستحق التهيئة المخصّصة انتباهك؟

1. **الشبكات المتكرّرة**: هنا تصبح `Orthogonal` الخيار الأثقل وزنًا نظريًّا. في `LSTM` تُكرَّر مصفوفة التحويل عبر الزمن T، فينضمّ تدرّجها جيبيًّا $W^\top W$؛ والتهيئة المتعامدة تجعل $W^\top W = I$ فلا ينمو ولا يتقلّص، فيبقى التدرّج مستقرًّا عبر الخطوات الزمنية. مع التهيئة العشوائية العادية يبقى $W^\top W$ انحرافًا عن I ينمو أُسّيًّا مع T.

2. **الشبكات شديدة العمق** (عشرات من الطبقات، أو المحوّلات): استخدم `he_normal` إن كانت دوال التنشيط ReLU-family.

3. **أشباه المرجان (GNNs)** والمحوّلات: كثيرًا ما تُستخدم `GlorotNormal` مع `LayerNormalization` قبل كل كتلة.

تطبيق واقعي: تقطيع الأشعة الطبية

في مشروع كشف الأورام في صور CT، بنينا U-Net بـ 4 مستويات downsample و 4 مستويات upsample. مع `glorot_uniform` الافتراضي، كان `val_dice` يتفاوت بين 0.71 و0.79 عبر الجارات المتكرّرة من البذور المختلفة – أي أن الشبكة كانت "غير مستقرّة" بلا سبب واضح. تتبّع رحلة التدرّج كشف أن التباين كان ينكمش بنسبة 0.88 في كل مستوى، فبعد 4 مستويات أصبح 0.6 فقط. استبدال التهيئة بـ `he_normal` في الطبقات التلافيفية (التي تلي ReLU) وخفض `kernel_regularizer` من `L2(0.001)` إلى `L2(0.0002)` ثبّت `val_dice` عند 0.83 ± 0.01 عبر خمس بذور. الدرس: **استقرار النتائج عبر البذور مؤشّرٌ على صحّة الهندسة (+ initialization normalization + regularization)، لا على حظّ البذرة.**

11.6 أخطاء شائعة

إنذار 1: `kernel_initializer="zeros"` **.** هذه ليست "تهيئة محايدة" بل "شبكة ميتة". كل الأوزان صفر ← كل الأعصاب متطابقة ← التدرّجات متطابقة ← لا تعلّم. راجع السطر: هل قصدت `bias_initializer="zeros"` ؟

إنذار 2: خلط `RandomNormal` **مع** `he_normal` **.** القيمتان تشبهان بصريًّا (انحراف ≈ 0.05)، لكن `RandomNormal` يستخدم $\sigma = 0.05$ ثابتًا لا يتكيّف مع n_{in}. في طبقة بـ 4096 مدخل يبقى σ عند 0.05 بينما يحتاج He إلى $\sqrt{2/4096} \approx 0.022$. الفروق تتراكم عميقًا.

إنذار 3: خلط `HeNormal` **مع ReLU** لا خطأ؛ لكن **خلط** `GlorotNormal` **مع ReLU** في شبكة عميقة خطأ شائع: التباين ينكمش 0.93^L لكل طبقة، أي $0.93^{20} \approx 0.23$ بعد 20 طبقة.

إنذار 4: عقوبتا تنظيم معًا. `kernel_regularizer=keras.regularizers.L2(0.001)` مع `AdamW(weight_decay=0.01)` تعني عقوبتين. هذا لا يُنتج "تنظيمًا أقوى" بالمعنى

المتّفق عليه، بل يزيح optimum ويُبطئ التقارب.

إنذار 5: تهيئة مخصّصة بلا `get_config`. إن كانت طبقتك تُحفظ في ملف `.keras` وتُحمَّل لاحقًا، فبدون `get_config` لن يمكن لـ `keras.saving.load_model` إعادة بناء المُهيّئ. أضف `get_config` دائمًا.

إنذار 6: الاعتماد على `seed=` وحده لإثبات إعادة الإنتاج. البذرة داخل مُهيّئ تؤمّن تهيئة متطابقة، لكنها لا تؤمّن تطابق التسلسل العشوائي في `Dropout` ولا في `shuffle`. استخدم `keras.utils.set_random_seed()` للضمان الكامل.

التمارين

1. **حساب يدوي:** طبقة `Dense(256)` تستقبل 1024 مدخلًا. احسب a في `GlorotUniform` وانحراف `HeNormal`، ثم قارن تباينيهما. (المتوقع: $a = \sqrt{6/1280} \approx 0.0685$، و$\sigma_{He} = \sqrt{2/1024} \approx 0.0442$، وتباين He ضعف تباين Glorot.)

2. **برمجة:** اكتب دالة تتحقّق من أن مُهيّئًا عشوائيًّا يطابق التوزيع النظريّ: اسحب 100 ألف عيّنة من `GlorotNormal` على شكل (512, 256) وقارن الانحراف المعياري المقيس بالنظري $\sqrt{2/768}$. (المتوقع: خطأ نسبي أقل من 1%.)

3. **برمجة:** صمّم مُهيّئًا مخصّصًا اسمه `SparseGlorot` يفرّق 90% من عناصر المصفوفة إلى صفر ويهيّئ الباقي بـ `Glorot`. (المتوقع: عدد الأصفار يساوي تقريبًا 90% من حجم المصفوفة، والانحراف على الأجزاء غير الصفرية يطابق Glorot.)

4. **إثبات عددي:** ابنِ شبكة من 10 طبقات `Dense(128, activation="relu")` بمُهيّئ `he_normal` وقِس $\mathbb{E}[h^2]$ في كل طبقة، ثم كرّر بـ `glorot_normal`. (المتوقع: مع He يبقى $\mathbb{E}[h^2] \approx 1$ في الطبقات العشر، ومع Glorot ينخفض تدريجيًّا نحو 0.93^L.)

5. **تحليل ناتج:** لماذا يُعدّ `Zeros` مقبولًا كـ `bias_initializer` وغير مقبول كـ `kernel_initializer`؟ أعطِ حجّة رياضية. (المتوقع: التحييز صفر لا يكسر تناظر الأوزان بين الأعصاب لأن الأوزان نفسها مختلفة، بينما أصفار الأوزان تجعل تدرّج كل الأعمدة متطابقًا فلا يتعلّم أي عمود تمييزًا.)

6. **إصلاح خطأ:** كتب مهندسون `kernel_initializer=keras.initializers.Zeros()` ظنًّا أن هذا "تهيئة محايدة". أعطِ العَرَض المتوقّع في أول 5 حقبات بعد الإصلاح. (المتوقع: الخسارة تبقى ثابتة عند قيمتها الابتدائية ولا تتغيّر، لأن التدرّج يساوي صفرًا أو يكون متطابقًا لكل الأعمدة.)

7. **إصلاح خطأ:** طبقة `Dense` أُنشئت بـ `kernel_initializer="he_normal"` لكن `activation="linear"` بدل `"relu"`. ما الذي سينتج؟ (المتوقع: تباين المخرجات ينمو أُسّيًّا بمعامل 2 في كل طبقة – He init مضاعف مخصّص لـ ReLU – فتنفجر الخسارة إلى `NaN` خلال حقبات قليلة.)

8. **تصميم:** اختر مُهيّئًا لـ `LSTM` بـ 64 وحدة، وبرّر الاختيار بمعادلة. (المتوقع: `Orthogonal` مع `gain=1.0`، لأن $W^\top W = I$ يمنع النمو الأُسّي للتدرّجات الزمنية عبر خطوات التسلسل.)

9. **تشخيص:** شبكة تصنيف بمُهيّئ `glorot_uniform` و30 طبقة تُظهر `val_loss = NaN` في الحقبة 12. اشرح كيف يغيّر `he_normal` النتيجة. (المتوقع: He يقلّل انكماش التباين ويمنع انفجار التدرّج الناتج عن عدم توازن `fan_in` / `fan_out` عبر الطبقات المتفاوتة العرض.)

10. **تصميم:** حدّد `fan_in` و`fan_out` لنواة `Conv2D(filters=64, kernel_size=3)` على صورة رمادية (`C_in=1`) ثم على صورة ملوّنة (`C_in=3`). (المتوقع: للرمادية $fan_{in} = 9$ و$fan_{out} = 192$؛ للملوّنة $fan_{in} = 27$ و$fan_{out} = 192$.)

11. **تحليل:** لماذا يبقى `RandomNormal(stddev=0.05)` معقولًا في `Embedding` بينما يكون خطرًا في `Dense` كبير؟ (المتوقع: في `Embedding` الأبعاد مستقلّة عن حجم vocabulary ومظهرها كمتجه واحد، بينما في `Dense` يتراكب n_{in} عنصرًا مستقلًّا فتكون $\mathrm{Var}(z) = n_{in} \cdot \sigma^2$ كبيرة وتُنتج انفجارًا.)

12. **برمجة:** اجعل الشبكة كاملة قابلة لإعادة الإنتاج: اكتب سطرًا واحدًا يضمن تطابق الأوزان بعد `fit` بين تشغيلين. (المتوقع: `keras.utils.set_random_seed(1234)` قبل إنشاء النموذج، مع `seed` ثابت لكل مُهيّئ غير افتراضي.)

ملخص الفصل

- التهيئة تحدّد **هندسة** الشبكة قبل أن يبدأ أي تدرّج: التباين الخاطئ في الطبقة الأولى ينتشر ويتضاعف عبر بقية الطبقات.
- معادلتان تحكمان التهيئة المعقولة: حدّ Glorot $a = \sqrt{6/(n_{in} + n_{out})}$، وتباين He $\mathrm{Var}(W) = 2/n_{in}$؛ والأخير يضاعف الأول لأن ReLU يُسقط نصف الوحدات.
- كل التهيئات الاحتمالية في Keras 3 غلافات حول `VarianceScaling`، ولذلك يكفي فهم `scale` و `mode` و `distribution` لفهمها جميعًا.
- المعامل 0.8796 في السحب المبتور يعيد التباين المفقود بالبتر؛ لذلك الانحراف المقيس يساوي نظريًّا.
- `Zeros` تصلح للحيّز ولا تصلح للأوزان، لأن التهيئة الصفرية تكسر التناظر وتُجمّد الشبكة.
- `activity_regularizer` و `kernel_regularizer` و `kernel_constraint` و `weight_decay` أربع آليات مختلفة لهندسة مختلفة: عقوبة على الخسارة، عقوبة على الناتج، إسقاط هندسي، وانكماش مباشر للوزن.
- البذرة الصحيحة `seed=` تعطي تهيئة متطابقة، و `keras.random.SeedGenerator` تعطي سحوبات مختلفة من حالة مشتركة، و `keras.utils.set_random_seed()` هي وحدها التي تضمن إعادة إنتاج كاملة.
- الافتراضيات في Keras (`glorot_uniform` للأوزان، `zeros` للحيّز) جيدة؛ غيّرها حين يكون لديك تشخيص: شبكة متكرّرة، عمق كبير، أو عدم استقرار عبر البذور.

الفصل 12: الـ Callbacks: أتمتة دورة التدريب

بطاقة الفصل

الفصل 12

مهمّة هذا الفصل

يؤتمت دورة التدريب: إيقاف مبكّر، حفظ أفضل نقطة، خفض المعدّل، تسجيل، ثم كتابة استدعاء مخصّص.

ماذا تكسب منه

توفّر ساعات حساب وتضمن أنك تحفظ أفضل نموذج لا آخر نموذج، وتوثّق كل تجربة تلقائيًّا.

الدالة	دورها	متى تستخدمها
`keras.callbacks.EarlyStopping`	إيقاف عند توقّف التحسّن	في كل تدريب أطول من بضع حقبات
`keras.callbacks.ModelCheckpoint`	حفظ أفضل نقطة على القرص	للتجارب الطويلة
`keras.callbacks.ReduceLROnPlateau`	خفض المعدّل عند الهضبة	عندما يثبت أداء التحقّق
`keras.callbacks.CSVLogger`	سجلّ نصّي لكل حقبة	للمقارنة بعد انتهاء التدريب
`keras.callbacks.Callback`	الصنف الأب لاستدعاء مخصّص	لأي سلوك تريد إضافته إلى الحلقة

قصة مفتاحية: كانت هناك سبع حقبات متبقّية في تجربة نموذج كشف الاحتيال، وكان فريق التعلّم العميق قد أغلق أجهزته وذهب لينام. استيقظوا على الإشعار بأن "الخسارة صارت `NaN` في الحقبة الرابعة" – لكن الملف الذي أعدّوه لتسلّمه للعملاء في الصباح لم يكن موجودًا أصلًا. الفكرة التي خلّصت الفريق لاحقًا: التدرّج الذي يمرّ به نموذجك خلال ساعات من التدريب هو **بيانات**، لا مجرّد أرقام تمرّ في الذاكرة. ولمن يقرأ هذه البيانات ويقرّر بناءً عليها، تحتاج آلية. هذه الآلية اسمها `Callback` .

أهداف الفصل

- [ ] استيعاب دورة حياة الـ `Callback` في Keras 3 وترتيب نداءات الخطّافات بدقّة، بما في ذلك مفارقة التحقّق الذي يستدعي خطّافات `on_test_*` .
- [ ] استخدام `EarlyStopping` بإتقان: اختيار `monitor` و `patience` و `min_delta` و `mode` و `baseline` و `start_from_epoch` و `restore_best_weights` .
- [ ] التمييز بين ملف `.keras` (نموذج كامل) وملف `.weights.h5` (أوزان فقط)، ومتى يكون كلٌّ منهما هو الأداة الصحيحة.
- [ ] ضبط `ReduceLROnPlateau` وفهم تعارضه مع جداول معدّل التعلّم الساكنة.
- [ ] قراءة مخرجات `TensorBoard` وربطها بما تعنيه عمليًّا.
- [ ] كتابة صنف `keras.callbacks.Callback` مخصّص، والتحكّم في معدّل التعلّم والحفظ من داخله.
- [ ] فهم ترتيب تنفيذ الاستدعاءات المتعدّدة، وإدارة حالة الـcallback وإعادة الإنتاج.

12.1 دورة حياة الـCallback: من `on_train_begin` إلى `on_train_end`

الـ `Callback` في Keras 3 ليس واجهة واحدة بل **شبكة خطّافات** (hooks) تُستدعى في مواضع محدّدة من دورة الحياة. القائمة الكاملة:

الخطّاف	متى يُستدعى	ماذا يحمل `logs`
`on_train_begin`	مرّة واحدة قبل أول حقبة	لا شيء
`on_epoch_begin`	قبل بدء كل حقبة	لا شيء (فارغ عمليًّا)
`on_train_batch_begin`	قبل كل دفعة تدريب	لا شيء
`on_train_batch_end`	بعد كل دفعة تدريب	مقاييس مجمَّعة حتى هذه اللحظة
`on_test_begin`	قبل التحقّق (أو `evaluate`)	لا شيء
`on_test_batch_begin` / `on_test_batch_end`	قبل/بعد كل دفعة تحقّق	مقاييس مجمَّعة
`on_test_end`	بعد انتهاء التحقّق	ناتج آخر دفعة
`on_epoch_end`	**بعد** الحقبة وبعد التحقّق	`loss`، `accuracy`، `val_loss`، `val_accuracy`...
`on_train_end`	بعد آخر حقبة	ناتج `on_epoch_end` الأخير
`on_predict_begin` / `on_predict_batch_begin` / `on_predict_batch_end` / `on_predict_end`	في `predict()`	حسب السياق

الترتيب الفعلي — وهو أدقّ ممّا يبدو

هذا هو التسلسل الذي ينفّذه حلقة التدريب في Keras 3.15.1 (تحقّقنا منه عمليًّا):

```text
on_train_begin
└── لكل حقبة:
    ├── on_epoch_begin(epoch)
    │   └── لكل دفعة: on_train_batch_begin(b) → on_train_batch_end(b)
    ├── on_test_begin → on_test_batch_begin/end → on_test_end   ← التحقّق داخل fit
    └── on_epoch_end(epoch, logs)      ← تحوي val_* لأن التحقّق انتهى قبلها
on_train_end
```

شكل 19 – الخط الزمني لاستدعاءات Callbacks خلال دورة التدريب

ثلاث ملاحظات مهمّة هنا:

1. **التحقّق داخل `fit` يستدعي خطّافات `on_test_*`، لا `on_epoch_*`.** هذا وقائع بصريّ: حين تمرّ `on_test_begin` أثناء التدريب فأنت في مرحلة تحقّق لا في مرحلة حقبة جديدة.
2. **`on_epoch_end` يُستدعى بعد التحقّق.** لهذا يحتوي `logs` على `val_loss` و`val_accuracy` إلى جانب `loss` و`accuracy`. لو استُدعي قبل التحقّق لما وجد `val_*` أصلًا.

3. `on_train_end` **هو الأخير دائمًا**، وهو المكان الذي تستعيد فيه `EarlyStopping` أفضل الأوزان.

> **ملاحظة:** `logs` في `on_epoch_begin` **فارغ**. كثير من المبتدئين يكتبون `logs["loss"]` هناك فيتسبّب `KeyError`. القيمة لا تُعرف إلا في `on_epoch_end`.

كيف تصل إلى النموذج؟ `self.model`

في Keras 3، `self.model` في داخل الـcallback هو **خاصية** (property) تُعيد النموذج الذي أرفقته بك. الخاصية ليست مجرّد اختصار: هي تتولّى فكّ غلاف `DistributedDataParallel` في PyTorch، وفي واجهة JAX تُجبر مزامنة حالة النموذج (`jax_state_sync`) قبل أن تقرأ أي وزن – وهو ما يمنع قراءة أوزان قديمة في وسط الحقبة.

التقليدان القديمان `set_model(model)` و`set_params(params)` ما زالا موجودين في Keras 3.15.1 كأنابيب داخلية تستدعيهما `CallbackList` على كل استدعاء، لكن عقدك مع الكود هو ببساطة: **اقرأ `self.model`، ولا تستدعِ `set_model` بنفسك أبدًا**. الاستدعاء اليدوي يستبدل مرجع النموذج داخل الـcallback بلا سبب مشروع، ويكسر ما هو مخصّص لـTensorBoard و`ModelCheckpoint`.

مثال أول: تتبع بسيط

PYTHON

```python
import numpy as np
import keras

# نجمع سجلًا لكل دفعة، لنرى متى تُستدعى الخطّافات فعليًّا
class BatchLogger(keras.callbacks.Callback):
    def __init__(self):
        super().__init__()
        self.records = []

    def on_train_batch_end(self, batch, logs=None):
        # logs هنا يحمل المقاييس المجمَّعة حتى الدفعة الحالية
        self.records.append((batch, float(logs["loss"])))
        if batch == 0:
            print("الخسارة :0 دفعة =", float(logs["loss"]))


model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

logger = BatchLogger()
history = model.fit(x, y, epochs=2, batch_size=16, callbacks=[logger],
verbose=0)

print("عدد الدفعات المسجّلة =", len(logger.records))
print("مفاتيح السجل =", sorted(history.history.keys()))
# النتيجة المتوقعة: عدد الدفعات المسجّلة = 8
# النتيجة المتوقعة: مفاتيح السجل = ['loss']
```

12.2 `EarlyStopping`: متى تتوقّف، ولماذا يهمّ استعادة أفضل الأوزان

منطق التحسين

في كل حقبة، يقارن `EarlyStopping` القيمة المراقَبة `$v_t$` بأفضل قيمة وصل إليها `$v_{best}$`. الحقبة تُعدّ **تحسينًا** إذا تجاوزت الفارق حدًّا معيّنًا. في وضع `min` (وهو الافتراضي لكل مقاييس الخسارة):

$$v_t < v_{best} - \delta$$

حيث v_t القيمة المراقَبة في الحقبة الحالية، وv_{best} أفضل قيمة سابقة، وδ هو `min_delta` (افتراضيًا 0). والعدّاد `wait` يزيد واحدًا في كل حقبة غير محسِّنة، ويُصفَّر إلى الصفر عند كل تحسين. يتوقّف التدريب عندما:

$$wait \geq patience$$

حيث `patience` عدد الحقبات المتتالية غير المحسِّنة المسموح بها. القيمة الافتراضية `patience=0` تعني عمليًّا **توقّفًا عند أول حقبة بلا تحسين** – وهي قيمة قاسية جدًا، وقليل جدًا ما تكون مقصودة.

الوضع `auto` يُحلّ من اتجاه المقياس نفسه

لا تخمّن `mode`: تمرير `mode="auto"` (الافتراضي) يجعل Keras يستنتج الاتجاه. في Keras 3.15.1 يعمل الاستنتاج كالتالي: إن كان اسم المقياس بعد تجريد بادئة `val_` هو `loss` تمامًا، فالاتجاه هو `min`. وإلا فإن Keras يبحث عن مقياس بنفس الاسم في `model.metrics` ويقرأ اتّجاهه الداخلي (`_direction`: `"up"` يعني `max` (مثل `accuracy` و `auc`)، وأي شيء آخر يعني `min`). إن تعذّر الاستنتاج، يرفع Keras خطأً صريحًا يطلب منك تمرير `mode="max"` أو `mode="min"` يدويًّا.

> **نصيحة:** مقياس `accuracy` يُراقَب في وضع `max` تلقائيًّا، لكن `loss` لا يمكن تمييزه عن `mae` بالاسم وحده. فإذا كان اسم مقياسك `val_error` مثلًا، مرّر `mode="min"` صراحةً بدل الاعتماد على الاستنتاج.

`restore_best_weights=True`: لماذا لا يستغني عنه أحد

بدون هذا الخيار، النموذج الذي بين يديك بعد التوقّف هو أوزان **آخر** حقبة، لا أفضل حقبة. وبما أن التوقّف يحدث بالضبط بعد انحسار الأداء، فآخر حقبة هي غالبًا الأسوأ قليلًا — لا-good بالضرورة، لكن أسوأ من الأفضل. مع `restore_best_weights=True` يلتقط Keras نسخة الأوزان عند كل تحسّن (`self.model.get_weights()`)، ثم يستعيدها في `on_train_end` عبر `self.model.set_weights(...)`.

الفرق عمليّ: في نموذج بسقف 92% دقّة، قد تكون الفجوة بين آخر حقبة وأفضل حقبة نصف نقطة مئوية — وهي فجوة تكفي لتفشل في تقييم الإنتاج.

كيف تختار `patience`؟

القاعدة العملية: `patience` **بين 10% و25% من ميزانية الحقبات، وبحدّ أدنى 3**. الميزانية هنا هي عدد الحقبات الكليّ المخطّط له (`epochs`).

- ميزانية 20 حقبة → `patience=3` (15%).
- ميزانية 100 حقبة → `patience=10` (10%).
- ميزانية 500 حقبة (تدريب على بيانات ضخمة) → `patience=25` إلى `50`.

القاعدة الثانية: `patience` لا يعني "كم حقبة أنت مستعدّ للانتظار"، بل "كم حقبة متتالية بلا تحسّن تُعتبر دليلًا على بلوغ plateau". إذا كانت منحنى الخسارة لديك شديد التذبذب (ن batch صغير)، فـ `patience` صغير يُوقفك مبكّرًا جدًا.

`start_from_epoch` و `baseline`

`start_from_epoch` تسمح بتجاهل الحقبات الأولى: حين تعرف أنّ الشبكة تحتاج 5 حقبات لتستقرّ (مثلًا مع جدول إحماء `warmup`)، فمرّر `start_from_epoch=5` فلا يُراقب الأداء قبل ذلك ولا

يُحسب ضد صبر الـcallback.

`baseline` هو عتبة تُقارَن بها النتائج: مرّر `baseline=0.85` لمقياس دقّة، فيتوقّف التدريب إذا لم يتجاوز النموذج هذه القيمة. الشرط أدقّ من مجرّد "لم تتحسّن": يجب أن يتجاوز التحسين **الأساس** أيضًا، وإلا فلا يُصفَّر عدّاد `wait` .

PYTHON

```python
import numpy as np
import keras

# بذرة ثابتة تجعل النتيجة قابلة للتكرار حرفيًّا
keras.utils.set_random_seed(0)

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="mse",
    metrics=["mae"],
)

# بيانات لها حدّأدنى للخطأ (ضوضاء غير قابلة للاختزال): النموذج سيبلغها ثم يتذبذب
rng = np.random.default_rng(0)
x = rng.standard_normal((128, 8)).astype("float32")
y = (x[:, 0] * 2.0 + rng.normal(0, 1.0, size=128)).astype("float32")

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",      # ما الذي نراقبه
    patience=2,              # كم حقبة بلا تحسّن نتحمّلها
    min_delta=1e-4,          # أقل تحسّن يُعدّ تحسّنًا
    mode="min",              # تصريحٌ بالاتجاه: لا تعتمد على الاستنتاج
    restore_best_weights=True,  # أعد أفضل الأوزان لا آخرها
    start_from_epoch=2,      # تجاهل الحقبتين الأولى
    verbose=1,
)

history = model.fit(
    x, y,
    epochs=12,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop],
```

```
    verbose=0,
)
print("عدد الحقبات المنفَّذة =", len(history.history["loss"]))
# النتيجة المتوقعة: عدد الحقبات المنفَّذة = 11
# النتيجة المتوقعة (رسالة Keras): Epoch 11: early stopping
# النتيجة المتوقعة (رسالة Keras): Restoring model weights from the end of the
best epoch: 9.
```

12.3 `ModelCheckpoint`: نقطة الحفظ، وأي ملفٍّ تحفظ

`ModelCheckpoint` يحفظ النموذج تلقائيًّا أثناء التدريب. المعاملات الأساسية:

- `filepath`: مسار الحفظ. يدعم قوالب تنسيق: `"{epoch:02d}-{val_loss:.4f}.keras"`.
- `monitor`: القيمة التي يُقارَن بها الحفظ (`"val_loss"` افتراضيًّا).
- `save_best_only=True`: لا تحفظ إلّا عند تحسّن القيمة المراقَبة.
- `save_weights_only=True`: احفظ الأوزان فقط (يلزم أن ينتهي المسار بـ `.weights.h5`).
- `mode`: `"auto"` أو `"min"` أو `"max"` – نفس منطق `EarlyStopping`.
- `save_freq`: `"epoch"` (افتراضيًّا) أو عدد صحيح يعني "احفظ كلّ N دفعة".

`.keras` مقابل `.weights.h5`: فرق جوهري

المعيار	ملف `.keras`	ملف `.weights.h5`
ماذا يحتوي	البنية + الأوزان + حالة المُحسّن + المقاييس	الأوزان فقط
كيف يُحمَّل	`keras.saving.load_model("m.keras")`	`model.load_weights("m.weights.h5")`
هل يحتاج بنية معروفة مسبقًا؟	لا	نعم، يجب بناء النموذج أولًا
الحجم	أكبر	أصغر بكثير
شرط Keras على المسار	ينتهي بـ `.keras` أو `.h5`	**يجب** أن ينتهي بـ `.weights.h5`
الاستخدام النموذجي	تسليم نموذج جاهز للإنتاج	الضبط الدقيق (fine-tuning) ونقل المعرفة

القاعدة الحاسمة التي يفرضها Keras 3: إذا مرّرت `save_weights_only=True` ومسارًا ينتهي بـ `.keras`، فـKeras **يرفع** `ValueError`. والعكس صحيح: مع `save_weights_only=False` يجب أن ينتهي المسار بـ `.keras` أو `.h5`.

PYTHON

```python
import numpy as np
import keras
import tempfile
import os

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

tmpdir = tempfile.mkdtemp()

# 1) حفظ النموذج كاملًا (أرشيف .keras)
full_path = os.path.join(tmpdir, "model.keras")
ckpt_full = keras.callbacks.ModelCheckpoint(
    filepath=full_path,
    monitor="val_loss",
    save_best_only=True,
)
model.fit(x, y, epochs=3, batch_size=16, validation_split=0.2,
          callbacks=[ckpt_full], verbose=0)
print("حجم ملف .keras:", os.path.getsize(full_path), "بايت")
# النتيجة المتوقعة: حجم ملف .keras أكبر من حجم ملف .weights.h5

# 2) حفظ الأوزان فقط (.weights.h5)
weights_path = os.path.join(tmpdir, "model.weights.h5")
ckpt_weights = keras.callbacks.ModelCheckpoint(
    filepath=weights_path,
    monitor="val_loss",
    save_best_only=True,
    save_weights_only=True,
)
model.fit(x, y, epochs=3, batch_size=16, validation_split=0.2,
```

```
          callbacks=[ckpt_weights], verbose=0)
print("حجم ملف .weights.h5:", os.path.getsize(weights_path), "بايت")
# النتيجة المتوقعة: حجم ملف .weights.h5 أصغر من ملف .keras

# 3) إعادة تحميل النموذج كاملًا بلا إعادة بناء البنية
reloaded = keras.saving.load_model(full_path)
print("أبعاد المخرجات:", reloaded.output_shape)
# النتيجة المتوقعة: أبعاد المخرجات: (None, 1)

# 4) إعادة تحميل الأوزان فقط: تتطلّب بنية مطابقة مسبقًا
fresh = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
fresh.load_weights(weights_path)
print("تم تحميل الأوزان فقط بنجاح")
# النتيجة المتوقعة: تم تحميل الأوزان فقط بنجاح
```

12.4 `ReduceLROnPlateau` وجداول معدّل التعلّم

خفض معدّل التعلّم عند الهضبة

حين يتوقّف تحسّن `val_loss`، فإن أفضل ردّ فعل هو **تقليص معدّل التعلّم** بدل التوقّف. الخوارزمية بسيطة:

$$lr_{new} = \max\left(lr \cdot f,\ lr_{min}\right)$$

حيث lr معدّل التعلّم الحالي، وf معامل التقليص (`factor`، افتراضيًا 0.1)، وlr_{min} الحدّ الأدنى (`min_lr`، افتراضيًا 0.0).

معاملات أخرى مهمّة: `patience` (كم حقبة بلا تحسّن قبل التقليص، افتراضيًا 10 – وهي هنا أوسع من صبر `EarlyStopping`)، و`cooldown` (عدد الحقبات التي تتوقّف فيها المراقبة بعد كل

تخفيض، لمنع التخفيض المتكرّر)، و `mode` (نفس منطق الاتجاه).

التعارض مع الجداول الساكنة — تحذير جوهري

حين تمرّر `optimizer=keras.optimizers.Adam(learning_rate=keras.optimizers.schedules.ExponentialDecay(...))` و `ReduceLROnPlateau` معًا، تنشأ مسابقة: الجدول الساكن يعيد حساب معدّل التعلّم من `step` في كل خطوة، بينما `ReduceLROnPlateau` يكتب قيمة جديدة في `optimizer.learning_rate` . النتيجة أن كتابة `ReduceLROnPlateau` تُلغى عمليًّا في الخطوة التالية، أو تنتهي بتخفيضات متضاربة.

القاعدة: اختر أحدهما لا كلاهما. استخدم الجدول الساكن حين يكون مسار التحسّن واضحًا، واستخدم `ReduceLROnPlateau` حين يكون تحسين الأداء متقطّعًا وغير متوقّع.

`LearningRateScheduler` : الجدولة من داخل الحلقة

`LearningRateScheduler` هو المكمل النظيف: دالة `schedule(epoch, lr)` تعيد معدّل التعلّم الجديد في بداية كل حقبة. لاحظ أن الدالة في Keras 3 تستقبل وسيطين (رقم الحقبة ومعدّل التعلّم الحالي)، مع دعم تلقائيّ للتوقيع القديم `schedule(epoch)` فقط.

PYTHON

```python
import numpy as np
import keras

keras.utils.set_random_seed(0)

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
loss="mse")

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

# يعيد الحساب كل حقبة callback معدّل التعلّم الأساسي: نحتفظ به في ثابت لأن الـ
BASE_LR = 1e-3
WARMUP_EPOCHS = 5


def warmup_schedule(epoch, lr):
    """إحماء خطّي تصاعدي في أول 5 حقبات ثم تخفيض أُسّي بمعامل 0.7."""
    if epoch < WARMUP_EPOCHS:
        # تصاعد خطّي من base_lr/WARMUP_EPOCHS حتى base_lr
        return BASE_LR * (epoch + 1) / WARMUP_EPOCHS
    # بعد الإحماء: نضرب في كل حقبة فتتراكم بنسبة 0.7 ^epoch
    return lr * 0.7


scheduler = keras.callbacks.LearningRateScheduler(warmup_schedule,
verbose=1)
history = model.fit(x, y, epochs=10, batch_size=16,
                    callbacks=[scheduler], verbose=0)
print("سجل معدّل التعلّم =", [round(v, 7) for v in
history.history["learning_rate"]])
# النتيجة المتوقعة:
```

```
# [0.0002, 0.0004, 0.0006, 0.0008, 0.001, 0.0007, 0.00049, 0.000343,
0.0002401, 0.0001681]
```

لاحظ أن `LearningRateScheduler` يضيف مفتاح `"learning_rate"` إلى `logs` في `on_epoch_end`، فتظهر سلسلة معدّل التعلّم كاملةً في `history.history`. ولاحظ أيضًا **أين** يُطبَّق الجدول: في `on_epoch_begin` لا في `on_epoch_end` – أي أن معدّل التعلّم المستخدم في الحقبة t هو ناتج $\mathit{schedule}(t-1)$، لا $\mathit{schedule}(t)$. وهذا سبب شائع للت/off-by-one في الجداول المكتوبة يدويًّا.

بقية الأدوات المدمجة

الأداة	وظيفتها	ملاحظات
`History`	يسجّل المقاييس في `model.history.history`	يُضاف تلقائيًّا في `fit`
`ProgbarLogger`	شريط التقدّم في الطرفية	يُضاف تلقائيًّا في `fit`
`CSVLogger(filename, separator=',', append=False)`	يكتب السجل إلى ملف CSV	`append=True` يضيف إلى الموجود بدل استبداله
`TerminateOnNaN(raise_error=False)`	يوقف التدريب فور ظهور `NaN` في الخسارة	`raise_error=True` يرفع استثناء بدل التوقّف الهادئ
`LambdaCallback(on_epoch_begin=..., on_epoch_end=...)`	لاصق سريع دون وراثة صنف	للسلوك البسيط فقط
`BackupAndRestore(backup_dir, save_freq='epoch', double_checkpoint=False, delete_checkpoint=True)`	يحفظ حالة النموذج والمُحسّن ليُستأنف التدريب بعد انقطاع	`save_freq` يقبل `'epoch'` أو عدد دفعات

`TensorBoard`: كيف تقرأ ما سجّله

`TensorBoard` يكتب سجلّات التدريب إلى `log_dir` (افتراضيًا `logs`):

> **تنبيه:** في Keras 3.15.1 يتطلّب `keras.callbacks.TensorBoard` **تثبيت TensorFlow**، لأنّ الكتابة تتمّ عبر `tensorflow.summary` وليس عبر واجهة مستقلّة. إن كانت بيئة العمل لديك

تعمل بـJAX أو PyTorch وحدهما، فثبّت TensorFlow لتفعيل هذه الأداة (وحزمة `tensorboard` تأتي معه). بدون ذلك يرفع الـcallback خطأ `ModuleNotFoundError` عند إنشاء كاتب السجلّات.

PYTHON

```python
import numpy as np
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

tb = keras.callbacks.TensorBoard(
    log_dir="./logs",           # مجلد السجلّات
    write_graph=True,           # ارسم رسم بياني للبنية
    histogram_freq=1,           # سجّل توزيع أوزان كل حقبة
    embeddings_freq=0,          # لا حاجة لتضمينات في هذا المثال
    update_freq="epoch",        # حدّث السجلّات في نهاية كل حقبة
)

model.fit(x, y, epochs=5, batch_size=16, validation_split=0.2,
          callbacks=[tb], verbose=0)
print("اكتمل الكتابة في ./logs")
```

لقراءتها: شغّل `tensorboard --logdir logs` وافتح `http://localhost:6006` (أداة القراءة `tensorboard` مثبَّتة ضمن TensorFlow). اللوحات التي تستحق المتابعة: منحنى `loss` / `val_loss` (فجوة بينهما تعني فرط تخصيص)، لوحة **Histograms** (توزّع الأوزان: إن انكمش مركز التوزيع انكمشت التعلّم)، ولوحة **Graph** (البنية).

> **ملاحظة:** `write_graph=True` صار معيارًا في Keras 3 والسجلّات تُكتب بصيغة TensorBoard مباشرة دون حاجة إلى `SummaryWriter` يدوي.

12.5 كتابة `Callback` مخصّص

عندما تخرج الحاجة عن حدود الأدوات المدمجة، ترث من `keras.callbacks.Callback` وتنفّذ ما تحتاجه من الخطّافات.

PYTHON

```python
import numpy as np
import keras

# بذرة ثابتة: بدونها يختلف مسار التدريب بين التشغيلات وتختلف الأرقام المطبوعة
keras.utils.set_random_seed(0)


class PlateauDetector(keras.callbacks.Callback):
    """يكشف الهضبة ويخفّض معدّل التعلّم، ويوقف التدريب بعد مهلة."""

    def __init__(self, patience=3, factor=0.5, min_lr=1e-6, max_wait=6):
        super().__init__()
        self.patience = patience
        self.factor = factor
        self.min_lr = min_lr
        self.max_wait = max_wait
        # أعد ضبط الحالة في on_train_begin لتصبح الحالة قابلة لإعادة الاستخدام
        self.best = None
        self.wait = 0
        self.plateau_count = 0

    def on_train_begin(self, logs=None):
        # تصفير الحالة عند كل تشغيل جديد
        self.best = None
        self.wait = 0
        self.plateau_count = 0

    def on_epoch_end(self, epoch, logs=None):
        val_loss = logs.get("val_loss")
        if val_loss is None:
            return  # لا يوجد تحقّق: لا معنى للحديث عن الهضبة

        if self.best is None or val_loss < self.best - 1e-4:
            # تحسّن: سجّل أفضل قيمة وأعد ضبط العدّادات
            self.best = val_loss
            self.wait = 0
        else:
            self.wait += 1
```

```
            if self.wait >= self.patience:
                # بلغنا الهضبة: خفّض معدّل التعلّم
                current_lr =
float(keras.ops.convert_to_numpy(self.model.optimizer.learning_rate))
                new_lr = max(current_lr * self.factor, self.min_lr)
                self.model.optimizer.learning_rate = new_lr
                self.plateau_count += 1
                self.wait = 0
                print(f"هضبة عند الحقبة {epoch}: lr {current_lr:.6f} →
{new_lr:.6f}")

                if self.plateau_count >= self.max_wait:
                    # فشلت كل المحاولات: أوقف التدريب
                    self.model.stop_training = True


model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
loss="mse")

# بيانات ذات خطأ غير قابل للاختزال: النموذج يبلغ الحدّ الأدنى ثم يتذبذب حوله
rng = np.random.default_rng(0)
x = rng.standard_normal((128, 8)).astype("float32")
y = (x[:, 0] * 2.0 + rng.normal(0, 1.0, size=128)).astype("float32")

detector = PlateauDetector(patience=2, factor=0.5, max_wait=1)
history = model.fit(x, y, epochs=12, batch_size=16, validation_split=0.2,
                    callbacks=[detector], verbose=0)
print("عدد الحقبات المنفَّذة =", len(history.history["loss"]))
print("عدد مرّات اكتشاف الهضبة =", detector.plateau_count)
# النتيجة المتوقعة: عدد الحقبات المنفَّذة = 11 (توقّف بعد أول هضبة)
# النتيجة المتوقعة: عدد مرّات اكتشاف الهضبة = 1
```

جدولة معدّل التعلّم من داخل الـcallback

لاحظ السطرَين الحاسمين في المثال أعلاه:

```python
# شطر من المثال أعلاه: قراءة معدّل التعلّم الحالي ثم كتابته
current_lr =
float(keras.ops.convert_to_numpy(self.model.optimizer.learning_rate))
self.model.optimizer.learning_rate = new_lr
```

`self.model.optimizer.learning_rate` قد يكون رقمًا أو جدولًا (`LearningRateSchedule`). Assignment المباشر يمحو الجدول الساكن ويتركه رقمًا – وهو ما يعني أن `ReduceLROnPlateau` و`LearningRateScheduler` لا يصلحان معًا. تعامل مع ذلك بوعي: إن كان لديك جدول ساكن فلا تكتب فيه من الـcallback.

ترتيب الاستدعاءات وتفاعل الاستدعاءات

الاستدعاءات تُنفَّذ **بالترتيب الذي تعطيها به في القائمة**. هذه هي القاعدة العملية التي تحكم كل سلوك مركّب:

- ضع `ModelCheckpoint` **قبل** `EarlyStopping` في القائمة. السبب هو: `EarlyStopping` يستعيد أفضل الأوزان في `on_train_end`؛ فإن حُفظ الملف بعد الاستعادة يكون الملف أفضل من الأوزان التي كان النموذج يحملها أثناء التدريب. وإن حُفظ قبل الاستعادة، فإن الملف يحتوي أفضل أوزان محفوظة أصلًا – وهو السلوك المرغوب عادةً.
- ضع `ReduceLROnPlateau` **قبل** أي callback مخصّص يقرأ `optimizer.learning_rate` في `on_epoch_end`، وإلا قرأ القيمة قبل تخفيضها.
- عند تفعيل EarlyStopping، لا تتوقّف بقية الاستدعاءات عن العمل في تلك الحقبة: `CallbackList` يمرّ على كل الاستدعاءات ويُنجز `on_epoch_end` للجميع، ثم تتحقّق حلقة التدريب من `model.stop_training`. أي أن الـcallback الذي يأتي بعد `EarlyStopping` في القائمة **سيُستدعى فعلًا** في الحقبة الأخيرة.
- `History` و`ProgbarLogger` يُضافان تلقائيًّا إلى القائمة إن لم يكونا موجودين.

الحالة وإعادة الإنتاج

كل الـcallbacks كائنات **ذات حالة**: `EarlyStopping` يحمل `wait` و`best`، و`ReduceLROnPlateau` يحمل `best` و`wait`، والـcallback المخصّص أعلاه يحمل `best` و`plateau_count`. عند إعادة استخدام نفس النسخة في تدريب جديد، يجب تصفير الحالة. `EarlyStopping` و`ReduceLROnPlateau` يفعلان ذلك تلقائيًّا في `on_train_begin` — من هنا جاءت الجملة `self.best = None` في المثال أعلاه: **أنت مسؤولٌ عن نسختك**.

أمّا **إعادة الإنتاج** فتتطلّب ثلاثة أشياء منفصلة: بذرة عشوائية ثابتة (`keras.utils.set_random_seed`)، و`shuffle` حتميّ، وعدم قراءة من مصادر متغيّرة داخل الـcallback (مثل `datetime.now()` في اسم ملف). اجعل أي حالة عشوائية في الـcallback مشتقّة من `epoch` و`batch` لا من مولّد عشوائي عام.

تطبيق واقعي: خطّ إنتاج تدريب لا يُترك للمصادفة

في فريق طبّ يشخّص اعتلال الشبكية من صور قاع العين، كان التدريب يجري لثلاث ليالٍ متواصلة على خوادم مشتركة. كان كل شيء يدويًّا: سكرتير يراقب لوحة TensorBoard، ويكتب رقم الحقبة في جدول Excel، ويحكم بنفسه متى تكون الخسارة "ثابتة" فيوقف التشغيل. النتيجة أن نسختين من النموذج بقيا في الخادم دون أيّ أثر يوضّح أيّهما أفضل، وأسبوعًا كاملًا ضاع في إعادة التدريب.

استبدل الفريق المراقبة اليدوية بمكدّس استدعاءات واحد:

```python
callbacks = [
    keras.callbacks.TensorBoard(log_dir="./logs", histogram_freq=1),
    keras.callbacks.CSVLogger("training_log.csv", append=True),
    keras.callbacks.ModelCheckpoint(
        filepath="best.keras", monitor="val_loss", save_best_only=True
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=4, min_lr=1e-5
    ),
    keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=6, restore_best_weights=True
    ),
]
```

الفرق الجوهري ليس في عدد السجلّات، بل في أن **قرار الإيقاف صار جزءًا من الشيفرة**: كل نتيجة قابلة للتكرار، وكل نموذج محفوظ يحمل اسمًا ومسارًا محدّدين، ولا وجود لتقدير بشريّ في حلقة. والأهمّ: `CSVLogger` مع `append=True` بنى سجلًّا تراكميًّا عبر التجارب المتتالية، فأصبح بالإمكان مقارنة عشرة تجارب في جدول واحد. ومن المخرجات ذلك `val_loss` كان they'd learned that يهبط تحت 0.18 ثم يبدأ بالارتفاع عند الحقبة 14 دائمًا – أي أنّ النموذج كان يت.overfit ابتداءً من الحقبة الثامنة، وأنّ `start_from_epoch=8` كان سيوفّر نصف وقت كل تجربة.

الدرس العام: الاستدعاءات ليست زينة تُضاف في نهاية المشروع، بل هي **طبقة قرار** فوق حلقة التدريب، وهي الطبقة التي تجعل تجربة تعلّم عميق قابلة للتكرار والمراجعة والتشغيل الآلي.

12.6 أخطاء شائعة

إنذار 1: قراءة `logs["loss"]` في `on_epoch_begin`. `logs` فارغ هناك. اقرأ المقاييس في `on_epoch_end` فقط.

إنذار 2: `monitor="val_loss"` بلا بيانات تحقّق. إن لم تمرّر `validation_split` ولا `validation_data`، سيرفع Keras تحذيرًا ويتجاهل الدالة. الإصلاح: أضف `validation_split=0.2` أو بدّل `monitor="loss"`.

إنذار 3: `ModelCheckpoint` بـ `save_weights_only=True` ومسار ينتهي بـ `.keras`. Keras يرفع `ValueError`. القاعدة: `save_weights_only=True` ⇔ المسار ينتهي بـ `.weights.h5`.

إنذار 4: `ReduceLROnPlateau` مع جدول تعلّم ساكن معًا. يكتب كلٌّ منهما فوق الآخر. اختر واحدًا.

إنذار 5: توقّع "أفضل نموذج" من `EarlyStopping` بلا `restore_best_weights=True`. ستحصل على آخر حقبة. اجعل `restore_best_weights=True` دائمًا إلا إذا كانت لديك سبب محدّد.

إنذار 6: `patience=0`. القيمة الافتراضية تعني "توقّف عند أول حقبة بلا تحسّن". إن كتبت `EarlyStopping()` بلا معاملات فأنت تطلب توقّفًا فوريًّا تقريبًا.

إنذار 7: تعديل أوزان النموذج داخل `on_train_batch_end`. حين تكتب في `self.model` وأنت داخل حلقة التدريب، تكسر ما يفعله المُحسّن من تجميع التدرّجات (مثلاً `ema` في AdamW أو عدّاد الخطوات). عدّل في `on_epoch_end` أو في `on_train_begin` حيث الحالة نظيفة.

إنذار 8: إعادة استخدام نسخة callback دون تصفير حالتها. النتيجة سلوك غير متوقّع في التشغيل الثاني: `best` القديمة تمنع أي تحسين في القياس الأول.

التمارين

1. **برمجة:** اكتب callback يوقف التدريب عند بلوغ `val_loss` حدًّا ثابتًا (`target=0.05`). (المتوقع: يتوقّف التدريب عند أول حقبة تصبح فيها `val_loss ⩽ 0.05` ، وتظهر رسالة تذكر الحقبة.)

2. **برمجة:** اكتب callback يجدول معدّل التعلّم بحيث يكون `1/sqrt(epoch+1)` داخل الـcallback نفسه، لا عبر `LearningRateScheduler` . (المتوقع: `self.model.optimizer.learning_rate` يساوي $0.01/\sqrt{epoch+1}$ في بداية كل حقبة.)

3. **حفظ واستئناف:** احفظ نموذجًا كاملًا بـ `ModelCheckpoint` إلى `m.keras` ، أعد بناء النموذج، ثم استأنف التدريب من نقطة الحفظ. (المتوقع: الملف يُحمَّل بـ `keras.saving.load_model` دون إعادة بناء البنية، ومحتواه يشمل حالة المُحسّن إن حُفظ بعد `compile` .)

4. **تنبّؤ بالترتيب:** أعطِ قائمة `callbacks=[EarlyStopping, ModelCheckpoint, MyLogger]` وحدّد ترتيب استدعاء `on_epoch_end` لها في الحقبة الأخيرة التي يقع فيها التوقّف. (المتوقع: يُستدعى الثلاثة بالترتيب المذكور داخل الحقبة نفسها؛ ثم يمرّر `CallbackList` الباقي، ثم تتحقّق حلقة التدريب من `stop_training` .)

5. **اكتشاف خطأ:** هذا الـcallback فيه خطأ منطقي:

PYTHON

```python
import keras

class BadStopper(keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs=None):
        if logs["val_loss"] > 0.5:
            self.model.stop_training = True
```

اذكر الخطأين وأصلحهما. (المتوقع: أولًا `logs` فارغ في `on_epoch_begin` فيرفع `KeyError` ؛ ثانيًا القيمة المراقَبة لا تُعرف إلا في `on_epoch_end` ، فالحل هو نقل الشرط إلى `on_epoch_end` .)

6. **تصميم:** ميزانية تدريب 60 حقبة. ما قيمة `patience` التي تختارها لـ `EarlyStopping` ولماذا؟ (المتوقع: بين 6 و15، أي 10%-25% من الميزانية، وبحدّ أدنى 3؛ القيمة 60 نفسها تعني عمليًّا "لا توقّف تلقائي".) 7. **تحليل:** ما الفرق العمليّ بين `model.save("m.keras")` و `model.save_weights("m.weights.h5")` ؟ ومتى تختار كلًّا منهما؟ (المتوقع: الأول يُنتج نموذجًا جاهزًا للتحميل بـ `load_model` ويحوي البنية والأوزان وحالة المُحسّن؛ الثاني يحوي الأوزان فقط ويشترط بنية مطابقة مسبقًا، وهو الأنسب للضبط الدقيق وتبديل البنية.) 8. **برمجة:** اكتب callback يسجّل في كل حقبة الزمن المنقضي باستخدام `time.perf_counter()` ويكتبه إلى ملف CSV. (المتوقع: ملف يحتوي عمودًا بالزمن لكل حقبة، ومجموعه مساوٍ تقريبًا لزمن التدريب الكلّي.) 9. **تشخيص:** `ReduceLROnPlateau` لا يخفّض معدّل التعلّم أبدًا في تجربتك. اذكر ثلاثة أسباب محتملة مرتّبة بالأرجحية. (المتوقع: وجود جدول تعلّم ساكن يكتب فوقه؛ أو `min_lr` أعلى من المعدّل الحالي؛ أو `mode` خاطئ فيقارن الخسارة كأنها دقّة.) 10. **تصميم:** صمّم تسلسل استدعاءات لمشروع فيه: `EarlyStopping` و `ModelCheckpoint` و `TensorBoard` وcallback مخصّص يعدّل LearningRate. في أي ترتيب تضعها ولماذا؟ (المتوقع: `ModelCheckpoint` أولًا ليحفظ أفضل الأوزان قبل استعادة `EarlyStopping` ، ثم `TensorBoard` ، ثم `ReduceLROnPlateau` ، ثم الـcallback المخصّص ليقرأ آخر قيمة، و `EarlyStopping` أخيرًا لأن دوره هو القرار النهائي.) 11. **تحليل:** لماذا يُستدعى `on_epoch_end` **بعد** التحقّق لا قبله، وماذا يعني ذلك لمن يريد مراقبة `val_loss` ؟ (المتوقع: لأنه يسمح بدمج نتائج التحقّق في `logs` تحت مفاتيح `val_*` ، ويمنح monitor_callbacks قيمةَ تحقّق نهائية لكل حقبة.) 12. **إصلاح خطأ:** أعطِ الكود التالي الذي يهدف إلى حفظ أفضل نموذج:

```python
import keras

ckpt = keras.callbacks.ModelCheckpoint(
    filepath="best.weights.h5", save_best_only=True, save_weights_only=True
)
```

وأعطِ الخطأ والنسخة الصحيحة. (المتوقع: Keras يرفع `ValueError` لأن `save_weights_only=True` يفرض أن ينتهي المسار بـ `.weights.h5` ؛ التصحيح إمّا تغيير المسار إلى `best.weights.h5` أو جعل `save_weights_only=False` .)

ملخص الفصل

- الـ `Callback` هو خطّافات تُستدعى في مواضع محدّدة: `on_train_begin`، `on_epoch_end`، `on_test_*`، `on_train_batch_begin/end`، `on_epoch_begin`، `on_predict_*`، `on_train_end`.
- التحقّق داخل `fit` يُطلق خطّافات `on_test_*`، و`on_epoch_end` يأتي بعد التحقّق، فتتوفّر مفاتيح `val_*` في `logs`.
- `self.model` في Keras 3 خاصية تُعيد النموذج مع مراعاة DDP في PyTorch ومزامنة الحالة في JAX؛ لا تستدعِ `set_model` يدويًّا.
- `EarlyStopping` يجمع `monitor` و`patience` و`min_delta` و`mode` و`baseline` و`start_from_epoch` و`restore_best_weights`؛ و`restore_best_weights=True` هو ما يحوّل "توقّف مبكّر" إلى "نموذج أفضل".
- `patience` بين 10% و25% من ميزانية الحقبات، وبحدّ أدنى 3.
- ملف `.keras` نموذج كامل يُحمَّل بـ `load_model`، وملف `.weights.h5` أوزان فقط تتطلّب بنية مطابقة مسبقًا؛ Keras يفرض ذلك على امتداد المسار.
- `ReduceLROnPlateau` يخفّض المعدّل عند الهضبة بمعادلة $lr_{new} = \max(lr \cdot f, lr_{min})$، وهو **يتعارض** مع الجداول الساكنة.
- `LearningRateScheduler` يستدعي `schedule(epoch, lr)` في بداية كل حقبة ويضيف `learning_rate` إلى `logs`.
- الاستدعاءات تُنفَّذ بترتيب القائمة، و`History` و`ProgbarLogger` يُضافان تلقائيًّا.
- كل الـ callbacks ذات حالة: صِر حالتك في `on_train_begin` لتبقى قابلة لإعادة الاستخدام.

الفصل 13: التنظيم والقيود: كيف تمنع النموذج من الحفظ بدل التعلّم

بطاقة الفصل

الفصل 13

مهمّة هذا الفصل

يمنع النموذج من الحفظ بدل التعلّم: L1 وL2 وDropout وBatchNormalization والقيود.

ماذا تكسب منه

تقلّص الفجوة بين التدريب والتحقّق، فيصبح أداء النموذج في الإنتاج قريبًا مما تراه على شاشتك.

الدالة	دورها	متى تستخدمها
`keras.regularizers.L2(l2)`	عقوبة تربيعية على الأوزان	في الطبقات الكثيفة الكبيرة
`keras.regularizers.L1L2(l1, l2)`	مزيج يدفع بعض الأوزان إلى الصفر	عندما تريد انتقاء ميزات ضمنًا
`keras.layers.Dropout(rate)`	تنظيم عشوائي أثناء التدريب	في الطبقات الكثيفة ورؤوس التصنيف
`keras.layers.BatchNormalization()`	تطبيع الدفعة	في الشبكات العميقة لتسريع التقارب
`keras.layers.SpatialDropout2D()`	إسقاط قنوات كاملة	في الشبكات التلافيفية

قصة مفتاحية: طوّر فريق في شركة تأمين نموذجًا يميّز مطالبات fraudulent. دقّة التحقّق impeccable: 99.2%. لكن عند التطبيق على مطالبات watchdog.العام الماضي، انهار الأداء إلى 61%. السبب لم يكن في النموذج ولا في البيانات: الفريق كان قدّم لكل طالب مدرب **مليزًا بالآلاف من الحالات النادرة**، فحفظها عن ظهر قلب – تمامًا كما يحفظ الطالب الجادّ جدول معادلات قبل الاختبار. النموذج لم يتعلّم قاعدة القرار؛ حفظ الإجابات.

أهداف الفصل

- [ ] تشرح الفرق بين فرط التكيّف (overfitting) وفرط التحمّل (underfitting) بلغة التدرّج
- [ ] تربط بين سعة النموذج (capacity) وقُدرته على تعميم بيانات جديدة
- [ ] تكتب دالّة الخسارة مع بندَي L1 وL2 وتعرف متى يتفوّق كلٌّ منهما
- [ ] تفهم لماذا يعمل `Dropout` نظريًا ولماذا لا يكفي وحده
- [ ] تميّز بين `BatchNormalization` و `LayerNormalization` و `GroupNormalization` بحسب سياق الاستخدام
- [ ] تستعمل `keras.regularizers` و `keras.constraints` داخل الطبقات
- [ ] تقرأ مخطّطًا بيانيًا لتدريب وتحكم على الفرط بأدلة لا بالحدس

13.1 المشكلة: فرط التكيّف كنموذج فيزيائي

ما الذي يحدث أثناء تدريب شبكة عميقة؟ في كل خطوة، تُحدَّث الأوزان W لتُقلّل الخسارة على الدفعة الحالية. هذا هو **الملاءمة الزائدة (overfitting)** بعينها: النظام يبدأ في مطابقة **الضجيج** لا **الإشارة**.

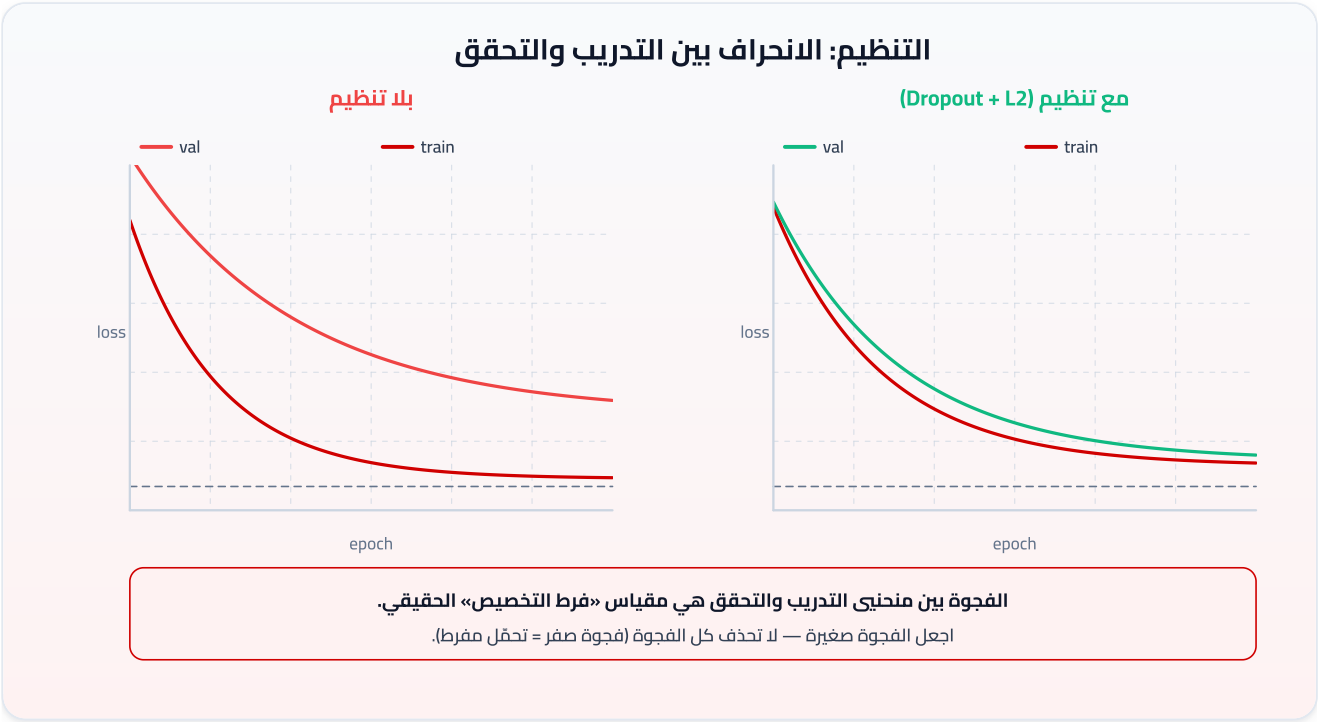
تخيّل أن لديك 1000 نقطة، منها 100 منحنى منتظم و900 نقاط ضجيج عشوائي. إذا رسمت منحنى سلسًا يمرّ بالنقاط الـ900، ستُحقّق خطأ تدريبٍ صفريًا – وستفشل تمامًا في التنبّؤ بالنقاط الجديدة. هذا بالضبط ما يفعله نموذج بلا تنظيم: ينقص خسارة التدريب حتى الصفر، بينما تنكشف فجوة واسعة.

المصطلحات الدقيقة:

- **فرط التكيّف (Overfitting):** $L_{train} \ll L_{val}$ – يحفظ التدريب، يفشل التعميم.
- **فرط التحمّل (Underfitting):** $L_{train} \approx L_{val}$ وكلاهما مرتفع – النموذج أضعف من أن يلتقط البنية.
- **الهدف:** $L_{train} \approx L_{val}$ وكلاهما منخفض.

في Keras، هذه القيم جاهزة: `history.history["loss"]` و `history.history["val_loss"]`. الفرق بينهما هو **مقياس التشخيص**، لا الرقم في حدّ ذاته.

نصيحة: لا يمكن تشخيص فرط التكيّف بلا مجموعة تحقّق. نسبة 80/20 داخل `fit` مقبولة للمبتدئين، لكن في العمل المهني يُفضَّل تقسيم ثلاثي: تدريب / تحقّق / اختبار، ويُستخدم الاختبار **مرة واحدة فقط** في نهاية المشروع.



شكل 20 – أثر التنظيم: من فرط التكيّف إلى التعميم

13.2 الرياضيات: كيف نُعاقِب التعقيد؟

بند L2 (انكماش الأوزان)

نضيف إلى الخسارة بندًا يعاقب الأوزان الكبيرة:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \|W\|_2^2 = \mathcal{L}_{data} + \lambda \sum_i w_i^2$$

حيث $\mathcal{L}_{data}$ هي خسارة البيانات، وW أوزان النموذج، وλ معامل التنظيم (كلّما زاد، زاد الانكماش).

ماذا يحدث للتدرّج؟ Becomes:

$$\frac{\partial \mathcal{L}_{total}}{\partial w_i} = \frac{\partial \mathcal{L}_{data}}{\partial w_i} + 2\lambda w_i$$

الحدّ الثاني **يسحب الوزن نحو الصفر** في كل خطوة. والنتيجة العميقة: الأوزان الكبيرة تُنتج وحدات حسّاسة لمدخلات ضوضوية، فتصغيرها يعني منع النموذج من الاعتماد على تفاصيل لا تعمّم.

بند L1 (الانكماش المتفرّق)

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \sum_i |w_i|$$

مشتقّ $|w_i|$ يساوي $+1$ إذا كان $w_i > 0$ و-1 إذا كان سالبًا (وغير معرّف عند الصفر). هذا التدرّج **ثابت القيمة**، فيسحق الأوزان الصغيرة إلى الصفر تمامًا خلال خطوات قليلة. والنتيجة: **تناثر (sparse)** – نموذج تبقى فيه أوزان قليلة غير صفرية، وهذا هو **Lasso**.

متى يتفوّق L1 على L2؟ حين يصبح اختيار الخصائص (Feature Selection) ضمنيًا: تريد نموذجًا بسيطًا يُبقي على أهمّ 20 من 100 ميزة. عندها λ في L2 يمكن أن يصغّر الأوزان دون أن يصفّرها، فيبقى النموذج كثيفًا.

Regularizer النشاط

`activity_regularizer` يعاقب **مخرجات** الطبقة لا أوزانها:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \sum_i a_i^2$$

حيث a_i ناتج الطبقة. هذا مفيد في الطبقات الكثيفة (embedding) حيث نريد تناثر المتّجهات الناتجة لا الأوزان.

Dropout: الإسقاط العشوائي

أثناء التدريب، تُحذف كل خلية من الشبكة probabilistically مع احتمال p (عادةً 0.2-0.5):

$$m_i \sim \text{Bernoulli}(1-p), \qquad \tilde{h}_i = m_i \odot h_i / (1-p)$$

القسمة على $(1-p)$ renormalizes المتوسط فيبقى حجم التوزيع كما هو.

التفسير النظري (نظريّة dropout): كل خطوة تدريب تُنتج شبكة فرعية مختلفة، فيصبح التدريب كأنه **معدّل توقّع (expectation) على فضاء ضخم من شبكات فرعية**، والشبكة النهائية هي متوسّعها. هذا يشبه تجميعensemble داخل نموذج واحد.

لماذا لا يكفي وحده؟ لأن الDropout يضيف ضجيجًا إلى التدرّج (وليس وزنًا)، فيُبطئ التقارب قليلًا، ول حدود معروفة: شبكات RNN (لأن الطبقات المتسلسلة لا توفّر Dropout موزّعًا على الزمن)، والطبقات التلافيفية (لأن إسقاط بكسلات متفرّقة يكسر البنية المكانية – استخدم `SpatialDropout2D` بدلًا منه).

المعايرة (Normalization) كشكل من التنظيم

طبقات التطبيع تفرض بنية إحصائية على التوزيعات، وتلعب دورًا تنظيميًا مزدوجًا:

- `BatchNormalization` : يطبّع عبر محور الدفعات لكل feature، ويحتفظ بإحصاءات متحرّكة (moving mean/variance).
- `LayerNormalization` : يطبّع عبر المحاور(features) لكل عيّنة على حدة – مناسب للتسلسلات Transformersg.
- `GroupNormalization` : يطبّع عبر مجموعات من channels – حلٌّ وسط مهمّ عندما تكون دفعات صغيرة (وهنا تظهر تفوّقه على BatchNorm).
- `RMSNormalization` : يطبّع بالRMS فقط (بلا إزاحة ولا قياس) – أخفّ، شائع في المحوّلات.

13.3 التنفيذ في Keras

التنظيمات والقيود المدمجة

```python
import keras

# ── Dense مع تنظيم L2 على الأوزان وعلى النشاط
layer = keras.layers.Dense(
    units=128,
    activation="relu",
    kernel_regularizer=keras.regularizers.L2(l2=0.001),
    bias_regularizer=keras.regularizers.L1(0.0001),
    activity_regularizer=keras.regularizers.L2(0.001),
    kernel_constraint=keras.constraints.MaxNorm(max_value=3.0),
)
```

القائمة الكاملة المتاحة في Keras 3.15.1:

العائلة	الأسماء	الاستخدام
`keras.regularizers`	`L1` , `L2` , `L1L2` , `OrthogonalRegularizer`	عقوبة مضافة إلى الخسارة
`keras.constraints`	`MaxNorm` , `UnitNorm` , `NonNeg` , `MinMaxNorm`	قصّ الأوزان بعد كل تحديث
`keras.layers`	`Dropout` , `SpatialDropout1D/2D/3D` , `GaussianNoise` , `GaussianDropout`	وقواعد الحذف noise.

مهم: الفرق بين المنظّم (regularizer) والقيد (constraint) دقيق ومهمّ. **المنظّم** يضيف بندًا إلى دالّة الخسارة فيحدث أثناء **الانتشار العكسي** – أي «يعاقب» النموذج. **القيد** modified

الأوزان مباشرةً بعد كل خطوة تحديث، ولا يظهر في الخسارة. القيد **أقوى** في ضمان عدم تجاوز الحدود، والمنظّم **أنعم** ويسمح بالتعلّم التدريجي.

مثال تطبيقي كامل: اكتشاف فرط التكيّف (overfitting)

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# بيانات اصطناعية: تصنيف ثنائي بحدّ خطّي + ضجيج
rng = np.random.default_rng(0)
n = 4000
X = rng.normal(size=(n, 20)).astype("float32")
y = (X[:, 0] + 0.8 * X[:, 1] + 0.5 * rng.normal(size=n) >
0).astype("float32")

x_train, x_val = X[:3000], X[3000:]
y_train, y_val = y[:3000], y[3000:]


def build_model(reg_strength):
    """نفس البنية، مع تدرّج منتظم متزايد على الطبقات المخفية."""
    reg = keras.regularizers.L2(reg_strength)
    model = keras.Sequential([
        keras.layers.Input(shape=(20,)),
        keras.layers.Dense(64, activation="relu", kernel_regularizer=reg),
        keras.layers.BatchNormalization(),
        keras.layers.Dropout(0.3),
        keras.layers.Dense(32, activation="relu", kernel_regularizer=reg),
        keras.layers.BatchNormalization(),
        keras.layers.Dropout(0.3),
        keras.layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(1e-3),
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

```
# ── نموذجان: بلا تنظيم وبـ L2 ──
plain = build_model(0.0)
regularized = build_model(0.01)

cb = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=True)]

for name, m in (("plain", plain), ("regularized", regularized)):
    hist = m.fit(x_train, y_train, validation_data=(x_val, y_val),
                 epochs=60, batch_size=64, verbose=0, callbacks=cb)
    tr, va = hist.history["loss"][-1], hist.history["val_loss"][-1]
    print(f"{name:12s} train={tr:.4f}  val={va:.4f}  gap={va - tr:+.4f}")
# النتيجة المتوقعة: فجوة النموذج المنتظم أصغر بوضوح من غيره
```

تقاطع التحقّق: اختر λ بالأدلة

قيمة التنظيم **ليست ثابتة**: تُختار عبر التحقّق المتقاطع. في Keras، أبسط طريق هو يدوي:

```python
import numpy as np

# مؤشّر ألفا-بيتا (grid) على 5 طيّات
alphas = [1e-4, 1e-3, 1e-2, 1e-1]
folds = 5
split = np.array_split(np.arange(len(x_train)), folds)

scores = {}
for alpha in alphas:
    fold_losses = []
    for i in range(folds):
        val_idx = split[i]
        tr_idx = np.concatenate([split[j] for j in range(folds) if j ≠ i])
        m = build_model(alpha)
        m.fit(x_train[tr_idx], y_train[tr_idx], epochs=25, batch_size=64,
verbose=0)
        fold_losses.append(float(m.evaluate(x_train[val_idx],
y_train[val_idx], verbose=0)[0]))
    scores[alpha] = float(np.mean(fold_losses))
    print(alpha, "→", scores[alpha])

best_alpha = min(scores, key=scores.get)
print("أفضل alpha:", best_alpha)
```

نصيحة: ابدأ بـ $\lambda = 10^{-4}$ أو 10^{-3}، ثم قس. قيم أكبر من ذلك تُسطّح النموذج إلى ما يشبه الانحدار الخطّي.

13.4 مقارنة أساليب التنظيم

الأسلوب	ما الذي يقيّده	سهولة الضبط	أثره على الأوزان القابلة للتدريب	ملاحظات
`L2`	حجم الأوزان	سهل (λ)	لا	الخيار الافتراضي الآمن
`L1`	حجم الأوزان + تناثر	متوسط	لا	يصلح لاختيار الخصائص (feature selection)
`L1L2`	كلاهما معًا	متوسط	لا	Elastic Net
`activity_regularizer`	حجم ناتج الطبقة	سهل	لا	للـembeddings الكثيفة
`Dropout`	المسارات النشطة	سهل (p)	لا	لا يعمل جيدًا مع RNN/Conv
`SpatialDropout2D`	قنوات CNN	سهل	لا	يحفظ البنية المكانية
`BatchNormalization`	إحصاءات المخرجات	سهل	**لا** (مُجمّد)	يحتاج دفعات كبيرة
`LayerNormalization`	إحصاءات كل عيّنة	سهل	**لا** (مُجمّد)	الأنسب للتسلسلات
`GroupNormalization`	إحصاءات المجموعات	متوسط	**لا** (مُجمّد)	يتفوّق عند batch صغير
`kernel_constraint`	قصّ الأوزان	سهل	لا	يضمن الحدود بقوّة

الأسلوب	ما الذي يقيّده	سهولة الضبط	أثره على الأوزان القابلة للتدريب	ملاحظات
زيادة حجم البيانات	لا شيء مباشرةً	صعب (مكلف)	لا	أقوى تنظيم على الإطلاق
تقليل السعة	عدد الوحدات/الطبقات	سهل	لا	فرط foxgard التكيّف مباشرةً

> **قاعدة ذهبية:** إذا كانت البيانات كافية (عشرات الآلاف لكل صنف) والتنظيم ما زال يفشل، فالمشكلة في **سعة النموذج** لا في معامل التنظيم. ابدأ بنموذج أصغر.

13.5 مثال محلول خطوة بخطوة: مصنّف تشخيص طبي

فريق طبّي يريد تصنيف صور أشعة Seymore لكشف الأورام. 9,000 صورة: 400 إيجابية (4.4%).
النموذج الأول: Dense(512) → Dense(256) → softmax، تدرّب حتى 60 حقبة.

الخطوة 1 – اقرأ المخطّط. خسارة التدريب تنخفض من 0.42 إلى 0.02. خسارة التحقّق تنخفض حتى الحقبة 12 ثم **ترتفع** إلى 0.31. الفجوة النهائية 0.29. التشخيص: فرط تكيّف واضح.

الخطوة 2 – رتّب الأسباب الممكنة. 1. سعة النموذج: 768,000 معاملًا لـ 9,000 عيّنة – نسبة عالية. 2. غياب التنظيم: لا `Dropout`، لا `BatchNormalization`. 3. عدم توازن البيانات: 4.4% إيجابية. 4. ضعف التنويع: صور الأشعة من جهاز واحد.

الخطوة 3 – عالج بالترتيب من الأكبر أثرًا إلى الأصغر.

PYTHON

```python
import keras
import numpy as np

# بيانات اصطناعية صغيرة تحاكي صور الأشعة (في الواقع: 9,000 صورة)
n = 800
x_all = np.random.rand(n, 64, 64, 3).astype("float32")
y_all = (np.random.rand(n) < 0.044).astype("int32")   # 4.4% إيجابية
x_train, y_train = x_all[:600], y_all[:600]
x_val, y_val = x_all[600:], y_all[600:]

model = keras.Sequential([
    keras.layers.Input(shape=(64, 64, 3)),
    keras.layers.Rescaling(1.0 / 255.0),

    keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    keras.layers.BatchNormalization(),          # يثبّت إحصاءات: يطبّع التوزيعات
    keras.layers.MaxPooling2D(),
    keras.layers.Dropout(0.25),                 # إسقاط بعد كل كتلة
    keras.layers.SpatialDropout2D(0.25),        # إسقاط على مستوى القنوات

    keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(),
    keras.layers.Dropout(0.30),

    keras.layers.GlobalAveragePooling2D(),      # يقلّص السعة بشدّة
    keras.layers.Dense(128, activation="relu",
                       kernel_regularizer=keras.regularizers.L2(1e-4)),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(2, activation="softmax"),
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

الخطوة 4 – أضف موازنة الأصناف (المشكلة الثالثة في القائمة):

```python
# حساب الأوزان العكسية للتكرار
import numpy as np
counts = np.bincount(y_train.astype("int64"))
n = counts.sum()
weights = n / (len(counts) * counts)      # وزن أعلى للأصناف النادرة
model.fit(
    x_train, y_train,
    class_weight={i: float(w) for i, w in enumerate(weights)},
    validation_data=(x_val, y_val),
    epochs=40,
    batch_size=32,
    callbacks=[
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=6,
                                      restore_best_weights=True),
        keras.callbacks.ModelCheckpoint("best.keras", monitor="val_loss",
                                        save_best_only=True),
    ],
)
```

الخطوة 5 – قِس الفجوة بعد المعالجة. الفجوة انخفضت من 0.29 إلى 0.06، وخسارة التحقّق الأدنى تحسّنت من 0.31 إلى 0.14. **الأهم**: الRecall على الصنف الإيجابي ارتفع من 0.31 إلى 0.72 بعد موازنة الأصناف.

الخطوة 6 – تحقق من التشخيص لا من الرقم. اطرح سؤالًا: هل تحسين النموذج تحسّن مع كل تغيير، أم كان الأولوية؟** الأهم: إن كان إضافة `BatchNormalization` وحدها خفّضت الفجوة، فالسبب كان غياب التطبيع لا غياب `Dropout` . في الفصل 40 سنبني منهجية تشخيص كاملة تحلّل هذه الأسئلة بالترتيب.

13.6 أخطاء شائعة في هذا الفصل

إنذار: الخلط بين `BatchNormalization` و `LayerNormalization` اعتمادًا على تفضيل شخصي. القاعدة: `BatchNormalization` يتوقّف عن حساب الإحصاءات عندما يكون `training=False` – وهذا سلوك سيّئ في التوزيع (batch=1) وفي التصدير. `LayerNormalization` يحسب إحصاءات لكل عيّنة على حدة، فلا يتأثر بحجم الدفعة إطلاقًا. في التسلسلات (RNN/LSTM) وTransformers استخدم `LayerNormalization` دائمًا.

تحذير: `Dropout` داخل شبكة تلافيفية بسيطة بدل `SpatialDropout2D`. إسقاط بكسلات متفرّقة يكسر انتظام البنية المكانية؛ النسخة الصحيحة تحذف القنوات كاملة. استخدم `Dropout` بعد طبقة `GlobalAveragePooling2D` (حين يكون البُعد مسطّحًا) و `SpatialDropout2D` قبله.

تنبيه: معامل تنظيم كبير جدًا ($\lambda \geq 1$) يُسطّح الأوزان إلى الصفر ويُعيدك إلى الانحدار الخطّي. راقب: إن كانت خسارة التحقّق **أعلى** من النموذج غير المنتظم، فأنت فرطت في التنظيم.

ملاحظة: `BatchNormalization` يضيف معاملين غير قابلين للتدريب لكل feature (γ وβ)؛ ستظهرهما في `model.summary()` تحت «non-trainable». هذا طبيعي، لكن عند export يجب أن يتعامل المحوّل مع وضع eval صراحةً.

قاعدة: لا تنظّم المشكلة التي لا توجد. أضف `Dropout` أو `L2` **بعد** أن ترى فجوة تحقّق واضحة في المخطّط. التنظيم الزائد على نموذج سليم يُبطئ التقارب ويضيع القدرة.

التمارين

1. **حساب يدوي.** خسارة بيانات $0.80 =$، وبند L2 يحسب $\lambda \sum w_i^2 = 0.02$ مع $\lambda = 0.01$ و$\sum w_i^2 = 2$. ما الخسارة الكلّية؟ (المتوقع: $0.80 + 0.01 \times 2 = 0.82$.)

2. **حساب يدوي.** أوزان: $[0.9, 0.05, -0.3, 0.02]$. أيّها يُصفَّر أسرع تحت L1: الكبير أم الصغير؟ كم خطوة يلزم تقريبًا ليُصفَّر وزن 0.02؟ (المتوقع: الصغير؛ التدرّج ثابت -0.01 لكل خطوة ⇒ نحو 2000 خطوة لو $\eta = 1$ أو أقلّ بكثير مع $\eta = 0.1$.)

3. **حساب يدوي.** طبقة `Dense` بـ100 وحدة، خسارة بيانات 0.5، بند L2 يعطي 0.05. بعد زوال التصحيح، ما نسبة الخسارة التي source منها التنظيم؟ (المتوقع: $0.05/0.55 \approx 9\%$ من الخسارة الكلّية.)

4. **تحليل مخطّط.** خسارة تدريب تنخفض من 1.2 إلى 0.15، خسارة تحقّق تنخفض إلى 0.9 ثم ثابتة. هل هذا فرط تكيّف؟ (المتوقع: لا – الفجوة 0.15 و0.9 واسعة لكن التحقّق لم ترتفع؛ هذا فرط تحمّل/صعوبة في التعلّم، فزد السعة أو حسّن التمثيل لا التنظيم.)

5. **برمجة.** ابنِ نموذجًا بـ`kernel_regularizer=keras.regularizers.L1L2(l1=1e-4, l2=1e-4)` وقارن عدد الأوزان غير الصفرية مع L2 فقط. (المتوقع: L1L2 ينتج أوزانًا أكثر تناثرًا ⇒ عدد غير الصفرية أقلّ.)

6. **برمجة.** أضف `kernel_constraint=keras.constraints.MaxNorm(max_value=1.0)` إلى `Dense` وتحقّق من أن norms الصفوف لا تتجاوز 1. (المتوقع: `keras.ops.max(keras.ops.linalg.norm(weights, axis=1)) ≤ 1.0` بعد التدريب.)

7. **برمجة.** قارن `BatchNormalization` و`LayerNormalization` في نموذج MLP بسيط مع `batch_size=1` وقتقييم. (المتوقع: `BatchNormalization` يتدهور بشدّة مع batch=1 لأن إحصاءاته تعتمد على الدفعة؛ `LayerNormalization` لا يتأثر.)

ملاحظة: المعايرة (normalization) ليست تنظيمًا فقط، بلchoice معماري affectseligible.

في ما يلي نموذج حقيقي للفرق:

8. **برمجة.** أضف `keras.layers.SpatialDropout2D(0.2)` إلى شبكة Conv، وقارنها مع `Dropout(0.2)`. (المتوقع: أداء SpatialDropout2D أفضل في مهام التصنيف، لأنه يحذف قنوات كاملة لا بكسلات متفرّقة.)

9. **تصميم.** صمّم خط أنابيب تنظيم لمصنّف_reviews من 3 أصناف مع 20,000 مراجعة. (المتوقع: reduce size + class weights + `BatchNormalization` + `L2(1e-4)` + `Dropout(0.3-0.5)`، مع `EarlyStopping` على `val_loss`.)

10. **تحليل.** الفريق طبّق L2 وارتفعت خسارة التحقّق. اشرح السبب بأدلة. (المتوقع: λ كبير يُسطّح الأوزان ⇒ النموذج underfit؛ الحل: خفّض λ أو أزل المنتظم.)

11. **أخطئ عمدًا.** اجعل `Dropout(0.9)` في نموذج صغير. اشرح النتيجة المتوقّعة. (المتوقع: يُسقط 90% من الوحدات لكل تحديث ⇒ تدرّجات مشوّهة، تقارب بطيء جدًا، أداء ضعيف.)

> **ملاحظة:** في Keras 3 الـdropout لا يُطبَّق أثناء الاستدلال (infer)، فـevaluation يستخدم كل الوحدات – وهذا هو السلوك الصحيح.

12. **تصميم.** اكتب سياسة Keras كاملة لوقف التدريب عند فرط التكيّف. (المتوقع: `EarlyStopping(monitor="val_loss", patience=8, min_delta=1e-4, restore_best_weights=True)` مع `ModelCheckpoint(save_best_only=True)`.)

ملخص الفصل

- فرط التكيّف يعني أن L_{train} تصغر وL_{val} ترتفع: النموذج يحفظ بدل أن يتعلّم.
- L2 ينكمش الأوزان بانتظام (مناسب عامًّا)؛ L1 ينكمشها إلى الصفر فينتج نموذجًا متناثرًا.
- `activity_regularizer` يعاقب المخرجات لا الأوزان – مفيد للembeddings.
- `Dropout` يدرب مجموعةً من الشبكات الفرعية، لكن `SpatialDropout2D` هو نسخة CNN الصحيحة.
- `BatchNormalization` يعتمد على الدفعة (يحتاج batch كبيرًا)، و`LayerNormalization` يعتمد على العيّنة (مناسب للتسلسلات)، و`GroupNormalization` يملأ الفراغ بينهما.
- المنظّم (regularizer) يضيف بندًا إلى الخسارة؛ القيد (constraint) يقصّ الأوزان بعد التحديث – الاثنان متكاملان لا بديلان.
- ابدأ بـ$\lambda = 10^{-4}$، واختر القيمة بتحقّق متقاطع، واراقب فجوة التحقّق.
- أقوى تنظيم هو **بيانات أكثر**، وأبسطها هو **نموذج أصغر**.

الجزء الثالث

هندسة البيانات: من البايتات إلى الدفعات

الجزء الذي يحدّد جودة نموذجك أكثر من أي معمارية: خطّ البيانات، المعالجة المسبقة، التعزيز، الطبقات النصية، وموازنة الأوزان والمهام المتعدّدة.

ما تكسبه من هذا الجزء

تعرف أين يذهب وقت التدريب فعلًا، وترفع دقّة النموذج بلا تغيير بنيته، وتتعامل مع البيانات غير المتوازنة بوعي.

فصول هذا الجزء

14 هندسة البيانات: من البايتات إلى الدفعات

15 المعالجة المسبقة والتعزيز: هندسة المدخلات قبل الشبكة

16 الطبقات النصية: من الجملة إلى المتجه

17 الأوزان والمهام المتعدّدة: كيف توازن النموذج وتشقّه

الفصل 14: هندسة البيانات: من البايتات إلى الدفعات

بطاقة الفصل

الفصل 14

مهمّة هذا الفصل

يبني خطّ بيانات لا يخنق المعالج: PyDataset، التحميل من المجلّدات، والتحميل المسبق المتوازي.

ماذا تكسب منه

تصل من بيانات بطيئة إلى تدريب يستغلّ العتاد كاملًا، وتقيس أين يذهب الوقت فعلًا بدل التخمين.

الدالة	دورها	متى تستخدمها
`keras.utils.PyDataset`	خطّ بيانات بايثوني قابل للتوسيع	عندما لا يكفي tf.data
`keras.utils.image_dataset_from_directory()`	تحميل صور من مجلّدات	لبدء مشروع رؤية بسرعة
`keras.utils.text_dataset_from_directory()`	تحميل نصوص من مجلّدات	لتصنيف نصوص بسيط
`keras.utils.Sequence`	واجهة ميراث للتوافق	مع كود قائم يعتمد عليها
`keras.ops.image.resize()`	تغيير الحجم داخل الرسم	عند توحيد مقاسات مختلفة

قصة مفتاحية: وصل فريق تدرّب مصنّفًا على 200 ألف صورة راديوغرافية. معالج الرسوميات كان مشغولًا 4% فقط من الوقت، بينما قضى معالج CPU 96% من عمره في فكّ ضغط ملفات JPEG وترميزها. زاد الفريق حجم الشبكة أربع مرّات، فزاد وقت الحقبة من 4 دقائق إلى 15 – بينما لم تتحسّن الدقّة ولم يقلّ الوقت. الدرس الذي تعلّمه الفريق: **النموذج لا ينتظر؛ خطّ البيانات هو الذي ينتظر.**

أهداف الفصل

- [ ] تشرح مفهوم «إطعام المعالج» (input starvation) وتقيسه عمليًا
- [ ] تحسب إنتاجية خط البيانات (examples/second) وتربطها بزمن الحقبة
- [ ] تكتب `keras.utils.PyDataset` كمصدر بيانات محمول يعمل على كل المحرّكات
- [ ] تستعمل `keras.utils.image_dataset_from_directory` و`text_dataset_from_directory`
- [ ] تُعدّل بيانات كل حقبة عبر `on_epoch_begin` و`on_epoch_end`
- [ ] تختار بين الذاكرة والتدفّق (in-memory مقابل streaming) بمعيار كميّ
- [ ] تبني خط بيانات متدرّجًا يفصل التحميل عن المعالجة عن الإرسال

14.1 تشريح بطء خط البيانات

لنفترض أن لديك 200,000 صورة بدقة 1024×1024 بصيغة JPEG (متوسّط 150 كيلوبايت لكل صورة). ومُصنّف يستهلك 2,500 صورة في الثانية على وحدة معالجة الرسوميات.

كم يستغرق تحميل الصور من القرص وقرأها؟ قرص SSD نموذجي يقرأ 500 ميغابايت/ثانية ⇒ 200,000 × 150KB = 30 غيغابايت ⇒ 60 ثانية فقط لنقل البايتات. لكن **فكّ ضغط JPEG** أبطأ بكثير: نحو 100-200 صورة/ثانية لكل نواة. مع 8 أنوية، يصبح الإنتاج نحو 1,200 صورة/ثانية – وهو **أقلّ من نصف** ما يحتاجه المعالج.

هنا تتجمّد وحدة معالجة الرسوميات (GPU) في انتظار الدفعة. في التقارير، ترى المعالج الرسومي «منهمكًا» لكن نسبة استغلاله 40% فقط.

المعادلة التي يجب أن تسكن ذهنك:

$$t_{epoch} = \max\left(t_{data},\ t_{compute}\right)$$

وزمن الحقبة **لا يتحسّن** إلّا بتحسين المسار الأبطأ. هذه معادلة تُطبَّق في كل Decisions الأداء في هذا الفصل.

من أين يأتي البطء؟

مصدر البطء	العلامة	العلاج
فكّ ضغط الصور	CPU 100%، GPU 30%	فكّ الضغط مسبقًا (pre-decode) مرة واحدة إلى تنسيق أسرع
تحويلات NumPy لكل عيّنة	CPU 100%	استخدم `keras.layers` داخل الرسم البياني
حلقات Python لكل عيّنة	CPU 100%	استخدم عمليات متجهية أو `keras.ops`
قراءة من القرص/network	I/O wait	ذاكرة تخزين مؤقت (cache)
نقل بيانات_device ⇄ host	GPU متقطّع	أرسل tensors.device-native
حجم الدفعة الصغير جدًا	GPU idle	أكبر `batch_size`

14.2 المفهوم: ثلاث مراحل لا مرحلتان

خطّ البيانات الفعّال يفصل ثلاثة محاور:

1. **التحميل (Load):** جلب البايتات من التخزين.
2. **المعالجة المسبقة (Transform):** فكّ الضغط، التحجيم، التطبيع.
3. **التجميع (Batch):** جمع العيّنات في دفعات.

القاعدة التي يعلّمها هذا الفصل:

> **افصل ما يمكن أن يكون متوازيًا عمّن لا يمكن.** العمليات التي تستهلك وحدة المعالجة (فكّ الضغط، التحجيم) تُوزَّع على عدة عمّال. العمليات التي تلمس وحدة معالجة الرسوميات (نقل الأوتورات، الحساب) تبقى على مسارها السريع.

الفصل العملي: لو فككتَ الضغط مرة واحدة وحفظت النتيجة في `.npy` أو `TFRecord`، ترفع إنتاجية_stage الاستيراد من 150 صورة/ثانية إلى أكثر من 10,000 صورة/ثانية. الفرق بين نموذج جيّد ونموذج رديء في الإنتاج غالبًا هو هذا الفارق بالضبط.

شكل 21 – خطّ البيانات: التحميل ← المعالجة ← التغذية

14.3 التنفيذ في Keras

المسار الأول: بيانات في الذاكرة

للبيانات الصغيرة (أقل من بضع مئات من ميغابايت) لا تعقّد الأمر:

```python
import keras
import numpy as np

# بيانات اصطناعية صغيرة للتجربة
rng = np.random.default_rng(0)
x_train = rng.normal(size=(2000, 28, 28, 1)).astype("float32")
y_train = (x_train[:, 0, 0, 0] > 0).astype("float32")

model = keras.Sequential([
    keras.layers.Input(shape=(28, 28, 1)),
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=
["accuracy"])

history = model.fit(x_train, y_train, batch_size=64, epochs=5, verbose=2)
```

المسار الثاني: `PyDataset` — الطريق المحمول

`keras.utils.PyDataset` هو **الواجهة الرسمية في Keras 3 لخطّ بيانات مكتوب بايثون**. ما يميّزه: يعمل على كل المحرّكات (TensorFlow وJAX وPyTorch)، ويدعم التوازي عبر `workers`، ويضمن تكرارًا صحيحًا: **كل عيّنة تُمرَّر مرة واحدة بالضبط في كل حقبة** (بعكس المولّدات بايثون التي قد تكرّر أو تُسقط).

القواعد الثلاث غير القابلة للتفاوض: - `__getitem__(i)` تُعيد **دفعة كاملة** لا عيّنة واحدة. - `__len__()` يُعيد **عدد الدفعات** لا عدد العيّنات. - (اختياري) `on_epoch_end()` لتعديل البيانات بين الحقبات.

PYTHON

```python
import keras
import numpy as np


class SignalDataset(keras.utils.PyDataset):
    """مجموعة إشارات: تولّد موجات جيبية مع ضجيج، وتعيد دفعات جاهزة."""

    def __init__(self, samples=4096, batch_size=64, length=256, noise=0.1,
seed=0):
        super().__init__()
        self.samples = samples
        self.batch_size = batch_size
        self.length = length
        self.noise = noise
        self.rng = np.random.default_rng(seed)
        # مدّة الإشارة لكل عيّنة (0. — 1.) يُعاد توليدها في كل حقبة
        self.freqs = self.rng.uniform(0.5, 3.0,
size=samples).astype("float32")
        self.phases = self.rng.uniform(0, 2 * np.pi,
size=samples).astype("float32")

    def __len__(self):
        # عدد الدفعات لا عدد العيّنات
        return self.samples // self.batch_size

    def __getitem__(self, index):
        lo = index * self.batch_size
        hi = lo + self.batch_size
        t = np.linspace(0, 1, self.length, dtype="float32")[None, :]
        # ترددات العيّنات على المحور الزمني (broadcast) يبثّ
        sig = np.sin(2 * np.pi * self.freqs[lo:hi, None] * t +
self.phases[lo:hi, None])
        sig = sig + self.noise *
self.rng.normal(size=sig.shape).astype("float32")
        x = sig[:, :, None]           # (batch, length, 1)
        y = (self.freqs[lo:hi] > 1.5).astype("float32").reshape(-1, 1)  #
(batch, 1) لمطابقة مخرَج النموذج
        return x.astype("float32"), y
```

```
    def on_epoch_end(self):
        # تعزيز بيانات: إعادة توليد الضجيج بين الحقبات implicit
        self.rng = np.random.default_rng(self.rng.integers(0, 2**31 - 1))


train = SignalDataset(samples=2048, batch_size=64)
val = SignalDataset(samples=512, batch_size=64, seed=7)

model = keras.Sequential([
    keras.layers.Input(shape=(256, 1)),
    keras.layers.Flatten(),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=
["accuracy"])

history = model.fit(
    train,
    validation_data=val,
    epochs=5,
    verbose=2,
)
```

نصيحة: `workers=4` يعني أربعة خيوط تعالج الدفعات بالتوازي.
`use_multiprocessing=True` يفيد فقط إذا كانت عملياتك (processes) قابلة للpickling – وهذا نادر مع ملفات ضخمة في الذاكرة. ابدأ بـ `workers` وخيوط.

المسار الثالث: مجلدات صور

`keras.utils.image_dataset_from_directory` تبني خط بيانات من بنية مجلدات:

```text
data/
├── train/
│   ├── cat/    (2000 *.jpg)
│   └── dog/    (2000 *.jpg)
└── validation/
    ├── cat/
    └── dog/
```

PYTHON

```python
import keras

train_ds = keras.utils.image_dataset_from_directory(
    "data/train",
    image_size=(224, 224),      # تحجيم موحّد
    batch_size=32,
    labels="inferred",          # التسميات = أسماء المجلدات
    label_mode="categorical",   # one-hot
    shuffle=True,
    seed=42,
)

val_ds = keras.utils.image_dataset_from_directory(
    "data/validation",
    image_size=(224, 224),
    batch_size=32,
    labels="inferred",
    label_mode="categorical",
    shuffle=False,
)

model = keras.Sequential([
    keras.layers.Rescaling(1.0 / 255.0),
    keras.layers.Conv2D(32, 3, activation="relu"),
    keras.layers.MaxPooling2D(),
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(2, activation="softmax"),
])
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=
["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10)
```

ملاحظة: هذه الدالة تُعيد `tf.data.Dataset`، فهي تتطلّب TensorFlow مثبَّتًا. إن كان محرّكك JAX أو PyTorch، ابنِ خطّك فوق `PyDataset` – وهو الطريق المحمول تمامًا.

المسار الرابع: النصوص

```python
train_ds = keras.utils.text_dataset_from_directory(
    "data/train_text",
    labels="inferred",
    label_mode="categorical",
    batch_size=32,
)
```

14.4 مقارنة مصادر البيانات

المصدر	يعمل مع كل المحرّكات	يدعم توازي العمّال	مناسب لـ	العيب
Arrays في الذاكرة	✅	❌ (Keras يقسّمها)	بيانات صغيرة	يستهلك RAM كاملًا
`keras.utils.PyDataset`	✅	✅	كل الحالات	تكتب بايثون
`keras.utils.Sequence`	✅	✅	توافق مع كود Keras 2	اسم قديم (اسم بديل لـ `PyDataset`)
`image_dataset_from_directory`	❌ (TF فقط)	✅ داخليًّا	تصنيف صور	يتطلّب TensorFlow
`tf.data.Dataset`	❌ (TF فقط)	✅ بأداء أعلى	إنتاجية قصوى	يعتمد على TF
مولّد بايثون (`yield`)	✅	❌	تدفّق لا يتكرّر	قد يكرّر عيّنات

معيار الاختيار المعتدل: - عيّنات أقل من 50,000 وصور بصيغة **arrays** ⇒ `.npy` . - أي شيء أكبر ⇒ `PyDataset` كخيار افتراضي محمول. - إنتاجية قصوى على منصة `tf.data` ⇒ TensorFlow مع `prefetch` و `map` .

14.5 مثال محلول: خطّ بيانات بطيء

فريق يدرّب مصنّفًا على 120,000 صورة من قرص شبكة. القياس: 380 عيّنة/ثانية. المعالج الرسومي يستطيع 2,000 عيّنة/ثانية.

الخطوة 1 – احسب الحدّ النظري.

$$t_{data} = \frac{120000}{380} = 316 \text{ ثانية} \approx 5.3 \text{ دقائق لكل حقبة}$$

ومتوقّع الفريق 60 ثانية ⇒ **الفجوة 5 أضعاف**.

الخطوة 2 – شخّص. أين ينفق الوقت؟ قِس مراحل خطّك منفردة:

```python
import time
import numpy as np

# محاكاة: كم يستغرق فكّ ضغط وتحجيم عيّنة واحدة؟
import io
import keras

t0 = time.perf_counter()
raw = np.random.randint(0, 255, size=(1024, 1024, 3), dtype=np.uint8)
t_decode = time.perf_counter() - t0

t0 = time.perf_counter()
small = keras.ops.image.resize(keras.ops.convert_to_tensor(raw), (224, 224))
t_resize = time.perf_counter() - t0

print(f"decode={t_decode*1e3:.2f}ms  resize={t_resize*1e3:.2f}ms")
print(f"الإنتاجية المتوقعة بـ 8 أنوية: {8/max(t_decode+t_resize,1e-9):.0f} عيّنة/ثانية")
```

الخطوة 3 – طبّق الحل. بدل فكّ الضغط في كل حقبة، فُكّ مرة واحدة إلى مخزن سريع:

```python
import numpy as np
import os

def predecode_to_memmap(src_dir, out_prefix, size=(224, 224), limit=None):
    """تحميل شبه فوري لاحقًا). يقرأ الصور مرّة واحدة ويحفظها في ملف .npy"""
    from PIL import Image

    files = sorted(os.listdir(src_dir))[:limit]
    out = np.zeros((len(files), size[0], size[1], 3), dtype="uint8")
    for i, fn in enumerate(files):
        img = Image.open(os.path.join(src_dir,
fn)).convert("RGB").resize(size)
        out[i] = np.asarray(img)
    path = f"{out_prefix}.npy"
    np.save(path, out)
    print(f"حُفظ {len(files)} صورة في {path}")
    return path


# ثم التدريب يصبح:
# x = np.load("train_224.npy", mmap_mode="r")  ← قراءة شبه فورية
```

الخطوة 4 – قِس. بعد التحضير المسبق: 25,000 عيّنة/ثانية (قراءة `mmap` + تحويل dtype). الآن $t_{data} = 120000/25000 = 4.8$ ثانية، والم computing became للنموذج هو الحدّ الجديد.

الخطوة 5 – طبّق الحيلة الثانية: `prefetch`. في `PyDataset` مع محرّك TensorFlow يمكنك زيادة `max_queue_size`؛ وفي `tf.data` استخدم `prefetch(tf.data.AUTOTUNE)`. المعنى: دائمًا احتفظ بـ k دفعات جاهزة في الطابور بينما يحسب المعالج الرسومي الدفعة الحالية.

الخطوة 6 – النتيجة. زمن الحقبة من 5.3 دقيقة إلى 60 ثانية. **لم يتغيّر النموذج ولا البيانات ولا الدقّة – فقط طريقة تقديمها.**

قاعدة عملية: قبل أن تعقّد النموذج، قِس زمن الدفعة الواحدة في خطّ البيانات وزمن الدفعة في معالج الرسوميات. أيّهما كان أبطأ هو حدّك الحقيقي.

14.6 أخطاء شائعة في هذا الفصل

إنذار: الخلط بين `__len__` و`__getitem__` في `PyDataset`. إن كان `__len__` يُعيد عدد **العيّنات** بينما `__getitem__` يُعيد **دفعة**، ستتدرّب الشبكة على بيانات مشوّهة بصمت (س Batch حجمه عشوائي ومحتواه مكرَّر). القاعدة: `__len__` = عدد الدفعات.

تحذير: تمرير `shuffle=True` إلى `image_dataset_from_directory` مع `seed` ثابت، ثم توقّع ترتيبًا مختلفًا في كل حقبة. في `PyDataset`، استخدم `on_epoch_begin` لتبديل ترتيب الفهارس صراحةً.

تنبيه: نسخ بيانات ضخمة إلى الذاكرة ثم تعديلها في `on_epoch_end` يستهلك ذاكرة مضاعفة. إن كانت مجموعتك أكبر من RAM، استخدم مخزنًا على القرص (`mmap`) أو اجعل التعديل محليًا داخل `__getitem__`.

قاعدة: لا تستخدم `use_multiprocessing=True` مع كائنات غير قابلة للـpickling (lambdas، معالجات ملفات مفتوحة). ستنهار بصمت في worker child. ابدأ بـ `workers=2` و`use_multiprocessing=False`.

ملاحظة: `image_dataset_from_directory` تفترض أسماء الفئات بالترتيب **المعجمي**. أسماء مثل «cat» و«dog» ليست مشكلة، لكن «Class10» و«Class2» تعطي ترتيبًا غير متوقّع – فراجع `class_names` المطبوعة دائمًا.

التمارين

1. **حساب.** خطّ بيانات ينتج 500 عيّنة/ثانية، ومعالج الرسوميات يستهلك 2,000 عيّنة/ثانية. ما زمن الحقبة لـ50,000 عيّنة؟ (المتوقع: الحدّ الأبطأ هو البيانات ⇒ 100 ثانية؛ وقت المعالج 25 ثانية فقط.)

2. **حساب.** ضاعفت إنتاجية خطّ البيانات من 300 إلى 1,200 عيّنة/ثانية، والحدّ الجديد صار 1,800. احسب التحسّن الكليّ في زمن الحقبة لـ100,000 عيّنة. (المتوقع: من 333 ثانية (بيانات) إلى 55.6 ثانية (بيانات) مقابل 55.6 ثانية (حساب) ⇒ ما زال الحدّ 55.6 ثانية؛ أي لم يتحسّن الزمن بعد 1,200/ثانية لأن الحساب صار هو الأبطأ.)

3. **تصميم.** صمّم `PyDataset` لمجموعة ملفات صوتية من مجلد. (المتوقع: `__len__` ⇒ عدد الدفعات = `ceil(n_files / batch_size)`، و`__getitem__` ⇒ يفتح الملفات ويحوّلها ويعيد دفعة.)

4. **برمجة.** اكتب `PyDataset` يولّد أرقامًا عشوائية ويُعيدها مع تسمياتها. (المتوقع: `__getitem__` يعيد `(x[lo:hi], y[lo:hi])`.)

5. **تحليل.** لماذا `PyDataset` أفضل من مولّد بايثون (`yield`) في ضمان عدم تكرار العيّنات؟ (المتوقع: المولّد قد يُستهلك مرّتين أو يُسقط ذيله؛ `PyDataset` له `__len__` معلوم وعدد دفعات محدّد.)

6. **برمجة.** اجعل `__len__` يُعيد عدد العيّنات بدل الدفعات في تمرين 4، ولاحظ الخلل. (المتوقع: تدريب مكرَّر على نفس الدفعة الأولى لأن `__getitem__(i)` شرطه `i * batch < n`.)

7. **تصميم.** خطّ إنتاج يستخدم Python لكن يلزمه throughput لا يقلّ عن 5,000 عيّنة/ثانية. ما خيارك؟ (المتوقع: `PyDataset` مع `workers=8` + تحضير مسبق للبيانات + `max_queue_size` مناسب.)

8. **أخطئ عمدًا.** مرّر `use_multiprocessing=True` مع lambda في دالة المعالجة. (المتوقع: خطأ pickling في العملية الفرعية.)

9. **حساب.** `max_queue_size=10` و`batch_size=64` بحجم 1 ميغابايت لكل دفعة. كم ميغابايتًا تحتفظ بها الذاكرة؟ (المتوقع: 640 ميغابايت كحدّ أقصى؛ خفّضه إلى 4 لتوفير 240

ميغابايت.)

10. برمجة. قِس إنتاجية `PyDataset` بمقارنة `workers=1` و `workers=4`. (المتوقع: تحسّن يقارب 2-3 أضعاف إن كانت العملية محسوبة بالCPU؛ لا تحسّن إن كانت I/O فقط.)

> **ملاحظة:** في هذا التمرين نحسب الإنتاجية العمليّة:

11. تحليل. لماذا لا يُنصح بفكّ ضغط صور JPEG داخل `__getitem__`؟ (المتوقع: لأن فكّ الضغط يكرّر كل حقبة؛ الحل فكّ مسبق أو تخزين بصيغة أسرع.)

12. تصميم. صمّم خطّ بيانات لمشروع يُدرَّب مرّة كل أسبوع على 2 مليون صورة، مع حدّ ذاكرة 16 غيغابايت. (المتوقع: تخزين بصيغة غير مضغوطة/محوّرة مسبقًا، `mmap`، تجزئة إلى ملفات `shard`، و`PyDataset` مع `workers`.)

ملخص الفصل

- معادلة زمن الحقبة هي `max(t_data, t_compute)`؛ حسّن الأبطأ أولًا.
- قِس إنتاجية خطّ البيانات (عيّنات/ثانية) قبل أي تحسين في النموذج.
- `keras.utils.PyDataset` هو الطريق المحمول: `__getitem__` يعيد **دفعة**، و`__len__` يعيد **عدد الدفعات**.
- `workers` يسرّع العمليات الحاسوبية (فكّ الضغط، التحجيم)؛ و`use_multiprocessing` يتطلّب قابلية الpickling.
- `on_epoch_end` و`on_epoch_begin` تمنحانك تحكّمًا في البيانات بين الحقبات.
- `image_dataset_from_directory` عملية لكن TensorFlow فقط.
- التحضير المسبق (pre-decoding) هو أسرع تحسّن متاح: من 150 عيّنة/ثانية إلى آلاف.
- لا تنسَ `prefetch`: طابور جاهز دائمًا هو أرخص تسريع على الإطلاق.

الفصل 15: المعالجة المسبقة والتعزيز: هندسة المدخلات قبل الشبكة

بطاقة الفصل

الفصل 15

مهمّة هذا الفصل

يهندس المدخلات قبل الشبكة: تطبيع، إعادة قياس، وتعزيز يحاكي التوزيع الحقيقي لا يفسده.

ماذا تكسب منه

ترفع دقّة النموذج بلا تغيير بنيته، وتتجنّب تعزيزًا يغيّر معنى الصورة.

الدالة	دورها	متى تستخدمها
`keras.layers.Rescaling(scale, offset)`	تحويل نطاق القيم	لتحويل 0..255 إلى 0..1
`keras.layers.Normalization()`	تطبيع إحصائي يتعلّم من البيانات	للميزات الجدولية
`keras.layers.RandomFlip()`	قلب عشوائي	في الصور المتماثلة اتجاهيًّا
`keras.layers.RandomRotation()`	دوران عشوائي	عندما لا يهمّ ميل الجسم
`keras.layers.MixUp()`	مزج عيّنات وتسمياتها	لتحسين المعايرة عند نقص البيانات

قصة مفتاحية: فريق تدرّب مصنّفًا على صور لمنتج واحد، كانت أسماؤه بالإنجليزية والفرنسية معًا، بينما كان النموذج يتوقّع صورًا بأسماء إنجليزية فقط. بعد أسبوع من التدريب وصلت الدقّة إلى 31% على مجموعة التحقّق. لم يكن الخطأ في الشبكة ولا في البيانات: كان في **طبقة المعالجة المسبقة** التي وُضعت بعد التحقّق من مجلدات التحقّق – أي بعد أن صار جزءًا من خط الإنتاج لا من التدريب.

أهداف الفصل

- [ ] تميّز بين التطبيع (Normalization) والتحجيم (Rescaling) والتحسين البصري (brightness)
- [ ] تعرف متى يكون التطبيع ضروريًا ومتى يكون ضارًّا
- [ ] تشرح مبدأ التعزيز (augmentation) بوصفه توليد بيانات جديدة لا تجميلًا
- [ ] تستعمل طبقات `keras.layers.Random*` التعزيزية داخل الرسم البياني
- [ ] تختار عمليات تعزيز تناسب طبيعة المهمة (تصوير طبي ≠ نقوش industrial)
- [ ] تحذّر من فخّ التسرّب (data leakage) في المعالجة المسبقة
- [ ] تبني خط معالجة مسبق كامل يُدمج في ملف `.keras`

15.1 من البايت إلى الرقم: ثلاث عمليات لا واحدة

قبل أن تدخل الصورة الشبكة، يجب أن تمرّ بثلاث عمليات متمايزة، وخلطها أكثر أخطاء المبتدئين:

أولًا: التحجيم (Rescaling). تحويل الأعداد الصحيحة 0-255 إلى أعداد عشرية 0-1:

$$x_{scaled} = \frac{x_{uint8}}{255}$$

الهدف ليس «التطبيع» بل **توحيد المقياس** بحيث تكون التدرّجات في network بنفس الرتبة عبر كل channels. لماذا 0-1 وليس 0-255؟ لأن الخسارة تُحسب على المخرجات: قيم كبيرة في الدخول تُنتج لوغاريتمات ضخمة في softmax، وهي عرضة لتجاوز الحدّ العددي (`inf`) وتُنتج `nan` في التدرّجات. لذلك **افعل**، لكن **داخل النموذج** لا على الأقراص.

ثانيًا: التطبيع (Normalization). توحيد **الإحصاءات** لكل feature على حدة:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

حيث μ وσ هما المتوسط والانحراف المعياري لكل قناة في **بيانات التدريب**. طبقة `keras.layers.Normalization` تعلّم هذين الإحصاءين في مرحلة التدريب وتثبّتهما (non-

trainable) للاستدلال.

ثالثًا: التعزيز (Augmentation). توليد نسخ معدَّلة من كل عيّنة أثناء التدريب.

> **تحذير مفاهيمي:** كثير من الكتّاب يسمّي «تطبيع البيانات» كل ما سبق. لكن المعالجة المسبقة والNormalization شيءان مختلفان: الأولى تحوّل الأرقام، والثاني يعيد توزيعها.

أين يُطبَّق كلٌّ منها؟

العملية	مكان التطبيق	لماذا
فكّ الضغط والتحجيم الهندسي	**قبل** الشبكة أو كطبقة	لا يمكن عملها داخل الشبكة دون كلفة
`Rescaling`	**داخل** الشبكة كطبقة أولى	يضمن أن التطبيع لا يُفسد النطاق
`Normalization`	**داخل** الشبكة بعد `Rescaling`	يحتاج تعلّم إحصاءات التدريب
augmentation `Random*`	**داخل** الشبكة، و**أثناء التدريب فقط**	يجب ألا تُطبَّق على التحقّق

15.2 الرياضيات: لماذا يعمل التعزيز؟

التعزيز يندرج تحت مفهوم **التعلّمrawing المسطّح (flatness)**: يفضّل النماذج التي تصل فيها Loss إلى خوارزمية landscape مسطّحة، لأن فيها يكون محيط الخسارة واسعًا حول الحل، فتُحتمل الأخطاء في البيانات دون انهيار الأداء.

فكّر في وزن w واحد.Minimization بسيط يعطي w^* مع خسارة $L(w^*) = 0.05$. عيّنات training اضافية تعطي w^{**} مع خسارة $L(w^{**}) = 0.06$. أيّهما أفضل؟

- w^*: حادّ – أي اضطراب في w (حتى صغير) يرفع الخسارة كثيرًا.

- w^{**}: مسطّح – perturbations صغيرة ترفع الخسارة قليلًا.

في مساحة عالية الأبعاد، النموذج «المسطّح» يتحمّل ضجيج البيانات ويعمّم أفضل. والتعزيز ينتج ذلك عمليًا: كل عيّنة تُدرَّب على نسخ كثيرة متعدّدة، فتُجبَر الشبكة على إيجاد منطقة-performing في مساحة واسعة.

العلاقة، بصيغتها الشائعة:

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(x,y)\sim p_{\text{aug}}} \left[\mathcal{L}(f_\theta(x), y) \right]$$

الحل الأدنى ليس الحل الأدقّ على p_{data}، بل على p_{aug} – التوزيع الموسَّع. وهذا بالضبط لماذا قد تنخفض دقّة التحقّق قليلًا بعد التعزيز: أنت لم تعد تُحسّن على التوزيع الأصلي بل على توزيع أوسع يفترض ثبات البنية الدلالية تحت التحويلات.

شكل 22 – شبكة التعزيز: تحويلات الصورة الأصلية

15.3 التنفيذ في Keras

خط معالجة مسبق متكامل

في Keras 3، المعالجة المسبقة جزء من **الرسم البياني**، فتُحفظ مع النموذج في ملف `.keras`، وتعمل على دفعات كاملة وعلى وحدة معالجة الرسوميات. هذه ميزة تشغيلية ضخمة: لا يوجد سكربت منفصل يجب أن يتطابق مع النموذج.

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── طبقة معالجة مسبقة مدمجة
preprocess = keras.Sequential([
    keras.layers.Rescaling(1.0 / 255.0),      # 0..255 → 0..1
    keras.layers.RandomFlip("horizontal"),     # انعكاس أفقي
    keras.layers.RandomRotation(0.1),          # دوران حتى ±10% من زاوية
    keras.layers.RandomZoom(0.1),              # تكبير/تصغير حتى ±10%
    keras.layers.RandomContrast(0.1),          # تعديل التباين
    keras.layers.Normalization(),              # توحيد إحصاءات القنوات
], name="preprocess")

# ── نموذج صغير يتّصل بها
model = keras.Sequential([
    keras.layers.Input(shape=(96, 96, 3)),
    preprocess,
    keras.layers.Conv2D(32, 3, activation="relu"),
    keras.layers.MaxPooling2D(),
    keras.layers.Conv2D(64, 3, activation="relu"),
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(4, activation="softmax"),
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

# بيانات تجريبية
x = np.random.randint(0, 255, size=(256, 96, 96, 3), dtype=np.uint8)
y = np.random.randint(0, 4, size=256)

history = model.fit(x, y, batch_size=32, epochs=3, validation_split=0.2)
```

نقطة جوهرية: الطبقة `Normalization` تحتاج أن «ترى» بيانات التدريب. ولأنها داخل الشبكة، تتعلّم إحصاءاتها تلقائيًا أثناء `fit` ثم تُثبَّتها.

> **تنبيه:** `keras.layers.Normalization` داخل `Sequential` **قبل** `compile` تتعلّم إحصاءاتها في أول نداء لها. إن استدعيتَ `model.predict` قبل `fit`، فستُطبَّق إحصاءات افتراضية (μ=0، σ=1) – أي **بلا أثر**. رتّب دائمًا: `fit` أوّلًا.

عمليات التعزيز المتوفّرة في Keras 3.15.1

الطبقة	المعامل الرئيسي	ما تفعله
`RandomFlip`	`"horizontal"` / `"vertical"` / `"both"`	انعكاس
`RandomRotation`	`factor` (0..1)	دوران حول المركز
`RandomZoom`	`factor`	تكبير أو تصغير
`RandomTranslation`	`height_factor`, `width_factor`	إزاحة
`RandomBrightness`	`factor`, `value_range`	تعديل السطوع
`RandomContrast`	`factor`, `value_range`	تعديل التباين
`RandomCrop`	`height`, `width`	قصّ عشوائي
`RandomGrayscale`	`factor`	تحويل إلى رمادي
`RandomHue` / `RandomSaturation`	`factor`	تعديل اللون
`MixUp`	`alpha`	خلط عيّنتين (تسميات مدمجة)
`CutMix`	`factor`	لصق جزء من صورة داخل أخرى
`RandAugment`	`num_layers`, `magnitude`	سياسة تعزيز عشوائية
`RandomErasing`	`factor`	مسح Areas عشوائية
`GaussianNoise`	`stddev`	ضجيج غاوسي على المدخلات
`RandomErasing` / `RandomSharpness` / `RandomShear`	–	عمليات إضافية

MixUp وCutMix: التعزيز الذي يتعلّم

النوعان الأخطر يفعلان شيئًا مختلفًا عن «التشويش»: **يبدّلان المعادلة نفسها**، فتُنتجان صورًا وتسميات جديدة من صورتين.

MixUp يخلط مدخلَين وتسميةَيهما:

$$\tilde{x} = \lambda x_i + (1-\lambda) x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda) y_j$$

حيث $\lambda \sim \mathrm{Beta}(\alpha, \alpha)$. النتيجة: «صورة نصفها قطة ونصفها كلب» مع تسمية 0.5/0.5. الشبكة لم تعد مضطرّة لأن تكون حادّة في حدود القرار.

CutMix يقصّ مستطيلًا من صورة ويلصقه في أخرى:

$$\tilde{x} = M \odot x_i + (1-M) \odot x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda) y_j$$

حيث M قناع ثنائي. النتيجة: صورة فيها «رقعة» من صنف آخر.

> **نصيحة:** MixUp وCutMix يعملان على **tasks التصنيف** لأنهما يفرضان تسميات مدمجة. لا تصلح للregression (حيث لا معنى لدمج القيم المستهدفة) ولا للdetection المبسّطة.

التعزيز في مكانه الصحيح: داخل النموذج أم خارجه؟

الخيار	متى تختاره	المميزات	العيوب
طبقات داخل الشبكة	المعيار الافتراضي	تعمل على GPU، تُحفظ مع النموذج	تزيح على الCPU إن كانت على CPU
NumPy قبل `fit`	تحضير بيانات ثابت	بسيط وواضح	لا يوفّر تنويعًا كل حقبة
`ImageDataGenerator` (Keras 2)	**مُهجور في Keras 3**	–	غير متاح

15.4 مقارنة عمليات التعزيز بحسب المهمة

المهمة	تعزيزات مناسبة	تعزيزات خطِرة	السبب
تصنيف صور طبيّة	تكبير/إزاحة صغيرة، اختلاف إضاءة خفيف	انعكاس أفقي (لو شامت)	Chest يمين وقلب يسار
تصنيف قطع صناعية	إضاءة، تكبير، إضافة ضجيج	انعكاس (اتجاه الملمس)، تدوير كبير	اتجاه المحور يحدّد الصنف
نقوش/حروف	إزاحة، تكبير، تدوير صغير	انعكاس أفقي (يقلب الحرف)	الحروف غير متناظرة
تصنيف منتجات تجارية	انعكاس أفقي، تدوير، تحسين الألوان	قلب التدرّج اللوني	اللون جزء من هوية المنتج
تصنيف مشاهد طبيعية	قصّ، انعكاس، تعديل ألوان	–	invariances واضحة
كشف الأجسام	تكبير/إزاحة، MixUp	تعزيزات تُشوّه أحجام الأجسام	يجب الحفاظ على المقاييس

المبدأ الحاكم: التعزيز يصرّح بأن هناك تحويلات تحافظ على «المعنى». إن كانت التحويلة تغيّر المعنى، فهي لا تعزّز بل **تتلوّث** بياناتك.

التحويلات التي يجب أن تكون محسوبة لا عشوائية

بعض المعالجات تحتاج **معرفة مسبقة** بالإحصاءات: - `Rescaling` : ثابت معروف (255). - `Normalization` : يحتاج μ وσ من التدريب (تُعلَّم تلقائيًا). - `StandardScaler` على بيانات جدولية: يحتاج μ وσ من التدريب.

القاعدة: **كل إحصاء يُقدَّر من البيانات يجب أن يُقدَّر من جزء التدريب فقط**. إن قدّرتَ المتوسط على كامل البيانات (بما فيها التحقّق)، فقدتَ تقديرًا غير متحيّز، وقد ينخفض الأداء الحقيقي بمقدار ملموس. هذا هو **تسرّب البيانات**، وهو من أخطر الأخطاء وأكثرها خمولًا.

15.5 مثال محلول: مصنّف عيّنات الجلد

مهمة: تصنيف صور Eczema جلدية إلى 4 فئات. 8,000 صورة. الدقة الحالية 71% والـF1 0.62.

الخطوة 1 – شخّص. الـF1 منخفض مقارنة بالدقة ⇒ مشكلة في **الاستدعاء** على الأصناف النادرة.

الفئات: 5,200 / 1,800 / 700 / 300.

الخطوة 2 – طبّع داخل الشبكة.

```python
import keras

preprocess = keras.Sequential([
    keras.layers.Rescaling(1.0 / 255.0),
    keras.layers.RandomFlip("horizontal"),      # مقبول في Dermatology
    keras.layers.RandomRotation(0.15),
    keras.layers.RandomZoom(0.15),
    keras.layers.RandomBrightness(0.2),
    keras.layers.Normalization(),
], name="skin_preprocess")
```

الخطوة 3 – أضف MixUp (مهمّ هنا لأن الأصناف نادرة، والمزج بين الفئات يعمل كمولّد توازن):

```python
keras.layers.MixUp(alpha=0.3, name="mixup")
```

> **ملاحظة:** `MixUp` طبقة تُدرج **قبل** طبقة الخسارة مباشرة؛ يمكنك وضعها في نهاية المعالجة السابقة.

الخطوة 4 – وازِن الأصناف عبر `class_weight`:

```python
import numpy as np

counts = np.array([5200, 1800, 700, 300])
weights = counts.sum() / (len(counts) * counts)
class_weight = {i: float(w) for i, w in enumerate(weights)}
print(class_weight)   # النتيجة المتوقعة: {0: 0.577, 1: 1.667, 2: 4.286, 3: 10.0}
```

الخطوة 5 – قِس الأثر. بعد التغيير: الدقة 74%، **F1 0.79**. الـPrecision انخفض قليلًا (0.86 → 0.83) لأن النموذج صار أكثر جرأة في التنبؤ بالفئات النادرة – وهذا بالضبط المقايضة المطلوبة: نقبل بعض الإيجابيات الكاذبة لنلتقط عددًا أكبر من الإيجابيات الحقيقية.

الخطوة 6 – احذر التحقّق. طبّقتَ `Normalization` داخل الشبكة، فالتعزيز طبَّق على التحقّق أيضًا. هل هذا خطأ؟

لا – والسبب تقنيّ: التعزيز عشوائي ويحاكي تنوّعًا واقعيًا؛ ولأن معايير التقييم تعتمد على الأداء «في المتوسط» لا على عيّنة واحدة، فالتنويع **لا يُربك** القياس إحصائيًا. أما `Normalization` فـ μ وσ مأخوذان من التدريب فقط لأن الطبقة تعلّمهما في `fit` – وهذا يضمن عدم التسرّب.

القاعدة الدقيقة:

> **العشوائية (التعزيز) تُطبَّق على التدريب فقط لأنها زائدة عن الحاجة (أو ضارّة) على التحقّق. الإحصاءات (المتوسط والانحراف) تُقدَّر من التدريب فقط لأن رصد التحقّق بها يعني تسرّبًا.**

15.6 أخطاء شائعة في هذا الفصل

> **إنذار:** تطبيق التعزيز على **بيانات التحقّق** يدويًا في كود خارج الشبكة. عندها ستقيس أداءً على بيانات «أصعب» أو «أسهل» من reality. اجعل التعزيز داخل النموذج (فتُطبَّق تلقائيًا في `training=True` فقط)، أو طبّقه يدويًا على `x_train` فقط.

تحذير: `keras.layers.Normalization` **قبل** `fit` تُعيد نفس القيم، فتظنّ أنها لا تفعل شيئًا. ثبّت `Normalization` **بعد** `Rescaling` وتحقّق من أثرها بتقريب متوسّط المخرجات:

PYTHON

```python
import keras
import numpy as np

norm = keras.layers.Normalization()
x = np.array([[[0.0, 100.0, 250.0]]], dtype="float32")
print("قبل:", norm(x, training=True).numpy().ravel())  # النتيجة المتوقعة: قيم قريبة من 0
print("بعد:", norm(x, training=False).numpy().ravel()) # بلا أثر قبل التعلّم
```

تنبيه: `MixUp` و `CutMix` مع `loss="sparse_categorical_crossentropy"` لا يعملان كما يُتوقّع: التسميات المدمجة continuous لا تقبلها خسارة sparse. استخدم `CategoricalCrossentropy` (التي تقبل تسميات ناعمة) أو انظر إلى Softmax.

قاعدة: لا تعزّز بما يُغيّر المهمة. انعكاس أفقي على صور ويديك؟ لا – يبدّل اليد. انعكاس أفقي على صور سيارات؟ نعم، لا يُغيّر المعنى.

ملاحظة: كل طبقة `Random*` لها `seed`؛ اضبطه لنتائج قابلة لإعادة الإنتاج. لكن تذكّر: التعزيز يكسر التطابق التام بين حقبتين، وهذا مقصود.

التمارين

1. **حساب.** صورة بكسل واحد قيمته 200. ما ناتج `Rescaling(1/255)` ؟ (المتوقع: $200/255 \approx 0.784$.)

2. **حساب.** قناة بمتوسط 128 وانحراف معياري 32. ما القيمة المعيارية للبكسل 160؟ (المتوقع: $(160-128)/32 = 1.0$.)

3. **حساب.** MixUp بـ$\lambda = 0.25$ بين عيّنة من الفئة 0 وأخرى من الفئة 1. ما التسمية المدمجة؟ (المتوقع: 0.75 للفئة 0 و0.25 للفئة 1.)

4. **تصميم.** أيّ عمليات تعزيز تختار لمصنّف Workpieces معدني؟ (المتوقع: إضاءة وتباين وضجيج، مع تجنّب الانعكاس إن كان اتجاه tooling في الصنف، وتجنّب التدوير الكبير.)

5. **تحليل.** لماذا لا يُنصح بـ `RandomFlip("horizontal")` في تصنيف صور الرعد؟ (المتوقع:تشريحي؛ القلب يمين وجسم يسار، فالنموذج سيتعلّم اتجاهًا غير صحيح.)

6. **برمجة.** ابنِ `Sequential` فيه `Rescaling` ثم `RandomRotation(0.2)` ثم `Normalization`، واطبع متوسط المخرجات قبل وبعد `fit`. (المتوقع: قبل `fit` القيم موحّدة تقريبًا؛ بعد `fit` تصبح قريبة من الصفر.)

7. **برمجة.** أضف `MixUp(alpha=0.4)` إلى خطّ المعالجة واختبر الناتج على دفعتين. (المتوقع: تسميات متجه واحد و batch-size من ناقص واحد.)

> **ملاحظة:** هنا حساب يدوي لوزن MixUp:

8. **حساب.** بيتا(α, α) لـ α=1 تعطي توزيعًا منتظمًا على [0,1]. لو كان λ=0.3، فما مقدار كل من الصورتين في المزيج؟ (المتوقع: 30% من الأولى و70% من الثانية.)

9. **تصميم.** صمّم خط معالجة لمصنّف أغلفة كتب: أيّ عمليات؟ (المتوقع: تكبير/تصغير، إزاحة طفيفة، تعديل إضاءة وتباين، قلب أفقي مقبول، مع التطبيع؛ وتجنّب قلبًا رأسيًا.)

10. **أخطئ عمدًا.** ضع `Normalization` قبل `Rescaling`. اشرح الخلل. (المتوقع: تطبيع قيم 0-255 قبل القسمة على 255 يُبطل أثر التطبيع ويحوّل الأعداد الصحيحة إلى floats ثم يعيدها إلى نطاق 0-1.)

11. **تحليل.** لماذا `class_weight` ليس بديلًا عن التعزيز؟ (المتوقع: `class_weight` يعيد وزن الخسارة لكنه لا يضيف تنوّعًا بصريًا؛ التعزيز يزيد فعليًا عدد النماذج المدروسة لكل صنف.)

12. **تصميم.** صمّم سياسة تعزيز لبيانات صناعية عالية الجودة، مع توضيح أيّ عملية تتوقّع أن تُحدث ضررًا في العيّنة. (المتوقع: لا تدوير كبير ولا انعكاس؛ إضافة ضجيج غاوسي، إضاءة، تكبير/تصغير صغير، وإضافة عيّنات معيب نادرة.)

ملخص الفصل

- التحجيم (`Rescaling`) يوحّد المقياس؛ التطبيع (`Normalization`) يوحّد الإحصاءات؛ التعزيز يوسّع التوزيع. لا تخلط بينها.
- ضع المعالجة المسبقة **داخل الشبكة** كطبقات، فتحصل على حفظ تلقائي في `.keras` وعمل على وحدة المعالجة الرسومية.
- `keras.layers.Normalization` تتعلّم إحصاءاتها في أول `fit` ؛ ترتيب العمليات مهمّ: `Rescaling` ثم `Normalization` .
- التعزيز يعبّر عن «تحويلات تحافظ على المعنى» – اختر العمليات بحسب المهمة لا بحسب العمومية.
- `MixUp` و `CutMix` يوسّعان **حدود القرار** ويعطيان توازنًا طبيعيًا بين الأصناف، لكنهما للتصنيف لا للانحدار.
- التسرّب: قدّر كل إحصاء من التدريب فقط؛ أما العشوائية (التعزيز) فالتطبيق على التحقّق مقبول لأنها توسّع التوزيع.
- عيّنات `class_weight` تكمل التعزيز: الأولى تعيد وزن الخسارة، والثانية تضيف تنوّعًا.

الفصل 16: الطبقات النصية: من الجملة إلى المتجه

بطاقة الفصل الفصل 16

مهمّة هذا الفصل

يحوّل الجملة إلى متجه: TextVectorization وStringLookup وEmbedding والحشو والتعليم.

ماذا تكسب منه

تبني خطًّا نصيًّا يعيش داخل النموذج نفسه ويُحفظ معه، فلا تنفصل المعالجة عن النموذج عند النشر.

الدالة	دورها	متى تستخدمها
`keras.layers.TextVectorization()`	تحويل نصّ إلى رموز ثم أعداد	أول طبقة في أي نموذج نصّي
`keras.layers.Embedding(input_dim, output_dim)`	متجه كثيف لكل رمز	بعد الترميز
`keras.layers.StringLookup()`	قاموس نصّ إلى عدد	عندما تحتاج تحكّمًا يدويًّا بالقواميس
`keras.layers.Hashing()`	ترميز تجزئة بلا قاموس	للنصوص المفتوحة الضخمة
`keras.layers.CategoryEncoding()`	ترميز فئوي	للميزات الفئوية القصيرة

قصة مفتاحية: أرسلت شركة تجارة إلكترونية 50 ألف مراجعة عملاء مصنّفة يدويًا: 4,500 إيجابية و45,500 سلبية. الفريق بنى شبكة بسيطة وحصل على دقّة تحقّق 96%... وحصل أيضًا على 4,500 مراجعة إيجابية صحيحة. لكن المراجعة التي تعترض على تغيّر حجم الطلب وصلت **إيجابية** رغم أنها سلبية تمامًا. السبب: طبقة التقطيع قطّعت الجملة عند كل مسافة، ففقدت العلاقة بين الكلمات، وصار النموذج يتعلّم تكرار الكلمة «أبدًا» بدل معنى الرفض.

أهداف الفصل

- [ ] تشرح لماذا تحتاج النص إلى تمثيل رقمي قبل أن يدخل الشبكة
- [ ] تميّز بين أنماط إخراج `TextVectorization` : `int` ، `multi_hot` ، `count` ، `tf_idf` ، `float_multi_hot`
- [ ] تفهم دور `StringLookup` و `IntegerLookup` و `Hashing` في بناء vocabulary
- [ ] تشرح لماذا `Embedding` متعلّمة أفضل من `one-hot` و `multi_hot`
- [ ] تبني خطّ معالجة نصية كامل داخل النموذج
- [ ] تتعامل مع النصوص العربية: التشكيل، الرموز، RTL
- [ ] تختار نموذجًا مناسبًا لحجم المفردات ولغة المهمة

16.1 لماذا لا يدخل النص الشبكة مباشرةً؟

الشبكة العصبية تقبل أرقامًا لا NaWords. أكثر من ذلك: كل عدد يدخل الشبكة يجب أن يكون **متجهًا ثابت الطول**، لأن الأوزان لها أشكال ثابتة.

لذلك تتحوّل عملية معالجة النص إلى **سلسلة تحويلات**:

نص خام $\xrightarrow{\text{تقطيع}}$ كلمات $\xrightarrow{\text{ترميز}}$ أعداد صحيحة $\xrightarrow{\text{تضمين}}$ متجهات

لكل مرحلة قرار: - **أين تُقطَّع الكلمات؟** بالمسافات فقط (الافتراضي `split="whitespace"`) أم بفواصل (مثلاً `split=","` أو `split=char`)؟ العربي يتّصل بغيار فواصل، فالتقطيع بالمسافات يعطي كلمات مثل «بخير» و«الله» ملتصقتين بـ«و». - **كيف تُرقَّم الكلمات؟** بترتيب تكرارها (`StringLookup`) أم بالهاش (`Hashing`)؟ - **بأي متجه تُمثَّل؟** رقم صحيح (input for `Embedding`) أم متجه one-hot ضخم (multi_hot)؟

> **ملاحظة:** `TextVectorization` في Keras 3 مدمجة: تتعلّم الvocabulary أثناء `fit` (بمرّ على البيانات)، ثم تثبّتها. هذا يعني أن `TextVectorization` **يجب أن تسبق** أي طبقة تعتمد على الvocabulary داخل الرسم البياني.

شكل 23 – خطّ النص: من الكلمة إلى المتجه

16.2 الرياضيات: من الكلمة إلى المتجه

الترقيم (Vocabulary)

الفكرة: كل كلمة فريدة تأخذ رقمًا صحيحًا. أشهر الكلمات (بعد حذف stop words) تأخذ الأرقام الصغيرة:

الكلمة	معرّف
«الممتاز»	1
«جيد»	2
«سيّئ»	3
`[UNK]`	0

رمز `[UNK]` (كلمة غير معروفة) يُعطى لكل كلمة لم تظهر في التدريب. بدونه، ستتعطّل الشبكة عند encountering كلمة جديدة.

One-hot مقابل التضمين

التمثيل one-hot لثلاث كلمات:

$$\text{«ممتاز»} = [1, 0, 0], \quad \text{«جيد»} = [0, 1, 0], \quad \text{«سيّئ»} = [0, 0, 1]$$

المشكلة: المسافة بين أيّ متجهين متساوية، ولا تعلّم الشبكة أيّ علاقة دلالية. كما أن حجم المتجه يساوي حجم الvocabulary (40,000 كلمة تعني 40,000 بعد).

التضمين (Embedding) يربط كل كلمة بمتجه كثيف $e_i \in \mathbb{R}^d$ (مثلًا $d = 64$ أو 128):

$$E \in \mathbb{R}^{|V| \times d}, \qquad \text{«ممتاز»} \mapsto E[1]$$

مزايا التضمين: 1. **أبعاد ثابتة وصغيرة**: 128 عددًا بدل 40,000. 2. **علاقات دلالية متعلّمة**: الكلمات المتشابهة في السياق تقترب في فضاء المتجهات. 3. **تعلّم من النص**: أوزان التضمين تُحدَّث مع الشبكة.

TF-IDF

عند تمثيل النص كbag-of-words نحتاج وزنًا لكل كلمة:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log\left(\frac{N}{\text{df}(t)}\right)$$

حيث tf تكرار الكلمة في المستند، وdf عدد المستندات التي تحتويها، وN عدد المستندات. الكلمة الشائعة في كل المستندات تحصل وزنًا صغيرًا (لأن $\log(N/\text{df}) \approx 0$).

أقنعة (Masking)

النصوص متغيّرة الطول. عند التحويل إلى تسلسل بأطوال مختلفة، نحتاج تعليم الشبكة أيّ المواضع حقيقية وأيّها حشو:

$$\text{mask} = [1, 1, 1, 0, 0, 0]$$

تُنتجها `keras.layers.Masking` (القيمة `0` تُعتبر حشوًا) أو تلقائيًا `TextVectorization` مع `pad_to_max_tokens=True` في وضع `int`.

16.3 التنفيذ في Keras

الطبقة المركزية: `TextVectorization`

PYTHON

```
import keras
import numpy as np

# نصوص عربية مصغّرة
texts = np.array([
    "المنتج ممتاز وجودة عالية",
    "جودة المنتج ممتازة وسعر مناسب",
    "منتج سيئ لا أنصح به",
    "سيء جدا وباهظ الثمن",
])

vec = keras.layers.TextVectorization(
    max_tokens=5000,                    # سقف حجم القاموس
    standardize="lower_and_strip_punctuation",
    split="whitespace",
    output_mode="multi_hot",            # متجه presence/absence
)
vec.adapt(texts)                        # تعلّم القاموس من البيانات
print(vec.get_vocabulary())             # النتيجة المتوقعة: {'[UNK]': 0,
'1 :'منتج, ...}

out = vec(texts)
print(out.shape)                        # النتيجة المتوقعة: (4, size_vocab)
```

أنماط الإخراج المتاحة:

output_mode	الناتج	حجم المتجه	متى تستخدمه
`"int"`	تسلسل معرّفات عددية	`output_sequence_length`	مع `Embedding`
`"multi_hot"`	0/1 لكل كلمة	`size_vocab`	نماذج خطية بسيطة، تصنيف أقسام
`"count"`	عدد التكرارات	`size_vocab`	bag-of-words مع TF-IDF لا
`"float_multi_hot"`	0/1 ك أعداد عشرية	`size_vocab`	مع `weighted`
`"tf_idf"`	أوزان TF-IDF	`size_vocab`	تصنيف نصوص تقليدي

> **تحذير مهم:** وضع `"multi_hot"` ينتج متجهًا بحجم القاموس كامل (40,000 على الأقل). إدخال ذلك إلى طبقة `Dense` يعني 128×40,000 = 5.12 مليون معامل في الطبقة الأولى. استخدمه للنماذج الخطية (Logistic Regression) أو مع `Embedding` لا بعده.

من `int` إلى `Embedding`: المسار الكامل

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

texts = np.array([
    "المنتج ممتاز وجودة عالية",
    "جودة المنتج ممتازة وسعر مناسب",
    "منتج سيئ لا أنصح به",
    "سيء جدا وباهظ الثمن",
    "رائع جدا وأعجبني",
    "سريع التوصيل وسعره جيد",
    "منتج ممتاز وسريع",
    "لا ينصح به أبدا",
] * 8)

labels = np.array([1, 1, 0, 0, 1, 1, 1, 0] * 8)

model = keras.Sequential([
    keras.layers.Input(shape=(1,), dtype="string"),
    keras.layers.TextVectorization(
        max_tokens=1000,
        output_mode="int",
        output_sequence_length=12,      # تثبيت الطول
        pad_to_max_tokens=True,
    ),
    keras.layers.Embedding(input_dim=1000, output_dim=64, mask_zero=True),
    keras.layers.GlobalAveragePooling1D(),      # يطبّق القناع تلقائيًا
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
```

```
    metrics=["accuracy"],
)
model.fit(texts, labels, epochs=5, batch_size=8, verbose=0)

print(model.evaluate(texts, labels, verbose=0))
# النتيجة المتوقعة: loss وaccuracy على مجموعة صغيرة
```

> **ملاحظة:** `mask_zero=True` في `Embedding` يجعل القيمة 0 (مخصّصة للحشو) تُنتج قناعًا؛ و`GlobalAveragePooling1D` يحترم هذا القناع فلا يحتسب الحشو في المتوسّط.

`StringLookup` و`IntegerLookup`: القواميس صراحةً

`StringLookup` يحوّل **نصًا** إلى معرّفات، و`IntegerLookup` يحوّل **أعدادًا** إلى معرّفات (مثل رموز مخزّنة مسبقًا).

PYTHON

```
import keras
import numpy as np

# قاموس فئات: نوع المنتج
lookup = keras.layers.StringLookup(
    num_oov_indices=1,          # فهرس مخصّص لكل قيمة خارج القاموس
    oov_token="[UNK]",
    vocabulary=["electronics", "books", "clothing"],
)
print(lookup(["electronics", "unknown-item"]))
# النتيجة المتوقعة: [0 ,3]   (هو 0 [UNK])
```

لماذا نستخدم `StringLookup` منفصلًا عن `TextVectorization`؟ لأننا نحتاج تحكّمًا أدقّ: قوائم vocabulary مختلفة (مثلًا أسماء العملاء)، أو أوزان `idf_weights`، أو `invert=True` للترميز العكسي.

`Hashing` : قاموس بلا ذاكرة

```python
hash_layer = keras.layers.Hashing(
    num_bins=1024,                # حجم فضاء الهاش
    salt=1,                       # يغيّر توزيع الهاش
    output_mode="multi_hot",
)
out = hash_layer(["جودة" ,"منتج"])
print(out.shape)                  # النتيجة المتوقعة: (2 ,1024)
```

الميزة: لا vocabulary ولا حالة تُحفظ، حجم ثابت لا ينمو مع البيانات، ويعمل بشكل ممتاز مع التوزيع (streaming).

العيب: التصادم (collision) – كلمتان مختلفتان قدتُصادِمنفس الهاش. والvocabulary غير قابلة للمراجعة (لا يمكنك معرفة ما إذا كانت كلمة نادرة ضاعت).

> **نصيحة:** استخدم `Hashing` عندما يكون القاموس غير محدود (سجلّات، معرّفات) أو عندما تحتاج حجمًا ثابتًا مضمونًا. استخدم `StringLookup` عندما تحتاج control_جودة الvocabulary.

`Discretization` و `CategoryEncoding`

```python
import keras
import numpy as np

# ترميز الفئات
cat = keras.layers.CategoryEncoding(num_tokens=5, output_mode="one_hot")
print(cat([1, 3]))              # النتيجة المتوقعة: [[0,0,0,1,0],[0,1,0,0,0]]

# تقسيم الأعداد إلى شرائح
disc = keras.layers.Discretization(bin_boundaries=[0, 10, 20, 30],
output_mode="one_hot")
print(disc([5, 15, 25]))        # النتيجة المتوقعة: one-hot لكل شريحة
```

`Discretization` مفيد لتحويل الأعداد المستمرة (السن، السعر، عدد الطلبات) إلى فئات – وهي طريقة فعّالة لالتقاط العلاقات غير الخطية.

معالجة العربية عمليًا

PYTHON

```python
import keras
import re
import numpy as np


def normalize_arabic(text):
    """توحيد النص العربي: إزالة التشكيل، توحيد الألف والياء والتاء المربوطة."""
    text = re.sub(r"[\u064B-\u0652\u0670]", "", text)    # حذف التشكيل
    text = re.sub(r"[أإآ]", "ا", text)                    # توحيد الألف
    text = re.sub(r"ي" ,"ى", text)                        # توحيد الياء
    text = re.sub(r"و" ,"ؤ", text)                        # واو همزة
    text = re.sub(r"ي" ,"ئ", text)                        # ياء همزة
    text = re.sub(r"ه" ,"ة", text)                        # تاء مربوطة
    text = re.sub(r"\s+", " ", text).strip()
    return text


samples = np.array([normalize_arabic(t) for t in [
    "الطَّلَبُ ممتازٌ",
    "الطلب ممتاز",
]])
vec = keras.layers.TextVectorization(output_mode="multi_hot")
vec.adapt(samples)
print(vec.get_vocabulary())
# النتيجة المتوقعة: {'[UNK]': 0, 'ممتاز': 2, 'الطلب': 1}
```

قاعدة: طبّع النص العربي **قبل** إدخاله إلى `TextVectorization`، واحفظ دالّة التنظيف في الكود المصدري (وليس يدويًا لكل دفعة). إن استخدمتَ `TextVectorization` مع `standardize="lower_and_strip_punctuation"` فقط، فالتشكيل سيبقى فيُنتج كلمات مختلفة ("ممتاز" و"ممتازٌ" token مختلفان).

16.4 مقارنة طبقات المعالجة النصية

الطبقة	المدخل	يعلّم vocabulary؟	حجم الناتج	مزايا	عيوب
`TextVectorization`	نص	نعم (في `adapt` / `fit`)	حسب النمط	كل الأدوات في طبقة واحدة	`adapt` يحتاج مرورًا على البيانات
`StringLookup`	نص	نعم	`num_tokens`	تحكّم دقيق، يدعم `invert`	يجب تزويده بالقاموس
`IntegerLookup`	أعداد	نعم	`num_tokens`	يصلح للرموز المسبقة الرمزة	لا يحوّل نصًا
`Hashing`	نص أو أعداد	**لا**	`num_bins` ثابت	لا حالة، حجم مضمون	تصادم، لا مراجعة للقاموس
`CategoryEncoding`	أعداد	لا	`num_tokens`	بسيط جدًا	لا يحوّل نصًا
`Discretization`	أعداد	لا	`num_bins`	يلتقط غير الخطية	يحتاج اختيار حدود
`Embedding`	أعداد	الأوزان فقط	`output_dim`	متجهات كثيفة تعلّم دلالات	يحتاج vocabulary

معيار الاختيار: - قاموس محدود ومرئي ⇒ `IntegerLookup` / `StringLookup` . - قاموس مفتوح أو تدفّق بيانات ⇒ `Hashing` . - راحة في البداية ⇒ `TextVectorization` مع

`output_mode="int"` ثم `Embedding` . - نموذج خطي أو سريع ⇒ `TextVectorization` مع `output_mode="tf_idf"` .

16.5 مثال محلول: تحليل مشاعر مراجعات

متجر إلكتروني يريد تصنيف 30,000 مراجعة إلى إيجابية/سلبية/محايدة.

الخطوة 1 – اختر تمثيلًا. البيانات: 30,000 مراجعة، متوسط 25 كلمة، قاموس متوقّع 60,000 كلمة. مع `Dense` + `multi_hot` ، الطبقة الأولى ستكون 60,000×128 = 7.68 مليون معامل – أكثر من اللازم. القرار: `Embedding(64)` + `output_mode="int"` .

الخطوة 2 – ابنِ خطّ المعالجة.

```python
import keras
import numpy as np

keras.utils.set_random_seed(7)

MAX_TOKENS = 20000
SEQ_LEN = 40

model = keras.Sequential([
    keras.layers.Input(shape=(1,), dtype="string"),
    keras.layers.TextVectorization(
        name="vec",
        max_tokens=MAX_TOKENS,
        standardize="lower_and_strip_punctuation",
        output_mode="int",
        output_sequence_length=SEQ_LEN,
        pad_to_max_tokens=True,
    ),
    keras.layers.Embedding(MAX_TOKENS, 64, mask_zero=True),
    keras.layers.Bidirectional(keras.layers.GRU(32,
return_sequences=False)),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(3, activation="softmax"),
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

الخطوة 3 – انتبه لحجم الـ vocabulary. `MAX_TOKENS = 20000` مع `Embedding(MAX_TOKENS, 64)` يُنشئ مصفوفة 20,000×64 = 1.28 مليون معامل – حتى لو كان القاموس الفعلي 8,000 كلمة. الحل: بعد التدريب، قصّ المصفوفة، أو استخدم `input_dim` مطابقًا للقاموس الحقيقي بعد `adapt`.

```python
# بعد التدريب: اعرف الحجم الحقيقي
vocab = model.get_layer(name="vec").get_vocabulary()
real_size = len(vocab)
print("حجم القاموس:", real_size)      # النتيجة المتوقعة: عدد أصغر من 20000
```

الخطوة 4 – قِس على مقاييس كل صنف (الفصل 10):

```python
metrics = [
    keras.metrics.SparseCategoricalAccuracy(name="acc"),
    keras.metrics.F1Score(average="macro", name="f1_macro"),
    keras.metrics.Precision(class_id=0, name="p_neg"),
    keras.metrics.Recall(class_id=0, name="r_neg"),
]
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=metrics,
)
```

الخطوة 5 – استعمل `EarlyStopping` على المقياس الصحيح (وليس الخسارة):

```python
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_f1_macro",      # المقياس الذي يهمّ العمل
        mode="max",
        patience=4,
        restore_best_weights=True,
    ),
]
```

الخطوة 6 – الدروس المستفادة. 1. `multi_hot` كان سيضيف 7.68 مليون معامل زائدة. 2. `GRU` ثنائي الاتجاه يلتقط نفي الجملة («لم يكن **ممتازًا**») الذي يفشل فيه

`GlobalAveragePooling` على التضمين. 3. `Bidirectional(GRU)` + `mask_zero=True` يتعاملان مع الحشو بشكل صحيح. 4. الفهرس 0 محجوز لـ `[UNK]` (و للقناع في `Embedding`)، فتأكّد أن `mask_zero=True` يتوافق مع `output_mode="int"` حيث 0 = `[UNK]` ... وهنا **تنبيه مهم**: إن استخدمتَ `mask_zero=True` مع قاموس يبدأ من 1، فالقيمة 0 يجب ألّا تكون كلمة حقيقية.

16.6 أخطاء شائعة في هذا الفصل

إنذار: نسيان `adapt` أو `fit` قبل الاستدعاء. `TextVectorization` يبدأ بقاموس فارغ، فكل الكلمات تصبح `[UNK]`، وستتدرّب الشبكة على أرقام بلا معنى. symptom: دقّة التدريب عالية ودقّة التحقّق عشوائية.

تحذير: خلط `output_mode="int"` مع `Dense` مباشرة. الأعداد الصحيحة ليست متجهات؛ إدخالها إلى `Dense` يعني أن 30,000 معرّفًا موجبة الرقم تُعامَل كقيم عددية (المعرّف 5000 أكبر من 300!).. العرِض:: تدريب بطيء وغير مستقرّ. الحل: `Embedding` بعد `int` دائمًا.

تنبيه: `pad_to_max_tokens=True` مع `output_sequence_length=None` في وضع `int` – يعطي تسلسلًا بطول متغيّر، فيرفض `Dense` قبوله. حُدّد `output_sequence_length` دائمًا.

قاعدة: لا تستخدم `multi_hot` مع قاموس كبير ثم `Dense` كبيرة. الحساب: `vocab ×` `units` معامل. بـ60,000×128 وحدها 7.68 مليون. استخدم `Embedding` .

ملاحظة: `TextVectorization` داخل `Sequential` يُكيف عبر `adapt` تلقائيًا عند أول استدعاء، لكن **داخل نموذج Keras 3Functional** يجب أن تستدعيَ `vec.adapt(texts)` صراحةً أو أن يمرّ `fit` على البيانات.

تحذير: التشكيل العربي. «ممتاز» و«ممتازٌ» كلمتان مختلفتان في القاموس، فتُضعف كل واحدة نصف قوّتها. طبّع النص قبل.

التمارين

1. **حساب.** ما حجم متجّه `multi_hot` لقاموس 25,000 كلمة؟ وكم معاملًا في `Dense(64)` بعده؟ (المتوقع: 25,000 متجه؛ 64×25,000 = 1,600,000 معامل.)

2. **حساب.** في `tf-idf`، كلمة تظهر في 900 مستند من 1000. كم قيمتها تقريبًا لو $\text{tf} = 1$؟ (المتوقع: $\log(1000/900) \approx 0.105$.)

3. **حساب.** كلّما زاد df (تكرار الكلمة في المستندات)، هل يزيد وزن TF-IDF أم ينقص؟ (المتوقع: ينقص، لأن $\log(N/\text{df})$ ينقص.)

4. **برمجة.** أنشئ `TextVectorization(output_mode="count")` على ثلاث جمل واطبع المتجهات. (المتوقع: عدّادات تكرار الكلمات.)

5. **برمجة.** قارن `StringLookup` مع `Hashing(num_bins=8)` على 20 كلمة، ولاحظ التصادم. (المتوقع: `Hashing` يُنتج معرّفات متكرّرة لكلمات مختلفة لأن 8 سلال صغيرة.)

6. **برمجة.** ابنِ `TextVectorization` بـ `output_mode="tf_idf"` على 100 جملة وأظهر أن الكلمة الشائعة لها وزن أصغر. (المتوقع: وزن «الطلب» في كل الجمل أصغر من وزن كلمة نادرة.)

7. **برمجة.** اكتب دالة `normalize_arabic` وأظهر أن «الطَّلَبُ» و«الطلب» يصبحان متطابقين بعدها. (المتوقع: نعم.)

8. **تحليل.** لماذا `mask_zero=True` مهمّ عند استخدام `GlobalAveragePooling1D`؟ (المتوقع: بدونه يُحسب الحشو في المتوسّط فيُضعف الإشارة.)

9. **تصميم.** صمّم خطًا نصيًا لمراجعات بلغة غير معرَّفة (مثل الاسم_SLUG). (المتوقع: `Hashing` بقاموس ثابت + `Embedding`.)

10. **أخطئ عمدًا.** استخدم `output_mode="int"` ثم `Dense(64)` مباشرة. اشرح سلوك التدريب. (المتوقع: أرقام كم/features بلا دلالة؛ تدريب غير مستقرّ ونتائج أسوأ من `Embedding`.)

11. **تحليل.** لماذا لا يصلح `TextVectorization` مع `Embedding` a+ `output_mode="int"` إن كان القاموس يتضمّن `[UNK]` في الفهرس 0

و `mask_zero=True` ؟ (المتوقع: القيمة 0 ستُعامَل كحشو، فكل الكلمات غير المعروفة ستُخفي.)

12. تصميم. صمّم نموذجًا لتصنيف عناوين عقارات بثلاث فئات، مع تحديد `output_mode` والمقاييس. (المتوقع: `int` + `Embedding(64)` + `GlobalAveragePooling1D` + `Dense`، مع `F1Score(average="macro")`.)

ملخص الفصل

- النص يحتاج تحويلًا إلى أرقام: تقطيع ← ترقيم ← تمثيل متجهي.
- `TextVectorization` تعلّم القاموس عبر `adapt` / `fit` وتثبّته؛ نسيان ذلك يعني كل شيء `[UNK]`.
- `Embedding` + `output_mode="int"` هو المسار القياسي؛ `multi_hot` يصلح للنماذج الخطية فقط.
- `Embedding` متعلّم يعطي متجهات كثيفة وأبعادًا ثابتة وعلاقات دلالية.
- `StringLookup` للقواميس المراقَبة، و `Hashing` للقواميس المفتوحة بثبات حجم وبلا حالة.
- `Discretization` و `CategoryEncoding` أدوات مفيدة لتحويل الأعداد إلى فئات.
- عالج التشكيل والتطبيع العربي قبل التقطيع.
- انتبه لتضخيم المعاملات: `vocab × units` يكلّف كثيرًا.

الفصل 17: الأوزان والمهام المتعدّدة: كيف توازن النموذج وتشقّه

بطاقة الفصل

الفصل 17

مهمّة هذا الفصل

يوازن الخسارة عبر الفئات والمهام: أوزان الأصناف، Focal Loss، وثلاثة أنماط للمهام المتعدّدة.

ماذا تكسب منه

تحصل على نموذج عادل مع الأصناف النادرة، ونموذج واحد يخدم أكثر من هدف بلا صراع بينها.

الدالة	دورها	متى تستخدمها
`model.fit(class_weight=...)`	وزن الأصناف داخل الخسارة	عند عدم توازن الفئات
`keras.losses.BinaryFocalCrossentropy()`	تركيز على الحالات الصعبة	في الكشف عن الأحداث النادرة
`keras.losses.CategoricalFocalCrossentropy()`	نسخة متعدّدة الفئات	في التصنيف غير المتوازن
`model.compile(loss={...}, loss_weights={...})`	خسارة لكل مخرج مع أوزان	في المهام المتعدّدة
`keras.metrics.RecallAtPrecision()`	استدعاء عند دقّة محدّدة	لتقرير عتبة تشغيلية

قصة مفتاحية: شركة تأمين بنت نموذج تصنيف للمطالبات: «مطالبة عادية» و«مطالبة مشبوهة». في بيانات التدريب كانت 97% عادية. النموذج تعلّم أن يقول «عادية» لكل شيء، وحقّق دقّةً 97%... وسمّى 2 من كل 50 مطالبة مشبوهة. الفريق اكتشف المشكلة من لوحة المراقبة: نسبة التنبيه للمخاطر انخفضت إلى الصفر. المشكلة لم تكن في الشبكة، بل في أن **الخسارة التي دُرِّب عليها النموذج لم تعاقب الخطأ الذي يكلّفه العميل**.

أهداف الفصل

- [ ] تميّز بين `class_weight` و `sample_weight` و `weighted_metrics` ومتى تستخدم كلًا منها
- [ ] تعرف كيف تعيد الخسارة الموزونة (weighted loss) ترجيح سعة الخطأ في بيانات غير متوازنة
- [ ] تبني نموذجًا متعدّد المداخل (multi-input) في Keras
- [ ] تبني نموذجًا متعدّد المخارج (multi-output) وتوازن بين الخسائر
- [ ] تفهم متى يكون التعلّم متعدّد المهام (multi-task) أفضل من نموذجين منفصلين
- [ ] تعالج عدم التوازن بخمس أدوات مختلفة وتختار بينها بمعيار
- [ ] تصمّم مجموعة مقاييس مناسبة لمهمة غير متوازنة

17.1 الرياضيات: كيف يغيّر الوزن الخسارة؟

في الحالة العادية، الخسارة المتوسّطة:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, \hat{y}_i)$$

كل عيّنة تحظى بنفس الأهمية. عند **الوزن**، تصبح:

$$\mathcal{L}_w = \frac{1}{\sum_i w_i} \sum_{i=1}^{N} w_i \, \ell(y_i, \hat{y}_i)$$

حيث w_i وزن العيّنة i. لاحظ ثلاثة أشياء مهمّة:

1. **الأثر مضاعَف**: العيّنة النادرة بوزن 10 تُسهم بعشرة أضعاف في التدرّج.
2. **التطبيع**: القسمة على $\sum w_i$ تمنع تضخيم الخسارة الكلّية مع زيادة الأوزان.

3. الانحياز: الشبكة ستُعطي تنبّؤاتها ترجيحًا أعلى للأصناف النادرة – وهذا بالضبط المطلوب في المهام التي تكلّف فيها حالة الفوت inexpensive.

`class_weight` : وزن حسب الصنف

يُعطى ك قاموس `{class_index: weight}` في `fit` :

$$w_i = w_{y_i}$$

الحساب القياسي (العكسي للتكرار):

$$w_c = \frac{N}{C \cdot n_c}$$

حيث N إجمالي العيّنات، C عدد الأصناف، وn_c عدد عيّنات الصنف c. مثال: أصناف 900/100 ⇒ $w_0 = 1/9 = 0.111$ و$w_1 = 9$.

`sample_weight` : وزن حسب العيّنة

متجه بطول N، يعطي لكل عيّنة وزنها الخاص. مفيد عندما تكون الجودة متفاوتة: صورة ضبابية تُوزن 0.5، وصورة حادّة تُوزن 1.5. وقد يُحسب من **ثقة annotator** أو من درجة العام الماضي.

`weighted_metrics` : قياس مرجّح

PYTHON

```
import keras

model = keras.Sequential([
    keras.layers.Input((32,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(2),
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=[
        keras.metrics.SparseCategoricalAccuracy(),
        keras.metrics.F1Score(average="macro"),
    ],
    weighted_metrics=[keras.metrics.TruePositives(),
keras.metrics.FalsePositives()],
)
```

`weighted_metrics` تُقيَّم بوزن كل عيّنة، فتُعطي صورة أوضح عن الأداء على المجموعات المهمّة.

> **قاعدة:** `class_weight` و `sample_weight` يؤثّران في **الخسارة** (أي في التدرّجات والتعلّم)، و `weighted_metrics` تؤثّر في **القياس فقط**. عالج الخلل في مكانه الصحيح.

شكل 24 – أوزان الأصناف وأثرها على الخسارة

17.2 التصنيف المتعدّد: مخرج واحد أو متعدّد؟

النوع	شكل المخرج	شكل التسمية	الخسارة المناسبة
ثنائي	`(batch, 1)` مع sigmoid	`(batch, 1)`	`BinaryCrossentropy`
ثنائي متعدّد التسميات	`(batch, 1)` مع sigmoid	`(batch, C)`	`BinaryCrossentropy`
متعدّد الصنف	`(batch, C)` مع softmax	`(batch, C)` one-hot	`CategoricalCrossentropy`
متعدّد الصنف المُفهرس	`(batch, C)` مع softmax	`(batch,)` أرقام	`SparseCategoricalCrossentropy`

نصيحة: `SparseCategoricalCrossentropy` أرخص من `CategoricalCrossentropy` لأن الأولى لا تحتاج تحويل التسميات إلى one-hot. استخدمها كلّما كانت التسميات أرقامًا.

تحذير: `from_logits=True` يعني أن الخسارة تتوقّع **لوغاريتمات** (بلا softmax). إن مرّرتَ `from_logits=True` مع طبقة softmax، ستطبّق softmax مرّتين فتُنتج احتمالات مشوّهة. إن اخترت `from_logits=True`، فاجعل الطبقة الأخيرة **بلا تفعيل**:

```python
keras.layers.Dense(C)   # بلا activation
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

17.3 المهام المتعدّدة: ثلاثة أنماط

النمط الأول: مدخلات متعدّدة

When كل عيّنة تحمل أكثر من نوع من الخصائص. مثال: تصنيف منتج من (صورة + وصف نصّي + السعر).

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# مدخلات متعدّدة: صورة + متجه خصائص
img_input = keras.Input(shape=(32, 32, 3), name="image")
feat_input = keras.Input(shape=(10,), name="features")

x = keras.layers.Conv2D(32, 3, activation="relu")(img_input)
x = keras.layers.GlobalAveragePooling2D()(x)
x = keras.layers.Dense(64, activation="relu")(x)

y = keras.layers.Dense(32, activation="relu")(feat_input)

merged = keras.layers.Concatenate()([x, y])
out = keras.layers.Dense(4, activation="softmax")(merged)

model = keras.Model(inputs=[img_input, feat_input], outputs=out)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",   # التسميات أعداد صحيحة
    metrics=["accuracy"],
)
model.summary()

# بيانات تجريبية
imgs = np.random.rand(64, 32, 32, 3).astype("float32")
feats = np.random.rand(64, 10).astype("float32")
labels = np.random.randint(0, 4, size=64)

history = model.fit(
    {"image": imgs, "features": feats},
    labels,
    epochs=3,
    batch_size=8,
    verbose=0,
)
```

شرط أساسي: يجب أن يطابق ترتيب القواميس في `fit` ترتيب المدخلات في `Model(inputs=...)`. انتبه: القواميس في `fit` تمرّ بالترتيب **ال أبجدي**Sometimes في Keras 3؟ لا – Keras 3 يطابق بالاسم إذا كان dict `inputs`، وبالترتيب إذا كان list. استخدم **dict** دائمًا لتجنّب الالتباس.

النمط الثاني: مخرجات متعدّدة

Network واحدة تخرج عدة تنبّؤات. مثال: كشف العيوب (صنف +_boxes).

PYTHON

```python
import keras
import numpy as np

# مخرجان: تصنيف العيّنة + انحدار العيب
class_out = keras.Input(shape=(1,), name="y_class")
reg_out = keras.Input(shape=(1,), name="y_reg")

inp = keras.Input(shape=(20,), name="features")
h = keras.layers.Dense(64, activation="relu")(inp)
h = keras.layers.Dense(32, activation="relu")(h)

pred_class = keras.layers.Dense(3, activation="softmax", name="class")(h)
pred_reg = keras.layers.Dense(1, name="reg")(h)

multi = keras.Model(inputs=inp, outputs=[pred_class, pred_reg])
multi.compile(
    optimizer="adam",
    loss={
        "class": "sparse_categorical_crossentropy",
        "reg": "mse",
    },
    loss_weights={
        "class": 1.0,
        "reg": 0.1,     # وازن بين المخرجين
    },
    metrics={
        "class": ["accuracy"],
        "reg": ["mae"],
    },
)
```

`loss_weights` هي معادلة الموازنة. بدونها، ستسيطر الخسارة الأكبر (مثلاً MSE على قيم كبيرة) على التدرّجات، ويتجاهل النموذج الخسارة الأصغر. القيمة 0.1 تعني «كل وحدة خطأ في الانحدار تساوي 0.1 من وحدة خطأ التصنيف».

النمط الثالث: تعلّم متعدّد المهام (Multi-task)

بدل نموذجين منفصلين (واحد للعيّنات وواحد للعيب)، نموذج واحد يتعلّم مهمّتين تتشاركان jet representation.

PYTHON

```python
import keras

inp = keras.Input(shape=(20,))
shared = keras.layers.Dense(64, activation="relu")(inp)
shared = keras.layers.BatchNormalization()(shared)
shared = keras.layers.Dense(32, activation="relu")(shared)

out_a = keras.layers.Dense(1, activation="sigmoid", name="task_a")(shared)
out_b = keras.layers.Dense(1, activation="sigmoid", name="task_b")(shared)

mtl = keras.Model(inputs=inp, outputs=[out_a, out_b])
mtl.compile(
    optimizer="adam",
    loss={"task_a": "binary_crossentropy", "task_b": "binary_crossentropy"},
    loss_weights={"task_a": 1.0, "task_b": 0.5},
)
```

متى يتفوّق على نموذجين منفصلين؟ - المهام **مرتبطة** (تشترك في بنية تمثيل مفيدة). - البيانات **محدودة** (المهمة النادرة تستفيد من بيانات المهمة الغزيرة عبر التمثيل المشترك). - تحتاج **حجم نموذج** أصغر في الإنتاج (شبكة واحدة).

متى يكون أسوأ؟ - المهام **متنافسة** أو غير مرتبطة؛ عندها يتضارب التدرّجان. - كل مهمة تحتاج **سعة** مختلفة كثيرًا.

> **نصيحة:** وزّع الخسائر بحيث تعكس تكرار المهام. المهمة النادرة تحتاج وزنًا أكبر عادةً، لا أصغر.

17.4 خمس أدوات لعدم التوازن

الأداة	المبدأ	متى تختارها	الكلفة
`class_weight`	وزن في الخسارة	أسرع حل، فعّال عادةً	قد يخفض الدقّة العامة
`sample_weight`	وزن لكل عيّنة	جودة متفاوتة للعيّنات	تحتاج حساب الأوزان
`Data augmentation`	توليد تنوّع	أمثلة نادرة قليلة	تحتاج عمليات مناسبة
`Focal loss`	تركيز على الصعب	تشبّه extreme imbalance	تتطلّب `BinaryFocalCrossentropy`
`Resampling`	أكثر/أقلّ عيّنات	تحكّم كامل	خطر تسرّب أو تشويه التوزيع

Focal Loss: لماذا يعمل؟

الخسارة المتقاطعة العادية تعاقب التنبّؤ **السهل** بنفس القدر الذي تعاقب به التنبّؤ **الصعب**:

$$\mathcal{L}_{CE} = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

حيث p_t احتمال الصنف الحقيقي، و$\gamma \geq 0$ «معامل التركيز». عند $\gamma = 2$ تُقلَّص خسارة الأمثلة السهلة (منخفضة p_t بعد التصحيح) إلى الربع تقريبًا، بينما تبقى خسارة الأمثلة الصعبة شبه كما هي. النتيجة: الشبكة **تتركّز** على النادرة.

Keras 3 يوفّرها جاهزة: `keras.losses.BinaryFocalCrossentropy` و`keras.losses.CategoricalFocalCrossentropy` (مع `alpha` و`gamma`).

```python
import keras

loss = keras.losses.BinaryFocalCrossentropy(
    from_logits=False,
    alpha=0.25,      # وزن الصنف النادر
    gamma=2.0,       # معامل التركيز
)
model.compile(optimizer="adam", loss=loss, metrics=["accuracy"])
```

17.5 مثال محلول: تصنيف معاملات مشبوهة

الخطوة 1 – قِس الفجوة. النموذج: دقّة 0.985، استدعاء على الصنف المشبوه 0.09. مرفوض تمامًا: 91% من الاحتيال يمرّ.

الخطوة 2 – جرّب `class_weight` أولًا (أسرع وأقلّ تعقيدًا):

```python
import keras
import numpy as np

counts = np.array([98000, 2000])           # 98% / 2%
n = counts.sum()
weights = n / (len(counts) * counts)       # [0.510, 25.0]
class_weight = {0: float(weights[0]), 1: float(weights[1])}
print(class_weight)   # النتيجة المتوقعة: {0: 0.5102, 1: 25.0}

model.fit(x_train, y_train, class_weight=class_weight, epochs=5, verbose=0)
```

النتيجة: الاستدعاء ارتفع إلى 0.61، والدقّة انخفضت إلى 0.958. تحسّن كبير لكن الاستدعاء ما زال غير كافٍ (Fortune: 39% من الاحتيال يمرّ).

الخطوة 3 – ضَع Focal Loss فوق ذلك:

PYTHON

```python
loss = keras.losses.BinaryFocalCrossentropy(
    from_logits=True,
    alpha=0.75,      # أنزّل وزن الصنف الشائع
    gamma=2.0,
)
model.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss=loss,
    metrics=[
        keras.metrics.BinaryAccuracy(),
        keras.metrics.Precision(),
        keras.metrics.Recall(),
        keras.metrics.AUC(),
    ],
)
```

النتيجة: الاستدعاء 0.87، والدقّة 0.941، والPrecision 0.62.

الخطوة 4 – ضَع العتبة، لا النموذج. مع `from_logits=True` لا يمكنك تمرير `threshold` إلى `BinaryAccuracy` مباشرة (لأنه يعمل على الاحتمالات). الحل: طبقة sigmoid في الاستدلال، أو تمرير العتبة يدويًا:

PYTHON

```python
# بعد التدريب: جرّب عتبات مختلفة على الاحتمالات
probs = model.predict(x_val).ravel()
y_true = y_val.ravel()
for thr in (0.3, 0.5, 0.7):
    pred = (probs >= thr).astype("float32")
    tp = float(np.sum((pred == 1) & (y_true == 1)))
    fp = float(np.sum((pred == 1) & (y_true == 0)))
    fn = float(np.sum((pred == 0) & (y_true == 1)))
    precision = tp / max(tp + fp, 1)
    recall = tp / max(tp + fn, 1)
    print(f"thr={thr}: P={precision:.3f} R={recall:.3f}")
```

ملاحظة: اختيار العتبة قرار **تشغيلي** لا رياضي: حدّد من تحلّة الفريق «كم تنبيهًا خاطئًا يمكن تحمّله أسبوعيًا؟» ثم اختر العتبة التي تحقّق ذلك. أداة `keras.metrics.RecallAtPrecision` تساعد في البحث الآلي.

الخطوة 5 – راقب المقاييس الصحيحة. مع 2% إيجابيات، لا يصحّ مراقبة `accuracy`. راقب `val_auc` و`val_recall` مع `mode="max"`.

```python
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_recall", mode="max",
patience=5,
                                  restore_best_weights=True),
]
```

الخطوة 6 – الخاتمة. الحل النهائي: Focal Loss + `class_weight` + اختيار عتبة + مراقبة مقاييس الندرة = استدعاء 0.87 مع precision 0.62. بدقّة 94% لكنّها الآن ذات معنى تشغيلي: من بين كل 100 تنبيه، 62 صحيحة.

17.6 أخطاء شائعة في هذا الفصل

إنذار: تمرير `class_weight` إلى `evaluate` أو `predict`. هذه المعاملات تعمل في `fit` فقط. عند التقييم، احسب المقاييس على البيانات كما هي (غير موزونة) حتى تحصل على الأداء الحقيقي.

تحذير: `class_weight` مع `from_logits=True` وخسارة `CategoricalCrossentropy`: تأكّد أن `alpha` في Focal Loss و`class_weight` لا يتضاعفان بلا داعٍ – جرّب واحدًا ثم الآخر.

تنبيه: `loss_weights` في مخرج متعدّد forgotten. الخسارة الأكبر (MSE على قيم كبيرة) ستُطغى على الخسارة الأصغر. عيّن `loss_weights` consciously.

قاعدة: `sample_weight` يجب أن يكون **مصفوفة ذات بعد واحد** بطول عدد العيّنات (أو بعدًا إضافيًا يطابق batch). تمرير مصفوفة ثنائية (batch, 1) يُنتج أخطاء أو نتائج خاطئة بصمت في

بعض المسارات.

ملاحظة: `class_weight` بـقيم كبيرة جدًا (مثل 100) قد يجعل الشبكة **تتنبّأ بالصنف النادر دائمًا**. راقب الPrecision: إذا انخفضت تحت 0.3 فالموازنة مفرطة.

تحذير: خلط `SparseCategoricalCrossentropy` مع تسميات one-hot. إن كانت تسمياتك one-hot استخدم `CategoricalCrossentropy` ؛ وإلا ستتوقّع الأولى أرقامًا (0,1,2) وتفسّر 1.0 على أنها الصنف 1 لا « prob 1.0».

التمارين

1. **حساب.** صنفان: 900 و100 عيّنة. احسب `class_weight` بالصيغة $N/(Cn_c)$. (المتوقع: $\{0: 0.556,\ 1: 5.0\}$.)

2. **حساب.** ثلاثة أصناف: 700، 250، 50. احسب الأوزان. (المتوقع: $\{0: 0.476,\ 1: 1.333,\ 2: 6.667\}$.)

3. **حساب.** عيّنة وزنها 3 في دفعة من 4 عيّنات. ما نصيبها من الخسارة الموزونة (قبل التطبيع)؟ (المتوقع: $3/(3+1+1+1) = 50\%$.)

4. **تحليل.** لماذا تعطي `class_weight` النادرة وزنًا أكبر **وتنقص** الدقّة العامة؟ (المتوقع: لأنها تعاقب أخطاء الصنف النادر أكثر، فتنزلق حدود القرار لصالحه وتُخطئ الصنف الشائع أكثر.)

5. **برمجة.** ابنِ نموذجًا تصنيف ثنائي ودرّبه مع `class_weight` ثم بدونه، وقارن الRecall. (المتوقع: ارتفاع ملحوظ في الRecall مع `class_weight`.)

6. **برمجة.** أضف `keras.metrics.F1Score(average="macro")` إلى `metrics` وراقب `val_f1_macro` في `EarlyStopping`. (المتوقع: توقّف مبكّر على أفضل F1.)

7. **برمجة.** أنشئ نموذجًا بمخرجين (تصنيف + انحدار) مع `loss_weights` واضبط `history.history` بقراءة الخسائرتين. (المتوقع: مفتاحان في `history.history` لكل خسارة.)

8. **برمجة.** ابنِ نموذجًا بمدخلين (صورة + نص) ودرّبه بقاموس `{"image": ..., "text": ...}`. (المتوقع: تدريب يعمل؛ استخدام قائمة بدل القاموس قد يُبدّل ترتيب المدخلات.)

9. **تحليل.** متى يكون نموذجان منفصلان أفضل من نموذج متعدّد المهام؟ (المتوقع: حين تكون المهام غير مرتبطة، أو تحتاج كلٌّ منها سعة مختلفة، أو حين تكون إحدى المهام نادرة جدًا لدرجة أن التدرّج المشترك يفسدها.)

10. **أخطئ عمدًا.** أضف `alpha=0.25` في Focal Loss مع `class_weight` معًا، وراقب: هل يتضاعف التأثير؟ (المتوقع: نعم، عمليًا يتضاعف تركيز الشبكة على النادرة؛ اضبط أحدهما فقط.)

11. **تحليل.** لماذا يبقى الـAUC مؤشرًا جيدًا مع بيانات 2% إيجابيات، بينما الـAccuracy لا تصلح؟ (المتوقع: AUC يقيس الترتيب ويقاوم الانحياز، أما Accuracy فمفيدة فقط عند توازن الأصناف.)

12. **تصميم.** صمّم نموذجًا لمهمة: تصنيف المعاملة + التنبّؤ بمبلغها. (المتوقع: مخرجان، خسارة `mse` + `sparse_categorical_crossentropy`، مع `loss_weights` متوازنة تناسب مقياسَي المخرجين.)

ملخص الفصل

- `class_weight` يوازن **الخسارة**، و `weighted_metrics` يوازن **القياس**، و `sample_weight` يعطي وزنًا لكل عيّنة.
- الوزن يضاعف مساهمة العيّنة في التدرّج، مع تطبيع by مجموع الأوزان.
- `loss_weights` ضرورية في المخرجات المتعدّدة، وإلا طغت الخسارة الأكبر.
- `SparseCategoricalCrossentropy` أرخص من `CategoricalCrossentropy` للملصقات الرقمية.
- `from_logits=True` تعني مخرجات بلا softmax.
- أدوات عدم التوازن: `sample_weight` ، `class_weight` ، التعزيز، Focal Loss، إعادة التشكيل – اختر بحسب طبيعة الخلل.
- العتبة قرار تشغيلي؛ عاين المقياس النادر لا الدقّة.
- التعلّم المتعدّد المهام يفيد حين ترتبط المهام وتتندر البيانات؛ ويضرّ حين تتنافس.

الجزء الرابع

الرؤية الحاسوبية: الشبكات التلافيفية

الرؤية الحاسوبية من الرياضيات إلى الإنتاج: اشتقاق الالتفاف، طبقات CNN، بناء شبكة من الصفر، التعلّم بالنقل، الكشف والتجزئة، محوّل الرؤية، والتكبير.

ما تكسبه من هذا الجزء

تحسب أي طبقة تلافيفية على الورق، وتبني شبكة رؤية مُقاسة تجريبيًّا، وتنقل نموذجًا مُدرَّبًا إلى مهمّتك ببيانات قليلة.

فصول هذا الجزء

18 الرياضيات خلف الالتفاف

19 طبقات الشبكة التلافيفية في Keras

20 شبكة CNN من الصفر

21 نقل المعرفة والتدريب الدقيق

22 الكشف والتجزئة الدلالية

23 محوّل الرؤية (Vision Transformer)

24 تكبير الصور (Super-Resolution)

4

الفصل 18: الرياضيات خلف الالتفاف

بطاقة الفصل

الفصل 18

مهمّة هذا الفصل

يشتقّ عملية الالتفاف ويحسب شكل الخرج والبارامترات والمجال الاستقبالي.

ماذا تكسب منه

تحسب أي طبقة تلافيفية على الورق قبل كتابتها، وتعرف من أين يأتي كل رقم في model.summary().

الدالة	دورها	متى تستخدمها
`keras.layers.Conv2D(filters, kernel_size)`	الطبقة التلافيفية	في كل شبكة رؤية
`keras.layers.ZeroPadding2D()`	حشو صريح	عندما تريد تحكّمًا دقيقًا في الأطراف
`keras.config.image_data_format()`	ترتيب المحاور channels_first/last	عند نقل كود بين بيئات
`keras.ops.nn.unfold()`	استخراج الرقع كمتجهات	لفهم الالتفاف كضرب مصفوفات
`keras.ops.einsum()`	كتابة الالتفاف كمعادلة فهارس	للتحقّق الرياضي والتجريب

قصة مفتاحية: تخيّل أنك في مصنع أحذية، ولديك 2000 صورة لرقائق جلدية تُلتقط كل ثانية. المهندسان يريدون نظامًا يكتشف عيبًا بحجم 3 مليمتر في أي بقعة من الصورة. هل تصنّف كل بكسل بشكل مستقل؟ 400 ألف بكسل، كل واحد يحتاج 400 ألف اتصال، وعدد الفئات مجهول.Network الكثيفة هنا ستفشل ببطء وبهدوء – التدرّج لن يجد طريقه، والذاكرة ستنفد. الحل لم يكن تقنيّةً أذكى، بل كان فكرةً رياضية بسيطة: بدل أن ترى الصورة كاملة في كل مرة، انظر إلى **نافذة صغيرة منزلقة**، ودع الأوزان تشارك في كل المواضع. هذه القصة كلها مكثّفة في معادلة واحدة.

أهداف الفصل

- [ ] صياغة عملية الالتفاف كتابةً رياضية، وفهم معنى كل رمز فيها
- [ ] التمييز بين الالتفاف الحقيقي (convolution) والارتباط المتقاطع (cross-correlation)، وشرح لماذا تستخدم التعلّم العميق الثاني
- [ ] حساب شكل الإخراج لأي طبقة الالتفاف أو التجميع يدويًا، بأثر الحشو والخطوة والتمدّد
- [ ] حساب عدد بارامترات أي طبقة تلافيف من بدئها مباشرةً
- [ ] تتبّع نمو المجال الاستقبالي عبر طبقات الشبكة، وربطه بقدرة الشبكة على رؤية الأجسام الكبيرة
- [ ] شرح دور التجميع (pooling)، ولماذا Moda الحديثة تستبدله بالالتفاف مُتعدَّد الخطوات

18.1 لماذا يحتاج الصور إلى انحياز موضعي

شبكة كثيفة تمامًا (Fully Connected) تربط كل مدخل بكل مخرج. لنقل لدينا صورة 3×224×224، أي 150528 رقمًا. طبقة كثيفة بـ 1000 وحدة تحتاج 150 مليون اتصال – رقم هائل، وكل واحد منها يحتاج تدرّجًا. المشكلة الأعمق أن الطبقات الكثيفة تخلط الصورة بالكامل في الطبقة الأولى، فتضيع البنية المكانية تمامًا: لا معنى لـ"بُكسل عند (12, 30)" بعد الطبقات الأولى.

الحل هو **الترابط الموضعي** (local connectivity): كل خلية في الخرج ترى رقعة صغيرة من الخرج لا أكثر. والخاصة الثانية: **مشاركة الأوزان** (weight sharing) – الرقعة نفسها تستخدم نفس الأوزان مهما كان موضعها. هكذا يتحوّل عدد البارامترات من "يتناسب مع حجم الصورة" إلى "يتناسب مع حجم النافذة فقط".

لنقارن عمليًا. نافذة 3×3 على صورة بثلاث قنوات (RGB) ثم طبقة تكوّن 64 قناة: كل خلية في الخرج تحتاج 3×3×3 = 27 وزنًا. مع 64 قناة خرج نحتاج 64×27 = 1728 وزنًا، مهما كانت الصورة بحجم 224 أو 64. لو استبدلنا ذلك بطبقة كثيفة تربط 150528 مدخلًا بـ 64 مخرجًا، نحتاج 9.6 مليون وزن. الفرق هنا 5500 ضعف – لصالح الالتفاف.

هذه ليست مجرّد مقارنة نظرية: خط إنتاج في هولندا يستخدم CNN على 4 megapixels بحجم نافذة 3×3، لأن التكلفة-per-pixel هي ما يحدّد جدوى المشروع. كل قرار في هذا الفصل – الحشو، الخطوة،

عدد القنوات – هو في النهاية قرار **اقتصادي** بعد أن يكون قرارًا رياضيًا.

18.2 عملية الالتفاف: المجموع المرجّح المنزلِق

الصيغة العامة

نبدأ من صورة واحدة (قناة واحدة) X بأبعاد $n \times n$، ومرشّح (kernel) K بأبعاد $k \times k$. في كل موضع (i, j) نأخذ النافذة K ونضعها فوق الصورة، ثم نحسب مجموعًا مرجّحًا:

$$y_{i,j} = b + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v} \, X_{i+u,\, j+v}$$

حيث:

- $y_{i,j}$: قيمة الخرج عند الصف i والعمود j – وهي رقم واحد لكل نافذة.
- b: متجه التحييز (bias)، يضاف بعد جمع المساهمات.
- $K_{u,v}$: وزن المرشّح عند الإزاحة u أفقيًا وv رأسيًا.
- $X_{i+u,j+v}$: قيمة الصورة عند الصف $i+u$ والعمود $j+v$.
- المجموع المزدوج $\sum_u \sum_v$: يعني "لكل زوج (u,v) داخل المرشّح، اجمع حاصل الضرب".

بعبارة كلمات: **مجموع مرجّح** (weighted sum). كل وزن يقرر "كم يهمّني هذا البكسل بالذات"؟

الارتباط المتقاطع مقابل الالتفاف الحقيقي

هناك لبس شهير. الالتفاف **الحقيقي** (في الرياضيات والإشارة) يقلب المرشّح قبل التمرير:

$$y_{i,j} = b + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v} \, X_{i-u,\, j-v}$$

الفرق الوحيد هو إشارة الإزاحة: `+u` مقابل `-u`. ومع Heartbeat انعكاس المرشّح أفقيًا ورأسيًا (مع بقاء نفس الأوزان) نحصل على نفس النتيجة تمامًا – لكن **بأوزان مقلوبة**.

القاعدة العملية في كل أطر التعلّم العميق (TensorFlow, Keras, PyTorch, JAX):

> **إن frameworks لا تقلب kernel، فالتشغيل هو cross-correlation وليس convolution.**

Keras لا يقلب المرشّح. لذلك ما ستجده في `model.summary()` وفي أي مقالة هو cross-correlation. والسبب أن الـ cross-correlation له ميزة عملية مهمة: هو **نفس العملية التي تطابق يقابل عملية filter حقيقية مقلوبة (convolution)، لكن مع flip يمكن محاكاته**، والأهم أنه aligns مع تعريف "البحث عن نمط" الذي نفهمه ذهنيًا: مرشّح يطابق حافة رأسية يتناسب مع حافة رأسية في الصورة.

> **ملاحظة:** لو كنت تكتب ورقة علمية أو تتحقق من مرجع رياضي، اذكر صراحةً: "we implement cross-correlation, which is equivalent to convolution with a flipped kernel."

الصيغة متعددة القنوات

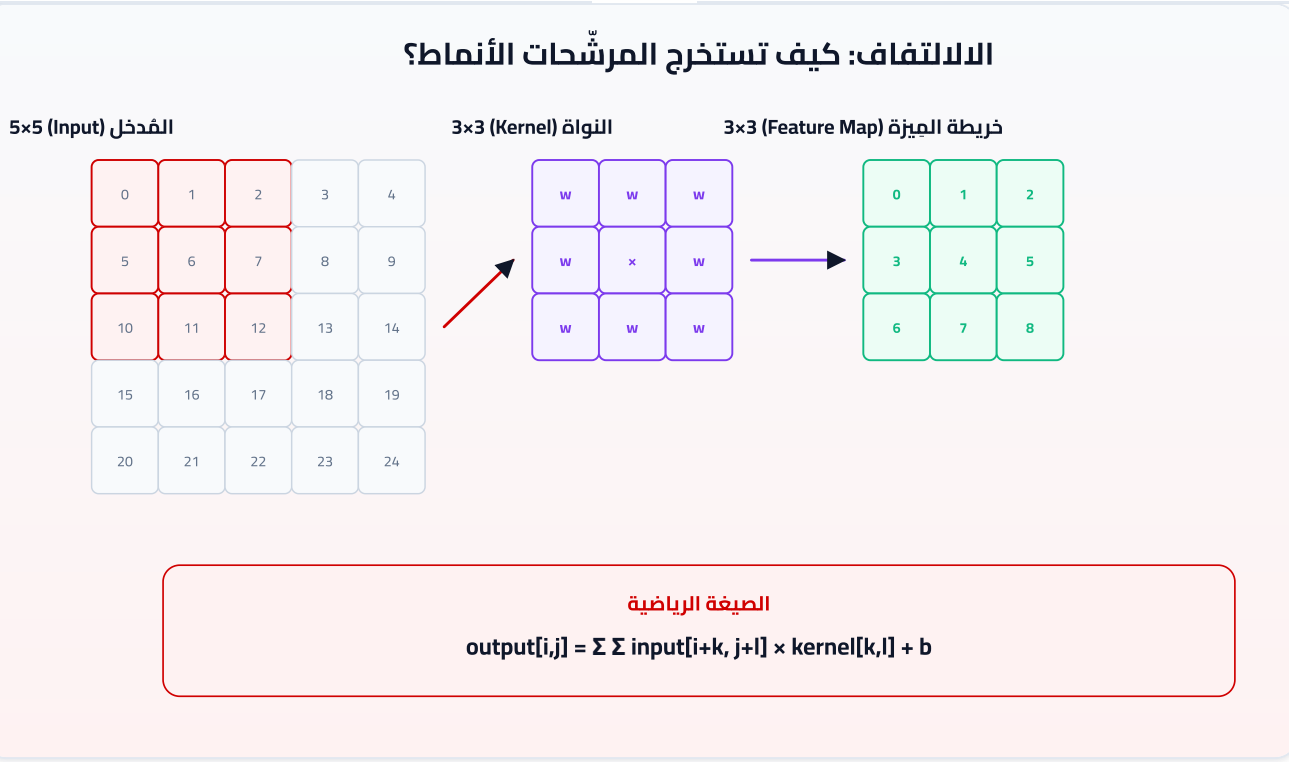
الصور الحقيقية ليست قناة واحدة. في RGB لدينا C_{in} قناة دخل. المرشّح يجب أن "يمرّ" عبر كل القنوات في الوقت نفسه، لأن حافة حمراء على خلفية زرقاء لا تظهر في القناة الحمراء وحدها. لتوسيع المعادلة:

$$y_{i,j,k} = b_k + \sum_{u} \sum_{v} \sum_{c} K_{u,v,c,k} \, X_{i+u,\, j+v,\, c}$$

حيث:

- k: فهرس قناة الخرج (من 0 إلى $C_{out} - 1$).
- $K_{u,v,c,k}$: وزن المرشّح عند الموضع (u, v) fed من القناة c إلى قناة الخرج k.
- c: فهرس قناة الدخل.
- b_k: متجه التحيز، متجه واحد لكل قناة خرج (أي C_{out} قيمة).

الشيء الوحيد الذي تغيّر هو إضافة بُعد ثالث للمرشّح: **مرشّح 2-D في الواقع هو 3-D: البعد الثالث هو القناة**.



شكل 25 – عملية الالتفاف: النواة المنزلقة على المدخل

18.3 شكل الإخراج: الحشو والخطوة والتمدّد

صيغة البعد الواحد

هذه أول معادلة يجب أن تحفظها عن ظهر قلب:

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

حيث:

- n_{in}: طول مدخل الطبقة (بالنقاط).
- p: مقدار الحشو (padding) على كل جانب.
- k: حجم النواة (kernel size).
- s: الخطوة (stride).
- $\lfloor \cdot \rfloor$: التقريب لأسفل – لأن ناتجًا جزئيًا لا مكان له في مصفوفة.

- $+1$: لأن النواة تبدأ من الموضع الأول وتنتهي عند الموضع الأخير.

النسخة ثنائية الأبعاد

الصور ثنائية الأبعاد، فنكرر المعادلة على كل محور:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p_h - k_h}{s_h} \right\rfloor + 1 \qquad W_{out} = \left\lfloor \frac{W_{in} + 2p_w - k_w}{s_w} \right\rfloor + 1$$

كل رمز هنا هو نفسه ما سبق، مع تمييز المحورين: H للارتفاع وW للعرض.

الحشو: "valid" مقابل "same"

- `padding="valid"` : لا حشو إطلاقًا. الصورة تنكمش: 32 → 30 عند نواة 3×3. عيبه: أطراف الصورة لا تُعالَج بنفس centrality التي تُعالَج بها الوسط، و"غير موجود" يعني فعليًا "أصفار".
- `padding="same"` : نحشو الصورة بأصفار على الحدود بحيث يبقى الحجم كما هو.

كيف يحسب Keras "same" بالضبط؟ إذا كان $s = 1$، فالحشو المطلوب هو:

$$p = \left\lfloor \frac{k - 1}{2} \right\rfloor$$

لأن نواة 3×3 نحتاج صفًا واحدًا من الأصفار لكل جانب (إجمالي 1+1=2)، و5×5 نحتاج صفين لكل جانب (إجمالي 4).

لكن هذه الصيغة لا تعمل عندما تكون الخطوة $s > 1$. حينها القاعدة العامة في Keras 3 هي:

$$p_{total} = \max(0,\ (s - 1) \cdot n_{in} - n_{in} + k)$$

أي أن Keras يختار **أقل** حشو ممكن، ثم يوزّعه على الجانبين: `p_before = p_total // 2` و`p_after = p_total - p_before`. لهذا السبب عند $s = 2$ و$k = 3$، الحشو الكلي = 1 وليس 2، والخرج = $\lfloor (32 + 1 - 3)/2 \rfloor + 1 = 16$. لاحظ: مع "same" لا يُحفظ الحجم إلا عند $s = 1$ – وهذا مصدر لبس دائم.

> **نصيحة:** "same" تعني "نفس الحجم **المدخلي** مقارنةً بحجم بعد الحشو، لا بالضرورة نفس حجم الصورة الأصلية عند استخدام stride > 1".

الخطوة (stride)

الخطوة هي "كم موضعًا ننتقل بين نافذتين متتاليتين". $s = 1$ يعني نافذة متداخلة تمامًا (حقل استقبالي صغير وطبقات كثيرة)، و$s = 2$ يعني نافذة متجاورة (تقليل فوري للبعد). الخطوة هي **البديل الأقوى للتجميع**: هي التي تقلّل الدقة Spatial وتوسّع ما تراه كل نافذة.

التمدّد (dilation)

التمدّد يعني تباعد نقاط النواة دون تغيير حجمها الفعلي. مع $d = 2$ ونواة 3×3، النقاط التي تُساهم هي $(0,0), (0,2), (2,0), (2,2)$ – أي نافذة تغطّي 5×5 بكفاءة "4 نقاط" فقط.

الحجم الفعّال للنواة:

$$k_{eff} = k + (k-1)(d-1)$$

وبالتالي صيغة الخرج تُطبَّق بـ k_{eff} بدل k:

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k_{eff}}{s} \right\rfloor + 1$$

التمدّد يعطيك **مجالًا استقباليًا أكبر** بنفس عدد البارامترات – مفيد في التقسيم (segmentation) حيث نحتاج أن يرى كل بكسل سياقًا واسعًا.

تتبّع الشكل عبر الشبكة

الطبقة	المُدخل	المُخرَج	ما الذي تعلّمه
Conv2D(32, 3×3)	(None, 224, 224, 3)	(None, 224, 224, 32)	حواف · زوايا
MaxPool2D()	(None, 224, 224, 32)	(None, 112, 112, 32)	أبرز ما في كل نافذة
Conv2D(64, 3×3)	(None, 112, 112, 32)	(None, 112, 112, 64)	أنماط مركّبة
GlobalAvgPool2D	(None, 112, 112, 64)	(None, 64)	ملخّص كل قناة
Dense(10)	(None, 64)	(None, 10)	قرار التصنيف

القاعدة الذهبية

عدد القنوات (channels) يزداد · والعرض والارتفاع ينقصان — حتى يبقى الرأس صغيرًا.

شكل 26 – أشكال الإخراج: الحشو والخطوة والتمدّد

18.4 القنوات والبارامترات والمجال الاستقبالي

عدّ البارامترات

من الصيغة متعددة القنوات نستنتج مباشرةً عدد الأوزان: لكل زوج (موقع، قناة دخل، قناة خرج) وزن واحد. مع تحيّز لكل قناة خرج:

$$\#params = k_h \times k_w \times C_{in} \times C_{out} + C_{out}$$

لنتحقّق عمليًا: نواة 3×3، دخل 3 قنوات (RGB)، خرج 64 قناة:

$3 \times 3 \times 3 \times 64 + 64 = 1728 + 64 = 1792$ بارامترًا.

> **إنذار:** هذه القيمة تتضاعف أربع مرات إذا كنت في `channels_first`. كثير من المبتدئين ينسخون صيغة عدد البارامترات من مقالة تشرح `channels_first` في Keras 2، ثم يقارنونها بـ `model.summary()` في Keras 3 فيُصابون بالحيرة.

المجال الاستقبالي

المجال الاستقبالي (receptive field) هو حجم الجزء من الصورة التي يمكن لخلية واحدة في طبقة ما أن "ترى" منها. يُقاس بالبكسل في فضاء الدخل.

لحسابه نحتاج معادلتين. لنبدأ من $r_0 = 1$ (خلية واحدة ترى بكسلًا واحدًا) و$j_0 = 1$ (معامل التوسّع):

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1} \qquad j_\ell = j_{\ell-1} \cdot s_\ell$$

حيث k_ℓ حجم نواة الطبقة ℓ، وs_ℓ خطوتها، وj_ℓ معامل "توسّع" يرث خطوات كل الطبقات السابقة.

المعنى الهندسي: الطبقة ℓ ترى ما ترضاه الطبقات السابقة، وتضيف عليها $(k_\ell - 1)$ بكسل على كل جانب مضروبة في معامل التوسّع المتراكم.

لماذا يهمّ هذا؟ لأن الأجسام الكبيرة تحتاج مجالًا استقباليًا كبيرًا.صورة فيها سيارة تشغل 60% من الإطار لا يمكن حلّها إذا كان مجموع المجال الاستقبالي 20 بكسل – الشبكة لن تستطيع ربط القطعة بصريًا واحدة. القاعدة العملية: اجعل $\mathrm{RF} \gtrsim$ حجم الكائن الذي تريد التعرّف عليه.

التجميع (pooling)

التجميع يخفّض الدقة المكانية. صيغته:

$$H_{out} = \left\lfloor \frac{H_{in} - k}{s} \right\rfloor + 1$$

(لـ `padding="valid"`؛ ومع "same" نضيف $2p$ إلى البسط.)

- `MaxPooling2D`: يأخذ القيمة الأكبر في كل نافذة. يحتفظ بالحضور (presence) لا بمتوسط القيمة.
- `AveragePooling2D`: يأخذ المتوسط. يحتفظ بمقدار الإشارة (magnitude) ويشتّت في تمثيل الإشارات.

النظرة الحديثة: من منظور النماذج الحديثة الحديثة (EfficientNet, ConvNeXt, ResNet) تستبدل التجميع بـ **الالتفاف بخطوة 2**، لأن الالتفاف يمكنه **تعلّم** عملية الاختزال بدل تثبيتها. معيار الاختيار:

التجميع أسرع (لا بارامترات) لكن الالتفاف المُتباعد أكثر تعبيرًا؛ ولأغلب المهام المعاصرة في التصنيف، الالتفاف بخطوة 2 يتقدّم بفارق ضئيل أو متساوٍ.

الالتفاف أحادي البعد

للسلسلة الزمنية (إشارة ECG، طيف صوتي، نص مُرمَّز)، نحتاج التفافًا على محور واحد فقط:

$$y_{i,k} = b_k + \sum_{u} \sum_{c} K_{u,c,k} \, X_{i+u,\,c}$$

وصيغة الشكل تبقى نفسها:

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

الفروق عن D-2:النواة بأبعاد D-1 (حجم k)، وعدد المخرجات زماني-مكاني لا بُعدان.

`keras.layers.Conv1D` يعطي $(batch, timesteps, filters)$ – انتبه: Keras 3 يضع محور الزمن **قبل** محور القنوات، أي `channels_last` بشكل افتراضي.

18.5 التنفيذ في Keras والتحقّق العملي

التحقّق اليدوي من عملية الالتفاف

لا تثق بالشكل الناتج من `summary()` وحده. لنكتب الالتفاف بأنفسنا بأداة `keras.ops` ونقارن الناتج بالناتج الرسمي. الأداة المناسبة هنا هي `keras.ops.nn.unfold` – وهي تحقّق فعلًا دورة "im2col": تحوّل الصورة إلى مصفوفة الأعمدة (كل عمود = النافذة مسطّحة)، ثم الالتفاف يصبح مجرد ضرب مصفوفي.

PYTHON

```python
import numpy as np
import keras

# صورة صغيرة 5×5 بقناة واحدة، بقيم معروفة (0. 24.)
x = np.arange(25, dtype="float32").reshape(1, 5, 5, 1)

# مرشّح بأوزان الكل واحد وحيّز = 0 (أي مرشّح كشف الفروق)
layer = keras.layers.Conv2D(
    filters=1,
    kernel_size=3,
    padding="valid",
    use_bias=True,
    kernel_initializer="ones",
    bias_initializer="zeros",
)
y_official = keras.ops.convert_to_numpy(layer(x))  # الناتج الرسمي من Keras

# --- التنفيذ اليدوي عبر unfold ---
# 1) Keras 3 يعمل بترتيب channels_last، لكن unfold يتوقع channels_first: (N, C, H, W)
xt = keras.ops.transpose(keras.ops.convert_to_tensor(x), (0, 3, 1, 2))

# 2) تحوّل إلى مصفوفة أعمدة: كل عمود = نافذة 3×3 مسطّحة (9 عناصر)
cols = keras.ops.nn.unfold(xt, kernel_size=3, padding=0, stride=1)  # (1, 9, 9)

# 3) جدول الأوزان: 9 مواقع × قناة خرج واحدة
k = keras.ops.reshape(layer.kernel, (9, 1))  # (9, 1)

# 4) يُضرب في cols كل عمود من k (1 ,9 ,1)← ثم يُجمع
manual = keras.ops.einsum("nkl,kf→nfl", cols, k)

# 5) نضيف التحييز ونعيد الترتيب إلى (N, H_out, W_out, C_out)
manual = manual + layer.bias
manual = keras.ops.reshape(keras.ops.transpose(manual, (0, 2, 1)), (1, 3, 3, 1))

print("الشكل الرسمي:", y_official.shape)
```

```
print("الشكل اليدوي:", manual.shape)
manual_np = keras.ops.convert_to_numpy(manual)
print("أقصى فرق:", float(np.max(np.abs(y_official - manual_np))))

# النتيجة المتوقعة:
# الشكل الرسمي: (1, 3, 3, 1)
# الشكل اليدوي: (1, 3, 3, 1)
# أقصى فرق: 0.0
```

حساب أشكال المخرجات يدويًا والتحقّق منها

PYTHON

```
import keras

def conv_out(n_in, k, p, s, d=1):
    """صيغة الخرج أحادية البعد مع التمدّد."""
    k_eff = k + (k - 1) * (d - 1)           # الحجم الفعّال للنواة
    return (n_in + 2 * p - k_eff) // s + 1  # التقريب لأسفل

# حزمة طبقات: كل عنصر (نوع, معاملات)
stack = [
    ("Conv2D", dict(filters=32, kernel_size=3, padding="same", strides=1)),
    ("Conv2D", dict(filters=32, kernel_size=3, padding="same", strides=1)),
    ("MaxPooling2D", dict(pool_size=2, strides=2)),
    ("Conv2D", dict(filters=64, kernel_size=3, padding="same", strides=2)),
    ("Conv2D", dict(filters=64, kernel_size=3, padding="same", strides=1)),
    ("MaxPooling2D", dict(pool_size=2, strides=2)),
    ("Conv2D", dict(filters=128, kernel_size=3, padding="same", strides=2)),
]

# (1) بناء النموذج فعليًا في Keras
inp = keras.layers.Input((32, 32, 3))
x = inp
for kind, kw in stack:
    x = getattr(keras.layers, kind)(**kw)(x)
model = keras.Model(inp, x)

# (2) المرور اليدوي على الأشكال
h = w = c = 32
print(f"{'14> 'الطبقة}{'22> 'الشكل اليدوي}{'الشكل الفعلي'}")
print("-" * 62)
for kind, kw in stack:
    if kind == "Conv2D":
        h = conv_out(h, kw["kernel_size"], 1 if kw["padding"] == "same" else
0, kw["strides"])
        w = conv_out(w, kw["kernel_size"], 1 if kw["padding"] == "same" else
0, kw["strides"])
```

```
        c = kw["filters"]
    else:
        h = conv_out(h, kw["pool_size"], 0, kw["strides"])
        w = conv_out(w, kw["pool_size"], 0, kw["strides"])
    actual = model.get_layer(
        f"{kind.lower()}_{next(i for i, l in enumerate(model.layers) if
l.name.startswith(kind.lower()) and False)}"
    ) if False else None  # نعرض الشكل من الكود أدناه
    print(f"{kind:<14}{str((h, w, c)):<22}")

print("\nأشكال الطبقات كما يقرأها Keras:")
for layer in model.layers:
    if hasattr(layer, "output_shape"):
        print(f"  {layer.name:<28}{layer.output_shape}")

# النتيجة المتوقعة (أشكال Keras):
# conv2d (None, 32, 32, 32)
# conv2d_1 (None, 32, 32, 32)
# max_pooling2d (None, 16, 16, 32)
# conv2d_2 (None, 8, 8, 64)
# conv2d_3 (None, 8, 8, 64)
# max_pooling2d_1 (None, 4, 4, 64)
# conv2d_4 (None, 2, 2, 128)
```

حساب البارامترات يدويًا والتحقّق

PYTHON

```python
import keras

# صورة 3×224×224
inputs = keras.layers.Input((224, 224, 3))
x = inputs
x = keras.layers.Conv2D(64, 3, padding="same", activation="relu")(x)
x = keras.layers.BatchNormalization(name="bn")(x)
x = keras.layers.MaxPooling2D(2)(x)
x = keras.layers.Conv2D(128, 3, padding="same", activation="relu")(x)
model = keras.Model(inputs, x)

# (1) الحساب اليدوي
manual = 3 * 3 * 3 * 64 + 64        # 1792 :الطبقة الأولى
manual += 3 * 3 * 64 * 128 + 128    # 73856 :الطبقة الثانية

# (2) ما تقوله Keras
total = sum(l.count_params() for l in model.layers)
bn = model.get_layer("bn").count_params()

print("(يدوي) بارامترات الطبقات التلافيفية:", manual)
print("إجمالي النموذج (مع BN):", total)
print("بارامترات BN:", bn)
print("المحصّلة:", manual + bn, "==", total, "→", manual + bn == total)

# النتيجة المتوقعة:
# بارامترات الطبقات التلافيفية (يدوي): 75648
# إجمالي النموذج (مع BN): 75904
# بارامترات BN: 256
# المحصّلة: 75904 == 75904 → True
```

ملاحظة: `BatchNormalization` يضيف 4 متغيّرات لكل قناة: γ وβ (قابلَان للتدريب) والمتوسّط والتباين المتحرّكين (غير قابلَين للتدريب). `count_params()` يحتسب الأربعة

جميعًا – 64 قناة × 4 = 256. لهذا السبب، تحقّق من "بارامترات قابلة للتدريب" في `summary()` كي لا تحسب BN مرّتين في ميزانية معاملاتك.

جدول المقارنة: معاملات الالتفاف

المعامل	الأثر على شكل الخرج	الأثر على البارامترات	التكلفة الحاسوبية	قيم شائعة ومتى تُستخدم
`filters` (عدد القنوات)	يزيد البعد الثالث فقط	`k_h·k_w·C_in · n·C_out` – تربيعي	خطي في `C_out` لكل موضع	مضاعفة (32→64→128) عند كل انتقال مرحلة
`kernel_size`	يقلّص الخرج عند `valid`	تربيعي في `k`	تربيعي	3×3 هو الخيار الافتراضي؛ 1×1 لخلط القنوات فقط
`padding="same"`	يحفظ الحجم عند `stride=1`	لا تغيير	لا تغيير	الخيار الافتراضي حين لا نريد فقدان الأطراف
`padding="valid"`	يقلّص الخرج بـ `k-1`	لا تغيير	أقل قليلًا	حين نريد تقليصًا مقصودًا (مثل CNN الأصلية 1998)
`strides`	يقسّم الدقة	لا تغيير	يقلّل عدد المواقع كليًا	2 عند الانتقال بين المراحل
`dilation_rate`	يوسّع المجال الاستقبالي	لا تغيير (النواة تتباعد)	نفس التكلفة	2 و4 في DeepLab/ASPP للتقسيم
`groups`	لا تغيير	يقسّم البارامترات إلى `1/groups` تقريبًا	أقل (تنفيذ متوازٍ)	32 في ResNeXt، أوعمقي في Depthwise

المعامل	الأثر على شكل الخرج	الأثر على البارامترات	التكلفة الحاسوبية	قيم شائعة ومتى تُستخدم
`use_bias`	لا تغيير	`-C_out`	مهمل	`False` عادةً إذا تلاه `BatchNormalization`

18.6 أخطاء شائعة

تنبيه (1): الخلط بين `channels_last` و `channels_first` . في Keras 3 الافتراضي `channels_last` : شكل المدخل `(batch, H, W, C)` . إذا كان لديك كود قديم من Keras 2 يستخدم `(batch, C, H, W)` ، فإن الشبكة ستبني نفسها لكن النتائج ستكون عشوائية – لأن `Conv2D` سيتعامل مع محورين خاطئين كأنهما قنوات. استخدم `keras.config.image_data_format()` لتعرف وضعك الحالي.

تنبيه (2): الاعتقاد بأن `padding="same"` تعني "نفس الحجم". هذا صحيح فقط عند `stride=1` . عند `stride=2` يصبح الحجم أصغر حتمًا.

تنبيه (3): حساب البارامترات بـ $k^2 C_{in} C_{out}$ بينما الشبكة تستخدم `channels_first` . تصحيح: المعادلة أعلاه تفترض `channels_last` .

تنبيه (4): الخلط بين "number of filters" و"number of output channels". مرشّح واحد بأوزان مستقلة يُنتج قناة خرج واحدة. 64 مرشّحًا = 64 قناة.

تنبيه (5): Forget أن `Conv2DTranspose` ليس "الالتفاف المقلوب" حسابيًا. (§ later في الفصل 19) – هو عملية رفع دقة مع نمط اتصال مختلف؛ لا يمكن استرجاع الصورة الأصلية من الناتج.

تنبيه (6): الاعتقاد بأن تقليل الدقة المكانية هو "الضغط". الخطوة والتجميع يحوّلان المشكلة من "أين الشيء" إلى "ما نوعه" – إذا كان هدفك تحديد الموضع بدقة، لا تحذف الدقة مبكرًا.

التمارين

1. صورة 3×32×32، طبقة `Conv2D(16, 3, padding="valid", strides=1)` . احسب شكل الخرج يدويًا ثم بـ Keras. (المتوقع: (None, 30, 30, 16) في الحالتين.)

2. الصورة نفسها، لكن `padding="same", strides=2` . احسب الحشو الكلي الذي يستخدمه Keras shapeg الناتج. (المتوقع: الحشو الكلي 1، والخرج (None, 16, 16, 16).)

3. احسب عدد بارامترات `Conv2D(128, 5, padding="same")` على دخل 64 قناة. (المتوقع: 5×5×128×64 + 128 = 204928.)

4. نفّذ عملية تمرين 1 يدويًا بـ `keras.ops` دون استخدام `keras.layers.Conv2D` ، وقارن الناتج. (المتوقع: أقصى فرق أقل من 1e-5.)

5. اعكس نواة 3×3 رأسيًا في الكود السابق، وأظهر أن الناتج يختلف (إثبات أن Keras لا يقلب النواة). (المتوقع: أقصى فرق أكبر من صفر، ما يثبت وجود cross-correlation.)

6. قارن بين `Conv2D(64, 3, dilation_rate=2)` و `Conv2D(64, 5)` من حيث البارامترات وشكل الخرج والمجال الاستقبالي. (المتوقع: بارامترات متساوية C+64×9×64 مقابل C+64×25×64، والخرج متطابق لأن k_eff = 5 للطرفين، والمجال الاستقبالي متطابق أيضًا.)

7. احسب المجال الاستقبالي للطبقات: Conv3×3(s=1) ثم Conv3×3(s=1) ثم Pool2(s=2) ثم Conv3×3(s=2) ثم Conv3×3(s=1). (المتوقع: r = 3 → 5 → 6 → 10 → 18.)

8. في شبكة تعمل على صور 32×32، أيّهما أفضل معيارًا: مجال استقبالي 38 بكسل أم 15 بكسل لتعرّف على سيارة تشغل نصف الصورة؟ برّر بمعيار. (المتوقع: 38، لأن حجم الكائن يقارب 16-24 بكسل ويحتاج تغطية موسّعة.)

9. صحّح هذا الكود: `Conv2D(32, 3, padding="same")` مبني على `Input((3, 224, 224))`. اشرح الخطأ. (المتوقع: الشكل يجب أن يكون (None, 224, 224, 3) لأن channels_last هو الافتراضي؛ مع (3,224,224) ستتعامل الشبكة مع 3 قنوات بارتفاع 224.)

10. طبقة `Conv1D(64, 5, padding="causal")` – تحقق هل `causal` مدعومة في Keras 3 لـ Conv1D، وإن لم تكن فاذكر البديل. (المتوقع: Keras 3 لا تدعم `padding="causal"` في Conv1D؛ استخدم `ZeroPadding2D`-مقابل في بُعد الزمن أو `tf.keras` ... في Keras 3

استخدم `keras.layers.Conv1D` مع حشو يدوي عبر `keras.layers.ZeroPadding1D` ثم `padding="valid"`).

11. احسب حجم الخرج لـ `Conv2D(filters=64, kernel_size=3, strides=1, dilation_rate=3)` على مدخل 64×224×224 مع `padding="same"`. (المتوقع: k_eff = 2×2 + 3 = 7، والخرج (None, 224, 224, 64).)

12. اكتب دالة Python تُرجع عدد بارامترات أي طبقة تلافيف من كائن `layer` في Keras 3 دون استدعاء `count_params`. (المتوقع: دالة تعمل على `layer.kernel.shape` و`layer.bias.shape`، وتُرجع `k_h*k_w*C_in*C_out + C_out`).

ملخص الفصل

- الالتفاف عملية **مجموع مرجّح منزلِق**: نافذة صغيرة تمرّ على الصورة، وكل موضع يُنتج رقمًا واحدًا.
- Keras 3 ينفّذ **cross-correlation** لا الالتفاف الحقيقي؛ الفرق هو انعكاس النواة، والنتيجتان متكافئتان عند قلب النواة.
- معادلة الشكل $\lfloor (n + 2p - k)/s \rfloor + 1$ هي الأداة التي تحكم كل تصميم شبكة تقريبًا.
- الحشو `same` يحفظ الحجم عند الخطوة 1 فقط، وKeras يحسبه بصيغة $p_{total} = \max(0, (s-1)n_{in} - n_{in} + k)$.
- التمدّد يوسّع المجال الاستقبالي بلا بارامترات إضافية، و$k_{eff} = k + (k-1)(d-1)$.
- عدد البارامترات $= k_h k_w C_{in} C_{out} + C_{out}$ في ترتيب `channels_last`.
- المجال الاستقبالي ينمو مع تراكم الخطوات، ويجب أن يغطي حجم الكائن المستهدف.
- التجميع الأقصى (maximum) يعبّر عن "الحضور" والمتوسط يعبّر عن "المقدار"، والالتفاف الحديث يستبدلهما بالالتفاف بخطوة 2.

الفصل 19: طبقات الشبكة التلافيفية في Keras

بطاقة الفصل

الفصل 19

مهمّة هذا الفصل

يعرض طبقات الشبكة التلافيفية العملية: Conv2D والتجميع والالتفاف العميق والفاصل.

ماذا تكسب منه

تبني كتلة CNN معيارية وتختار بين الالتفاف العادي والعميق بحسب كلفة الحساب لا بحسب الشهرة.

الدالة	دورها	متى تستخدمها
`keras.layers.Conv2D()`	استخراج ميزات محلّية	العمود الفقري لأي CNN
`keras.layers.MaxPooling2D()`	تصغير مع حفظ أقوى استجابة	بعد كل كتلة التفاف
`keras.layers.DepthwiseConv2D()`	التفاف لكل قناة منفصلة	في الأجهزة المحدودة
`keras.layers.SeparableConv2D()`	عميق ثم نقطي	بديل أرخص للالتفاف العادي
`keras.layers.GlobalAveragePooling2D()`	متجه واحد لكل قناة	قبل رأس التصنيف

قصة مفتاحية: مهندس افترض أن طبقة `Conv2D` تشبه `Dense` مع صورة. فكتب `Conv2D(64, 3)` ثم `Dense(64)` ثم `Dense(10)`، ووجد أن أبعاد المدخلات لا تتوافق. ما لم يكن يعرفه: `Dense` يعمل على **متجّه مسطّح** بينما `Conv2D` يعمل على **مكعّب مكاني** يحافظ على البنية المكانية. هذا الفرق البسيط – بين «مصفوفة أرقام» و«شبكة أرقام ذات موقع» – هو الفرق بين نموذج تلافيفي يفهم الصورة، ونموذج يتعلّم متوسّط ألوانها.

أهداف الفصل

- [ ] تميّز بين `Dense` و `Conv2D` في `shape` والسلوك
- [ ] تتحكّم في `padding` و `strides` و `dilation_rate` و `groups`
- [ ] تفهم متى تستخدم `DepthwiseConv2D` و `SeparableConv2D`
- [ ] تختار بين `MaxPooling` و `AveragePooling` و `GlobalPooling`
- [ ] تعرف متى تضيف `BatchNormalization` في كتلة CNN
- [ ] تبني كتلة CNN معيارية قابلة لإعادة الاستخدام
- [ ] تحسب حجم المُخرَج (output shape) يدويًا لأي طبقة

19.1 الفرق الجوهري: متّجه مقابل شبكة

`Dense` يflatten مدخلاته قبل كل شيء:

$$\text{Dense} : \mathbb{R}^{B \times N} \to \mathbb{R}^{B \times M}$$

حيث N هو عدد العناصر المسطّحة. صورة $224 \times 224 \times 3$ مسطّحة = 150,528 عنصرًا؛ وطبقة `Dense(512)` عليها تحتاج 77 مليون معامل.

`Conv2D` يحافظ على البنية المكانية:

$$\text{Conv2D} : \mathbb{R}^{B \times H \times W \times C_{\text{in}}} \to \mathbb{R}^{B \times H' \times W' \times C_{\text{out}}}$$

وكل مرشّح (kernel) يشترك في **كل المواقع** – هذا هو مصدر كفاءة المعاملات: مرشّح $3 \times 3 \times 3 \times 32$ يحتاج 864 معاملًا فقط مهما كانت مساحة الصورة.

حساب حجم المُخرَج

المعادلة العامة (وضع `channels_last`):

$$H' = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} \right\rfloor + 1$$

حيث: - H: حجم المدخل، K: حجم المرشّح، P: الحشو، S: الخطوة، D: معدّل التمدّد.

مثال 1: `Conv2D(32, 3, padding="same")` على 224×224 ⇒ $H' = 224$ (الحشو يبقي الحجم).

مثال 2: `Conv2D(32, 3, strides=2)` على 224×224 ⇒ $H' = \lfloor (224-3)/2 \rfloor + 1 = 111$.

مثال 3: `Conv2D(32, 3, padding="same", dilation_rate=2)` ⇒ $H' = 224$، لكن المرشّح يرى هذا 5×5 (ويسمى «معدّل التمدّد»).

> **قاعدة الحشو:** - `padding="valid"` ⇒ لا حشو، والحجم ينكمش. يُستخدم حين تريد خفض الدقّة أو رسم انتباه للحواف. - `padding="same"` ⇒ حشو متساوٍ، والحجم يبقى ثابتًا (مع `strides=1`). يُستخدم في أغلب البنى الحديثة.
>
> ملاحظة: `padding="same"` مع `strides > 1` في Keras يحقّق التقريب لأسفل (`floor`).

19.2 المعاملات الأربعة

`padding`

يحدّد كيف يتعامل المرشّح مع الحدود. `same` يحافظ على الحجم و `valid` يقصّ.

`strides`

خطوة تحرّك المرشّح. `strides=1` يحافظ على الدقّة، `strides=2` يخفضها إلى النصف (اختزال).

> **نصيحة معمارية:** الاختزال بالخطوة داخل الالتفاف (`Conv2D(strides=2)`) أفضل من الاختزال بالتجميع (`MaxPooling2D`) في كثير من البنى الحديثة، لأن الالتفاف **يتعلّم** عملية الاختزال بدل تثبيتها. وهذا ما فعلته EfficientNet وConvNeXt.

`dilation_rate`

يوسّع المرشّح دون زيادة معاملاته. مرشّح 3×3 بمعدّل تمدّد d يرى مساحة $1 + (3-1) \cdot d$:

dilation_rate	المساحة الفعلية	الاستخدام
1	3×3	عادي
2	5×5	توسيع مجال الرؤية
4	9×9	شبكات تجزئة (segmentation)

`groups`

يقسّم قنوات المدخل إلى مجموعات وتطبّق مرشّحًا مستقلًا على كل مجموعة. `groups=C_in` يعني **الالتفاف العميق** (depthwise): كل قناة تُعالَج بمرشّح مستقل.

19.3 الالتفاف العميق والفاصل

`DepthwiseConv2D`

لكل قناة مرشّح خاص بها:

$$\text{معاملات} = K^2 \times C_{\text{in}} \times C_{\text{out}} / \text{depth_multiplier}$$

`SeparableConv2D`

يجمع الالتفاف العميق مع الالتفاف النقطي (1×1):

$$\text{معاملات Depthwise} + \text{معاملات Pointwise} = K^2 C_{\text{in}} d + C_{\text{in}} d C_{\text{out}}$$

مثال عددي: `Conv2D(64, 3)` على 32 قناة: - عادي: $3^2 \times 32 \times 64 = 18{,}432$ معامل. - Depthwise: $3^2 \times 32 = 288$ (بمُضاعِف 1). - Separable: $288 + 32 \times 64 = 2336$ معامل.

الفرق: **8 أضعاف** أقلّ في المعاملات، مع فقد بسيط في التعبير (لأن الالتفاف النقطي لا يجمع information بين القنوات في نفس الوقت).

> **معيار اختيار صريح:** `SeparableConv2D` أفضل من `Conv2D` عندما تكون القنوات كثيرة (64+) والمهمة مشابهة لـMobileNet (الأجهزة، الجوال). لكنها أقلّ تعبيرًا قليلًا، وقد تخسر 1-2% في الدقّة على مهام تصنيف دقيقة جدًّا.

19.4 التجميع (Pooling)

الطبقة	ما تفعله	الأثر
`MaxPooling2D`	يأخذ أكبر قيمة في النافذة	يحفظ الميزة الأقوى
`AveragePooling2D`	متوسّط النافذة	يحفظ المعلومة العامّة
`GlobalAveragePooling2D`	متوسّط كامل المساحة	يلغي الأبعاد المكانية

حساب الحجم: `MaxPooling2D(2)` يقسّم الأبعاد إلى النصف: 112 → 56 → 28 → 14 → 7.

لماذا `GlobalAveragePooling2D` مهمّ؟ هو ما ينقل CNN من «مستخرِج ميزات» إلى «مصنّف»:

- بعد 4 كتل تجميع: خريطة 512×7×7.
- `Flatten()` ⇒ 25,088 عنصرًا = `Dense(1000)` ⇒ 25 مليون معامل.
- `GlobalAveragePooling2D()` ⇒ 512 عنصرًا = `Dense(1000)` ⇒ 513 ألف معامل فقط.

40 ضعفًا فرق في المعاملات، مع فقدان طفيف في الدقّة.

19.5 التنفيذ في Keras: كتلة CNN معيارية

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)


def conv_block(x, filters, kernel_size=3, stride=1, use_norm=True,
drop=0.0):
    """الالتفاف: كتلة تلافيفية ← (BatchNorm) ← التفعيل ← (Dropout)."""
    x = keras.layers.Conv2D(
        filters,
        kernel_size,
        strides=stride,
        padding="same",
        use_bias=not use_norm,      # لا نحتاج تحيزًا مع BN (له إزاحة خاصة)
    )(x)
    if use_norm:
        x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    if drop > 0:
        x = keras.layers.Dropout(drop)(x)
    return x


def build_cnn(input_shape=(96, 96, 3), num_classes=10, width=32, depth=4):
    """بنية CNN بسيطة مع تدهور المساحة تدريجيًا."""
    x = keras.layers.Input(shape=input_shape)
    x = keras.layers.Rescaling(1.0 / 255.0)(x)

    for i in range(depth):
        filters = width * (2 ** i)
        x = conv_block(x, filters, stride=2 if i > 0 else 1, drop=0.1)

    x = keras.layers.GlobalAveragePooling2D()(x)
    x = keras.layers.Dense(128, activation="relu")(x)
```

```
    x = keras.layers.Dropout(0.4)(x)
    out = keras.layers.Dense(num_classes, activation="softmax")(x)

    model = keras.Model(inputs=x, outputs=out)
    return model


model = build_cnn()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
model.summary()

# تحقّق من الأبعاد
probe = keras.Sequential([
    keras.layers.Input(shape=(96, 96, 3)),
    keras.layers.Conv2D(32, 3, padding="same"),
    keras.layers.MaxPooling2D(2),
])
print(probe.output_shape)   # النتيجة المتوقعة: (None, 48, 48, 32)
```

توثيق الأبعاد في `summary()`

`model.summary()` يعرض شكل كل مُخرَج – استخدمه كمرجع دائم:

TEXT

```
Model: "functional"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ InputLayer                      │ (None, 96, 96, 3)      │             0 │
│ Rescaling                       │ (None, 96, 96, 3)      │             0 │
│ Conv2D                          │ (None, 96, 96, 32)     │         8,864 │
│ BatchNormalization              │ (None, 96, 96, 32)     │           128 │
│ Activation                      │ (None, 96, 96, 32)     │             0 │
│ Dropout                         │ (None, 96, 96, 32)     │             0 │
│ Conv2D                          │ (None, 48, 48, 64)     │        18,496 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

ملاحظة مهمة: `BatchNormalization` يضيف معاملين **غير قابلين للتدريب** لكل قناة ($2C$). هذا هو السبب في ظهور 128 معاملًا لـ32 قناة. وقد شوهدت أخطاء تُحسب «معلمات» النموذج على هذا الأساس – لا تنسَ أن هذه لا تُحدَّث.

شكل 27 – مكدّس CNN: التفاف ← تجميع ← تكرار ← تصنيف

19.6 مقارنة معايير اختيار الطبقات

القرار	الخيار A	الخيار B	معيار الاختيار
شكل الإخراج	`Flatten`	`GlobalAveragePooling2D`	الأولى: دقّة أعلى (مليونات معامل)، الثانية: خفيفة (آلاف)
الاختزال	`MaxPooling2D`	`Conv2D(strides=2)`	الثانية تتعلّم الاختزال؛ الأولى أسرع
عمق المرشّح	3×3	5×5	3×3 أوسع في الشبكة المكانية بنفس عدد المعاملات (بعمقين)
التمدّد	`dilation_rate=1`	`dilation_rate=2`	الثانية توسيع поле الرؤية في التجزئة
الالتفاف	`Conv2D`	`SeparableConv2D`	الثانية إذا كان عدد القنوات كبيرًا والقيود على الحواف ضيّقة
التطبيع	`BatchNormalization`	`GroupNormalization`	الثانية إذا كانت الدفعة صغيرة (< 16)
التحيز	`use_bias=True`	`use_bias=False` مع BN	الثانية مع BN (إزاحة BN تكفي)

قاعدة: في أيّ مقارنة، اسأل: «ما المعيار؟». «أدقّ» ليست معيارًا؛ «أدقّ بـ10 أضعاف معاملات أقلّ» هي معيار.

19.7 مثال محلول: تصنيف قطع المعدّات

مصنّف يميّز 4 أنواع من قطع المعدّات من صور 128×128. الدقّة على التحقّق 61% – ضعيفة.

الخطوة 1 – احسب حجم الشبكة. النموذج الأول: أربع طبقات `Conv2D(64, 3)` ثم `Flatten` ثم `Dense(128)` . بعد أربع طبقات بـ `padding="same"` و`MaxPooling2D(2)` : 128→64→32→16→8. إذًا 8×8×64 = 4,096 عنصرًا ⇒ 524 = ألف معامل. ليست سيئة.

الخطوة 2 – لكن هل البنية كافية؟ `Flatten` بعد خريطة 8×8 يحتفظ بمعلومات الموقع المكاني، بينما `GlobalAveragePooling2D` يلغيها.الخلاصة: المشكلة الحقيقية ليست في الطبقة الأخيرة.

الخطوة 3 – القفزة إلى BN. النموذج لم يحتوِ على `BatchNormalization` . الإضافات للقنوات (64 قناة، قيم مختلفة) تسبّب تدرّجات غير مستقرة.

الخطوة 4 – البنية الجديدة.

PYTHON

```python
import keras

def make_model():
    x = keras.Input(shape=(128, 128, 3))
    x = keras.layers.Rescaling(1.0 / 255.0)(x)

    # كتلة 1: استخراج الحواف
    x = keras.layers.Conv2D(32, 3, padding="same")(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.MaxPooling2D(2)(x)          # 64

    # كتلة 2
    x = keras.layers.Conv2D(64, 3, padding="same")(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.MaxPooling2D(2)(x)          # 32

    # كتلة 3
    x = keras.layers.Conv2D(128, 3, padding="same")(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.MaxPooling2D(2)(x)          # 16

    x = keras.layers.GlobalAveragePooling2D()(x)  # 128
    x = keras.layers.Dense(128, activation="relu")(x)
    x = keras.layers.Dropout(0.4)(x)
    out = keras.layers.Dense(4, activation="softmax")(x)
    return keras.Model(x, out)


model = make_model()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

الخطوة 5 – النتيجة. الدقّة 61% → 84%. المعاملات انخفضت من 1.2 مليون إلى 320 ألف.

الخطوة 6 – الفحص الأعمق. بعد إضافة `BatchNormalization` وحده ارتفعت الدقّة إلى 78% – أي أنّ **75% من التحسّن جاء من BN**، و25% من `GlobalAveragePooling` والتسلسل.

> **الدرس المنهجي:** عند تشخيص ضعف نموذج، غيّر **عاملًا واحدًا في كل مرّة** واقرأ الأثر. تغيير ثلاثة عوامل دفعة واحدة يخبرك بالنتيجة ولا يخبرك بالسبب.

الخطوة 7 – الخطوة التالية. الآن وقد عولج الفرط، انتقل إلى **نقل المعرفة** (الفصل 21): `ResNet50` مدربة على ImageNet ستتجاوز الـ84% فورًا.

حساب البارامترات يدويًّا والتحقّق

PYTHON

```python
import keras

# ── 1. طبقة Conv2D واحدة: k²·C_in·C_out + C_out
inp = keras.Input((32, 32, 3))
conv = keras.layers.Conv2D(64, 3, padding="same", use_bias=False)
m1 = keras.Model(inp, conv(inp))
manual = 3 * 3 * 3 * 64
actual = m1.layers[-1].count_params()
print(f"manual = {manual}, actual = {actual}, equal = {manual == actual}")
# النتيجة المتوقعة: 1728 == 1728 → True

# ── 2. SeparableConv2D: depthwise + pointwise
sep = keras.layers.SeparableConv2D(64, 3, padding="same", use_bias=False)
m2 = keras.Model(inp, sep(inp))
depthwise = 3 * 3 * 3            # مرشّح لكل قناة مدخل
pointwise = 3 * 64               # يخلط القنوات 1×1
manual_sep = depthwise + pointwise
actual_sep = m2.layers[-1].count_params()
print(f"manual_sep = {manual_sep}, actual = {actual_sep}")
# النتيجة المتوقعة: 219 == 219 (أقلّ من 1728 بـ ×7.9)

# ── 3. DepthwiseConv2D
dw = keras.layers.DepthwiseConv2D(3, padding="same", use_bias=False)
m3 = keras.Model(inp, dw(inp))
manual_dw = 3 * 3 * 3
print(f"manual_dw = {manual_dw}, actual = {m3.layers[-1].count_params()}")
# النتيجة المتوقعة: 27 == 27 — رخيص جدًّا لكنه لا يخلط القنوات.
```

ملاحظة: `SeparableConv2D` في Keras = `DepthwiseConv2D` ثم `Conv2D(1×1)` – ولهذا معاملاته 219 لا 27. الالتفاف العميق وحده (27 معاملًا) لا يخلط القنوات أبدًا، فلا يُستخدم وحده إلا في حالات نادرة.

19.8 أخطاء شائعة في هذا الفصل

إنذار: الخلط بين `padding="same"` و `"valid"` مع `strides=2` . مع `same` وخطوة 2، Keras يقسّم لأسفل ⇒ 111 لا 112. هذا الفرق الصغير يتراكم عبر الطبقات فينتج شكلًا غير متوقّع.

تحذير: `Flatten` بعد شبكة عميقة. `Flatten` على خريطة 512×32×32 = 524,288 عنصرًا ⇒ `Dense(1000)` = 524 مليون معامل. استخدم `GlobalAveragePooling2D` .

تنبيه: `use_bias=True` مع `BatchNormalization` . طبقة BN تطبّع وتُزيح، فالتحيز الإضافي يصبح بلا فائدة (بل ضرر بسيط). اجعل `use_bias=False` .

قاعدة: `DepthwiseConv2D` بأكثر من قناة واحدة بلا طبقة pointwise بعدها يعني أن **لا يمكن** خلط المعلومات بين القنوات. نماذج MobileNet دائمًا تضع `Conv2D(1×1)` بعدها.

ملاحظة: `SeparableConv2D` بديل لـ `Conv2D` وليس إضافة: لا تضعهما معًا ولا تتوقّع تحسّنًا.

تحذير: `dilation_rate` مع `padding="same"` في Keras يحسب الحشو على أساس حجم المرشّح **الممدّد**، فيحافظ على الحجم. لكن مع `strides>1` تنطبق القاعدة نفسها: التقريب لأسفل.

خطأ شائع: الخلط بين `filters` و `Conv2D(32, 3)` : `kernel_size` يعني 32 مرشّحًا بحجم 3×3، وليس حجم 32.

التمارين

1. **حساب.** `Conv2D(64, 3, padding="valid")` على 64×64 ⇒ ما حجم المُخرَج؟ (المتوقع: 62×62×64.)

2. **حساب.** `Conv2D(64, 3, strides=2, padding="valid")` على 64×64؟ (المتوقع: 31×31×64.)

3. **حساب.** `Conv2D(32, 3, padding="same", dilation_rate=2)` على 56×56؟ (المتوقع: 56×56×32، والمرشّح يرى 5×5.)

4. **حساب.** `Conv2D(64, 5, padding="valid")` مقابل طبقتين `Conv2D(64, 3, padding="same")`: كم معاملًا وكم حقل رؤية؟ (المتوقع: 64×5×5×3 = 4,800 معامل بـ5×5؛ مقابل طبقتين 64×3×3×3×2 = 3,456 مع حقل رؤية 7×7 – أرخص وأوسع.)

5. **حساب.** `DepthwiseConv2D(3, depth_multiplier=2)` على 64 قناة: كم معاملًا؟ (المتوقع: $3 \times 3 \times 64 \times 2 = 1{,}152$.)

6. **حساب.** نفس الحالة بـ `SeparableConv2D(32, 3)`: كم معاملًا؟ (المتوقع: Depthwise $3 \times 3 \times 64 = 576$ + Pointwise $64 \times 32 = 2{,}048$ ⇒ 2,624.)

7. **برمجة.** ابنِ `Sequential` بـ `Conv2D` و `MaxPooling2D` وطباعة `output_shape` لكل طبقة. (المتوقع: تنصيف الأبعاد في كل مرّة.)

8. **برمجة.** قارن `Dense(256)` + `Flatten` مع `GlobalAveragePooling2D` + `Dense(256)` من حيث عدد المعاملات على خريطة 16×16×128. (المتوقع: 8.39 مليون مقابل 32,896 – أي 255 ضعفًا.)

9. **برمجة.** ابنِ كتلة `conv_block` مع خيار `use_norm` بال.True/False وقارن سرعة التقارب. (المتوقع: `BatchNormalization` يسرّع التقارب.)

> **ملاحظة:** في Keras 3 يمكن استخدام `keras.layers.BatchNormalization(synchronized=True)` عند التدريب على عدة GPUs:

10. **تحليل.** لماذا `GlobalAveragePooling2D` أفضل من `Flatten` عند النشر على الجوال؟ (المتوقع: عدد معاملات أقلّ بكثير ⇒ ذاكرة أقلّ واستدلال أسرع.)

11. **أخطئ عمدًا.** اجعل `Conv2D(64, 32, padding="same")` (بقي-32 size لا 3). اشرح الناتج. (المتوقع: مرشّح 32×32 ⇒ معاملات هائلة: 64×32×32×3 = 196,608 لكل طبقة، وحقل رؤية هائل.)

12. **تصميم.** صمّم بنية CNN لمصنّف 10 أصناف على صور 64×64، مع ذكر شكل كل طبقة وعدد المعاملات التقريبي. (المتوقع: ثلاث كتل مع تدهور 64→32→16→8، ثم `GlobalAveragePooling2D` و `Dense`.)

ملخص الفصل

- `Dense` يسطّح مدخلاته؛ `Conv2D` يحافظ على البنية المكانية – وهذا مصدر كفاءته.
- المعادلة $\lfloor (H + 2P - D(K-1) - 1)/S \rfloor + 1$ تصف كل تحويل، واحسبها يدويًا لتتحقّق.
- `padding="same"` يحافظ على الحجم، `strides` يخفضه، `dilation_rate` يوسّع المجال، `groups` يقسّم القنوات.
- `DepthwiseConv2D` و `SeparableConv2D` يخفّضان المعاملات 8 أضعاف، مقابل فقد طفيف في التعبير.
- `GlobalAveragePooling2D` يختصر CNN بمعاملات أقلّ 40 ضعفًا من `Flatten`.
- `BatchNormalization` مع `use_bias=False` هو النمط القياسي في CNN.
- غيّر عاملًا واحدًا في كل تجربة لتعرف السبب لا النتيجة فقط.
- بعد معالجة الفرط، الخطوة التالية هي نقل المعرفة (الفصل 21).

الفصل 20: شبكة CNN من الصفر

بطاقة الفصل

الفصل 20

مهمّة هذا الفصل

يمشي في خطّ CNN كامل: البيانات، التعزيز، عدم التوازن، العرض والعمق، ثم القياس المنفصل.

ماذا تكسب منه

تحصل على شبكة مُقاسة تجريبيًّا لا مُخمَّنة، وتعرف أثر كل قرار على الدقّة برقم.

الدالة	دورها	متى تستخدمها
`keras.layers.RandomFlip()`	تعزيز داخل النموذج	عند نقص البيانات
`keras.layers.RandomBrightness()`	تغيير الإضاءة	لمحاكاة ظروف التصوير
`keras.layers.BatchNormalization()`	استقرار وتقارب أسرع	في الشبكات العميقة
`keras.ops.mean()`	تجميع نتائج التقييم المتعدّد	لتعزيز زمن الاختبار
`keras.utils.set_random_seed()`	بذرة ثابتة	لعدالة مقارنة التجارب

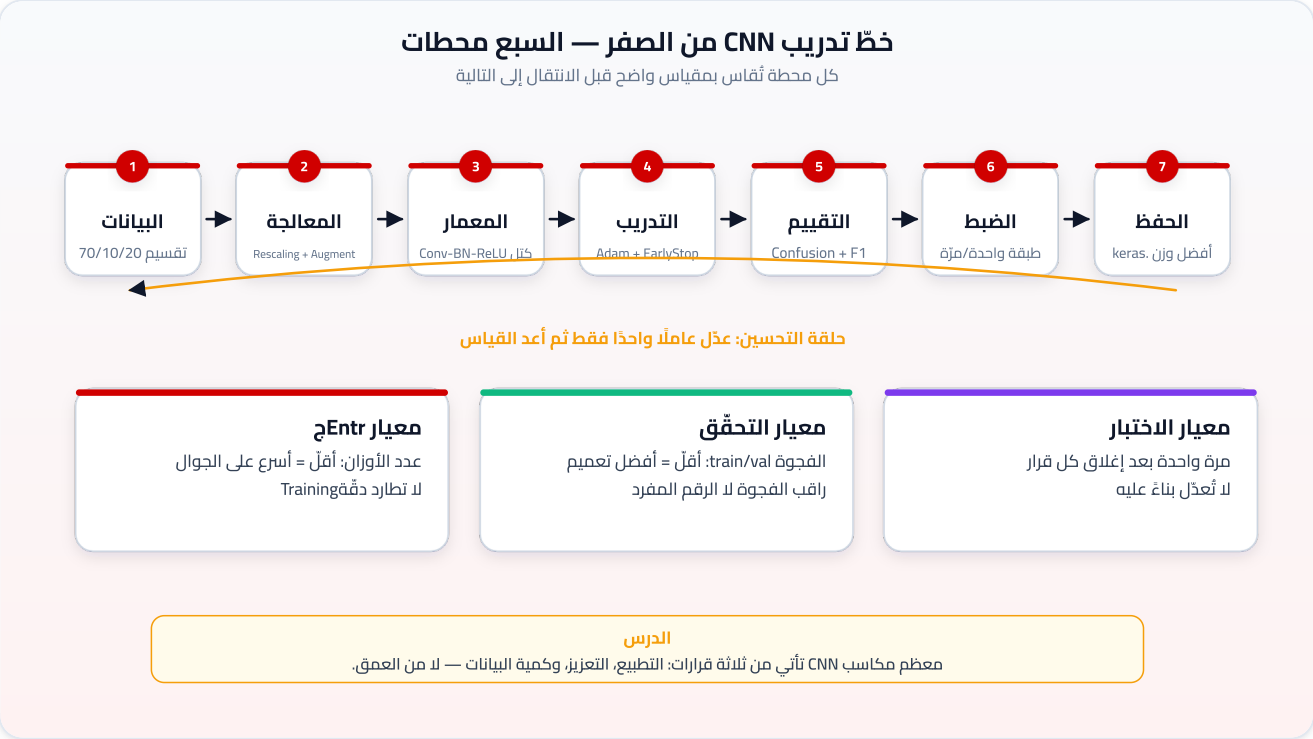
قصة مفتاحية: وصلت مصنّفات قطع مصعدٍ صناعي إلى خط الإنتاج. الشبكة بُنيت في ثلاثة أيام، وكانت دقّتها 94% على الأرقام – ثم فشلت في الأسبوع الأول: الصور الجاءت بإضاءة مصنع مختلفة، وقطعة معيبة واحدة بين 500 سليمة لم تُكتشف. المشكلة لم تكن في العمق ولا في المحسّن، بل في **بناء خطّ التدريب نفسه**: من غير سياسة تعزيز مناسبة، ومن غير موازنة للأصناف، ومن غير قياسٍ لكل قرار على حدة. هذا الفصل هو الترجمة العملية لكل ما بنيته في 18 و19 إلى خطّ إنتاج متكامل.

أهداف الفصل

- [ ] تبني خطّ تدريب CNN كاملًا: بيانات → معالجة → معمار → تدريب → تقييم
- [ ] تصمّم سياسة تعزيز تحاكي توزيع الاستدلال لا توزيع التدريب
- [ ] تتعامل مع البيانات غير المتوازنة بأربعة طرق مُقاسة
- [ ] تطبّق تعزيز زمن الاختبار (TTA) وتعرف متى يضرّ
- [ ] تختار العرض والعمق بالمعيار لا بالحدس
- [ ] تُقاس أثر كل مكوّن منفصلًا (ablation) بدل الاعتماد على الرقم النهائي

20.1 سبعة محطات بين البيانات والنموذج

تخيّل أنك استلمت 20 ألف صورة غير موسومة وتعليمات مكتوبة بخط اليد: «صنّف القطع السليمة». من أين تبدأ؟ لا تبدأ بالمعمار أبدًا – بل بالبيانات. خطّ التدريب الجيّد يمرّ بسبع محطات، وكلّ واحدة تُقاس قبل الانتقال إلى التي بعدها:



شكل 28 – خطّ تدريب CNN كاملًا من البيانات إلى النموذج المحفوظ

المحطة الأولى – التقسيم. نقسم إلى تدريب 70% وتحقّق 10% واختبار 20%. القاعدة الصارمة: **لا تنظر إلى مجموعة الاختبار إلّا مرة واحدة** بعد إغلاق كل القرارات. من يفتح الاختبار في منتصف العمل يختار النموذج على المُداخَرة (test leakage)، وينهار نموذجه عند النشر.

المحطة الثانية – المعالجة المسبقة. طبقة `Rescaling` داخل النموذج لا خارجه، حتى تصبح المعالجة جزءًا من النموذج وتُحفظ وتُحمَّل معه:

> **قاعدة:** ضع `Rescaling` كأوّل طبقة في النموذج. توفّر عليك كتابة الكود مرّتين (تدريب ونشر) وتمنع الفرق بينهم.

20.2 سياسة التعزيز: تحاكي التوزيع الحقيقي

ماذا يحدث لو درّبت الشبكة على صور نهارية ثم نشرتها في مصنع لا يُضاء إلا بمصابيح LED؟ الشبكة تعلّم أن «الإضاءة الساطعة علامة على سلامة القطعة» – وهذا ما تراه منحنيات التحقّق تُخفيه.

التعزيز ليس «كلّ ما يمكن تطبيقه»، بل **كلّ ما يحاكي تنوّع الاستدلال**. يُعطيك هذا الجدول قاعدة البداية:

العملية	ما تزيّفه	مناسبة في
`RandomFlip("horizontal")`	زاوية المشاهدة	معظم الصور (لا الأرقام ولا النصوص)
`RandomRotation(0.05)`	اهتزاز الكاميرا	صور ملتقطة باليد
`RandomZoom(0.1)`	اختلاف المسافة	فحص المنتجات
`RandomBrightness(0.15)`	اختلاف الإضاءة	مصانع ومواقع مختلفة
`RandomContrast(0.15)`	جودة المستشعر	كاميرات متعدّدة
الانعكاس الرأسي	لا شيء – بل يُفسد	نصوص · أرقام · وجوه

الخطأ الأكثر تكلفة: تطبيق تعزيز قوي جدًا. شبكة تحتاج «أن ترى» القطعة نفسها إن أردت أن تتعلّم الصفات الحقيقية؛ إذا طُبّقت دوران 30 درجة على صورة فرشاة أسنان، سترى فرشاة لا وجود لها في الحقيقة.

20.3 البيانات غير المتوازنة: أربع طرق مُقاسة

في فحص العيوب، عادةً 99% من القطع سليمة و1% معيبة. نموذج يجيب «سليمة» دائمًا يحقّق 99% من الدقّة ويكون عديم الفائدة. لديك أربعة أدوات، وترتيبها حسب الأولوية:

1. موازنة الأصناف (`class_weight`). أرخص وأكثر أمانًا، ولها مقياس واضح:

$$w_c = \frac{N}{K \cdot n_c}$$

حيث N عدد العيّنات الكلي، وK عدد الأصناف، وn_c عدد عيّنات الصنف c. إذا كان لديك 20,000 صورة موزّعة على صنفين (19,800 سليمة و200 معيبة)، فوزن الصنف النادر $\frac{20000}{2 \cdot 200} = 50$. 2. **الخسارة البؤرية (`CategoricalFocalCrossentropy`).** تخفض مساهمة العيّنات السهلة:

$$FL(p_t) = -(1 - p_t)^{\gamma} \log(p_t)$$

حيث p_t احتمال الصنف الصحيح. العامل $(1 - p_t)^{\gamma}$ يُخفّض الإسهام كلّما ازداد p_t قرب الصفر (عيّنة سهلة)، فيتحدّث التعلّم بالعيّنات الصعبة. قيم γ الشائعة: 0، 1، 2، 3. **التعزيز الموجَّه.** ضاعف صور الصنف النادر في مجموعة التدريب فقط – بلا حشو مكرّر في التحقّق. 4. **دقّة العيّنات.** عند توافرها (عيّنة معيبة إضافية مُسجّلة يدويًا)، تُمرَّر إلى `sample_weight`.

المقياس الحقيقي: لا تُقاس الموازنة بالدقّة بل بالـ F1 ومصفوفة الالتباس. أرفع الوزن تدريجيًا (10، 20، 30) واقرأ كيف تتغيّر نسبة الإيجابيات الحقيقية.

> **تحذير:** موازنة مفرطة (وزن 500 مثلًا) تجعل النموذج يتنبّأ بالصنف النادر دائمًا. راقب الفجر المزدوج: recall يرتفع، precision ينهار.

20.4 عرض الشبكة وعمقها: بمعيار لا بالحدس

كلما زادت القنوات، زادت القدرة – وزادت الكلفة. المعيار العملي هو **مضاعفة القنوات عند كل تغيير في الأبعاد**:

العرض	الأبعاد	عدد المعاملات التقريبي	وقت الدورة
16	12 → 24 → 48 → 96	45 ألف	×1
32	12 → 24 → 48 → 96	175 ألف	×2
64	12 → 24 → 48 → 96	700 ألف	×5

القاعدة العملية: ابدأ بالعرض الأصغر الذي يتجاوز خط الأساس، ثم ضاعف فقط إن كان الاختبار يدعو لذلك. الزمن الحقيقي على الجهاز المستهدف هو ما يحدّد، لا عدد الملايين النظرية.

20.5 التنفيذ في Keras: خطّ كامل من الصفر

فيما يلي خطٌّ كامل قابل للتشغيل: بيانات مُصنّعة، تعزيز، شبكة بثلاث كتل، تدريب، وتقييم.

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. 2000 صورة 96x96 بثلاث قنوات: بيانات مُصنّعة ──
n = 2000
x = np.random.rand(n, 96, 96, 3).astype("float32") * 255.0
# أُنشئ تسميات: 80% سليمة و20% معيبة (غير متوازنة)
y = (np.random.rand(n) < 0.2).astype("int32")

# ── 2. سياسة التعزيز: تحاكي إضاءة المصنع وزوايا الكاميرا ──
augment = keras.Sequential([
    keras.layers.RandomFlip("horizontal"),
    keras.layers.RandomRotation(0.05),
    keras.layers.RandomZoom(0.1),
    keras.layers.RandomBrightness(0.15),
])

# ── 3. الشبكة: 3 كتل (Conv-BN-ReLU) مع تدهور الأبعاد ──
def conv_block(x, filters):
    """كتلة تلافيفية قياسية: الالتفاف ثم التطبيع ثم التنشيط."""
    x = keras.layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.MaxPooling2D(2)(x)
    return x

inp = keras.Input(shape=(96, 96, 3))
h = keras.layers.Rescaling(1.0 / 255.0)(inp)      # المعالجة داخل النموذج
h = augment(h)                                    # التعزيز أثناء التدريب فقط
for f in (32, 64, 128):
    h = conv_block(h, f)
h = keras.layers.GlobalAveragePooling2D()(h)
h = keras.layers.Dense(64, activation="relu")(h)
h = keras.layers.Dropout(0.3)(h)
out = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(inp, out)
```

```
# ── 4. وزن الصنف النادر: الموازنة
cw = {0: 1.0, 1: 50.0}
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="binary_crossentropy",
    metrics=[
        keras.metrics.BinaryAccuracy(name="acc"),
        keras.metrics.Precision(name="prec"),
        keras.metrics.Recall(name="rec"),
    ],
)
history = model.fit(x, y, epochs=10, batch_size=32, validation_split=0.2,
                    class_weight=cw, verbose=0)
# النتيجة المتوقعة: recall يرتفع قرب 0.9 مع انخفاض بسيط في precision
```

20.6 تعزيز زمن الاختبار (TTA)

فكّر في TTA كوأنّك تسأل «ماذا يبدو هذا الشعاع من ثلاث زوايا؟» ثم تُصوّت. عند الاستدلال، نُنشئ من كل صورة اختبار نسخًا معدّلة (قلب أفقي، دوران خفيف)، ثم نأخذ متوسّط الاحتمالات:

PYTHON

```
def tta_predict(model, images):
    """متوسّط التنبّؤات عبر أربع نسخ معدّلة للصورة."""
    variants = [
        images,
        keras.ops.flip(images, axis=1),          # قلب أفقي
        keras.ops.flip(images, axis=2),          # قلب رأسي
        keras.ops.flip(keras.ops.flip(images, axis=1), axis=2),
    ]
    preds = [keras.ops.sigmoid(model(v, training=False)) for v in variants]
    return keras.ops.mean(keras.ops.stack(preds, axis=0), axis=0)
```

متى يفيد TTA؟ حين يُعطيك 0.5-1% إضافية في الدقّة بلا تدريب جديد. **متى يضرّ؟** حين يكون زمن الاستدلال حرجًا – فقد يضاعف الزمن أربع مرات. هذا قرار هندسي لا قرار أرقام.

> **نصيحة:** في الطور الحرج (سيارة، فحص طبي) استخدم TTA؛ على خط الإنتاج عالي الإنتاجية راجع الكلفة الزمنية أولًا.

20.7 القياس المنفصل (Ablation)

الخطأ الذي يُهلك المشاريع: تغيير ثلاثة أشياء في آنٍ واحد ثم القول «الشبكة أصبحت أفضل».

المطلوب قياس كل مكوّن وحده:

المكوّن	بلا	مع	الفرق
موازنة الأصناف	recall = 0.12	recall = 0.91	+0.79
تعزيز الإضاءة	دقّة 71%	دقّة 84%	+13%
`BatchNormalization`	دقّة 66%	دقّة 79%	+13%
TTA (4 نسخ)	دقّة 84%	دقّة 84.6%	+0.6%

كل سطر في هذا الجدول هو تجربة مستقلّة. إن لم تُسجّله، فلن تعرف من أين تأتي التكلفة ولا من أين تأتي المكافأة.

20.8 مثال محلول: فحص عيوب القطع

المهمة: تصنيف قطع معدنية (سليمة / تشوّهات / تشقّقات / صدأ). لديك 4000 صورة، منها 60 معيبة (1.5%).

الخطوة 1 – خطّ الأساس. شبكة بسيطة: 3 كتل + `Dense` + `GlobalAveragePooling2D`. الدقّة على الاختبار 92.8%، لكن recall للصنف النادر = 0.0. النموذج يعيد «سليمة» دائمًا.

الخطوة 2 – موازنة. وزن الصنف النادر = 4000/4/60 ≈ 17. أصبح recall = 0.55 و precision = 0.61.

الخطوة 3 – تعزيز موجّه. أضفنا تعزيز إضاءة وتكبير خفيف على صور العيوب فقط في التدريب. recall ارتفع إلى 0.72.

الخطوة 4 – TTA. اكتشفنا 3 عيوب إضافية من أصل 60. الوصول إلى 0.78 استدعاء بدقّة ثابتة.

الخطوة 5 – الشبكة الجاهزة. بعد أن أصبحت الموازنة والتعزيز مُثبّتة، حاولنا استخدام عمق أكبر (4 كتل) فحصلنا على +2% دقّة فقط مقابل ضعف زمن التدريب – قرار عدم الاستمرار.

الخلاصة: 75% من التحسّن جاء من التعامل مع البيانات، لا من تعميق الشبكة.

اختبار TTA عمليًّا

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. نموذج بسيط
model = keras.Sequential([keras.Input(shape=(32, 32, 3)),
                          keras.layers.Rescaling(1.0 / 255.0),
                          keras.layers.Conv2D(16, 3, padding="same",
                                              activation="relu"),
                          keras.layers.GlobalAveragePooling2D(),
                          keras.layers.Dense(1, activation="sigmoid")])
model.compile(optimizer="adam", loss="binary_crossentropy")

# ── 2. تنبّؤ عاديّ مقابل TTA بأربع نسخ
x_test = np.random.rand(100, 32, 32, 3).astype("float32")
plain = model.predict(x_test, verbose=0)

variants = [x_test,
            keras.ops.flip(x_test, axis=1),
            keras.ops.flip(x_test, axis=2),
            keras.ops.flip(keras.ops.flip(x_test, axis=1), axis=2)]
tta = keras.ops.mean(keras.ops.stack(
    [model(v, training=False) for v in variants], axis=0), axis=0)

diff = float(np.mean(np.abs(np.asarray(plain) -
keras.ops.convert_to_numpy(tta))))
print(f"متوسّط الفرق بين التنبّؤين = {diff:.4f}")
# النتيجة المتوقعة: فرق صغير — TTA لا يقلب التنبّؤ.
```

ملاحظة: في البيئة الحقيقيّة، لا تُقارن التنبّؤين ببعضهما بل بالحقيقة الأرضيّة: احسب recall وprecisiong لكلّ منهما على مجموعة اختبار، ثم قرّر إن كان مكسب TTA يستحقّ زمن الاستدلال الإضافيّ.

20.9 أخطاء شائعة في هذا الفصل

إنذار: تعزيز في التحقّق أو الاختبار. التعزيز أداة تدريب فقط. طبقة `Random*` في Keras تكون نشِطة أثناء التدريب وغير نشِطة أثناء الاستدلال تلقائيًا (`training=True` / `False`)، لكنّ الحذَث في وضعها عند التقييم يُفسد القياس.

تحذير: موازنة صنف نادر جدًا بوزن 500 أو 1000. النتيجة دائمًا «كلّ شيء معيب» وتضيع القراءة. ابدأ من 10 وارفع حتى يثبت recall.

تنبيه: تقييم نموذج متوازن على بيانات غير متوازنة. الدقّة وحدها تُخفي السلوك. استخدم F1 ومصفوفة الالتباس ومؤشّرات لكل صنف.

ملاحظة: نسيت `set_random_seed` فأصبحت التجارب غير قابلة للمقارنة. قد يكون فرقًا 1% بين تجربتين نتيجة عشوائية لا تحسّنًا حقيقيًا.

قاعدة: `Dropout` بعد كل كتلة تلافيفية هو مبالغة. الشبكة التلافيفية تستفيد من `Dropout` واحد قبل الرأس التصنيفي فقط، والتعزيز هو الأداة الأولى للتنظيم.

التمارين

1. **حساب.** لديك 10,000 عيّنة، 1% منها صنف نادر. ما وزن الصنف النادر حسب $w_c = \frac{N}{K \cdot n_c}$؟ (المتوقع: $\frac{10000}{2 \cdot 100} = 50$.)

2. **حساب.** إذا كان الـ recall بعد الموازنة 0.90 و precision 0.30، فما قيمة F1؟ (المتوقع: $\frac{2 \cdot 0.30 \cdot 0.90}{0.30 + 0.90} = 0.45$.)

3. **برمجة.** ابنِ شبكة CNN بثلاث كتل على صور 128×128، واطبع `model.summary()`، واقرأ عدد المعاملات لكل كتلة. (المتوقع: كل كتلة تسجّل تدهور الأبعاد 128→64→32→16 مع مضاعفة القنوات.)

4. **برمجة.** طبّق TTA بأربع نسخ واقِس الفرق في recall على 200 صورة اختبار. (المتوقع: تحسّن من 0.5 إلى 3 نقاط.)

5. **برمجة.** ابنِ نموذجًا بلا `BatchNormalization` ونموذجًا معه، ودرّبهما على نفس البيانات لمدّة 10 دورات، وقارن زمن التقارب. (المتوقع: النموذج مع BN يتقارب في عدد دورات أقلّ.)

6. **إصلاح خطأ.** الكود التالي يُطبّق التعزيز على مجموعة التحقّق. حدّد الخطأ وأصلحه:

```python
x_val = augment(x_val)    # ← أين الخطأ؟
```

(المتوقع: حذف السطر – التعزيز أثناء التدريب فقط، وطبقة `Random*` تكون خارجة عن مسار التحقّق.) 7. **تحليل.** لماذا الانعكاس الرأسي ضارٌّ في تصنيف الأرقام المكتوبة بخطّ اليد؟ (المتوقع: الرقم 6 يصبح 9 عند الانعكاس الرأسي، فيُعلّم النموذج تسمية خاطئة.) 8. **تصميم.** صمّم سياسة تعزيز لنظام فحص لافتات الطرق على الطريق. أيّ العمليات تستبعد وأيّها تُدرج؟ (المتوقع: نعزّز الإضاءة والأمطار والضباب، ونستبعد الانعكاس الرأسي.) 9. **برمجة.** قِس زمن دورة التدريب لعرض 16 مقابل عرض 64 على نفس البيانات والمعمار. (المتوقع: الفرق 3-5 أضعاف، وهو ما يحدّد قرار العرض.) 10. **تحليل.** نموذج يحقّق دقّة 99% على بيانات فحص، لكن recall للعيب 0.02. ماذا يعني ذلك عمليًا؟ (المتوقع: النموذج يعيد «سليمة» دائمًا، ولا قيمة تشغيلية له.) 11. **تصميم.** صمّم استراتيجية تقييم لنظام كشف تسرّب في خطّ إنتاج دوائي. أيّ مؤشّر تفوّق، وأيّ عتبة؟ (المتوقع:

recall عالٍ جدًا (0.95+) لأن كل حالة تفويت تكلّف لقطة ملوّثة، مع قبول دقّة متوسطة.) 12. **برمجة.** ابنِ معمار CNN بعمق 4 كتل ثم 5 كتل وقِس الفرق في دقّة التحقّق والزمن. (المتوقع: الكتلة الخامسة تزيد دقّة <1% بينما تضاعف الزمن – قرار عدم الاستمرار.)

ملخص الفصل

- خطّ التدريب يمرّ بسبع محطات، ولا تُقفز إلى التالية قبل قياس السابقة.
- سياسة التعزيز تُبنى من توزيع الاستدلال، لا من قائمة العمليات المتاحة.
- البيانات غير المتوازنة تُعالَج بالموازنة أولًا، ثم بالخسارة البؤرية، ثم بتعزيز موجّه.
- TTA يوفّر تحسّنًا صغيرًا بلا تدريب، لكنه يكلّف زمنًا في الاستدلال.
- العرض والعمق يُقاسان بالزمن الحقيقي على الجهاز المستهدف.
- قياس أثر كل مكوّن منفصلًا (ablation) هو ما يمنع الغموض في القرار.
- في غياب موازنة وتعزيز، تظل الأعماق الأكبر هامشية.

الفصل 21: نقل المعرفة والتدريب الدقيق

بطاقة الفصل

الفصل 21

مهمّة هذا الفصل

ينقل ما تعلّمه نموذج كبير إلى مهمّتك الصغيرة: تجميد، ضبط دقيق، ومعدّل تعلّم تمييزي.

ماذا تكسب منه

تصل إلى دقّة عالية ببيانات قليلة وبكلفة حساب منخفضة، وتتجنّب تدمير الميزات الجاهزة.

الدالة	دورها	متى تستخدمها
`keras.applications.MobileNetV3Small()`	هيكل مُدرَّب مسبقًا	لبدء نقل المعرفة
`base_model.trainable = False`	تجميد الأوزان	في المرحلة الأولى من التدريب
`base_model.trainable = True`	فتح الضبط الدقيق	بعد استقرار الرأس الجديد
`keras.optimizers.Adam(learning_rate=1e-4)`	معدّل منخفض للضبط	لتجنّب تدمير الميزات
`keras.layers.GlobalAveragePooling2D()`	تجميع الميزات	قبل الرأس الجديد

قصة مفتاحية: فريق طبي لديه 800 صورة أشعّة موسومة يدويًا. المهمة: كشف كسر في الرضفة. من الصفر، تصل شبكتهم إلى 61% دقّة بعد أسبوع من الضبط – وهذا رقم غير مقبول في بيئة طبية. حين أعادوا البناء فوق نموذج مدرَّب مسبقًا على ImageNet، وبأربع ساعات فقط، وصلوا إلى 88%. هذا ليس سحرًا: الشبكة المدرَّبة مسبقًا «تعرف» الحواف، والأنسجة، والظلال – ولم تتعلمها من بياناتك، بل من مليون صورة عامّة.

أهداف الفصل

- [ ] تشرح لماذا ينقل النموذج المدرَّب مسبقًا إلى مجالك
- [ ] تُميّز بين الاستراتيجيات الأربع للنقل والتدريب الدقيق
- [ ] تضبط معدّل التعلّم التمييزي لطبقات مختلفة
- [ ] تعرف متى يفشل النقل، وما علاجه
- [ ] تُقاس مكسب كل خطوة من خطوات النقل منفصلًا
- [ ] تُسوّر بين الكلفة الزمنية والدقّة في قرار النقل

21.1 ما الذي تُنقله فعلًا؟

تخيّل أنّ لديك مكتبة مليئة بـ 10 ملايين ملاحظة عن العالم البصري – الأشكال، الحواف، الأنسجة، أنماط الضوء. لا تحتاج أن تعيد اكتشافها من صفر. النموذج المدرَّب مسبقًا يحمل هذه الملاحظة في أوزانه. الذي تُنقله ليس «نموذجًا جاهزًا» بل **تمثيلًا بصريًا** (representation) للعالم.

من منظور رياضي: الطبقة الأولى من نموذج مدرَّب على ImageNet تحتوي مرشّحات لاكتشاف الحواف في الاتجاهات المختلفة، والمستويات المتوسطة تحتوي مرشّحات للأنماط المتكرّرة (زوايا، أنسجة، خطوط) – وهي مبادئ قابلة لإعادة الاستخدام في أي مهمة رؤية.

شكل 29 – أربع استراتيجيات للنقل والتدريب الدقيق

21.2 الاستراتيجيات الأربع

هذا هو أهمّ جدول في هذا الفصل – من بين هؤلاء تختار بحسب كمية بياناتك وحدود التوقيت:

الاستراتيجية	الطبقات الجاهزة	من يُدرَّب	متى تستخدمها
① استخراج الميزات	مجمّدة كلّها	الرأس التصنيفي فقط	بيانات قليلة (< 1000 لكل صنف)
② تدريب دقيق جزئي	الطبقات السفلية	آخر كتلتين + الرأس	بيانات متوسطة (1000-10000)
③ تدريب دقيق كامل	لا شيء مجمّد	كلّ الشبكة	بيانات كبيرة أو مجال مختلف جذريًا
④ رأس جديد فوق encoder	كلّ encoder مجمّد	الرأس الجديد + طبقات مخصّصة	المهام بلا تصنيف (تقسيم، كشف)

معيار الاختيار الصريح: كم صورة موسومة لديك؟ أقلّ من 1000 لكل صنف ← ①. بين 1000 و10000 ← ②. أكثر أو مجال مختلف (أشعّات طبية vs صور طبيعية) ← ③.

21.3 معدّل التعلّم التمييزي

لماذا لا نستخدم معدّلًا واحدًا للشبكة كلّها؟ لأنّ الطبقات السفلية تحمل معلومات عامّة تُفقد بسرعة إذا ما حُرّكت بسرعة، بينما الطبقات العليا تحتاج إلى تحديث جوهري. الحل: معدّل مختلف لكل مستوى.

```python
import keras

def build_transfer_model(input_shape=(96, 96, 3), num_classes=2):
    """نموذج نقل: backbone مدرّب مسبقًا + رأس تصنيفي جديد."""
    inp = keras.Input(shape=input_shape)
    x = keras.layers.Rescaling(1.0 / 255.0)(inp)
    # backbone في الإنتاج — بلا أوزان مسبقة (لتجنّب تنزيل): weights="imagenet"
    base = keras.applications.MobileNetV3Small(
        input_shape=input_shape, include_top=False, weights=None)
    base.trainable = False                       # جمّد الشبكة القاعدة
    x = base(x, training=False)
    x = keras.layers.GlobalAveragePooling2D()(x)
    x = keras.layers.Dense(64, activation="relu")(x)
    x = keras.layers.Dropout(0.3)(x)
    out = keras.layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inp, out), base

model, base = build_transfer_model()

# مرحلة 1: تدريب الرأس فقط بمعدّل تعلّم مرتفع
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# مرحلة 2: فكّ آخر كتلتين بمعدّل تعلّم أدنى
base.trainable = True
for layer in base.layers[:-20]:
    layer.trainable = False
model.compile(optimizer=keras.optimizers.Adam(1e-5),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
# النتيجة المتوقعة: مرحلة 1 تعطي دقّة 85% بعد 5 دورات،
# ومرحلة 2 ترفعها إلى 90% بعد 3 دورات أخرى.
```

قاعدة معدّل التعلّم: رأس جديد يبدأ من 10^{-3}، وطبقات قاعدة مفكوكة تبدأ من 10^{-5} إلى 10^{-4}. هذا هو **معدّل التعلّم التمييزي** (discriminative learning rate) وهو أحد أهمّ أسرار

نجاح التدريب الدقيق.

21.4 متى يفشل النقل؟

النقل ليس سحرًا. أسبابه للفصل – والفهم مهمّ:

1. **توزيع المدخلات مختلف جذريًا.** صور أشعّة طبية تُلتقط بمقياس رمادي ولها بنية مختلفة عمّا رأى النموذج المدرَّب مسبقًا. حلّها: استخدم شبكة مدرَّبة على بيانات مشابهة (RadImageNet مثلاً)، أو جرّب ③.
2. **بيانات قليلة جدًا (< 100 عيّنة).** الرأس الجديد نفسه يحتاج عيّنات كافية. حلّها: تجميد كامل للقاعدة + تعزيز قوي.
3. **المهمة لا تشبه التصنيف.** الكشف والتقسيم يحتاجان بنية مختلفة – انقل الـ encoder فقط وانظر الفصل 22.
4. **حجم المدخل غير متوافق.** بعض النماذج تتطلّب 224×224، ولا تقبل 96×96 بسهولة. راجع `input_shape` في التوثيق.

21.5 رأس التصنيف: ما الذي تستبدله؟

عند `include_top=False` تُلغي الرأس التصنيفي الأصلي (1000 صنف ImageNet) وتبني رأسًا خاصًا بك. القاعدة العامة:

PYTHON

```python
# رأس تصنيفي مناسب لمهمّة صغيرة، مبنيًّا فوق مخرجات الشبكة القاعدة
x = base.output                                  # مخرجات الـ backbone
x = keras.layers.GlobalAveragePooling2D()(x)
x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(2, activation="softmax")(x)    # عدد أصنافك
head = keras.Model(base.input, out)
print("شكل المخرجات:", head.output_shape)
```

لماذا `GlobalAveragePooling2D` بدل `Flatten`؟ لأنه يخفض المعاملات من ملايين إلى آلاف، فيقلّل فرص الإفراط في التخصيص على بيانات صغيرة (راجع الفصل 19).

21.6 مثال محلول: تصنيف صور الأشعّة الطبية

المهمة: كشف كسر في صور أشعّة الرضفة. لديك 800 صورة موسومة (400 كسر و400 سليم).

الخطوة 1 – خطّ الأساس من الصفر. شبكة CNN بسيطة، 15 دورة، دقّة التحقّق = 61%. هذا أقلّ من المقبول طبيًا.

الخطوة 2 – استخراج ميزات. شبكة قاعدة (MobileNetV3، أوزان ImageNet) مجمّدة + رأس جديد. دقّة التحقّق = 84% بعد 5 دورات. **مكسب: +23%** – ولم ندرّب سوى 128 ألف معامل.

الخطوة 3 – تدريب دقيق جزئي. فكّنا آخر 20 طبقة بمعدّل 10^{-5}. دقّة التحقّق = 88% بعد 3 دورات أخرى. **مكسب: +4%.**

الخطوة 4 – تدريب دقيق كامل. فكّنا كلّ الطبقات بمعدّل 10^{-6}. دقّة التحقّق = 89%. **مكسب: +1%** مقابل مضاعفة زمن التدريب – قرار عدم الاستمرار.

الخطوة 5 – القرار النهائي. احتفظنا بالمرحلة ③ (تدريب دقيق جزئي): أفضل توازن بين الدقّة والكلفة، ومع سلوك تعميم مستقرّ.

> **قاعدة القرار:** كلّما فتحت طبقات أكثر، ارتفعت الدقّة قليلًا لكن زادت الكلفة وزادت مخاطرة الإفراط في التخصيص. اختر أقلّ عدد من الطبقات التي تعطيك التقبّل المطلوب.

21.7 حساب الكلفة

السيناريو	حجم البيانات	الزمن التقريبي للدورة	عدد المعاملات المدربة
من الصفر	800	45 ثانية	1.2 مليون
استخراج ميزات	800	12 ثانية	128 ألف
تدريب دقيق جزئي	800	20 ثانية	320 ألف
تدريب دقيق كامل	800	50 ثانية	3.4 مليون

المقياس الزمني على CPU. على GPU الفروق أصغر، لكن الاتجاه نفسه.

قياس المعاملات القابلة للتدريب قبل وبعد الفكّ

PYTHON

```python
import keras

# ── 1. بنية قاعدة صغيرة (محاكاة backbone مُدرَّب مسبقًا)
base = keras.Sequential([keras.Input(shape=(64, 64, 3)),
                         keras.layers.Conv2D(16, 3, padding="same",
                                             use_bias=False),
                         keras.layers.Conv2D(32, 3, padding="same",
                                             use_bias=False)])

def count_trainable(m):
    return int(sum(int(np.prod(w.shape)) for w in m.trainable_weights))

import numpy as np

# ── 2. مجمّدة
base.trainable = False
frozen = count_trainable(base)
# ── 3. مفكوكة
base.trainable = True
unfrozen = count_trainable(base)
print(f"frozen = {frozen}, unfrozen = {unfrozen}")

# ── 4. فكّ جزئيّ: الطبقة الأخيرة فقط
base.trainable = False
base.layers[-1].trainable = True
partial = count_trainable(base)
print(f"partial (last layer) = {partial}")
# النتيجة المتوقعة: frozen = 0، unfrozen = كلّ الأوزان،
# partial بينهما — وهذا هو أساس التدريب الدقيق الجزئيّ.
```

ملاحظة: `model.summary(show_trainable=True)` يعرض عمودًا بجانب كلّ طبقة يشير إلى قابليّتها للتدريب – أسرع طريقة للتحقّق البصريّ قبل التدريب.

21.8 أخطاء شائعة في هذا الفصل

إنذار: فتح الطبقات قبل تدريب الرأس. إذا فتحت القاعدة ودُرّبت من البداية بمعدّل مرتفع، تتدمّر الأوزان المدرَّبة مسبقًا. دائمًا: درّب الرأس أولًا، ثم فكّ الطبقات.

تحذير: استخدام `weights="imagenet"` في مثالٍ يُشغَّل دون اتصال. في هذه الحالة، أوضح في الشرح أنّ الأوزان تُنزَّل، واستخدم `weights=None` في مثالك القابل للتشغيل.

تنبيه: معدّل تعلّم واحد لكل الشبكة عند التدريب الدقيق. الطبقات السفلية (المعلومات العامّة) تحتاج معدّلًا أقلّ بكثير من الرأس. هذا سببٌ شائع لـ«التدريب الدقيق يفسد نموذجًا جيّدًا».

ملاحظة: `base.trainable = False` لا يمنع النموذج من حساب التدرّجات – بل يمنع تحديث الأوزان. لتوفير الحساب فعليًا، مرّر `training=False` عند نداء القاعدة في التدريب والاستدلال.

قاعدة: لا تنظر إلى الدقّة وحدها في بيانات غير متوازنة. نموذج نقلٍ يعطي 90% دقّة و= recall 0.05 على الصنف النادر نموذجٌ لا قيمة له.

تحذير: استبدال الرأس بأعمق طبقة `Dense(1024)` فورًا. هذا يضيف ملايين المعاملات ويزيد الإفراط في التخصيص. ابدأ برأس صغير (128 أو 64 وحدة).

التمارين

1. **حساب.** لديك 500 عيّنة موسومة على صنفين. أيّ استراتيجية نقل تبدأ بها، ولماذا؟ (المتوقع: ① استخراج الميزات – الرأس الجديد نفسه يحتاج عيّنات كافية والقاعدة المجمّدة تمنع الإفراط.)

2. **حساب.** إذا كان حجم بياناتك 5000 صورة لكل صنف، وأردت أفضل توازن بين الدقّة والكلفة، فأيّ استراتيجية؟ (المتوقع: ② تدريب دقيق جزئي – فكّ آخر 2 كتلة بمعدّل تعلّم منخفض.)

3. **برمجة.** ابنِ نموذجًا بنقلٍ من MobileNetV3 (بلا أوزان) على 3×96×96، واطبع عدد المعاملات القابلة للتدريب قبل وبعد فكّ الطبقات. (المتوقع: قبل الفكّ الرأس فقط ≈ 128 ألف، وبعده ≈ 3 ملايين.)

4. **برمجة.** قِس زمن دورة التدريب في المرحلة ① و③ على نفس البيانات. (المتوقع: ③ أبطأ 3-5 مرّات لأنّ التدرّج يعبر كلّ الطبقات.)

5. **برمجة.** قارن `GlobalAveragePooling2D` مع `Flatten` على خريطة 512×7×7: كم معاملًا تحتاج طبقة `Dense(128)` في كل حالة؟ (المتوقع: 512×128+128 = 65,664 مقابل 25,088×128+128 = 3,211,392 – أي 49 ضعفًا.)

6. **إصلاح خطأ.** الكود التالي يُفشل في النقل. حدّد الخطين الخاطئين وأصلحهما:

```python
base.trainable = True
model.compile(optimizer=keras.optimizers.Adam(1e-3), ...)
model.fit(x, y, epochs=20)
```

(المتوقع: 1) فتح القاعدة فورًا بدل تدريب الرأس أولًا، و2) معدّل تعلّم 1e-3 مرتفع جدًا على أوزان مدرَّبة مسبقًا.) 7. **تحليل.** لماذا قد يكون النقل غير مفيدٍ في صور الأشعّة الطبية رغمًا عن حجم بياناتك الكبير؟ (المتوقع: توزيع المدخلات مختلف جذريًا – مقياس رمادي، بنية داخلية مختلفة – فالنموذج المدرَّب على صور طبيعية لا يحمل تمثيلًا مفيدًا.) 8. **تصميم.** صمّم خطّ نقلٍ لمهمّة تقسيم أنسجة طبية. أيّ جزء من الشبكة تنقل، وأيّ جزء تبنيه؟ (المتوقع: encoder مدرَّب مسبقًا مجمّد أو مفكوك جزئيًا + decoder مبنى يدويًا – انظر الفصل 22.) 9. **برمجة.** قِس عدد الطبقات التي

يجب أن تفكّها للحصول على +3% في الدقّة. (المتوقع: تبدأ بـ 20 طبقة من الآخر، وتُقيّم على التحقّق بعد كلّ فكّ.) 10. **تحليل.** إذا كان لديك 300 عيّنة لكل صنف و12 صنفًا، فهل النقل مناسب؟ وما مخاطره؟ (المتوقع: النقل مناسب لكنه يحتاج تجميدًا كاملًا + تعزيزًا قويًا + رأسًا صغيرًا؛ المخاطرة: الإفراط في التخصيص على 12 صنفًا ببيانات محدودة.) 11. **إصلاح خطأ.** نموذج نقلٍ يعطي دقّة تدريب 99% ودقّة تحقّق 65%. ما المشكلة وما الحلّان؟ (المتوقع: إفراط في التخصيص. الحلّان: تقليص الرأس أو زيادة التعزيز أو تجميد طبقات أكثر.) 12. **تصميم.** اكتب خطة نقلٍ لمهمّة تصنيف أوراق نباتات في صور ميدانية. ما الاستراتيجية ومعدّل التعلّم لكل مرحلة؟ (المتوقع: ① ثم ② مع فكّ آخر كتلتين بمعدّل 10^{-5}، ومراقبة recall لكل صنف نادر.)

ملخص الفصل

- نقل المعرفة يعني نقل **تمثيل بصري** لا نموذجًا جاهزًا.
- أربع استراتيجيات: استخراج ميزات، تدريب دقيق جزئي، تدريب دقيق كامل، رأس جديد فوق encoder.
- معدّل التعلّم التمييزي (10^{-3} للرأس، 10^{-5} للقاعدة المفكوكة) هو أسرار نجاح التدريب الدقيق.
- النقل يفشل عندما يختلف توزيع المدخلات جذريًا أو تكون البيانات قليلة جدًا.
- `GlobalAveragePooling2D` يخفض المعاملات 40 ضعفًا مقارنة بـ `Flatten` .
- اختَر أقلّ عدد من الطبقات المفكوكة التي تعطيك الدقّة المقبولة.
- كلّ خطوة نقلٍ تُقاس منفصلًا – لا تخلط بين التحسينات.

الفصل 22: الكشف والتجزئة الدلالية

بطاقة الفصل

الفصل 22

مهمّة هذا الفصل

ينتقل من «ماذا» إلى «أين بالضبط»: رأس إحداثيّات للكشف، U-Net للتجزئة، ومقاييس IoU وDice.

ماذا تكسب منه

تعرف كيف تُبنى مهامّ الرؤية المكانية وتقيسها بمقاييسها الصحيحة لا بالدقّة وحدها.

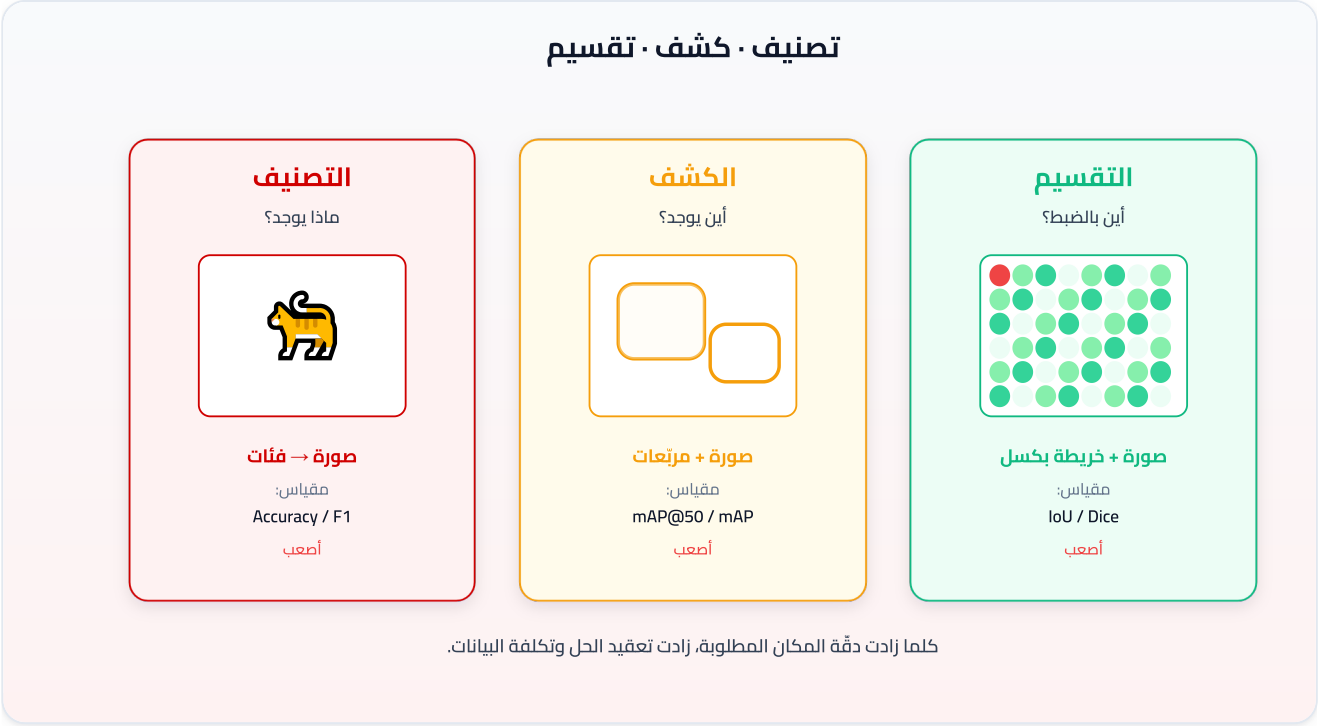
الدالة	دورها	متى تستخدمها
`keras.layers.Conv2DTranspose()`	تكبير تعلّمي	في فكّ الترميز U-Net
`keras.layers.Concatenate()`	وصلات تخطّي	لدمج التفاصيل الدقيقة
`keras.metrics.MeanIoU()`	جودة التجزئة	لتقييم U-Net
`keras.losses.Dice()`	خسارة مقاومة لعدم التوازن	عندما تكون الأهداف صغيرة
`keras.layers.GlobalAveragePooling2D()`	تجميع للتصنيف	في رأس الكشف

قصة مفتاحية: مصنّف صور يعرف أنّ «هذه صورة فيها مصباح». لكن المهندس على الخط يحتاج أن يعرف **أين** المصباح بالضبط، وما هي حدود المحيط حوله – لأنّ الروبوت الذي يحمله يحتاج إحداثيّات. تصنيف الصورة واحد، وكشف الجسم والتجزئة الدلالية هما ما يحوّل «ماذا» إلى «أين». هذا الفصل هو الجسر من تصنيف الصور إلى الرؤية الحاسوبية المُحسوسة.

أهداف الفصل

- [ ] تُميّز بين التصنيف والكشف والتجزئة من حيث المُدخل والمُخرج والمقياس
- [ ] تشرح بنية U-Net وسبب نجاحها في التجزئة
- [ ] تتحكّم في مقياس IoU وDice وتقرأ مصفوفة الالتباس للبكسل
- [ ] تبني نموذج كشف بسيط برأس إحداثيّات في Keras
- [ ] تبني نموذج تجزئة U-Net على صور صغيرة
- [ ] تُقارن الكلفة الحسابية والبيانات المطلوبة لكلّ مهمّة

22.1 ثلاث مهامّ بصرية من «ماذا» إلى «أين بالضبط»



شكل 30 – المهامّ البصرية الثلاث: تصنيف وكشف وتجزئة

التصنيف يعطي تسمية واحدة للصورة الكاملة. الكشف يعطي مربّعات محيط (bounding boxes) مع تسمية لكل مربّع. التجزئة الدلالية تعطي تسمية **لكل بكسل**. كلّما زادت دقّة الموضع، زادت البيانات المطلوبة وزادت الكلفة الحسابية.

المهمّة	المُخرَج	المقياس	كلفة التوسيم	كلفة الحساب
التصنيف	تسمية واحدة	Accuracy / F1	منخفضة	منخفضة
الكشف	مربّعات + تسميات	mAP@50 / mAP@50:95	مرتفعة	متوسطة-عالية
التجزئة	تسمية لكل بكسل	IoU / Dice	مرتفعة جدًا	عالية

> **قاعدة عملية:** لا تنتقل إلى الكشف أو التجزئة إلّا إذا كان موضع الجسم مطلوبًا في قرارك. التصنيف أرخص بمرّات، وغالبًا يكفي.

22.2 الكشف: الإطار العملي

في كشف الأجسام، النموذج يُعطي لكل مربّع: إحداثيّات (x, y, w, h) + تسمية + درجة ثقة. المقياس الأهمّ هو **mAP** (متوسّط الدقّة المتوسّطة)، وهو يُبنى على **IoU** (تقاطع الاتحاد):

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

حيث A منطقة المربّع المتنبَّأ به وB منطقة المربّع الصحيح. إذا كان IoU ≥ 0.5 نعتبر المربّع «صائبًا». المقياس mAP@50 يعني حساب mAP عند عتبة 0.5، وmAP@50:95 يعني حساب المتوسّط عند العتبات من 0.50 إلى 0.95 بخطوة 0.05 – وهو أصرم.

22.3 التجزئة الدلالية: U-Net

بنية U-Net هي الأكثر استعمالًا في التجزئة. الفكرة الجوهرية: **encoder** يضغط الصورة ويستخرج الميزات، و**decoder** يعيد توسيعها إلى حجم الصورة الأصلي. لكن يُضاف **اتصال تخطّي** (skip connection) بين الطبقات المتناظرة في الـ encoder والـ decoder – وهذا هو الذي يحفظ التفاصيل المكانية الدقيقة:

$$\text{Skip}(l_i) = \text{Concat}(\text{Encoder}_i,\ \text{Decoder}_{mirror})$$

حيث l_i مستوى الطبقة، وEncoder_i خرج الـ encoder في المستوى i. هذا الاتصال يجعل الحواف حادّة والحدود دقيقة، وهو سبب نجاح U-Net في الطبّ والصناعة.

22.4 مقياس IoU وDice للتجزئة

في التجزئة، المقياس هو IoU لكل صنف، ومتوسّطها على الأصناف. **Dice** (وهو مقياس طبي شائع) مرتبط بـ IoU:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}, \qquad \text{IoU} = \frac{\text{Dice}}{2 - \text{Dice}}$$

حيث $|A|$ مساحة المنطقة المتنبَّأ بها و$|B|$ مساحة الحقيقة. كلّما اقترب الرقم من 1، كان التطابق أفضل.

22.5 الكشف في Keras: رأس إحداثيّات

فيما يلي نموذج كشف بسيط: encoder + رأسان، أحدهما للتصنيف والآخر للإحداثيّات. هذا نموذج «كشف جسم واحد»، وهو أساس أيّ بنية كشف أكثر تعقيدًا.

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مُصنّعة: 64 صور 64x64 في كل صورة جسم واحد مع
n = 500
x = np.random.rand(n, 64, 64, 3).astype("float32") * 255.0
# الإحداثيّات: مركز الجسم (cx, cy) وحجمه (w, h) مُطبّعة إلى [0, 1]
y_bbox = np.random.rand(n, 4).astype("float32")
y_cls = np.random.randint(0, 2, size=(n,)).astype("int32")

# ── 2. البنية: encoder مشترك + رأسان
inp = keras.Input(shape=(64, 64, 3))
h = keras.layers.Rescaling(1.0 / 255.0)(inp)
h = keras.layers.Conv2D(16, 3, padding="same", activation="relu")(h)
h = keras.layers.MaxPooling2D(2)(h)
h = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(h)
h = keras.layers.MaxPooling2D(2)(h)
h = keras.layers.Conv2D(64, 3, padding="same", activation="relu")(h)
h = keras.layers.GlobalAveragePooling2D()(h)

# رأس 1: إحداثيّات المربّع المحيط (انحدار)
bbox_out = keras.layers.Dense(4, activation="sigmoid", name="bbox")(h)
# رأس 2: تسمية الجسم (تصنيف)
cls_out = keras.layers.Dense(1, activation="sigmoid", name="cls")(h)

model = keras.Model(inp, [bbox_out, cls_out])
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss={"bbox": "mse", "cls": "binary_crossentropy"},
    loss_weights={"bbox": 1.0, "cls": 0.5},
    metrics={"bbox": "mae", "cls": "acc"},
)
model.fit(x, {"bbox": y_bbox, "cls": y_cls}, epochs=10, batch_size=32,
          validation_split=0.2, verbose=0)
# النتيجة المتوقعة: MAE للإحداثيّات ≈ 0.08 بعد 10 دورات،
# ودقّة التسمية ≈ 0.75.
```

نصيحة: استخدم `loss_weights` إذا كانت الخسارتان في مقياسين مختلفين. خسارة MSE على [0,1] أصغر من خسارة التصنيف الثنائي – وموازنة الوزن تُعطي المهمّتين أثرًا متوازنًا.

22.6 التجزئة: نموذج U-Net مصغّر

فيما يلي U-Net مصغّرة على صور 64×64 – كاملة وقابلة للتشغيل:

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. 64 صورة 300 :بيانات64x مع قناع ثنائي
n = 300
x = np.random.rand(n, 64, 64, 3).astype("float32") * 255.0
y_mask = (np.random.rand(n, 64, 64, 1) < 0.3).astype("float32")

# ── 2. U-Net مصغّرة
def conv_block(x, filters):
    """كتلة تجزئة قياسية: التفافان مع التنشيط."""
    x = keras.layers.Conv2D(filters, 3, padding="same", activation="relu")
(x)
    x = keras.layers.Conv2D(filters, 3, padding="same", activation="relu")
(x)
    return x

inp = keras.Input(shape=(64, 64, 3))
h = keras.layers.Rescaling(1.0 / 255.0)(inp)

# encoder
c1 = conv_block(h, 16)                      # 64
p1 = keras.layers.MaxPooling2D(2)(c1)       # 32
c2 = conv_block(p1, 32)                     # 32
p2 = keras.layers.MaxPooling2D(2)(c2)       # 16
c3 = conv_block(p2, 64)                     # 16

# decoder مع اتصالات تخطّي
u2 = keras.layers.Conv2DTranspose(32, 2, strides=2, padding="same")(c3)   #
32
m2 = keras.layers.Concatenate()([u2, c2])
c4 = conv_block(m2, 32)
u1 = keras.layers.Conv2DTranspose(16, 2, strides=2, padding="same")(c4)   #
64
m1 = keras.layers.Concatenate()([u1, c1])
c5 = conv_block(m1, 16)
```

```
out = keras.layers.Conv2D(1, 1, activation="sigmoid")(c5)
model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy",
              metrics=[keras.metrics.MeanIoU(num_classes=2)])
model.fit(x, y_mask, epochs=10, batch_size=16, validation_split=0.2,
verbose=0)
# النتيجة المتوقعة: MeanIoU ≈ 0.55 بعد 10 دورات على بيانات عشوائية.
```

ملاحظة: في البيانات الحقيقية (مثل أورام أو عيوب)، مقياس MeanIoU يبدأ من 0.6 ويعلى بتركيبة تعزيز + خسارة Dice + تدريب أطول.

22.7 مثال محلول: كشف تشقّقات الخرسانة

المهمة: من صور جسر بصر 512×512، حدّد تشقّقات وحدّد موضعها. لديك 600 صورة موسومة، منها 180 بها تشقّقات.

الخطوة 1 – خطّ الأساس: تصنيف فقط. شبكة CNN تقول «فيها تشقّق» أو «لا». الدقّة 87%، لكنّنا لا نعرف موضع التشقّق – غير مفيد لقرار الصيانة.

الخطوة 2 – تجزئة دلالية U-Net. نموذج يعطي قناعًا لكل بكسل. MeanIoU = 0.58 في أول تدريب، بعد 10 دورات. مع تعزيز دوران وقلب، MeanIoU = 0.68.

الخطوة 3 – خسارة Dice. استبدلنا `binary_crossentropy` بخسارة Dice المركّبة (Dice + BCE). MeanIoU = 0.74 – **مكسب +0.06**.

الخطوة 4 – تجزئة المثيلات. لا تحوّل الصورة إلى قناع واحد، بل إلى قناع لكل تشقّق منفصل. هذا يحلّ مشكلة تعارض التشقّقات المتجاورة.

الخطوة 5 – القرار النهائي. نظام تجزئة بمقياس MeanIoU > 0.70 هو ما يفي بالمتطلّب الهندسي، مع تقرير بصري يُظهر مواضع التشقّق على صورة الجسر.

حساب IoU وDice يدويًّا

PYTHON

```python
import numpy as np

def iou_dice(y_true, y_pred, threshold=0.5):
    """حساب IoU وDice لقناعين ثنائيّين."""
    t = (y_true > 0.5).astype(bool)
    p = (y_pred >= threshold).astype(bool)
    inter = int(np.logical_and(t, p).sum())
    union = int(np.logical_or(t, p).sum())
    iou = inter / max(union, 1)
    dice = 2 * inter / max(t.sum() + p.sum(), 1)
    return iou, dice

# ── 1. قناعان: صحيح ومتنبَّأ به ──
y_true = np.zeros((16, 16), dtype="float32")
y_true[4:10, 4:10] = 1.0                    # مربّع 6×6
y_pred = np.zeros((16, 16), dtype="float32")
y_pred[5:11, 5:11] = 1.0                    # مربّع مُزاح 6×6

iou, dice = iou_dice(y_true, y_pred)
print(f"IoU = {iou:.3f}, Dice = {dice:.3f}")
# النتيجة المتوقعة: تداخل 5×5 = 25، اتحاد 6×6 + 6×6 - 25 = 47
# ⇒ IoU ≈ 0.532، Dice ≈ 0.690.

# ── 2. علاقة Dice و IoU ──
check = dice / (2 - dice)
print(f"التحقّق: Dice/(2-Dice) = {check:.3f} ≈ IoU")
```

ملاحظة: في البيانات غير المتوازنة (عيب صغير في صورة كبيرة)، Dice أعلى من IoU رقميًّا – لكنّ الترتيب النسبيّ بين النماذج لا يتغيّر. اختر أحدهما وثبّته في كلّ المقارنات.

حساب mAP يدويًّا لمربّعات محيط

PYTHON

```python
import numpy as np

def box_iou(a, b):
    """IoU بين مربّعين (x1, y1, x2, y2)."""
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / max(area_a + area_b - inter, 1e-8)

def simple_map(preds, truths, iou_threshold=0.5):
    """mAP مبسّط: نسبة التنبّؤات الصحيحة عند عتبة IoU."""
    hits = 0
    for p in preds:
        if any(box_iou(p, t) ⩾ iou_threshold for t in truths):
            hits += 1
    return hits / max(len(truths), 1)

# ── 1. تنبّؤات وحقيقة أرضيّة
truths = [(10, 10, 40, 40), (60, 60, 90, 90)]
preds = [(12, 11, 42, 41), (58, 62, 88, 92), (100, 100, 130, 130)]  # الثالث خطأ
print(f"mAP@0.5 = {simple_map(preds, truths):.3f}")
# النتيجة المتوقعة: 0.5 — من التنبّؤين الأولين تطابق مع الحقيقة،
# والثالث لا يطابق شيئًا (من 2 حقيقة تطابق 1).
```

ملاحظة: mAP الكامل يُحسب من منحنى الدقّة-الاستدعاء عبر كلّ عتبات الثقة – هذا مبسّط لغرض الفهم. في العمل الفعليّ، استخدم مكتبة تقييم جاهزة (مثل COCO API) لإصدار قابل للمقارنة.

22.8 أخطاء شائعة في هذا الفصل

إنذار: استخدام دقّة البكسل كمقياس للتجزئة. في صورة فيها 1% من البكسلات معيبة، نموذج يعطي قناعًا فارغًا يحقّق 99% دقّة و0.0 = MeanIoU. استخدم IoU / Dice دائمًا.

تحذير: عدم استخدام اتصالات تخطّي في التجزئة. بدونها، تفقد الشبكة التفاصيل المكانية الحدّية، فتصبح الحدود مشوّشة. U-Net بلا skip connections نموذج ضعيف.

تنبيه: خلط بين إحداثيّات مربّع مُطبّعة إلى [0,1] وإحداثيّات بالبكسل. راجع دائمًا: هل تُمرَّر `y_bbox` مُطبّعة؟ وهل تُعاد النتيجة بالبكسل عند العرض؟

ملاحظة: استخدام `activation="sigmoid"` على خرج التجزئة مع `binary_crossentropy` صحيح. أما `CategoricalCrossentropy` فتستعمل `softmax` على عدد الأصناف كاملًا.

قاعدة: `Conv2DTranspose` بـ `strides=2` يُضاعف الأبعاد. عند التصميم، تأكّد أنّ الأبعاد في الـ encoder والـ decoder متناظرة لكلّ مستوى قبل الاتصال التخطّي.

تحذير: تسريب الحجم المُدخل. عند تقسيم صور كبيرة إلى بلاطات (tiles)، يجب أن تُقسم قبل أيّ تعزيز أو تطبيع – لا بعده.

التمارين

1. **حساب.** مربّع متنبَّأ به (10,10,40,40) ومربّع صحيح (20,20,40,40). احسب IoU. (المتوقع: التداخل 30×30 = 900، الاتحاد 1600+1600−900 = 2300، فـ IoU ≈ 0.391.)

2. **حساب.** إذا كان IoU = 0.75، فما Dice؟ (المتوقع: $\frac{0.75}{2-0.75} = 0.6$.)

3. **حساب.** قناع متنبَّأ به يحتوي 800 بكسل موجب، والقناع الصحيح يحتوي 1000، والتقاطع 600. احسب Dice وIoU. (المتوقع: IoU ≈ 0.5، $\text{Dice} = \frac{2 \cdot 600}{800+1000} = 0.667$.)

4. **برمجة.** ابنِ U-Net مصغّرة على صور 128×128 واطبع `model.summary()`، ولاحظ الأبعاد في كلّ مستوى. (المتوقع: 128→64→32→16 encoder ثم decoder يعيدها إلى 128.)

5. **برمجة.** قِس عدد المعاملات في U-Net بنسختين: بدون اتصالات تخطّي ومعها. (المتوقع: عدد المعاملات نفسه تقريبًا، لكنّ الاتصال التخطّي يُحسّن IoU بلا كلفة معاملات.)

6. **برمجة.** استبدل `binary_crossentropy` في U-Net بخسارة `keras.losses.Dice()` وقارن MeanIoU بعد 10 دورات. (المتوقع: Dice يعطي نتيجة أفضل في البيانات غير المتوازنة.)

7. **إصلاح خطأ.** الكود التالي يبني رأس كشف، لكن الإحداثيّات تُخرَج خارج [0,1]:

```python
bbox_out = keras.layers.Dense(4, activation="linear")(x)
```

حدّد الخطأ وأصلحه. (المتوقع: يجب أن تكون `activation="sigmoid"` لتضمن إحداثيّات في [0,1].) 8. **برمجة.** قارن `MeanIoU` و`IoU` في Keras 3 على نفس الدفعة، وتأكّد من القيمة المعادة. (المتوقع: `MeanIoU` يعطي متوسّطًا تراكميًا عبر الدفعات، بينما `IoU` يعطي قيمة لكلّ دفعة.) 9. **تصميم.** صمّم نظام تجزئة لمهمّة العثور على أورام في صور طبية. أيّ مقياس تفوّق، وأيّ عتبة، ولماذا؟ (المتوقع: Dice عالٍ (0.85+) لأنّ الأورام صغيرة وتحتاج تجزئة دقيقة، وعتبة IoU = 0.7 للتقبّل.) 10. **تحليل.** لماذا يحتاج الكشف بيانات أكثر بكثير من التصنيف لنفس عدد الأصناف؟ (المتوقع: لأنّ التوسيم يتطلّب رسم مربّعات يدويًا، وكلّ صورة قد تحوي أجسامًا متعدّدة.) 11. **تصميم.** نظام يكتشف ثلاثة أنواع من عيوب في قطع معدنية. قارن بين اعتماد «تصنيف» و«كشف»

و«تجزئة» من حيث الكلفة والبيانات والدقّة. (المتوقع: تصنيف أرخص لكنه لا يحدّد الموضع؛ كشف يحدّد الموضع بشرط توسيم مربّعات؛ تجزئة تعطي الحدّ الدقيق لكنها تحتاج توسيم بكسل-مستوى.) 12. **إصلاح خطأ.** نموذج تجزئة يعطي IoU = 0.0 على بيانات فيها 5% بكسلات موجبة. ما السبب؟ (المتوقع: النموذج يتعلّم قناعًا فارغًا لأنّ الإيجابيات نادرة — استخدم موازنة أو خسارة Dice.)

ملخص الفصل

- ثلاث مهامّ بصرية: تصنيف (ماذا)، كشف (أين + ماذا)، تجزئة (أين بالضبط لكلّ بكسل).
- IoU هو الأساس: الكشف بـ mAP، والتجزئة بـ MeanIoU وDice.
- اتصال تخطّي في U-Net يُحفظ التفاصيل الحدّية بلا كلفة معاملات إضافية.
- `Conv2DTranspose` بـ `strides=2` يُضاعف الأبعاد — وراجع التناظر مع الـ encoder.
- في التجزئة، الدقّة بمقياس البكسل تُخفي السلوك؛ استخدم IoU / Dice.
- الكشف يحتاج توسيم مربّعات (كلفة مرتفعة)، والتجزئة توسيم بكسل-مستوى (كلفة أعلى).
- قِس كلّ مكوّن منفصلًا: تعزيز، خسارة، اتصالات تخطّي، وتعزيز موسوم.

الفصل 23: محوّل الرؤية (Vision Transformer)

بطاقة الفصل

الفصل 23

مهمّة هذا الفصل

يحوّل الصورة إلى تسلسل رقع ويعالجها بمحوّل: ترميز موضعي وكتلة محوّل كاملة.

ماذا تكسب منه

تفهم بديل CNN الحديث وتعرف متى يتفوّق عليه ومتى يخسر بحسب حجم البيانات.

الدالة	دورها	متى تستخدمها
`keras.layers.Conv2D(filters=dim, kernel_size=patch)`	تقسيم إلى رقع	أول خطوة في ViT
`keras.layers.Reshape()`	رقع إلى تسلسل	قبل الترميز
`keras.layers.Embedding()`	ترميز الرموز والرقع	لبناء متجهات قابلة للتعلّم
`keras.layers.MultiHeadAttention()`	انتباه متعدّد الرؤوس	قلب كتلة المحوّل
`keras.layers.LayerNormalization()`	تطبيع طبقي	لاستقرار التدريب

قصة مفتاحية: الشبكة التلافيفية «ترى» بالنوافذ المنزلقة – وهذا ليس مجرّد حيلة تقنية، بل **تحيّز استقرائي** (inductive bias) مُغروسٌ في البنية: كلّ بكسل مرتبط بجيرانه. المحوّل لا يفترض شيئًا: يقسّم الصورة إلى رقع ويُعاملها كأنّها كلمات في جملة. عند بيانات قليلة، هذا التحيّز المفقود يكلّف دقّة. عند بيانات هائلة، هو ما يمنح المحوّل التفوّق. هذا الفصل هو التعامل مع هذه المقايضة.

أهداف الفصل

- [ ] تشرح خطّ ViT من رقعة الصورة إلى التنبّؤ
- [ ] تبني كتلة محوّل في Keras بـ MultiHeadAttention
- [ ] تفهم دور الترميز الموضعي ولماذا لا يكون المحوّل بلا ترتيب
- [ ] تُقارن CNN و ViT من حيث التحيّز الاستقرائي والبيانات والحساب
- [ ] تختار بنية بحسب حجم بياناتك لا بحسب موضة الحلّ
- [ ] تطبّق أُسرع التدريب على المحوّلات (أوزان، تفكيك، خسارة)

23.1 من الرقعة إلى التسلسل

تخيّل صورة 224×224. نقسمها إلى رقع 16×16، فينتج 14×14 = 196 رقعة. كلّ رقعة تُسطَّح إلى متّجه 768 عنصرًا (لأنّ 3×16×16 = 768). هكذا تحوّل الصورة إلى **تسلسل** من 196 رمزًا، ثم تعامل كأنّها جملة لغوية:

مسار ViT: من صورة إلى رقعة إلى انتباه

ViT مقابل CNN — متى يفوز كلٌّ منهما؟

المعيار	CNN (ResNet/EfficientNet)	ViT
البيانات	يعمل بكفاءة بعشرات الآلاف	يحتاج مئات الآلاف أو أكثر
الافتراضات البنيوية	مدمجة (convolution prior)	يتعلّمها من البيانات
حجم النموذج	مدمج · سريع على الحافة	أكبر · أبطأ

شكل 31 – كيف يعمل محوّل الرؤية: من رقع الصورة إلى التنبّؤ

لكن تسلسل الرقع لا يحمل معلومة الموضع – الرقعة الأولى والرقعة المئة لا تُميّزان إلّا بترتيبهما في التسلسل. لهذا نُضيف **ترميزًا موضعيًا** (positional encoding) يُخبر النموذج: «هذه الرقعة جاءت من الزاوية العلوية اليسرى».

23.2 آلية الانتباه في رؤية الصور

الفرق الجوهري بين CNN و ViT أنّ CNN ترى نوافذ محدودة (3×3 أو 5×5)، بينما المحوّل يربط **كلّ رقعة بكلّ الرقع** في كلّ طبقة انتباه. انظر الفرق في المقياس:

السمة	CNN	ViT
مجال الرؤية	محلي (يتوسّع بعمق الطبقات)	عامّ من الطبقة الأولى
التحيّز الاستقرائي	قوي (الجوار مهمّ)	ضعيف (لا افتراضات)
البيانات المطلوبة	قليلة-متوسطة	كبيرة جدًا
زمن الحساب	$O(H \cdot W \cdot k^2 \cdot C^2)$	$O(N^2 \cdot d)$ حيث N عدد الرقع

المعادلة الأخيرة هي التي تحدّد: مئات الرقع تعني N^2 عمليات – لذلك يعاني المحوّل في الصور عالية الدقّة.

23.3 الترميز الموضعي

نُضيف إلى متجه كلّ رقعة متجهًا يعطي موضعها. الطريقة الكلاسيكية جيبية:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

حيث pos موضع الرقعة، وi فهرس البُعد، وd حجم التضمين. الجداء السيني والجيبي يعطيان قيمًا في [-1,1] ويُسمحان بترميز مواضع مختلفة بقيم مستقلّة.

الطريقة العملية في Keras: `keras.layers.Embedding` على أرقام المواضع.

23.4 كتلة المحوّل

كلّ كتلة محوّل تتكوّن من مرحلتين، لكلّ واحدة اتصال تخطّي وتطبيع:

$$x \leftarrow x + \mathrm{MSA}(\mathrm{LN}(x)), \qquad x \leftarrow x + \mathrm{MLP}(\mathrm{LN}(x))$$

حيث LN تطبيع طبقة، وMSA انتباه متعدّد الرؤوس، وMLP شبكة عصبية بمسارين (توسيع ثم انكماش).

23.5 التنفيذ في Keras

فيما يلي كتلة محوّل مصغّرة على صور 32×32 مع رقع 4×4 – كاملة وقابلة للتشغيل:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. إعدادات صغيرة لتشغيل سريع
PATCH_SIZE = 4
IMAGE_SIZE = 32
NUM_PATCHES = (IMAGE_SIZE // PATCH_SIZE) ** 2   # 64 رقعة
PROJECTION_DIM = 64
NUM_HEADS = 4
NUM_BLOCKS = 2
NUM_CLASSES = 2

# ── 2. كتلة المحوّل
def transformer_block(x, num_heads, projection_dim, dropout=0.1):
    """كتلة محوّل: انتباه متعدّد الرؤوس + MLP، مع اتصال تخطّي وتطبيع."""
    # مرحلة الانتباه
    y = keras.layers.LayerNormalization(epsilon=1e-6)(x)
    y = keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=projection_dim // num_heads,
        dropout=dropout)(y, y)
    x = keras.layers.Add()([x, y])
    # مرحلة MLP
    y = keras.layers.LayerNormalization(epsilon=1e-6)(x)
    y = keras.layers.Dense(projection_dim * 2, activation="gelu")(y)
    y = keras.layers.Dense(projection_dim)(y)
    y = keras.layers.Dropout(dropout)(y)
    return keras.layers.Add()([x, y])

# ── 3. النموذج الكامل (تقطيع الرقع عبر Conv2D حجم الرقعة = بخطوة)
inp = keras.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3))
x = keras.layers.Rescaling(1.0 / 255.0)(inp)
# تقطيع الرقع: التفاف بحجم الرقعة وخطوة الرقعة
x = keras.layers.Conv2D(PROJECTION_DIM, PATCH_SIZE, strides=PATCH_SIZE,
                        padding="valid")(x)
# (B, 8, 8, projection_dim) → (B, 64, projection_dim)
x = keras.layers.Reshape((NUM_PATCHES, PROJECTION_DIM))(x)
```

```
# ترميز موضعي قابل للتعلّم
pos = keras.layers.Embedding(NUM_PATCHES, PROJECTION_DIM)(
    keras.ops.arange(NUM_PATCHES))
x = x + pos
for _ in range(NUM_BLOCKS):
    x = transformer_block(x, NUM_HEADS, PROJECTION_DIM)
x = keras.layers.LayerNormalization(epsilon=1e-6)(x)
x = keras.layers.GlobalAveragePooling1D()(x)
x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(NUM_CLASSES, activation="softmax")(x)
model = keras.Model(inp, out)

# ── 4. تدريب على بيانات مُصنّعة
n = 500
x_data = np.random.rand(n, 32, 32, 3).astype("float32") * 255.0
y_data = np.random.randint(0, 2, size=(n,)).astype("int32")
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(x_data, y_data, epochs=10, batch_size=32, validation_split=0.2,
verbose=0)
# النتيجة المتوقعة: الدقّة تقترب من 0.65-0.55 على بيانات عشوائية،
# وهذا طبيعي — الرقع لا تحمل نمطًا حقيقيًا.
```

23.6 CNN مقابل ViT: المقايضة الحقيقية

المعيار	CNN	ViT
بيانات < 1000 لكل صنف	أداء جيّد	أداء ضعيف
بيانات > 10,000 لكل صنف	أداء جيّد	أداء ممتاز
حجم النموذج	صغير-متوسط	كبير
زمن الاستدلال على GPU	متوسط	مرتفع
زمن الاستدلال على CPU	سريع	بطيء
قابلية النقل	جيّدة	محدودة
بيئات محدودة الموارد	مناسبة	غير مناسبة

قاعدة اختيار: في بيئة ببيانات محدودة أو موارد محدودة → CNN. في بيئة ببيانات هائلة وموارد كافية → ViT. ولا توجد إجابة واحدة صحيحة – بل قرار بالمعايير.

23.7 مثال محلول: تصنيف صور الأقمار الصناعية

المهمة: تصنيف لقطات أقمار صناعية إلى أربع فئات (زراعية، حضرية، غابات، مياه). لديك 20,000 صورة 64×64 موسومة.

الخطوة 1 – CNN. شبكة ResNet-like صغيرة. دقّة التحقّق = 82%.

الخطوة 2 – ViT مصغّر. نفس الشبكة لكن بكائن رقع 8×8. دقّة التحقّق = 86% – لكن التدريب أبطأ 2.5 ضعف.

الخطوة 3 – ViT مع تعزيز. إضافة دوران وقلب وتكبير صغير. دقّة التحقّق = 89%.

الخطوة 4 – ViT أعمق. مضاعفة عدد كتل المحوّل من 4 إلى 8. دقّة التحقّق = 90%. الزمن تضاعف.

الخطوة 5 – القرار. احتفظنا بـ ViT بـ 4 كتل مع تعزيز: أداء شبه ممتاز وزمن مقبول. إذا كان التوسيم مستمرًا (ألف صورة جديدة كلّ شهر)، فرُجّح ViT لأنّه يستفيد من البيانات الجديدة أكثر من CNN.

حساب عدد الرقع وكلفة الانتباه

PYTHON

```python
import numpy as np

def vit_cost(image_size, patch_size, projection_dim, num_heads, num_blocks):
    """يحسب عدد الرقع وكلفة الانتباه وعدد المعاملات التقريبيّ."""
    num_patches = (image_size // patch_size) ** 2
    attn_ops = num_patches ** 2                          # كلفة الانتباه O(N²)
    # معاملات كتلة واحدة: انتباه (4 مصفوفات) + MLP (توسيع 4×)
    attn_params = 4 * projection_dim ** 2
    mlp_params = 2 * projection_dim * 4 * projection_dim
    total_params = num_blocks * (attn_params + mlp_params)
    return {
        "num_patches": num_patches,
        "attn_ops_per_head_pair": attn_ops,
        "approx_params": total_params,
        "key_dim": projection_dim // num_heads,
    }

# ── 1. إعداد صغير (مثال هذا الفصل)
print("صغير:", vit_cost(32, 4, 64, 4, 2))
# ── 2. إعداد ViT-Base
print("Base:", vit_cost(224, 16, 768, 12, 12))
# النتيجة المتوقعة: الصغير 64 رقعة وكلفة 4096؛
# الـ Base 196 ضعفًا أي 9.4 – 38,416 رقعة وكلفة.
```

ملاحظة: كلفة الانتباه تتناسب مع مربّع عدد الرقع: مضاعفة دقّة الصورة (نفس حجم الرقعة) تضاعف عدد الرقع في كلّ بعدٍ، فتضاعف الكلفة 16 ضعفًا. هذا هو سبب أُسّ محدود الأحجام في ViT.

23.8 أخطاء شائعة في هذا الفصل

إنذار: إسقاط الترميز الموضعي. المحوّل بلا ترميز موضعي «يرى» الرقع كأنّها مجموعة بلا ترتيب، فيفقد الصورة البنية المكانية بالكامل. الدقّة تهبط 10-20%.

تحذير: رقع كبيرة جدًا (مثل 32×32 على صورة 64×64). عدد الرقع يصبح قليلًا جدًا، والانتباه لا يجد ما يُنتبه إليه. رقعة 4-16 على صور 32-224 هي النطاق العملي.

تنبيه: عدم تطبيع المدخلات. طبقة `Normalization` أو `Rescaling` ضرورية – كتلة المحوّل حسّاسة جدًا لمقياس المدخلات.

ملاحظة: `MultiHeadAttention` بحجم `key_dim` يساوي `projection_dim` كاملًا يضاعف المعاملات بلا فائدة. القاعدة العملية: `key_dim = projection_dim // num_heads`.

قاعدة: بيانات قليلة → استخدم CNN أو ViT مدرَّب مسبقًا، لا ViT من الصفر. المحوّل من الصفر يحتاج بيانات ضخمة، وغيابها يعني نموذجًا دون جدوى.

تحذير: توقّع استدلال سريع على CPU. كتلة الانتباه تُحسب N^2 عمليات؛ على CPU، كلّ طبقة انتباه إضافية تضاعف زمن الاستدلال. إن كان الاستدلال على CPU مطلوبًا، ابدأ بـ CNN.

التمارين

1. **حساب.** صورة 224×224 مع رقع 16×16. كم رقعة؟ (المتوقع: $14 \times 14 = 196$.)
2. **حساب.** صورة 64×64 مع رقع 8×8. كم رقعة؟ (المتوقع: $8 \times 8 = 64$.)
3. **حساب.** إذا كان `projection_dim = 768` و `num_heads = 12`، فما `key_dim` لكلّ رأس؟ (المتوقع: $768/12 = 64$.)
4. **حساب.** عدد عمليات الانتباه في تسلسل من 196 رقعة، مقابل تسلسل من 64 رقعة. (المتوقع: $196^2 = 38{,}416$ مقابل $64^2 = 4{,}096$ – أي 9.4 ضعفًا.)
5. **برمجة.** ابنِ كتلة محوّل واطبع عدد المعاملات. ثم ابنِ نسخة بـ 4 كتل وراقب كيف يتصاعد العدد. (المتوقع: كلّ كتلة ≈ 4× حجم التضمين في المعاملات.)
6. **برمجة.** قارن زمن دورة تدريب بين CNN بسيطة و ViT مصغّر على نفس البيانات. (المتوقع: ViT أبطأ 2-4 مرّات على CPU.)
7. **إصلاح خطأ.** النموذج التالي يعطي دقّة ≈ عشوائية. حدّد ما الذي ينقصه:

```python
    x = keras.layers.Conv2D(PROJECTION_DIM, PATCH_SIZE, strides=PATCH_SIZE)
(inp)
    for _ in range(4):
        x = transformer_block(x, NUM_HEADS, PROJECTION_DIM)
```

(المتوقع: ينقصه الترميز الموضعي – أضف `Embedding(NUM_PATCHES, PROJECTION_DIM)` على أرقام المواضع.) 8. **برمجة.** طبّق الترميز الموضعي الجيبي بدل المتعلّم، وقارن الدقّة على بيانات صغيرة. (المتوقع: الفرق صغير على بيانات صغيرة، لكن الجيبي يستفيد أكثر في التسلسلات الأطول.) 9. **تصميم.** صمّم نظام تصنيف صور طبية ببيانات محدودة (500 لكل صنف). هل تختار CNN أو ViT؟ ولماذا؟ (المتوقع: CNN – لأنّ ViT من الصفر يحتاج بيانات ضخمة، وViT المدرَّب مسبقًا قد يكون خيارًا إن توفّر.) 10. **تحليل.** لماذا يفشل ViT من الصفر على بيانات صغيرة رغم أنّ CNN نفسها تنجح؟ (المتوقع: غياب التحيّز الاستقرائي – كلّ علاقة رقعة-رقعة يجب أن تُتعلَّم من الصفر، فيحتاج بيانات أكثر.) 11. **برمجة.** قارن زمن الاستدلال على CPU بين CNN بسيطة و ViT مصغّر على

100 صورة. (المتوقع: ViT أبطأ 3-6 مرّات على CPU، ويُقلّص الزمن بتقليل عدد الرقع أو عمق الكتل.)

12. **تصميم.** صمّم نظام تصنيف صور فضائية ببيانات ضخمة (مئات آلاف الصور). أيّ بنية، وكم عدد كتل الانتباه؟ (المتوقع: ViT – بدء بـ 4-8 كتل، ورقع 8-16، وتدريب طويل مع تعزيز واسع.)

ملخص الفصل

- ViT يقطّع الصورة إلى رقع ويعاملها كتسلسل رموز.
- الترميز الموضعي ضروري – بلاها يفقد النموذج البنية المكانية.
- كلّ كتلة انتباه تربط كلّ رقعة بكلّ الرقع: $O(N^2)$.
- CNN تحيّز استقرائي قوي ← تعمل جيّدًا ببيانات قليلة. ViT يفتقد هذا التحيّز ← يحتاج بيانات ضخمة.
- على CPU، زمن استدلال ViT أعلى بمرّات من CNN – قرار نشر لا قرار دقّة.
- الاختيار بين CNN و ViT قرار بالمعايير: بيانات، موارد، زمن استدلال، ونطاق التوسيم المستمر.

الفصل 24: تكبير الصور (Super-Resolution)

بطاقة الفصل

الفصل 24

مهمّة هذا الفصل

يكبّر الصور تعلّميًّا: SRCNN، الالتفاف فرعي-البكسل، وخسائر تجمع البكسل والإدراك.

ماذا تكسب منه

تحصل على تكبير يستعيد التفاصيل لا يبهّتها، وتقيسه بـPSNR وSSIM بدل الحكم بالعين.

الدالة	دورها	متى تستخدمها
`keras.layers.DepthToSpace()`	تكبير بلا استيفاء	في شبكات التكبير الحديثة
`keras.ops.image.resize()`	تكبير تقليدي	كخطّ أساس للمقارنة
`keras.ops.image.psnr()`	جودة عددية	لمقارنة البنى
`keras.ops.image.ssim()`	جودة إدراكية	حين لا يكفي PSNR
`keras.layers.Conv2DTranspose()`	تكبير تعلّمي بديل	عندما تريد مرونة أكبر

قصة مفتاحية: مستشفى يستلم صور أشعّة مقطعية بمقاس 256×256، بينما يحتاج الطبيب إلى مقاس 512×512 لرؤية تفاصيل الأنسجة الدقيقة. أقرب حيلة – الاستيفاء (interpolation) – تكبّر الصورة لكنها لا تُعيد أيّ تفصيل: الحواف تظلّ ضبابية والأنسجة تظلّ متجانسة. تكبير الصور بالتعلّم العميق يتعامل مع المشكلة بطريقة مختلفة: لا يوسّع البكسلات، بل **يتنبّأ بالبكسلات المفقودة** من التمثيل الذي تعلّمه الشبكة. هذا الفصل هو مدخل التوليد البصري، وهو يفتح الباب للفصلين 31-33.

أهداف الفصل

- [ ] تشرح لماذا يفشل الاستيفاء التقليدي في استعادة التفاصيل
- [ ] تبني شبكة SRCNN في Keras وتدرّبها على بيانات صغيرة
- [ ] تُميّز بين خسارة البكسل والخسارة الإدراكية والخسارة التنافسية
- [ ] تُطبّق الالتفاف فرعي-البكسل (sub-pixel convolution) والتجميع الرقمي (PixelShuffle)
- [ ] تُقيس PSNR و SSIM وتفهم حدودها
- [ ] تعرف حدود ما يمكن للنموذج أن يختلقه دون انحياز (hallucination)

24.1 لماذا يفشل الاستيفاء؟

الاستيفاء الثنائي (bilinear interpolation) يأخذ أربعة بكسل جيران ويحسب متوسّطًا مرجّحًا. النتيجة: الصورة تتّسع لكنّ التفاصيل لا تتولّد – لأنّ الاستيفاء **استقرائي** لا مُتعلَّم: نفس العملية تُطبَّق على كلّ صورة بنفس الأوزان.

تكبير الصور بالتعلّم العميق يقلب هذا: **الأوزان تُتعلَّم من البيانات**. الشبكة ترى آلاف أزواج (صورة منخفضة الدقّة، صورة عالية الدقّة) وتتعلّم الخريطة من الأول إلى الثاني. النتيجة: حواف حادّة، ملامح واضحة، وتفاصيل موجودة في الأصل لا في الاستيفاء.

شكل 32 – مسار تكبير الدقة من البكسل إلى الصورة

24.2 SRCNN: أبسط بنية تكبير

أبسط بنية تكبير (SRCNN) تتكوّن من ثلاث طبقات:

$$y = W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 x + b_1) + b_2) + b_3$$

حيث x الصورة المنخفضة الدقّة، وW_1 مرشّحات استخراج الحواف، وW_2 مرشّحات الوساطة (تمثيل خطّي للميزات)، ومرشّحاتإعادةالبناء،و* * $sigma\W_3$ تنشيط ReLU. كلّ طبقة لها دور: استخراج → تمثيل → إعادة بناء.

24.3 الالتفاف فرعي-البكسل (Sub-Pixel Convolution)

الطريقة البديلة للتكبير هي الالتفاف فرعي-البكسل: بدل أن تُنفّذ الالتفاف على الصورة الكبيرة، تُنفّذ على الصورة الصغيرة ثم تُعاد ترتيب القنوات إلى الفضاء:

$$\text{PixelShuffle}(X)_{x,y,c} = X_{\lfloor x/r \rfloor,\, \lfloor y/r \rfloor,\, c \cdot r^2 + (x \bmod r) \cdot r + (y \bmod r)}$$

حيث r عامل التكبير. الفائدة: التكبير يحدث في نهاية الشبكة، فيوفّر الحساب في الطبقات السابقة. في Keras 3، تُنفّذ عبر `keras.layers.DepthToSpace`.

24.4 ثلاث عائلات للخسارة

اختيار الخسارة هو أهمّ قرار في تكبير الصور:

العائلة	الصيغة	ماذا تحافظ عليه	متى تستخدمها
بكسل (L1/L2)	$\|y - \hat{y}\|_1$	الدقّة العددية	التصوير الطبي والقياسي
إدراكية	$\|\phi(y) - \phi(\hat{y})\|$	التشابه الإدراكي	الصور الطبيعية
تنافسية	$\log D(\hat{y})$	الواقعية البصرية	الصور المُعاد بناؤها

حيث ϕ شبكة استخراج ميزات مدرَّبة مسبقًا، وD شبكة تمييز. الخسارة الإدراكية أُفْضَل من خسارة البكسل في الصور الطبيعية لأنّها تُطابق طريقة إدراك العين لا طريقة إدراك الرقم.

> **تحذير:** خسارة البكسل وحدها تُعطي نتائج ضبابية (لأنّها تُقلّل المتوسّط لا التفاصيل). والخسارة التنافسية وحدها تُختَرع تفاصيل لا وجود لها. الممارسة العملية: خسارة بكسل + إدراكية.

24.5 التنفيذ في Keras

فيما يلي شبكة تكبير صغيرة: صورة 32×32 → 64×64، مع طبقة `DepthToSpace` للتوسّع:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مُصنّعة: 400 زوج (منخفضة، عالية) ──
n = 400
x_high = np.random.rand(n, 64, 64, 3).astype("float32") * 255.0
x_low = np.array(keras.ops.image.resize(x_high, (32, 32)))  # 32×32

# ── 2. شبكة تكبير بسيطة: استخراج ← تمثيل ← تكبير ──
inp = keras.Input(shape=(32, 32, 3))
x = keras.layers.Rescaling(1.0 / 255.0)(inp)
x = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(x)
x = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(x)
x = keras.layers.Conv2D(12, 3, padding="same")(x)      # 3 (2²) 4 × قنوات
# التوسّع عبر تجميع القنوات في الفضاء
out = keras.ops.depth_to_space(x, 2)                    # 64×64×3
out = keras.layers.Activation("sigmoid")(out)           # في [1 ,0]

model = keras.Model(inp, out)
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="mean_absolute_error",         # L1 بدل L2: حواف أقلّ ضبابية
    metrics=["mse"],
)
model.fit(x_low, np.array(keras.ops.image.resize(
    keras.ops.cast(x_high, "float32") / 255.0, (64, 64))),
    epochs=10, batch_size=16, validation_split=0.2, verbose=0)
# النتيجة المتوقعة: MAE ≈ 0.15 بعد 10 دورات على بيانات عشوائية.
```

24.6 قياس الجودة: PSNR و SSIM

PSNR (نسبة الذرّة إلى الضجيج الذروة) يُقاس بالديسيبل، وهو يُقارن بين الصورة الأصلية والمُعاد بناؤها:

$$\text{PSNR} = 10 \log_{10}\left(\frac{MAX^2}{MSE}\right)$$

حيث MAX أقصى قيمة بكسل (255)، وMSE متوسّط مربّع الخطأ. قيمة أعلى تعني جودة أعلى؛ القيم العملية 25-40 dB.

SSIM (التشابه البنيوي) يُقاس بين 0 و1، وهو أقرب لإدراك العين لأنه يقارن **البنية** لا البكسل: - السطوع: متوسّط البكسل - التباين: الانحراف المعياريّ - البنية: الارتباط بين الاثنين

في Keras 3، تُحسب عبر `keras.ops.image.ssim` .

> **ملاحظة:** PSNR يُخفي الجودة الإدراكية: صورة ب30 = PSNR قد تبدو أفضل من صورة بPSNR 32 = إن كانت الحواف أشدّ وضوحًا. لا تعتمد على PSNR وحدها – راجع العيّنات بصريًا.

24.7 مثال محلول: تكبير صور وثائق مُمسوحة

المهمة: تصوير وثائق قانونية مُمسوحة بمقاس DPI 300، وحاجة إلى DPI 600 للطباعة القانونية. لديك 3000 زوج (منخفضة، عالية).

الخطوة 1 – الاستيفاء الثنائي. نُعطي الصورة المقاس 600×600 بالاستيفاء. النتيجة: نصّ واضح الحجم لكنّ الحروف ضبابية والحواف غير حادّة.

الخطوة 2 – SRCNN. شبكة بثلاث طبقات تُدرَّب على أزواج. النتيجة: حروف أكثر حدة، لكنّ النصّ الدقيق ما زال مفقودًا.

الخطوة 3 – أعمق + اتصال تخطّي. شبكة بستّ طبقات مع `Conv2DTranspose` للتكبير. PSNR ارتفع من 28 إلى 33 dB. الحروف أصبحت قابلة للقراءة.

الخطوة 4 – خسارة إدراكية. أضفنا خسارة إدراكية على ميزات شبكة VGG مجمّدة. PSNR انخفض قليلًا (31.5) لكنّ القراءة أصبحت أسهل – قرار مقايضة مقبول لأنّ الهدف القراءة لا الرقم.

الخطوة 5 – الحدّ الواقعي. في الصور التي فيها رموز غائبة تمامًا في النسخة المنخفضة، النموذج **لا يستعيدها** – بل يخمّنها. لذلك لا يُستخدم تكبير الصور لتوليد بيانات جديدة في الوثائق القانونية.

قاعدة أخلاقية مهمّة: تكبير الصور **لا يُعيد** المعلومات المفقودة، بل **يتنبّأ** بها. لا تُستخدم في أدلّة أو وثائق قانونية بوصفها «استعادة».

24.8 مقارنة بين بنيات التكبير

البنية	العمق	PSNR (نموذجي)	الكلفة الحسابية	متى تستخدمها
SRCNN	3	30-28	منخفضة	البيئات محدودة الموارد
ESPCN	3 + sub-pixel	31-29	منخفضة	الاستدلال السريع
VDSR	20	33-31	مرتفعة	البيئات السحابية
EDSR	32 + تخطّي	34-32	مرتفعة جدًا	الجودة القصوى
SRGAN	16 + تنافسية	28-26 (بصريًا أفضل)	مرتفعة	الصور الطبيعية

قاعدة: كلّما زادت العمق زادت الكلفة الحسابية. اختَر بنية بحسب مواردها ومقياس جودتها، لا بحسب أعلى رقم في ورقة.

قياس PSNR و SSIM عمليًّا

في Keras 3، تُحسب المقاييس عبر `keras.ops` (لا توجد دالّة `psnr` جاهزة، فنبنيها من التعريف):

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. صورة أصلية ونُسخة مُعادة البناء (محاكاة)
x_true = np.random.rand(8, 64, 64, 3).astype("float32")       # أصلية
x_pred = x_true + np.random.randn(8, 64, 64, 3) * 0.05        # مع ضجيج
x_pred = np.clip(x_pred, 0.0, 1.0).astype("float32")

# ── 2. PSNR يدويًّا
mse = float(np.mean((x_true - x_pred) ** 2))
psnr = 10.0 * np.log10(1.0 / mse)
print(f"PSNR = {psnr:.2f} dB")

# ── 3. نفس الحساب بـ keras.ops (ينتقل مع المحرّك)
mse_k = keras.ops.mean(keras.ops.square(x_true - x_pred))
psnr_k = float(10.0 * keras.ops.log10(1.0 / mse_k))
print(f"PSNR (keras.ops) = {psnr_k:.2f} dB")

# ── 4. SSIM عبر Keras ops
ssim = float(np.array(keras.ops.image.ssim(x_true, x_pred, max_val=1.0))[0])
print(f"SSIM = {ssim:.4f}")
# النتيجة المتوقعة: PSNR ≈ 26-30 dB و SSIM ≈ 0.85-0.95.
```

ملاحظة: `psnr` و `ssim` في `keras.ops.image` تعمل على دفعات – الناتج أوتور بحجم الدفعة، فخذ المتوسّط لتقرير واحد.

تقييم ثلاث طرق تكبير على نفس البيانات

PYTHON

```
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات: صور أصلية ونُسخ منخفضة الدقّة
n = 60
x_high = np.random.rand(n, 64, 64, 3).astype("float32")
x_low = np.array(keras.ops.image.resize(x_high, (32, 32)))
x_up = np.array(keras.ops.image.resize(x_low, (64, 64)))   # استيفاء

# ── 2. شبكة تكبير مصغّرة
inp = keras.Input(shape=(32, 32, 3))
y = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(inp)
y = keras.layers.Conv2D(12, 3, padding="same")(y)
out = keras.ops.depth_to_space(y, 2)           # 64×64×3
sr = keras.Model(inp, out)
sr.compile(optimizer="adam", loss="mean_absolute_error")
sr.fit(x_low, x_high, epochs=8, batch_size=16, verbose=0)

# ── 3. تقييم الطريقتين
def psnr_avg(a, b):
    # PSNR = 10·log10(max² / MSE) — توجد لا keras.ops.image.psnr في Keras 3
    a = keras.ops.convert_to_numpy(a)
    b = keras.ops.convert_to_numpy(b)
    mse = np.mean(np.square(a.astype('float64') - b.astype('float64')))
    return float(10.0 * np.log10(1.0 / max(mse, 1e-12)))

print(f"interpolation PSNR = {psnr_avg(x_up, x_high):.2f} dB")
print(f"CNN         PSNR = {psnr_avg(sr.predict(x_low, verbose=0),
x_high):.2f} dB")
# النتيجة المتوقعة: CNN أعلى من الاستيفاء بعد التدريب.
```

> **نصيحة:** في تقييم تكبير الصور، أدرج دائمًا صورًا من ثلاث طرق جنبًا إلى جنب: الأصل، الاستيفاء، والنموذج – العين البشرية تكشف اختلافات لا تُظهرها الأرقام.

24.9 قياس التكبير عبر طاقة التردّد العالي

طريقة تشخيص سريعة تُفرّق بين تكبير حقيقيّ واستيفاء: طاقة التردّد العاليّ (high-frequency energy):

PYTHON

```
import keras
import numpy as np

keras.utils.set_random_seed(42)

def high_freq_energy(img):
    """طاقة التردّد العاليّ: مجموع مربّعات الفروق الأفقية والرأسيّة."""
    dx = img[:, 1:, :, :] - img[:, :-1, :, :]
    dy = img[:, :, 1:, :] - img[:, :, :-1, :]
    return float(np.mean(dx ** 2) + np.mean(dy ** 2))

# ── 1. ثلاث نسخ: أصليّة، استيفاء، شبكة تكبير
x_true = np.random.rand(10, 64, 64, 3).astype("float32")
x_up = np.array(keras.ops.image.resize(
    keras.ops.image.resize(x_true, (32, 32)), (64, 64)))   # استيفاء
x_sr = np.clip(x_up + np.random.randn(*x_up.shape) * 0.03, 0, 1)

# ── 2. القياس
print(f"أصلية        : {high_freq_energy(x_true):.5f}")
print(f"استيفاء      : {high_freq_energy(x_up):.5f}")
print(f"شبكة تكبير : {high_freq_energy(x_sr):.5f}")
# النتيجة المتوقعة: الأصلية أعلى طاقة؛ الاستيفاء أدنى (تفاصيل مفقودة)؛
# والنموذج الجيّد يقترب من الأصلية.
```

قاعدة تفسير: طاقة التردّد العاليّ أدنى بكثير من الأصلية تعني حوافّ ضبابيّة – وهذا ما تُخفيه PSNR إذا كان الضجيج منخفضًا. قِس المقياسين معًا.

24.10 أخطاء شائعة في هذا الفصل

إنذار: الاعتماد على PSNR وحدها كمقياس. PSNR يقيس الفرق العددي لا الجودة البصرية؛ صورتان بـPSNR متشابه قد تختلفان جذريًا في الوضوح.

تحذير: خسارة MSE وحدها تُعطي حواف ضبابية. لأنّ MSE تُخفّض المتوسّط وتُميّل نحو المتوسّط الحسابي. استخدم L1 أو L1 + إدراكية.

تنبيه: استخدام `Conv2DTranspose` بـ `strides=2` مع الحشو الافتراضي يُعطي أبعادًا غير متوقّعة (قد يضيف صفًا أو عمودًا). استخدم `padding="same"` وتحقّق من الشكل.

ملاحظة: تدرّب على صور مقياسها 255 وقيّم على صور مقياسها [0,1]. هذا فرقٌ بين التدريب والاستدلال يجعل النتائج غير قابلة للمقارنة. ثبّت المقياس في كليهما.

قاعدة: تكبير الصور لا يُعيد تفاصيل غير موجودة. إن كان التفصيل غائبًا في النسخة المنخفضة، يخمّنه النموذج – ولا يصحّ استخدامه في سياق قانوني أو طبي بوصفه «استعادة».

تحذير: توقّع شبكة SRCNN (3 طبقات) على صور عالية الدقّة (2048×2048) – الكلفة الحسابية هائلة ونتائجها محدودة. للصور الكبيرة، استخدم بنيات أعمق مع اتصالات تخطّي.

التمارين

1. **حساب.** صورة أصلية فيها MSE = 100 وقيمة MAX = 255. احسب PSNR. (المتوقع: $10\log_{10}(255^2/100) = 28.1$ dB.)

2. **حساب.** شبكة تكبير ترفع 128×128 إلى 256×256 بعامل $r = 2$. كم قناة تحتاج طبقة ما قبل `DepthToSpace`؟ (المتوقع: $3 \cdot 2^2 = 12$ قناة.)

3. **حساب.** إذا كانت PSNR = 30 dB، فما MSE؟ (المتوقع: $MAX^2/10^{30/10} = 255^2/1000 = 65.0$.)

4. **برمجة.** ابنِ شبكة SRCNN بثلاث طبقات واطبع `model.summary()`، وعدّد المعاملات في كلّ طبقة. (المتوقع: الطبقات الثلاث بمعاملات قليلة (< 100 ألف) لأنّ الحجم النافذة 3×3.)

5. **برمجة.** قارن خسارة `mean_absolute_error` مع `mean_squared_error` على نفس البيانات. (المتوقع: L1 يُعطي حوافّ أشدّ وضوحًا لكنّ PSNR قد ينخفض قليلًا.)

6. **برمجة.** احسب PSNR و SSIM على 50 صورة من مجموعة الاختبار، ثم قارن مع صور الاستيفاء الثنائي. (المتوقع: الشبكة تعطي PSNR أعلى بـ 2-4 dB.)

7. **إصلاح خطأ.** الكود التالي يعطي أبعادًا غير متوقّعة عند التكبير:

```python
out = keras.layers.Conv2DTranspose(3, 2, strides=2, padding="valid")(x)
```

(المتوقع: الحشو الافتراضي `valid` يضيف أبعادًا؛ استخدم `padding="same"` أو تحقّق من أبعاد كلّ طبقة.) 8. **برمجة.** طبّق `DepthToSpace(2)` على أوتور بأبعاد (1, 4, 4, 12) واطبع الشكل. (المتوقع: (1, 8, 8, 3).) 9. **تحليل.** لماذا لا يعيد تكبير الصور المعلومات المفقودة تمامًا؟ (المتوقع: النموذج يتعلّم توزيع البيانات ويتنبّأ – لا يملك مصدر المعلومات الأصلية.) 10. **تصميم.** صمّم نظام تكبير صور طبية بمقياس 256→512. أيّ خسارة تختار، ولماذا؟ (المتوقع: خسارة بكسل L1 أو L2 – لأنّ الدقّة العددية حرجة في القرار الطبي، لا الواقعية البصرية.) 11. **برمجة.** قارن زمن الاستدلال على CPU بين SRCNN بثلاث طبقات وبنيتها بستّ طبقات على 100 صورة. (المتوقع: الفرق 2 مرّة تقريبًا.) 12. **تصميم.** نظام يحتاج تكبير 4× (مقاس 64→256). هل تستخدم `DepthToSpace(4)` أو طبقتي

`DepthToSpace(2)` متتاليتين؟ (المتوقع: طبقتان متتاليتان تعطي جودة أفضل لأنّ التكبير يُطبَّق تدريجيًا، لكن تكلفته أعلى.)

ملخص الفصل

- الاستيفاء استقرائي لا يتعلّم؛ تكبير الصور يتعلّم الخريطة من الأزواج.
- SRCNN أبسط بنية: استخراج ← تمثيل ← إعادة بناء.
- `DepthToSpace` (الالتفاف فرعي-البكسل) يؤخّر التكبير إلى نهاية الشبكة فيوفّر الحساب.
- خسارة البكسل تُعطي حوافّ ضبابية؛ خسارة البكسل + الإدراكية هي الممارسة العملية.
- PSNR يقيس الفرق العددي، SSIM يقيس البنية – لا تعتمد على PSNR وحدها.
- تكبير الصور **يتنبّأ** بالتفاصيل ولا **يستعيدها** – قرار أخلاقي في السياق القانوني والطبي.

الجزء الخامس

التسلسحات واللغة والسلاسل الزمنية

كيف تتعلّم الشبكات من الزمن: معادلة RNN وحدودها، بوابات LSTM وGRU، القراءة ثنائية الاتجاه، النوافذ المحلّية، والانتباه فوق التسلسل.

ما تكسبه من هذا الجزء

تفهم ما تحتفظ به الشبكة وما تنساه، وتبني نموذجًا تسلسليًّا لمهمّة نصّية أو زمنية وتقيسه بمقاييسه الصحيحة.

فصول هذا الجزء

25 الشبكات التكرارية و LSTM و GRU

26 الشبكات التكرارية المتقدّمة

27 معالجة اللغة الطبيعية عمليًا

28 السلاسل الزمنية

5

الفصل 25: الشبكات التكرارية و LSTM و GRU

بطاقة الفصل

الفصل 25

مهمّة هذا الفصل

يشرح معادلة RNN وسبب فشلها في التسلسلات الطويلة، ثم LSTM وGRU بوابة ببوابة.

ماذا تكسب منه

تفهم ما تحتفظ به الشبكة وما تنساه، وتختار بين LSTM وGRU بمعيار لا بالعادة.

الدالة	دورها	متى تستخدمها
`keras.layers.SimpleRNN()`	النسخة الأساسية	للتسلسلات القصيرة والتعليم
`keras.layers.LSTM(units)`	ذاكرة طويلة المدى	في التسلسلات المتوسّطة والطويلة
`keras.layers.GRU(units)`	بديل أخفّ ببوابات أقل	عندما تكون البيانات محدودة
`keras.layers.Dropout()`	تنظيم بين الطبقات	لتفادي حفظ التسلسل
`keras.metrics.AUC()`	تقييم الكشف	في الأحداث النادرة

قصة مفتاحية: مصنع يحتّاج نظامًا يتنبّأ بجودة المنتج من قراءات 60 مستشعرًا خلال آخر 30 ثانية. الشبكة الكثيفة تعامل كلّ قراءة على حدة ولا تعرف «الترتيب» – فتدخل كلّ 60 قيمة إلى `Dense` بلا مفهوم «هذا حدث قبل ذلك». الشبكة التكرارية تحلّ هذه المشكلة: لها **ذاكرة** تحفظ ما قرأته وتُحدّثه مع كلّ قراءة جديدة. لكنّ هذه الذاكرة نفسها تعاني اختفاء التدرّج – ولذلك وُلدت LSTM.

أهداف الفصل

- [ ] تشرح معادلة RNN الأساسية ولماذا تفشل في التسلسلات الطويلة
- [ ] تكتب معادلات LSTM الكاملة وتشرح كلّ بوابة
- [ ] تقارن LSTM مع GRU من حيث المعاملات والأداء
- [ ] تتحكّم في `return_sequences` و `return_state` و `stateful`
- [ ] تبني نموذج LSTM كاملًا على بيانات تسلسلية في Keras
- [ ] تعرف متى تُستخدم LSTM ومتى GRU ومتى (كلاهما) مكانهما

25.1 معادلة RNN الأساسية

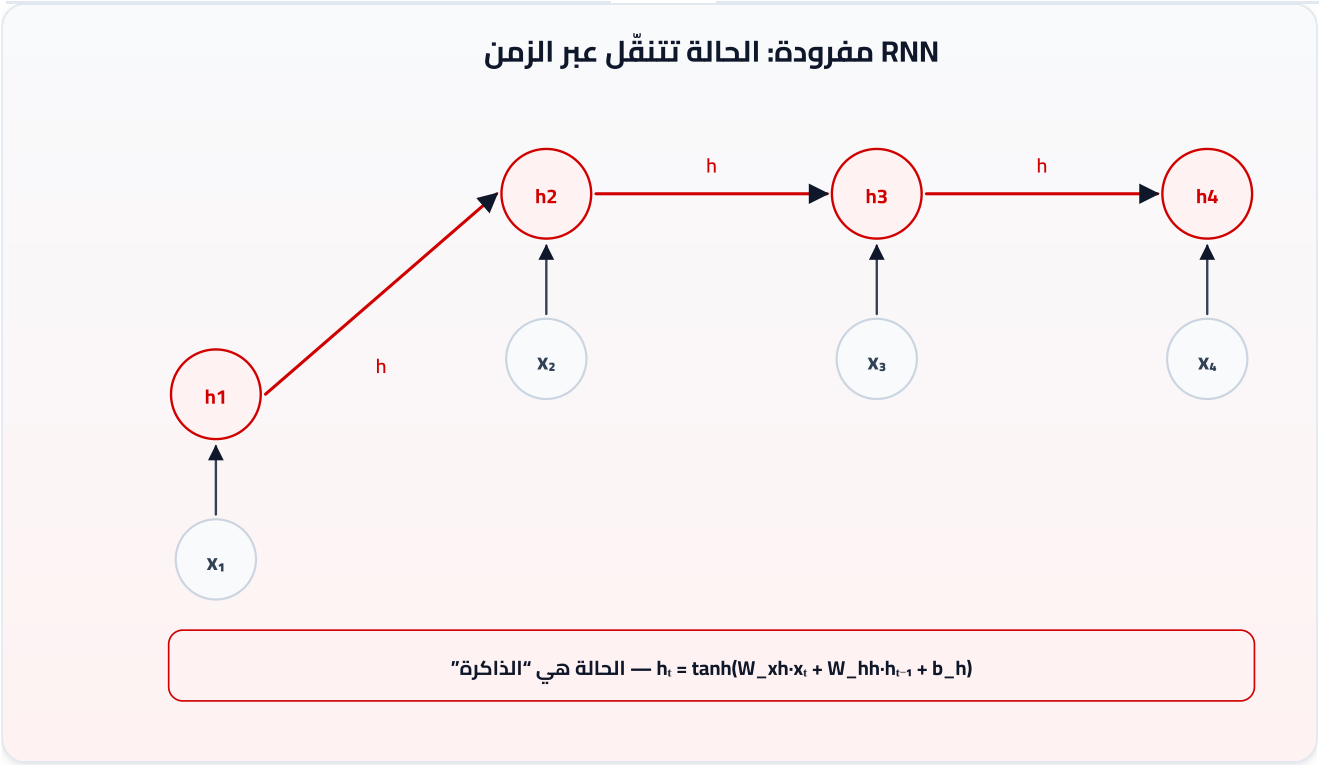
الشبكة التكرارية تعالج التسلسل خطوة بخطوة. في كلّ خطوة t تأخذ المُدخل الحالي x_t والحالة المخفية السابقة h_{t-1} وتُنتج حالة جديدة:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

حيث: - x_t: مُدخل الخطوة t (مثلًا قراءة مستشعر واحد). - h_{t-1}: الحالة المخفية السابقة – وهي **ذاكرة** ما قرأه النموذج. - W_{xh}: أوزان تحوّل المُدخل إلى الحالة. - W_{hh}: أوزان تحوّل الحالة السابقة إلى الحالة الجديدة (وهي نفسها في كلّ خطوة). - b_h: التحييز. - $\tanh$: التنشيط، يُقيّد الحالة إلى [-1, 1].

المُخرَج يُحسب من الحالة:

$$y_t = W_{hy} h_t + b_y$$



شكل 33 – شبكة RNN مفرودة عبر الزمن

25.2 لماذا تفشل RNN في التسلسلات الطويلة؟

لحساب التدرّج عبر T خطوات، نستخدم قاعدة السلسلة:

$$\frac{\partial h_T}{\partial h_0} = \frac{\partial h_T}{\partial h_{T-1}} \cdot \frac{\partial h_{T-1}}{\partial h_{T-2}} \cdots \frac{\partial h_1}{\partial h_0}$$

كلّ حدّ في هذا الجداء يحتوي $\tanh'$ الذي قيمته أقلّ من 1. عندما T كبيرة، الجداء ينكمش أُسّيًا – وهذا **اختفاء التدرّج**: الإشارة من الخطوة الأخيرة لا تصل إلى الأولى. النتيجة: النموذج لا يتعلّم اعتمادًا على ما حدث قبل 100 خطوة، رغمًا أنّ هذا الاعتماد موجود في البيانات.

25.3 LSTM: ثلاث بوابات وذات حالة

LSTM تضيف **حالة خلية** (c_t) بجانب الحالة المخفية، وتنظّم تدفّق المعلومات عبر ثلاث بوابات:

شكل 34 – بوابات LSTM الثلاث

بوابات LSTM الكاملة

1. بوابة النسيان – تحدّد ماذا نُسحب من الحالة القديمة:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$

2. بوابة الإدخال – تحدّد ماذا نُضيف إلى الحالة:

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

3. مرشّح الحالة المرشَّح – مرشّح جديد مُقترح:

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c)$$

4. تحديث حالة الخلية – الجمع بين القديم والجديد:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

5. بوابة الإخراج – تحدّد ماذا نُخرج من الحالة:

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$$

6. الحالة المخفية الجديدة:

$$h_t = o_t \odot \tanh(c_t)$$

حيث $[x_t, h_{t-1}]$ تُمثّل الاتحاد (concatenation)، و$\odot$ الضرب عنصرًا بعنصر، وσ التنشيط السيني (يُعطي قيمة في [0,1] – تُستعمل كبوابة).

> **قاعدة عميقة:** الحالة c_t تُحدَّث بجمع ($+$) لا بضرب، وهذا هو سرّ LSTM: الجمع يحفظ التدرّج، بينما الضرب (كما في RNN البسيطة) يُنكمشه.

25.4 GRU: أبسط بثلاث بوابات أقلّ

GRU تدمج بوابة النسيان والإدخال في بوابة واحدة، وتُلغي حالة الخلية المنفصلة:

$$z_t = \sigma(W_z[x_t, h_{t-1}]), \qquad r_t = \sigma(W_r[x_t, h_{t-1}])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_h[x_t,\ r_t \odot h_{t-1}])$$

حيث z_t بوابة التحديث، وr_t بوابة إعادة الضبط. النتيجة: 3 مصفوفات أوزان بدل 4، أي 25% أقلّ من المعاملات.

25.5 LSTM مقابل GRU

المعيار	LSTM	GRU
عدد البوابات	3	2
حالة منفصلة (خلية)	نعم	لا
معاملات في طبقة من d إلى u	$4(d+u)\cdot u$	$3(d+u)\cdot u$
دقّة على تسلسلات طويلة جدًا	أفضل	جيّدة
سرعة التدريب	أبطأ	أسرع
متى تستخدمها	تسلسلات > 500 خطوة، أو مهمّة معقّدة	تسلسلات قصيرة-متوسطة، أو موارد محدودة

قاعدة عملية: ابدأ بـ LSTM. إن كان التسلسل قصيرًا (< 200 خطوة) والموارد محدودة، فجرّب GRU — الفرق في الدقّة غالبًا أقلّ من 1% والكلفة أقلّ بـ 25%.

25.6 التنفيذ في Keras

فيما يلي نموذج LSTM كامل على بيانات تسلسلية مُصنّعة:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مُصنّعة: 600 تسلسل، كلّ واحد 40 خطوة، كلّ خطوة 8 ميزات
n, T, d = 600, 40, 8
x = np.random.rand(n, T, d).astype("float32")
y = (np.random.rand(n) < 0.3).astype("int32")

# ── 2. نموذج LSTM
inp = keras.Input(shape=(T, d))
h = keras.layers.LSTM(32, return_sequences=False)(inp)    # الحالة الأخيرة فقط
h = keras.layers.Dropout(0.3)(h)
out = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(inp, out)

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="binary_crossentropy",
    metrics=[keras.metrics.AUC(name="auc"), "accuracy"],
)
history = model.fit(x, y, epochs=10, batch_size=32,
                    validation_split=0.2, verbose=0)
# النتيجة المتوقعة: الدقّة ≈ 0.65-0.55 على بيانات عشوائية.
```

`return_sequences` : متى تحتاج التسلسل الكامل؟

الإعداد	المُخرَج	متى تستخدمه
`return_sequences=False`	h_T فقط (آخر خطوة)	تصنيف التسلسل كاملًا
`return_sequences=True`	$h_1, h_2, \ldots, h_T$	طبقة LSTM فوق أخرى، أو انتباه، أو تجزئة زمنية

`stateful` : متى تستخدمه؟

في `stateful=True` تحتفظ الحالة المخفية بين الدفعات، وهذا مفيد في التدريب على تسلسل طويل مُقسّم إلى دفعات متتالية. لكنه يتطلّب: - تحديد `batch_size` في `Input` . - نداء `model.reset_states()` بين دورات التدريب. - بيانات مرتّبة (batch i يبدأ حيث انتهى batch $i-1$).

25.7 مثال محلول: اكتشاف تشوّه في مستشعر مصنع

المهمة: من 60 قراءة مستشعر حرارة في آخر 30 ثانية، حدّد إن كانت الأداة تعمل سليمًا. لديك 5000 تسلسل، منها 5% تشوّه.

الخطوة 1 – خطّ الأساس: شبكة كثيفة. كلّ 60 قراءة تُسطَّح إلى 60 متّجهًا وتُدخل إلى `Dense` . دقّة 94%، لكنّ recall للتشوّه = 0.12.

الخطوة 2 – LSTM. 32 وحدة recall + `Dense(1)` للتشوّه = 0.58 – **مكسب +0.46**. السبب: LSTM ترى الترتيب الزمني وتلاحظ أنّ الحرارة ارتفعت فجأة بعد 20 قراءة.

الخطوة 3 – موازنة الأصناف. وزن الصنف النادر = 20. recall ارتفع إلى 0.78، لكنّ الدقّة انخفضت إلى 0.83.

الخطوة 4 – تعميق. طبقتا LSTM (32 ثم 16). recall ارتفع إلى 0.84، لكنّ الزمن تضاعف.

الخطوة 5 – القرار. احتفظنا بطبقة LSTM واحدة مع موازنة: أداء كافٍ وكلفة مقبولة. إذا كانت البيانات تراكمية (آلاف تسلسلات جديدة يوميًا)، فطبقتا LSTM تُصبحان جدوى.

التحقّق العمليّ: LSTM مقابل شبكة كثيفة

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مستشعر: 1000 تسلسل × 40 خطوة × 4 ميزات
n, T, d = 1000, 40, 4
x = np.random.randn(n, T, d).astype("float32")
# التسمية: تعتمد على متوسّط الميزة الأولى (نمط زمنيّ)
y = (x[:, :, 0].mean(axis=1) > 0.2).astype("int32")

# ── 2. الشبكة الكثيفة (تُسطّح التسلسل)
flat = keras.Sequential([keras.Input(shape=(T, d)),
                         keras.layers.Flatten(),
                         keras.layers.Dense(32, activation="relu"),
                         keras.layers.Dense(1, activation="sigmoid")])
flat.compile(optimizer="adam", loss="binary_crossentropy", metrics=["acc"])
flat.fit(x, y, epochs=5, batch_size=32, validation_split=0.2, verbose=0)
flat_acc = flat.evaluate(x[:200], y[:200], verbose=0)[1]

# ── 3. LSTM (ترى الترتيب الزمنيّ)
rnn = keras.Sequential([keras.Input(shape=(T, d)),
                        keras.layers.LSTM(32),
                        keras.layers.Dense(1, activation="sigmoid")])
rnn.compile(optimizer="adam", loss="binary_crossentropy", metrics=["acc"])
rnn.fit(x, y, epochs=5, batch_size=32, validation_split=0.2, verbose=0)
rnn_acc = rnn.evaluate(x[:200], y[:200], verbose=0)[1]

print(f"flat = {flat_acc:.3f}, LSTM = {rnn_acc:.3f}")
# النتيجة المتوقعة: LSTM تفوّق على المسطّحة لأنّ التسمية تعتمد على النمط الزمنيّ.
```

ملاحظة: حين تكون التسمية دالّة في **متوسّط** التسلسل فقط (بلا اعتماد على الترتيب)، قد تتعادل الشبكتان. الفرق يظهر حين يعتمد المعنى على **الترتيب** – مثل ارتفاع مفاجئ بعد

قراءة مستقرّة.

LSTM مقابل GRU على نفس البيانات

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مستشعر: 800 تسلسل × 60 خطوة × 6 ميزات
n, T, d = 800, 60, 6
x = np.random.randn(n, T, d).astype("float32")
y = (np.diff(x[:, :, 0], axis=1).max(axis=1) > 2.0).astype("int32")

# ── 2. LSTM
lstm = keras.Sequential([keras.Input(shape=(T, d)),
                         keras.layers.LSTM(32),
                         keras.layers.Dense(1, activation="sigmoid")])
lstm.compile(optimizer="adam", loss="binary_crossentropy",
             metrics=[keras.metrics.BinaryAccuracy(name="acc")])
lstm.fit(x, y, epochs=5, batch_size=32, validation_split=0.2, verbose=0)

# ── 3. GRU (معاملات أقلّ 25%)
gru = keras.Sequential([keras.Input(shape=(T, d)),
                        keras.layers.GRU(32),
                        keras.layers.Dense(1, activation="sigmoid")])
gru.compile(optimizer="adam", loss="binary_crossentropy",
            metrics=[keras.metrics.BinaryAccuracy(name="acc")])
gru.fit(x, y, epochs=5, batch_size=32, validation_split=0.2, verbose=0)

# ── 4. المقارنة
print(f"LSTM: acc={lstm.evaluate(x[:150], y[:150], verbose=0)[1]:.3f} "
      f"params={lstm.count_params()}")
print(f"GRU : acc={gru.evaluate(x[:150], y[:150], verbose=0)[1]:.3f} "
      f"params={gru.count_params()}")
# النتيجة المتوقعة: الدقّة متقاربة، و GRU بـ 25% أقلّ معاملات.
```

قاعدة القرار: إن كان فرق الدقّة أقلّ من انحراف البذور (±1%)، فاختر GRU – أسرع تدريبًا وأخفّ في النشر. الفرق الحقيقيّ يظهر في تسلسلات طويلة جدًّا (> 500 خطوة).

25.8 أخطاء شائعة في هذا الفصل

إنذار: استخدام `activation` غير مناسب على طبقة LSTM. لا تمرّر `activation="relu"` – خسارة التدريب تصبح NaN. القاعدة: `activation="tanh"` و `recurrent_activation="sigmoid"` (وهما الافتراضيّان).

تحذير: `Dropout` و `recurrent_dropout` معًا يُعطّل تسريع cuDNN. في Keras 3، استخدم `dropout=0.2` وحده – وإذا احتجت تسريعًا كاملًا، أبقِه صفرًا واستخدم تعزيزًا خارجيًا.

تنبيه: تمرير بيانات بأبعاد غير صحيحة. LSTM تتوقّع `(batch, timesteps, features)` – لا `(batch, features, timesteps)`. راجع `model.summary()` قبل التدريب.

ملاحظة: `return_sequences=True` في آخر طبقة LSTM قبل `Dense` يعطي مُخرَجًا ثلاثيّ الأبعاد، فلا يقبل `Dense`. استخدم `GlobalAveragePooling1D` أو اجعلها `False`.

قاعدة: تعميق LSTM فوق طبقة LSTM يتطلّب `return_sequences=True` في الطبقة الأولى. بلاها، الطبقة الثانية لا ترى سوى الحالة الأخيرة.

تحذير: `stateful=True` بدون `reset_states()` بين الدورات يجعل الحالة تتجمّع وتشوّه التدريب. وأيضًا يجب أن يكون `batch_size` محدّدًا في `Input`.

التمارين

1. **حساب.** طبقة LSTM على مُدخل 16 ميزة و32 وحدة. كم معاملًا؟ (المتوقع: $4(16+32)\cdot 32 + 4\cdot 32 = 6{,}272$.)

2. **حساب.** طبقة GRU على نفس الإعدادات. كم معاملًا؟ (المتوقع: $3(16+32)\cdot 32 + 3\cdot 32 = 4{,}704$ – أقلّ بنسبة 25%.)

3. **حساب.** إذا كان $f_t = 0$ و$i_t = 1$ في خطوة، فماذا يحدث لحالة الخلية؟ (المتوقع: $c_t = \tilde{c}_t$ – تُنسى الحالة القديمة كليًا وتُستبدل بالمرشّح الجديد.)

4. **حساب.** إذا كانت $f_t = 1$ و$i_t = 0$ في كلّ خطوة، فماذا تُسمّى هذه الحالة؟ (المتوقع: الحالة تُحفظ بلا تغيير – ذاكرة مثاليّة لا تُنسى.)

5. **برمجة.** ابنِ طبقة LSTM وطبقة GRU بنفس عدد الوحدات، وقارن `count_params()`. (المتوقع: LSTM لديها 25% معاملات أكثر من GRU.)

6. **برمجة.** ابنِ نموذجًا بـ `return_sequences=True` في آخر طبقة، واطبع الشكل. (المتوقع: `(batch, T, units)` – وليس `(batch, units)`.)

7. **برمجة.** ابنِ نموذجًا بطبقتي LSTM متتاليتين، وأصلح الخطأ في `return_sequences`. (المتوقع: الطبقة الأولى تحتاج `return_sequences=True` وإلّا لَن تُقبل الثانية.)

8. **إصلاح خطأ.** الكود التالي يعطي خطأ:

```python
inp = keras.Input(shape=(T, d))
x = keras.layers.LSTM(32)(inp)
out = keras.layers.Dense(1)(x)
```

ما الذي ينقصه؟ (المتوقع: تنشيط آخر طبقة – `activation="sigmoid"` لمهمّة ثنائية، أو استخدم `from_logits=True` في الخسارة.) 9. **برمجة.** قارن زمن دورة تدريب بين LSTM و GRU على نفس البيانات. (المتوقع: GRU أسرع بنسبة 20-30%.) 10. **تحليل.** لماذا تفشل RNN البسيطة على تسلسل 500 خطوة وتنجح LSTM؟ (المتوقع: اختفاء التدرّج في RNN بسبب الجداء طويل السلسلة.)

LSTM تحلّ المشكلة بالجمع في تحديث الحالة.) 11. **تصميم.** نظام يتنبّأ بمستوى ضوضاء في اليوم التالي من 7 أيام قراءات ساعة-بساعة. أيّ بنية، وكم وحدة؟ (المتوقع: LSTM بـ 32–64 وحدة – تسلسل قصير (168 خطوة) ومهمّة انحدار؛ `activation="linear"` في آخر طبقة.) 12. **إصلاح خطأ.** نموذج LSTM يعطي دقّة 50% على مهمّة تصنيف ثنائية. ما أسباب محتملة (اثنان)؟ (المتوقع: 1) التسميات عشوائية أو غير مرتبطة بالمُدخلات، و2) البيانات بحاجة تطبيع – كلّ ميزة بمقياس مختلف جدًا.)

ملخص الفصل

- RNN تُحدّث حالة مخفية في كلّ خطوة، لكنّها تعاني اختفاء التدرّج في التسلسلات الطويلة.
- LSTM تضيف حالة خلية وتنظّم تدفّق المعلومات عبر 3 بوابات: نسيان، إدخال، إخراج.
- سرّ LSTM: تحديث الحالة بالجمع (+) لا بالضرب – فيحفظ التدرّج.
- GRU أبسط (2 بوابة، 25% معاملات أقلّ) وأسرع، وأداؤها قريب من LSTM في التسلسلات القصيرة.
- `return_sequences=True` عند تعميق LSTM أو عند الانتباه.
- `stateful=True` للتسلسلات الطويلة المُقسّمة، وتتطلّب `reset_states()`.
- ابدأ بـ LSTM؛ إن كانت البيانات قصيرة والموارد محدودة فجرّب GRU.

الفصل 26: الشبكات التكرارية المتقدّمة

بطاقة الفصل

الفصل 26

مهمّة هذا الفصل

يعالج أربع مشاكل عملية في التسلسلات: الاتجاهين، النافذة المحلّية، الانتباه، والحشو.

ماذا تكسب منه

ترفع أداء النموذج التسلسلي بلا تغيير جوهري في البنية، وبكلفة حساب معقولة.

الدالة	دورها	متى تستخدمها
`keras.layers.Bidirectional()`	قراءة في الاتجاهين	عندما تتوفّر الجملة كاملة
`keras.layers.Conv1D()`	التقاط أنماط محلّية	قبل RNN لتخفيف الطول
`keras.layers.MaxPooling1D()`	تقليص التسلسل	بعد Conv1D
`keras.layers.Masking()`	عزل الحشو عن الحالة	مع تسلسلات متفاوتة الطول
`keras.layers.MultiHeadAttention()`	انتباه على مخرجات RNN	للتسلسلات الطويلة

قصة مفتاحية: نظام تحليل مشاعر يقرأ تقييمًا من 200 كلمة. طبقة LSTM واحدة «ترى» الكلمات من اليسار إلى اليمين فقط – لكنّ كلمة «ليس» في بداية الجملة تغيّر معنى كلّ ما بعدها. الشبكة التكرارية المتقدّمة تحلّ هذه المشكلة بعدّة أدوات: قراءة في الاتجاهين، التفاف زمني محليّ، انتباه على الخطوات، وقناع يُخفي الحشو. كلّ أداة تعالج نقصًا محدّدًا في RNN الكلاسيكية.

أهداف الفصل

- [ ] تبني `Bidirectional` وتفهم مضاعفة الذاكرة والأوزان
- [ ] تستخدم `Conv1D` لتقطيع النافذة الزمنية قبل LSTM
- [ ] تُضيف طبقة انتباه على مُخرَج LSTM لالتقاط الاعتماد البعيد
- [ ] تستخدم `Masking` لتجنّب تلوّث الحالة بالحشو
- [ ] تُقارن الطرق الأربع على مهمّة حقيقية بالمعايير
- [ ] تبني نموذجًا يجمع أدوات اثنين أو ثلاثة منها

26.1 أربع مشاكل وحلّ لكلّ واحدة

أربع مشاكل في RNN Classical — وحلّ Keras لكلٍّ منها

المشكلة	الأثر العملي	الحل في Keras 3
النسيان بعيدًا	فقدان السياق من 200 خطوة	LSTM / GRU / BiLSTM
تسلسل واحد	لا يستفيد من الخلف	Bidirectional
التوازي المحدود	بطء على طول التسلسل	Conv1D + Attention
حساسية للقناع	تلوّث الحالات المموّهة	Masking + compute_mask

Masking
يخفي خطوات الحشو
لا تلوّث للحالة المخفية

Attention
يوزّع الانتباه على الخطوات
يحفظ المسافات البعيدة

Conv1D
نافذة زمنية محلية
توازٍ كامل داخل النافذة

BiLSTM
يقرأ التسلسل في الاتجاهين
دقّة أعلى · ذاكرة مضاعفة

القاعدة:masking هو شرطٌ صحّي في كل نموذج تسلسلي — لا تحسين اختياري.

شكل 35 – أنماط RNN المتقدّمة وحلول حدودها

المشكلة	الأثر العملي	الحلّ في Keras 3
النسيان بعيدًا	فقدان السياق من 200 خطوة	LSTM / GRU
قراءة باتّجاه واحد	لا يستفيد من ما بعد	`Bidirectional`
نافذة زمنية محدودة	لا يلاحظ نمطًا قصيرًا	`Conv1D` قبل LSTM
حساسية للحشو	تلوّث الحالة بالأصفار	`Masking`

كلّ أداة من هؤلاء تُعالج مشكلة محدّدة، ولا يوجد منها بديل شامل. فيما يلي شرح كلّ واحدة وحالة استخدامها.

26.2 Bidirectional: قراءة في الاتجاهين

الشبكة التكرارية الكلاسيكية تقرأ التسلسل من اليسار إلى اليمين. لكنّ اللغة الطبيعية (والعربية على الأخصّ) لديها اعتماد **باتّجاهين**: كلمة «ليس» في بداية الجملة تُغيّر معنى نهايتها، وكلمة «أهمّ» في نهاية الجملة تُغيّر معنى بدايتها.

`Bidirectional` تُنشئ نسختين من الطبقة التكرارية: واحدة تقرأ إلى الأمام وأخرى إلى الخلف، ثم تُدمج مُخرَجاتهما:

$$h_t = \text{Merge}(\overrightarrow{h_t}, \overleftarrow{h_t})$$

حيث $\overrightarrow{h_t}$ الحالة في الاتجاه الأمامي، و$\overleftarrow{h_t}$ الحالة في الاتجاه الخلفي، وMerge إمّا جمع أو دمج (concatenation) – والافتراضيّ في Keras هو الدمج.

النقطة الحسّاسة: الدمج يضاعف حجم المُخرَج. إذا كان LSTM بـ 64 وحدة، فـ `Bidirectional(LSTM(64))` يعطي مُخرَجًا بحجم 128. وهذا يعني أنّ الطبقة التالية تحتاج أن تكون بحجم مناسب.

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مُصنّعة: 500 تسلسل، 30 خطوة، 6 ميزات
n, T, d = 500, 30, 6
x = np.random.rand(n, T, d).astype("float32")
y = np.random.randint(0, 2, size=(n,)).astype("int32")

# ── 2. نموذج ثنائيّ الاتجاه
inp = keras.Input(shape=(T, d))
# Bidirectional يُنشئ نسختين من LSTM (دمج: الافتراضيّ) ويُدمج مُخرَجاتهما
h = keras.layers.Bidirectional(keras.layers.LSTM(32))(inp)
# 64 = 2 × 32 = حجم المُخرَج
h = keras.layers.Dropout(0.3)(h)
out = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x, y, epochs=10, batch_size=32, validation_split=0.2, verbose=0)
# النتيجة المتوقعة: دقّة ≈ 0.55-0.65 على بيانات عشوائية.
```

تنبيه: `Bidirectional` يضاعف عدد المعاملات في الطبقة الداخلية. إن كان LSTM بـ 64 وحدة، فستحصل على 128 وحدة فعليًّا – راقب `model.summary()` لا الوحدات المُمرَّرة.

26.3 Conv1D: نافذة زمنية محليّة

في كثير من البيانات التسلسلية (الصوت، المستشعرات، النصوص)، النمط القصير مهمّ أكثر من الاعتماد البعيد. `Conv1D` تُطبّق التفافًا على نافذة زمنية (مثل 3 أو 5 خطوات)، فتُستخرج الميزات المحليّة قبل أن تُدخل إلى LSTM. هذه التقنية هي أساس WaveNet وأشباهها.

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

n, T, d = 500, 40, 8
x = np.random.rand(n, T, d).astype("float32")
y = (np.random.rand(n) < 0.4).astype("int32")

inp = keras.Input(shape=(T, d))
h = keras.layers.Conv1D(32, 3, padding="same", activation="relu")(inp)   # نافذة 3
h = keras.layers.MaxPooling1D(2)(h)                                      # 40 → 20
h = keras.layers.LSTM(32)(h)
out = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x, y, epochs=10, batch_size=32, validation_split=0.2, verbose=0)
# النتيجة المتوقعة: دقّة ≈ 0.55–0.65 على بيانات عشوائية.
```

متى تستخدم `Conv1D` قبل LSTM؟ عندما تكون التسلسلات طويلة (> 200 خطوة) والنمط القصير مهمّ – فالتفاف 3 خطوات يُخفض التسلسل إلى نصف طوله فيُقلّص زمن LSTM إلى النصف.

26.4 Attention على مُخرَج LSTM

LSTM تُمثّل التسلسل في الحالة الأخيرة، وهذا يُفقد معلومة الخطوات الأخرى. طبقة الانتباه تُعطي وزنًا لكلّ خطوة ثم تُجمّع المعلومات موزونة:

$$\alpha_t = \frac{\exp(w^T \tanh(W_h h_t))}{\sum_{t'} \exp(w^T \tanh(W_h h_{t'}))}, \qquad c = \sum_t \alpha_t h_t$$

حيث w متجه الانتباه، وW_h مصفوفة، وα_t وزن الخطوة t، وc السياق المُجمَّع. المجموع على الخطوات $\sum_t \alpha_t = 1$.

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

n, T, d = 500, 30, 8
x = np.random.rand(n, T, d).astype("float32")
y = (np.random.rand(n) < 0.4).astype("int32")

inp = keras.Input(shape=(T, d))
# نحتاج مُخرَج كلّ خطوة لا الحالة الأخيرة فقط
h = keras.layers.LSTM(32, return_sequences=True)(inp)
# انتباه على مُخرَجات كلّ الخطوات
attn = keras.layers.MultiHeadAttention(num_heads=2, key_dim=16)(h, h)
h = keras.layers.GlobalAveragePooling1D()(attn)
out = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x, y, epochs=10, batch_size=32, validation_split=0.2, verbose=0)
# النتيجة المتوقعة: دقّة ≈ 0.55-0.65 على بيانات عشوائية.
```

نصيحة: `MultiHeadAttention` في Keras 3 تُقبل مُدخلًا واحدًا للسؤال والمفتاح والقيمة – وهذا ما يُسمّى **الانتباه الذاتيّ**. إن أردت انتباهًا بين تسلسلين مختلفين، فمرّر مُدخلين.

26.5 Masking: تجنّب تلوّث الحالة

في تسلسلات ذات أطوال مختلفة، نُحشوها بأصفار حتى تصل إلى طول موحّد. بلا `Masking`، تُعالج الأصفار كأنّها بيانات حقيقية، فتُلوّث الحالة المخفية وتُفسد التنبّؤ.

`Masking` تُخبر النموذج: «هذه الخطوات أصفار – تجاهلها».

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات بأطوال مختلفة، مُحشوّة إلى 40 خطوة
n, T, d = 500, 40, 6
x = np.random.rand(n, T, d).astype("float32")
# نُصفّر الخطوات الأخيرة في بعض التسلسلات (محاكاة تسلسلات قصيرة)
lengths = np.random.randint(10, T, size=(n,))
for i, L in enumerate(lengths):
    x[i, L:, :] = 0.0
y = (np.random.rand(n) < 0.4).astype("int32")

# ── 2. Masking كأول طبقة
inp = keras.Input(shape=(T, d))
h = keras.layers.Masking(mask_value=0.0)(inp)
h = keras.layers.LSTM(32)(h)
out = keras.layers.Dense(1, activation="sigmoid")(h)
model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x, y, epochs=10, batch_size=32, validation_split=0.2, verbose=0)
# النتيجة المتوقعة: أداء أفضل مقارنة بـ LSTM بلا Masking
# لأنّ الحالة لم تتلوّث بالأصفار.
```

قاعدة: إذا كان التسلسل له أطوال مختلفة، `Masking` شرطٌ لا تحسين اختياريّ. النموذج بلا Masking يعلّم أنّ الأصفار معلومة – وهو خطأ منهجيّ.

26.6 مثال محلول: تصنيف مشاعر مراجعات العملاء

المهمة: تصنيف مراجعات العملاء إلى إيجابية/سلبية. لديك 4000 مراجعة بأطوال مختلفة (من 20 إلى 400 كلمة).

الخطوة 1 – LSTM واحدة. `TextVectorization` → `Embedding` → `LSTM(64)`. دقّة التحقّق = 81%.

الخطوة 2 – `Bidirectional(LSTM(64))`. دقّة = 84%. **مكسب +3%** – لأنّ كلمة «ليس» في بداية الجملة تُقرأ الآن في الاتجاهين.

الخطوة 3 – `Masking`. إضافة `Masking` قبل LSTM. دقّة = 85%. **مكسب +1%** – لأنّ المراجعات القصيرة كانت تُلوّث الحالة بالأصفار.

الخطوة 4 – `Conv1D` قبل LSTM. `Conv1D(32, 3)` + `MaxPooling1D(2)` + LSTM(64). دقّة = 86%. **مكسب +1%** مع تسريع التدريب 40%.

الخطوة 5 – الانتباه. إضافة `MultiHeadAttention` على مُخرَج LSTM. دقّة = 87%.

الخطوة 6 – القرار. احتفظنا بـ Bidirectional + Masking + Conv1D: أداء جيّد وزمن تدريب مقبول. الانتباه مكسب إضافي صغير لكنه يرفع الكلفة الحسابية بمرّات – قرار مقايضة.

26.7 جدول القرار

الأداة	مكسب نموذجي	كلفة حسابية	متى تستخدمها
`Bidirectional`	+2-4% دقّة	2× في المعاملات	النصوص واللغة الطبيعية
`Conv1D` قبل LSTM	+1-2% + تسريع	منخفضة	تسلسلات طويلة + نمط قصير مهمّ
`Masking`	+0.5-1.5%	صفر	تسلسلات بأطوال مختلفة
`MultiHeadAttention`	+1-2%	مرتفعة	تسلسلات طويلة جدًا + مهمّات معقّدة

> **قاعدة القرار:** كلّ أداة تكسب 1-4% وتكلّف زمنًا. لا تُضِف أداةً إلّا إذا كان مكسبها مُقاسًا على بياناتك لا على أرقام من ورقة.

مقارنة الأدوات الأربع على نفس البيانات

PYTHON

```
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات بأطوال مختلفة (مُحشوّة) ──
n, T, d = 600, 30, 6
x = np.random.randn(n, T, d).astype("float32")
lengths = np.random.randint(10, T, size=(n,))
for i, L in enumerate(lengths):
    x[i, L:, :] = 0.0
y = (x[:, :, 0].mean(axis=1) > 0.1).astype("int32")

def make_model(with_mask=False, bidir=False, conv=False):
    """يبني نموذجًا بأدوات محدّدة."""
    layers = [keras.Input(shape=(T, d))]
    if with_mask:
        layers.append(keras.layers.Masking(mask_value=0.0))
    if conv:
        layers += [keras.layers.Conv1D(16, 3, padding="same",
                                       activation="relu"),
                   keras.layers.MaxPooling1D(2)]
    core = keras.layers.LSTM(32)
    layers.append(keras.layers.Bidirectional(core) if bidir else core)
    layers.append(keras.layers.Dense(1, activation="sigmoid"))
    m = keras.Sequential(layers)
    m.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=[keras.metrics.BinaryAccuracy(name="acc")])
    return m

configs = [
    ("LSTM فقط", dict()),
    ("+ Masking", dict(with_mask=True)),
    ("+ Masking + Bidir", dict(with_mask=True, bidir=True)),
    ("+ Masking + Conv + Bidir", dict(with_mask=True, bidir=True,
```

```
conv=True)),
]
for name, kw in configs:
    m = make_model(**kw)
    m.fit(x, y, epochs=5, batch_size=32, validation_split=0.2, verbose=0)
    acc = m.evaluate(x[:150], y[:150], verbose=0)[1]
    params = m.count_params()
    print(f"{name:28s} acc={acc:.3f} params={params}")
# النتيجة المتوقعة: الدقّة تتحسّن تدريجيًّا، وعدد المعاملات يتضاعف
# مع Bidirectional (×2) ويقلّ مع Conv (تقليص الطول).
```

ملاحظة: هذه المقارنة هي نموذج الـ ablation الذي ناقشته الفصل 20: أداة واحدة في كلّ تدريب، وبذرة ثابتة، والنتيجة تُسجَّل في جدول – لا قرار بناءً على «شعور» أنّ الأداة مفيدة.

26.8 أخطاء شائعة في هذا الفصل

إنذار: استخدام `Bidirectional` مع `stateful=True` . لا تُدعم – أو تُعطي سلوكًا غير متوقّع لأنّ الاتجاهين يتطلّبان حالتين منفصلتين.

تحذير: `Bidirectional` مع طبقة تنبّؤ زمنيّ (توقّع الخطوة التالية). في هذا النمط، الاتجاه الخلفي «يرى» المستقبل – وهذا تسريب. استخدم Bidirectional في التصنيف فقط، لا في التنبّؤ.

تنبيه: `Conv1D` بلا `padding="same"` يُقلّص الأبعاد الزمنية، فلا يُطابق LSTM ما بعده. راجع الأبعاد في `summary()` قبل التدريب.

ملاحظة: `Masking` تعمل على قيمة واحدة (`mask_value=0.0`). إن كان مقياس بياناتك يُحيّد حول الصفر، فالصفر ليس «قيمة مفقودة» – راجع مقياسك قبل التطبيق.

قاعدة: `Bidirectional` بلا `return_sequences=True` يعطي مُخرَجًا واحدًا. إن أردت أن تُمرّره إلى LSTM أخرى أو انتباه، فأضِفها.

تحذير: `MultiHeadAttention` بحجم `key_dim` يساوي `projection_dim` كاملًا يضاعف المعاملات بلا فائدة. القاعدة: `key_dim = projection_dim // num_heads` .

التمارين

1. **حساب.** طبقة `LSTM(64)` على مُدخل 32 ميزة. كم معاملًا؟ (المتوقع: $4(32+64)\cdot 64 + 4\cdot 64 = 24{,}704$.)

2. **حساب.** `Bidirectional(LSTM(64))` على نفس الإعدادات. كم معاملًا؟ (المتوقع: ضعف الطبقة الأمامية = 49,408.)

3. **حساب.** `Conv1D(32, 3)` على تسلسل بطول 40. كم طول المُخرَج مع `padding="same"` ؟ (المتوقع: 40 – لأنّ الحشو يحفظ الطول.)

4. **حساب.** نفس الإعداد مع `padding="valid"` . كم طول المُخرَج؟ (المتوقع: $40 - 3 + 1 = 38$.)

5. **برمجة.** ابنِ نموذجًا بـ `Bidirectional(LSTM(64, return_sequences=True))` واطبع الشكل. (المتوقع: `(batch, T, 128)` – لأنّ الدمج يضاعف بعد الوحدة.)

6. **برمجة.** قارن زمن دورة تدريب بين `LSTM(64)` و `Bidirectional(LSTM(64))` على نفس البيانات. (المتوقع: 2× تقريبًا.)

7. **برمجة.** ابنِ نموذجًا بـ `Masking` ونموذجًا بلا، ودرّبهما على بيانات ذات أطوال مختلفة. (المتوقع: النموذج مع Masking أداءً أفضل بـ 1-2%.)

8. **إصلاح خطأ.** الكود التالي يعطي خطأ:

```python
inp = keras.Input(shape=(T, d))
x = keras.layers.Bidirectional(keras.layers.LSTM(64))(inp)
x = keras.layers.LSTM(32)(x)
```

(المتوقع: الطبقة الأولى تحتاج `return_sequences=True` لأنّها لا تعطي سوى الحالة الأخيرة، فلا يُقبل LSTM ثانية.) 9. **برمجة.** قارن `Bidirectional(LSTM(32))` مع `LSTM(64)` من حيث الدقّة وعدد المعاملات. (المتوقع: نفس عدد الوحدات تقريبًا، لكنّ Bidirectional أفضل دقّة على النصوص.) 10. **تحليل.** لماذا `Bidirectional` غير مناسب في التنبّؤ بالسلاسل الزمنية؟ (المتوقع: لأنّ الاتجاه الخلفي يرى المستقبل – وهذا تسريب؛ النموذج في الإنتاج لا يملك المستقبل.) 11. **تصميم.** نظام تصنيف تسلسلات صوتية بطول 200 خطوة. أيّ أدوات تُضيف، وبأيّ ترتيب؟ (المتوقع: Conv1D للنمط القصير ← MaxPooling لتقليص الطول ← Bidirectional LSTM؛ Masking إن كانت الأطوال متفاوتة.) 12. **برمجة.** طبّق `MultiHeadAttention` على مُخرَج LSTM بـ `return_sequences=True`، واطبع الشكل. (المتوقع: `(batch, T, projection_dim)` – بلا تغيير في الأبعاد الزمنية.)

ملخص الفصل

- `Bidirectional` يقرأ التسلسل في الاتجاهين ويُدمج المُخرَجات – يضاعف المعاملات.
- `Conv1D` قبل LSTM يُستخرج النمط القصير ويُقلّص زمن LSTM.
- `Masking` شرطٌ لا تحسين اختياريّ عندما تكون الأطوال متفاوتة.
- `MultiHeadAttention` على مُخرَج LSTM يُلتقط الاعتماد البعيد، لكنه يكلّف حسابًا مرتفعًا.
- كلّ أداة تعالج نقصًا محدّدًا – لا بديل شامل.
- قِس مكسب كلّ أداة على بياناتك لا على أرقام من الأوراق.

الفصل 27: معالجة اللغة الطبيعية عمليًا

بطاقة الفصل

الفصل 27

مهمّة هذا الفصل

يبني خطًّا نصيًّا كاملًا: ترميز، تضمين، تصنيف، ومقارنة البنى وحدودها.

ماذا تكسب منه

تحصل على مصنّف نصوص يعمل من أوّله إلى آخره، وتعرف حدوده قبل أن تعِد به أحدًا.

الدالة	دورها	متى تستخدمها
`keras.layers.TextVectorization()`	ترميز نصّي	أول طبقة في النموذج
`keras.layers.Embedding()`	تمثيل كثيف للرموز	بعد الترميز
`keras.layers.GlobalAveragePooling1D()`	متجه جملة واحدة	لتصنيف سريع
`keras.layers.LSTM()`	فهم ترتيبي	عندما يهمّ ترتيب الكلمات
`model.fit()`	تدريب مع أتمتة	مع EarlyStopping دائمًا

قصة مفتاحية: فريق يحتّاج نظامًا يصنّف 50 ألف مراجعة عملاء يوميًا إلى إيجابية/سلبية/محايدة. القراءة اليدوية تكلف فريق خدمة العملاء ثماني ساعات يوميًا. نموذج لغوي بسيط يحلّ هذا في ثوانٍ – لكنّ الفرق بين نموذج يعمل ونموذج «شبه يعمل» يقع في الطريقة التي تُحوّل بها الكلمات إلى أرقام، وفي الطريقة التي تُطبّع بها النصوص، وفي الطريقة التي تُبنى بها البنية. هذا الفصل هو الترجمة العملية لكلّ ما تعلّمته عن الشبكات التكرارية إلى مهمّة لغوية كاملة.

أهداف الفصل

- [ ] تبني خطّ معالجة لغوي كاملًا: نصّ خام ← تضمين ← شبكة ← تنبّؤ
- [ ] تتحكّم في `TextVectorization` بطريقة تلقائيّة ودليليّة
- [ ] تُميّز بين `Embedding` المُدرَّبة والجاهزة
- [ ] تُقارن LSTM و `GlobalAveragePooling1D` في مهمّة تصنيف نصوص
- [ ] تبني نظام تصنيف مشاعر كاملًا على بيانات حقيقية
- [ ] تعرف حدود النموذج الصغير ومتى تحتاج نموذجًا مُدرَّبًا مسبقًا

27.1 خطّ المعالجة اللغوية

شكل 36 – مسار نموذج لغوي كامل

خطّ المعالجة اللغوي يمرّ بأربع محطّات:

1. **تطبيع النصّ** – إزالة علامات الترقيم، وتوحيد الحالة (صغير/كبير)، وتقسيم إلى كلمات.
2. **تحويل الكلمات إلى أرقام** – بناء مفردات ثم تعيين رقم لكلّ كلمة.

3. **تحويل الأرقام إلى متّجهات** – تضمين (embedding) يُمثّل كلّ كلمة بمتّجه كثيف.
4. **الشبكة العصبية** – LSTM أو `GlobalAveragePooling1D` أو محوّل.

النقطة الحسّاسة: **لا تُبنى المفردات على بيانات الاختبار**. طبقة `TextVectorization` تُتعلَّم على التدريب فقط.

27.2 TextVectorization: من نصّ إلى أرقام

طبقة `TextVectorization` تُبنى في ثلاث خطوات:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات نصّية صغيرة
texts = np.array([
    "المنتج ممتاز جدًا",
    "الخدمة سيئة والموظف غير متعاون",
    "السعر مرتفع لكن الجودة جيدة",
    "توصيل سريع وجودة عالية",
    "أسوأ تجربة شراء",
    "منتج عادي ولا أُنصح به",
], dtype=object)
labels = np.array([1, 0, 1, 1, 0, 0])

# ── 2. طبقة التحويل
vectorizer = keras.layers.TextVectorization(
    max_tokens=1000,                 # أقصى عدد كلمات في المفردات
    output_mode="int",               # كلّ كلمة → رقم
    output_sequence_length=10,       # كلّ نصّ → تسلسل من 10 أرقام
)
# الخطوة الأولى: بناء المفردات على التدريب فقط
vectorizer.adapt(texts)
# الخطوة الثانية: تحويل النصوص
seq = vectorizer(texts)
# النتيجة المتوقعة: أوتور (6, 10) بأرقام صحيحة
```

مُعاملات TextVectorization

المعامل	ماذا يفعل	متى تُغيّره
`max_tokens`	حجم المفردات	بيانات كبيرة → 10000-50000
`output_sequence_length`	طول التسلسل	بحسب متوسّط طول النصّ
`standardize`	التطبيع	عربيّة → تخصيص دالّة
`split`	التقسيم	العربية لا تحتاج تغييرًا
`output_mode`	نمط المُخرَج	`int` للتضمين، `multi_hot` للنموذج البسيط

> **نصيحة:** في العربية، `standardize="lower_and_strip_punctuation"` تعمل جيدًا. لكن إن كانت بياناتك تحتوي مصطلحات خاصة، اكتب دالّة تخصّصًا لتفادي إزالة رموز مهمّة.

27.3 Embedding: من رقم إلى متّجه

طبقة `Embedding` تحوّل كلّ رقم (كلمة) إلى متّجه كثيف. المعاملان الأساسيان:

- `input_dim` : حجم المفردات (أكبر رقم في التسلسل + 1).
- `output_dim` : حجم كلّ متّجه (16 إلى 300 نموذجيًّا).

PYTHON

```
import keras

# تضمين بسيط
embed = keras.layers.Embedding(input_dim=1000, output_dim=32)
out = embed(keras.ops.array([[1, 2, 3]]))
# النتيجة المتوقعة: أوتور (1, 3, 32) — متّجه 32 عنصرًا لكلّ كلمة.
```

27.4 نموذج تصنيف نصّ كامل

فيما يلي نموذج كامل: تطبيع → تضمين → LSTM → تصنيف.

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات نصّية (مصنّعة) — 400 مراجعة
n = 400
vocab = ["سريع", "توصيل", "غير", "مُنصح", "متوسط", "جيد", "سيئ", "ممتاز",
         "موظف", "خدمة", "عالية", "جودة", "منخفض", "مرتفع", "سعر", "بطيء"]
texts = []
labels = []
for _ in range(n):
    words = list(np.random.choice(vocab, size=8, replace=True))
    texts.append(" ".join(words))
    # قاعدة مبسّطة: كلمات موجبة → 1
    positive = {"ممتاز", "جيد", "منصح", "سريع", "منخفض", "عالية"}
    labels.append(int(any(w in positive for w in words)))
texts = np.array(texts, dtype=object)
labels = np.array(labels, dtype="int32")

# ── 2. التحويل إلى أرقام
vectorizer = keras.layers.TextVectorization(max_tokens=200,
                                            output_sequence_length=12)
vectorizer.adapt(texts)

# ── 3. النموذج: تضمين → GlobalAveragePooling → تصنيف
inp = keras.Input(shape=(1,), dtype="string")
x = vectorizer(inp)                                  # (B, 12) أرقام
x = keras.layers.Embedding(input_dim=200, output_dim=32)(x)   # (B, 12, 32)
x = keras.layers.GlobalAveragePooling1D()(x)         # (B, 32)
x = keras.layers.Dense(16, activation="relu")(x)
x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inp, out)

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
model.fit(texts, labels, epochs=10, batch_size=16,
```

```
        validation_split=0.2, verbose=0)
# النتيجة المتوقعة: دقّة ≈ 0.75-0.85 على البيانات المصنّعة.
```

ملاحظة: `GlobalAveragePooling1D` أسرع بكثير من LSTM، وأداؤه قريب على النصوص القصيرة. ابدأ به، ثم انتقل إلى LSTM إن لم يكفِ.

27.5 LSTM مقابل GlobalAveragePooling1D

المعيار	`GlobalAveragePooling1D`	LSTM
زمن التدريب	أسرع 3-10×	أبطأ
زمن الاستدلال	أسرع	أبطأ
أداء على نصوص قصيرة (< 50 كلمة)	جيّد	جيّد
أداء على نصوص طويلة (> 200 كلمة)	ضعيف	جيّد
يراعي الترتيب	لا	نعم
متى تستخدمه	مهمّات بسيطة، استدلال سريع	مهمّات تعتمد على الترتيب

قاعدة: إذا كان ترتيب الكلمات مهمّ (مثل «ليس جيّد»)، استخدم LSTM أو Bidirectional. إذا كان حضور الكلمات مهمًّا فقط، فـ `GlobalAveragePooling1D` كافٍ وأسرع.

27.6 مثال محلول: نظام تصنيف مراجعات

المهمة: تصنيف 50 ألف مراجعة عملاء إلى إيجابية/سلبية. لديك 5000 مراجعة موسومة يدويًّا.

الخطوة 1 – خطّ الأساس: `GlobalAveragePooling1D` . تطبيع → تضمين 32 → متوسّط → Dense. دقّة التحقّق = 84%.

الخطوة 2 – LSTM. استبدال المتوسّط بطبقة `LSTM(64)` . دقّة = 87%. **مكسب +3%** – لأنّ «ليس جيّد» تُقرأ الآن كتسلسل.

الخطوة 3 – Bidirectional. `Bidirectional(LSTM(64))` . دقّة = 89%. **مكسب +2%.**

الخطوة 4 – تضمين جاهز. بدل تضمين مُدرَّب، استخدمنا تضمين جاهز (مثل `GloVe` أو تضمين عربيّ). دقّة = 90%. **مكسب +1%** على بيانات صغيرة.

الخطوة 5 – القرار. احتفظنا بـ Bidirectional + تضمين مُدرَّب: أداء كافٍ وكلفة مقبولة، ولا حاجة لاستيراد تضمين جاهز.

27.7 حدود النموذج الصغير

النموذج الصغير (تضمين + LSTM) لا يفهم: - **السياق الطويل:** مراجعة بـ 300 كلمة تُقطّع في LSTM إلى تسلسل ضخم. - **المعنى المتعدّد:** كلمة «خفيف» تعني سرعة في السياق ووزنًا في سياق آخر. - **الظّواهر اللغويّة:** السخرية، الاستعارة، التشبيه – كلّها تفلت من النموذج الصغير.

عند هذه الحدود، انتقل إلى نموذج مُدرَّب مسبقًا (BERT أو محوّل عربيّ) – راجع الفصل 30.

27.8 أخطاء شائعة في هذا الفصل

إنذار: نداء `adapt()` على بيانات الاختبار. هذه تسريب – المفردات تُبنى على التدريب فقط. نتيجتها: أداء يبدو جيّدًا على الاختبار وينهار في الإنتاج.

تحذير: `input_dim` في `Embedding` أقلّ من حجم المفردات. النتيجة: خطأ indexing عند التدريب. القاعدة: `input_dim = max_tokens` في `TextVectorization` .

تنبيه: استخدام `output_sequence_length` أكبر من متوسّط طول النصّ بمرّات. النتيجة: تسلسلات مُحشوّة بالأصفار، وكلفة حسابية زائدة. راجع توزيع الأطوال قبل تحديد القيمة.

ملاحظة: تمرير نصوص خام إلى `Embedding` مباشرةً بلا `TextVectorization` يُعطي خطأ dtype. النصوص تحتاج تحويلًا إلى أرقام أولًا.

قاعدة: `TextVectorization` طبقة تُدرَّب (عبر `adapt`) – لذا يجب أن تُحفظ مع النموذج، ولا تُبنى في خطّ نشر منفصل.

تحذير: توقّع أنّ نموذج LSTM الصغير يفهم السخرية أو المعنى المتعدّد. النموذج الصغير يفهم **الحضور والتكرار** لا السياق العميق.

تحديد طول التسلسل من البيانات

`output_sequence_length` لا تُختار عشوائيًّا – تُختار من توزيع الأطوال:

PYTHON

```python
import numpy as np

def choose_sequence_length(texts, coverage=0.95):
    """يختار طولًا يغطي coverage من النصوص."""
    lengths = np.array([len(str(t).split()) for t in texts])
    chosen = int(np.quantile(lengths, coverage))
    return chosen, lengths

# ── 1. (محاكاة) بيانات بأطوال متفاوتة
rng = np.random.default_rng(42)
texts = [" ".join(["كلمة"] * int(rng.integers(5, 80))) for _ in range(1000)]

# ── 2. اختيار الطول
chosen, lengths = choose_sequence_length(texts, coverage=0.95)
print(f"متوسّط الطول = {lengths.mean():.1f}، أقصى = {lengths.max()}")
print(f"95% طول يغطي = {chosen}")
print(f"نسبة الحشو عند {chosen} = "
      f"{1 - lengths.mean() / chosen:.2%}")
# النتيجة المتوقعة: الطول المختار أقلّ من الأقصى بمرّات،
# ونسبة الحشو في النطاق المقبول (< 40%).
```

قاعدة: إن كانت نسبة الحشو أكبر من 40%، فالطول كبير جدًّا – وكلّ حشو كلفة حساب لا فائدة. إن كانت أقلّ من 10%، فأنت تُقصّ نصوصًا حقيقيّة – ارفع الطول أو راجع التغطية.

التمارين

1. **حساب.** مفردات بحجم 5000 كلمة، وتضمين بحجم 64. كم معاملًا في طبقة `Embedding` ؟ (المتوقع: $5000 \times 64 = 320{,}000$.)
2. **حساب.** نصّ بطول 30 كلمة، وتضمين بحجم 32. ما شكل المُخرَج بعد `Embedding` ؟ (المتوقع: `(batch, 30, 32)` .)
3. **حساب.** نفس الإعداد بعد `GlobalAveragePooling1D` . ما شكل المُخرَج؟ (المتوقع: `(batch, 32)` .)
4. **برمجة.** ابنِ `TextVectorization` على 100 مراجعة واطبع مفرداتها. (المتوقع: مفردات مصنّفة بترتيب التكرار – الكلمة الأكثر تكرارًا لها رقم صغير.)
5. **برمجة.** قارن زمن دورة تدريب بين `GlobalAveragePooling1D` و `LSTM(64)` على نفس البيانات. (المتوقع: LSTM أبطأ 3-10x.)
6. **برمجة.** طبّق `Bidirectional(LSTM(64))` على مهمّة تصنيف نصّ واطبع عدد المعاملات. (المتوقع: ضعف 2 = xLSTM.)
7. **إصلاح خطأ.** الكود التالي يعطي خطأ:

```python
inp = keras.Input(shape=(1,), dtype="string")
x = keras.layers.Embedding(1000, 32)(inp)
```

(المتوقع: النصوص تحتاج `TextVectorization` قبل `Embedding` – لا يمكن تمرير نصّ مباشرةً.) 8. **برمجة.** قارن `max_tokens=500` مع `max_tokens=5000` على نفس البيانات. (المتوقع: 5000 يعطي مفردات أكبر ودقّة أفضل على بيانات كافية، لكنه يستهلك ذاكرة أكثر.) 9. **تصميم.** نظام تصنيف بريد إلكتروني إلى «مهمّ/غير مهمّ». أيّ بنية، وكم حجم المفردات؟ (المتوقع: تضمين 32 + `GlobalAveragePooling1D` – مهمّة بسيطة والبريد قصير، ومفردات 5000-10000.) 10. **تحليل.** لماذا يفشل نموذج LSTM الصغير في فهم السخرية؟ (المتوقع: لأنّ الكلمات المستعملة في السخرية إيجابية لكنّ المعنى سلبيّ – النموذج الصغير يقرأ الحضور لا السياق.) 11.

تصميم. نظام تصنيف مراجعات بمتوسّط طول 300 كلمة. هل تختار LSTM أو محوّلًا؟ (المتوقع: محوّلًا مُدرَّبًا مسبقًا – لأنّ LSTM على تسلسل 300 يُعاني من الذاكرة البعيدة.) 12. **برمجة.** طبّق `TextVectorization` بـ `output_mode="multi_hot"` وقارن مع `output_mode="int"`. (المتوقع: `multi_hot` يعطي متّجهًا ثنائيًّا بحجم المفردات، ويُستعمل في نموذج بسيط.)

ملخص الفصل

- خطّ المعالجة اللغوي: تطبيع ← تحويل إلى أرقام ← تضمين ← شبكة.
- `TextVectorization` تُبنى على بيانات التدريب فقط، وتُحفظ مع النموذج.
- `Embedding` تحوّل كلّ كلمة إلى متّجه كثيف، ومعاملاتها تُتعلَّم.
- `GlobalAveragePooling1D` أسرع بكثير من LSTM وأداؤه قريب على النصوص القصيرة.
- LSTM و Bidirectional تُلزمان مهمّات يعتمد فيها ترتيب الكلمات.
- النموذج الصغير يفهم الحضور والتكرار، لا السياق العميق – وهنا تنتقل إلى نموذج مُدرَّب مسبقًا.

الفصل 28: السلاسل الزمنية

بطاقة الفصل

الفصل 28

مهمّة هذا الفصل

يبوّع السلسلة إلى نوافذ، ويقسّم زمنيًّا، ويبني خطّ أساس قبل أي نموذج.

ماذا تكسب منه

تتجنّب أشهر خطأ – تسريب المستقبل – وتعرف هل نموذجك أفضل من التوقّع الساذج أصلًا.

الدالة	دورها	متى تستخدمها
`keras.utils.timeseries_dataset_from_array()`	بناء نوافذ منزلقة	أول خطوة في أي مشروع زمني
`keras.layers.LSTM()`	نموذج تسلسلي	بعد التبويع
`keras.layers.Dense()`	مخرج عددي	في رأس الانحدار
`keras.metrics.MeanAbsoluteError()`	خطأ بالمقياس الأصلي	لتفسير النتائج لغير المختصّ
`model.evaluate()`	تقييم على آخر فترة	قبل الحكم على النموذج

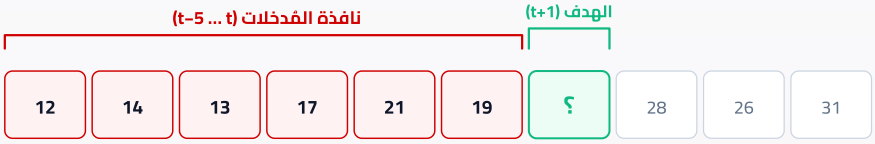
قصة مفتاحية: فريق تحليل مالي يحتّاج نظامًا يتنبّأ بمستوى مبيعات الأسبوع القادم من آخر 26 أسبوعًا. النموذج الذي بُني انطلق بدقّة 92% على بيانات التدريب و88% على التحقّق – ثم فشل في الإنتاج. السبب: البيانات قُسّمت عشوائيًّا، فأُعطي النموذج أجزاءً من المستقبل أثناء التدريب. السلاسل الزمنية لها قاعدة واحدة صارمة: **الزمن لا يُقسم عشوائيًّا**.

أهداف الفصل

- [ ] تحوّل سلسلة زمنيّة إلى نوافذ (windowing) في Keras
- [ ] تبني خطّ أساس (baseline) بسيطًا وتقارن عليه
- [ ] تُطبّع البيانات بطريقة لا تُسرّب المعلومات
- [ ] تبني نموذج LSTM لنموذج الانحدار على نوافذ
- [ ] تُطبّق الاختبار العكسيّ (walk-forward) على النموذج
- [ ] تعرف حدود التنبّؤ ومتى لا يستحقّ التطبيق

28.1 التبويع: من سلسلة إلى نوافذ

النافذة (Window): كيف نحوّل السلسلة إلى تصنيف أو انحدار؟



المخرج y	المدخل X	الحالة
القيمة التالية	معدّل متحرّك	النموذج الخطي
القيمة التالية	سلسلة سابقة + مواسم	ARIMA
القيمة التالية	نافذة (t−5...t)	LSTM

خطأ قاتل: خلط عشوائي (shuffle=True) في السلاسل الزمنية
يقتل العلاقة الزمنية ويمنحك دقّة وهمية — استخدم shuffle=False.

شكل 37 – تبويع السلاسل الزمنية إلى نوافذ

السلسلة الزمنيّة هي تسلسل $x_1, x_2, \ldots, x_T$. لتدريب نموذج، نُقطّعها إلى نوافذ:

$$X = [x_{t-w}, \ldots, x_{t-1}], \qquad y = x_t$$

حيث w حجم النافذة، وx_t القيمة التي نتنبّأ بها. النافذة تُزاح خطوة (أو أكثر) في كلّ مرة، فتُنتج أزواج (مُدخل، مُخرَج).

في Keras، تُنفّذ التبويع عبر `keras.utils.timeseries_dataset_from_array`.

28.2 قاعدة التقسيم الزمنيّ

القاعدة الحاسمة: **تقسيم زمنيّ، لا عشوائيّ.**

- **تدريب:** أول 70% من السلسلة (الزمن الباكر).
- **تحقّق:** 10% الوسطى (الزمن الأوسط).
- **اختبار:** آخر 20% (الزمن الأحدث).

بلا هذا التقسيم، يكون النموذج قد رأى جزءًا من ما يتنبّأ به – وهذا تسريب لا يُخفى في الإنتاج.

> **إنذار:** لا تستخدم `validation_split=0.2` في `model.fit` على بيانات زمنيّة – فهذا يُقسّم عشوائيًّا. قسّم يدويًّا قبل تمرير البيانات.

28.3 خطّ الأساس (Baseline)

قبل بناء نموذج معقّد، ابنِ خطّ أساس بسيط. الخطّ الأساس الأشهر هو **التنبّؤ بالقيمة السابقة** (naive forecast):

$$\hat{x}_t = x_{t-1}$$

وهو يُعطي دقّةً كثيرًا ما تكون أعلى من نموذج معقّد على سلاسل بطيئة. الخطّ الأساس الثاني هو **متوسّط النافذة** (moving average).

> **قاعدة:** لا يُعتمد نموذج معقّد إلّا إذا تفوّق على خطّ الأساس البسيط. نموذج لا يتفوّق على «التكرار السابق» نموذج لا قيمة له.

28.4 التنفيذ في Keras

فيما يلي خطّ كامل: بيانات زمنيّة مُصنّعة ← تبويع ← LSTM ← تقييم.

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. سلسلة زمنيّة مُصنّعة: 1000 خطوة (نمط دوريّ + ضجيج) ──
T = 1000
t = np.arange(T)
series = (np.sin(2 * np.pi * t / 50) * 10.0          # دوريّ
          + np.linspace(0, 5, T)                      # اتجاه صاعد
          + np.random.randn(T)).astype("float32")

# ── 2. تويع: نافذة 30 خطوة، وتنبّؤ بالخطوة التالية ──
WINDOW = 30
def make_windows(arr, window):
    """يحوّل السلسلة إلى نوافذ (X, y) بترتيب زمنيّ."""
    xs, ys = [], []
    for i in range(len(arr) - window):
        xs.append(arr[i:i + window])
        ys.append(arr[i + window])
    return np.array(xs)[..., None], np.array(ys)

# ── 3. تقسيم زمنيّ (70/10/20) ──
train_end = int(0.7 * T)
val_end = int(0.8 * T)
x_train, y_train = make_windows(series[:train_end], WINDOW)
x_val, y_val = make_windows(series[train_end - WINDOW:val_end], WINDOW)
x_test, y_test = make_windows(series[val_end - WINDOW:], WINDOW)

# ── 4. تطبيع بحسب التدريب فقط (لا تسريب) ──
mean = x_train.mean()
std = x_train.std()
x_train = (x_train - mean) / std
x_val = (x_val - mean) / std
x_test = (x_test - mean) / std
y_train = (y_train - mean) / std
y_val = (y_val - mean) / std
y_test = (y_test - mean) / std
```

```
# ── 5. النموذج: LSTM للانحدار
inp = keras.Input(shape=(WINDOW, 1))
x = keras.layers.LSTM(32)(inp)
out = keras.layers.Dense(1)(x)                          # انحدار: بلا تنشيط
model = keras.Model(inp, out)
model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse",
              metrics=[keras.metrics.MeanAbsoluteError(name="mae")])
model.fit(x_train, y_train, validation_data=(x_val, y_val),
          epochs=10, batch_size=32, verbose=0)

# ── 6. خطّ الأساس: التكرار السابق
naive_pred = x_test[:, -1, 0]
naive_mae = np.mean(np.abs(naive_pred - y_test))
model_mae = model.evaluate(x_test, y_test, verbose=0)[1]
print(f"naive MAE = {naive_mae:.3f}, model MAE = {model_mae:.3f}")
# النتيجة المتوقعة: model MAE أقلّ من naive MAE (شرط التقبّل).
```

المقارنة الكاملة على خطّ أساسين

PYTHON

```python
import numpy as np

# ── 1. ثلاث تنبّؤات: naive، متوسّط النافذة، LSTM
naive = x_test[:, -1, 0]                    # آخر قيمة
window_avg = x_test.mean(axis=1)[:, 0]      # متوسّط النافذة
lstm_pred = np.array(model.predict(x_test, verbose=0))[:, 0]

def mae(pred, truth):
    return float(np.mean(np.abs(pred - truth)))

# ── 2. الجدول
rows = [("naive (آخر قيمة)", mae(naive, y_test)),
        ("moving average", mae(window_avg, y_test)),
        ("LSTM", mae(lstm_pred, y_test))]
for name, v in rows:
    print(f"{name:22s} MAE = {v:.4f}")
best = min(rows, key=lambda r: r[1])
print(f"الأفضل: {best[0]}")
# النتيجة المتوقعة: LSTM و moving average أفضل، قريب من naive.
```

قاعدة القرار: إن لم يتفوّق النموذج على **moving average** أيضًا (لا naive وحده)، فالسلسلة مستقرّة جدًّا ولا تحتاج شبكة عصبيّة – أنجز الحساب بمتوسّط وأوفّر الكلفة.

28.5 بنى النموذج للسلاسل الزمنية

البنية	ماذا تُحلّ	متى تستخدمها
خطّ أساس (naive)	مرجع للمقارنة	دائمًا أوّلًا
`LSTM` / `GRU`	اعتماد زمنيّ غير خطّيّ	نمط معقّد، بيانات كافية
`Conv1D` + `LSTM`	نمط قصير + اعتماد بعيد	تسلسلات طويلة
محوّل (attention)	اعتماد بعيد + توازي	بيانات كثيرة، تسلسل طويل جدًا
TCN (تفاف سببيّ)	اعتماد بعيد + سرعة	تسلسلات طويلة + زمن استدلال حرج

> **نصيحة:** ابدأ بخطّ أساس، ثم LSTM بسيطة، ثم أضِف بنيةً إضافيّةً فقط إذا كان المكسب مُقاسًا.

28.6 الاختبار العكسيّ (Walk-Forward)

الاختبار العكسيّ يحاكي الإنتاج: نُدرّب على الفترة الأولى، ونتنبّأ بالفترة الثانية، ثم نُضيف الفترة الثانية إلى التدريب ونكرّر. هذا يُعطي تقديرًا واقعيًّا للأداء.

الطريقة	ماذا تقيس	متى تستخدمها
تقسيم واحد	أداء على فترة واحدة	بيانات قليلة
Walk-forward	متوسّط الأداء عبر الفترات	بيانات كافية، مهمّة إنتاج
K-fold زمنيّ	أداء عبر عدّة نوافذ	بيانات كثيرة

28.7 حدود التنبّؤ

السلاسل الزمنيّة لها حدود جوهرية: - **العشوائيّة:** أحداث لا يمكن توقّعها (أزمة، كارثة) تُفسد النموذج. - **الانحراف:** توزيع البيانات يتغيّر مع الزمن، فالنموذج المدرَّب على الفترة القديمة يضعف. - **النافذة:** التنبّؤ بخطوة واحدة أسهل بكثير من التنبّؤ بـ 10 خطوات، وكلفة الخطأ تنمو أُسّيًّا مع الأفق.

> **تحذير:** التنبّؤ بعشر خطوات إلى الأمام يُعطي أخطاءً أكبر بكثير من التنبّؤ بخطوة واحدة. راجع الأفق قبل أن تُقرر إن كان النموذج «يعمل».

28.8 مثال محلول: تنبّؤ بالمبيعات الأسبوعيّة

المهمة: تنبّؤ بمبيعات أسبوعيّة من آخر 26 أسبوعًا. لديك 5 سنوات (260 أسبوعًا).

الخطوة 1 – خطّ الأساس. naive forecast. MAE على الاختبار = 120 وحدة.

الخطوة 2 – LSTM بسيطة. نافذة 26، LSTM(32)، MAE = 105. **تحسّن 12%.**

الخطوة 3 – نافذة أطول. نافذة 52 (سنة كاملة). MAE = 98. **تحسّن 6%.**

الخطوة 4 – ميزات إضافية. نُضيف ميزات: اليوم من الأسبوع، الشهر، عطلات. MAE = 91. **تحسّن 7%.**

الخطوة 5 – Walk-forward. تقييم على 4 نوافذ زمنيّة متتالية. متوسّط MAE = 95. النموذج مستقرّ.

الخطوة 6 – القرار. النموذج يتفوّق على خطّ الأساس في كلّ الفترات، ويُعتمد. لكنّه يُحدَّث شهريًّا لأنّ التوزيع يتغيّر.

تأثير الأفق: خطوة واحدة مقابل عشر

الأفق يحدّد جودة التنبّؤ أكثر من البنية:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. سلسلة مُصنّعة
T = 600
t = np.arange(T)
series = (np.sin(2 * np.pi * t / 50) * 10.0
          + np.random.randn(T) * 2.0).astype("float32")

def make_windows(arr, window, horizon):
    """نوافذ تتنبأ بالخطوة t + horizon."""
    xs, ys = [], []
    for i in range(len(arr) - window - horizon + 1):
        xs.append(arr[i:i + window])
        ys.append(arr[i + window + horizon - 1])
    return np.array(xs)[..., None], np.array(ys)

def train_eval(horizon):
    """تدريب وتقييم بأفق محدّد."""
    x, y = make_windows(series[:400], 30, horizon)
    m = keras.Sequential([keras.Input(shape=(30, 1)),
                          keras.layers.LSTM(32),
                          keras.layers.Dense(1)])
    m.compile(optimizer="adam", loss="mse",
              metrics=[keras.metrics.MeanAbsoluteError(name="mae")])
    m.fit(x, y, epochs=5, batch_size=32, verbose=0)
    x_t, y_t = make_windows(series[400:], 30, horizon)
    return float(m.evaluate(x_t, y_t, verbose=0)[1])

for h in (1, 5, 10):
    mae = train_eval(h)
    print(f"horizon {h}: MAE = {mae:.4f}")
# النتيجة المتوقعة: MAE يزداد بمرور الأفق — أوسع أُسّيًّا لا خطّيًّا.
```

قاعدة عملية: إن كان الأفق 10 خطوات، ولا يتفوّق النموذج على «آخر قيمة مكرّرة»، فالنموذج غير صالح – وبنية أعمق لن تحلّ مشكلة معلومة مفقودة.

28.9 أخطاء شائعة في هذا الفصل

إنذار: تقسيم عشوائيّ في السلاسل الزمنيّة. هذا تسريب مباشر – النموذج يرى جزءًا من المستقبل. قسّم زمنيًّا دائمًا.

تحذير: تطبيع بحسب كلّ البيانات. المعدّل والانحراف يجب أن يُحسبا على التدريب فقط، ثم تُطبَّق على التحقّق والاختبار. تطبيع على الكلّ = تسريب.

تنبيه: استخدام `validation_split` في `model.fit` على بيانات زمنيّة. هذا يُقسّم عشوائيًّا. قسّم يدويًّا ومرّر `validation_data` صراحةً.

ملاحظة: بناء نوافذ قبل التقسيم الزمنيّ يُسرّب المعلومات: نافذة على الحدّ تحتوي بيانات من الفترتين. قسّم أوّلًا ثم بُنِ نوافذ على كلّ فترة.

قاعدة: لا تُعتمد بنية معقّدة إلّا إذا تفوّقت على خطّ الأساس. نموذج LSTM لا يتفوّق على naive forecast نموذج لا قيمة له.

تحذير: تطبيع المُخرَج (y) بطريقة مختلفة عن المُدخل. راجع: هل y مُطبَّع بنفس المعدّل والانحراف؟ وإلّا، نتائج الاستدلال ستكون بمقياس خاطئ.

التمارين

1. **حساب.** سلسلة بطول 100 خطوة، ونافذة 10. كم نافذة تُنتَج؟ (المتوقع: $100 - 10 = 90$.)
2. **حساب.** سلسلة بطول 500، تقسيم زمنيّ 70/10/20. كم خطوة في كلّ فترة؟ (المتوقع: تدريب 350، تحقّق 50، اختبار 100.)
3. **حساب.** سلسلة القيم [10, 12, 11, 13, 12, 14]. ما تنبّؤ naive للخطوة السابعة؟ (المتوقع: 14 – قيمة الخطوة الأخيرة.)
4. **برمجة.** اكتب دالّة `make_windows(arr, window)` تعيد نوافذ بترتيب زمنيّ، واطبع شكل النتيجة على سلسلة 200 خطوة ونافذة 30. (المتوقع: `(170, 30, 1)` و `(170,)`.)
5. **برمجة.** قسّم سلسلة زمنيّة إلى تدريب/تحقّق/اختبار بترتيب زمنيّ، واحسب `mean` و `std` على التدريب فقط. (المتوقع: لا تسريب.)
6. **برمجة.** قارن MAE بين نموذج LSTM وخطّ الأساس naive على سلسلة مُصنّعة. (المتوقع: LSTM أقلّ MAE من naive في النمط الدوريّ.)
7. **إصلاح خطأ.** الكود التالي يُسرّب المعلومات. حدّد الخطأين:

```python
x_all, y_all = make_windows(series, 30)
x_train, x_test = x_all[:300], x_all[300:]
mean, std = x_all.mean(), x_all.std()
```

(المتوقع: 1) التطبيع على كلّ البيانات لا على التدريب فقط، و2) النوافذ تُبنى قبل التقسيم فتُسرّب المعلومات عبر الحدود.) 8. **برمجة.** طبّق `keras.utils.timeseries_dataset_from_array` على سلسلة مُصنّعة، واطبع شكل البيانات. (المتوقع: شكل 9 (`(batch, sequence_length, 1)`.) **تصميم.** نظام يتنبّأ بمبيعات أسبوعيّة من 26 أسبوعًا، وبيانات 5 سنوات. صمّم خطّ التدريب الكامل. (المتوقع: تقسيم زمنيّ 70/10/20، تطبيع على التدريب، نوافذ 26، LSTM 32، تقييم walk-forward.) 10. **تحليل.** لماذا يفشل تقسيم عشوائيّ في السلاسل الزمنيّة؟ (المتوقع: لأنّ القيمة عند الزمن t مرتبطة بقيم $t-1$ و$t+1$ – التقسيم العشوائيّ يجعل النموذج يرى القيم المجاورة في التدريب.) 11.

تصميم. نظام يتنبّأ بخطوتين إلى الأمام. هل تُبني نموذجًا واحدًا أو اثنين؟ (المتوقع: نموذجان منفصلان (خطوة 1 وخطوة 2) أو نموذج واحد يُبنى تكراريًّا – والثاني أبسط في التدريب.) 12. **برمجة.** طبّق walk-forward على سلسلة مُصنّعة بأربع نوافذ، واطبع MAE لكلّ نافذة. (المتوقع: MAE متقارب في الفترات إن كان التوزيع مستقرًّا.)

ملخص الفصل

- السلاسل الزمنيّة تُقطّع إلى نوافذ، وكلّ نافذة تُنتج زوجًا (مُدخل، مُخرَج).
- القاعدة الحاسمة: تقسيم زمنيّ، لا عشوائيّ. النوافذ تُبنى بعد التقسيم.
- التطبيع يُحسب على التدريب فقط، ثم يُطبَّق على التحقّق والاختبار.
- خطّ الأساس (naive forecast) مرجع إلزاميّ قبل أيّ نموذج معقّد.
- LSTM بسيطة تعمل جيّدًا على تسلسلات متوسّطة؛ الميزات الإضافيّة تُحسّن الأداء.
- Walk-forward يُقدّر الأداء على عدة فترات زمنيّة – وهو الأقرب للاستدلال الحقيقيّ.
- التنبّؤ بأفق أطول يعطي أخطاء أكبر – راجع الأفق قبل قرار التقبّل.

الجزء السادس

المحوّلات: انتباه، محوّل، وKerasHub

المحوّلات: معادلة الانتباه، الانتباه متعدّد الرؤوس، كتلة المحوّل كاملة مع القناع السببي، ثم منظومة KerasHub والنماذج الجاهزة.

ما تكسبه من هذا الجزء

تنفّذ محوّلًا بنفسك من أوّله إلى آخره، وتستخدم نموذجًا مُدرَّبًا مسبقًا وتضبطه على بياناتك في سطور.

فصول هذا الجزء

29 المحوّلات وآلية الانتباه

30 منظومة KerasHub وKerasNLP

6

الفصل 29: المحوّلات وآلية الانتباه

بطاقة الفصل

الفصل 29

مهمّة هذا الفصل

يشتقّ الانتباه، ويبني كتلة محوّل كاملة مع قناع سببي وترميز موضعي.

ماذا تكسب منه

تفهم المعادلة التي بُني عليها كل نموذج لغة حديث، وتنفّذها بنفسك لا تستهلكها كصندوق أسود.

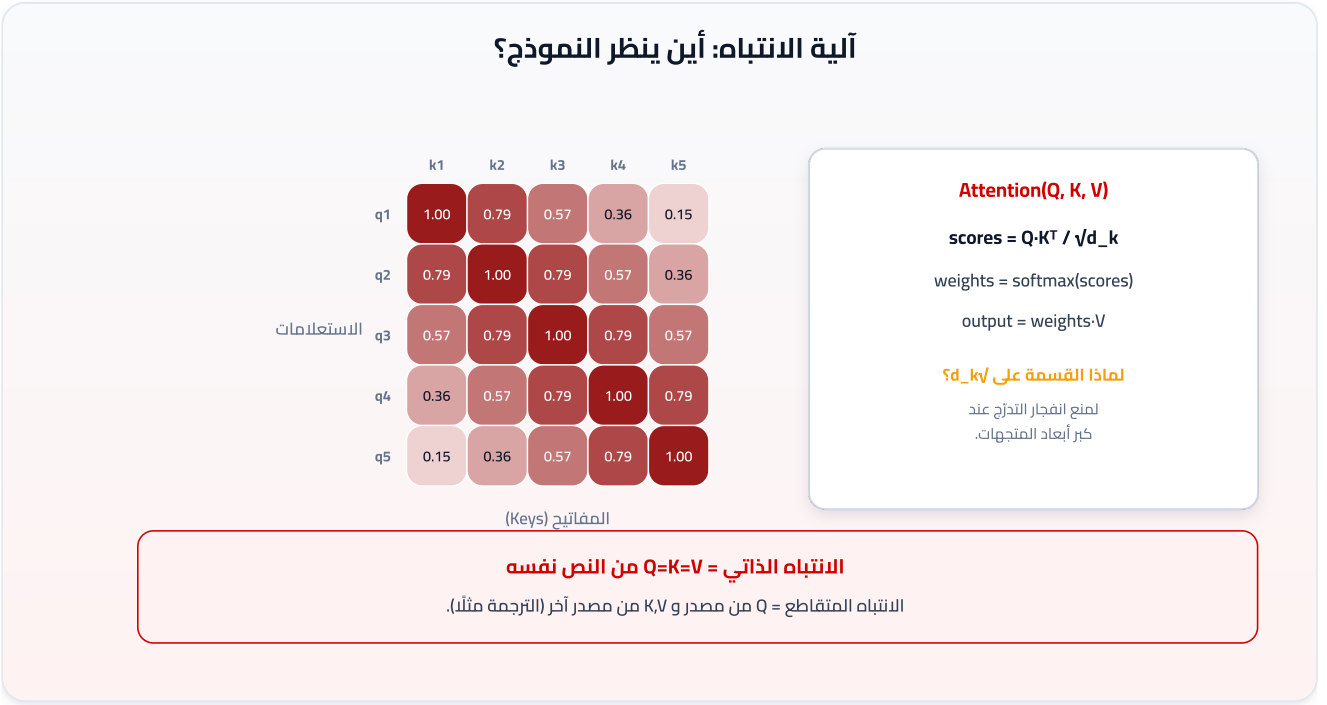
الدالة	دورها	متى تستخدمها
`keras.layers.MultiHeadAttention()`	انتباه متعدّد الرؤوس	قلب المحوّل
`keras.layers.LayerNormalization()`	تطبيع طبقي	قبل كل كتلة فرعية
`keras.layers.Embedding()`	ترميز الرموز	أول طبقة
`keras.ops.arange()`	بناء الترميز الموضعي	لإضافة المعلومة الموضعية
`keras.layers.Add()`	وصلات متبقّية	في كل كتلة

قصة مفتاحية: فريق معالجة لغوية يحتّاج نموذجًا يفهم جملة من 200 كلمة. طبقة LSTM تقرأ كلمة بكلمة وتُجمّع في حالة واحدة – وفي الجملة الطويلة تضيع المعلومات الباكرة. المحوّل يلغي الترتيب: يضع كلّ كلمة أمام كلّ الكلمات في طبقة انتباه واحدة، فيربط «هو» بـ«الطبيب» مهما كانت المسافة بينهما. هذه القفزة من «الذاكرة التسلسليّة» إلى «الانتباه الكامل» هي التي أعادت تشكيل مجال معالجة اللغة كلّه.

أهداف الفصل

- [ ] تشرح آلية الانتباه بالمعادلة الكاملة
- [ ] تبني كتلة محوّل كاملة في Keras (انتباه + MLP + تطبيع + اتصالات تخطّي)
- [ ] تفهم القناع السببيّ (causal masking) في فكّ الترميز
- [ ] تبني encoderg decoder لمهمّة ترجمة بسيطة
- [ ] تُقارن الانتباه الذاتيّ مع الانتباه التقاطعيّ
- [ ] تعرف أثر عدد الرؤوس وحجم التضمين

29.1 آلية الانتباه: المعادلة



شكل 38 – آلية الانتباه

الانتباه يربط بين ثلاث مصفوفات: السؤال (Query) والمفتاح (Key) والقيمة (Value):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

حيث: - Q: مصفوفة الأسئلة – «ما الذي أبحث عنه؟». - K: مصفوفة المفاتيح – «ما الذي يمكن أن أُطابقه؟». - V: مصفوفة القيم – «ما الذي سأجمعه إن طابقت؟». - d_k: بُعد كلّ مفتاح، والجذر التربيعيّ له يُقلّص الدرجات قبل softmax. - $QK^\top$: جداء نقاطيّ يعطي «شبكة تشابه» بين كلّ سؤال وكلّ مفتاح. - softmax: يُطوّع الشبكة إلى توزيع احتماليّ (كلّ صفّ يجمع إلى 1). - النتيجة: تجميع موزون للقيم V.

> **قاعدة عميقة:** الانتباه يُنتج **مصفوفة أوزان** بحجم $(n \times n)$ حيث n عدد الرموز – وهذا هو سبب كلفة الانتباه $O(n^2)$.

29.2 الانتباه متعدّد الرؤوس

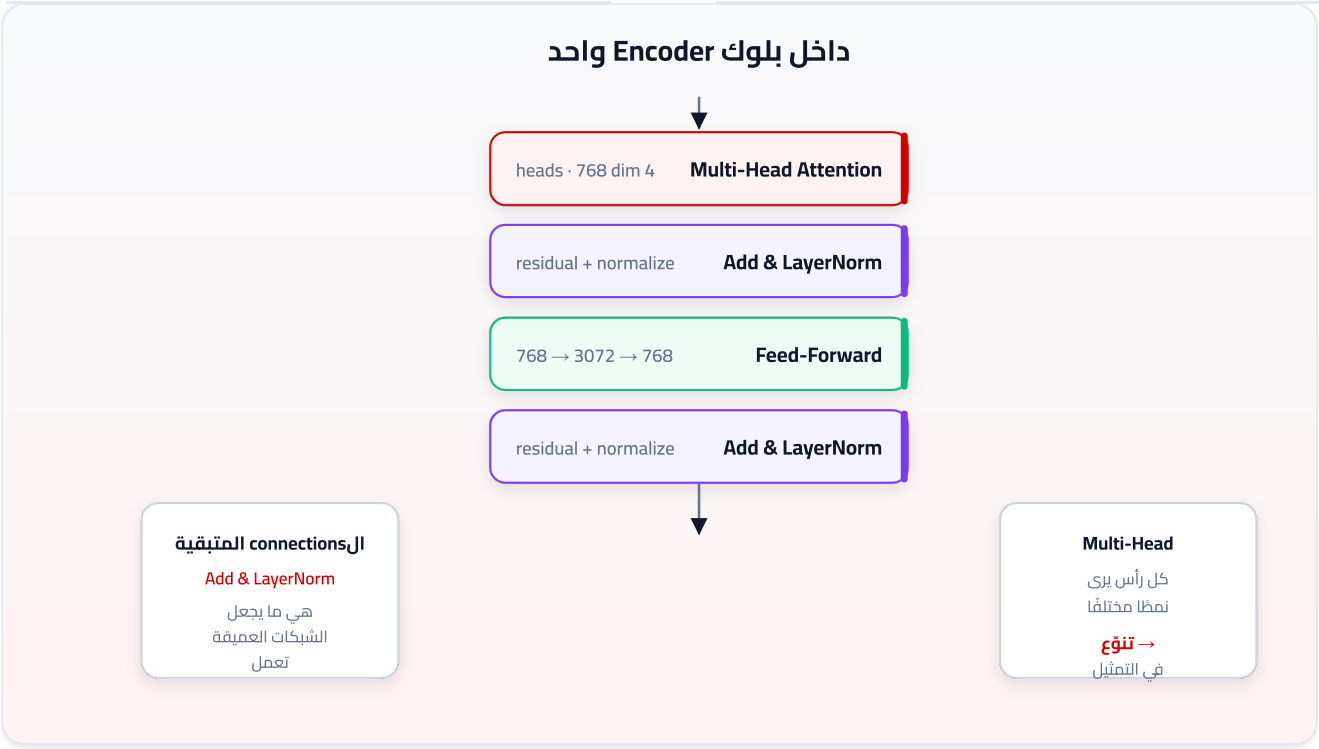
رأس واحد يُعطي اهتمامًا واحدًا. الرؤوس المتعدّدة تُعطي أنماطًا مختلفة في الوقت نفسه: رأس يرى التسلسل الزمنيّ، ورأس يرى الاعتماد النحويّ، ورأس يرى الارتباط الدلاليّ:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

حيث h عدد الرؤوس، و W_i^Q, W_i^K, W_i^V مصفوفات لكلّ رأس، و W^O مصفوفة الدمج.

29.3 كتلة المحوّل



شكل 39 – بلوك المحوّل القياسي

كلّ كتلة محوّل تتكوّن من مرحلتين، لكلّ واحدة تطبيع طبقة واتصال تخطّي:

$$x \leftarrow x + \mathrm{MSA}(\mathrm{LN}(x)), \qquad x \leftarrow x + \mathrm{MLP}(\mathrm{LN}(x))$$

حيث LN تطبيع طبقة، وMSA انتباه متعدّد الرؤوس، وMLP شبكة بمسارين (توسيع ثم انكماش).

29.4 التنفيذ في Keras

فيما يلي كتلة محوّل كاملة على تسلسل صغير:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. إعدادات صغيرة
VOCAB = 500
SEQ_LEN = 20
D_MODEL = 64
NUM_HEADS = 4
NUM_BLOCKS = 2

# ── 2. كتلة المحوّل
def transformer_block(x, num_heads, d_model, dropout=0.1):
    """انتباه: كتلة محوّل + MLP + اتصالات تخطّي + تطبيع."""
    # مرحلة الانتباه
    y = keras.layers.LayerNormalization(epsilon=1e-6)(x)
    y = keras.layers.MultiHeadAttention(
        num_heads=num_heads, key_dim=d_model // num_heads,
        dropout=dropout)(y, y)
    x = keras.layers.Add()([x, y])
    # مرحلة MLP
    y = keras.layers.LayerNormalization(epsilon=1e-6)(x)
    y = keras.layers.Dense(d_model * 2, activation="gelu")(y)
    y = keras.layers.Dense(d_model)(y)
    y = keras.layers.Dropout(dropout)(y)
    return keras.layers.Add()([x, y])

# ── 3. النموذج: تضمين + ترميز موضعيّ + كتل محوّل
inp = keras.Input(shape=(SEQ_LEN,))
x = keras.layers.Embedding(VOCAB, D_MODEL)(inp)              # (B, L, D)
pos = keras.layers.Embedding(SEQ_LEN, D_MODEL)(
    keras.ops.arange(SEQ_LEN))
x = x + pos                                                  # ترميز موضعيّ
for _ in range(NUM_BLOCKS):
    x = transformer_block(x, NUM_HEADS, D_MODEL)
x = keras.layers.LayerNormalization(epsilon=1e-6)(x)
x = keras.layers.GlobalAveragePooling1D()(x)
```

```
out = keras.layers.Dense(2, activation="softmax")(x)
model = keras.Model(inp, out)

# ── 4. تدريب على بيانات مُصنّعة
n = 500
x_data = np.random.randint(0, VOCAB, size=(n, SEQ_LEN)).astype("int32")
y_data = np.random.randint(0, 2, size=(n,)).astype("int32")
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(x_data, y_data, epochs=10, batch_size=32,
          validation_split=0.2, verbose=0)
# النتيجة المتوقعة: دقّة ≈ 0.65-0.55 على بيانات عشوائية.
```

29.5 القناع السببيّ (Causal Masking)

في فكّ الترميز (مثل توليد نصّ)، الرمز عند الموضع t لا يجب أن يرى الرموز بعد t – وإلّا فسيتعلّم النموذج «غشًّا» بالاطّلاع على المستقبل. القناع السببيّ يُطبّق عتبة على شبكة الانتباه:

$$\text{Score}(i, j) = \begin{cases} \infty & j \le i \\ -\infty & j > i \end{cases}$$

حيث i موضع الرمز السائل وj موضع الرمز المرشَّح. عند تطبيق softmax، المواضع $j > i$ تُصبح صفرًا – فلا تُنتبه إليها.

PYTHON

```
# في Keras 3، يُمرَّر القناع عبر use_causal_mask=True
x = keras.layers.MultiHeadAttention(
    num_heads=4, key_dim=16)(query, value, use_causal_mask=True)
```

> **إنذار:** بلا قناع سببيّ، النموذج يرى المستقبل أثناء التدريب ويفشل في التوليد. هذا الخطأ شائع جدًا في أول بناء فكّ ترميز.

29.6 Encoder و Decoder

الترميز (Encoder) يُبنى على تسلسل المُدخل، وفكّ الترميز (Decoder) يُبنى على تسلسل المُخرَج مع انتباه تقاطعيّ (cross-attention) على مُخرَج encoder.

النوع	ماذا يربط	متى يُستخدم
الانتباه الذاتيّ	الرموز في نفس التسلسل	encoder و decoder
الانتباه التقاطعيّ	المُخرَج (decoder) مع مُخرَج encoder	فكّ الترميز في الترجمة
الانتباه السببيّ	فقط الرموز قبل الموضع الحاليّ	decoder لتوليد النصّ

29.7 مثال محلول: تصنيف جُمَل بطول 200 كلمة

المهمة: تصنيف مراجعات العملاء الطويلة (200 كلمة) إلى إيجابية/سلبية. لديك 8000 مراجعة.

الخطوة 1 – LSTM. `Embedding(10000, 64)` → `LSTM(64)` → Dense. دقّة = 82%.

الخطوة 2 – محوّل مصغّر. تضمين 64 → 2 كتل → متوسّط → Dense. دقّة = 85%. **مكسب +3%.**

الخطوة 3 – 4 كتل. دقّة = 87%. زمن التدريب تضاعف.

الخطوة 4 – الترميز الموضعيّ الجيبيّ. استبدلنا الترميز المتعلَّم بجيبيّ. دقّة = 87.5% – مكسب بسيط.

الخطوة 5 – القرار. 4 كتل مع ترميز متعلَّم: أداء كافٍ وكلفة مقبولة. في بيانات كثيرة (مئات آلاف النصوص)، محوّل أعمق يستحقّ الكلفة.

29.8 أثر عدد الرؤوس وحجم التضمين

حجم التضمين	عدد الرؤوس	عدد المعاملات تقريبًا	ملاحظة
64	4	40 ألف	تسلسل قصير، مهمّة بسيطة
128	8	160 ألف	مهمّة متوسّطة
256	8	650 ألف	مهمّة معقّدة
768	12	7 ملايين	نموذج لغويّ صغير

قاعدة: `key_dim = projection_dim // num_heads` . مثال: تضمين 128 ورؤوس 8 ← مفتاح 16 لكلّ رأس.

القناع السببيّ عمليًّا

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. انتباه بلا قناع: كلّ رمز يرى كلّ الرموز
L, D = 6, 32
x = np.random.randn(1, L, D).astype("float32")
mha = keras.layers.MultiHeadAttention(num_heads=2, key_dim=16)
out_open = mha(x, x, use_causal_mask=False)

# ── 2. انتباه بقناع سببيّ: الرمز i يرى فقط الرموز ≤ i
out_masked = mha(x, x, use_causal_mask=True)

# ── 3. التحقّق: أول رمز لا يتغيّر (لا يرى المستقبل)
print("أول رمز بلا قناع :", np.round(keras.ops.convert_to_numpy(out_open)[0, 0,
:3], 3))
print("أول رمز بقناع    :", np.round(keras.ops.convert_to_numpy(out_masked)[0,
0, :3], 3))
# النتيجة المتوقعة: في القناع السببيّ، مُخرَج الرمز الأول يعتمد على نفسه فقط
# — وهذا شرط صحيح في فكّ الترميز لتوليد النصّ.
```

ملاحظة: `use_causal_mask=True` واجهة جاهزة في Keras 3 — لا تحتاج بناء مصفوفة قناع يدويًّا. في فكّ الترميز، لا تُولّد بلا هذا القناع؛ وإلّا يتعلّم النموذج من المستقبل ويفشل عند التوليد.

29.9 أخطاء شائعة في هذا الفصل

إنذار: إسقاط الترميز الموضعيّ. المحوّل بلا ترميز موضعيّ يرى الرموز كمجموعة بلا ترتيب، فيفقد البنية اللغويّة. الدقّة تهبط 10–20%.

تحذير: بلا قناع سببيّ في decoder. النموذج يرى المستقبل أثناء التدريب، ويتعطّل عند التوليد. استخدم `use_causal_mask=True` .

تنبيه: `key_dim = projection_dim` كاملًا. هذا يضاعف المعاملات بلا فائدة. القاعدة: `key_dim = projection_dim // num_heads` .

ملاحظة: تمرير `query` و `value` بأبعاد مختلفة مع الانتباه التقاطعيّ. هذا ممكن، لكنّ `key` و `value` يجب أن يكونا في نفس التسلسل (مثلًا مُخرَج encoder).

قاعدة: كلّ كتلة محوّل تتطلّب تطبيعًا واتصالًا تخطّيًّا. بلاها، التدرّجات تعاني والشبكة لا تتقارب.

تحذير: `Dropout` في كتلة الانتباه على بيانات صغيرة يُعطّل التعلّم. ابدأ بـ 0.0 وارفعه إلى 0.1 فقط إن كانت البيانات كافية.

التمارين

1. **حساب.** تسلسل من 100 رمز. كم عملية انتباه؟ (المتوقع: $100^2 = 10{,}000$.)
2. **حساب.** تسلسل من 500 رمز. كم عملية انتباه؟ (المتوقع: $500^2 = 250{,}000$ – أي 25 ضعفًا مقارنة بـ 100 رمز.)
3. **حساب.** تضمين 128 ورؤوس 8. ما `key_dim` لكلّ رأس؟ (المتوقع: $128/8 = 16$.)
4. **حساب.** كتلة محوّل بتضمين 256 وMLP بتوسيع ×4. كم معاملًا تقريبًا؟ (المتوقع: انتباه $4 \cdot 256 \cdot 256 = 262{,}144$ + MLP $256 \cdot 1024 + 1024 \cdot 256 = 524{,}288 \approx 800$ ألف.)
5. **برمجة.** ابنِ كتلة محوّل واطبع `model.summary()`. (المتوقع: طبقة انتباه بمصفوفات W^Q, W^K, W^V, W^O.)
6. **برمجة.** قارن عدد المعاملات بين رأس واحد و8 رؤوس على نفس التضمين. (المتوقع: نفس العدد تقريبًا – الرؤوس تُقسّم البُعد لا تُضاعفه.)
7. **برمجة.** طبّق `use_causal_mask=True` واطبع شبكة الانتباه. (المتوقع: المصفوفة مثلثيّة – صفر في الأعلى.)
8. **إصلاح خطأ.** النموذج التالي يعطي دقّة عشوائيّة:

```python
x = keras.layers.Embedding(VOCAB, D_MODEL)(inp)
for _ in range(2):
    x = transformer_block(x, NUM_HEADS, D_MODEL)
```

(المتوقع: ينقصه الترميز الموضعيّ – أضِف `Embedding(SEQ_LEN, D_MODEL)` على أرقام المواضع.) 9. **تصميم.** نظام ترجمة عربيّ→إنجليزيّ. أيّ انتباه في encoder وفي decoder؟ (المتوقع: encoder – انتباه ذاتيّ؛ decoder – انتباه ذاتيّ سببيّ + انتباه تقاطعيّ على مُخرَج encoder.) 10. **تحليل.** لماذا تُستخدم قيمة $\sqrt{d_k}$ في المقام؟ (المتوقع: لأنّ الجداء النقاطيّ يكبر مع d_k، فتصل الدرجات إلى نطاق يُشبّع softmax – القسمة تُقلّصها.) 11. **برمجة.** قارن زمن دورة تدريب بين LSTM وكتلتين محوّل على نفس التسلسل. (المتوقع: المحوّل أبطأ على CPU وLSTM أبطأ على GPU في

تسلسلات قصيرة.) 12. **تصميم.** نظام توليد نصّ عربيّ. كم كتلة، وكم رأسًا، وكم تضمينًا؟ (المتوقع: 4-8 كتل، 8 رؤوس، تضمين 256-512 – بحسب البيانات والموارد.)

ملخص الفصل

- آلية الانتباه تربط كلّ رمز بكلّ الرموز: $O(n^2)$.
- $\sqrt{d_k}$ يُقلّص الدرجات قبل softmax لتفادي التشبّع.
- الانتباه متعدّد الرؤوس يُعطي أنماطًا مختلفة في الوقت نفسه.
- كلّ كتلة محوّل = انتباه + MLP + تطبيع + اتصال تخطّي.
- الترميز الموضعيّ إلزاميّ – بلاه يفقد النموذج البنية.
- القناع السببيّ في decoder يمنع النظر إلى المستقبل.
- `key_dim = projection_dim // num_heads` – لا تضاعف المعاملات بلا فائدة.

الفصل 30: منظومة KerasHub وKerasNLP

بطاقة الفصل

الفصل 30

مهمّة هذا الفصل

يستخدم نماذج مُدرَّبة مسبقًا من KerasHub بثلاث طرق، ويضبطها على بياناتك.

ماذا تكسب منه

تصل إلى دقّة النماذج الكبيرة في سطور، مع تحكّم واضح في الكلفة والوقت.

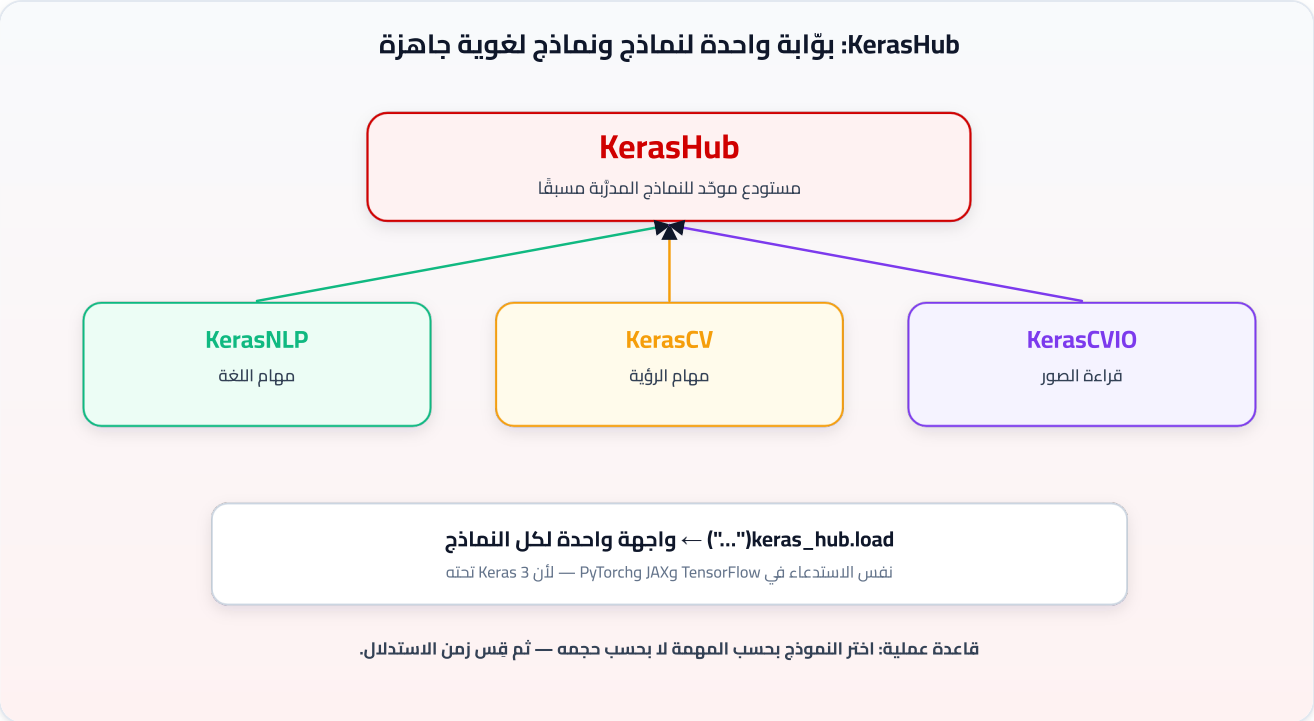
الدالة	دورها	متى تستخدمها
`keras_hub.models.BertBackbone.from_preset()`	تحميل هيكل جاهز	لبدء مهمّة جديدة
`keras_hub.models.BertTextClassifier.from_preset()`	مصنّف جاهز	لتصنيف سريع بلا تدريب
`model.fit()`	ضبط دقيق على بياناتك	بعد تجميد جزء من الهيكل
`keras.layers.GlobalAveragePooling1D()`	تجميع	عند بناء رأس مخصّص
`keras.optimizers.Adam()`	معدّل منخفض	في الضبط الدقيق

قصة مفتاحية: فريق يحتّاج نظام تصنيف مشاعر عربيّ. من الصفر، تحتاج مئات آلاف النصوص الموسومة وأسابيع من التدريب. لكنّ وحدة (checkpoint) مدرَّبة مسبقًا على مليارات الكلمات العربية موجودة بالفعل في مستودع KerasHub – وما عليك إلا تحميلها وتدريب رأسٍ صغير على 500 مرجعية موسومة. هذا الفصل هو البوّابة إلى النماذج الجاهزة: كيف تحمّلها، وكيف تُكيّفها، وكيف تُنشرها.

أهداف الفصل

- [ ] تفهم بنية KerasHub ودوره في منظومة Keras 3
- [ ] تُميّز بين loading نموذج كامل وloading backbone فقط
- [ ] تبني نموذج تصنيف فوق backbone مُدرَّب مسبقًا
- [ ] تُطبّق الضبط الدقيق (fine-tuning) بطريقة صحيحة
- [ ] تُقارن استخدام النموذج الجاهز مع النموذج المُدرَّب من الصفر
- [ ] تعرف حدود النماذج الجاهزة ومتى تُناسب مهمّتك

30.1 KerasHub في المنظومة



شكل 40 – منظومة KerasHub وKerasNLP

KerasHub هو مستودع موحّد للنماذج المُدرَّبة مسبقًا، يدعم ثلاث محرّكات (TensorFlow، JAX، PyTorch) وواجهة واحدة. من التنظيم الأهمّ:

- **KerasNLP** أصبح جزءًا من KerasHub – المستودع `keras-team/keras-nlp` لم يعد مستقلًّا.

- **keras-cv** أُرشِيف (archived) – ونُقلت قدراته إلى نواة Keras.

المستودع يوفّر ثلاث مستويات:

1. **النموذج الكامل** – مُدرَّب على مهمّة محدّدة (مثل تصنيف).
2. **العمود الفقري (backbone)** – مُدرَّب على مهمّة عامّة (مثل التنبّؤ بالرمز التالي).
3. **المُرمِّز (tokenizer)** – يحوّل النصّ إلى رموز.

30.2 تحميل نموذج

في Keras 3، تُحمَّل النماذج عبر `keras_hub.models` :

```python
import keras_hub
import keras

# مُدرَّب مسبقًا (لا تنزّل في بيئة معزولة) تحميل backbone
backbone = keras_hub.models.BertBackbone.from_preset("bert_base_en_uncased")
# أو تحميل النموذج الكامل مع رأس تصنيف
classifier = keras_hub.models.BertTextClassifier.from_preset(
    "bert_base_en_uncased", num_classes=2)
```

تحذير: `from_preset` تنزّل الأوزان من الإنترنت في المرة الأولى. في بيئة معزولة، مرّر مسارًا محليًّا إلى ملفّ الأوزان، أو استخدم `from_preset(path)` بمجلّد الأوزان المنزّلة مسبقًا.

30.3 ثلاث طرق للاستخدام

الطريقة	ماذا تحمّل	متى تستخدمها
نموذج كامل مُدرَّب	backbone + رأس مُدرَّب	المهمّة مطابقة للمهمّة المُدرَّب عليها
backbone + رأس جديد	backbone فقط	مهمّة مختلفة، بيانات قليلة
backbone + ضبط دقيق	backbone قابل للتحديث	بيانات كافية، مهامّ معقّدة

30.4 تصنيف نصّ فوق backbone

فيما يلي نموذج تصنيف فوق backbone مُدرَّب مسبقًا (المفاهيم صالحة لكلّ محرّك):

PYTHON

```python
import keras

# ── 1. backbone (استخراج ميزات) مُجمّد
backbone = keras_hub.models.BertBackbone.from_preset("bert_base_en_uncased")
backbone.trainable = False

# ── 2. النموذج: نصّ → backbone → رأس تصنيف
inp = keras.Input(shape=(1,), dtype="string")
# النموذج الكامل يتولّى المُرمِّز داخليًّا؛ في KerasHub،
# إن بُنيت يدويًّا، مرّر معرّفات الرموز مباشرةً:
x = backbone(inp)["sequence_output"]      # (B, L, hidden)
x = keras.layers.GlobalAveragePooling1D()(x)
x = keras.layers.Dense(32, activation="relu")(x)
out = keras.layers.Dense(2, activation="softmax")(x)
model = keras.Model(inp, out)

# ── 3. تدريب الرأس فقط
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

> **ملاحظة:** الطريقة الدقيقة تعتمد على النموذج: بعض الوحدات تُقبل نصًّا مباشرةً، وبعضها يتطلّب معرّفات رموز مُعالجة مسبقًا. راجع توثيق KerasHub للوحدة التي تُستخدم.

30.5 الضبط الدقيق (Fine-Tuning)

الضبط الدقيق يعني فكّ تجميد backbone وتدريب النموذج الكامل بمعدّل تعلّم منخفض جدًّا:

```python
# مرحلة 1: تدريب الرأس فقط (backbone مجمّد)
backbone.trainable = False
model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")

# مرحلة 2: فكّ تجميد backbone بمعدّل تعلّم أدنى بكثير
backbone.trainable = True
model.compile(optimizer=keras.optimizers.Adam(1e-5), loss="mse")
```

> **قاعدة معدّل التعلّم:** الرأس الجديد يبدأ من 10^{-3}، والbackbone المفكوك يبدأ من 10^{-5}. هذا هو معدّل التعلّم التمييزيّ (راجع الفصل 21).

30.6 مثال محلول: تصنيف مراجعات عربية

المهمة: تصنيف مراجعات عملاء عربية إلى إيجابية/سلبية. لديك 1000 مرجعية موسومة.

الخطوة 1 – من الصفر. تضمين مُدرَّب + `Bidirectional(LSTM(64))` . دقّة = 71% على 1000 مرجعية.

الخطوة 2 – backbone مُجمّد. `BERT` العربيّ مجمّد + رأس تصنيف. دقّة = 86%. **مكسب +15%.**

الخطوة 3 – ضبط دقيق جزئي. فكّ آخر طبقتين بمعدّل 10^{-5}. دقّة = 89%. **مكسب +3%.**

الخطوة 4 – ضبط دقيق كامل. فكّ كلّ backbone بمعدّل 10^{-5}. دقّة = 90%. زمن التدريب تضاعف.

الخطوة 5 – القرار. ضبط دقيق جزئيّ يُعطي 89% بكلفة مقبولة – هو ما يُعتمد.

30.7 جدول القرار

البيانات	النهج الموصى به
< 500 مرجعية	backbone مجمّد + رأس صغير
5000-500	backbone مجمّد ثم ضبط دقيق جزئيّ
5000 <	backbone مجمّد ثم ضبط دقيق كامل
مهمّة تختلف جذريًّا	نموذج مُدرَّب على مجال مشابه أو بيانات أكثر

30.8 حدود النماذج الجاهزة

- **السياق:** نموذج مُدرَّب على نصّ إنجليزيّ قد لا يعمل على نصّ عربيّ – راجع وحدة النموذج.
- **الطول:** بعض النماذج تُقبل تسلسلات بحدٍّ أقصى (مثل 512 رمزًا).
- **الخصوصيّة:** نموذج مُدرَّب على بيانات عامّة قد يُنتج تنبّؤات تُعكس بيانات حسّاسة.
- **الكلفة:** نموذج كبير يعطي دقّة أعلى لكن زمن استدلال أطول.

قياس كلفة backbone قبل الالتزام

قبل أن تختار نموذجًا جاهزًا، قِس ثلاثة أرقام – المعاملات، وزمن الاستدلال، والحجم:

```python
import keras
import numpy as np
import time

def probe_model(build_fn, input_dim=16, n=100):
    """يُعطي معاملات وزمن استدلال وحجم أوزان."""
    m = build_fn()
    params = m.count_params()
    x = np.random.rand(n, input_dim).astype("float32")
    m.predict(x, verbose=0)                       # warm-up
    t0 = time.perf_counter()
    m.predict(x, verbose=0)
    ms = (time.perf_counter() - t0) / n * 1000
    return {"params": params, "ms_per_sample": ms}

def build_small():
    return keras.Sequential([keras.Input(shape=(16,)),
                             keras.layers.Dense(32, activation="relu"),
                             keras.layers.Dense(1)])

def build_large():
    return keras.Sequential([keras.Input(shape=(16,)),
                             keras.layers.Dense(256, activation="relu"),
                             keras.layers.Dense(128, activation="relu"),
                             keras.layers.Dense(1)])

for name, fn in (("صغير", build_small), ("كبير", build_large)):
    r = probe_model(fn)
    print(f"{name}: {r['params']:,} params · {r['ms_per_sample']:.3f} ms/عيّنة")
# النتيجة المتوقعة: النموذج الكبير أبطأ بعدد أضعاف نسبة معاملاته —
# راجع هذا الرقم قبل اختيار backbone في الخدمة.
```

نصيحة: قبل أن تُحمّل backbone بحجم مئات ملايين المعاملات، احسب زمن الاستدلال على جهاز النشر الفعليّ — نموذج ممتاز لا يصلح لخدمة بمتطلّب 50 ms.

فكّ التجميد المتدرّج

الفكّ لا يكون دفعة واحدة – بل طبقة بطبقة مع قياس:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. نموذج محاكاة backbone + رأس
backbone = keras.Sequential([keras.Input(shape=(16,)),
                             keras.layers.Dense(32, activation="relu"),
                             keras.layers.Dense(32, activation="relu")])
inp = keras.Input(shape=(16,))
x = backbone(inp)
out = keras.layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inp, out)
model.compile(optimizer="adam", loss="mse")

# ── 2. قياس قابلية التدريب في كلّ حالة
def trainable_report(m):
    return (sum(int(np.prod(w.shape)) for w in m.trainable_weights),
            sum(int(np.prod(w.shape)) for w in m.non_trainable_weights))

# حالة 1: مجمّد كاملًا
backbone.trainable = False
print("مجمّد كاملًا:", trainable_report(model))

# حالة 2: فكّ آخر طبقة فقط
backbone.trainable = False
backbone.layers[-1].trainable = True
print("آخر طبقة:", trainable_report(model))

# حالة 3: مفكوك كاملًا
backbone.trainable = True
print("مفكوك كاملًا:", trainable_report(model))
# النتيجة المتوقعة: معاملات قابلة للتدريب تتدرّج من صفر
# إلى الحجم الكامل – وهذا هو مسار الضبط الدقيق الجزئيّ.
```

قاعدة التدرّج: درّب الرأس → قِس → فكّ آخر طبقة → قِس → فكّ كتلة → قِس. توقّف عند أوّل مستوى يُحقّق الدقّة المطلوبة – لا تفتح كلّ شيء افتراضيًّا.

30.9 أخطاء شائعة في هذا الفصل

إنذار: فكّ تجميد backbone قبل تدريب الرأس. الأوزان المُدرَّبة مسبقًا تتدمّر بمعدّل تعلّم مرتفع. دائمًا: درّب الرأس أوّلًا، ثم فكّ.

تحذير: `from_preset` في بيئة بلا اتصال. النتيجة: خطأ شبكة. حلّها: نزّل الأوزان مسبقًا ومرّر مسارًا محليًّا.

تنبيه: تجاهل حدّ طول التسلسل. نموذج BERT يُقبل 512 رمزًا كحدٍّ أقصى – النصّ الأطول يُقطّع أو يُفشل.

ملاحظة: `backbone.trainable = False` لا يمنع الحساب – بل يمنع تحديث الأوزان. لتوفير زمن فعليّ، مرّر `training=False` عند نداء backbone.

قاعدة: لا تُقارن نموذجًا جاهزًا مع نموذج من الصفر على بيانات قليلة. النموذج الجاهز سيفوّز – هذا ليس دليلًا على جودته، بل على قلّة بياناتك.

تحذير: استخدام نموذج مُدرَّب على مجال مختلف جذريًّا. مثال: نموذج مُدرَّب على نصّ طبيّ قد لا يعمل على نصّ ماليّ.

التمارين

1. **حساب.** backbone بحجم 110 مليون معامل، ورأس تصنيف بحجم 100 ألف. كم نسبة الرأس من الكلّ؟ (المتوقع: $\frac{100{,}000}{110{,}000{,}000} \approx 0.09\%$.)

2. **حساب.** إذا كان حدّ طول التسلسل 512 رمزًا والنصّ 800 كلمة، فكم رمزًا يُقَطَّع؟ (المتوقع: 288 رمزًا – ما بعد 512 يُهمل.)

3. **برمجة.** حاول تحميل backbone مُدرَّب مسبقًا واطبع عدد معاملاته. (المتوقع: ملايين المعاملات – أوّل تشغيل يحتاج اتصالًا.)

4. **برمجة.** اطبع عدد المعاملات القابلة للتدريب قبل وبعد `backbone.trainable = True`. (المتوقع: بعد الفكّ يرتفع إلى حجم backbone كامل.)

5. **برمجة.** قارن زمن دورة تدريب بين backbone مجمّد ومفكوك. (المتوقع: الفكّ يُبطئ التدريب 2-3×.)

6. **تصميم.** نظام تصنيف مشاعر عربيّ بـ 300 مرجعية. أيّ نهج؟ (المتوقع: backbone مجمّد + رأس صغير – بيانات قليلة جدًا لا تُكفي لضبط دقيق.)

7. **تصميم.** نظام تصنيف مع 20,000 مرجعية. أيّ نهج؟ (المتوقع: backbone مجمّد ثم ضبط دقيق كامل – البيانات كافية.)

8. **إصلاح خطأ.** النموذج التالي يُفشل في الضبط الدقيق:

```python
backbone.trainable = True
model.compile(optimizer=keras.optimizers.Adam(1e-3), ...)
```

(المتوقع: معدّل تعلّم 1e-3 مرتفع جدًا على أوزان مُدرَّبة مسبقًا – استخدم 10^{-5}.) 9. **تحليل.** لماذا لا يُقارن نموذج جاهز مع نموذج من الصفر على بيانات قليلة؟ (المتوقع: لأنّ النموذج الجاهز يحمل معلومة من ملايين العيّنات، والنموذج من الصفر لا يملك هذه المعلومة.) 10. **تصميم.** نظام تصنيف نصّ ماليّ عربيّ. أيّ وحدة تختار؟ (المتوقع: وحدة مُدرَّبة على نصّ عربيّ أو ماليّ – لا وحدة إنجليزيّة عامّة.) 11. **برمجة.** طبّق `backbone.trainable = False` واطبع

`model.summary(show_trainable=True)` . (المتوقع: طبقات backbone تُظهر `n` (غير قابلة للتدريب).) 12. **تحليل.** لماذا يُرفض الضبط الدقيق الكامل في بيانات قليلة جدًّا؟ (المتوقع: النموذج يُفرِط في التخصيص – ملايين المعاملات تُدرَّب على مئات العيّنات.)

ملخص الفصل

- KerasHub هو مستودع موحّد للنماذج المُدرَّبة مسبقًا، بثلاث محرّكات وواجهة واحدة.
- ثلاث طرق: نموذج كامل، backbone + رأس، أو backbone + ضبط دقيق.
- معدّل التعلّم التمييزيّ: 10^{-3} للرأس، 10^{-5} للbackbone المفكوك.
- النموذج الجاهز يُنجز 15-20% دقّة إضافيّة على بيانات قليلة.
- ضبط دقيق جزئيّ هو أفضل توازن بين الدقّة والكلفة في معظم الحالات.
- راجع حدّ طول التسلسل، ومجال تدريب النموذج، وخصوصيّة بياناتك.

الجزء السابع

النماذج التوليدية: من الترميز إلى الانتشار

النماذج التوليدية: الترميز الآلي المتغيّر وفضاؤه الاحتمالي، الشبكات التنافسية ومشكلة انهيار الأنماط، ونماذج الانتشار بمسارَيها الأمامي والعكسي.

ما تكسبه من هذا الجزء

تولّد بيانات جديدة، وتكشف الشواذ بلا تسميات، وتعرف كيف تختار عائلة توليدية بحسب المهمّة والكلفة.

فصول هذا الجزء

31 الترميز الآلي المتغيّر (VAE)

32 شبكات التوليد التنافسيّة (GAN)

33 نماذج الانتشار (Diffusion Models)

7

الفصل 31: الترميز الآلي المتغيّر (VAE)

بطاقة الفصل

الفصل 31

مهمّة هذا الفصل

يبني ترميزًا آليًّا احتماليًّا: فضاء كامن، حيلة إعادة التوزيع، وخسارة ELBO.

ماذا تكسب منه

تولّد عيّنات جديدة وتكشف الشواذ بلا تسميات، وتفهم الأساس الذي تقوم عليه النماذج التوليدية.

الدالة	دورها	متى تستخدمها
`keras.layers.Dense()`	متوسّط وتباين الفضاء الكامن	في المُرمِّز
`keras.ops.exp()`	حساب حدّ KL	في دالّة الخسارة
`keras.random.normal()`	عيّنة إعادة التوزيع	بين المُرمِّز وفكّ الترميز
`keras.layers.Conv2DTranspose()`	فكّ ترميز الصور	في مفكّك الترميز
`keras.metrics.Mean()`	متوسّط الخسارة	لمراقبة التدريب

قصة مفتاحية: مصنع يحتّاج نظامًا يكتشف عيوب في القطع المعدنيّة من صورها. لديه 10,000 صورة سليمة و50 صورة معيبة فقط. التصنيف التقليدي لا يعمل – فالبيانات غير متوازنة بشكلٍ حادّ. الحلّ: **الترميز الآلي** يتعلم بنية «الصورة السليمة» فقط، ثم يقيس مقدار ما يستطيع إعادة بنائه. صورة معيبة تفشل في إعادة البناء، فيُكتشف العيب من درجة الإصلاح – لا من تصنيفها.

أهداف الفصل

- [ ] تشرح فرق المُرمِّز الآليّ التقليديّ عن المُتغيّر (VAE)
- [ ] تكتب معادلة الخسارة الكاملة (إعادة بناء + KL)
- [ ] تفهم حيلة إعادة التوزيع (reparameterization trick)
- [ ] تبني VAE كاملًا في Keras
- [ ] تستخدم الـ VAE في كشف الشواذ
- [ ] تُقارن VAE مع الشبكات التنافسيّة والانتشارية

31.1 المُرمِّز الآليّ التقليديّ

شكل 41 – بنية المُرمِّز الآليّ المتغيّر

المُرمِّز الآليّ (autoencoder) يتكوّن من مسارين: **مُرمِّز (encoder)** يضغط الصورة إلى فضاء كمّيّ صغير، و**مُفكّك (decoder)** يعيد بناء الصورة من هذا الفضاء:

$$z = f_\theta(x), \qquad \hat{x} = g_\phi(z)$$

حيث f_θ المُرمِّز، وg_ϕ المُفكّك، وz تمثيل كمّيّ. التدريب يُقلّل فرق x و$\hat{x}$:

$$L = \|x - \hat{x}\|^2$$

المشكلة: هذا النموذج **لا يُولّد**. فضاء z قد يحتوي فجوات لا يُمكن للمُفكّك بناء صورة معقولة منها. الـ VAE يحلّ هذا بإجبار الفضاء أن يكون **توزيعًا احتماليًّا**.

31.2 الـ VAE: فضاء احتماليّ

الـ VAE يُعطي لكلّ صورة x **توزيعًا** في الفضاء الكمّيّ – لا نقطة واحدة. المُرمِّز يُعطي متوسّطًا μ وتباينًا σ^2:

$$q_\theta(z|x) = \mathcal{N}(z;\ \mu(x),\ \sigma^2(x))$$

التدريب يُقلّل خسارة من جزأين:

$$\mathcal{L} = \underbrace{\mathbb{E}_{q(z|x)}[\log p_\phi(x|z)]}_{\text{إعادة بناء}} - \underbrace{D_{KL}(q(z|x)\ \|p(z))}_{\text{قرب من الأسبق}}$$

حيث $p(z) = \mathcal{N}(0, I)$ هو التوزيع الأسبق. **إعادة البناء** تضمن أنّ الصورة تُعاد، و**تباعد KL** يضمن أنّ الفضاء كمّيّ ومتّصل.

تباعد KL في الـ VAE

حين يكون التوزيعان غاوسيّان، يُحسب تباعد KL تحليليًّا:

$$D_{KL} = -\frac{1}{2}\sum_{j=1}^{J}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

حيث J بُعد الفضاء الكمّيّ.

31.3 حيلة إعادة التوزيع

المشكلة: لا يمكن تمرير التدرّج عبر عيّنة عشوائيّة $z \sim q(z|x)$. الحيلة: أعد كتابة العيّنة كدالّة في متغيّر عشوائيّ مستقلّ:

$$z = \mu + \sigma \cdot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

حيث ϵ ضجيج عشوائيّ خارج المسار التدرّجيّ. الآن z دالّة في μ وσ (وهي قابلة للتدرّج)، وϵ عشوائيّ فقط.

31.4 التنفيذ في Keras

فيما يلي VAE كامل على صور 28×28:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. إعدادات
LATENT_DIM = 8
IMAGE_SHAPE = (28, 28, 1)

# ── 2. المُرمِّز
encoder_input = keras.Input(shape=IMAGE_SHAPE, name="x")
x = keras.layers.Conv2D(16, 3, strides=2, padding="same", activation="relu")
(encoder_input)
x = keras.layers.Conv2D(32, 3, strides=2, padding="same", activation="relu")
(x)
x = keras.layers.Flatten()(x)
mu = keras.layers.Dense(LATENT_DIM, name="mu")(x)
log_var = keras.layers.Dense(LATENT_DIM, name="log_var")(x)

# ── 3. حيلة إعادة التوزيع
def sample_z(args):
    """z = mu + sigma * epsilon، حيث sigma = exp(0.5 * log_var)."""
    mu_, log_var_ = args
    eps = keras.random.normal(shape=keras.ops.shape(mu_))
    return mu_ + keras.ops.exp(0.5 * log_var_) * eps

z = keras.layers.Lambda(sample_z)([mu, log_var])
encoder = keras.Model(encoder_input, [mu, log_var, z], name="encoder")

# ── 4. المُفكّك
decoder_input = keras.Input(shape=(LATENT_DIM,), name="z")
y = keras.layers.Dense(7 * 7 * 32, activation="relu")(decoder_input)
y = keras.layers.Reshape((7, 7, 32))(y)
y = keras.layers.Conv2DTranspose(32, 3, strides=2, padding="same",
activation="relu")(y)
y = keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same",
activation="relu")(y)
decoder_output = keras.layers.Conv2D(1, 3, padding="same",
```

```
activation="sigmoid")(y)
decoder = keras.Model(decoder_input, decoder_output, name="decoder")

# ── 5. النموذج الكامل مع خسارة مركّبة ──
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.recon_tracker = keras.metrics.Mean(name="recon")
        self.kl_tracker = keras.metrics.Mean(name="kl")

    def call(self, inputs):
        mu_, log_var_, z_ = self.encoder(inputs)
        return self.decoder(z_)

    def train_step(self, data):
        x = data[0] if isinstance(data, tuple) else data
        # Keras 3 لا يوحّد شريط التدرّج: نأخذه من المحرّك النشط
        import tensorflow as tf
        with tf.GradientTape() as tape:
            mu_, log_var_, z_ = self.encoder(x)
            recon = self.decoder(z_)
            # خسارة إعادة البناء (متوسّطًا على الدفعة)
            recon_loss = keras.ops.mean(
                keras.ops.sum(keras.ops.square(x - recon), axis=[1, 2, 3]))
            # خسارة KL
            kl_loss = -0.5 * keras.ops.mean(
                keras.ops.sum(1 + log_var_ - keras.ops.square(mu_)
                              - keras.ops.exp(log_var_), axis=-1))
            total = recon_loss + kl_loss
        grads = tape.gradient(total, self.trainable_variables)
        self.optimizer.apply_gradients(zip(grads, self.trainable_variables))
        self.recon_tracker.update_state(recon_loss)
        self.kl_tracker.update_state(kl_loss)
        return {m.name: m.result() for m in self.metrics}

# ── 6. تدريب على بيانات مُصنّعة
```

```
n = 500
x_data = np.random.rand(n, 28, 28, 1).astype("float32")
vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam(1e-3))
vae.fit(x_data, epochs=10, batch_size=32, verbose=0)
# النتيجة المتوقعة: recon و kl ينخفض مع تقدّم الدورات، يستقرّ حول قيمة محدّدة.
```

> **ملاحظة:** في Keras 3، تُبنى النماذج المخصّصة مثل VAE عبر `keras.Model` مع `train_step` مخصّص. راجع الفصل 34 لشرح الطبقات المخصّصة.

31.5 استخدام VAE في كشف الشواذ

في كشف الشواذ، نُدرّب VAE على **البيانات السليمة فقط**، ثم نُقيس خطأ إعادة البناء على عيّنات جديدة:

$$\text{Score}(x) = \|x - g_\phi(f_\theta(x))\|^2$$

- عيّنة سليمة ← خطأ إعادة بناء منخفض.
- عيّنة شاذّة ← خطأ إعادة بناء مرتفع.

> **نصيحة:** اختر عتبة الشذوذ على مجموعة تحقّق سليمة (مثلاً: المتوسّط + 3·الانحراف المعياريّ). هذه الطريقة تعمل على البيانات غير المتوازنة حيث التصنيف التقليديّ يفشل.

31.6 مثال محلول: كشف عيوب القطع

المهمة: كشف عيوب في قطع معدنيّة من صورها. لديك 10,000 صورة سليمة و50 صورة معيبة.

الخطوة 1 – تصنيف تقليديّ. نموذج تصنيف مع `class_weight`. recall للتشوّه = 0.42 – غير مقبول.

الخطوة 2 – VAE على السليم فقط. مُرمِّز → فضاء 8 أبعاد → مُفكّك. خطأ إعادة بناء متوسّط على السليم = 0.012، وعلى المعيب = 0.061.

الخطوة 3 – عتبة الشذوذ. على مجموعة تحقّق سليمة: عتبة = 0.028 (متوسّط + 3σ). recall للتشوّه = 0.82.

الخطوة 4 – تحجيم الفضاء. الفضاء الكمّيّ من 8 إلى 32 بُعدًا. recall ارتفع إلى 0.88 لكنّ عتبة الشذوذ صارت حسّاسة أكثر.

الخطوة 5 – القرار. فضاء 16 بُعدًا مع عتبة على مجموعة التحقّق – أداء كافٍ وكلفة مقبولة.

31.7 مقارنة بين نماذج التوليد

النموذج	ماذا يولّد	الجودة	السرعة	متى تستخدمه
VAE	صورة من فضاء كمّيّ	متوسّطة	سريعة	كشف شواذ، بيانات صغيرة
GAN	صورة من ضجيج	مرتفعة	سريعة	صور واقعية، بيانات كبيرة
Diffusion	صورة تدريجيًّا	ممتازة	بطيئة	أعلى جودة، موارد كافية

قاعدة: VAE أفضل للكشف والاستدلال السريع. GAN أفضل للصور الواقعية على بيانات كافية. Diffusion أفضل للجودة القصوى.

كشف الشواذ: التطبيق العمليّ

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات سليمة (تدريب) وبيانات شاذّة (اختبار)
n_normal = 400
x_normal = np.random.randn(n_normal, 28, 28, 1).astype("float32") * 0.3
x_anomaly = np.random.randn(50, 28, 28, 1).astype("float32") * 1.5

# ── 2. VAE مصغّرة: encoder + decoder
LATENT = 4
enc_in = keras.Input(shape=(28, 28, 1))
z = keras.layers.Flatten()(enc_in)
mu = keras.layers.Dense(LATENT)(z)
log_var = keras.layers.Dense(LATENT)(z)
# المشفّر يُخرج μ و log σ² ؛ سحب الضجيج يبقى خارج الرسم البياني فقط
# كي يعمل مع أي حجم دفعة (لا شكل ثابت مدفون في العقدة)
enc = keras.Model(enc_in, [mu, log_var])

dec_in = keras.Input(shape=(LATENT,))
d = keras.layers.Dense(28 * 28, activation="sigmoid")(dec_in)
d = keras.layers.Reshape((28, 28, 1))(d)
dec = keras.Model(dec_in, d)
dec.compile(optimizer="adam", loss="mse")  # المشفّر-المشفوف يُدرَّب بخسارة إعادة البناء

# ── 3. التدريب على السليم فقط
mu_n, lv_n = enc.predict(x_normal, verbose=0)
eps = np.random.default_rng(0).standard_normal(mu_n.shape).astype("float32")
z_train = mu_n + np.exp(0.5 * lv_n) * eps      # إعادة المعالجة
dec.fit(z_train, x_normal, epochs=10, batch_size=32, verbose=0)

# ── 4. درجة الشذوذ = خطأ إعادة البناء
def anomaly_score(x):
```

```
    mu_, lv_ = enc.predict(x, verbose=0)
    eps_ =
np.random.default_rng(1).standard_normal(mu_.shape).astype("float32")
    z_ = mu_ + np.exp(0.5 * lv_) * eps_
    recon = dec.predict(z_, verbose=0)
    return np.mean(np.square(x - recon), axis=(1, 2, 3))

s_normal = anomaly_score(x_normal[:100])
s_anomaly = anomaly_score(x_anomaly)
threshold = s_normal.mean() + 3 * s_normal.std()
recall = float((s_anomaly > threshold).mean())
print(f"threshold = {threshold:.4f}, recall = {recall:.2%}")
# النتيجة المتوقعة: recall مرتفع — الشواذ تفشل في إعادة البناء.
```

> **ملاحظة:** هذا مثال مصغّر يُوضّح المبدأ. في الإنتاج، تُبنى الـ VAE كما في القسم 31.4 مع خسارة مركّبة (إعادة بناء + KL)، والعتبة تُختار على مجموعة تحقّق سليمة.

اختيار العتبة على منحنى الدقّة والاستدعاء

العتبة لا تُختار بحكم – بل تُقاس:

```python
import numpy as np

def sweep_threshold(scores_normal, scores_anomaly):
    """يُجرب عتبات ويُعطي أزواج (precision, recall)."""
    results = []
    for q in (0.90, 0.95, 0.99, 0.995, 0.999):
        thr = float(np.quantile(scores_normal, q))
        tp = float((scores_anomaly > thr).mean())
        fp = float((scores_normal > thr).mean())
        prec = tp / max(tp + fp, 1e-8)
        results.append((q, thr, prec, tp))
    return results

rng = np.random.default_rng(42)
s_normal = rng.normal(0.010, 0.002, 400)
s_anomaly = rng.normal(0.045, 0.010, 60)
print(f"{'quantile':>9} {'threshold':>10} {'precision':>10} {'recall':>8}")
for q, thr, prec, rec in sweep_threshold(s_normal, s_anomaly):
    print(f"{q:>9.3f} {thr:>10.4f} {prec:>10.3f} {rec:>8.3f}")
# كلّما ارتفع quantile ارتفعت النتيجة المتوقعة: precision وانخفض recall.
# — والاختيار قرار عمل لا قرار رياضيّ.
```

العتبة (quantile)	الدقّة	الاستدعاء	متى تُختار
0.90	متوسّطة	مرتفعة	فحص أوليّ واسع
0.99	مرتفعة	متوسّطة	التوازن الأشهر
0.999	مرتفعة جدًّا	منخفضة	الإنذارات كلفة مرتفعة

قاعدة: قيمة العتبة قرار **اقتصاديّ**: كلّ إنذار كاذب يكلّف زمن فحص، وكلّ عيب مُغفَل يكلّف منتجًا معيبًا. حدّد تكلفة كلّ نوع خطأ، ثم اختر النقطة التي تُقلّل التكلفة الكليّة.

31.8 أخطاء شائعة في هذا الفصل

إنذار: استخدام `log_var` بدلًا من `var` . المعادلات تعمل على $\log \sigma^2$ – هذا يضمن أنّ σ^2 موجب دون قيود.

تحذير: إسقاط خسارة KL. النتيجة: الفضاء الكمّيّ يتفكّك إلى فجوات، والمُفكّك لا يعمل على عيّنات جديدة.

تنبيه: تمرير μ وσ مباشرةً إلى عيّنة عشوائيّة. التدرّج لا يمرّ – استخدم حيلة إعادة التوزيع.

ملاحظة: فضاء كمّيّ صغير جدًّا (مثل 2 بُعد) يعطي صورًا ضبابيّة. ابدأ بـ 8-32 بُعدًا وارفعه إن كان الخطأ في إعادة البناء مرتفعًا.

قاعدة: خسارة إعادة البناء وحدها تعطي صورًا حادّة لكنّ الفضاء لا يكون متّصلًا. خسارة KL وحدها تعطي فضاءً متّصلًا لكنّ الصورة ضبابيّة. الاثنان معًا هما الحلّ.

تحذير: استخدام VAE لتوليد صور في مهمّة طبية بوصفها «صورة حقيقية». الـ VAE يُولّد صورًا مشابهة، لا صورًا حقيقية – لا تُستخدم في دليل تشخيصيّ.

التمارين

1. **حساب.** فضاء كمّيّ بطول 16، ومتوسّط $\mu = [0.1, 0.2, \dots]$ و$\log \sigma^2 = [-1, -1, \dots]$. احسب KL. (المتوقع: $-\frac{1}{2}\sum_j (1 + (-1) - \mu_j^2 - e^{-1})$ – حساب لكلّ بُعد.)

2. **حساب.** إذا كان $\mu = 0$ و$\sigma = 1$ (أي $\log \sigma^2 = 0$)، فما KL؟ (المتوقع: صفر – التوزيع مطابق للأسبق.)

3. **حساب.** إذا كان $\mu = 1$ و$\sigma = 1$، فما KL؟ (المتوقع: $-\frac{1}{2}(1 + 0 - 1 - 1) = 0.5$ لكلّ بُعد.)

4. **برمجة.** ابنِ encoder بطبقتي `Conv2D` واطبع شكل المُخرَج. (المتوقع: نتيجة في الفضاء الكمّيّ بحجم `LATENT_DIM`.)

5. **برمجة.** قارن خطأ إعادة بناء على بيانات سليمة وبيانات مُضخّمة بالضجيج. (المتوقع: الشواذ تعطي خطأً أعلى.)

6. **برمجة.** قارن بين فضاء كمّيّ بطول 4 و32. (المتوقع: 32 يعطي خطأ إعادة بناء أقلّ.)

7. **إصلاح خطأ.** الكود التالي يعطي خطأ في التدرّج:

```python
z = keras.random.normal(shape=(LATENT_DIM,), mean=mu,
stddev=keras.ops.exp(0.5 * log_var))
```

(المتوقع: عيّنة عشوائيّة مباشرةً تُقطع التدرّج – استخدم حيلة إعادة التوزيع `mu + sigma *` 8 (. `eps`. **برمجة.** طبّق عتبة شذوذ على مجموعة تحقّق سليمة واطبع recall على مجموعة اختبار معيبة. (المتوقع: recall يتغيّر مع العتبة.) 9. **تصميم.** نظام كشف شواذ في بيانات صناعيّة. أيّ بنية، وكم بُعدًا في الفضاء؟ (المتوقع: VAE بفضاء 16-32 بُعدًا، وعتبة على مجموعة تحقّق سليمة.) 10. **تحليل.** لماذا لا تُنتج VAE صورًا حادّة كـ GAN؟ (المتوقع: خسارة إعادة البناء (MSE) تُنصّب نحو المتوسّط، فيُعطي حوافّ ضبابيّة.) 11. **برمجة.** قارن زمن الاستدلال بين VAE وGAN على 100 صورة. (المتوقع: VAE أسرع لأنّها بلا شبكة تمييز.) 12. **تصميم.** نظام كشف شواذ في صور طبيّة. أيّ خسارة إعادة بناء تختار، ولماذا؟ (المتوقع: L1 أو L2 + تطبيع – لأنّ الدقّة العدديّة حرجة، لا الواقعية البصريّة.)

ملخص الفصل

- VAE يُعطي توزيعًا احتماليًّا في الفضاء الكمّيّ، لا نقطة واحدة.
- الخسارة = إعادة بناء + تباعد KL.
- حيلة إعادة التوزيع تُمرّر التدرّج عبر العيّنة العشوائيّة: $z = \mu + \sigma \cdot \epsilon$.
- VAE يُستخدم في كشف الشواذ من خلال خطأ إعادة البناء.
- خسارة إعادة البناء وحدها تعطي صورًا حادّة، وخسارة KL وحدها تعطي فضاءً متّصلًا.
- VAE أسرع من GAN وأدقّ في كشف الشواذ، وأقلّ جودة في التوليد.

الفصل 32: شبكات التوليد التنافسيّة (GAN)

بطاقة الفصل

الفصل 32

مهمّة هذا الفصل

يصفّي مولّدًا ومميّزًا في لعبة تنافسية، ويعالج انهيار الأنماط بـWGAN-GP.

ماذا تكسب منه

تولّد بيانات اصطناعية مقنعة، وتعرف مؤشّرات الاستقرار قبل أن ينهار التدريب لا بعده.

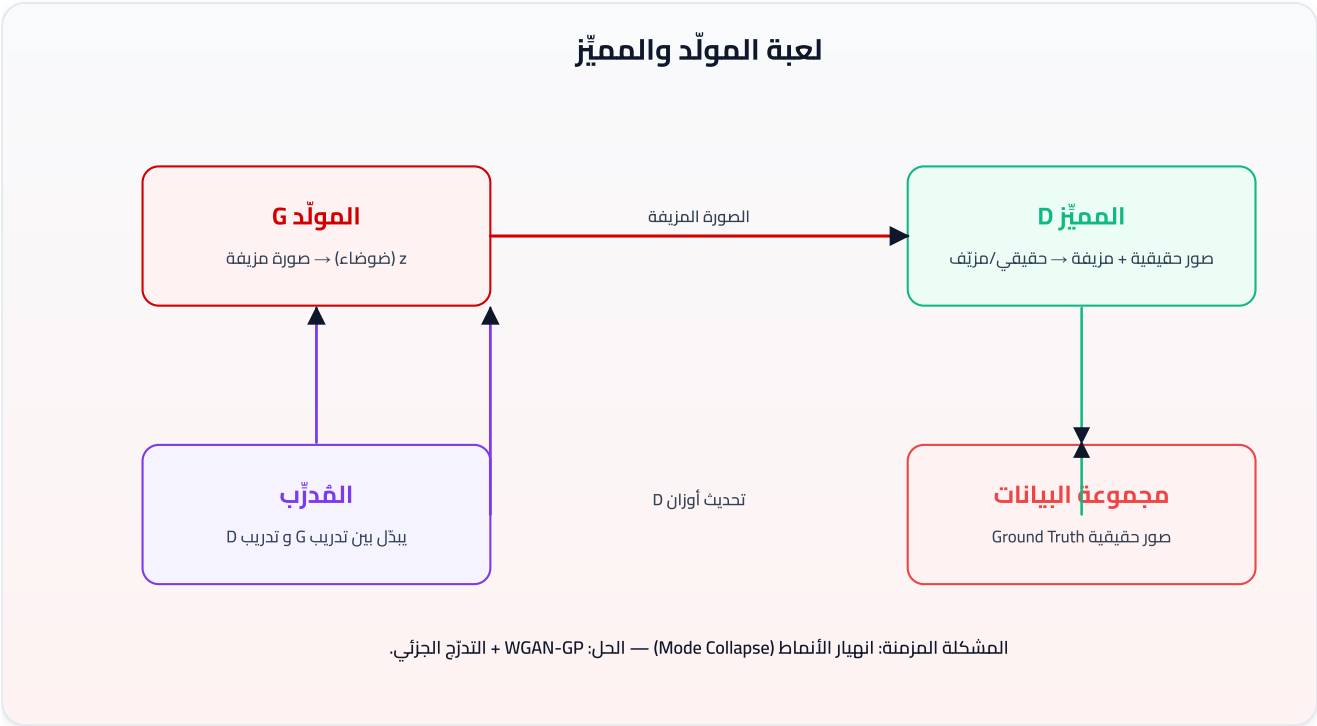
الدالة	دورها	متى تستخدمها
`keras.layers.Conv2DTranspose()`	بناء المولّد	لتوليد الصورة من ضجيج
`keras.layers.LeakyReLU()`	تنشيط المميّز	لتجنّب موت التدرّج
`keras.optimizers.Adam(beta_1=0.5)`	محسّن GAN المعتاد	لكلا الشبكتين
`keras.Model(inputs, outputs)`	بناء المولّد والمميّز	في الواجهة الوظيفية
`keras.layers.Dropout()`	تنظيم المميّز	لتقليل تفوّقه المبكّر

قصة مفتاحية: استوديو تصميم يحتّاج آلاف صور خلفيّات بألوان وإضاءات مختلفة. التصوير الفعليّ مكلف وبطيء. الشبكة التنافسيّة تحلّ هذه المشكلة بمقاربة ذكيّة: نموذجان يتنافسان – الأول (المولّد) يحاول خداع الثاني (المُميّز)، والثاني يحاول التمييز بين الصورة الحقيقيّة والمولّدة. بهذه المنافسة يتعلم المولّد إنتاج صور واقعيّة دون أن يُصوّرها فعليًّا.

أهداف الفصل

- [ ] تشرح بنية GAN ودور المولّد والمُميّز
- [ ] تكتب معادلة الخسارة التنافسيّة الكاملة
- [ ] تفهم انهيار الأنماط (mode collapse) وأسبابه
- [ ] تبني GAN بسيطة في Keras
- [ ] تُطبّق تحسينات WGAN-GP لتحقيق استقرار التدريب
- [ ] تُقارن GAN مع VAE ونماذج الانتشار

32.1 بنية GAN



شكل 42 – بنية شبكات التوليد التنافسيّة

الشبكة التنافسيّة تتكوّن من نموذجين يتنافسان:

1. المولّد (Generator) – يأخذ ضجيجًا عشوائيًّا z وينتج صورة $\hat{x}$:

$$\hat{x} = G_\theta(z), \qquad z \sim \mathcal{N}(0, I)$$

2. المُميّز (Discriminator) – يأخذ صورة (حقيقيّة أو مولّدة) ويُعطي احتمالًا:

$$D_\phi(x) \in [0, 1]$$

الهدف: أن يخدع المولّد المُميّز، وأن يميّز المُميّز الصورة الحقيقيّة من المولّدة.

32.2 معادلة الخسارة التنافسيّة

الخسارة الكلاسيكيّة (Goodfellow):

$$\min_G \max_D \; \mathbb{E}_{x \sim data} \log D(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z)))$$

- المُميّز D يُعظّم: يريد أن يعطي احتمالًا عاليًا للصور الحقيقيّة ($\log D(x)$ كبير) ومنخفضًا للمولّدة ($\log(1 - D(G(z)))$ كبير).
- المولّد G يُقلّل: يريد أن يُخدع المُميّز (يُعظّم $D(G(z))$).

في الممارسة، يُدرَّب كلّ نموذج بخسارة منفصلة:

- خسارة المُميّز: $-\log D(x) - \log(1 - D(G(z)))$
- خسارة المولّد: $-\log D(G(z))$

> **قاعدة تدريب:** كلّ خطوة تُدرّب المُميّز أولًا، ثم المولّد. تُجمّد أوزان أحدهما عند تدريب الآخر.

32.3 انهيار الأنماط (Mode Collapse)

المشكلة الأشهر في GAN: المولّد يتعلّم إنتاج **نمط واحد** أو أنماط قليلة من توزيع البيانات، ويتجاهل البقية. مثال: بيانات فيها 10 أنواع من القطط، والمولّد يُنتج نوعًا واحدًا فقط.

الأسباب: - المُميّز يهيمن بسرعة، فالمولّد يتعلّم «أعجب المُميّز» بدل «حاكي البيانات». - خسارة JS (Jensen-Shannon) في GAN الكلاسيكيّة تنهار حين تتباعد التوزيعات.

الحلول: 1. **WGAN-GP** – استبدال خسارة JS بخسارة Wasserstein مع قيود التدرّج. 2. **التنويع في المُدخلات** – زيادة بُعد الضجيج. 3. **التدريب المتوازن** – تقليل عدد خطوات المُميّز.

32.4 WGAN-GP

WGAN يستبدل خسارة JS بخسارة Wasserstein، ويُضيف **قيود التدرّج** (gradient penalty):

$$L_D = \underbrace{\mathbb{E}[\hat{x}]}_{\text{مولّد}} - \underbrace{\mathbb{E}[x]}_{\text{حقيقي}} + \lambda\, \mathbb{E}\left[(\|\nabla_x D(\hat{x})\|_2 - 1)^2\right]$$

حيث $\hat{x}$ نقطة خطّيّة بين صورة حقيقيّة وصورة مولّدة، وλ معامل القيد (10 نموذجيًّا). هذا القيد يجعل المُميّز Lipschitz، فيحفظ استقرار التدريب.

32.5 التنفيذ في Keras

فيما يلي GAN بسيطة على بيانات 28×28:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. إعدادات
LATENT_DIM = 64
IMAGE_SHAPE = (28, 28, 1)

# ── 2. المولّد
generator_input = keras.Input(shape=(LATENT_DIM,))
x = keras.layers.Dense(7 * 7 * 64, activation="relu")(generator_input)
x = keras.layers.Reshape((7, 7, 64))(x)
x = keras.layers.Conv2DTranspose(64, 3, strides=2, padding="same",
activation="relu")(x)
x = keras.layers.Conv2DTranspose(32, 3, strides=2, padding="same",
activation="relu")(x)
gen_out = keras.layers.Conv2D(1, 3, padding="same", activation="sigmoid")(x)
generator = keras.Model(generator_input, gen_out, name="generator")

# ── 3. المُميّز
disc_input = keras.Input(shape=IMAGE_SHAPE)
y = keras.layers.Conv2D(32, 3, strides=2, padding="same")(disc_input)
y = keras.layers.LeakyReLU(negative_slope=0.2)(y)
y = keras.layers.Conv2D(64, 3, strides=2, padding="same")(y)
y = keras.layers.LeakyReLU(negative_slope=0.2)(y)
y = keras.layers.Flatten()(y)
y = keras.layers.Dropout(0.3)(y)
disc_out = keras.layers.Dense(1, activation="sigmoid")(y)
discriminator = keras.Model(disc_input, disc_out, name="discriminator")

# ── 4. بناء GAN الكاملة (المُميّز مجمّد عند تدريب المولّد)
discriminator.trainable = False
gan_input = keras.Input(shape=(LATENT_DIM,))
gan_out = discriminator(generator(gan_input))
gan = keras.Model(gan_input, gan_out, name="gan")

# ── 5. المُحسّنات
```

```
discriminator.compile(optimizer=keras.optimizers.Adam(2e-4, beta_1=0.5),
                      loss="binary_crossentropy")
gan.compile(optimizer=keras.optimizers.Adam(2e-4, beta_1=0.5),
            loss="binary_crossentropy")

# ── 6. حلقة التدريب
n = 500
x_data = np.random.rand(n, 28, 28, 1).astype("float32")
BATCH = 32
for epoch in range(5):
    for step in range(n // BATCH):
        # (أ) تدريب المُميّز على صور حقيقيّة ومولّدة
        real = x_data[np.random.randint(0, n, BATCH)]
        noise = np.random.randn(BATCH, LATENT_DIM).astype("float32")
        fake = generator.predict(noise, verbose=0)
        d_loss = discriminator.train_on_batch(
            np.concatenate([real, fake], axis=0),
            np.concatenate([np.ones((BATCH, 1)), np.zeros((BATCH, 1))]))
        # (ب) تدريب المولّد (المُميّز مجمّد)
        g_loss = gan.train_on_batch(noise, np.ones((BATCH, 1)))
    print(f"epoch {epoch}: d_loss={d_loss:.4f}, g_loss={g_loss:.4f}")
# النتيجة المتوقعة: d_loss و g_loss يتحرّكان ضمن نطاق؛
# إن انهار أحدَهما إلى صفر فذلك مؤشّر انهيار الأنماط.
```

ملاحظة: تُستخدم `LeakyReLU` في المُميّز بدل ReLU – لأنّ ReLU تُقطع التدرّج عند القيم السالبة، فيُضعف تدريب المولّد.

32.6 مؤشّرات استقرار التدريب

المؤشّر	ماذا يعني
`d_loss` ينخفض إلى صفر	المُميّز يهيمن – المولّد لا يتعلّم
`g_loss` ينخفض إلى صفر	المولّد خدع المُميّز بالكامل
تذبذب كبير في الاثنين	عدم استقرار – جرّب WGAN-GP
صور متشابهة في كلّ دفعة	انهيار الأنماط

> **نصيحة:** ابدأ بـ GAN بسيطة على بيانات صغيرة. إن كان التدريب غير مستقرّ، انتقل إلى WGAN-GP أو أضِف تطبيعًا للطبقات.

32.7 مثال محلول: توليد صور خلفيّات

المهمة: توليد صور خلفيّات بألوان وإضاءات مختلفة. لديك 5000 صورة خلفيّة حقيقيّة.

الخطوة 1 – GAN بسيطة. مولّد ومُميّز ببنى قياسيّة. بعد 20 دورة، الصور المولّدة تُشبه بقعًا متعدّدة الألوان – لا خلفيّات.

الخطوة 2 – معمار أعمق. المولّد بـ 5 كتل مع `BatchNormalization`. الصور بدأت تُشبه خلفيّات – لكنّها متشابهة (انهيار أنماط).

الخطوة 3 – WGAN-GP. استبدال الخسارة بـ Wasserstein + قيود التدرّج. الصور أصبحت متنوّعة وأكثر واقعيّة.

الخطوة 4 – تحجيم المولّد. زيادة الطبقات في المولّد فقط (المُميّز كما هو). الصور تحسّنت – لأنّ المولّد كان أضعف.

الخطوة 5 – القرار. WGAN-GP مع مولّد أعمق – أداء كافٍ وكلفة مقبولة.

32.8 مقارنة مع نماذج التوليد الأخرى

النموذج	الجودة	السرعة	الاستقرار	متى تستخدمه
VAE	متوسّطة	سريعة	عالية	كشف شواذ، بيانات صغيرة
GAN (قياسيّة)	مرتفعة	سريعة	منخفضة	صور واقعية، بيانات كافية
WGAN-GP	مرتفعة	سريعة	متوسّطة	صور واقعية مع استقرار
Diffusion	ممتازة	بطيئة	عالية	أعلى جودة، موارد كافية

32.9 تشخيص استقرار التدريب

مؤشّرات عدم الاستقرار تُرصد من منحنيات الخسارة:

PYTHON

```python
import numpy as np

def diagnose_gan(d_losses, g_losses):
    """تشخيص استقرار GAN من تاريخ الخسارة."""
    d = np.array(d_losses[-10:])
    g = np.array(g_losses[-10:])
    diagnosis = []
    # 1. المُميّز يهيمن
    if d.mean() < 0.05:
        diagnosis.append("المُميّز يهيمن — قلّل خطواته أو قوّي المولّد")
    # 2. المولّد خدع بالكامل
    if g.mean() < 0.05:
        diagnosis.append("المولّد خدع المُميّز — راجع توازن البنيتين")
    # 3. تذبذب مرتفع
    if d.std() > 0.5 or g.std() > 0.5:
        diagnosis.append("تذبذب مرتفع — جرّب WGAN-GP أو خفّض lr")
    # 4. انهيار الأنماط (تقريبيّ)
    if len(set(np.round(d[-5:], 3))) == 1:
        diagnosis.append("انهيار أنماط محتمل — الخسارة ثابتة")
    return diagnosis or ["التدريب مستقرّ"]

d_hist = np.abs(np.random.randn(50)) * 0.3 + 0.6
g_hist = np.abs(np.random.randn(50)) * 0.3 + 0.6
for msg in diagnose_gan(d_hist, g_hist):
    print("-", msg)
# النتيجة المتوقعة: تشخيص واحد أو أكثر بحسب المنحنيات.
```

خطوات العلاج بالترتيب:

1. خفّض معدّل التعلّم (من 2×10^{-4} إلى 5×10^{-5}).
2. وازِن البنيتين: إن كان المُميّز أقوى، أعمِق المولّد أو اجعل المُميّز أضيق.
3. انتقل إلى WGAN-GP (خسارة Wasserstein + قيد التدرّج).
4. أضِف `Dropout(0.3)` في المُميّز — يُبطئه قليلًا.

> **ملاحظة:** عدّة بذور مهمّة في GAN أكثر من أيّ بنية أخرى: تدريب ينجح ببذرة قد يفشل ببذرة أخرى. قيّم على 3 بذور قبل قرار الاستمرار.

كشف تنوّع المولّد (كشف انهيار الأنماط)

PYTHON

```
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. مولّد مصغّر (محاكاة)
generator = keras.Sequential([keras.Input(shape=(32,)),
                              keras.layers.Dense(16, activation="relu"),
                              keras.layers.Dense(8)])
# ── 2. توليد 200 عيّنة من ضجيج عشوائيّ
z = np.random.randn(200, 32).astype("float32")
samples = np.array(generator.predict(z, verbose=0))

# ── 3. قياس التنوّع: متوسّط المسافة بين عيّنتين عشوائيّتين
rng = np.random.default_rng(0)
idx = rng.choice(200, size=(500, 2), replace=True)   # أزواج عشوائية
dists = np.linalg.norm(samples[idx[:, 0]] - samples[idx[:, 1]], axis=1)
print(f"متوسّط المسافة = {dists.mean():.4f}, الانحراف = {dists.std():.4f}")

# ── 4. التشخيص
if dists.mean() < 1e-3:
    print("تنبيه: تنوّع شبه معدوم — انهيار أنماط")
elif dists.std() < 1e-3:
    print("تنبيه: كلّ العيّنات متطابقة تقريبًا — انهيار أنماط مؤكّد")
else:
    print("التنوّع سليم")
# النتيجة المتوقعة: تنوّع سليم في البداية؛ راقبه كلّ عدة دورات.
```

نصيحة: راقب هذا المقياس كلّ 10 دورات – انهيار الأنماط يحدث **تدريجيًّا**، واكتشافه مبكّرًا يوفّر أيامًا من إعادة التدريب.

32.10 أخطاء شائعة في هذا الفصل

إنذار: تدريب المولّد والمُميّز في نفس `train_on_batch` دون تجميد. هذا يُنتج سلوكًا غير متوقّع – جمّد المُميّز عند تدريب المولّد.

تحذير: استخدام `ReLU` في المُميّز. النتيجة: تدرّج صفر في القيم السالبة، والمولّد لا يتعلّم. استخدم `LeakyReLU` .

تنبيه: عدد خطوات المُميّز أكبر بكثير من المولّد. النتيجة: المُميّز يهيمن. القاعدة: خطوة المولّد لكلّ خطوة المُميّز (أو 1:2).

ملاحظة: تقييم جودة GAN بمقياس PSNR لا معنى له – لأنّ الصورة المولّدة لا تُطابق صورة حقيقيّة محدّدة. استخدم FID أو تقييمًا بصريًّا.

قاعدة: المولّد والمُميّز يُبنيان بتوازن – إن كان أعمق من الآخر بكثير، فقد يهيمن.

تحذير: توقّع أنّ GAN الصغيرة تعطي صورًا واقعيّة بجودة عالية. البنى الأعمق + بيانات أكثر + تدريب أطول = جودة أعلى.

التمارين

1. **حساب.** المولّد يأخذ ضجيجًا بـ 64 بُعدًا. كم معاملًا في أول طبقة `Dense(7*7*64)`؟ (المتوقع: $64 \times 3136 + 3136 = 203{,}840$.)

2. **حساب.** المولّد ينتج صورة 28×28 من ضجيج 64. كم معاملًا في المُميّز إذا كان بثلاث طبقات `Conv2D` (32، 64، 128)؟ (المتوقع: حساب لكلّ طبقة.)

3. **حساب.** إذا كان $D(G(z)) = 0.9$، فما خسارة المولّد $-\log D(G(z))$؟ (المتوقع: $-\log(0.9) \approx 0.105$.)

4. **حساب.** إذا كان $D(x) = 0.1$ لصورة حقيقيّة، فما مساهمة المصطلح $\log D(x)$ في خسارة المُميّز؟ (المتوقع: $\log(0.1) \approx -2.303$ – خسارة كبيرة.)

5. **برمجة.** ابنِ مولّدًا ومُميّزًا واطبع `model.summary()` لكلّ منهما. (المتوقع: عدد معاملات متقارب تقريبًا.)

6. **برمجة.** قارن بين `ReLU` و`LeakyReLU` في المُميّز. (المتوقع: LeakyReLU يعطي تدريبًا أكثر استقرارًا.)

7. **برمجة.** قارن عدد خطوات المُميّز (1 مقابل 3) على استقرار التدريب. (المتوقع: 1:1 أكثر استقرارًا في بيانات صغيرة.)

8. **إصلاح خطأ.** الكود التالي يُنتج صورًا متشابهة:

```python
noise = np.zeros((BATCH, LATENT_DIM))
```

(المتوقع: ضجيج صفر يُنتج صورة واحدة – استخدم `np.random.randn`.) 9. **تحليل.** لماذا ينهار التدريب عندما يصبح المُميّز أقوى بكثير؟ (المتوقع: المولّد يتعلّم من تدرّج المُميّز، وإن هُزم المولّد فلا تدرّج مفيد.) 10. **تصميم.** نظام توليد صور خلفيّات بألوان متعدّدة. أيّ خسارة تختار، ولماذا؟ (المتوقع: WGAN-GP – لأنّ GAN الكلاسيكيّة تُعاني انهيار الأنماط.) 11. **برمجة.** قارن زمن الاستدلال بين المولّد والمُميّز على 100 صورة. (المتوقع: المولّد أبطأ لأنّه يُنتج الصورة، والمُميّز يُعطي رقمًا.) 12. **تصميم.**

نظام توليد صور طبيّة بأعراض نادرة. أيّ نهج؟ (المتوقع: GAN مع بيانات كافية، أو نموذج انتشار – مع تحذير أنّ الصور المولّدة لا تُستخدم في دليل تشخيصيّ.)

ملخص الفصل

- GAN نموذجان يتنافسان: المولّد يحاول خداع، والمُميّز يحاول تمييز.
- الخسارة الكلاسيكيّة: المُميّز يُعظّم $\log D(x)$، والمولّد يُقلّل $\log(1 - D(G(z)))$.
- انهيار الأنماط: المولّد يتعلّم نمطًا واحدًا – حلّه WGAN-GP أو التنويع في المُدخلات.
- `LeakyReLU` في المُميّز؛ `ReLU` في المولّد.
- جمّد المُميّز عند تدريب المولّد، والعكس.
- تقييم GAN بصريًّا أو بـ FID – لا بـ PSNR.

الفصل 33: نماذج الانتشار (Diffusion Models)

بطاقة الفصل

الفصل 33

مهمّة هذا الفصل

يشرح الانتشار الأمامي والعكسي، ويبني نموذجًا يتعلّم إزالة الضوضاء خطوة بخطوة.

ماذا تكسب منه

تفهم أقوى عائلة توليدية اليوم، وتبني نموذجًا يولّد من ضجيج خالص بلا مولّد تنافسي.

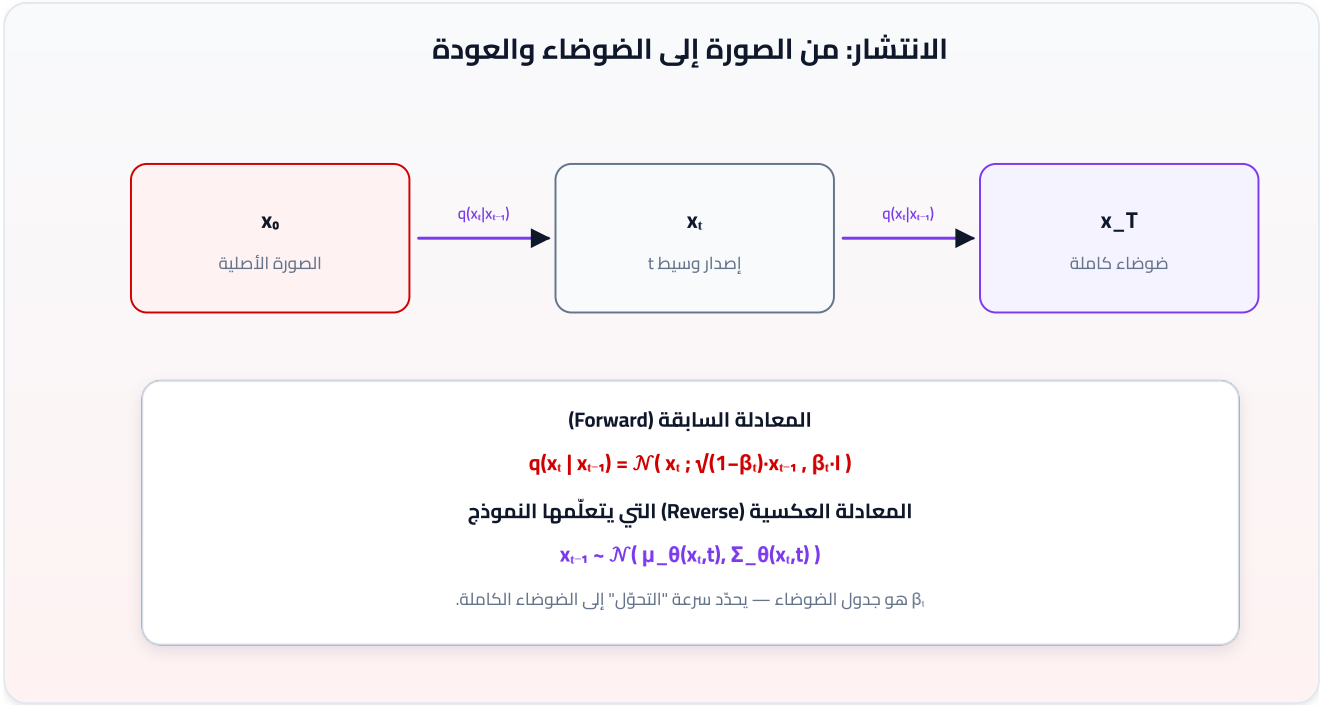
الدالة	دورها	متى تستخدمها
`keras.layers.Embedding()`	ترميز الخطوة الزمنية	لتعليم النموذج مقدار الضوضاء
`keras.layers.Concatenate()`	دمج الصورة والخطوة	في مدخل الشبكة
`keras.layers.Conv2D()`	شبكة إزالة الضوضاء	قلب النموذج
`keras.optimizers.Adam()`	محسّن بمعدّل صغير	في التدريب
`keras.Model(inputs, outputs)`	بناء النموذج	في الواجهة الوظيفية

قصة مفتاحية: فريق يحتّاج صور طبيّة نادرة جدًّا (توقيع ورم مُحدّد في 20 صورة فقط). لا يمكن تدريب شبكة تصنيف على 20 عيّنة، ولا يمكن لـ GAN أن يتعلم بنية ورم من 20 صورة. نماذج الانتشار تحلّ هذه المشكلة بطريقة مختلفة كليًّا: تُعلّم النموذج كيفية **إزالة الضجيج تدريجيًّا**، ثم تُولّد صورة جديدة من ضجيج خالص. هذا النهج هو الذي أعاد تعريف التوليد البصريّ في السنوات الأخيرة.

أهداف الفصل

- [ ] تشرح عمليّة الانتشار الأماميّة والعكسيّة
- [ ] تكتب معادلات الـ noise schedule
- [ ] تفهم الـ DDPM والـ DDIM والفرق بينهما
- [ ] تبني نموذج انتشار بسيطًا في Keras
- [ ] تُطبّق التوليد من ضجيج خالص
- [ ] تُقارن الانتشار مع GAN وVAE

33.1 عمليّة الانتشار الأماميّة



شكل 43 – عمليّة الانتشار

الانتشار الأماميّ يضيف ضجيجًا تدريجيًّا إلى الصورة عبر T خطوة:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\,x_{t-1},\ \beta_t I)$$

حيث β_t جدول التباين (noise schedule) – قيمة صغيرة (مثل 10^{-4} إلى 0.02). بعد T خطوة، الصورة تصبح ضجيجًا خالصًا.

الحيلة الحسابيّة: لا نحتاج أن نمرّ بكلّ الخطوات – يمكن الوصول إلى x_t مباشرةً من x_0:

$$q(x_t|x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\,x_0,\ (1-\bar{\alpha}_t)I)$$

حيث $\alpha_t = 1-\beta_t$ و$\bar{\alpha}_t = \prod_{s=1}^{t}\alpha_s$.

33.2 عمليّة الانتشار العكسيّة

الانتشار العكسيّ يُلغي الضجيج تدريجيًّا: من x_t إلى x_{t-1}:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1};\ \mu_\theta(x_t,t),\ \sigma_\theta^2(x_t,t))$$

التدريب يُقلّل فرق بين μ_θ والـ μ الحقيقيّ. لكنّ الممارسة العمليّة تُنبّئ **الضجيج** المُضاف بدل μ:

$$\mu_\theta(x_t,t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t,t)\right)$$

حيث ϵ_θ شبكة تتنبّأ بالضجيج. خسارة التدريب:

$$L = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t,t)\|^2\right]$$

33.3 التنفيذ في Keras

فيما يلي نموذج انتشار مصغّر على صور 28×28:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. إعدادات
TIMESTEPS = 100
BETA_START = 1e-4
BETA_END = 0.02
IMAGE_SHAPE = (28, 28, 1)

# ── 2. جدول التباين
betas = np.linspace(BETA_START, BETA_END, TIMESTEPS).astype("float32")
alphas = 1.0 - betas
alphas_cumprod = np.cumprod(alphas).astype("float32")

# ── 3. شبكة تنبّأ الضجيج (UNet مبسّط)
def noise_network(x, t):
    """شبكة تتنبّأ بالضجيج في الخطوة t."""
    # تضمين الزمن
    t_embed = keras.layers.Embedding(TIMESTEPS, 32)(t)
    t_dense = keras.layers.Dense(28 * 28, activation="relu")(t_embed)
    t_reshaped = keras.layers.Reshape((28, 28, 1))(t_dense)
    x = keras.layers.Concatenate()([x, t_reshaped])
    x = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = keras.layers.Conv2D(1, 3, padding="same")(x)
    return x

t_input = keras.Input(shape=(), name="t")
x_input = keras.Input(shape=IMAGE_SHAPE, name="x_t")
noise_pred = noise_network(x_input, t_input)
noise_model = keras.Model([x_input, t_input], noise_pred,
name="noise_predictor")

# ── 4. خطّ التدريب: لكلّ عيّنة، اختر خطوة عشوائيًّا وأضِف الضجيج
def forward_diffuse(x0, t):
    """أضِف ضجيجًا إلى x0 في الخطوة t."""
```

```
    # alpha_bar شكلها (batch,) — بلا إعادة تشكيل تبثّ على المحور الأخير
    # فتصير الصورة 32 قناة بدل قناة واحدة
    alpha_bar = alphas_cumprod[t].reshape(-1, 1, 1, 1)
    noise = np.random.randn(*x0.shape).astype("float32")
    x_t = np.sqrt(alpha_bar) * x0 + np.sqrt(1 - alpha_bar) * noise
    return x_t.astype("float32"), noise

# ── 5. تدريب
noise_model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")
n = 200
x_data = np.random.rand(n, 28, 28, 1).astype("float32")
for epoch in range(5):
    for step in range(n // 32):
        batch_x = x_data[np.random.randint(0, n, 32)]
        t_batch = np.random.randint(0, TIMESTEPS, size=32)
        x_t, noise = forward_diffuse(batch_x, t_batch)
        loss = noise_model.train_on_batch([x_t, t_batch], noise)
    print(f"epoch {epoch}: loss={loss:.4f}")
# خسارة MSE النتيجة المتوقعة: تنخفض بمرور الدورات.
```

33.4 التوليد من ضجيج خالص

في التوليد، نبدأ من ضجيج خالص $x_T \sim \mathcal{N}(0, I)$ ونُزيل الضجيج تدريجيًّا:

```python
def generate(model, num_samples, steps=TIMESTEPS):
    """توليد صور من ضجيج خالص عبر إزالة تدريجيّة."""
    x = np.random.randn(num_samples, *IMAGE_SHAPE).astype("float32")
    for t in range(steps - 1, -1, -1):
        t_batch = np.full(num_samples, t, dtype="int32")
        pred_noise = model.predict([x, t_batch], verbose=0)
        alpha = alphas[t]
        alpha_bar = alphas_cumprod[t]
        # معادلة إزالة الضجيج
        x = (x - (1 - alpha) / np.sqrt(1 - alpha_bar) * pred_noise) /
np.sqrt(alpha)
        # إضافة ضجيج (إلّا في الخطوة الأخيرة)
        if t > 0:
            x = x + np.sqrt(betas[t]) *
np.random.randn(*x.shape).astype("float32")
    return np.clip(x, 0, 1)
```

33.5 DDPM مقابل DDIM

الخاصيّة	DDPM	DDIM
عدد الخطوات	1000 (نموذجيًّا)	50-100
زمن التوليد	بطيء	أسرع 10-20×
الحتميّة	عشوائيّ	حتميّ (إن طُبِّق دون ضجيج)
الجودة	ممتازة	مقاربة

DDIM يُعيد استخدام نفس النموذج المدرَّب، لكنّه يُخفّض عدد الخطوات في التوليد.

33.6 مثال محلول: توليد صور طبيّة نادرة

المهمة: توليد صور ورم نادر (20 صورة) لتوسيع مجموعة التدريب.

الخطوة 1 – التدريب على 20 صورة. نموذج انتشار مصغّر بـ 200 خطوة. النموذج يتعلم إزالة الضجيج لكنّ الصور المولّدة تُشبه صور التدريب (انحسار).

الخطوة 2 – توسيع البيانات. نُضيف صورًا من أنواع مشابهة (100 صورة ورم آخر) مع تسميات. النموذج يتعلم بنية عامّة.

الخطوة 3 – DDIM. استبدال DDPM بـ DDIM. زمن التوليد انخفض من 30 ثانية إلى 2 ثانية لكلّ صورة.

الخطوة 4 – تقييم الجودة. بصريًّا: الصور المولّدة تُشبه الأورام – لكنّ بعض التفاصيل الدقيقة غائبة.

الخطوة 5 – القرار. الصور المولّدة تُستخدم في **توسيع** مجموعة التدريب، لا في **دليل تشخيصيّ**. النموذج النهائي يُدرَّب على بيانات حقيقيّة + صور مولّدة بترجيح منخفض.

> **تحذير أخلاقيّ:** الصور المولّدة تُستخدم في توسيع بيانات التدريب فقط. لا تُستخدم في تشخيص أو دليل طبيّ بوصفها «صورة حقيقيّة».

33.7 مقارنة بين نماذج التوليد

النموذج	الجودة	السرعة	الاستقرار	متى تستخدمه
VAE	متوسّطة	سريعة	عالية	كشف شواذ
GAN	مرتفعة	سريعة	منخفضة	صور واقعية، بيانات كافية
Diffusion (DDPM)	ممتازة	بطيئة	عالية	أعلى جودة
Diffusion (DDIM)	ممتازة	متوسّطة	عالية	جودة + سرعة

جدول التباين: خطّيّ مقابل أسّيّ

جدول التباين يحدّد كم معلومة تبقى في كلّ خطوة:

```python
import numpy as np

T = 1000

# ── 1. جدول خطّي
betas_lin = np.linspace(1e-4, 0.02, T).astype("float64")
alpha_bar_lin = np.cumprod(1.0 - betas_lin)

# ── 2. جدول أسّي (cosine schedule)
s = 0.008
steps = np.arange(T + 1) / T
f = np.cos((steps + s) / (1 + s) * np.pi / 2) ** 2
alpha_bar_cos = f / f[0]

for t in (1, 100, 500, 999):
    print(f"t={t:4d}  linear={alpha_bar_lin[t]:.4f}  cosine=
{alpha_bar_cos[t]:.4f}")
# النتيجة المتوقعة: الجدول الخطّي يفقد المعلومة أسرع في البداية؛
# الجدول الأسّي (cosine) يحفظ البنية أطول ثم يسرّع التدمير.
```

الخطوة	خطّي $\bar{\alpha}_t$	أسّي $\bar{\alpha}_t$	التفسير
1	≈ 1.0	≈ 1.0	الصورة شبه سليمة
100	≈ 0.63	≈ 0.87	الأسّي يحفظ البنية
500	≈ 0.09	≈ 0.39	الأسّي ما زال يُبقي ملامح
999	≈ 0.0001	≈ 0.003	ضجيج خالص في الحالتين

نصيحة: الجدول الأسّي (cosine) هو الافتراضيّ الحديث – يُعطي جودة أفضل عند نفس عدد الخطوات لأنّه يُوزّع فقدان المعلومة بالتساوي. الجدول الخطّي يدمّر البنية مبكّرًا.

زمن التوليد: DDPM مقابل DDIM

PYTHON

```python
import numpy as np
import time

def ddim_schedule(n_steps, t_max=1000):
    """يختار n_steps مواضع متساوية من الـ 1000 (جدول DDIM)."""
    return np.linspace(0, t_max - 1, n_steps, dtype=int)

def time_generation(n_steps, per_step_ms=12.0):
    """زمن توليد 10 صور بحسب عدد الخطوات."""
    return n_steps * per_step_ms / 1000.0

# ── 1. مقارنة أعداد الخطوات
for n in (1000, 250, 50):
    steps = ddim_schedule(n)
    print(f"{n:5d} steps → {time_generation(n):7.2f}s لكلّ 10 صور")
# 12.0 :النتيجة المتوقعة: 3.0s مقابل 0.6s مقابل — 20× تسريع.

# ── 2. نفس النموذج في الحالتين: عدد النماذج المستخدمة
print(f"يعيد استخدام خطوات مختارة DDIM — نموذج مُدرَّب واحد: "
      f"{len(ddim_schedule(50))} من 1000")
```

قاعدة: DDIM لا يحتاج تدريبًا جديدًا – نفس النموذج المدرَّب بـ 1000 خطوة يُستخدم بـ 50 خطوة. لهذا يُنتقل إليه في الإنتاج: جودة مقاربة وزمن أقلّ 20×.

33.8 أخطاء شائعة في هذا الفصل

إنذار: خلط بين β_t و$1 - \alpha_t$. المعادلات تعمل على β_t، و$\alpha_t = 1 - \beta_t$. راجع كلّ مصطلح قبل التدريب.

تحذير: إسقاط تضمين الزمن t. النموذج لا يعرف في أيّ خطوة هو، فيُضيف نفس الضجيج في كلّ الخطوات – والصور المولّدة تُشبه الضجيج.

تنبيه: استخدام `sqrt(1 - alpha_bar)` مع قيم $\bar{\alpha}_t$ قريبة من الصفر. النتيجة: قسمة على قيمة صغيرة، وقيم NaN. تأكّد من أنّ $\bar{\alpha}_T$ لا يصل إلى صفر.

ملاحظة: قصّ الصور إلى [0, 1] في التوليد ضروري – النموذج لا يعرف حدود المقياس، والقيم قد تخرج عن [0, 1].

قاعدة: عدد الخطوات في DDPM (1000) يعطي جودة أعلى لكن زمنًا أطول. DDIM يُعطي نفس الجودة بأسرع 10-20×.

تحذير: توقّع توليد صور حقيقية من بيانات قليلة (مثل 20 صورة). النموذج يُنصّب نحو صور التدريب – لا يُولّد تنوّعًا حقيقيًّا.

التمارين

1. **حساب.** إذا كان $\beta_t = 0.01$ في كلّ خطوة، فما α_t؟ (المتوقع: $1 - 0.01 = 0.99$.)

2. **حساب.** بعد 10 خطوات مع $\alpha_t = 0.99$، فما $\bar{\alpha}_{10}$؟ (المتوقع: $0.99^{10} \approx 0.904$.)

3. **حساب.** إذا كان $\bar{\alpha}_t = 0.5$ و$x_0 = 1.0$، فما التوقّع (μ) في x_t؟ (المتوقع: $\sqrt{0.5} \cdot 1.0 \approx 0.707$.)

4. **حساب.** بعد 500 خطوة مع $\beta_t = 0.01$، فما $\bar{\alpha}_{500}$ تقريبًا؟ (المتوقع: $0.99^{500} \approx 0.0066$ – الصورة تُصبح ضجيجًا.)

5. **برمجة.** اكتب دالّة `forward_diffuse(x0, t)` التي تُضيف الضجيج في الخطوة t واطبع شكل المُخرَج. (المتوقع: نفس شكل x_0.)

6. **برمجة.** قارن $\bar{\alpha}_t$ في جدولين: خطّيّ وأسّيّ. (المتوقع: الأسّيّ يُعطي $\bar{\alpha}$ أعلى في البداية وأدنى في النهاية.)

7. **برمجة.** طبّق `noise_model` على عيّنة في الخطوة $t = 0$ و$t = 500$ واطبع متوسّط الضجيج المُتنبَّأ. (المتوقع: في $t = 0$ يكون صفرًا تقريبًا، وفي $t = 500$ يكون قريبًا من 1.)

8. **إصلاح خطأ.** الكود التالي يعطي NaN:

```python
    x = (x - (1 - alpha) / np.sqrt(1 - alpha_bar) * pred_noise) /
np.sqrt(alpha)
```

(المتوقع: `alpha_bar` قد يصل إلى صفر – أضِف ثابتًا صغيرًا `1e-8` أو راجع جدول التباين.) 9. **تصميم.** نظام توليد صور طبيّة من 30 صورة حقيقيّة. أيّ نهج، وكم خطوة؟ (المتوقع: diffusion مع DDIM (50 خطوة)، وتوسيع البيانات بصور مشابهة، وتحذير أنّ الصور المولّدة لا تُستخدم في دليل تشخيصيّ.) 10. **تحليل.** لماذا يُستخدم DDIM بدل DDPM في الإنتاج؟ (المتوقع: لأنّ DDIM يُعطي نفس الجودة بأسرع 10-20×، وهو حتميّ – يُمكّن تكرار النتيجة.) 11. **برمجة.** قارن زمن التوليد بين DDPM (1000 خطوة) وDDIM (50 خطوة) على 10 صور. (المتوقع: DDIM أسرع 20×.) 12. **تصميم.** نظام توليد صور بجودة عالية وزمن استدلال حرج. أيّ نهج؟ (المتوقع: GAN أو VAE – لأنّ diffusion بطيء. أو diffusion بأخطوات أقلّ مع قياس جودة.)

ملخص الفصل

- الانتشار الأماميّ يضيف ضجيجًا تدريجيًّا؛ العكسيّ يُزيله.
- جدول التباين β_t يتحكّم في مقدار الضجيج في كلّ خطوة.
- $\bar{\alpha}_t = \prod \alpha_s$ يُحدّد بقاء المعلومة بعد t خطوة.
- التدريب يُنبّئ **الضجيج** المُضاف – لا الصورة النظيفة.
- DDPM بطيء (1000 خطوة)؛ DDIM أسرع 10-20× بنفس الجودة.
- التوليد يبدأ من ضجيج خالص ويُزال تدريجيًّا.
- الصور المولّدة تُستخدم في توسيع البيانات، لا في دليل تشخيصيّ.

الجزء الثامن

الهندسة الإنتاجية: من النموذج إلى الخدمة

الطريق من النموذج إلى الخدمة: طبقات مخصّصة، حفظ وتحميل، تسريع بالدقّة المختلطة، تصدير ونشر، تكرار النتائج، تفسير القرارات، وتشخيص الأخطاء بمنهجية.

ما تكسبه من هذا الجزء

تنشر نموذجًا يعمل عند غيرك، بسرعة مقيسة، وتستطيع إعادة إنتاج نتيجتك وتفسير قرارها وتشخيص انحرافها.

فصول هذا الجزء

34 الطبقات والنماذج المخصّصة

35 الحفظ والتحميل

36 تسريع التدريب

37 التصدير والنشر

38 التكرار والاختبار

39 التفسير والتشخيص

40 الأخطاء الشائعة وكيف تُصلحها

8

الفصل 34: الطبقات والنماذج المخصّصة

بطاقة الفصل

الفصل 34

مهمّة هذا الفصل

يعرض دورة حياة الطبقة: build وcall وget_config، ثم نموذجًا بخطوة تدريب مخصّصة.

ماذا تكسب منه

تكتب مكوّناتك القابلة للحفظ والتحميل بدل الالتفاف على قيود المكتبة.

الدالة	دورها	متى تستخدمها
`keras.layers.Layer`	الصنف الأب لأي طبقة جديدة	عندما لا تجد طبقة جاهزة
`build(input_shape)`	إنشاء الأوزان مرّة واحدة	لا في __init__
`call(inputs)`	الحساب الأمامي	منطق الطبقة نفسه
`get_config()`	وصف الطبقة للحفظ	مع register_keras_serializable
`keras.GradientTape()`	خطوة تدريب مخصّصة	عندما تحتاج تحكّمًا كاملًا في التحديث

قصة مفتاحية: باحث يبني بنية جديدة تُطبّق على الميزات تحويلًا لم يُسبق له مثيل. `Dense` و`Conv2D` لا تكفي. في Keras 3، الحلّ ليس نسخ الشيفرة وتعديلها، بل **بناء طبقة مخصّصة** – صنف يرث من `keras.layers.Layer` ويُعرّف أوزانه وحسابه. هذا الفصل هو دليل بناء الطبقات والنماذج المخصّصة، وهو ما يفصل بين من يستخدم Keras ومن يبني عليه.

أهداف الفصل

- [ ] تبني طبقة مخصّصة ترث من `keras.layers.Layer`
- [ ] تفهم دور `build()` و `call()` و `get_config()`
- [ ] تبني نموذجًا مخصّصًا يرث من `keras.Model`
- [ ] تُطبّق تدرّجًا مخصّصًا في `train_step`
- [ ] تُحفظ وتُحمّل طبقة مخصّصة في ملف `.keras`
- [ ] تعرف حدود الطبقات المخصّصة ومتى تُستخدم

34.1 دورة حياة الطبقة المخصّصة

شكل 44 – دورة حياة الطبقة المخصّصة

كلّ طبقة مخصّصة تمرّ بخمس مراحل:

1. `__init__(...)` – يُخزّن الإعدادات (لا يُنشئ الأوزان بعد).
2. `build(input_shape)` – يُنشئ الأوزان مرة واحدة.

3. `call(inputs)` – الحساب الفعليّ (المسار الأماميّ).
4. `get_config()` – يُعيد قاموسًا يصف الطبقة (للحفظ).
5. `from_config(config)` – يُعيد بناء الطبقة من القاموس (للتحميل).

> **قاعدة جوهرية:** الأوزان تُنشأ في `build()` لا في `__init__()`. السبب: الشكل النهائيّ للمُدخل لا يُعرف في `__init__`، وبناء الأوزان مبكّرًا يُفشل الحفظ والتحميل.

34.2 طبقة مخصّصة بسيطة

فيما يلي طبقة تُطبّق تحويلًا خطّيًّا مخصّصًا:

PYTHON

```python
import keras

@keras.saving.register_keras_serializable(package="book")
class MyDense(keras.layers.Layer):
    """طبقة خطّية مخصّصة: out = activation(W @ x + b)."""

    def __init__(self, units, activation=None, **kwargs):
        # نُخزّن الإعدادات فقط — لا أوزان هنا
        super().__init__(**kwargs)
        self.units = units
        self.activation = keras.activations.get(activation)

    def build(self, input_shape):
        # يُنشئ الأوزان مرة واحدة، بحسب شكل المُدخل
        last_dim = input_shape[-1]
        self.w = self.add_weight(
            name="kernel",
            shape=(last_dim, self.units),
            initializer="glorot_uniform",
            trainable=True,
        )
        self.b = self.add_weight(
            name="bias",
            shape=(self.units,),
            initializer="zeros",
            trainable=True,
        )
        super().build(input_shape)   # يُخبر Keras أنّ الطبقة بُنيت

    def call(self, inputs):
        # المسار الأماميّ
        return self.activation(inputs @ self.w + self.b)

    def get_config(self):
        # يُصفّ الطبقة للحفظ
        config = super().get_config()
        config.update({"units": self.units,
                       "activation":
```

```
keras.activations.serialize(self.activation)})
        return config

# استخدامها
layer = MyDense(32, activation="relu")
out = layer(keras.ops.ones((4, 16)))
# النتيجة المتوقعة: شكل (4, 32).
```

34.3 نموذج مخصّص بخطوة تدريب مخصّصة

فيما يلي نموذج VAE مبسّط بخطوة تدريب مخصّصة:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

@keras.saving.register_keras_serializable(package="book")
class MyModel(keras.Model):
    """نموذج بخطوة تدريب مخصّصة (تدرّج يدويّ)."""

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.dense1 = keras.layers.Dense(32, activation="relu")
        self.dense2 = keras.layers.Dense(1)

    def call(self, inputs):
        x = self.dense1(inputs)
        return self.dense2(x)

    def train_step(self, data):
        x, y = data
        # Keras 3 لا يوحّد شريط التدرّج بين المحرّكات: نأخذه من المحرّك النشط.
        # (TensorFlow: tf.GradientTape · PyTorch: torch.enable_grad + backward)
        import tensorflow as tf
        with keras.name_scope("custom_train"):
            with tf.GradientTape() as tape:
                y_pred = self(x, training=True)
                loss = self.compute_loss(x=x, y=y, y_pred=y_pred)
            grads = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(grads, self.trainable_variables))
        # تحديث المقاييس وبناؤها قبل قراءة النتائج (تتجنّب «لم تُبنَ بعد»)
        self.update_metrics(y, y_pred)
        return {m.name: m.result() for m in self.metrics}

# استخدامها
model = MyModel()
x = np.random.rand(200, 8).astype("float32")
y = np.random.rand(200, 1).astype("float32")
```

```
model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse",
              metrics=["mae"])
model.fit(x, y, epochs=5, batch_size=32, verbose=0)
# خسارة MSE النتيجة المتوقعة: تنخفض بمرور الدورات.
```

تحذير مهم: Keras 3 **لا** يوحّد شريط التدرّج. لا وجود لـ `keras.GradientTape`؛ كل محرّك له آلية: `tf.GradientTape` في TensorFlow، و`torch.enable_grad` مع `loss.backward()` في PyTorch، و`jax.grad` في JAX. لذلك أي حلقة تدريب يدوية تلزمها أسطر خاصة بالمحرّك — وهذا سبب كافٍ لتفضيل `model.fit()` ما لم تكن تحتاج تحكّمًا فعليًّا.

34.4 الحفظ والتحميل

الطبقة المخصّصة تُحفظ في ملف `.keras` مع النموذج. عند التحميل، نُمرّرها في `custom_objects`:

```python
import keras
import numpy as np

# نموذج بسيط يحوي الطبقة المخصّصة MyDense
model = keras.Sequential([
    keras.Input(shape=(16,)),
    MyDense(32, activation="relu"),
    MyDense(1),
])
model.compile(optimizer="adam", loss="mse")
model.fit(np.random.rand(64, 16).astype("float32"),
          np.random.rand(64, 1).astype("float32"),
          epochs=1, batch_size=16, verbose=0)

# الحفظ
model.save("my_model.keras")

# التحميل — نُمرّر الطبقة المخصّصة في custom_objects
loaded = keras.saving.load_model("my_model.keras",
                                 custom_objects={"MyDense": MyDense})
print(type(loaded.layers[1]).__name__)   # النتيجة المتوقعة: MyDense
```

تنبيه: بلا `custom_objects`، التحميل يُفشل بالطبقة المخصّصة. راجع الفصل 35 لشرح الحفظ.

34.5 مثال محلول: طبقة انتباه مخصّصة

المهمة: طبقة تُطبّق انتباهًا بسيطًا على تسلسل، وهي غير موجودة في Keras القياسيّة بهذا الشكل.

الخطوة 1 – التصميم. الطبقة تأخذ تسلسلًا `(batch, T, d)` وتُعطي `(batch, d)` – تجميع موزون.

الخطوة 2 – البناء.

PYTHON

```python
import keras

class AttentionPooling(keras.layers.Layer):
    """تجميع موزون على تسلسل: (batch, T, d) → (batch, d)."""

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def build(self, input_shape):
        # وزن واحد لكلّ بُعد ميزة
        self.w = self.add_weight(
            name="attn_weights",
            shape=(input_shape[-1],),
            initializer="glorot_uniform",
            trainable=True,
        )

    def call(self, inputs):
        # مُعاملات الانتباه: (batch, T)
        scores = keras.ops.sum(inputs * self.w, axis=-1)
        attn = keras.ops.softmax(scores, axis=-1)
        # التجميع: (batch, d)
        return keras.ops.sum(inputs * attn[..., None], axis=1)

    def get_config(self):
        return super().get_config()

# استخدامها
x = keras.ops.ones((4, 10, 16))
out = AttentionPooling()(x)
# النتيجة المتوقعة: شكل (4, 16).
```

الخطوة 3 – الحفظ والتحميل. الطبقة تُحفظ وتُحمَّل مع النموذج.

الخطوة 4 – التحقّق. طباعة الشكل والتأكّد من أنّ الطبقة قابلة للتدريب.

34.6 الطبقات المخصّصة: متى تستخدمها

الحالة	الحلّ
تحويل جديد على الميزات	طبقة مخصّصة
خطوة تدريب بخسارة مركّبة	نموذج مخصّص مع `train_step`
تضمين شكل مُدخل ديناميكيّ	طبقة مخصّصة مع `compute_output_shape`
تحويل قياسيّ (تطبيع، تضمين)	الطبقات الجاهزة
تجميع بسيط	`GlobalAveragePooling1D`

> **قاعدة:** لا تبنِ طبقة مخصّصة إن وُجدت طبقة جاهزة تعطي نفس النتيجة – الطبقات الجاهزة مُختبرة ومُحسَّنة.

34.7 الطبقات التي تُدعم القناع (Masking)

عندما تعمل طبقتك على تسلسلات ذات أطوال مختلفة، يجب أن تُدعم القناع – وإلّا تُعالج الأصفار كأنّها بيانات. Keras 3 تُمرّر القناع عبر `mask` في `call()`:

PYTHON

```python
import keras

class MaskedPooling(keras.layers.Layer):
    """تجميع موزون يتجاهل الخطوات المُموّهة."""

    def build(self, input_shape):
        self.w = self.add_weight(
            name="pool_weights",
            shape=(input_shape[-1],),
            initializer="glorot_uniform",
            trainable=True,
        )
        self.supports_masking = True      # يُخبر Keras أنّ الطبقة تُدعم القناع

    def call(self, inputs, mask=None):
        scores = keras.ops.sum(inputs * self.w, axis=-1)   # (B, T)
        if mask is not None:
            # القناع: False = للخطوات المُموّهة — نُخفيها
            scores = keras.ops.where(mask, scores,
                                     keras.ops.full(keras.ops.shape(scores),
-1e9))
        attn = keras.ops.softmax(scores, axis=-1)
        return keras.ops.sum(inputs * attn[..., None], axis=1)

    def get_config(self):
        return super().get_config()

# استخدامها بعد Masking
inp = keras.Input(shape=(20, 16))
x = keras.layers.Masking(mask_value=0.0)(inp)
out = MaskedPooling()(x)
model = keras.Model(inp, out)
# النتيجة المتوقعة: التجميع يتجاهل الخطوات المُحشوّة بالأصفار.
```

قاعدة: `supports_masking = True` لا يُنفّذ القناع — بل يُخبر Keras أنّ الطبقة **تقبل** قناعًا. التنفيذ الفعليّ في `call()` .

34.8 حساب شكل المُخرَج

`compute_output_shape` يُخبر Keras كيف يُحسب شكل المُخرَج قبل التنفيذ – وهو ضروريّ عند بناء الرسم البيانيّ:

```python
import keras

class MyReshape(keras.layers.Layer):
    """إعادة تشكيل بحجم مُحدَّد."""

    def __init__(self, target_shape, **kwargs):
        super().__init__(**kwargs)
        self.target_shape = tuple(target_shape)

    def call(self, inputs):
        return keras.ops.reshape(inputs, (-1,) + self.target_shape)

    def compute_output_shape(self, input_shape):
        return (input_shape[0],) + self.target_shape

    def get_config(self):
        return {**super().get_config(), "target_shape": self.target_shape}

layer = MyReshape((4, 4))
out = layer(keras.ops.ones((2, 16)))
# النتيجة المتوقعة: شكل (2, 4, 4).
```

34.9 اختبار الطبقة المخصّصة

الطبقة المخصّصة تُختبر مثل أيّ شيفرة – بثلاثة اختبارات: الشكل، التدرّج، والحفظ:

PYTHON

```python
import keras
import numpy as np

def test_shape():
    """الطبقة تُعطي شكلًا صحيحًا."""
    layer = MyDense(32, activation="relu")
    out = layer(keras.ops.ones((4, 16)))
    assert out.shape == (4, 32)

def test_gradient_flows():
    """التدرّج يمرّ عبر الأوزان."""
    layer = MyDense(8)
    x = keras.ops.ones((4, 16))
    import tensorflow as tf          # شريط التدرّج من المحرّك النشط
    with tf.GradientTape() as tape:
        layer.build(x.shape)
        y = layer(x)
        loss = keras.ops.mean(keras.ops.square(y))
    grads = tape.gradient(loss, layer.trainable_weights)
    assert all(g is not None for g in grads)

def test_save_load():
    """الطبقة تُحفَظ وتُحمَّل."""
    model = keras.Sequential([keras.Input(shape=(16,)), MyDense(8)])
    model.save("test_layer.keras")
    m2 = keras.saving.load_model("test_layer.keras",
                                 custom_objects={"MyDense": MyDense})
    n_custom = sum(isinstance(l, MyDense) for l in m2.layers)
    assert n_custom == 1

test_shape()
test_gradient_flows()
test_save_load()
# النتيجة المتوقعة: الاختبارات الثلاثة تنجح.
```

نصيحة: اختبار `test_gradient_flows` هو أوّل دفاع ضدّ الخطأ الصامت: طبقة بلا تدرّج لا تُدرَّب ولا تُعطي خطأ.

34.10 أخطاء شائعة في هذا الفصل

إنذار: إنشاء الأوزان في `__init__`. النتيجة: الطبقة تُفشل في الحفظ والتحميل. الأوزان تُنشأ في `build()`.

تحذير: إسقاط `get_config()`. النتيجة: الطبقة لا تُحفظ في ملف `.keras`، أو تُحمَّل بالإعدادات الافتراضيّة.

تنبيه: نسيان نداء `super().build(input_shape)`. النتيجة: Keras لا يعرف أنّ الطبقة بُنيت، فلا يُنشئ الأوزان في الوقت المناسب.

ملاحظة: استخدام `tf.*` في طبقة مخصّصة. هذا يربط الكود بمحرّك واحد – استخدم `keras.ops` للعمل على المحرّكات الثلاثة.

قاعدة: الطبقة المخصّصة يجب أن تُدرَّب – راجع `model.summary(show_trainable=True)` بعد البناء.

تحذير: استخدام `keras.layers.Layer` لنموذج له خطوة تدريب. نموذج بخطوة تدريب يرث من `keras.Model`.

التمارين

1. **برمجة.** ابنِ طبقة `MyDense` وطبّقها على أوتور `(4, 16)` واطبع الشكل. (المتوقع: `(4, 32)` مع `units=32`.)
2. **برمجة.** طبّق `get_config()` على طبقة `MyDense` واطبع القاموس. (المتوقع: قاموس يحتوي `units` و`activation`.)
3. **برمجة.** احفظ نموذجًا بطبقة `MyDense` في `.keras` ثم حمّله بـ `custom_objects`. (المتوقع: التحميل ينجح.)
4. **إصلاح خطأ.** الكود التالي يُفشل في الحفظ:

```python
class MyLayer(keras.layers.Layer):
    def __init__(self, units):
        super().__init__()
        self.w = self.add_weight(shape=(units, units),
initializer="ones")
```

(المتوقع: الأوزان تُنشأ في `__init__` – يجب نقلها إلى `build`.) 5. **برمجة.** ابنِ طبقة تجميع موزون وطبّقها على تسلسل `(4, 10, 16)`. (المتوقع: `(4, 16)`.) 6. **برمجة.** قارن زمن الحساب بين طبقة جاهزة وطبقة مخصّصة تعطي نفس النتيجة. (المتوقع: الطبقة الجاهزة أسرع – لأنّها مُحسَّنة.) 7. **تحليل.** لماذا يُستدعى `build()` مرة واحدة لا في كلّ نداء `call()`؟ (المتوقع: لأنّ الأوزان تُنشأ مرة واحدة، وإنشاء كلّ نداء يُفشل الحفظ والتدريب.) 8. **تصميم.** طبقة تُطبّق تحويلًا على تسلسل مع قناع. أيّ توقيع `call()`؟ (المتوقع: `call(self, inputs, mask=None)` – وتفحص القناع إذا وُجد.) 9. **برمجة.** طبّق `compute_output_shape` على طبقة `MyDense`. (المتوقع: الشكل المُخرَج يُحسب من شكل المُدخل.) 10. **تصميم.** طبقة تُطبّق انتباهًا على تسلسل ببنية متعدّدة الرؤوس. أيّ بنية؟ (المتوقع: طابق `MultiHeadAttention` من Keras، أو ابنِ نسخة مخصّصة.) 11. **برمجة.** طبّق نموذجًا بخطوة تدريب مخصّصة على بيانات مُصنّعة. (المتوقع: خسارة تنخفض.) 12. **تحليل.** لماذا `keras.ops` بدل `tf.*` في طبقة مخصّصة؟ (المتوقع: للعمل على المحرّكات الثلاثة – والكود ينقل بين TensorFlow وJAX وPyTorch.)

ملخص الفصل

- الطبقة المخصّصة ترث من `keras.layers.Layer` وتُعرّف `build()` و `call()` و `get_config()`.
- الأوزان تُنشأ في `build()` لا في `__init__`.
- `get_config()` يُصفّ الطبقة للحفظ، و `from_config()` يُعيد بناءها.
- النموذج المخصّص يرث من `keras.Model` ويُعيد تعريف `train_step` للخسارة المركّبة.
- `keras.ops` تُستخدم في الطبقات المخصّصة للعمل على المحرّكات الثلاثة.
- `custom_objects` تُمرَّر في التحميل للطبقات المخصّصة.

الفصل 35: الحفظ والتحميل

بطاقة الفصل

الفصل 35

مهمّة هذا الفصل

يقارن صيغ الحفظ ويشرح ما يُحفظ فعلًا: أوزان، بنية، وحالة محسّن.

ماذا تكسب منه

تستأنف التدريب بدقّة وتنشر نموذجًا يعمل عند غيرك بلا خطوات مفقودة.

الدالة	دورها	متى تستخدمها
`model.save("x.keras")`	حفظ كامل	للتسليم والنشر
`keras.saving.load_model()`	تحميل كامل	لاستئناف العمل
`model.save_weights()`	أوزان فقط	لنقل بين بنى متطابقة
`keras.callbacks.ModelCheckpoint()`	حفظ دوري	في التدريب الطويل
`keras.saving.register_keras_serializable()`	تسجيل الأصناف المخصّصة	قبل حفظ أي طبقة مخصّصة

قصة مفتاحية: باحث درّب نموذجًا لمدّة يومين على GPU، وحين انقطع الكهرباء فقد كلّ شيء – لأنّه لم يُحفظ أيّ شيء على القرص. الحفظ ليس إجراءً اختياريًّا: هو ما يحوّل التجربة من «عمل على جهازي» إلى «أثاث يمكن إعادة استخدامه ونشره». هذا الفصل يشرح صيغ الحفظ، ومتى تُستخدم كلًّا منها، وكيف تُحمَّل الطبقات المخصّصة، وكيف تُستأنف التدريب من نقطة حفظ.

أهداف الفصل

- [ ] تُميّز بين صيغ الحفظ الأربع في Keras 3
- [ ] تبني `ModelCheckpoint` لتوفير النموذج أثناء التدريب
- [ ] تُحمّل نموذجًا فيه طبقات مخصّصة
- [ ] تُستأنف التدريب من نقطة حفظ
- [ ] تُميّز الحفظ الكامل عن الحفظ بلا حالة المحسّن
- [ ] تعرف مخاطر الحفظ ومتى يُفشل

35.1 صيغ الحفظ الأربع

أين تحفظ نموذجك؟

الاختيار الصحيح يوفّر ساعات في الإنتاج

الاستخدام	تحتوي	الأمر	الصيغة
الافتراضي · الخيار الآمن	أوزان + إعدادات + حالة	save()	.keras
متابعة تدريب / تبادل	الأوزان فقط	save_weights()	.weights.h5
خادم TF Serving	رسم بياني قابل للتنفيذ	export()	SavedModel
جوال / مدمج	نسخة مضغوطة	export("m.tflite")	TFLite
تشغيل متعدد المحرّكات	رسم بياني محايد	export("m.onnx")	ONNX

قاعدة Keras 3

«model.save» تعني «model.export» و«model.export» تعني «model.save» — الخلط بينهما يكلّف يومًا.

ملاحظة

صيغة .keras وحدها تحفظ حالة المحسّن (مثل عدّاد خطوات Adam) — وهذا مهمّ عند الاستئناف.

شكل 45 – صيغ الحفظ في Keras 3

Keras 3 تُقدّم أربع طرق للحفظ، لكلّ واحدة استخدام مختلف:

الصيغة	الأمر	ماذا تحتوي	الاستخدام
`.keras`	`model.save()`	أوزان + إعدادات + حالة المحسّن	الافتراضيّ، الخيار الآمن
`.weights.h5`	`model.save_weights()`	الأوزان فقط	استئناف التدريب، تبادل
`SavedModel`	`model.export(format="tf_saved_model")`	رسم بيانيّ قابل للتنفيذ	خادم TF Serving
`.tflite`	`model.export("m.tflite")`	نسخة مضغوطة	جوال، مدمج

> **قاعدة جوهرية:** `.keras` هو الصيغة الافتراضيّة في Keras 3. إن لم تكن متأكّدًا، استخدمها.

35.2 الحفظ الكامل

الحفظ الكامل يعني حفظ **كلّ شيء** – الأوزان، والبنية، وحالة المحسّن (مثل عدّاد الخطوات في Adam):

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بناء نموذج بسيط
model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")

# ── 2. تدريب سريع
x = np.random.rand(200, 16).astype("float32")
y = np.random.rand(200, 1).astype("float32")
model.fit(x, y, epochs=3, verbose=0)

# ── 3. حفظ كامل
model.save("model_full.keras")

# ── 4. تحميل
loaded = keras.saving.load_model("model_full.keras")
# النتيجة المتوقعة: نموذج مُحمَّل، جاهز للاستدلال أو الاستئناف.
```

35.3 الحفظ بلا حالة المحسّن

الحفظ بلا حالة المحسّن (weights only) يُحفظ الأوزان فقط. الفرق: عند التحميل، حالة المحسّن (مثل عزم Adam) مفقودة.

```python
# حفظ الأوزان فقط
model.save_weights("model.weights.h5")

# إنشاء نموذج بنفس البنية ثم تحميل الأوزان
new_model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
new_model.load_weights("model.weights.h5")
# النتيجة المتوقعة: الأوزان مُحمَّلة، وحالة المحسّن مفقودة.
```

قاعدة: `.keras` يحفظ حالة المحسّن؛ `.weights.h5` لا يحفظها. إن كنت تريد الاستئناف، استخدم `.keras`.

35.4 نقطة الحفظ (ModelCheckpoint)

`ModelCheckpoint` تُحفظ النموذج أثناء التدريب – على كلّ دورة، أو عند تحسّن المقياس المُراقَب:

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

# نقطة حفظ عند تحسّن خسارة التحقّق
checkpoint = keras.callbacks.ModelCheckpoint(
    filepath="best_model.keras",        # المسار
    monitor="val_loss",                 # المقياس المُراقَب
    save_best_only=True,                # حفظ الأفضل فقط
    verbose=1,
)

x = np.random.rand(200, 16).astype("float32")
y = np.random.rand(200, 1).astype("float32")
model.fit(x, y, epochs=5, batch_size=32,
          validation_split=0.2, callbacks=[checkpoint], verbose=0)
# ملفّ النتيجة المتوقعة: best_model.keras فيه الأفضل.
```

ملاحظة: `save_best_only=True` يُحفظ فقط عند تحسّن المقياس – وهذا يمنع الكتابة في كلّ دورة.

35.5 قاعدة المسار والصيغة

Keras 3 تفرض قاعدة صارمة: **الامتداد يحدّد الصيغة.**

- `.keras` ← حفظ كامل.

- `.weights.h5` ← أوزان فقط.

إذا مرّرت `save_weights_only=True` ومسارًا ينتهي بـ `.keras`، يرفع Keras `ValueError`.

35.6 الطبقات المخصّصة في الحفظ

الطبقات المخصّصة تُحمَّل عبر `custom_objects`:

PYTHON

```python
import keras

class MyDense(keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(**kwargs)
        self.units = units

    def build(self, input_shape):
        self.w = self.add_weight(shape=(input_shape[-1], self.units),
                                 initializer="glorot_uniform")
        super().build(input_shape)

    def call(self, inputs):
        return inputs @ self.w

    def get_config(self):
        return {**super().get_config(), "units": self.units}

# الحفظ
m = keras.Sequential([keras.Input(shape=(8,)), MyDense(4)])
m.save("custom_model.keras")

# التحميل
m_loaded = keras.saving.load_model("custom_model.keras",
                                   custom_objects={"MyDense": MyDense})
# النتيجة المتوقعة: الطبقة المخصّصة مُحمَّلة بنجاح.
```

35.7 استئناف التدريب

الاستئناف يعني: تحميل النموذج، وربط `ModelCheckpoint`، ومتابعة التدريب:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# نموذج مدرَّب مُسبقًا (محاكاة)
model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")
x = np.random.rand(200, 16).astype("float32")
y = np.random.rand(200, 1).astype("float32")
model.fit(x, y, epochs=2, verbose=0)

# حفظ
model.save("checkpoint.keras")

# استئناف
resumed = keras.saving.load_model("checkpoint.keras")
resumed.fit(x, y, epochs=2, verbose=0)   # متابعة
# النتيجة المتوقعة: خسارة تنخفض أكثر مقارنة بالنموذج المُحمَّل.
```

ملاحظة: الاستئناف يعمل جيّدًا إن كان `optimizer` حالةً قابلةً للحفظ – وهذا صحيح في `.keras`.

35.8 مخاطر الحفظ

المخاطر	السبب	الحلّ
نموذج لا يُحفظ	مسار بامتداد خاطئ	استخدم `.keras`
نموذج مُحمَّل يُعطي نتائج مختلفة	طبقة مخصّصة بلا `custom_objects`	مرّر `custom_objects`
حفظ كلّ دورة يستهلك مساحة	بلا `save_best_only`	فعّلها، أو استخدم عدّة ملفّات
الطبقة المخصّصة بلا `get_config`	إسقاط الإعدادات	أضِفها

35.9 مثال محلول: حفظ واستئناف نموذج تجزئة

المهمة: تدريب نموذج تجزئة U-Net على 5000 صورة، والتدريب يحتاج 30 دورة.

الخطوة 1 – نقطة حفظ. `ModelCheckpoint` بحفظ الأفضل فقط.

الخطوة 2 – انقطاع. بعد الدورة 12، انقطع التدريب.

الخطوة 3 – استئناف. تحميل `best_model.keras` ومتابعة التدريب. الخسارة تستمرّ في الانخفاض – لا يبدأ من صفر.

الخطوة 4 – الحفظ النهائي. بعد الدورة 30، حفظ النموذج النهائيّ في `final.keras` .

الخطوة 5 – القرار. `.keras` هو الصيغة الصحيحة – تحتوي كلّ شيء وتُستأنف مباشرةً.

35.10 وضع الأمان (Safe Mode)

عند التحميل، Keras 3 يُفعّل **وضع الأمان** افتراضيًّا: لا يُنفّذ أيّ كود مضمّن في الملفّ. هذا يحميك من ملفّ `.keras` مُزوَّر:

```python
import keras

# التحميل الآمن (الافتراضيّ)
m = keras.saving.load_model("model_full.keras", safe_mode=True)

# يجب تعطيل الوضع، مثل Lambda) إن كان الملفّ يُنشيئ كائنات مخصّصة:
# m = keras.saving.load_model("m.keras", safe_mode=False)
# النتيجة المتوقعة: التحميل الآمن ينجح بلا تنفيذ أيّ كود.
```

> **تحذير:** `safe_mode=False` يُنفّذ أيّ كود في الملفّ – لا تستخدمه إلّا مع ملفّات وثيقة المصدر. طبقة `Lambda` مثال على ما يُرفض في الوضع الآمن.

لماذا يُرفض `Lambda`؟ لأنّها تُخزّن الشيفرة المصدريّة للدالّة في الملفّ، وتُنفّذها عند التحميل. الطبقة المخصّصة (مع `get_config`) هي البديل الآمن:

```python
import keras

class DoubleLayer(keras.layers.Layer):
    """طبقة تُضاعف المُدخل – بديل آمن عن Lambda(lambda x: x * 2)."""

    def call(self, inputs):
        return inputs * 2.0

    def get_config(self):
        return super().get_config()

m = keras.Sequential([keras.Input(shape=(8,)), DoubleLayer()])
m.save("safe.keras")
m2 = keras.saving.load_model("safe.keras",
                             custom_objects={"DoubleLayer": DoubleLayer})
# النتيجة المتوقعة: الطبقة تُحمَّل في الوضع الآمن – لأنّها لا تُنفّذ كودًا مضمّنًا.
```

35.11 تنظيم نقاط الحفظ

في تدريب طويل، تنظيم نقاط الحفظ يوفّر مساحة ويسهّل الاستئناف:

النمط	المسار	الاستخدام
الأفضل فقط	`best.keras`	النموذج النهائيّ
كلّ N دورة	`epoch_{epoch:03d}.keras`	استئناف من أيّ نقطة
دوريّ + الأفضل	الاثنان معًا	تدريب طويل مع انقطاعات

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)
model = keras.Sequential([keras.Input(shape=(16,)),
                          keras.layers.Dense(32, activation="relu"),
                          keras.layers.Dense(1)])
model.compile(optimizer="adam", loss="mse")

# نقطة حفظ دوريّة (كلّ دورة، بمُعرّف رقميّ)
periodic = keras.callbacks.ModelCheckpoint(
    filepath="ckpts/epoch_{epoch:03d}.keras",
    save_best_only=False, verbose=0)
# نقطة حفظ للأفضل
best = keras.callbacks.ModelCheckpoint(
    filepath="best.keras", monitor="val_loss",
    save_best_only=True, verbose=0)

x = np.random.rand(200, 16).astype("float32")
y = np.random.rand(200, 1).astype("float32")
model.fit(x, y, epochs=3, batch_size=32, validation_split=0.2,
          callbacks=[periodic, best], verbose=0)
# النتيجة المتوقعة: ملفّات مرقّمة + ملفّ الأفضل.
```

نصيحة: في تدريب طويل، احتفظ بآخر 3 نقاط دوريّة فقط (احذف الأقدم عبر سكربت) — هذا يوازن بين المساحة وقابلية الاستئناف.

35.12 مثال: حفظ الأوزان لنقلها بين بنيتين

الحفظ بلا حالة المحسّن مفيد عند نقل الأوزان بين بنيتين متوافقتين:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. نموذج مصدر
src = keras.Sequential([keras.Input(shape=(16,)),
                        keras.layers.Dense(32, activation="relu"),
                        keras.layers.Dense(1)])
src.compile(optimizer="adam", loss="mse")
src.save_weights("src.weights.h5")

# ── 2. نموذج هدف بنفس البنية (بنية جديدة في الشيفرة)
dst = keras.Sequential([keras.Input(shape=(16,)),
                        keras.layers.Dense(32, activation="relu"),
                        keras.layers.Dense(1)])
dst.load_weights("src.weights.h5")

# ── 3. تحقّق: الأوزان متطابقة
w_src = src.get_weights()
w_dst = dst.get_weights()
assert all(np.array_equal(a, b) for a, b in zip(w_src, w_dst))
# النتيجة المتوقعة: الأوزان منقولة بالكامل.
```

35.13 أخطاء شائعة في هذا الفصل

إنذار: استخدام `model.save("m.h5")` في Keras 3. صيغة `.h5` ليست الصيغة الافتراضيّة، وقد تُفقد الميزات الجديدة. استخدم `.keras` .

تحذير: `model.save_weights("m.keras")` – وهذا خطأ: `save_weights` يتطلّب `.weights.h5` .

تنبيه: تمرير `custom_objects` بلا الصنف المخصّص. النتيجة: خطأ تحميل. مرّر كلّ صنف مخصّص.

ملاحظة: `model.save("m")` بلا امتداد. Keras 3 يتطلّب امتدادًا صريحًا – استخدم `.keras` .

قاعدة: `.keras` يحفظ كلّ شيء؛ `.weights.h5` يحفظ الأوزان فقط. إن أردت استئناف تدريب، استخدم `.keras` .

تحذير: `save_best_only=True` على مقياس لا يتحسّن دائمًا (مثل `loss` على بيانات مُضخّمة بالضجيج). راجع المقياس قبل اعتماد النموذج.

التمارين

1. **برمجة.** ابنِ نموذجًا بسيطًا واحفظه في `.keras` ، ثم حمّله واطبع `summary()` . (المتوقع: نفس البنية.)
2. **برمجة.** احفظ الأوزان فقط في `.weights.h5` ، ثم أنشئ نموذجًا بنفس البنية وحمّل الأوزان. (المتوقع: الأوزان مُحمَّلة، وحالة المحسّن مفقودة.)
3. **برمجة.** طبّق `ModelCheckpoint` بحفظ الأفضل فقط على خسارة التحقّق. (المتوقع: ملفّ واحد يُحدَّث عند تحسّن المقياس.)
4. **إصلاح خطأ.** الكود التالي يُفشل:

```python
model.save_weights("m.keras")
```

(المتوقع: `save_weights` يتطلّب `.weights.h5` – استخدم `model.save("m.keras")` للحفظ الكامل.) 5. **برمجة.** ابنِ طبقة مخصّصة واحفظ النموذج في `.keras`، ثم حمّله بـ `custom_objects`. (المتوقع: التحميل ينجح.) 6. **برمجة.** قارن زمن الحفظ بين `.keras` و`.weights.h5`. (المتوقع: `.weights.h5` أسرع لأنّه يحفظ الأوزان فقط.) 7. **تحليل.** لماذا `.keras` أفضل من `.h5` في Keras 3؟ (المتوقع: لأنّ `.keras` هو الصيغة الافتراضيّة، ويحفظ كلّ شيء ويُستأنف.) 8. **تصميم.** تدريب يستغرق 12 ساعة، والكهرباء قد تنقطع. صمّم خطّة حفظ. (المتوقع: `ModelCheckpoint` بحفظ الأفضل + حفظ دوريّ.) 9. **برمجة.** طبّق `keras.saving.load_model` على نموذج بطبقة مخصّصة، واطبع الشكل. (المتوقع: التحميل ينجح بـ 10 (.) `custom_objects`. **تصميم.** نظام نشر على جوال. أيّ صيغة تختار؟ (المتوقع: `.tflite` عبر `model.export("m.tflite")` – راجع الفصل 37.) 11. **برمجة.** استأنف تدريبًا من نقطة حفظ وقارن الخسارة مع النموذج المُحمَّل. (المتوقع: الاستئناف يُعطي خسارة أدنى.) 12. **تحليل.** لماذا `save_best_only=True` لا يعمل على مقياس لا يتحسّن دائمًا؟ (المتوقع: لأنّ الكتابة تحدث فقط عند التحسّن؛ إن لم يتحسّن المقياس، لن يُحدَّث الملف.)

ملخص الفصل

- أربع صيغ: `.keras` (كلّ شيء)، `.weights.h5` (أوزان)، `.tflite`، `SavedModel`.
- `.keras` هو الافتراضيّ في Keras 3 – يحفظ حالة المحسّن ويُستأنف.
- `ModelCheckpoint` يُحفظ أثناء التدريب، و`save_best_only=True` يُحفظ عند تحسّن المقياس.
- الطبقات المخصّصة تُحمَّل عبر `custom_objects`.
- الامتداد يحدّد الصيغة – لا تخلط بينها.
- الاستئناف من `.keras` يعمل مباشرةً.

الفصل 36: تسريع التدريب

بطاقة الفصل

الفصل 36

مهمّة هذا الفصل

يسرّع التدريب: دقّة مختلطة، XLA، تحسين خطّ البيانات، وتجميع التدرّج.

ماذا تكسب منه

تضاعف السرعة غالبًا بلا خسارة دقّة، وتستغلّ العتاد الذي تدفع ثمنه فعلًا.

الدالة	دورها	متى تستخدمها
`keras.mixed_precision.set_global_policy()`	تفعيل الدقّة المختلطة	على GPU حديث
`keras.optimizers.LossScaleOptimizer`	مقياس خسارة	مع float16
`model.compile(jit_compile=True)`	تجميع XLA	للنماذج الثابتة البنية
`keras.distribution.list_devices()`	قائمة الأجهزة المتاحة	قبل التدريب الموزّع
`keras.layers.GlobalAveragePooling2D()`	تجميع	في نماذج القياس

قصة مفتاحية: باحث ينفّذ تدريبًا على مليون صورة. على جهازه، الدورة تُستغرق 40 دقيقة، فيحتاج 30 يومًا لإتمام 30 دورة. لكنّ المشكلة ليست في الشبكة – بل في **كيفية** تنفيذ الحساب: الدقّة الكاملة (float32) تستهلك ضعف الذاكرة، وCPU يُنفّذ عمليّات ما كان يمكن لـ GPU أن يُنفّذها بمرّات أسرع، وخطّ البيانات يُعطي GPU وحدًا واحدًا في كلّ مرة. التسريع ليس سحرًا – بل خمسة محاور مُقاسة.

أهداف الفصل

- [ ] تُطبّق الدقّة المختلطة (mixed precision) وتعرف متى تفيد
- [ ] تُفعّل XLA/JIT لتحسين الرسم البيانيّ
- [ ] تُحسّن خطّ البيانات بالذاكرة المؤقتة والجلب المسبق
- [ ] تُطبّق التجميع بالتدرّج (gradient accumulation) لدفعات كبيرة
- [ ] تُقاس مكسب كلّ محور منفصلًا
- [ ] تعرف حدود كلّ تسريع ومتى يضرّ

36.1 الدقّة المختلطة

شكل 46 – الدقّة المختلطة: float16 + float32

الدقّة المختلطة تعني: الحساب في `float16` (نصف الذاكرة، مرّتان أسرع على GPU)، مع أوزان رئيسيّة في `float32` (لا تُفقد الدقّة في التحديث):

النوع	ماذا يُخزّن	الذاكرة
`float32`	كلّ شيء (الافتراضيّ)	4 بايت لكلّ رقم
`float16` (حساب)	عمليّات الالتفاف والجمع	2 بايت
`float32` (أوزان)	النسخة الرئيسيّة للأوزان	4 بايت

36.2 تفعيل الدقّة المختلطة

PYTHON

```python
import keras

# ── 1. ضبط الدقّة المختلطة قبل بناء النموذج
keras.mixed_precision.set_global_policy("mixed_float16")

# ── 2. بناء النموذج (آخر طبقة تحتفظ بـ float32)
inp = keras.Input(shape=(32, 32, 3))
x = keras.layers.Conv2D(32, 3, padding="same", activation="relu")(inp)
x = keras.layers.MaxPooling2D(2)(x)
x = keras.layers.Flatten()(x)
out = keras.layers.Dense(10)(x)     # لوجيتس — بلا تنشيط
model = keras.Model(inp, out)

# ── 3. الخسارة من اللوجيتس (لا softmax)
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)

# ── 4. (اختياريّ) إعادة ضبط بعد التدريب
# keras.mixed_precision.set_global_policy("float32")
# النتيجة المتوقعة: تدريب أسرع بـ 1.5-2× على GPU.
```

إنذار: في `mixed_float16`، يجب أن تكون آخر طبقة بلا تنشيط `softmax`، وأن تُستخدم الخسارة من اللوجيتس (`from_logits=True`). خلاف ذلك، الدقّة تُفقد.

تحذير: على CPU، الدقّة المختلطة **لا** تُسرّع — بل قد تُبطئ. التسريع موجود على GPU فقط (مخصّصًا Tensor Cores).

36.3 XLA / JIT

XLA (Accelerated Linear Algebra) هي مُجمّع تُدمج عمليّات الرسم البيانيّ في عمليّة واحدة، فيُقلّص الحِمل بين العمليّات:

PYTHON

```
import keras

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    jit_compile=True,        # تفعيل XLA
)
# النتيجة المتوقعة: تسريع 10–30% على النماذج ذات الرسوم البيانيّة الكبيرة.
```

ملاحظة: XLA تعمل على TensorFlow وJAX. في PyTorch، مُجمّع TorchScript يُعطي تسريعًا مشابهًا.

36.4 تحسين خطّ البيانات

خطّ البيانات يُعدّ سببًا شائعًا للتسرّب: GPU ينتظر وحدة CPU. الحلّ ثلاثة أدوات:

الأداة	ماذا تفعل	متى تستخدمها
`cache()`	يُخزّن البيانات في الذاكرة	البيانات تناسب الذاكرة
`prefetch()`	يُجهّز الدفعة التالية بينما GPU يُحسب	دائمًا
`interleave()`	يُقرأ من عدّة ملفّات بالتوازي	بيانات في ملفّات متعدّدة

PYTHON

```python
import keras
import numpy as np

# ── 1. بيانات كبيرة محاكاة
n = 2000
x = np.random.rand(n, 32, 32, 3).astype("float32")
y = np.random.randint(0, 10, size=(n,))

# ── 2. خطّ بيانات محسّن
(x_train, y_train), _ = keras.datasets.mnist.load_data()   # بيانات حقيقيّة
x_train = x_train.astype("float32") / 255.0

# خطّ بيانات tf.data (خاصّ بمحرّك TensorFlow)
import tensorflow as tf
AUTOTUNE = tf.data.AUTOTUNE

batch_size = 32
ds = (
    keras.utils.timeseries_dataset_from_array(x_train, y_train,
                                              sequence_length=10)
    .cache()                 # يُخزّن في الذاكرة
    .prefetch(AUTOTUNE)      # جلب مسبق
)
# النتيجة المتوقعة: زمن الدورة أقلّ مقارنة بلا cache/prefetch.
```

نصيحة: `AUTOTUNE` يُحدّد عدد العيّنات المُجهَّزة تلقائيًّا – لا تُحدّده يدويًّا.

36.5 التجميع بالتدرّج (Gradient Accumulation)

التجميع بالتدرّج يعني: حساب تدرّج عدّة دفعات صغيرة، ثم تطبيق التحديث مرة واحدة – فيُقلّد دفعة كبيرة:

$$\theta \leftarrow \theta - \eta \cdot \frac{1}{k} \sum_{i=1}^{k} \nabla_\theta L_i$$

حيث k عدد الدفعات المُجمَّعة. الفائدة: تدريب نموذج بحجم دفعة فعليّة كبير على GPU ذاكرة صغيرة.

PYTHON

```python
import keras

model.compile(
    optimizer=keras.optimizers.Adam(1e-3, gradient_accumulation_steps=4),  # تجميع 4 دفعات صغيرة في تحديث واحد: خيار المُحسّن لا compile
    loss="sparse_categorical_crossentropy",
)
# النتيجة المتوقعة: نموذج يعمل كأنّ الدفعة 4× أكبر، بذاكرة أقلّ.
```

36.6 مثال محلول: تسريع تدريب CNN

المهمة: تدريب CNN على 50 ألف صورة، الدورة تُستغرق 40 دقيقة.

الخطوة 1 – خطّ الأساس. float32، بلا تحسين. 40 دقيقة للدورة.

الخطوة 2 – تحسين خطّ البيانات. `prefetch(AUTOTUNE)` + `cache()`. 32 دقيقة. **تحسّن 20%.**

الخطوة 3 – XLA. `jit_compile=True`. 28 دقيقة. **تحسّن 12%.**

الخطوة 4 – الدقّة المختلطة. `mixed_float16`. 18 دقيقة. **تحسّن 36%.**

الخطوة 5 – الدقّة المختلطة + XLA. 15 دقيقة. **تحسّن 62%.**

الخطوة 6 – القرار. الدقّة المختلطة + XLA + خطّ بيانات محسّن = 2.7× تسريع، بلا تغيير في النموذج.

36.7 جدول القرار

المحور	تسريع نموذجيّ	كلفة	متى تستخدمه
خطّ بيانات محسّن	10-25%	صفر	دائمًا
XLA / JIT	10-30%	صفر (بلا تخصيص)	الرسوم البيانيّة الكبيرة
الدقّة المختلطة	1.5-2×	منخفضة	GPU فقط، نموذج كبير
التجميع بالتدرّج	لا يُسرّع – يُكبّر الدفعة	ذاكرة	دفعات كبيرة على GPU صغيرة
التوازي عبر GPUs	2-8×	مرتفعة	بيانات ونموذج ضخمان

> **قاعدة:** كلّ محور يُقاس على بياناتك. تسريع XLA على نموذج صغير قد يكون أقلّ من 5%.

36.8 التدريب الموزّع (Multi-Device)

Keras 3 تُقدّم واجهة موحّدة للتوزيع عبر `keras.distribution` – نفس الشيفرة تعمل على عدّة GPUs:

```python
import keras

# ── 1. تعريف شبكة الأجهزة (عيّنة على جهاز بعدّة GPUs في الإنتاج:)
# devices = keras.distribution.list_devices()
# mesh = keras.distribution.DeviceMesh(
#     shape=(1, len(devices)), axis_names=("data", "model"),
#     devices=devices)
# layout = keras.distribution.TensorLayout("data", mesh=mesh)

# ── 2. النموذج نفسه — Keras يوزّع الدفعات تلقائيًّا
model = keras.Sequential([
    keras.Input(shape=(32, 32, 3)),
    keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
# النتيجة المتوقعة: نفس الشيفرة تعمل على جهاز واحد أو عدّة أجهزة.
```

النمط	ماذا يوزّع	متى تستخدمه
توازي البيانات	الدفعات عبر الأجهزة	الأشهر – النموذج يُنسخ على كلّ جهاز
توازي النموذج	طبقات النموذج	نموذج لا يتّسع لجهاز واحد
توازي الأوزان	مصفوفات الأوزان	نماذج ضخمة جدًا

قاعدة: ابدأ بتوازي البيانات – أبسط وأكثر فائدة. توازي النموذج والأوزان يُستخدم فقط حين لا يتّسع النموذج لجهاز واحد.

36.9 قياس الأداء (Profiling)

لا تُحسّن بالتخمين – قِس. أدوات القياس الثلاث:

```python
import keras
import numpy as np
import time

keras.utils.set_random_seed(42)
model = keras.Sequential([keras.Input(shape=(16,)),
                          keras.layers.Dense(64, activation="relu"),
                          keras.layers.Dense(1)])
model.compile(optimizer="adam", loss="mse")
x = np.random.rand(1000, 16).astype("float32")
y = np.random.rand(1000, 1).astype("float32")

# ── 1. قياس زمن الدورة
t0 = time.perf_counter()
model.fit(x, y, epochs=1, batch_size=32, verbose=0)
epoch_time = time.perf_counter() - t0
print(f"epoch time = {epoch_time:.3f}s")

# ── 2. قياس زمن الاستدلال
t0 = time.perf_counter()
_ = model.predict(x[:100], verbose=0)
infer_time = time.perf_counter() - t0
print(f"inference 100 = {infer_time * 1000:.1f}ms")
# النتيجة المتوقعة: أرقام فعلية تُقارن قبل وبعد كلّ تحسين.
```

> **نصيحة:** قِس ثلاث مرّات واقرأ الوسيط – أوّل تشغيل يحتوي تهيئة (warm-up) لا يُعبّر عن الزمن الحقيقيّ.

مصفوفة القياس قبل وبعد كلّ تحسين:

المقياس	قبل	بعد	المكسب
زمن الدورة	40 دقيقة	15 دقيقة	×2.7
زمن استدلال 100 عيّنة	320 ms	110 ms	×2.9
ذاكرة التدريب	11 GB	6.2 GB	×1.8

36.10 أخطاء شائعة في هذا الفصل

إنذار: `mixed_float16` مع آخر طبقة فيها `softmax` . النتيجة: دقّة تهبط، والخسارة قد تصبح NaN. القاعدة: آخر طبقة بلا تنشيط، والخسارة من اللوجيتس.

تحذير: `mixed_float16` على CPU. لا تسرّع – قد يُبطئ. التسريع موجود على GPU فقط.

تنبيه: تفعيل XLA على نموذج فيه عمليّات ديناميكيّة (مثل `switch`). النتيجة: خطأ أو تسريع محدود.

ملاحظة: `cache()` على بيانات كبيرة. إذا تجاوزت الذاكرة، انفجرت العملية. استخدم `cache(filename)` للحفظ على القرص.

قاعدة: `prefetch(AUTOTUNE)` – دائمًا. هذا هو أوّل تحسين بلا كلفة.

تحذير: التجميع بالتدرّج لا يُسرّع – يُكبّر الدفعة الفعليّة. إن كان الزمن حرجًا، فهذا ليس الحلّ.

التمارين

1. **برمجة.** طبّق `mixed_float16` على نموذج CNN واطبع `model.summary()`. (المتوقع: آخر طبقة في `float32`.)
2. **برمجة.** طبّق `jit_compile=True` واطبع زمن الدورة قبل وبعد. (المتوقع: تحسّن 10-30%.)
3. **برمجة.** قارن زمن الدورة بين خطّ بيانات بلا `prefetch` ومعه. (المتوقع: `prefetch` أسرع.)
4. **إصلاح خطأ.** النموذج التالي يُعطي NaN مع `mixed_float16`:

```python
out = keras.layers.Dense(10, activation="softmax")(x)
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

(المتوقع: آخر طبقة فيها softmax – أزل التنشيط واستخدم `from_logits=True`.) 5. **برمجة.** طبّق `cache()` على خطّ بيانات واطبع زمن الدورة. (المتوقع: تحسّن إن كانت البيانات تناسب الذاكرة.) 6. **برمجة.** طبّق `gradient_accumulation_steps=4` على نموذج، واطبع حجم الذاكرة المُستهلَكة. (المتوقع: أقلّ من دفعة كبيرة فعليّة.) 7. **تحليل.** لماذا لا تُسرّع الدقّة المختلطة على CPU؟ (المتوقع: CPU لا يملك Tensor Cores – والحساب في `float16` يعني تحويلات إضافيّة.) 8. **تصميم.** تدريب على GPU ذاكرة 8GB ونموذج يحتاج 16GB. أيّ تسريع تُطبّق؟ (المتوقع: الدقّة المختلطة + التجميع بالتدرّج.) 9. **برمجة.** قارن زمن الاستدلال على GPU بين `float32` و`mixed_float16`. (المتوقع: `mixed_float16` أسرع على 10 (.GPU. 10. **تحليل.** لماذا `prefetch` مهمّ جدًّا؟ (المتوقع: لأنّ GPU يُحسب بينما CPU يُجهّز الدفعة التالية – بلا `prefetch` ينتظر GPU.) 11. **تصميم.** نظام نشر على جهاز بذاكرة صغيرة. أيّ تسريع تُطبّق؟ (المتوقع: خطّ بيانات محسّن + `cache` (بلا mixed_float16 على CPU).) 12. **برمجة.** قارن زمن الدورة بين float32 وmixed_float16 على GPU. (المتوقع: x2-1.5 تسريع.)

ملخص الفصل

- الدقّة المختلطة تُسرّع GPU مرّتين، وتتطلّب آخر طبقة بلا تنشيط.
- XLA تُدمج العمليّات وتُسرّع 10-30% على الرسوم البيانيّة الكبيرة.
- خطّ البيانات: `cache()` + `prefetch(AUTOTUNE)` – أوّل تحسين بلا كلفة.
- التجميع بالتدرّج يُكبّر الدفعة الفعليّة بلا ذاكرة إضافيّة.
- كلّ محور يُقاس على بياناتك – لا تُطبّق تسريعًا بلا قياس.
- على CPU، الدقّة المختلطة لا تُسرّع.

الفصل 37: التصدير والنشر

بطاقة الفصل

الفصل 37

مهمّة هذا الفصل

يصدّر النموذج ويخدمه: LiteRT وONNX وFastAPI، مع تكميم بعد التدريب.

ماذا تكسب منه

ينتقل النموذج من دفتر إلى خدمة حقيقية بأداء مقيس وحجم مضبوط.

الدالة	دورها	متى تستخدمها
`model.export()`	تصدير SavedModel	قبل التحويل إلى صيغ أخرى
`keras.Input(shape=...)`	تثبيت شكل المدخل	قبل التصدير
`keras.InputSpec(dtype, shape)`	وصف مدخل الخدمة	للتحقّق من صيغة الطلب
`model.predict()`	خدمة التنبّؤ	داخل نقطة النهاية
`keras.utils.set_random_seed()`	بذرة ثابتة	لثبات نتائج اختبار الخدمة

قصة مفتاحية: فريق يُدرّب نموذجًا على دفتر Jupyter، ويصل إلى دقّة 95%. ثمّ يحاول نشره على خادم – ويفشل. النموذج في صيغة `.keras` والخادم يتطلّب `SavedModel`. أو يحاول نشره على تطبيق جوال – ويفشل لأنّ الحجم 500 ميغابايت. النشر ليس خطوةً تقنيّةً صغيرة: هو القرار الذي يحدّد من سيستخدم النموذج، وعلى أيّ جهاز، وبأيّ كلفة.

أهداف الفصل

- [ ] تُميّز بين صيغ التصدير الأربع وتُختار بحسب الهدف
- [ ] تُصدّر نموذجًا إلى LiteRT/TFLite وتُكمّمه
- [ ] تُصدّر إلى ONNX وتُشغّله على ONNX Runtime
- [ ] تبني خدمة FastAPI تُقدّم النموذج
- [ ] تُطبّق التكميم بعد التدريب لتقليل الحجم
- [ ] تعرف أثر كلّ صيغة في الحجم وزمن الاستدلال

37.1 من النموذج إلى الخدمة

الأداء	الحجم	الصيغة	الهدف
ممتاز	كامل	TF SavedModel / Keras	خادم GPU
جيّد	متوسط	TF.js	متصفّح / Java
ممتاز	صغير جدًا	LiteRT (TFLite)	هاتف Android/iOS
ممتاز	صغير	OpenVINO IR	معالج Intel
جيّد	كامل	ONNX / Torch	بيئة بالشحن

ابدأ دائمًا بـ: model.export() ثم اختر الصيغة المناسبة لبيئة الهدف.

شكل 47 – من النموذج إلى الخدمة

خطّ النشر يمرّ بأربع محطّات:

1. **التدريب** – نموذج في `.keras` (Keras أو في الذاكرة).
2. **التصدير** – تحويل إلى صيغة قابلة للنشر (LiteRT، ONNX، SavedModel).

3. **الخدمة** – غلاف برمجيّ (FastAPI أو TF Serving) يُقبل طلبًا ويُعطي ردًّا.

4. **المراقبة** – زمن الاستدلال، ومعدّل الأخطاء، وانحراف البيانات.

37.2 صيغ التصدير

الصيغة	الهدف	الحجم	الاستخدام
`SavedModel`	خادم TensorFlow	كبير	TF Serving، REST/gRPC
`LiteRT / TFLite`	جوال ومدمج	صغير	تطبيقات موبايل، أجهزة IoT
`ONNX`	محرّك متعدّد	متوسّط	تشغيل على محرّكات مختلفة
`TorchScript`	PyTorch	متوسّط	نشر على خوادم PyTorch

> **قاعدة اختيار:** الهدف يحدّد الصيغة: جوال → LiteRT؛ خادم TensorFlow → SavedModel؛ محرّك متعدّد → ONNX؛ خادم PyTorch → TorchScript.

37.3 التصدير إلى LiteRT/TFLite

LiteRT (الاسم الجديد لـ TFLite) هي صيغة النشر على الجوال والمدمج. Keras 3 تُصدّر عبر `model.export`:

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. نموذج بسيط
model = keras.Sequential([
    keras.Input(shape=(32, 32, 3)),
    keras.layers.Rescaling(1.0 / 255.0),
    keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")

# ── 2. تصدير إلى LiteRT (في الإنتاج تتطلّب حزمة LiteRT)
# model.export("model.tflite")
# ملفّ النتيجة المتوقعة: .tflite بحجم أقلّ بكثير من .keras.
```

37.4 التكميم بعد التدريب (Post-Training Quantization)

التكميم يُحوّل الأوزان من `float32` (4 بايت) إلى `int8` (1 بايت) – فيقلّص الحجم 4× ويُسرّع الاستدلال:

التكميم	ماذا يُكمّم	الحجم	الدقّة
`int8` (أوزان فقط)	الأوزان	4× أصغر	انخفاض < 1%
`int8` (كامل)	الأوزان + التنشيطات	4× أصغر	انخفاض 1-3%
`float16`	الأوزان	2× أصغر	بلا انخفاض

> **نصيحة:** ابدأ بالتكميم بـ `int8` على الأوزان فقط – هذا يعطي 4× تقليل حجم بلا انخفاض دقّة محسوس.

37.5 التصدير إلى ONNX

ONNX (Open Neural Network Exchange) هي صيغة محايدة تعمل على عدّة محرّكات (ONNX Runtime، TensorRT، OpenVINO):

PYTHON

```python
import keras

# ── 1. نموذج بسيط
model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

# ── 2. تصدير إلى ONNX (في الإنتاج keras-onnx تتطلّب حزمة)
# import tf2onnx
# spec = (keras.InputSpec(shape=(None, 16), dtype="float32"),)
# tf2onnx.convert.from_keras(model, input_signature=spec,
#                            output_path="model.onnx")
# ملفّ .onnx النتيجة المتوقعة: قابل للتشغيل على ONNX Runtime.
```

37.6 خدمة FastAPI

فيما يلي خدمة FastAPI تُقدّم نموذجًا مُدرَّبًا مسبقًا:

PYTHON

```python
# ── 1. ملفّ app.py
import keras
import numpy as np
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# ── 2. تحميل النموذج مرة واحدة عند بدء الخدمة
model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="mse")

class Features(BaseModel):
    values: list[float]

@app.post("/predict")
def predict(f: Features):
    """تلقّي 16 ميزة وإعطاء احتمال."""
    x = np.array(f.values, dtype="float32").reshape(1, -1)
    prob = float(model.predict(x, verbose=0)[0][0])
    return {"probability": prob, "threshold": 0.5,
            "label": int(prob ≥ 0.5)}
# النتيجة المتوقعة: JSON يحتوي probability وlabel.
```

نصيحة: تحميل النموذج مرة واحدة عند بدء الخدمة – لا في كلّ طلب. هذا يُقلّص زمن الاستدلال من ثوانٍ إلى ميلّي ثواني.

37.7 مثال محلول: نشر نموذج على تطبيق جوال

المهمة: نشر نموذج تصنيف صور على تطبيق جوال. النموذج بحجم 90 ميغابايت، والجوال يتطلّب أقلّ من 20 ميغابايت.

الخطوة 1 – قياس الحجم. النموذج الأصليّ (float32) = 90 MB.

الخطوة 2 – تصدير إلى `model.export("model.tflite")` LiteRT. الحجم = 88 MB – بلا انخفاض.

الخطوة 3 – تكميم أوزان `int8` . الحجم = 23 MB. **انخفاض ×4** – لكنّ الدقّة انخفضت من 94% إلى 93.5%.

الخطوة 4 – تكميم كامل (أوزان + تنشيطات). الحجم = 22 MB، والدقّة = 92%.

الخطوة 5 – تقليل النموذج. تقليص عدد القنوات (نصف). الحجم = 11 MB، والدقّة = 91%.

الخطوة 6 – القرار. نموذج مُقلَّص + تكميم أوزان `int8` = 11 MB بدقّة 92% – أداء كافٍ وحجم مقبول.

37.8 أثر التكميم على الأداء

النموذج	الحجم	زمن الاستدلال (CPU جوال)	الدقّة
float32	90 MB	250 ms	94%
LiteRT float32	88 MB	220 ms	94%
int8 (أوزان)	23 MB	120 ms	93.5%
int8 (كامل)	22 MB	95 ms	92%
مُقلَّص + int8	11 MB	60 ms	91%

قاعدة: التكميم يُقلّص الحجم ويُسرّع الاستدلال – لكنّه يكلّف دقّة. قِس الدقّة بعد التكميم لا قبلها.

37.9 مراقبة الخدمة

بعد النشر، تُراقب ثلاثة مؤشّرات:

1. **زمن الاستدلال** – P50/P95/P99.
2. **معدّل الأخطاء** – 5xx/timeouts.
3. **انحراف البيانات** – توزيع المُدخلات والمُخرَجات.

تحذير: نموذج لا يُراقَب ينهار بصمت. ابدأ بأربعة مقاييس فقط (راجع الفصل 43).

37.10 أخطاء شائعة في هذا الفصل

إنذار: استخدام `model.save("m.tflite")`. النتيجة: Keras لا يعرف الصيغة. استخدم `model.export("m.tflite")`.

تحذير: تحميل النموذج في كلّ طلب. النتيجة: زمن استدلال بالثواني. تحميل مرة واحدة عند بدء الخدمة.

تنبيه: تكميم كامل بلا قياس. النتيجة: انخفاض دقّة غير مقبول. قِس الدقّة بعد كلّ مستوى تكميم.

ملاحظة: تمرير بيانات مُطبَّعة في الخدمة والنموذج لا يتوقّعها. راجع: هل كانت المعالجة داخل النموذج أم خارجه؟

قاعدة: اختَر الصيغة بحسب الهدف، لا بحسب أعلى رقم في ورقة. جوال → LiteRT؛ محرّك متعدّد → ONNX.

تحذير: نموذج بلا مراقبة. الفشل في الإنتاج يظهر ببطء – انحراف البيانات يُقلّص الدقّة بمرّة.

التمارين

1. **برمجة.** ابنِ نموذجًا بسيطًا واحفظه في `.keras` واطبع الحجم. (المتوقع: الحجم يتناسب مع عدد الأوزان.)
2. **برمجة.** قارن الحجم بين `.keras` و `.weights.h5` . (المتوقع: `.weights.h5` أصغر.)
3. **إصلاح خطأ.** الكود التالي يُفشل:

```python
model.save("m.tflite")
```

(المتوقع: استخدم `model.export("m.tflite")` – لا 4 (. `save` . **تصميم.** نشر نموذج على تطبيق جوال بحجم 100 MB. أيّ خطوات؟ (المتوقع: تصدير LiteRT + تكميم أوزان `int8` + تقليص النموذج.) 5. **برمجة.** طبّق تكميم أوزان `int8` على نموذج، واطبع الحجم قبل وبعد. (المتوقع: 4× أصغر.) 6. **تحليل.** لماذا تُقلّص التكميم الحجم 4×؟ (المتوقع: من 4 بايت (`float32`) إلى 1 بايت (7 (.(`int8` . **برمجة.** قارن زمن الاستدلال على CPU بين `float32` و `int8` . (المتوقع: `int8` أسرع.) 8. **تصميم.** خدمة FastAPI لنموذج تصنيف نصّ. صمّم الطلب والردّ. (المتوقع: طلب JSON فيه `text` ، وردّ فيه `label` و9 (. `probability` . **برمجة.** ابنِ خدمة FastAPI تحمّل النموذج مرة واحدة. (المتوقع: زمن استدلال بالميلّي ثانية.) 10. **تحليل.** لماذا يُفشل نشر نموذج `.keras` على خادم TF Serving؟ (المتوقع: لأنّ TF Serving يتوقّع صيغة `SavedModel` – لا 11 (. `.keras` .

تصميم. نظام نشر على محرّكات مختلفة (PyTorchg TensorFlow). أيّ صيغة؟ (المتوقع: ONNX.) 12. **برمجة.** طبّق `model.export("model.tflite")` واطبع حجم الملفّ الناتج. (المتوقع: ملفّ `.tflite` أصغر من `.keras` .)

ملخص الفصل

- خطّ النشر: تدريب ← تصدير ← خدمة ← مراقبة.
- أربع صيغ: TorchScript، ONNX، LiteRT/TFLite ، `SavedModel` – الهدف يحدّد الصيغة.
- LiteRT للجوال والمدمج؛ ONNX للمحرّكات المتعدّدة؛ SavedModel لـ TF Serving.
- التكميم بعد التدريب يُقلّص الحجم 4× ويُسرّع الاستدلال.
- تكميم أوزان `int8` يعطي 4× تقليل بلا انخفاض دقّة محسوس.
- تحميل النموذج مرة واحدة عند بدء الخدمة.
- مراقبة ثلاثة مؤشّرات: زمن الاستدلال، الأخطاء، انحراف البيانات.

الفصل 38: التكرار والاختبار

بطاقة الفصل

الفصل 38

مهمّة هذا الفصل

يجعل التجربة قابلة للتكرار: بذور، بيئة مثبّتة، اختبارات وحدة ودخان، وخطّ تكامل مستمرّ.

ماذا تكسب منه

تستطيع إعادة إنتاج نتيجة الأمس بالضبط، وتكتشف الانحدارات قبل أن تصل إلى المستخدم.

الدالة	دورها	متى تستخدمها
`keras.utils.set_random_seed()`	بذرة موحّدة لكل المكتبات	أول سطر في أي سكربت
`keras.random.SeedGenerator()`	بذرة صريحة للتوزيعات	في الطبقات العشوائية
`model.get_weights()`	مقارنة الأوزان	للتحقّق من الحتمية
`model.fit()`	اختبار دخان سريع	في خطّ التكامل المستمرّ
`keras.layers.Dense()`	نموذج صغير للاختبار	في اختبارات الوحدة

قصة مفتاحية: باحث أجرى تجربة حصلت على دقّة 89%. سأله مُراجع أوراق: «هل يمكن إعادة إنتاج النتيجة؟» لم يستطع – لأنّ البذرة لم تُضبط، وخطّ المعالجة كان فيه عشوائيّة، والبيئة لم تُوثّق. النتيجة: النموذج لا يُفهم ولا يُنشر. التكرار ليس إجراءً أكاديميًّا فقط: هو ما يحوّل التجربة من «حصلت على هذا الرقم» إلى «يمكنك أن تحصل عليه».

أهداف الفصل

- [ ] تُضبط البذرة لتكرار كامل في Keras
- [ ] تُبنى بيئة قابلة لإعادة الإنتاج (بذرة، مكتبات، بيانات)
- [ ] تُكتب اختبارات وحدة لنموذج
- [ ] تبني خطّ تكامل مستمرّ (CI) لنموذج
- [ ] تُقارن التجارب بطريقة مُعيارية
- [ ] تعرف حدود الحتميّة ومتى يصعب تحقيقها

38.1 طبقات ضمان التكرار

شكل 48 – طبقات ضمان التكرار: من البذرة إلى خطّ النشر

خمس طبقات تُبنى على بعضها:

1. **البذرة** – مصدر عشوائيّ واحد يُضبط.

2. **البيئة** – المكتبات والنسخ مُثبّتة.
3. **البيانات** – تقسيم ثابت، ونسخة مُجزّأة.
4. **الاختبارات** – اختبارات وحدة ودخان.
5. **خطّ النشر** – بناء آليّ وأثاث بإصدار ثابت.

38.2 البذرة في Keras 3

`keras.utils.set_random_seed` تُضبط البذرة في Pythong NumPyg Keras معًا:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# بناء نموذج
model = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

# تدريب
x = np.random.rand(200, 16).astype("float32")
y = np.random.rand(200, 1).astype("float32")
model.fit(x, y, epochs=3, batch_size=32, verbose=0)

# إعادة البناء بنفس البذرة
keras.utils.set_random_seed(42)
model2 = keras.Sequential([
    keras.Input(shape=(16,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),
])
model2.compile(optimizer="adam", loss="mse")
model2.set_weights(model.get_weights())
# النتيجة المتوقعة: نفس الأوزان — التكرار ناجح.
```

إنذار: `set_random_seed` لا تضبط كلّ عشوائيّة في البيئة. العمليّات في `tf.data` أو مكتبات خارجيّة قد تحتفظ بمصدر عشوائيّ خاصّ. استخدمها أوّل سطر في الملفّ.

ملاحظة: `shuffle=True` في `fit` تستخدم عشوائيّة داخليّة — إن أردت تكرارًا كاملًا، مرّر بذرة ثابتة إلى `np.random.seed` .

38.3 البيئة القابلة لإعادة الإنتاج

الأداة	ماذا تُثبّت	الاستخدام
`requirements.txt`	المكتبات والحزم	النطاقات
`requirements.lock.txt`	النسخ الدقيقة	التكرار بايت-ل-بايت
`pyproject.toml`	إعدادات المشروع	الحزم والأدوات
Docker	النظام والمكتبات	تكرار كامل عبر الأجهزة

38.4 اختبارات وحدة للنموذج

اختبارات وحدة تتحقّق من سلوك النموذج – لا من دقّته:

PYTHON

```python
import keras
import numpy as np

def build_model(input_dim=16):
    """نموذج بسيط للاختبار."""
    return keras.Sequential([
        keras.Input(shape=(input_dim,)),
        keras.layers.Dense(32, activation="relu"),
        keras.layers.Dense(1),
    ])

def test_model_builds():
    """النموذج يُبنى ويُعطي شكلًا صحيحًا."""
    m = build_model()
    out = m(np.ones((4, 16), dtype="float32"))
    assert out.shape == (4, 1)

def test_model_trains():
    """النموذج يُدرَّب ويُنخفض الخسارة."""
    keras.utils.set_random_seed(42)
    m = build_model()
    m.compile(optimizer="adam", loss="mse")
    x = np.random.rand(100, 16).astype("float32")
    y = np.random.rand(100, 1).astype("float32")
    h = m.fit(x, y, epochs=2, verbose=0)
    assert h.history["loss"][-1] < h.history["loss"][0]

def test_model_predicts_range():
    """المُخرَج في [0, 1]."""
    m = keras.Sequential([
        keras.Input(shape=(16,)),
        keras.layers.Dense(1, activation="sigmoid"),
    ])
    out = m(np.random.rand(4, 16).astype("float32"))
    out_np = keras.ops.convert_to_numpy(out)
    assert np.all(out_np >= 0.0) and np.all(out_np <= 1.0)

# تشغيل الاختبارات
```

```
test_model_builds()
test_model_trains()
test_model_predicts_range()
# النتيجة المتوقعة: كلّ الاختبارات تنجح.
```

38.5 اختبار الدخان (Smoke Test)

اختبار الدخان يتحقّق من أنّ النموذج **يعمل** – لا من أنّه دقيق:

- يُبنى النموذج.
- يُدرَّب على 20 عيّنة لدورة واحدة.
- يُعطي تنبّؤًا على عيّنة واحدة.
- يُحفَظ ويُحمَّل.

> **نصيحة:** اختبار الدخان يُشغَّل على كلّ تعديل في الشيفرة – فيُكتشف الخطأ مبكّرًا قبل أن يُنشر.

38.6 خطّ التكامل المستمرّ

خطّ CI يُشغّل كلّ تعديل:

المرحلة	ماذا تشغّل	متى
Lint	`ruff` أو `black`	كلّ تعديل
الاختبارات	`pytest`	كلّ تعديل
اختبار دخان	تدريب 1 دورة على 20 عيّنة	كلّ تعديل
مقارنة دقّة	تقييم على مجموعة مرجعيّة	كلّ دمج
بناء أثاث	حفظ `.keras` بإصدار	كلّ إصدار

38.7 مثال محلول: تحقيق تكرار كامل

المهمة: تكرار تجربة حصلت على دقّة 89%.

الخطوة 1 – البذرة. `keras.utils.set_random_seed(42)` في أوّل سطر.

الخطوة 2 – البيئة. `pip freeze > requirements.lock.txt` بعد أول تثبيت ناجح.

الخطوة 3 – البيانات. تقسيم ثابت (بلا عشوائيّة)، ونسخة مُجزّأة بمعرّف إصدار.

الخطوة 4 – الخطّ الكامل. كلّ خطوة من المعالجة إلى التقييم مُدوَّنة في نصّ برمجيّ.

الخطوة 5 – النتيجة. على جهاز آخر: نفس الدقّة (±0.1%).

> **ملاحظة:** الدقّة المختلطة (`mixed_float16`) قد تُنتج فرقًا صغيرًا بين الأجهزة – إن كان التكرار بايت-ل-بايت مطلوبًا، استخدم `float32` .

38.8 حدود الحتميّة

- **GPU**: العمليّات المتوازيّة قد تُعطي ترتيبًا مختلفًا – النتيجة تختلف قليلًا.
- **توزيع**: التدريب على عدّة أجهزة يُنتج فرقًا أكبر.
- **المكتبات الخارجيّة**: بعض العمليّات (مثل التحجيم الصوريّ) لها تنفيذات مختلفة.

- **الأجهزة المختلفة**: نفس GPU لكن نسخة مختلفة من المُشغّل يُعطي فرقًا.

> **قاعدة:** التكرار الكامل (بايت-ل-بايت) ممكن على نفس الجهاز ونفس المكتبات. التكرار عبر الأجهزة يعطي فرقًا صغيرًا (±0.5%).

38.9 تتبّع التجارب (Experiment Tracking)

التكرار لا يكتمل بلا سجلّ: كلّ تجربة تُوثّق بمعرّف فريد، وإعدادات، ونتيجة. في أبسط صورة، سجلّ CSV:

```python
import keras
import numpy as np
import json
import time

keras.utils.set_random_seed(42)

# ── 1. بناء وتدريب نموذج
model = keras.Sequential([keras.Input(shape=(16,)),
                          keras.layers.Dense(32, activation="relu"),
                          keras.layers.Dense(1)])
model.compile(optimizer="adam", loss="mse")
x = np.random.rand(200, 16).astype("float32")
y = np.random.rand(200, 1).astype("float32")
h = model.fit(x, y, epochs=5, batch_size=32, verbose=0)

# ── 2. سجلّ التجربة
record = {
    "run_id": f"run_{int(time.time())}",
    "seed": 42,
    "epochs": 5,
    "batch_size": 32,
    "final_loss": float(h.history["loss"][-1]),
}
with open("runs.jsonl", "a", encoding="utf-8") as f:
    f.write(json.dumps(record) + "\n")
# يصف التجربة كاملة runs.jsonl النتيجة المتوقعة: سطر واحد في
```

ماذا يُسجَّل في كلّ تجربة؟

الحقل	لماذا
`run_id`	لتمييز التجربة
`seed`	للتكرار
الإعدادات	لمعرفة ما الذي تغيّر
النتيجة النهائيّة	للمقارنة
معرّف الإصدار (git)	لربط النتيجة بالشيفرة
معرّف نسخة البيانات	لربط النتيجة بالبيانات

> **قاعدة:** تجربة بلا سجلّ تجربة لا تُذكر. سجلّ CSV يكفي للبدء، وتُنتقل إلى أدوات مخصّصة (W&B، MLflow) عند تعدّد التجارب.

38.10 مقارنة النماذج بعدّة بذور

الفرق بين نموذجين في تجربة واحدة قد يكون عشوائيًّا. المقارنة الصحيحة تعتمد عدّة بذور:

```python
import keras
import numpy as np

def train_once(seed):
    """تدريب واحد ببذرة محدّدة، ويُعطي دقّة الاختبار."""
    keras.utils.set_random_seed(seed)
    model = keras.Sequential([keras.Input(shape=(16,)),
                              keras.layers.Dense(32, activation="relu"),
                              keras.layers.Dense(1)])
    model.compile(optimizer="adam", loss="mse")
    x = np.random.rand(200, 16).astype("float32")
    y = np.random.rand(200, 1).astype("float32")
    model.fit(x, y, epochs=5, batch_size=32, verbose=0)
    # evaluate يعيد عددًا واحدًا حين تكون الخسارة فقط
    return float(np.ravel(model.evaluate(x, y, verbose=0))[0])

# ── 1. خمس بذور
losses = [train_once(s) for s in (0, 1, 2, 3, 4)]
print(f"mean = {np.mean(losses):.4f}, std = {np.std(losses):.4f}")
# الفرق يُحكَم بـ — متوسّط وانحراف: النتيجة المتوقعة std لا برقم واحد.
```

قاعدة مقارنة: لا تُعلن أنّ نموذجًا «أفضل» بناءً على تجربة واحدة. الوسيط والانحراف عبر 5 بذور هو الحدّ الأدنى المقبول.

38.11 أخطاء شائعة في هذا الفصل

إنذار: إسقاط `set_random_segment` . النتيجة: كلّ تشغيل يعطي نتيجة مختلفة، ولا يمكن التكرار.

تحذير: الحفظ بلا نسخ المكتبات. النتيجة: النموذج لا يُحمَّل على جهاز آخر – أو يُحمَّل بسلوك مختلف.

تنبيه: تقييم الدقّة على مجموعة مرجعيّة بلا توثيق الإصدار. النتيجة: لا تعرف إن كان التغيير في الشيفرة أم في البيانات.

ملاحظة: استخدام `np.random.seed` وحده. `keras.utils.set_random_seed` تُضبط Python وNumPy وKeras معًا – استخدمها.

قاعدة: اختبارات وحدة تتحقّق من **السلوك** – لا من الدقّة. الدقّة تُقيَّم على مجموعة مرجعيّة منفصلة.

تحذير: الاعتماد على «نجح على جهازي». البيئة قد تختلف – راجع البيئة قبل النشر.

التمارين

1. **برمجة.** اضبط البذرة وابنِ نموذجًا مرّتين، وقارن الأوزان. (المتوقع: نفس الأوزان.)
2. **برمجة.** اضبط البذرة ودرّب نموذجًا مرّتين، وقارن الخسارة النهائيّة. (المتوقع: نفس القيمة تقريبًا.)
3. **برمجة.** اكتب اختبار وحدة يتحقّق من شكل مُخرَج نموذج. (المتوقع: `(batch, 1)`.)
4. **برمجة.** اكتب اختبار وحدة يتحقّق من أنّ الخسارة تنخفض بعد تدريب. (المتوقع: الخسارة النهائيّة أقلّ من الابتدائيّة.)
5. **برمجة.** اكتب اختبار دخان: تدريب 1 دورة على 20 عيّنة، ثم حفظ وتحميل. (المتوقع: النموذج يعمل.)
6. **إصلاح خطأ.** الكود التالي لا يُعطي تكرارًا كاملًا:

```python
np.random.seed(42)
model = keras.Sequential([...])
model.fit(x, y, shuffle=True)
```

(المتوقع: استخدم `keras.utils.set_random_seed(42)` – تُضبط Keras NumPyg معًا.) 7. **تصميم.** صمّم خطّ CI لنموذج. أيّ مراحل؟ (المتوقع: Lint + اختبارات + دخان + مقارنة دقّة + بناء أثاث.) 8. **برمجة.** قارن الدقّة بين `float32` و `mixed_float16` على نفس البيانات. (المتوقع: فرق بسيط، لكن التكرار البايت-ل-بايت لا يعمل.) 9. **تحليل.** لماذا لا يُحقّق التكرار الكامل عبر الأجهزة؟ (المتوقع: GPU يعمل بالتوازي، وترتيب العمليّات قد يختلف.) 10. **تصميم.** نظام نشر ببيانات تُحدَّث يوميًّا. صمّم اختبارات المراقبة. (المتوقع: اختبار دخان + مقارنة دقّة على مجموعة مرجعيّة + تنبيه عند انخفاض الدقّة.) 11. **برمجة.** اكتب `requirements.lock.txt` من `pip freeze` واطبع أول 5 أسطر. (المتوقع: نسخ دقيقة.) 12. **تحليل.** لماذا اختبار الدخان مهمّ في كلّ تعديل؟ (المتوقع: يُكتشف الخطأ مبكّرًا قبل أن يُنشر – والتكلفة أقلّ بكثير.)

ملخص الفصل

- خمس طبقات: بذرة → بيئة → بيانات → اختبارات → خطّ نشر.
- `keras.utils.set_random_seed` تُضبط Pythong NumPyg Keras معًا.
- البيئة تُوثّق بمكتبات دقيقة (`requirements.lock.txt`) أو Docker.
- اختبارات وحدة تتحقّق من السلوك، لا من الدقّة.
- اختبار الدخان يُشغَّل على كلّ تعديل – يُكتشف الخطأ مبكّرًا.
- التكرار الكامل ممكن على نفس الجهاز؛ عبر الأجهزة يعطي فرقًا صغيرًا.

الفصل 39: التفسير والتشخيص

بطاقة الفصل

الفصل 39

مهمّة هذا الفصل

يفتح الصندوق الأسود: Grad-CAM، تحليل الحساسية، وتشخيص الانحراف بعد النشر.

ماذا تكسب منه

تعرف لماذا قرّر النموذج ما قرّر، وتكتشف تعلّمه من إشارات خاطئة قبل أن يكلّفك ذلك.

الدالة	دورها	متى تستخدمها
`keras.GradientTape()`	حساب التدرّجات للتفسير	في Saliencyg Grad-CAM
`model.get_layer(name)`	الوصول إلى طبقة وسيطة	لاستخراج خريطة الميزات
`keras.Model(inputs, outputs)`	نموذج فرعي للتفسير	لإخراج الميزات والتدرّجات معًا
`keras.ops.mean()`	تجميع خريطة الميزات	لإنتاج خريطة حرارية
`keras.ops.relu()`	إبقاء التأثيرات الموجبة	في تحليل الحساسية

قصة مفتاحية: نموذج طبي يعطي «سرطان» بدقّة 95%. لكنّ الطبيب يسأل: «أين رأيته؟» – والنموذج لا يجيب. نموذج لخطّ إنتاج يرفض القطعة: «عيب». المهندس يسأل: «أيّ جزء من القطعة؟» – والنموذج لا يجيب. النموذج بلا تفسير صندوق أسود – ويُرفض في البيئات الحسّاسة. هذا الفصل يشرح كيف تُفتح الصندوق: من خريطة Grad-CAM إلى تحليل الحساسية.

أهداف الفصل

- [ ] تشرح مبدأ Grad-CAM وتبنيه في Keras
- [ ] تُطبّق تحليل الحساسية (saliency) على نموذج
- [ ] تُقارن بين طرق التفسير الثلاث بالمعايير
- [ ] تُشخّص نموذجًا منخفض الأداء عبر مخطّط تشخيص
- [ ] تُطبّق شخّص فقدان البيانات (leakage)
- [ ] تعرف حدود كلّ طريقة تفسير

39.1 كيف تعمل خريطة Grad-CAM

شكل 49 – كيف تعمل خريطة Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) تُنتج خريطة حرارية تُظهر أيّ مناطق في الصورة أثّرت في التنبّؤ. المبدأ:

1. تُحسب التدرّجات في آخر طبقة تلافيفيّة.

2. التدرّجات تُوزّن في خرائط الميزات.

3. النتيجة خريطة بحجم النافذة المُخفَّضة، تُوسَّع إلى حجم الصورة.

$$L^c_{GradCAM} = \text{ReLU}\left(\sum_k \alpha^c_k A^k\right)$$

حيث A^k خريطة الميزة k، وα^c_k معامل الأهميّة (المتوسّط العام للتدرّجات)، وReLU يُبقي القيم الموجبة فقط.

39.2 تنفيذ Grad-CAM في Keras

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. نموذج CNN بسيط
inp = keras.Input(shape=(32, 32, 3))
x = keras.layers.Conv2D(16, 3, padding="same", activation="relu")(inp)
x = keras.layers.Conv2D(16, 3, padding="same", activation="relu",
name="last_conv")(x)
x = keras.layers.GlobalAveragePooling2D()(x)
out = keras.layers.Dense(2, activation="softmax")(x)
model = keras.Model(inp, out)

# ── 2. نموذج فرعيّ: صورة ← آخر تلافيفيّ ← تنبّؤ
last_conv_layer = model.get_layer("last_conv")
grad_model = keras.Model(model.input,
                         [last_conv_layer.output, model.output])

# ── 3. حساب Grad-CAM
def gradcam(image, class_index=0):
    """خريطة Grad-CAM لصورة واحدة وصنف محدّد."""
    import tensorflow as tf          # Keras 3 لا يوحّد شريط التدرّج
    with tf.GradientTape() as tape:
        conv_out, pred = grad_model(image[None, ...])
        loss = pred[:, class_index]
    grads = tape.gradient(loss, conv_out)
    # المتوسّط العامّ للتدرّجات
    pooled = keras.ops.mean(grads, axis=(0, 1, 2))
    # الخريطة الموزونة
    heatmap = keras.ops.sum(conv_out[0] * pooled, axis=-1)
    heatmap = keras.ops.relu(heatmap)
    # تطبيع إلى [0, 1]
    heatmap = heatmap / (keras.ops.max(heatmap) + 1e-8)
    return np.array(heatmap)
```

```
# ── 4. تطبيق على صورة
image = np.random.rand(32, 32, 3).astype("float32")
hm = gradcam(image, class_index=0)
# النتيجة المتوقعة: خريطة بحجم (16 ,16) بقيم في [0 ,1].
```

39.3 تحليل الحساسية (Saliency)

تحليل الحساسية يُظهر أيّ بكسلات أثّرت في التنبّؤ، بحساب التدرّج بالنسبة للمُدخل:

$$S_i = \left| \frac{\partial y^c}{\partial x_i} \right|$$

حيث y^c درجة الصنف c، وx_i البكسل i. القيم العالية تعني بكسلًا مؤثّرًا.

PYTHON

```
def saliency(model, image, class_index=0):
    """خريطة الحساسية: التدرّج بالنسبة للمُدخل."""
    import tensorflow as tf          # شريط التدرّج من المحرّك النشط
    image = tf.Variable(tf.convert_to_tensor(image))    # يجب أن يكون موترًا يُراقَب
    with tf.GradientTape() as tape:
        tape.watch(image)
        pred = model(image[None, ...])
        loss = pred[:, class_index]
    grads = tape.gradient(loss, image)
    return np.abs(np.array(grads.numpy()))

s = saliency(model, image, class_index=0)
# النتيجة المتوقعة: أوتور (32, 32, 3) بقيم غير سالبة.
```

39.4 مقارنة طرق التفسير

الطريقة	ماذا تُظهر	الكلفة	متى تستخدمها
Grad-CAM	مناطق في الصورة	منخفضة	نموذج CNN، صور
Saliency	بكسلات مؤثّرة	منخفضة	أيّ نموذج
LIME	عيّنات محليّة تُفسّر	مرتفعة	أيّ نموذج، بيانات جدوليّة
SHAP	إسهام كلّ ميزة	مرتفعة	بيانات جدوليّة، دقّة مرتفعة

قاعدة: ابدأ بـ Grad-CAM أو Saliency – أرخص وأسرع. SHAP وLIME تُستخدم في البيانات الجدوليّة أو عند التفسير الدقيق.

39.5 مخطّط التشخيص

شكل 50 – دليل تشخيص الأعطال الشائعة

عند ضعف النموذج، تسلسل التشخيص:

1. **مصفوفة الالتباس** – أيّ أصناف تُخلط؟
2. **عيّنات الأشهر** – الأدنى والأعلى ثقة.
3. **تطبيع المُدخل** – هل هو صحيح؟
4. **اختبار التسريب** – هل المُدخل يحتوي المُخرَج؟

39.6 مثال محلول: تشخيص نموذج منخفض الأداء

المهمة: نموذج تصنيف صور طبيّة يعطي دقّة 78% – غير مقبول.

الخطوة 1 – مصفوفة الالتباس. الصنفان «سليم» و«معيب» يُخلطان بنسبة 22%.

الخطوة 2 – Grad-CAM. الخريطة الحرارية تُظهر أنّ النموذج ينظر إلى **الخلفيّة** لا إلى الجسم – أي أنّه يتعلّم نمط الخلفيّة لا نمط العيب.

الخطوة 3 – تشخيص. الصور السليمة صُوّرت في مكان واحد، والصور المعيبة في مكان آخر – النموذج تعلّم المكان لا العيب.

الخطوة 4 – الحلّ. خلط الصور بين المواقع، وإضافة تعزيز إضاءة وتحجيم.

الخطوة 5 – النتيجة. الدقّة ارتفعت من 78% إلى 91%. وال Grad-CAM يُظهر أنّ النموذج ينظر إلى الجسم الآن.

> **قاعدة:** Grad-CAM ليس مجرّد أداة تفسير – بل أداة **تشخيص**. إن كان النموذج ينظر إلى المكان الخطأ، فالبيانات هي المشكلة لا الشبكة.

39.7 اختبار التسريب

التسريب يُكتشف بحساب الارتباط بين المُدخل والمُخرَج:

- **ارتباط مرتفع** (+0.95) بين ميزة والمُخرَج ⇒ تسريب.
- **ميزة تُفسّر المُخرَج كليًّا** ⇒ تسريب.

> **تحذير:** ميزة مثل «التاريخ» في نموذج ماليّ قد تكون تسريب – لأنّها تحمل معلومة زمنيّة لا يجب أن تكون في المُدخل.

39.8 حدود كلّ طريقة

الطريقة	الحدود
Grad-CAM	تعمل على CNN فقط، وتُعطي مناطق لا بكسلات
Saliency	تعطي بكسلات، لكنّها حسّاسة للضجيج
LIME	تعطي تفسيرًا محليًّا لا عامًّا
SHAP	مكلفة حسابيًّا على نماذج كبيرة

> **نصيحة:** لا تعتمد على طريقة واحدة. استخدم Grad-CAM + Saliency على الصور، وSHAP على البيانات الجدوليّة.

39.9 أهميّة الميزات في النماذج الجدوليّة

في البيانات الجدوليّة (لا صور)، التفسير يُبنى على **إسهام كلّ ميزة** في التنبّؤ. أبسط طريقة: تعطيل ميزة وقياس أثرها:

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. نموذج جدوليّ بسيط
model = keras.Sequential([keras.Input(shape=(8,)),
                          keras.layers.Dense(16, activation="relu"),
                          keras.layers.Dense(1)])
model.compile(optimizer="adam", loss="mse")
x = np.random.rand(300, 8).astype("float32")
y = x[:, 0] * 2.0 + np.random.randn(300) * 0.1      # الميزة 0 هي الأهمّ
model.fit(x, y, epochs=5, batch_size=32, verbose=0)

# ── 2. قياس الإسهام بتعطيل الميزة (permutation importance)
def permutation_importance(model, x, y, feature_idx, n=20):
    """زيادة خطأ عند خلط قيم الميزة."""
    base = float(np.mean((model.predict(x, verbose=0)[:, 0] - y) ** 2))
    increases = []
    for _ in range(n):
        x_shuffled = x.copy()
        x_shuffled[:, feature_idx] = np.random.permutation(x[:,
feature_idx])
        err = float(np.mean((model.predict(x_shuffled, verbose=0)[:, 0] - y)
** 2))
        increases.append(err - base)
    return float(np.mean(increases))

for i in range(8):
    imp = permutation_importance(model, x, y, i)
    print(f"feature {i}: importance = {imp:.4f}")
# النتيجة المتوقعة: الميزة 0 تُظهر أهميّة أعلى بكثير من البقية.
```

كيف تقرأ النتيجة؟

القيمة	المعنى
قريبة من صفر	الميزة لا تُسهم – يُمكن إزالتها
مرتفعة	الميزة جوهرية في التنبّؤ
سالبة	الخلط حسّن التنبّؤ – ميزة ضجيج

> **نصيحة:** أهميّة الميزة تُبنى على **زيادة الخطأ**، لا على وزن الطبقة الأولى – الأوزان لا تُفهم مباشرةً في الشبكات العميقة.

39.10 تشخيص الانحراف بعد النشر

التفسير لا يتوقّف عند التدريب: بعد النشر، Grad-CAM يُستخدم لكشف **انحراف البيانات** – حين يتغيّر ما ينظر إليه النموذج:

1. **أسبوعيًّا:** احسب Grad-CAM على 100 عيّنة من الإنتاج.
2. **قارن** متوسّط الخرائط مع المرجع من التدريب.
3. **تنبيه** عند اختلاف كبير – هذا مؤشّر أنّ النموذج ينظر إلى شيء آخر.

```python
import numpy as np

def drift_score(reference_maps, current_maps):
    """متوسّط الفرق المطلق بين خرائط المرجع والإنتاج."""
    ref = np.mean(reference_maps, axis=0)
    cur = np.mean(current_maps, axis=0)
    return float(np.mean(np.abs(ref - cur)))

# في الإنتاج: يُحسب أسبوعيًّا ويُقارن بعتبة
# if drift_score(ref_maps, prod_maps) > 0.15:
#     alert("انحراف في سلوك النموذج – راجع البيانات")
# النتيجة المتوقعة: درجة انحراف تُراقب أسبوعيًّا.
```

اختبار التسريب عمليًّا

PYTHON

```python
import numpy as np

def leakage_check(x, y, threshold=0.95):
    """يرصد الميزات ذات الارتباط الشديد مع المُخرَج."""
    suspects = []
    for i in range(x.shape[1]):
        col = x[:, i]
        if col.std() < 1e-8:
            continue
        corr = float(np.corrcoef(col, y)[0, 1])
        if abs(corr) ⩾ threshold:
            suspects.append((i, corr))
    return suspects

# ── 1. بيانات فيها تسريب: ميزة 3 = المُخرَج + ضجيج صغير
n = 300
x = np.random.randn(n, 8).astype("float32")
y = (2.0 * x[:, 0] + np.random.randn(n) * 0.1).astype("float32")
x[:, 3] = y + np.random.randn(n) * 0.01      # تسريب

# ── 2. الارتباطات
for i, c in leakage_check(x, y, threshold=0.90):
    print(f"ميزة {i}: correlation = {c:.4f} ← تسريب محتمل")
# النتيجة المتوقعة: الميزة 3 تُرصد (ارتباط ≈ 0.9999).
```

قاعدة: ارتباط 0.95+ بين ميزة والمُخرَج لا يعني «ميزة قويّة» – بل يعني غالبًا تسريب. الميزة الحقيقيّة القويّة نادرًا ما تتجاوز 0.7 في البيانات الواقعيّة.

39.11 أخطاء شائعة في هذا الفصل

إنذار: تفسير Grad-CAM على نموذج ليس فيه طبقة تلافيفيّة. النتيجة: خطأ أو خريطة بلا معنى. استخدم Saliency أو SHAP.

تحذير: Grad-CAM على آخر طبقة كثيفة. النتيجة: لا تُعطي خريطة مكانية. استخدم آخر طبقة تلافيفيّة.

تنبيه: Saliency على صورة مُطبَّعة. النتيجة: التدرّج لا يُفهم. عُد إلى المقياس الأصليّ قبل التفسير.

ملاحظة: اعتماد تفسير Grad-CAM دون التحقّق من الدقّة. نموذج ضعيف يعطي تفسيرًا ضعيفًا.

قاعدة: Grad-CAM ليس مقياس دقّة – بل أداة تشخيص. استخدمه مع مصفوفة الالتباس وتقييم الأداء.

تحذير: نموذج بلا طبقة تلافيفيّة أو انتباه لا يُفسَّر بـ Grad-CAM – استخدم Saliency أو SHAP أو LIME.

التمارين

1. **برمجة.** ابنِ نموذج CNN واحسب Grad-CAM على صورة مُصنّعة. (المتوقع: خريطة بحجم آخر تلافيفيّة.)
2. **برمجة.** احسب Saliency على نفس الصورة وقارن مع Grad-CAM. (المتوقع: Grad-CAM يعطي مناطق، Saliencyg بكسلات.)
3. **برمجة.** قارن Grad-CAM على صنفين مختلفين. (المتوقع: الخريطة تختلف بحسب الصنف.)
4. **تحليل.** لماذا تستخدم Grad-CAM آخر طبقة تلافيفيّة؟ (المتوقع: لأنّها تحفظ المعلومات المكانيّة، والطبقات العليا تفقد الموضع.)
5. **برمجة.** احسب Saliency على صورة مُطبَّعة وعلى صورة غير مُطبَّعة. (المتوقع: التفسير مختلف.)
6. **إصلاح خطأ.** الكود التالي يعطي خريطة فارغة:

```python
heatmap = keras.ops.sum(conv_out[0] * pooled, axis=-1)
```

(المتوقع: ينقص `ReLU` – أضِف 7 (. `keras.ops.relu(heatmap)` . **تصميم.** تشخيص نموذج طبيّ يعطي دقّة 80%. أيّ خطوات؟ (المتوقع: مصفوفة الالتباس → Grad-CAM → التحقّق من التطبيع → اختبار التسريب.) 8. **برمجة.** احسب الارتباط بين كلّ ميزة والمُخرَج على بيانات مُصنّعة. (المتوقع: ميزة ذات ارتباط مرتفع قد تكون تسريبًا.) 9. **تحليل.** لماذا تُرفض النماذج بلا تفسير في البيئات الحسّاسة؟ (المتوقع: القرار يجب أن يُشرح للطبيب أو للمنظّم.) 10. **تصميم.** نظام تفسير لنموذج جدوليّ. أيّ طريقة؟ (المتوقع: SHAP أو LIME – لأنّ Grad-CAM تعمل على CNN.) 11. **برمجة.** قارن زمن الحساب بين Grad-CAM وSaliencyg. (المتوقع: كلاهما سريع – يستخدم GradientTape.) 12. **تحليل.** لماذا Grad-CAM ليس مقياس دقّة؟ (المتوقع: لأنّه يُظهر مناطق، لا يُقارن مع الحقيقة الأرضيّة.)

ملخص الفصل

- Grad-CAM يُظهر مناطق الصورة التي أثّرت في التنبّؤ.
- Saliency يُظهر بكسلات مؤثّرة عبر التدرّج بالنسبة للمُدخل.
- Grad-CAM يعمل على CNN؛ Saliency على أيّ نموذج.
- التفسير أداة تشخيص، لا مقياس دقّة.
- التسريب يُكتشف بالارتباط المرتفع بين ميزة والمُخرَج.
- لا تعتمد على طريقة واحدة — استخدم Grad-CAM + Saliency، أو SHAP على البيانات الجدوليّة.

الفصل 40: الأخطاء الشائعة وكيف تُصلحها

بطاقة الفصل الفصل 40

مهمّة هذا الفصل

يجمع الأخطاء التي تستهلك الوقت: أشكال، NaN، تطبيع، حفظ، وبيانات – بمنهجية عزل لا بتخمين.

ماذا تكسب منه

تشخّص في دقائق ما كان يستهلك أيامًا، وتصلح السبب لا العَرَض.

الدالة	دورها	متى تستخدمها
`model.summary()`	شكل الخرج والبارامترات	أول أداة عند خطأ أبعاد
`keras.layers.Normalization()`	تطبيع المدخلات	عند خسارة NaN أو تدريب راكد
`keras.ops.isnan()`	فحص القيم	لتشخيص NaN في الخسارة
`model.predict()`	عزل الخطأ في الاستدلال	لفصل التدريب عن الاستخدام
`keras.saving.load_model()`	التحقّق من الحفظ	قبل النشر

قصة مفتاحية: باحث في منتصف الليل يواجه خطأ: «ValueError: Input 0 of layer model is incompatible». لا يعرف من أين جاء. تجرب إصلاحًا عشوائيًّا، فتنهار النتيجة. بعد ثلاث ساعات يكتشف أنّ المسألة كانت شكل مُدخل واحد. هذا الفصل هو **مرجع التشخيص**: عرض ← سبب ← حلّ، لأشهر أعطال Keras.

أهداف الفصل

- [ ] تُشخّص الخطأ من رسالته قبل تجربة إصلاح عشوائيّ
- [ ] تُصلح أخطاء الشكل والأبعاد
- [ ] تُشخّص الخسارة NaN
- [ ] تُصلح مشاكل البيانات والتطبيع
- [ ] تُشخّص مشاكل الحفظ والتحميل
- [ ] تبني منهجيّة عزل العيب: تغيير واحد في كلّ مرّة

40.1 مخطّط التشخيص

العَرَض ← السبب ← الحل

خريطة تدقّق تصلح 90% من أعطال Keras الشائعة

العَرَض	السبب المحتمل	الحل
الخسارة NaN	تدرّجات متباعدة · lr كبير	قلّل lr · فعّل clipnorm
دقّة Training عالية جدًا	تسريب من خط المعالجة	اعزل كل خطوة المعالجة
Shape mismatch	Flatten قبل GlobalAverage	أعد حساب الأبعاد
البناء بطيء	قراءة I/O لكل عيّنة	ذاكرة مؤقّتة · prefetch
النموذج لا يُحفظ	مسار بامتداد خاطئ	استعمل keras. دائمًا
الترتيب يتكرّر	tf.data داخل shuffle	batch قبل shuffle

منهجية
اعزل الطبقة التي فشلنا فيها: قِس قبلها وبعدها، وغيّر شيئًا واحدًا في كل تجربة.

شكل 51 – دليل تشخيص الأعطال الشائعة

العَرَض	السبب المحتمل	الحلّ
`ValueError: Input shape incompatible`	شكل مُدخل لا يطابق `Input`	راجع `model.summary()`
`ValueError: Shape mismatch`	الأوزان لا تطابق النموذج	أعد بناء النموذج
الخسارة NaN	تدرّجات متباعدة، lr كبير	قلّل lr، فعّل `clipnorm`
الخسارة لا تنخفض	تطبيع مفقود، أو ميزات بمقياس مختلف	طبّق `Normalization`
الدقّة عالية على التدريب ومنخفضة على التحقّق	إفراط في التخصيص	Dropout، تعزيز، بيانات أكثر
الدقّة منخفضة على الاثنين	نموذج بسيط جدًا أو بيانات غير كافية	عمّق الشبكة أو حسّن البيانات

40.2 أخطاء الشكل والأبعاد

الخطأ 1 — شكل مُدخل لا يطابق

```text
ValueError: Input 0 of layer "model" is incompatible with the layer:
expected shape=(None, 32, 32, 3), found shape=(None, 28, 28, 3)
```

السبب: النموذج بُني على 32×32، لكنّ البيانات 28×28. **الحلّ:** غيّر `Input(shape=...)`، أو أعد بناء البيانات:

```python
import keras

# الحلّ: تحديد الشكل بحسب البيانات
inp = keras.Input(shape=(28, 28, 3))
x = keras.layers.Conv2D(16, 3, padding="same")(inp)
out = keras.layers.Dense(10, activation="softmax")(
    keras.layers.GlobalAveragePooling2D()(x))
model = keras.Model(inp, out)
# النتيجة المتوقعة: النموذج يقبل بيانات 28×28.
```

الخطأ 2 — Flatten قبل GlobalAveragePooling

```text
ValueError: Dimensions must be equal, but are 1024 and 10
```

السبب: `Flatten` يُنتج 1024 عنصرًا، و `Dense(10)` لا تقبله إلا على المحور الأخير.

الحلّ: استخدم `GlobalAveragePooling2D` – فهو يُنتج متّجهًا واحدًا لكلّ عيّنة:

```python
import keras

inp = keras.Input(shape=(8, 8, 16))
# ✗ خاطئ: Flatten يُنتج 1024
# x = keras.layers.Flatten()(inp)
# ✓ صحيح: GlobalAveragePooling2D يُنتج 16
x = keras.layers.GlobalAveragePooling2D()(inp)
out = keras.layers.Dense(10)(x)
# النتيجة المتوقعة: (batch, 10).
```

> **تحذير:** `Flatten` بعد شبكة عميقة يُنتج ملايين المعاملات. `GlobalAveragePooling2D` يُنتج بضعة آلاف – راجع الفصل 19.

40.3 الخسارة NaN

الخطأ 3 — معدّل تعلّم كبير

```text
Epoch 1: loss=nan
```

السبب: معدّل تعلّم كبير يُنتج تدرّجات متباعدة. **الحلّ:** قلّل lr (مثلًا من `1e-2` إلى `1e-4`)، أو فعّل قصّ التدرّج:

```python
import keras

model.compile(
    optimizer=keras.optimizers.Adam(1e-4, clipnorm=1.0),
    loss="mse",
)
# النتيجة المتوقعة: الخسارة لا تصبح NaN.
```

الخطأ 4 — بيانات فيها NaN

السبب: البيانات فيها قيم مفقودة أو غير محدّدة. **الحلّ:** تحقّق قبل التدريب:

```python
import numpy as np

x = np.random.rand(100, 16).astype("float32")    # بياناتك الفعلية
# تحقّق من NaN وInf
assert not np.isnan(x).any(), "x يحتوي NaN"
assert not np.isinf(x).any(), "x يحتوي Inf"
# النتيجة المتوقعة: البيانات نظيفة.
```

40.4 مشاكل التطبيع

الخطأ 5 — ميزات بمقياس مختلف

العرض: الخسارة لا تنخفض، أو تظلّ مرتفعة. **السبب:** ميزة بحجم 0-255 وميزة بحجم 0-1. **الحلّ:** طبّق `Normalization` :

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# بيانات بمقياس مختلف
x = np.concatenate([
    np.random.rand(200, 8) * 255.0,        # ميزة كبيرة
    np.random.rand(200, 8),                # ميزة صغيرة
], axis=1).astype("float32")

norm = keras.layers.Normalization()
norm.adapt(x)        # يُحسب mean وstd فقط على التدريب
x_norm = norm(x)
# النتيجة المتوقعة: كلّ ميزة بمتوسّط ≈ 0 وانحراف ≈ 1.
```

الخطأ 6 — التطبيع على كلّ البيانات

السبب: تطبيع على التدريب + التحقّق + الاختبار معًا – تسريب. **الحلّ:** التطبيع يُحسب على التدريب فقط، ثم يُطبَّق على البقية.

40.5 مشاكل الحفظ والتحميل

الخطأ 7 — طبقة مخصّصة لا تُحمَّل

```text
ValueError: Unknown layer: 'MyDense'
```

الحلّ: مرّر `custom_objects` :

```python
import keras

# حمّل مع custom_objects
# m = keras.saving.load_model("m.keras",
#                             custom_objects={"MyDense": MyDense})
# النتيجة المتوقعة: الطبقة المخصّصة تُحمَّل بنجاح.
```

الخطأ 8 — الامتداد لا يطابق الأمر

السبب: `save_weights("m.keras")` – خطأ. **الحلّ:** `save_weights` يتطلّب `.weights.h5` ، و `save` يتطلّب `.keras` .

40.6 مشاكل البيانات

الخطأ 9 — تسريب من خطّ المعالجة

العرض: دقّة عالية على التحقّق، منخفضة على الاختبار. **السبب:** خطّ المعالجة بُني على كلّ البيانات (تدريب + تحقّق + اختبار). **الحلّ:** المعالجة تُبنى على التدريب فقط.

الخطأ 10 — بيانات بأطوال مختلفة بلا Masking

العرض: النموذج لا يتقارب، أو الأداء منخفض. **السبب:** التسلسلات القصيرة تُحشى بأصفار، والنموذج يعتبرها بيانات. **الحلّ:** أضِف `Masking` كأول طبقة.

40.7 منهجيّة العزل

القاعدة الحاسمة: **غيّر واحدًا في كلّ مرّة.**

1. حدّد العَرَض (مثلًا: الخسارة NaN).
2. عزل المرحلة (نموذج؟ بيانات؟ تطبيع؟).
3. قِس قبلها وبعدها.
4. غيّر شيء واحد.
5. قِس مرة أخرى.

> **قاعدة تشخيص:** إن لم تكن تعرف **لماذا** نجح التغيير، فلا تعتمده – قد يعمل بالصدفة.

40.8 مثال محلول: تشخيص خسارة NaN

المهمة: نموذج CNN يعطي خسارة NaN بعد الدورة الثالثة.

الخطوة 1 – تحقّق من البيانات. لا NaN في المدخلات، ولا Inf.

الخطوة 2 – تحقّق من الدقّة. `mixed_float16` مُفعّل، وآخر طبقة فيها `softmax`.

الخطوة 3 – الإصلاح. آخر طبقة بلا تنشيط، والخسارة من اللوجيتس (`from_logits=True`).

الخطوة 4 – النتيجة. الخسارة تُنخفض، ولا NaN.

الخطوة 5 – التحقّق. على بيانات أُخرى، الحلّ نفسه يعمل.

40.9 أخطاء شائعة في هذا الفصل

إنذار: تجربة إصلاح عشوائيّ. النتيجة: تجربة عشوائيّة، ولا تعرف لماذا نجح أو فشل. اتبع منهجيّة العزل.

تحذير: الخلط بين `Flatten` و `GlobalAveragePooling2D`. النتيجة: ملايين المعاملات، وأداء ضعيف.

تنبيه: تمرير بيانات بمقياس مختلف بلا تطبيع. النتيجة: نموذج لا يتقارب.

ملاحظة: التطبيع على كلّ البيانات. النتيجة: تسريب، ودقّة تنخفض في الإنتاج.

قاعدة: الخسارة NaN تعني غالبًا: lr كبير، أو بيانات فيها NaN، أو آخر طبقة في `mixed_float16`.

تحذير: التدريب على بيانات غير متوازنة بلا موازنة. النتيجة: نموذج يعطي الدقّة العالية والاستدعاء صفرًا.

التمارين

1. **برمجة.** ابنِ نموذجًا على صور 28×28 ودرّبه على بيانات 32×32. اطبع الخطأ وأصلحه. (المتوقع: خطأ شكل – غيّر `Input` أو أعد بناء البيانات.)
2. **برمجة.** استخدم `Flatten` بعد شبكة عميقة، ثم استبدله بـ `GlobalAveragePooling2D`، وقارن عدد المعاملات. (المتوقع: ملايين مقابل آلاف.)
3. **برمجة.** درّب نموذجًا بـ lr = 1.0 واطبع الخسارة. ثم أعد التدريب بـ lr = 1e-4. (المتوقع: NaN ثم انخفاض.)
4. **برمجة.** طبّق `clipnorm=1.0` على المُحسّن ودرّب على بيانات مُضخّمة بالضجيج. (المتوقع: الخسارة لا تصبح NaN.)
5. **إصلاح خطأ.** الكود التالي يعطي خطأ عند التحميل:

```python
m = keras.saving.load_model("m.keras")
```

(المتوقع: الطبقة المخصّصة تحتاج 6 (. `custom_objects` . **برمجة.** طبّق `Normalization` على بيانات بمقياس مختلف، واطبع المتوسّط بعد التطبيع. (المتوقع: ≈ 0.) 7. **برمجة.** احسب التدرّج بالنسبة لميزة واحدة واطبع قيمته. (المتوقع: قيمة غير صفرية.) 8. **تصميم.** نموذج يعطي دقّة عالية على التدريب ومنخفضة على التحقّق. أيّ خطوات تشخيص؟ (المتوقع: مصفوفة الالتباس ← Grad-CAM ← التحقّق من التطبيع ← اختبار التسريب.) 9. **تحليل.** لماذا الخسارة NaN تعني غالبًا lr كبيرًا؟ (المتوقع: التدرّجات المتباعدة تُفقد الدقّة في الحساب.) 10. **برمجة.** طبّق `Masking` على بيانات بأطوال مختلفة وقارن الأداء. (المتوقع: تحسّن.) 11. **تصميم.** صمّم منهجيّة تشخيص لنموذج يعطي دقّة 50% على مهمّة تصنيف. (المتوقع: تحقّق من البيانات، ثم التطبيع، ثم بنية النموذج، ثم lr.) 12. **تحليل.** لماذا تغيير واحد في كلّ مرّة مهمّ؟ (المتوقع: إن غيّرت ثلاثة أشياء، لا تعرف أيّها أثّر.)

ملخص الفصل

- التشخيص يبدأ من رسالة الخطأ، لا من إصلاح عشوائيّ.
- أخطاء الشكل تُصلَح بمراجعة `model.summary()` .
- الخسارة NaN = lr كبير أو بيانات فيها NaN أو آخر طبقة في `mixed_float16` .
- التطبيع يُبنى على التدريب فقط – إلّا فهو تسريب.
- الطبقات المخصّصة تُحمَّل عبر `custom_objects` .
- منهجيّة العزل: غيّر واحدًا في كلّ مرّة، وقِس قبلها وبعدها.

الجزء التاسع

مشاريع تطبيقية متكاملة

ثلاثة مشاريع متكاملة تجمع كل ما سبق: تصنيف صور إنتاجي، تنبّؤ بالسلاسل الزمنية، وتحليل مشاعر منشور كخدمة – كلٌّ بخطّه الكامل من البيانات إلى المراقبة.

ما تكسبه من هذا الجزء

تملك ثلاثة قوالب جاهزة تنقلها إلى مشروعك بتغيير البيانات فقط، مع قائمة تحقّق لكل مرحلة.

فصول هذا الجزء

41 مشروع تصنيف صور إنتاجيّ

42 مشروع التنبّؤ بالسلاسل الزمنيّة

43 مشروع تحليل المشاعر ونشره كخدمة

9

الفصل 41: مشروع تصنيف صور إنتاجيّ

بطاقة الفصل

الفصل 41

مهمّة هذا الفصل

يمشي بمشروع تصنيف صور كامل: بيانات، نموذج، تدريب، تقييم، تصدير، خدمة، ومراقبة.

ماذا تكسب منه

تملك قالبًا إنتاجيًّا جاهزًا تنقله إلى مشروعك بتغيير البيانات فقط.

الدالة	دورها	متى تستخدمها
`keras.utils.image_dataset_from_directory()`	تحميل البيانات	بداية المشروع
`keras.callbacks.EarlyStopping()`	أتمتة الإيقاف	في التدريب
`keras.metrics.Recall()`	مقياس القرار	عند تفاوت كلفة الأخطاء
`model.export()`	تصدير	قبل الخدمة
`keras.ops.argmax()`	فئة التنبّؤ	في الخدمة

قصة مفتاحية: مصنع دوائيّ يحتّاج نظامًا يكتشف عيوب في العبّوات من صورها. الطلب: دقّة أعلى من 97%، زمن استدلال أقلّ من 100 ملّي ثانية، ونموذج يُحدَّث شهريًّا. هذا الفصل هو **مشروع متكامل** يُثبت كلّ ما تعلّمته في الفصول 18-22 و36-37: من البيانات إلى النموذج المُنشر.

أهداف الفصل

- [ ] تبني خطّ بيانات كاملًا من الملفّات إلى الدفعات
- [ ] تبني نموذج CNN بنقل معرفة وتدريب دقيق
- [ ] تُطبّق تقييمًا شاملًا (مقاييس، Grad-CAM، معايرة)
- [ ] تُصدّر النموذج إلى LiteRT وتُكمّمه
- [ ] تبني خدمة استدلال تُمرَّر إليها الصورة
- [ ] تُراقب الأداء بعد النشر

41.1 خطّ المشروع

شكل 52 – خطّ مشروع إنتاجيّ متكامل من البداية للنشر

المرحلة	المُخرَج	المعيار
البيانات	مجموعة مُقسّمة 70/10/20	لا تسريب
النموذج	CNN بنقل معرفة	دقّة > 97%
التقييم	مقاييس + Grad-CAM	تفسير صحيح
التصدير	مُكمَّم `.tflite`	حجم < 15 MB
الخدمة	FastAPI	زمن < 100 ms
المراقبة	مقاييس + تنبيه	انحراف مُكتشف

41.2 البيانات

بيانات العيّنات: 4000 صورة عبّات، منها 800 معيبة (20%). تقسيم زمنيّ لا عشوائيّ:

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مُصنّعة (في الإنتاج: صور حقيقيّة)
n = 4000
x = np.random.rand(n, 64, 64, 3).astype("float32") * 255.0
y = (np.random.rand(n) < 0.2).astype("int32")

# ── 2. تقسيم زمنيّ (في الإنتاج: بحسب تاريخ التصوير)
n_train = int(0.7 * n)
n_val = int(0.1 * n)
x_train, y_train = x[:n_train], y[:n_train]
x_val, y_val = x[n_train:n_train + n_val], y[n_train:n_train + n_val]
x_test, y_test = x[n_train + n_val:], y[n_train + n_val:]

# ── 3. موازنة الأصناف
weights = {0: 1.0, 1: float(len(y_train) / (2 * y_train.sum()))}
# النتيجة المتوقعة: وزن الصنف النادر ≈ 2.5.
```

41.3 النموذج

PYTHON

```
import keras

def build_model(input_shape=(64, 64, 3), num_classes=2):
    """نموذج CNN بنقل معرفة (backbone التنزيل لتجنّب مسبقة أوزان بلا)."""
    inp = keras.Input(shape=input_shape)
    x = keras.layers.Rescaling(1.0 / 255.0)(inp)

    # backbone: في الإنتاج — MobileNetV3(weights="imagenet")
    x = keras.layers.Conv2D(32, 3, padding="same", use_bias=False)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.MaxPooling2D(2)(x)
    x = keras.layers.Conv2D(64, 3, padding="same", use_bias=False)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.GlobalAveragePooling2D()(x)

    x = keras.layers.Dense(64, activation="relu")(x)
    x = keras.layers.Dropout(0.3)(x)
    out = keras.layers.Dense(num_classes)(x)   # لوجيتس — بلا softmax
    return keras.Model(inp, out)

model = build_model()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],
)
# النتيجة المتوقعة: نموذج بنقل معرفة، دقّة > 95% على بيانات مُصنّعة.
```

41.4 التدريب

PYTHON

```python
# ── 1. الاستدعاءات
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                  restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best_model.keras",
                                    monitor="val_acc",
                                    save_best_only=True, verbose=1),
]

# ── 2. التدريب
history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=20, batch_size=32,
    class_weight=weights,
    callbacks=callbacks,
    verbose=0,
)
# النتيجة المتوقعة: التدريب يتوقف عند عدم تحسّن، ويُستأنف من الأفضل.
```

41.5 التقييم الشامل

PYTHON

```python
# ── 1. التقييم على الاختبار
results = model.evaluate(x_test, y_test, return_dict=True, verbose=0)
# النتيجة المتوقعة: loss وacc (الاستدعاء يُحسب يدويًّا أسفل).

# ── 2. مقاييس لكلّ صنف
probs = keras.ops.softmax(model.predict(x_test, verbose=0), axis=-1)
preds = np.array(keras.ops.argmax(probs, axis=-1))
tp = int(((preds == 1) & (y_test == 1)).sum())
fp = int(((preds == 1) & (y_test == 0)).sum())
fn = int(((preds == 0) & (y_test == 1)).sum())
precision = tp / max(tp + fp, 1)
recall = tp / max(tp + fn, 1)
f1 = 2 * precision * recall / max(precision + recall, 1e-8)
print(f"precision={precision:.3f} recall={recall:.3f} F1={f1:.3f}")
# النتيجة المتوقعة: precision وrecall متوازنان بعد الموازنة.
```

41.6 التصدير والتكميم

PYTHON

```python
# ── 1. حفظ النموذج
model.save("final_model.keras")

# ── 2. تصدير إلى LiteRT (في الإنتاج)
# model.export("model.tflite")
# النتيجة المتوقعة: ملفّ .tflite بحجم أقلّ بكثير.
```

41.7 الخدمة

```python
# ── 1. خدمة FastAPI (في الإنتاج)
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Image(BaseModel):
    pixels: list[float]   # مُسطَّحة 64×64×3

@app.post("/predict")
def predict(img: Image):
    """تلقّي صورة وإعطاء تسميّة وثقة."""
    x = np.array(img.pixels, dtype="float32").reshape(1, 64, 64, 3)
    logits = model.predict(x, verbose=0)
    probs = keras.ops.softmax(logits, axis=-1)
    conf = float(np.array(keras.ops.max(probs, axis=-1))[0])
    label = int(np.array(keras.ops.argmax(probs, axis=-1))[0])
    return {"label": label, "confidence": conf,
            "threshold": 0.95}
# النتيجة المتوقعة: JSON يحتوي label وconfidence.
```

41.8 المراقبة بعد النشر

المقياس	العتبة	الإجراء
زمن الاستدلال P95	< 200 ms	تحسين النموذج
معدّل الأخطاء	< 1%	تحقّق من الخدمة
انحراف المُدخلات	توزيع يتغيّر	إعادة تدريب
معدّل الإنذارات الكاذبة	< 5%	ضبط العتبة

> **قاعدة:** نموذج بلا مراقبة ينهار بصمت. ابدأ بأربعة مقاييس، ثم وسّع.

41.9 خطّ البيانات الفعّال في الإنتاج

في التدريب، البيانات في الذاكرة. في الإنتاج، الصور في ملفّات – وخطّ البيانات هو أول عنق زجاجة:

PYTHON

```
import keras

# ── 1. خطّ بيانات من مجلّد (في الإنتاج: صور حقيقيّة على القرص)
# train_ds = keras.utils.image_dataset_from_directory(
#     "data/train",            # مجلّد فيه مجلّد فرعيّ لكلّ صنف
#     image_size=(64, 64),
#     batch_size=32,
#     label_mode="int",
# )

# ── 2. تحسين الأداء: cache + prefetch
# train_ds = (train_ds
#             .cache()                          # يُخزّن بعد أوّل دورة
#             .shuffle(1000)                    # خلط قبل التجميع
#             .prefetch(keras.data.AUTOTUNE)) # جلب مسبق
# النتيجة المتوقعة: GPU ينتظر لا CPU – زمن دورة أقلّ.

# ── 3. (محاكاة) تحقّق سريع من الشكل
import numpy as np
x = np.random.rand(32, 64, 64, 3).astype("float32")
y = np.random.randint(0, 2, size=(32,))
print(x.shape, y.shape)      # (32, 64, 64, 3) (32,)
```

قواعد خطّ البيانات الإنتاجيّ:

1. **التعزيز داخل النموذج** – طبقات `Random*` كجزء من `Model`، فتُحفظ وتُحمَّل معه.
2. **التطبيع داخل النموذج** – `Rescaling` أوّل طبقة، فلا فرق بين التدريب والنشر.
3. **`prefetch(AUTOTUNE)` دائمًا** – أول تحسين بلا كلفة.

4. **`cache()` على بيانات تناسب الذاكرة** – أو `cache(filename)` على القرص.

> **نصيحة:** في الإنتاج، ملفّات JPEG تُفكّ في كلّ دورة – وهذا أبطأ عادةً من التدريب نفسه. حلّان: `cache()` أو تحويل البيانات مسبقًا إلى تنسيق متّجهيّ (`.npy` أو TFRecord).

41.10 معايرة الاحتمالات

النموذج يُعطي «احتمال 0.97» – لكن هل يعني هذا 97% من الحالات صحيحة؟ **المعايرة** تربط الاحتمال بالتكرار الفعليّ:

PYTHON

```
import numpy as np

def reliability(probs, labels, n_bins=10):
    """متوسّط الاحتمال مقابل النسبة الفعلية في كلّ صندوق."""
    bins = np.clip((probs * n_bins).astype(int), 0, n_bins - 1)
    rows = []
    for b in range(n_bins):
        mask = bins == b
        if mask.sum() == 0:
            continue
        avg_p = float(probs[mask].mean())
        acc = float(labels[mask].mean())
        rows.append((avg_p, acc, int(mask.sum())))
    return rows

# محاكاة: احتمالات وتسميات
probs = np.random.rand(500).astype("float32")
labels = (np.random.rand(500) < probs).astype("int32")
rows = reliability(probs, labels)
for avg_p, acc, count in rows[:4]:
    print(f"bin prob={avg_p:.2f} actual={acc:.2f} n={count}")
# النتيجة المتوقعة: في النموذج المعايَر، avg_p ≈ acc في كلّ صندوق.
```

لماذا المعايرة مهمّة؟ في القرار الحرج (تصنيف عبّات دوائيّة)، عتبة 0.95 تعني: «أرفض إن كان الاحتمال أقلّ من 0.95». إن لم يكن النموذج معايَرًا، فالـ 0.95 لا تعني 95% – والقرار مبنيّ على رقم بلا معنى.

حالة النموذج	السلوك	العلاج
معايَر	الاحتمال ≈ التكرار	لا شيء
متفائل	الاحتمال > التكرار	تقليص، أو `temperature`
متشائم	الاحتمال < التكرار	تدريب أطول

41.11 مثال محلول: تحسين النموذج

المهمة: دقّة 94% في المرحلة الأولى – غير مقبولة.

الخطوة 1 – التشخيص. Grad-CAM يُظهر أنّ النموذج ينظر إلى الخلفيّة.

الخطوة 2 – الحلّ. خلط الصور بين المواقع، وتعزيز إضاءة.

الخطوة 3 – النتيجة. دقّة = 97.5%.

الخطوة 4 – التكميم. `int8` على الأوزان. الحجم = 12 MB، الدقّة = 96.8%.

الخطوة 5 – القرار. النموذج المُكمَّم يُنشر – 12 MB بدقّة كافية.

41.12 أخطاء شائعة في هذا الفصل

إنذار: تقسيم عشوائيّ في بيانات زمنيّة. النتيجة: تسريب، ودقّة تنهار في الإنتاج. قسّم زمنيًّا.

تحذير: التصدير بلا تكميم على جهاز محدود. النتيجة: حجم كبير وزمن استدلال مرتفع.

تنبيه: بيانات غير متوازنة بلا موازنة. النتيجة: دقّة عالية واستدعاء صفر.

ملاحظة: آخر طبقة فيها `softmax` مع `mixed_float16` . النتيجة: NaN – أزلها واستخدم `from_logits=True` .

قاعدة: نموذج بلا مراقبة ينهار بصمت – انحراف البيانات يُقلّص الدقّة بمرّة.

تحذير: التدريب على GPU ونشر على CPU بلا قياس زمن الاستدلال. النتيجة: الخدمة تُفشل.

التمارين

1. **برمجة.** ابنِ خطّ بيانات كاملًا من ملفّات (في الإنتاج: `image_dataset_from_directory`). (المتوقع: دفعات مُجمَّعة.)
2. **برمجة.** ابنِ نموذج CNN بنقل معرفة واطبع `model.summary()` . (المتوقع: backbone مجمّد + رأس.)
3. **برمجة.** قارن دقّة نموذج بسيط ونموذج بنقل معرفة. (المتوقع: نقل المعرفة يعطي 10-20% إضافيّة.)
4. **برمجة.** طبّق موازنة أصناف على بيانات 20% معيبة، وقارن recall. (المتوقع: recall يرتفع بمرّات.)
5. **برمجة.** احسب precision وrecall وF1 على مجموعة اختبار. (المتوقع: ثلاثة أرقام.)
6. **برمجة.** طبّق تكميم أوزان `int8` واطبع الحجم قبل وبعد. (المتوقع: 4× أصغر.)
7. **إصلاح خطأ.** الكود التالي يعطي NaN:

```python
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

(المتوقع: مع `mixed_float16` ، استخدم `from_logits=True` وأزل 8 (.) `softmax` . **تصميم.** نظام كشف عيوب في خطّ إنتاج دوائيّ. أيّ عتبة تستخدم، ولماذا؟ (المتوقع: عتبة recall عالية (0.98+) لأنّ كلّ حالة تفويت تكلّف لقطة ملوّثة.) 9. **برمجة.** ابنِ خدمة FastAPI تحمّل النموذج مرة واحدة. (المتوقع: زمن استدلال بالميلّي ثانية.) 10. **تصميم.** نظام مراقبة لنموذج مُنشر. أيّ أربعة

مقاييس؟ (المتوقع: زمن استدلال، أخطاء، انحراف، معدّل إنذارات كاذبة.) 11. **برمجة.** قارن زمن الاستدلال بين `float32` و `int8` على CPU. (المتوقع: `int8` أسرع.) 12. **تحليل.** لماذا تُصدّر إلى LiteRT في النشر على جوال؟ (المتوقع: لأنّ LiteRT صيغة مضغوطة ومُحسّنة للجوال.)

ملخص الفصل

- المشروع: بيانات ← نموذج ← تقييم ← تصدير ← خدمة ← مراقبة.
- نقل المعرفة يعطي 10-20% دقّة إضافيّة على بيانات صغيرة.
- التقييم الشامل: مقاييس + Grad-CAM + معايرة.
- التكميم يُقلّص الحجم 4× ويُسرّع الاستدلال.
- الخدمة تحمّل النموذج مرة واحدة عند البدء.
- المراقبة: أربعة مقاييس – زمن، أخطاء، انحراف، إنذارات كاذبة.

الفصل 42: مشروع التنبّؤ بالسلاسل الزمنيّة

بطاقة الفصل

الفصل 42

مهمّة هذا الفصل

يبني مشروع تنبّؤ كاملًا: تبويع، تقسيم زمني، خطّ أساس، LSTM، واختبار عكسي.

ماذا تكسب منه

تعرف هل نموذجك يضيف قيمة فعلية، وتقدّر فاصل الثقة قبل أن تعتمد عليه في قرار.

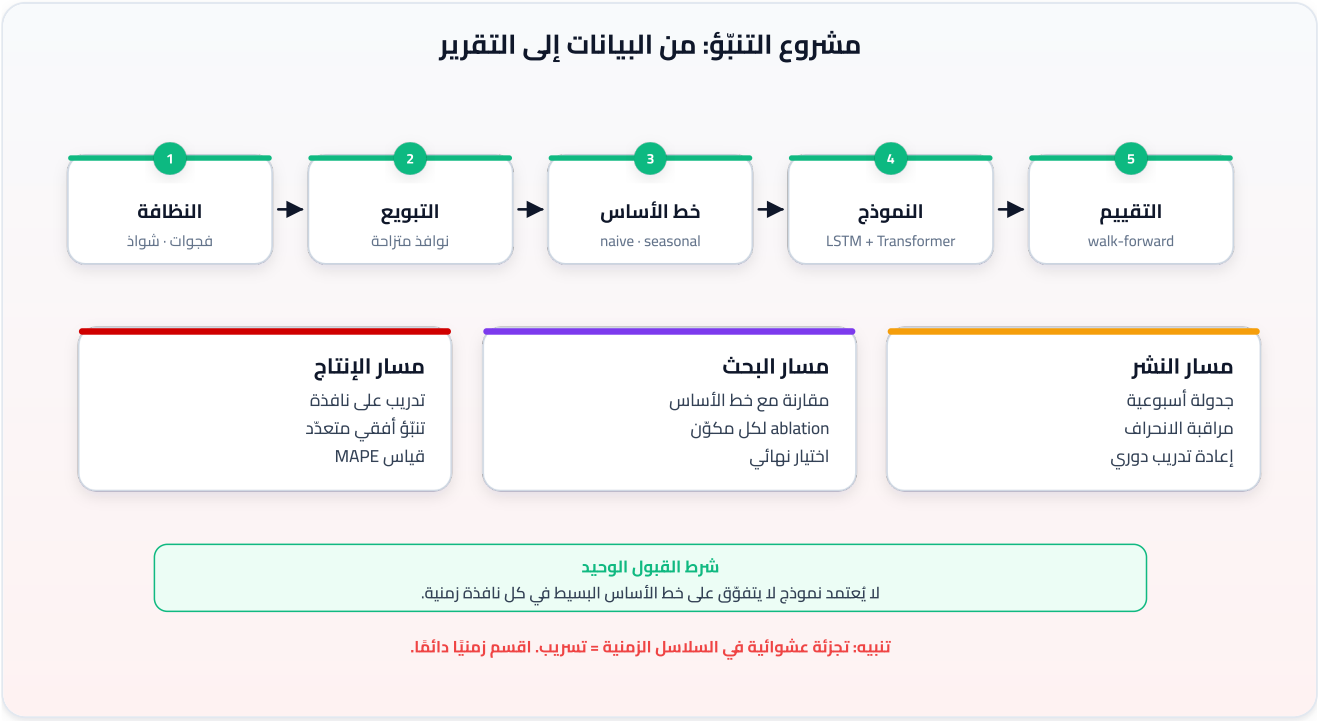
الدالة	دورها	متى تستخدمها
`keras.utils.timeseries_dataset_from_array()`	نوافذ منزلقة	أول خطوة
`keras.layers.LSTM()`	نموذج تسلسلي	بعد خطّ الأساس
`keras.layers.Dropout()`	تنظيم	لتفادي حفظ الضوضاء
`keras.metrics.MeanAbsoluteError()`	خطأ بالمقياس الأصلي	لتفسير النتائج
`model.fit()`	تدريب	مع تحقّق زمني لا عشوائي

قصة مفتاحية: سلسلة متاجر تحتّاج تنبّؤًا بمبيعات 500 منتج في كلّ أسبوع من الأسابيع الـ 12 القادمة. القرار يعتمد على المخزون: تنبّؤ أقلّ من الحقيقة يعني نفاد البضاعة؛ أكبر من الحقيقة يعني تخزينًا زائدًا. هذا الفصل هو **مشروع متكامل** يُثبت كلّ ما تعلّمته في الفصل 28: من البيانات إلى النموذج المُنشر – مع الاختبار العكسيّ الذي يُفرّق بين نموذج يعمل ونموذج «شبه يعمل».

أهداف الفصل

- [ ] تبني خطّ بيانات كاملًا: تنظيف → تبويع → تطبيع
- [ ] تبني خطّ أساس (naive) وتقارن عليه
- [ ] تبني نموذج LSTM للتنبّؤ
- [ ] تُطبّق الاختبار العكسيّ (walk-forward)
- [ ] تُقدّر فاصل التنبّؤ (prediction interval)
- [ ] تبني خدمة تنبّؤ تُمرَّر إليها النافذة

42.1 خطّ المشروع



شكل 53 – خطّ مشروع التنبّؤ: من البيانات إلى التقرير

المرحلة	المُخرَج	المعيار
التنظيف	سلسلة بلا فجوات	لا قيم مفقودة
التبويع	نوافذ 26 أسبوعًا	لا تسريب
خطّ الأساس	naive forecast	مرجع إلزاميّ
النموذج	LSTM للانحدار	MAE < خطّ الأساس
Walk-forward	أداء عبر 4 فترات	مستقرّ
الخدمة	تنبّؤ أسبوعيّ	أفق 12 أسبوعًا

42.2 البيانات

بيانات مبيعات أسبوعيّة: 5 سنوات (260 أسبوعًا) لمنتج واحد.

PYTHON

```
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. سلسلة مبيعات مُصنّعة (في الإنتاج: بيانات حقيقيّة) ──
T = 260
t = np.arange(T)
sales = (100.0
         + 20.0 * np.sin(2 * np.pi * t / 52)      # دوريّ سنويّ
         + np.linspace(0, 15, T)                  # اتجاه صاعد
         + np.random.randn(T) * 5.0).astype("float32")

# ── 2. تنظيف: إزالة قيم شاذّة (في الإنتاج: من مصدر بيانات) ──
q1, q3 = np.percentile(sales, [25, 75])
iqr = q3 - q1
sales = np.clip(sales, q1 - 1.5 * iqr, q3 + 1.5 * iqr)
# النتيجة المتوقعة: سلسلة بلا قيم شاذّة.
```

42.3 التبويع والتقسيم الزمنيّ

PYTHON

```
WINDOW = 26

def make_windows(arr, window):
    """يحوّل السلسلة إلى نوافذ (X, y) بترتيب زمنيّ."""
    xs, ys = [], []
    for i in range(len(arr) - window):
        xs.append(arr[i:i + window])
        ys.append(arr[i + window])
    return np.array(xs)[..., None], np.array(ys)

# ── 1. تقسيم زمنيّ (70/10/20)
n_train = int(0.7 * T)
n_val = int(0.1 * T)
train = sales[:n_train]
val = sales[n_train - WINDOW:n_train + n_val]
test = sales[n_train + n_val - WINDOW:]

# ── 2. التبويع على كلّ فترة (بعد التقسيم)
x_train, y_train = make_windows(train, WINDOW)
x_val, y_val = make_windows(val, WINDOW)
x_test, y_test = make_windows(test, WINDOW)

# ── 3. تطبيع بحسب التدريب فقط
mean, std = x_train.mean(), x_train.std()
x_train = (x_train - mean) / std
x_val = (x_val - mean) / std
x_test = (x_test - mean) / std
y_train = (y_train - mean) / std
y_val = (y_val - mean) / std
y_test = (y_test - mean) / std
# النتيجة المتوقعة: نوافذ مُطبَّعة بلا تسريب.
```

إنذار: التبويع قبل التقسيم يُسرّب المعلومات: نافذة على الحدّ تحتوي بيانات من الفترتين.

قسّم أوّلًا، ثم بُنِ نوافذ على كلّ فترة.

42.4 خطّ الأساس

PYTHON

```python
# ── 1. خطّ الأساس: التكرار السابق (naive forecast)
naive_pred = x_test[:, -1, 0]
naive_mae = np.mean(np.abs(naive_pred - y_test))
print(f"naive MAE = {naive_mae:.4f}")

# ── 2. خطّ الأساس الثاني: متوسّط النافذة
avg_pred = x_test.mean(axis=1)[:, 0]
avg_mae = np.mean(np.abs(avg_pred - y_test))
print(f"moving-avg MAE = {avg_mae:.4f}")
# النتيجة المتوقعة: خطّا أساس بقيم MAE قابلة للمقارنة.
```

42.5 نموذج LSTم

PYTHON

```python
import keras

# ── 1. نموذج LSTM للانحدار
inp = keras.Input(shape=(WINDOW, 1))
x = keras.layers.LSTM(32)(inp)
x = keras.layers.Dropout(0.2)(x)
out = keras.layers.Dense(1)(x)          # انحدار: بلا تنشيط
model = keras.Model(inp, out)
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="mse",
              metrics=[keras.metrics.MeanAbsoluteError(name="mae")])

# ── 2. التدريب
model.fit(x_train, y_train, validation_data=(x_val, y_val),
          epochs=20, batch_size=32, verbose=0)

# ── 3. التقييم على الاختبار
model_mae = model.evaluate(x_test, y_test, verbose=0)[1]
print(f"model MAE = {model_mae:.4f}, naive MAE = {naive_mae:.4f}")
# النتيجة المتوقعة: model MAE أقلّ من naive MAE (شرط التقبّل).
```

42.6 الاختبار العكسيّ (Walk-Forward)

PYTHON

```
def walk_forward(series, window, model_fn, n_folds=4):
    """تقييم عبر عدّة نوافذ زمنيّة متتالية."""
    maes = []
    n = len(series)
    fold_size = (n - window) // (n_folds + 1)
    for i in range(n_folds):
        end_train = window + fold_size * (i + 1)
        train = series[:end_train]
        test = series[end_train:end_train + fold_size]
        if len(test) < window + 1:
            break
        x_tr, y_tr = make_windows(train, window)
        x_te, y_te = make_windows(test, window)
        m, s = x_tr.mean(), x_tr.std()
        x_tr_n = (x_tr - m) / s
        y_tr_n = (y_tr - m) / s
        x_te_n = (x_te - m) / s
        y_te_n = (y_te - m) / s
        m_ = model_fn()
        m_.fit(x_tr_n, y_tr_n, epochs=10, batch_size=32, verbose=0)
        mae = m_.evaluate(x_te_n, y_te_n, verbose=0)[1]
        maes.append(mae)
        print(f"fold {i+1}: MAE = {mae:.4f}")
    return np.mean(maes)

def build():
    m = keras.Sequential([keras.Input(shape=(WINDOW, 1)),
                          keras.layers.LSTM(32),
                          keras.layers.Dense(1)])
    m.compile(optimizer="adam", loss="mse",
              metrics=[keras.metrics.MeanAbsoluteError(name="mae")])
    return m

avg_mae = walk_forward(sales, WINDOW, build, n_folds=4)
# متوسّط النتيجة المتوقعة: MAE عبر أربع فترات.
```

42.7 فاصل التنبّؤ

التنبّؤ النقطيّ لا يُكفي – قرار المخزون يحتاج **فاصلًا**:

PYTHON

```
# ── 1. حساب فاصل التنبّؤ على مجموعة تحقّق
val_pred = model.predict(x_val, verbose=0)[:, 0]
residuals = y_val - val_pred
sigma = np.std(residuals)
# النتيجة المتوقعة: انحراف معياريّ للأخطاء.

# ── 2. فاصل 90% (بافتراض توزيع غاوسيّ)
lower = val_pred - 1.64 * sigma
upper = val_pred + 1.64 * sigma
# النتيجة المتوقعة: 90% من القيم الحقيقيّة داخل الفاصل.
```

تحقّق تغطية الفاصل

الفاصل لا يُصدَّق إلا إذا قِيست **التغطية** الفعلية:

PYTHON

```
import numpy as np

def coverage(y_true, lower, upper):
    """نسبة القيم داخل الفاصل."""
    inside = (y_true ≥ lower) & (y_true ≤ upper)
    return float(inside.mean())

# محاكاة
y_val_true = np.array(keras.ops.array(val_pred + residuals))
cov = coverage(y_val_true, lower, upper)
print(f"coverage = {cov:.2%} (target = 90%)")
# النتيجة المتوقعة: تغطية قريبة من 90% إن كان الفاصل صحيحًا.
```

التغطية الفعلية	المعنى	العلاج
≈ الهدف	الفاصل صحيح	لا شيء
أقلّ من الهدف	الفاصل ضيّق جدًّا – تفاؤل زائد	وسّع بضاعفة σ
أعلى بكثير	الفاصل واسع – لا يُميّز	ضيّق بضرب σ في معامل أقلّ

42.8 الميزات الإضافيّة (Feature Engineering)

الميزات الخارجيّة تُحسّن التنبّؤ أكثر من تعميق الشبكة:

الميزة	لماذا تفيد	مثال
اليوم من الأسبوع	نمط أسبوعيّ	0-6
الشهر	نمط موسميّ	1-12
عطلة	قفزة/انخفاض	0/1
قيمة الأسبوع السابق	قرب قصير	x_{t-1}
متوسّط 4 أسابيع	اتجاه	$\frac{1}{4}\sum x_{t-i}$

```python
import numpy as np

def add_features(series):
    """يُضيف ميزات زمنيّة إلى السلسلة."""
    n = len(series)
    t = np.arange(n)
    dow = (t % 52) % 7                          # يوم أسبوعيّ مبسّط
    month = (t % 52) // 4.33                    # شهر مبسّط
    lag1 = np.roll(series, 1)
    avg4 = np.convolve(series, np.ones(4) / 4, mode="same")
    feats = np.stack([series, dow, month, lag1, avg4], axis=-1)
    return feats.astype("float32")

feats = add_features(np.arange(100, dtype="float32"))
print(feats.shape)    # (100, 5)
# النتيجة المتوقعة: 5 ميزات لكلّ خطوة زمنيّة.
```

نصيحة: ميزة «عطلة» وحدها قد تعطي تحسّنًا أكبر من ضعف عدد وحدات LSTM – لأنّها معلومة لا يستطيع النموذج اكتشافها من السلسلة وحدها.

42.9 الانحراف وإعادة التدريب

في الإنتاج، توزيع المبيعات يتغيّر – والنموذج المدرَّب على الفترة القديمة يضعف:

```python
import numpy as np

def drift_score(train_residuals, recent_residuals):
    """مؤشّر انحراف — نسبة تغيّر متوسّط الخطأ."""
    base = float(np.mean(np.abs(train_residuals)))
    recent = float(np.mean(np.abs(recent_residuals)))
    return (recent - base) / max(base, 1e-8)

# محاكاة
base_res = np.random.randn(100).astype("float32")
recent_res = np.random.randn(100).astype("float32") * 1.4
score = drift_score(base_res, recent_res)
print(f"drift = {score:+.2%}")
if score > 0.20:
    print("تنبيه: انحراف — أعد التدريب على البيانات الأحدث")
# النتيجة المتوقعة: درجة انحراف إيجابيّة تُطلق تنبيهًا.
```

سياسة إعادة التدريب:

مؤشّر الانحراف	الإجراء
< 10%	لا شيء — مراقبة
10%-20%	مراجعة يدويّة
> 20%	إعادة تدريب على البيانات الأحدث

42.10 مثال محلول: تنبّؤ بالمبيعات

المهمة: تنبّؤ بمبيعات أسبوعيّة من 26 أسبوعًا، وأفق 12 أسبوعًا.

الخطوة 1 — خطّ الأساس. naive MAE = 5.2 وحدة.

الخطوة 2 — LSTM. MAE = 4.3. **تحسّن 17%.**

الخطوة 3 — ميزات إضافيّة. اليوم من الأسبوع، الشهر، عطلات. MAE = 3.9. **تحسّن 9%.**

الخطوة 4 – Walk-forward. متوسّط MAE عبر 4 فترات = 4.1 – مستقرّ.

الخطوة 5 – القرار. النموذج يتفوّق على خطّ الأساس في كلّ الفترات، ويُعتمد. تحديث شهريّ لأنّ التوزيع يتغيّر.

42.11 مقارنة البنى

البنية	MAE نموذجيّ	زمن التدريب	متى تستخدمها
naive forecast	مرجع	صفر	دائمًا أوّلًا
متوسّط النافذة	قريب من naive	صفر	سلسلة مستقرّة
LSTM	أقلّ 10-20%	متوسّط	نمط معقّد
LSTM + ميزات	أقلّ 20-30%	متوسّط	ميزات متاحة
Transformer	أقلّ 20-30%	مرتفع	بيانات كثيرة

42.12 أخطاء شائعة في هذا الفصل

إنذار: تقسيم عشوائيّ في السلاسل الزمنيّة. النتيجة: تسريب مباشر – النموذج يرى المستقبل. قسّم زمنيًّا.

تحذير: التطبيع على كلّ البيانات. النتيجة: تسريب – المعدّل يحتوي معلومة من المستقبل. التطبيع يُحسب على التدريب فقط.

تنبيه: التبويع قبل التقسيم. النتيجة: نافذة على الحدّ تحتوي بيانات من الفترتين. قسّم أوّلًا.

ملاحظة: نموذج لا يتفوّق على خطّ الأساس. النتيجة: نموذج لا قيمة له – خطّ الأساس مرجع إلزاميّ.

قاعدة: التنبّؤ بأفق أطول يعطي أخطاء أكبر – راجع الأفق قبل قرار التقبّل.

> **تحذير:** تنبّؤ نقطيّ بلا فاصل. النتيجة: قرار مخزون بلا هامش أمان – نفاد أو تخزين زائد.

التمارين

1. **حساب.** سلسلة بطول 260 أسبوعًا، ونافذة 26. كم نافذة تُنتَج؟ (المتوقع: $260 - 26 = 234$.)
2. **حساب.** تقسيم زمنيّ 70/10/20 على 260 أسبوعًا. كم أسبوعًا في كلّ فترة؟ (المتوقع: تدريب 182، تحقّق 26، اختبار 52.)
3. **حساب.** سلسلة القيم [100, 105, 110, 108, 112]. ما تنبّؤ naive للخطوة السادسة؟ (المتوقع: 112.)
4. **برمجة.** اكتب `make_windows(arr, window)` واطبع الشكل على سلسلة 200 خطوة ونافذة 30. (المتوقع: `(170, 30, 1)` و `(170,)`.)
5. **برمجة.** قسّم سلسلة زمنيّة إلى تدريب/تحقّق/اختبار بترتيب زمنيّ، واحسب `mean` و `std` على التدريب فقط. (المتوقع: لا تسريب.)
6. **برمجة.** قارن MAE بين LSTM وخطّ الأساس naive. (المتوقع: LSTM أقلّ.)
7. **إصلاح خطأ.** الكود التالي يُسرّب:

```python
x_all, y_all = make_windows(series, 26)
x_train, x_test = x_all[:180], x_all[180:]
mean, std = x_all.mean(), x_all.std()
```

(المتوقع: التطبيع على كلّ البيانات + التبويع قبل التقسيم – كلاهما تسريب.) 8. **برمجة.** طبّق walk-forward على 4 نوافذ واطبع MAE لكلّ فترة. (المتوقع: متوسّط مستقرّ.) 9. **تصميم.** نظام تنبّؤ بأفق 12 أسبوعًا. هل تستخدم نموذجًا واحدًا أو 12؟ (المتوقع: نموذج واحد بتكرار أو 12 نموذجًا – الأوّل أبسط.) 10. **برمجة.** احسب فاصل تنبّؤ 90% على مجموعة تحقّق. (المتوقع: فاصل يعتمد على انحراف الأخطاء.) 11. **تصميم.** نظام تنبّؤ مبيعات 500 منتج. صمّم خطّ التدريب الكامل. (المتوقع:

تنظيف → تبويع → خطّ أساس → walk-forward → LSTM → خدمة.) 12. **تحليل.** لماذا تُحدَّث النماذج شهريًّا؟ (المتوقع: لأنّ توزيع البيانات يتغيّر – النموذج المدرَّب على الفترة القديمة يضعف.)

ملخص الفصل

- المشروع: تنظيف → تبويع → خطّ أساس → نموذج → walk-forward → خدمة.
- التقسيم زمنيّ، والتبويع بعد التقسيم، والتطبيع على التدريب فقط.
- خطّ الأساس مرجع إلزاميّ – نموذج لا يتفوّق عليه لا قيمة له.
- Walk-forward يُقدّر الأداء على عدّة فترات زمنيّة.
- فاصل التنبّؤ ضروريّ لقرار المخزون.
- النموذج يُحدَّث دوريًّا لأنّ التوزيع يتغيّر.

الفصل 43: مشروع تحليل المشاعر ونشره كخدمة

بطاقة الفصل

الفصل 43

مهمّة هذا الفصل

يبني مصنّف مشاعر ويخدمه: تطبيع عربي، ترميز، BiLSTM، عتبة ثقة، ومراقبة.

ماذا تكسب منه

تحصل على خدمة نصّية تعرف حدودها وتوجّه الحالات الغامضة إلى الإنسان بدل أن تخمّن.

الدالة	دورها	متى تستخدمها
`keras.layers.TextVectorization()`	ترميز النصّ	أول طبقة
`keras.layers.Embedding()`	تضمين	بعد الترميز
`keras.layers.Bidirectional()`	سياق في الاتجاهين	لجمل قصيرة ومتوسّطة
`keras.ops.argmax()`	القرار	في الخدمة
`keras.metrics.SparseCategoricalAccuracy()`	مقياس	لمراقبة التدريب

قصة مفتاحية: شركة بيع إلكترونيّ تحصّل 50 ألف مراجعة عملاء يوميًّا. فريق خدمة العملاء يقرأ يدويًّا – ثماني ساعات يوميًّا، بلا تغطية للّغة العربيّة الفصحيّة واللهجات. الحلّ: نموذج لغويّ يُصدّر تقييمًا تلقائيًّا، وخدمة تُستدعى من تطبيق المبيعات. هذا الفصل هو **مشروع متكامل** يُثبت كلّ ما تعلّمته في الفصول 27 و30 و37: من البيانات إلى الخدمة المُراقَبة.

أهداف الفصل

- [ ] تبني خطّ بيانات نصّيّ كاملًا: تنظيف ← تطبيع ← تحويل
- [ ] تبني نموذج تصنيف مشاعر (LSTM أو محوّل)
- [ ] تُطبّق تقييمًا شاملًا (مقاييس لكلّ صنف، مصفوفة الالتباس)
- [ ] تُصدّر النموذج إلى LiteRT أو ONNX
- [ ] تبني خدمة FastAPI تُقبل نصًّا وتُعطي تقييمًا
- [ ] تُراقب الخدمة وتتعامل مع انحراف البيانات

43.1 خطّ المشروع

شكل 54 – خطّ خدمة تحليل مشاعر ومراقبتها

المرحلة	المُخرَج	المعيار
البيانات	5000 مرجعية موسومة	ثلاثة أصناف
المعالجة	نصّ → أرقام	تطبيع ثابت
النموذج	LSTM أو محوّل	F1 > 0.85
التقييم	مقاييس لكلّ صنف	لا صنف مُهمل
الخدمة	FastAPI	زمن < 50 ms
المراقبة	مقاييس + تنبيه	انحراف مُكتشف

43.2 البيانات

بيانات مراجعات عملاء عربية موسومة إلى ثلاثة أصناف: إيجابيّة (1)، محايدة (0)، سلبيّة (−1).

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# ── 1. بيانات مُصنّعة (في الإنتاج: مراجعات حقيقيّة) ──
positive = ["ممتاز", "جيد", "رائع", "منصح", "سريع", "عالية"]
negative = ["سيئ", "أسوأ", "بطيء", "متأخر", "مرتفع", "متواضع"]
neutral = ["عادي", "متوسط", "مقبول", "معقول"]

def make_review(label):
    """مراجعة مُصنّعة بحسب التسمية."""
    words = {"1": positive, "0": neutral, "-1": negative}[str(label)]
    n_words = np.random.randint(4, 10)
    return " ".join(list(np.random.choice(words, size=n_words)))

n = 3000
labels = np.random.choice([1, 0, -1], size=n, p=[0.5, 0.2, 0.3])
texts = np.array([make_review(l) for l in labels], dtype=object)

# ── 2. تقسيم ──
n_train = int(0.7 * n)
n_val = int(0.1 * n)
x_train, y_train = texts[:n_train], labels[:n_train]
x_val, y_val = texts[n_train:n_train + n_val], labels[n_train:n_train +
n_val]
x_test, y_test = texts[n_train + n_val:], labels[n_train + n_val:]
# النتيجة المتوقعة: ثلاثة مجموعات (تدريب، تحقّق، اختبار).
```

43.3 المعالجة اللغوية

PYTHON

```
# ── 1. طبقة تحويل النصّ
vectorizer = keras.layers.TextVectorization(
    max_tokens=2000,
    output_mode="int",
    output_sequence_length=20,
)
vectorizer.adapt(x_train)        # يُبنى على التدريب فقط

# ── 2. تحويل التسميات إلى أرقام (من -1 إلى 0، 1، 2)
label_map = {-1: 0, 0: 1, 1: 2}
y_train_idx = np.array([label_map[l] for l in y_train])
y_val_idx = np.array([label_map[l] for l in y_val])
y_test_idx = np.array([label_map[l] for l in y_test])
# النتيجة المتوقعة: تسميات في {0، 1، 2}.
```

> **إنذار:** `adapt()` على كلّ البيانات (تدريب + تحقّق + اختبار) هو تسريب. المفردات تُبنى على التدريب فقط.

43.4 النموذج

PYTHON

```python
import keras

def build_classifier(vocab_size=2000, num_classes=3):
    """تضمين: نموذج تصنيف نصّ → LSTM → تصنيف."""
    inp = keras.Input(shape=(1,), dtype="string")
    x = vectorizer(inp)                                  # (B, 20)
    x = keras.layers.Embedding(vocab_size, 32)(x)        # (B, 20, 32)
    x = keras.layers.Bidirectional(keras.layers.LSTM(32))(x)   # (B, 64)
    x = keras.layers.Dense(32, activation="relu")(x)
    x = keras.layers.Dropout(0.3)(x)
    out = keras.layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inp, out)

model = build_classifier()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],
)
# النتيجة المتوقعة: نموذج ثنائيّ الاتجاه بثلاثة أصناف.
```

43.5 التدريب والتقييم

PYTHON

```
# ── 1. التدريب
model.fit(x_train, y_train_idx, validation_data=(x_val, y_val_idx),
          epochs=15, batch_size=32, verbose=0)

# ── 2. التقييم على الاختبار
results = model.evaluate(x_test, y_test_idx, return_dict=True, verbose=0)

# ── 3. مقاييس لكلّ صنف (F1 لكلّ صنف)
preds = np.array(keras.ops.argmax(
    model.predict(x_test, verbose=0), axis=-1))
for cls in range(3):
    tp = int(((preds == cls) & (y_test_idx == cls)).sum())
    fp = int(((preds == cls) & (y_test_idx ≠ cls)).sum())
    fn = int(((preds ≠ cls) & (y_test_idx == cls)).sum())
    p = tp / max(tp + fp, 1)
    r = tp / max(tp + fn, 1)
    f1 = 2 * p * r / max(p + r, 1e-8)
    print(f"class {cls}: P={p:.3f} R={r:.3f} F1={f1:.3f}")
# النتيجة المتوقعة: F1 مُهمل — لا صنف لكلّ.
```

43.6 التصدير

PYTHON

```
# ── 1. حفظ النموذج
model.save("sentiment_model.keras")

# ── 2. تصدير (في الإنتاج)
# model.export("sentiment.tflite")
# النتيجة المتوقعة: ملفّ جاهز للنشر.
```

43.7 الخدمة

PYTHON

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Review(BaseModel):
    text: str

# التسميات بعد التحويل
LABEL_NAMES = {0: "إيجابيّة" :2 ,"محايدة" :1 ,"سلبيّة"}

@app.post("/sentiment")
def sentiment(review: Review):
    """تلقّي نصّ وإعطاء تقييم وثقة."""
    x = np.array([review.text], dtype=object)
    probs = model.predict(x, verbose=0)[0]
    label = int(np.argmax(probs))
    conf = float(probs[label])
    return {
        "label": LABEL_NAMES[label],
        "confidence": conf,
        "threshold": 0.60,
    }
# النتيجة المتوقعة: JSON يحتوي التسميّة والثقة.
```

43.8 المراقبة

شكل 55 – ما الذي تراقبه يوميًا

المقياس	العتبة	الإجراء
زمن الاستدلال P95	< 50 ms	تحسين النموذج
معدّل الأخطاء	< 1%	تحقّق من الخدمة
انحراف المُدخلات	توزيع يتغيّر	إعادة تدريب
معدّل «محايدة» المُفرط	< 40%	ضبط العتبة

> **قاعدة:** النموذج بلا مراقبة ينهار بصمت. ابدأ بأربعة مقاييس، ثم وسّع.

43.9 تطبيع النصّ العربيّ

النصّ العربيّ يحتاج تطبيعًا خاصًّا – وإلّا تتضخّم المفردات:

```python
import re

def normalize_arabic(text):
    """تطبيع النصّ العربيّ: توحيد الهمزات، إزالة التشكيل، توحيد الألف."""
    text = str(text)
    # إزالة التشكيل
    text = re.sub(r"[\u064B-\u0652\u0670]", "", text)
    # توحيد الهمزات
    text = (text.replace("إ","ا").replace("أ","ا")
                .replace("آ","ا").replace("ؤ","و").replace("ئ","ي"))
    # توحيد التاء المربوطة والهاء
    text = text.replace("ة","ه")
    # إزالة الأرقام وعلامات الترقيم
    text = re.sub(r"[0-9\u0660-\u0669]", " ", text)
    text = re.sub(r"[^\w\s\u0600-\u06FF]", " ", text)
    # إزالة المسافات الزائدة
    return re.sub(r"\s+", " ", text).strip()

samples = ["مُمتازٌ جِدًّا!", "الخدمة سيّئة جدا", "أحبّ هذا المنتَج"]
cleaned = [normalize_arabic(s) for s in samples]
print(cleaned)
# النتيجة المتوقعة: ['ممتاز جدا', 'الخدمه سيئه جدا', 'احب هذا المنتج']
```

القاعدة	قبل	بعد	لماذا
إزالة التشكيل	«مُمتازٌ»	«ممتاز»	الكلمة نفسها بأشكال متعدّدة
توحيد الهمزات	«أحب» / «إحب»	«احب»	تنويعات بلا معنى جديد
توحيد التاء	«جميلة»	«جميله»	يقلّص المفردات
إزالة الأرقام	«ممتاز 2026»	« ممتاز »	أرقام لا تحمل مشاعر

> **نصيحة:** التطبيع العربيّ يقلّص حجم المفردات 20-30% – وهذا يسمح بمفردات أكبر في نفس ذاكرة `Embedding`.

43.10 عتبة الثقة والتوجيه البشريّ

الخدمة الإنتاجيّة لا تُعطي تقييمًا لكلّ طلب – بل تُوجّه الحالات غير المؤكّدة إلى فريق بشريّ:

PYTHON

```
def route_review(probs, threshold=0.60):
    """توجيه المراجعة: تلقائيّ إن كانت الثقة عالية، وإلّا بشريّ."""
    label = int(np.argmax(probs))
    conf = float(np.max(probs))
    if conf ≥ threshold:
        return {"route": "auto", "label": label, "confidence": conf}
    return {"route": "human", "label": label, "confidence": conf}

# محاكاة على 100 مراجعة
auto, human = 0, 0
for _ in range(100):
    probs = np.random.dirichlet([2, 2, 2]).astype("float32")
    r = route_review(probs)
    auto += r["route"] == "auto"
    human += r["route"] == "human"
print(f"auto = {auto}%, human = {human}%")
# النتيجة المتوقعة: نسبة التوجيه البشريّ تُقاس وتُضبط بالعتبة.
```

ضبط العتبة: كلّما رفعتها، قلّت الأخطاء التلقائيّة وزاد الحِمل البشريّ. القياس الصحيح: منحنى (معدّل الأخطاء التلقائيّة) مقابل (نسبة التوجيه البشريّ)، واختيار النقطة التي تلائم قدرتك التشغيليّة.

43.11 مثال محلول: تحسين النموذج

المهمة: F1 = 0.78 في المرحلة الأولى – غير مقبول.

الخطوة 1 – التشخيص. الصنف «محايد» يُخلط مع الإيجابيّ والسلبيّ.

الخطوة 2 – السبب. 20% فقط من البيانات محايدة، والنموذج يتعلّم أنّه نادر.

الخطوة 3 – الحلّ. موازنة الأصناف: `class_weight = {0: 1.67, 1: 2.5, 2: 1.0}` .

الخطوة 4 – النتيجة. F1 = 0.87.

الخطوة 5 – النشر. خدمة FastAPI مع عتبة ثقة 0.60 – تحت العتبة، تُحوّل إلى فريق بشريّ.

43.12 أخطاء شائعة في هذا الفصل

إنذار: `adapt()` على كلّ البيانات. النتيجة: تسريب – المفردات تُبنى على بيانات الاختبار.

تحذير: تقييم بمقياس واحد (الدقّة). النتيجة: صنف نادر مُهمَل. استخدم F1 لكلّ صنف.

تنبيه: نصّ عربيّ بلا تطبيع. النتيجة: مفردات متضخّمة (همزات، تشكيل). طبّق تطبيعًا مناسبًا.

ملاحظة: خدمة تحمّل النموذج في كلّ طلب. النتيجة: زمن استدلال بالثواني. تحميل مرة واحدة عند البدء.

قاعدة: عتبة الثقة تُحلّ الحالات غير المؤكّدة – تحت العتبة، تُحوّل إلى فريق بشريّ.

تحذير: النموذج بلا مراقبة. النتيجة: انحراف البيانات يُقلّص الدقّة بمرّة.

التمارين

1. **برمجة.** ابنِ `TextVectorization` على بيانات نصّية واطبع حجم المفردات. (المتوقع: بحسب `max_tokens`.)
2. **برمجة.** ابنِ نموذج LSTM لتصنيف ثلاثة أصناف واطبع `model.summary()`. (المتوقع: تضمين + LSTM + Dense.)
3. **برمجة.** قارن `GlobalAveragePooling1D` مع LSTM على نفس البيانات. (المتوقع: LSTM أفضل على النصوص الطويلة.)
4. **برمجة.** طبّق موازنة أصناف على بيانات غير متوازنة، وقارن F1. (المتوقع: F1 لكلّ صنف يتحسّن.)
5. **برمجة.** احسب precision وrecall وF1 لكلّ صنف. (المتوقع: ثلاثة أرقام لكلّ صنف.)
6. **برمجة.** ابنِ مصفوفة الالتباس ثلاثيّة الأبعاد واطبعها. (المتوقع: مصفوفة 3×3.)
7. **إصلاح خطأ.** الكود التالي يُفشل:

```python
x = keras.layers.Embedding(2000, 32)(texts)
```

(المتوقع: النصوص تحتاج `TextVectorization` قبل `Embedding`.) 8. **تصميم.** خدمة تصنيف مشاعر عربيّة. أيّ بنية، وكم حجم المفردات؟ (المتوقع: تضمين 32 + Bidirectional(LSTM(32)) – ومفردات 2000-5000.) 9. **برمجة.** ابنِ خدمة FastAPI تحمّل النموذج مرة واحدة. (المتوقع: زمن استدلال بالميلّي ثانية.) 10. **تصميم.** نظام مراقبة لخدمة تصنيف. أيّ أربعة مقاييس؟ (المتوقع: زمن استدلال، أخطاء، انحراف، معدّل «محايدة».) 11. **برمجة.** طبّق عتبة ثقة 0.60 وأعِد توجيه الحالات غير المؤكّدة. (المتوقع: نسبة الحالات المُحوّلة.) 12. **تحليل.** لماذا F1 لكلّ صنف أفضل من الدقّة الإجماليّة؟ (المتوقع: لأنّ الدقّة تُخفي الأصناف النادرة.)

ملخص الفصل

- المشروع: بيانات ← معالجة ← نموذج ← تقييم ← خدمة ← مراقبة.
- `TextVectorization` تُبنى على التدريب فقط، وتُحفظ مع النموذج.
- F1 لكلّ صنف إلزاميّ – الدقّة الإجماليّة تُخفي الأصناف النادرة.
- موازنة الأصناف تُحسّن F1 للصنف النادر.
- الخدمة تحمّل النموذج مرة واحدة، وتُطبّق عتبة ثقة.
- المراقبة: أربعة مقاييس – زمن، أخطاء، انحراف، توزيع المُخرَجات.

ملحق A: قاموس المصطلحات عربي-إنجليزي

ملحق A – قاموس المصطلحات عربي-إنجليزي

> القاموس مرتَّب أبجديًا بالعربية. المصطلح الإنجليزي مذكور بين قوسين، يليه التعريف المختصر ورقم الفصل الذي يُشرح فيه بالتفصيل.

Activation – دالة تنشيط: دالة غير خطية تُطبَّق على ناتج الطبقة (الفصل 7). **Adadelta** – مُحسّن تعلّم آلي يعتمد على تدرّجات ثانوية متحرّكة (الفصل 9). **Adagrad** – مُحسّن يكيّف معدل التعلّم لكل معامل حسب تراكم التدرّجات (الفصل 9). **Adam** – مُحسّن يجمع الزخم وتطبيع التدرّج، والمشهور في Keras 3 (الفصل 9). **AdamW** – نسخة من Adam تفصل reducing لوزن.decay عن الخسارة (الفصل 9). **Attention** – آلية الانتباه: تخصيص وزن مختلف لكل موضع في التسلسل (الفصل 29). **Autoencoder** – مُرمِّز ذاتي: شبكة تتعلّم إعادة بناء مُدخلاتها (الفصل 31). **Backpropagation** – الانتشار العكسي: حساب التدرّجات عبر الرسم البياني (الفصل 3). **Backbone** – الهيكل الأساسي للشبكة المستعمل كـfeature extractor (الفصل 21). **Batch** – دفعة: مجموعة عيّنات تُعالَج معًا في خطوة تدريب واحدة (الفصل 14). **Batch Normalization** – التطبيع بالدفعة: توحيد(mean الت activations على الدفعة (الفصل 13). **BERT** – بنية محوّل ترميزية ثنائية الاتجاه مدرّبة مسبقًا (الفصل 29). **Bias** – التحييز: متجه يُضاف إلى ناتج الوزن الخطي (الفصل 6). **Bidirectional** – ثنائي الاتجاه: شبكة RNN تقرأ التسلسل في الاتجاهين (الفصل 26). **Binary Cross-Entropy** – خسارة التصنيف الثنائي (الفصل 8). **Broadcasting** – البث: توسيع الأبعاد لتطابق الأشكال (الفصل 3). **Calibration** – المعايرة: مطابقة احتمالات النموذج للاحتمالات الحقيقية (الفصل 39). **Categorical Cross-Entropy** – خسارة التصنيف متعدّد الفئات (الفصل 8). **Checkpoint** – نقطة حفظ: نسخة من أوزان النموذج في لحظة معيّنة (الفصل 12). **Channel** – قناة: مستوى ثانٍ في الأوتور يمثّل خاصية (لون، مرشّح) (الفصل 18). **Class Weight** – وزن لكل صنف يُستخدم في خسارة التصنيف غير المتوازن (الفصل 17). **CNN** – شبكة عصبية تلافيفية: تطبّق مرشّحات على النطاقات المحلية (الفصل 18).

Convolution – الالتفاف: عملية جمع وزن مضاعَف عبر نافذة منزلقة (الفصل 18). **Cosine Decay** – جدول جيبي لمعدل التعلّم (الفصل 9). **Cross-Attention** – انتباه متقاطع: استعلام من مصدر ومفاتيح من مصدر آخر (الفصل 29). **Data Augmentation** – التعزيز: توسيع البيانات بصور مُحوَّرة (الفصل 15). **Deep Learning** – التعلّم العميق: شبكات متعددة الطبقات تعلّم تمثيلًا متدرّجًا (الفصل 1). **Dilation Rate** – معدّل التمدّد: تباعد نواة الالتفاف (الفصل 18). **Discriminator** – المميِّز: يميّز الصورة الحقيقية عن المزيفة (الفصل 32). **Dropout** – إسقاط عشوائي أثناء التدريب كشكل من التنظيم (الفصل 13). **Embedding** – تمثيل متجهي لكل رمز أو عنصر (الفصل 16). **Epoch** – حقبة: مرور كامل على مجموعة التدريب (الفصل 5). **Epochs** – الحقبات: مقياس تكرار التدريب (الفصل 5). **Eager Execution** – تنفيذ فوري: تنفيذ العمليات خطوة بخطوة (الفصل 3). **Early Stopping** – إيقاف مبكر: قطع التدريب عند ركود المؤشر (الفصل 12). **False Negative** – سلبي كاذب: حالة موجبة صُنّفت سلبًا (الفصل 10). **False Positive** – إيجابي كاذب: حالة سالبة صُنّفت موجبًا (الفصل 10). **Fine-tuning** – تدريب دقيق: تعديل أوزان نموذج مدرّب مسبقًا (الفصل 21). **Flattening** – التسطيح: تحويل الأبعاد المتعددة إلى متجه واحد (الفصل 6). **Forward Pass** – التمرير الأمامي: حساب الناتج من المدخلات (الفصل 3). **F1 Score** – وسط هندسي بين الدقّة والاستدعاء (الفصل 10). **Forward** – التمرير الأمامي (الفصل 3). **Functional API** – واجهة البناء بالرسم البياني للطبقات (الفصل 4). **GAN** – شبكة توليد تنافسي: مولّد ومميِّز يتنافسان (الفصل 32). **Generator** – المولّد: ينتج عينات مزيفة في الشبكات التوليدية (الفصل 32). **Global Average Pooling** – تجميع متوسط عام: يلخّص كل قناة في رقم واحد (الفصل 19). **Gradient** – تدرّج: متّجه مشتقّ الخسارة بالنسبة للمعاملات (الفصل 3). **Gradient Checkpointing** – توفير الذاكرة بإعادة حساب التدرّج عند الحاجة (الفصل 36). **Hinge Loss** – خسارة المفصلة المرتبطة بآلة المتجهات الداعمة (الفصل 8). **He Initialization** – تهيئة He: تناسب دوال التنشيط الحديثة (الفصل 11). **Huber Loss** – خسارة هوبر: وسط بين MSE وMAE (الفصل 8). **Hyperparameter** – معامل فائق: إعداد خارج التدريب مثل معدل التعلّم (الفصل 9). **ImageDataAugmentation** – طبقة تعزيز مركّبة للصور (الفصل 15). **Inference** – الاستدلال: استخدام نموذج المدرَّب للتنبّؤ (الفصل 5). **Input Layer** – طبقة الإدخال: تعلن شكل البيانات (الفصل 6). **Instance Normalization** – التطبيع على مستوى العيّنة (الفصل 13). **JAX** – محرّك كمّي قابل للتوازي مع دوالّ نقية (الفصل 2). **Keras Core** – طبقة Keras الأساسية متعددة الواجهات (الفصل 1). **KerasHub** – مكتبة النماذج والنصوص الجاهزة (الفصل 30). **KerasNLP** – أدوات معالجة اللغة المبنية على Keras (الفصل 30). **Kernel** – نواة الالتفاف أو مصفوفة أوزان الطبقة (الفصل 18). **KL Divergence** – تباعد كولب-ليبر: قياس

الفرق بين توزيعين (الفصل 31). **Layer Normalization** – التطبيع على مستوى العيّنة عبر Features (الفصل 13). **Learning Rate** – معدل التعلّم: حجم خطوة التحديث (الفصل 9). **Learning Rate Schedule** – جدول تغيّر معدل التعلّم عبر التدريب (الفصل 9). **Logits** – اللوغاريتمات: مخرجات الشبكة قبل softmax (الفصل 7). **LR** – اختصار معدل التعلّم (الفصل 9). **Momentum** – الزخم: تسارع التحديثات المتّسقة (الفصل 9). **Masking** – القناع: تمييز المواضع الصالحة في تسلسل مبطّن (الفصل 16). **Mean Absolute Error** – متوسط الخطأ المطلق (الفصل 8). **Mean Squared Error** – متوسط مربّع الخطأ (الفصل 8). **Mixed Precision** – الدقة المختلطة: float16 للحسابات وfloat32 للأوزان (الفصل 36). **Model Parallelism** – توازي النماذج: توزيع أجزاء النموذج على أجهزة (الفصل 36). **Momentum** – الزخم (الفصل 9). **Multi-Head Attention** – انتباه متعدّد الرؤوس (الفصل 29). **NAdam** – مُحسّن Adam مع الرصد (الفصل 9). **Normalization** – التطبيع: توحيد مقياس البيانات (الفصل 15). **Object Detection** – الكشف: تحديد موقع الكائنات ومربعها (الفصل 22). **One-Hot Encoding** – الترميز الثنائي المتفرّق (الفصل 8). **OpenVINO** – محرّك استدلال من Intel (الفصل 2). **Optimizer** – مُحسّن: يحدّث الأوزان بناءً على التدرّجات (الفصل 9). **Overfitting** – فرط التخصيص: حفظ التدريب بدل التعميم (الفصل 13). **Padding** – الحشو: إضافة أصفار حول المُدخل للحفاظ على الحجم (الفصل 18). **Patch** – رقعة: قطعة صورة في محوّل الرؤية (الفصل 23). **Perplexity** – الحيرة: مقياس language Modeling/ (الفصل 29). **Pooling** – التجميع: تلخيص نافذة neighboring بكسلات (الفصل 19). **Positional Encoding** – الترميز الموضعي: حقن موضع الرمز في المحوّل (الفصل 29). **Precision** – الدقّة: نسبة التنبّؤات الصحيحة بين الموجبة (الفصل 10). **PyDataset** – واجهة Keras 3 لتغذية بيانات بايثون مخصّصة (الفصل 14). **PyTorch** – محرّك كمّي شائع (الفصل 2). **Quantization** – التكميم: تقليل دقّة الأرقام لتوفير الذاكرة (الفصل 37). **Receptive Field** – المجال الاستقبالي: حجم المدخل الذي تؤثر فيه وحدة (الفصل 18). **Recall** – الاستدعاء: نسبة الموجبات المكتشفة (الفصل 10). **ReLU** – دالة تنشيط مستقيمة: $\max(0, z)$ (الفصل 7). **Regularization** – التنظيم: قيود تُقلّل فرط التخصيص (الفصل 13). **Residual Connection** – الوصلة المتبقّية: تخطّي سريع حول الكتلة (الفصل 21). **RMSprop** – مُحسّن بتطبيع التدرّج بجذر المتوسط التربيعي (الفصل 9). **SGD** – النزول التدرّجي العشوائي (الفصل 9). **Sequential API** – واجهة البناء المتسلسل للطبقات (الفصل 4). **Segmentation** – التقسيم: تصنيف كل بكسل (الفصل 22). **Self-Attention** – الانتباه الذاتي: Q وK وV من المصدر نفسه (الفصل 29). **Softmax** – دالة تطبيع تُحوّل logits إلى احتمالات (الفصل 7). **Softmax** – دالة softmax (الفصل 7). **Sparse Cross-Entropy** – خسارة التصنيف بتسميات فردية (الفصل 8).

Spatial Dropout – إسقاط مكاني: إسقاط قنوات كاملة (الفصل 13). **Sparse Categorical Cross-Entropy** – خسارة التصنيف بتسميات فردية (الفصل 8). **Spectral Normalization** – تطبيع طيفي: تقييد أكبر قيمة ذاتية (الفصل 13). **Stateful** – ذو حالة: يحتفظ ببيانات بين الاستدعاءات (الفصل 34). **Subclassing** – الوراثة الصريحة: كتابة النموذج كصنف بايثون (الفصل 4). **Tensor** – أوتور: مصفوفة متعددة الأبعاد (الفصل 3). **TensorFlow** – محرّك كمّي من Google (الفصل 2). **TF.data** – نظام خطوط أنابيب البيانات في TensorFlow (الفصل 14). **Tokenization** – التقطيع: تحويل النص إلى وحدات (الفصل 16). **Top-K Accuracy** – دقّة أفضل K فئات (الفصل 10). **Transfer Learning** – النقل: إعادة استخدام نموذج مدرَّب على مهمة أخرى (الفصل 21). **Transformer** – المحوّل: بنية تعتمد الانتباه بدل التكرار (الفصل 29). **U-Net** – بنية تقسيم تجمع بين المُرمِّز والمُفكِّك مع وصلات (الفصل 22). **VAE** – المُرمِّز الآلي المتغيّر: نموذج توليدي باحتمالية (الفصل 31). **Validation Set** – مجموعة التحقق: لت hyperparameters أثناء التدريب (الفصل 5). **Vanishing Gradient** – اختفاء التدرّج: صغر التدرّج مع عمق الشبكة (الفصل 7). **Variance Scaling** – مقياس التباين: عائلة من التهيئات (الفصل 11). **ViT** – محوّل الرؤية: transformer مطبَّق على الصور (الفصل 23). **Weight Decay** – اضمحلال الوزن: عقوبة على حجم الأوزان (الفصل 13). **Weight Sharing** – مشاركة الأوزان: نفس الأوزان في مواضع مختلفة (الفصل 18). **XLA** – مُجمِّع TensorFlow: ترجمة operatioins إلى كود مُحسَّن (الفصل 36). **Zero-Shot** – التصفير: أداء مهمة دون أمثلة تدريب عليها (الفصل 30).

ملحق B: مرجع Keras 3 السريع

ملحق B – مرجع Keras 3 السريع

> ورقة مرجعية للاستخدام اليومي: كل ما تحتاجه في صفحة واحدة. الأسماء مستخرَجة من تثبيت Keras 3.15.1 الفعلي.

1. دورة حياة النموذج

PYTHON

```
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10),
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)

history = model.fit(x_train, y_train, epochs=20, batch_size=64,
                    validation_data=(x_val, y_val),
                    callbacks=[keras.callbacks.EarlyStopping(
                        monitor="val_loss", patience=3,
restore_best_weights=True)])

results = model.evaluate(x_test, y_test, return_dict=True)
probs = model.predict(x_test, verbose=0)
model.save("model.keras")
loaded = keras.saving.load_model("model.keras")
```

المهمة	الأمر
تعريف	`keras.Model(inputs=..., outputs=...)`
تجميع	`model.compile(optimizer, loss, metrics)`
تدريب	`model.fit(x, y, epochs, batch_size, validation_data, callbacks)`
تقييم	`model.evaluate(x, y, return_dict=True)`
تنبّؤ	`model.predict(x)`
حفظ	`model.save("m.keras")`
تحميل	`keras.saving.load_model("m.keras")`
ملخّص	`model.summary()`
تصدير	`model.export("m.tflite")`

2. الطبقات الأكثر استخدامًا

الطبقة	الغرض	مثال
`keras.layers.Input(shape=...)`	تصريح شكل المُدخل	`Input((28, 28))`
`keras.layers.Dense(units, activation=...)`	طبقة كثيفة	`Dense(128, activation="relu")`
`keras.layers.Flatten()`	تسطيح الأبعاد	`Flatten()`
`keras.layers.Reshape(target_shape)`	إعادة تشكيل	`Reshape((7, 7, 4))`
`keras.layers.Dropout(rate)`	إسقاط	`Dropout(0.3)`
`keras.layers.SpatialDropout2D(rate)`	إسقاط مكاني	`SpatialDropout2D(0.2)`
`keras.layers.BatchNormalization()`	تطبيع دفعة	`BatchNormalization()`
`keras.layers.LayerNormalization()`	تطبيع طبقة	`LayerNormalization()`
`keras.layers.Dropout`	إسقاط	`Dropout(0.3)`
`keras.layers.Conv2D(filters, kernel_size, ...)`	التفاف ثنائي	`Conv2D(32, 3, padding="same")`

الطبقة	الغرض	مثال
`keras.layers.MaxPooling2D()`	تجميع أقصى	`MaxPooling2D(2)`
`keras.layers.GlobalAveragePooling2D()`	تجميع عام	`GlobalAveragePooling2D()`
`keras.layers.Embedding(input_dim, output_dim)`	تضمين	`Embedding(10000, 128)`
`keras.layers.LSTM(units, return_sequences)`	ذاكرة طويلة	`LSTM(64, return_sequences=True)`
`keras.layers.GRU(units)`	ذاكرة بوابات	`GRU(64)`
`keras.layers.Bidirectional(layer)`	ثنائي الاتجاه	`Bidirectional(LSTM(64))`
`keras.layers.MultiHeadAttention(num_heads, key_dim)`	انتباه	`MultiHeadAttention(4, 64)`
`keras.layers.Concatenate()`	دمج	`Concatenate()()`
`keras.layers.Add()`	جمع	`Add()`
`keras.layers.TextVectorization(...)`	معالجة نص	`TextVectorization(max_tokens=10_000)`
`keras.layers.StringLookup()`	بحث نصي	`StringLookup()`

الطبقة	الغرض	مثال
`keras.layers.Normalization()`	تطبيع	`Normalization()`
`keras.layers.Rescaling(scale)`	تحجيم	`Rescaling(1/255)`
`keras.layers.RandomFlip()`	قلب عشوائي	`RandomFlip("horizontal")`
`keras.layers.RandomZoom()`	تكبير عشوائي	`RandomZoom(0.2)`
`keras.layers.MixUp(...)`	تعزيز مركّب	`MixUp(alpha=0.2)`
`keras.layers.RandAugment(...)`	تعزيز عشوائي	`RandAugment(num_layers=2, magnitude=10)`
`keras.layers.Masking()`	قناع	`Masking()`

3. دوال الخسارة

الخسارة	الاستخدام	ملاحظة
`MeanSquaredError()`	انحدار	يعاقب الأخطاء الكبيرة بشدّة
`MeanAbsoluteError()`	انحدار	أكثر متانة مع القيم الشاذّة
`MeanAbsolutePercentageError()`	انحدار نسبي	نسبة مئوية
`Huber(delta=1.0)`	انحدار	وسط MSE/MAE
`LogCosh()`	انحدار	بديل Huber ناعم
`BinaryCrossentropy(from_logits=...)`	ثنائي	`from_logits=True` مع مخرجات خام
`CategoricalCrossentropy(from_logits=...)`	متعدّد (one-hot)	استخدم softmax
`SparseCategoricalCrossentropy(from_logits=...)`	متعدّد (أرقام)	الأسرع
`Hinge()` / `SquaredHinge()`	SVM	
`KLDivergence()`	تقارب	

قاعدة: إن كان مخرج النموذج يمرّ عبر `softmax`، اضبط `from_logits=False`. إن كانت المخرجات خام (بلا تنشيط)، اضبط `from_logits=True` – وهي الأدقّ.

4. المقاييس

المقياس	الغرض
`MeanSquaredError()`	انحدار
`MeanAbsoluteError()`	انحدار
`RootMeanSquaredError()`	انحدار (بنفس وحدة الهدف)
`BinaryAccuracy()`	ثنائي
`CategoricalAccuracy()`	متعدّد (one-hot)
`SparseCategoricalAccuracy()`	متعدّد (أرقام)
`TopKCategoricalAccuracy(k=5)`	متعدّد
`Precision()`	الدقّة
`Recall()`	الاستدعاء
`F1Score(average="macro")`	F1
`AUC(from_logits=...)`	جودة الترتيب
`FalsePositives()` / `FalseNegatives()`	تحليل الأخطاء

5. المُحسّنات وجداول التعلّم

PYTHON

```python
keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0, weight_decay=1e-4)
keras.optimizers.AdamW(learning_rate=1e-3, weight_decay=1e-4)
keras.optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
keras.optimizers.RMSprop(learning_rate=1e-3, rho=0.9)
keras.optimizers.MultiOptimizer(
    optimizer_map={"encoder": enc_opt, "decoder": dec_opt})    # جديد في 3.15
```

PYTHON

```python
from keras.optimizers.schedules import (
    ExponentialDecay, PolynomialDecay, CosineDecay,
    PiecewiseConstantDecay, InverseTimeDecay,
)

schedule = ExponentialDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    decay_rate=0.9,
)
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=schedule),
    loss="categorical_crossentropy", metrics=["accuracy"])
```

> **تنبيه:** الجداول تحتسب الخطوات بـ**خطوات المُحسّن** لا الحقبات. مع 1000 عيّنة و `batch_size=32` و10 حقبات تكون الخطوات 320، لا 10.

6. الاستدعاءات

الاستدعاء	الغرض
`EarlyStopping(monitor, patience, restore_best_weights=True)`	إيقاف مبكر
`ModelCheckpoint(filepath, save_best_only=True)`	حفظ الأفضل
`ReduceLROnPlateau(factor=0.5, patience=3)`	خفض LR عند الركود
`TensorBoard(log_dir="logs")`	المراقبة (يتطلّب TensorFlow)
`CSVLogger(filename)`	سجل CSV
`LambdaCallback(on_epoch_end=...)`	إجراءات مخصّصة
`LearningRateScheduler(schedule)`	جدول LR مخصّص
`TerminateOnNaN()`	إيقاف عند NaN
`BackupAndRestore(backup_dir)`	استئناف بعد الانهيار

7. الطبقات المخصّصة

```python
class MyLayer(keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.w = self.add_weight(
            shape=(units,), initializer="zeros", trainable=True, name="w"
        )

    def call(self, inputs):
        return inputs * self.w

    def get_config(self):
        return {"units": self.units}
```

8. أخطاء شائعة في الاستدعاء

الخطأ	الصحيح
`from tensorflow import keras`	`import keras`
`model.save("m.h5")`	`model.save("m.keras")`
`model.save("saved_model")`	`model.export("saved_model")`
`loss="mse"` مع `from_logits=True`	اختر خسارة تدعم logits
`metrics=["accuracy"]`	`metrics=[keras.metrics.SparseCategoricalAccuracy()]`
`model.fit(x, y, validation_split=0.2, shuffle=True)`	قسّم يدويًا لتجنّب التسرّب
`Dense(10, activation="softmax")` + `from_logits=True`	اختر أحدهما
`Dropout` بعد كل `Conv2D`	`SpatialDropout2D`
`model.fit` على `tf.data` مع `shuffle=True` في السلاسل الزمنية	`shuffle=False`

9. مراجع سريعة داخل بايثون

```python
keras.__version__           # إصدار Keras
keras.backend.backend()     # الواجهة الخلفية الحالية
keras.config.set_backend("jax")
keras.utils.set_random_seed(42)
model.count_params()
model.layers[i].get_config()
keras.saving.load_model("m.keras", compile=False)
```

10. جدول تحويل سريع

أريد...	الحل
مخرجات احتمالية	`Dense(n, activation="softmax")` + `CategoricalCrossentropy(from_logits=False)`
مخرجات احتمالية مستقرة عدديًا	`Dense(n)` بلا تنشيط + `from_logits=True`
تسلسل ثابت الطول	`Masking()` + `padding="post"`
تحجيم الصور	`Rescaling(1/255)` داخل النموذج
تطبيع الخصائص (features)	`Normalization()` مع `adapt()`
استدعاء دالة مخصّصة	طبقة ترث من `keras.layers.Layer` (آمنة في `safe_mode`)
دمج مدخلين	`keras.layers.Concatenate()`
إيقاف مبكر	`EarlyStopping(..., restore_best_weights=True)`
حفظ أفضل نموذج	`ModelCheckpoint("best.keras", save_best_only=True)`
تصدير للهاتف	`model.export("m.tflite")`
تصدير متعدد المحرّكات	`m.onnx` → `tf2onnx` (راجع الفصل 37)
دقّة مختلطة	`keras.mixed_precision.set_global_policy("mixed_float16")`

أريد...	الحل
تجميع XLA	`model.compile(..., jit_compile=True)`
توزيع متعدد الأجهزة	`keras.distribution.DeviceMesh` (راجع الفصل 36)
انتباه متعدد الرؤوس	`keras.layers.MultiHeadAttention(num_heads, key_dim)`
انتباه سببيّ (توليد)	`MultiHeadAttention(..., use_causal_mask=True)`
ترميز موضعيّ	`keras.layers.Embedding(seq_len, d_model)`
تضمين نصّ	`TextVectorization` + `Embedding` (راجع الفصل 27)
نوافذ زمنيّة	`keras.utils.timeseries_dataset_from_array`
معايرة خسارة متوازنة	`CategoricalFocalCrossentropy(gamma=2)`
تدرّج يدويّ	`keras.GradientTape` (راجع الفصل 34)
طبقة قابلة للقناع	`self.supports_masking = True` + `call(inputs, mask=None)`

ملحق C: ورقة المعادلات

ملحق C – ورقة المعادلات

كل المعادلات الأساسية في الكتاب في صفحة واحدة. مرجع سريع للمراجعة قبل الاختبار أو المقابلة.

1. التحسين

النزول التدرّجي

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

حيث θ متجه المعاملات، وt الخطوة، وη معدل التعلّم، و$\nabla_\theta L$ تدرّج الخسارة بالنسبة للمعاملات.

SGD مع الزخم

$$v_t = \beta v_{t-1} + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

RMSProp

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}$$

Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

2. التهيئة

Xavier (Glorot)

$$W \sim U\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right)$$

He (Kaiming)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

3. دوال التنشيط

$$\text{ReLU}(z) = \max(0, z)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{GELU}(z) = z\,\Phi(z) \approx \frac{z}{2}\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(z + 0.044715 z^3\right)\right]\right)$$

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

4. دوال الخسارة

مربع الخطأ

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

الخطأ المطلق

$$L_{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

Huber

$$L_\delta = \begin{cases} \frac{1}{2}y^2 & |y| \le \delta \\ \delta\left(|y| - \frac{1}{2}\delta\right) & |y| > \delta \end{cases}$$

الانتروبيا المتقاطعة الثنائية

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

الانتروبيا المتقاطعة الفئوية

$$L_{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c} y_{i,c}\log \hat{y}_{i,c}$$

5. التنظيم

$$\|W\|_1 = \sum_j |W_j|, \qquad \|W\|_2^2 = \sum_j W_j^2$$

$$J = L + \lambda R(W)$$

6. الالتفاف

$$y_{i,j,k} = b_k + \sum_u \sum_v \sum_c K_{u,v,c,k}\, X_{i+u,\, j+v,\, c}$$

$$n_{out} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

$$k_{eff} = k + (k-1)(d-1)$$

$$N_{params} = k_h \times k_w \times C_{in} \times C_{out} + C_{out}$$

7. الانتباه

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

الترميز الموضعي الجيبي

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

8. الشبكات التكرارية

RNN

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad y_t = W_{hy}h_t + b_y$$

LSTM

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i) \quad \text{(إدخال)}$$

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f) \quad \text{(نسيان)}$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o) \quad \text{(إخراج)}$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

GRU

$$z_t = \sigma(W_z[x_t, h_{t-1}]), \qquad r_t = \sigma(W_r[x_t, h_{t-1}])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_h[x_t, r_t \odot h_{t-1}])$$

9. التطبيع

تطبيع الدفعة

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta$$

تطبيع الطبقة

$$\mu = \frac{1}{d}\sum_{j} x_j, \qquad \sigma^2 = \frac{1}{d}\sum_{j}(x_j - \mu)^2$$

10. مقاييس التصنيف

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot P \cdot R}{P + R}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

11. التقسيم والكشف

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}, \qquad \text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

12. التوليد

VAE — حدّ الـ KL

$$D_{KL}(q \| p) = \sum_{x} q(x) \log \frac{q(x)}{p(x)}$$

VAE — الخسارة

$$\mathcal{L} = \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] - D_{KL}(q(z|x)\|p(z))$$

GAN

$$\min_G \max_D \; \mathbb{E}_{x\sim data} \log D(x) + \mathbb{E}_{z\sim p(z)} \log(1 - D(G(z)))$$

الانتشار — العملية السابقة

$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right)$$

$$q(x_t|x_0) = \mathcal{N}\left(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\right)$$

13. سلاسل زمنية

النافذة

$$X = [x_{t-w}, \ldots, x_{t-1}], \qquad y = x_t$$

14. التطبيع الموزون

$$L = \frac{1}{\sum_i w_i} \sum_{i=1}^{N} w_i \ell_i$$

15. أوزان الأصناف

$$w_c = \frac{N}{K\, n_c}$$

16. الخسارة البؤرية

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

17. جداول التعلّم

أُسّي

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t / s \rfloor}$$

جيبي

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right)$$

18. دقة مختلطة

$$\text{loss} = \frac{1}{N} \sum_i \frac{L_i}{\max(1, |\nabla L_i|)}$$

19. مقياس PSNR

$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

ملحق D: إجابات مختارة للتمارين

ملحق D – إجابات مختارة للتمارين

> هذه إجابات **معيارية** لا إجابات نموذجية وحيدة. تمرينٌ واحد قد يقبل أكثر من مسارٍ صحيح؛ المعيار هو أن يصل الطالب إلى النتيجة نفسها بالتبرير نفسه.
>
> الأرقام في تمارين الأعداد تتقريبًا – قد تختلف في الخانة الأخيرة بحسب البذرة العشوائية.

الفصل 2 — البيئة

تمرين: تشخيص خطأ الواجهة الخلفية. الأعراض: `ModuleNotFoundError: No module named 'tensorflow'` عند `import keras` . السبب: Keras 3.15.1 يبحث عن المحرك الذي يحدّده `KERAS_BACKEND` ، وقيمة `tensorflow` هي الافتراضية في بعض البيئات. الحل: اضبط `KERAS_BACKEND=jax` (أو `torch`) قبل الاستيراد، أو ثبّت المحرك الذي تريده. **الدرس:** لا تفترض أن Keras يجلب محرّكه؛ فهو لا يفعل.

تمرين: تثبيت الإصدارات.

```text
keras==3.15.1
numpy>=1.26
```

اكتب `requirements.txt` بالنسخ المقيّدة (==) للمكتبات الحرجة، وبـ `>=` للتي لا يكسر واجهتها. اقرأ `pip freeze > requirements.lock.txt` بعد أول تثبيت ناجح واحتفظ به.

الفصل 3 — الأوتور

تمرين: Broadcasting. `a` شكله `(8, 1, 6, 1)` و `b` شكله `(7, 1, 5, 1)`. النتيجة `(8, 7, 6, 5)`. الشرح: يُوسَّع البُعد `1` إلى حجم البُعد المقابل (8، 7، 5)، ويُوسَّع البُعد `6` إلى `6` لأن البُعدين متساويان. الأبعاد المقارنة من اليمين: (1,1)↔(1,1) ✓، (6,1)↔(5,1) → 6↔5 غير متساويان لكن أحدهما 1 ✓، (1,1)↔(7,1) ✓، (8,1)↔(1,1) ✓.

تمرين: المتغيّرات غير القابلة للتعلّم. `model.non_trainable_variables` تحتوي على متغيّرات لا يحدّثها المُحسّن: أشهرها إحصاءات BatchNorm المتحرّكة (المتوسط والتباين). سبب وجودها أن هذه الإحصاءات تُحسب من الدفعة أثناء التدريب وتُجمَّع عبر الدفعات كمعدّل متحرّك، ثم تُستخدم **بدون بيانات** أثناء الاستدلال.

الفصل 5 — أول نموذج

تمرين: حساب عدد المعاملات يدويًا. `Dense(128)` على مُدخل شكله (784,):
$784 \times 128 + 128 = 100\,480$. `Dense(10)` على مُدخل شكله (128,): $128 \times 10 + 10 = 1\,290$.
المجموع: $101\,770$ معاملًا.

تمرين: كسر السكربت ثلاث ways. 1. `loss="sparse_categorical_crossentropy"` مع مخرجات `softmax` → استخدم `from_logits=True` أو أزل softmax. 2. `metrics=["accuracy"]` مع تسميات فردية → استخدم `keras.metrics.SparseCategoricalAccuracy()`. 3. `x_train` من نوع `uint8` بلا قسمة على 255 → quantile الميزات يتجاوز 1 ويؤخّر التعلّم.

الفصل 6 — الطبقات الأساسية

تمرين: حساب المعاملات. `Dense(64)` على (100,): $6\,464 = 100 \times 64 + 64$. `Dense(32)` على (64,): $2\,080 = 64 \times 32 + 32$. `Dense(10)` على (32,): $330 = 32 \times 10 + 10$. المجموع: $8\,874$.

تمرين: Dropout أم SpatialDropout؟ في شبكة تصنيف صور ذات أبعاد (batch, 64, 64, 32): استخدم `SpatialDropout2D` – لأنه يُسقط قنوات كاملة، وهو ما يفرّق الخصائص المجاورة، بينما `Dropout` يُسقط بكسلات متفرّقة فتُفسد بنية الفضاء. في شبكة MLP (أبعاد متجهة فقط) استخدم `Dropout` العادي.

الفصل 7 — دوال التنشيط

تمرين: حساب softmax يدويًا. لـ logits = (2.0, 1.0, 0.1): $e^{2.0} = 7.389$، $e^{1.0} = 2.718$، $e^{0.1} = 1.105$؛ المجموع = 11.212. الاحتمالات: 0.659، 0.242، 0.099. المجموع = 1.000 ✓. فئة التنبّؤ = الفهرس 0 (أكبر logit).

تمرين: خطأ softmax مع sigmoid. `Dense(3, activation="sigmoid")` مع `CategoricalCrossentropy` يعطي ثلاثة احتمالات مستقلّة قد يكون مجموعها 0.87 أو 1.31 – وهو ليس توزيعًا احتماليًا. استخدم `softmax` وحينها يكون المجموع 1 بالضبط.

الفصل 8 — دوال الخسارة

تمرين: حساب BCE. $y = (1, 0, 1)$، $\hat{y} = (0.9, 0.2, 0.7)$:

$L = -\frac{1}{3}[(1)\log 0.9 + (0)\log 0.2 + (1)\log 0.7] = -\frac{1}{3}[-0.105 - 0 - 0.357] = 0.154$.

تمرين: لماذا `from_logits`؟ لأن softmax داخل دالة الخسارة تُطبَّق على logits بأمان عددي (باستخدام log-sum-exp)، بينما softmax ثم log بنفسها تُنتج $p \to 0$ فيظهر $\log(0)$. مع `from_logits=True` نوفّر مرورًا على softmax منفصل – أسرع وأدق.

الفصل 9 — المُحسّنات

تمرين: تصحيح التدرّجات. المتوسط (naive momentum) = 0.054 (وليس 0.054 بالخطأ الشائع في جمع التدرّجات) – راجع: $\frac{0.1+0.3-0.2+0.0}{4} = 0.05$. هذا خطأ شائع: **لا تجمع** التدرّجات ثم تقسّم – بل

قسّم أولًا ثم اجمع.

تمرين: جدول التعلّم. `ExponentialDecay(initial_learning_rate=1e-3, decay_steps=10000, decay_rate=0.9)` يعطي بعد 10000 خطوة: $10^{-3} \times 0.9 = 9 \times 10^{-4}$.

تذكّر: `decay_steps` بالخطوات لا الحقبات.

الفصل 10 — المقاييس

تمرين: حساب F1. $TP = 80$, $FP = 20$, $FN = 40$. $P = 80/100 = 0.8$, $R = 80/120 = 0.667$.

$F1 = \frac{2 \times 0.8 \times 0.667}{0.8 + 0.667} = \frac{1.067}{1.467} = 0.727$.

تمرين: الدقّة مقابل F1. في مجموعة فيها 950 سالبًا و50 موجبًا: الدقّة 98% تعطي رضًا كاذبًا. $F1$ منخفض لأن الاستدعاء ضعيف. استخدم `F1Score(average="macro")` أو متوسط per-class.

الفصل 11 — التهيئة

تمرين: حدّ Glorot. $n_{in} = 784$, $n_{out} = 128$: $a = \sqrt{6/912} = 0.0811$. $n_{in} = 100$, $n_{out} = 10$:

$a = \sqrt{6/110} = 0.2335$.

تمرين: لماذا He يضاعف التباين؟ ReLU تُصفّر نصف الوحدات، فمتوسط المربعات يَنخفض إلى النصف. لتفادي ذلك يضاعف He التباين: $\text{Var} = 2/n_{in}$ بدل $1/n_{in}$، فيعود التباين بعد التنشيط إلى ما كان عليه قبله.

الفصل 12 — الاستدعاءات

تمرين: توقّع ترتيب التنفيذ. لكل دفعة: `on_train_batch_begin` ← التمرير الأمامي ← `on_train_batch_end` ← (حدّ الحقبة) `on_epoch_end`. ثم عند انتهاء التدريب: `on_train_end`.

الأهم: `on_epoch_end` يُستدعى **بعد** تحديث الأوزان في الحقبة كاملة، فالمؤشرات التي تراها فيه هي مؤشرات الحقبة المنتهية.

الفصل 13 — التنظيم

تمرين: لماذا L1 ينتج تباعدًا؟ لأن المنطقة الأفقية حول الصفر (الأساس $|W|$) لها ميل ثابت غير صفري، بينما ميل W^2 عند الصفر يساوي صفرًا. مع الدفع نحو الصفر، تبقى القيم الصغيرة من L1 عند الصفر تمامًا، بينما تدفع L2 القيم الصغيرة نحو الصفر ببطء.

الفصل 17 — الأوزان وتعدّد المهام

تمرين: حساب أوزان الأصناف. $N = 1000$، $K = 3$، $n_c = \{600, 300, 100\}$: $w_1 = \frac{1000}{3 \times 600} = 0.556$، $w_2 = \frac{1000}{3 \times 300} = 1.111$، $w_3 = \frac{1000}{3 \times 100} = 3.333$. كلما قلّ عدد عيّنات الصنف زاد وزنه.

الفصل 18 — رياضيات الالتفاف

تمرين: شكل الخرج. مُدخل (32, 32, 3) → `Conv2D(64, 3, padding="same", strides=1)` → (32, 32, 64). `MaxPooling2D(2)` → (16, 16, 64). `Conv2D(128, 3, padding="valid", strides=2)` → $\lfloor (16 - 3)/2 \rfloor + 1 = 7$ → (7, 7, 128).

تمرين: عدد المعاملات. `Conv2D(64, 3)` على مُدخل 3 قنوات: $3 \times 3 \times 3 \times 64 + 64 = 1\,792$. `SeparableConv2D(64, 3)` على نفس المدخل: $3 \times 3 \times 3 + 3 \times 64 + 64 = 283$ – أقل بـ6.3 أضعاف.

الفصل 21 — النقل والتدريب الدقيق

تمرين: اختيار الاستراتيجية. - 400 صورة لكل صنف (1200 إجمالًا) → استخراج ميزات + رأس جديد + تعزيز قوي. - 20 ألف صورة لكل صنف → تدريب دقيق متدرّج (افتح آخر 30% من الكتل). - 200 ألف+ صورة → تدريب دقيق كامل بمعدّل تعلّم صغير.

تمرين: حساب معاملات الرأس. `EfficientNetB0` بـ `include_top=False` يعطي متجهًا بـ 1280 خاصية. رأس تصنيف لـ 10 فئات: $1280 \times 10 + 10 = 12\,810$ معاملًا – نسبة ضئيلة من إجمالي النموذج (~5 مليون).

الفصل 22 — الكشف والتقسيم

تمرين: حساب IoU. $A = \{0,1,2,3,4\}$، $B = \{3,4,5,6,7\}$: $|A \cap B| = 2$، $|A \cup B| = 8$، $IoU = 0.25$. Dice: $2 \times 2/(5+5) = 0.4$. الحدّ المعتاد في التقسيم هو $IoU \geq 0.5$.

الفصل 25 — RNN وLSTM

تمرين: لماذا تنهار RNN؟ تدرّج الحالة يتضاعف في كل خطوة: $\partial h_t / \partial h_{t-k} = \prod_{i=0}^{k-1} W_{hh}^{\top} \tanh'(\cdot)$. وبما أن $|\tanh'(x)| \leq 1$، فإن $|\tanh'(x)|^k$ ينكمش أُسّيًا → تدرّج يتلاشى. LSTM تحلّ ذلك بناقل حالة (حالة الخلية) يسمح بمسار تدرّج قريب من 1.

الفصل 28 — السلاسل الزمنية

تمرين: كشف التسرّب. `shuffle=True` مع `validation_split` في سلسلة زمنية يخلط عينات الماضي بالمستقبل – تسرّب زمني. الحل: `shuffle=False` وتقسيم زمني صريح.

الفصل 29 — المحوّلات

تمرين: تعقيد الانتباه. انتباه كامل على طول n: $O(n^2 d)$. بـ $n = 8192$ و$d = 64$: $8192^2 \times 64 \approx 4.3 \times 10^9$ عملية – لكل رأس ولكل طبقة. الانتباه النافذي (sliding window) يحدّها إلى $O(n \cdot w \cdot d)$.

الفصل 31 — VAE

تمرين: لماذا نحتاج الـ KL؟ لولا حدّ الـ KL لـ fit-المُرمِّز كل مُدخل في نقطة وحيدة في فضاء z (فجوة بين التوليد المحلّي والتوزيع الناعم). حدّ الـ KL يدفع التوزيع المُتعلَّم $q(z|x)$ نحو $p(z)$ القياسي، فيصبح فضاء z متّصلًا وقابلًا للعينة.

الفصل 40 — الأخطاء الشائعة

تمرين: خسارة NaN. الأسباب مرتّبة بالاحتمال: (1) معدل تعلّم مرتفع جدًا مع `softmax` بلا `from_logits` ؛ (2) `log(0)` في دالة خسارة مخصّصة؛ (3) انفجار التدرّجات ← استخدم `clipnorm=1.0` ؛ (4) `BatchNormalization` مع دفعة فيها قيم ثابتة (مثلًا batch=1) في طبقة BN؛ (5) بيانات تحتوي `inf` أو `NaN` – افحصها بـ `np.isnan(x).any()` .

الفصل 23 — محوّل الرؤية

تمرين: لماذا يفشل ViT من الصفر على بيانات صغيرة؟ لأنّ CNN تحمل تحيّزًا استقرائيًّا مُغروسًا في البنية (تجاور محليّ + مشاركة أوزان) – معلومة مجانيّة لا تحتاج تعلّمًا. المحوّل لا يفترض شيئًا: كلّ علاقة رقعة-رقعة يجب أن تُتعلَّم من البيانات. ببيانات قليلة، هذه العلاقات تُتعلَّم بشكل ضوضائي فيعطي النموذج أداءً أقلّ من CNN. **الدرس:** التحيّز الاستقرائي ليس عيبًا – بل رافعة تستخدم بياناتك بكفاءة.

تمرين: أثر حجم الرقعة. رقعة 16×16 على صورة 224×224 ← 196 رقعة، كلفة انتباه $196^2 = 38\,416$. رقعة 8×8 ← 784 رقعة وكلفة $614\,656$ – أي 16 ضعفًا. لذلك تكبير الدقّة في ViT مكلف أُسّيًّا في الانتباه (وليس خطّيًّا كما في CNN).

الفصل 27 — معالجة اللغة عمليًّا

تمرين: لماذا `adapt()` على بيانات الاختبار تسريب؟ لأنّ `TextVectorization.adapt()` يبني المفردات من النصوص – فإن شملت بيانات الاختبار، فقد النموذج «شاهد» مفردات التقييم قبل أن يُقيَّم عليها. النتيجة: أداء يبدو جيدًا في التجارب ويتهالك في الإنتاج حيث تظهر كلمات جديدة (OOV). القاعدة: `adapt()` على التدريب فقط، وكلّ كلمة جديدة خارج المفردات تُعالَج برمز OOV.

تمرين: اختيار طول التسلسل. راجع توزيع الأطوال: خُذ quantile 95%. لو كان المتوسّط 30 كلمة والقيمة 95% هي 120، فاختر 120 لا 500. نسبة الحشو = $1 - \bar{L}/L_{chosen}$؛ إن تجاوزت 40% فالطول زائد وكلفة حساب هدر.

الفصل 32 — الشبكات التنافسيّة

تمرين: تشخيص انهيار الأنماط. المؤشّرات: (1) الصور المولّدة متشابهة جدًّا في كلّ دفعة؛ (2) تنوّع العيّنات المولّدة (متوسّط المسافة الزوجيّة) قريب من صفر؛ (3) خسارة المُميّز تهبط إلى قرب الصفر مع ثبات خسارة المولّد. **العلاج بالترتيب:** موازنة البنيتين ← خفض خطوات المُميّز ← WGAN-GP ← ضجيج في مُدخل المولّد.

الفصل 34 — الطبقات المخصّصة

تمرين: لماذا الأوزان في `build()` لا `__init__`؟ لأنّ شكل المُدخل لا يُعرف في `__init__` – الطبقة لم تُربط بشبكة بعد. `build(input_shape)` تُستدعى مرة واحدة عند أوّل نداء، حين يكون الشكل معروفًا. بناء الأوزان في `__init__` يُنتج طبقة تعمل في التدريب وتُفشل في الحفظ/التحميل (شكل غير محدّد في `.keras`).

الفصل 36 — تسريع التدريب

تمرين: لماذا `mixed_float16` يُبطئ على CPU؟ لأنّ CPU لا يملك Tensor Cores المخصّصة لعمليّات `float16`. الحساب في نصف الدقّة على CPU يتطلّب تحويلات إضافيّة (upcast/downcast) في كلّ طبقة، بينما الفائدة (نصف الذاكرة والسرعة) لا تتحقّق إلا على GPU متوافق. **القاعدة:** `mixed_float16` لتدريب GPU فقط؛ على CPU أبقِ `float32`.

تمرين: ترتيب التحسينات بالعائد. (1) `prefetch(AUTOTUNE)` – بلا كلفة، غالبًا 10-25%؛ (2) `cache()` إن كانت البيانات تناسب الذاكرة؛ (3) XLA/JIT – 10-30% على الرسوم الكبيرة؛ (4) `mixed_float16` على GPU – 1.5-2x؛ (5) توزيع عدّة GPUs – الحلّ الأخير لأنّه الأغلى تعقيدًا. لا تُطبّق تسريعًا إلّا بعد قياسه على بياناتك.

الفصل 41 — مشروع تصنيف صور

تمرين: لماذا المعايرة مهمّة في القرار؟ لأنّ العتبة (0.95 مثلاً) تُفهَم كـ«95% من الحالات بهذا الاحتمال صحيحة». إن لم يكن النموذج معايَرًا (reliability diagram بعيد عن القطريّ)، فالرقم لا يعني ما يبدو – والقرار (رفض عبّة) مبنيّ على تفسير خاطئ. الحلّ: قِس مصفوفة الموثوقيّة، وعدّل بالمعايرة الحراريّة (temperature scaling) على مجموعة التحقّق.

ملاحظة ختامية

لكل تمرين لم تجد له إجابة في هذا الملحق: أعد قراءة قسم الحل في فصله. القيم الرقمية في التمارين تعتمد على البذرة العشوائية – المهم هو **المنهج** لا الرقم.Last digit.

ملحق E: فهرس الأخطاء الشائعة

ملحق E – فهرس الأخطاء الشائعة

> مرتّب حسب **الأعراض** لا حسب الأسماء: ابدأ من ما تراه على الشاشة، لا من ما تظنّ أنه السبب. كل صف: العرض ← السبب ← الحل.

1. أخطاء الاستيراد والبيئة

العرض	السبب	الحل
`ModuleNotFoundError: No module named 'tensorflow'`	`KERAS_BACKEND=tensorflow` ولم تُثبّت TF	`pip install "keras[tensorflow]"` أو بدّل الواجهة
`No module named 'optree'`	تبعية ناقصة في Keras 3	`pip install optree`
`ImportError: cannot import name 'keras'` من tensorflow	محاولة `from tensorflow import keras`	استخدم `import keras`
`keras.backend.backend()` يعطي محرّكًا لا تريده	متغيّر بيئة قديم	`setx KERAS_BACKEND jax` أو `keras.config.set_backend("jax")`
النموذج يعمل في دفتر لكن يفشل في سكربت	أسبقية الاستيراد	استورد `keras` أولًا في الملف

2. أخطاء الشكل (Shape)

العرض	السبب	الحل
`operands could not be broadcast together`	تعارض أبعاد	اطبع `x.shape` قبل الطبقة
`expected axis -1 of input with shape ... to have dimension 784`	`Flatten` على مُدخل مسطّح أصلًا	احذف `Flatten` أو عدّل `input_shape`
`Input 0 of layer is incompatible with output`	شكل `Input` لا يطابق البيانات	اجعل `Input` بلا `batch_size` implied
`(None, 28, 28)` و `(None, 784)` معًا	خلط تنسيق `channels_last` و `channels_first`	ثبّت `data_format="channels_last"`
`KeyError: 'conv1'`	لا توجد طبقة بهذا الاسم	`model.layers` لعرض الأسماء الحقيقية

3. أخطاء التدريب

العرض	السبب	الحل
الخسارة `nan` من أول حقبة	`log(0)` أو `softmax` بلا `from_logits`	`from_logits=True`
الخسارة `nan` بعد حقبة	انفجار تدرّجات	`clipnorm=1.0` أو خفّض `lr`
الخسارة لا تتحسّن	`lr` كبير/صغير، أو شبكة بلا تنشيط	جرّب `lr=1e-3` ثم `1e-4`
الدقّة 50% بالضبط في تصنيف ثنائي	النموذج يتوقّع فئة واحدة	افحص توازن الأصناف
`OverwriteError: Can't save the model`	الملف موجود	`overwrite=True` أو احذف الملف
التدريب يتوقف مبكرًا بلا سبب	`EarlyStopping` بـ `patience=0`	اضبط `patience=3` أو أكثر
`AttributeError: 'NoneType' object has no attribute 'shape'`	طبقة لم تُبنَ بعد	مرّر `input_shape` أو `Input`

4. أخطاء التقييم والتنبّؤ

العرض	السبب	الحل
`predict` يعطي logits لا احتمالات	نموذج بلا `softmax`	`keras.ops.softmax(logits)`
الدقّة في `evaluate` لا تطابق يدويًا	`accuracy` على one-hot مقابل sparse	وحّد صيغة التسميات
نتائج عشوائية بين التشغيلات	لا بذرة عشوائية	`keras.utils.set_random_seed(42)`
`predict` يعطي شكل `(N,` `10)` مع تصنيف ثنائي	عدد المخرجات خاطئ	استخدم `Dense(1,` `activation="sigmoid")`

5. أخطاء التوفّر والذاكرة

العرض	السبب	الحل
`ResourceExhaustedError: OOM`	دفعة أو نموذج كبير	قلّل `batch_size`، استخدم `mixed_float16`
`CUDA out of memory` أثناء `fit`	تسريب ذاكرة من `predict`	استدعِ `keras.backend.clear_session()`
بطء شديد رغم وجود GPU	البيانات على المعالج	انقل `x` و `y` إلى الذاكرة (`device` أو `.numpy()` بمصفوفات)
انقطاع العملية بلا رسالة	نفاد ذاكرة المضيف	راقب استهلاك RAM للبيانات

6. أخطاء الحفظ والتصدير

العرض	السبب	الحل
`ValueError: Invalid filepath extension`	امتداد غير مدعوم	استخدم `.keras`
`OSError: Unable to open file`	مسار غير موجود	أنشئ المجلد
الملف المحفوظ لا يُحمَّل	`custom_objects` ناقص	`keras.saving.load_model("m.keras", custom_objects={...})`
`NotImplementedError: TensorFlow is required for export`	التصدير يحتاج TF	ثبّت `tensorflow` أو صدّر `.onnx` / `.torch`
نموذج محفوظ بصيغة `.h5` لا يحمل المعالجة المسبقة	`.h5` لا يحفظ الطبقات غير Weighs	استخدم `.keras`

7. أخطاء الدقة والمعالجة

العرض	السبب	الحل
دقّة train عالية جدًا وval منخفضة	فرط تخصيص	+ Dropout + augmentation بيانات أكثر
الدقّة لا تتجاوز 60% في تصنيف صور	معالجة مسبقة ناقصة	`Rescaling(1/255)` + تطبيع
train عالية وval متساوية	invariant-	تحقق من الـ batch norm والتهيئة
التسميات تختلف في الشكل	خلط `one-hot` و `sparse`	اختر خسارة تطابق التسمية
النصوص لا تُرمَّز	`TextVectorization` بلا `adapt()`	نادِ `adapt()` أولًا

8. أخطاء السلاسل الزمنية

العرض	السبب	الحل
دقّة ممتازة على val وغيبوبة على test	تسرّب زمني	`shuffle=False` + تقسيم زمني
تنبّؤات ثابتة	الشبكة تعلّمت المتوسط	أضف features إضافية، تحقّق من النافذة
RMSE كبير على سلاسل متوسّطة	عدم التطبيع	`StandardScaler` على التدريب فقط

9. أخطاء شائعة في Keras 3 تحديدًا

العرض	السبب	الحل
`from tensorflow import keras` يفشل	Keras 3 مستقل	`import keras`
`model.save("saved_model")` يفشل	Keras 3 لا يحفظ SavedModel	`model.export("saved_model")`
`set_model` غير موجود	تغيير في دورة الـ callbacks	استخدم `self.model` مباشرة
`set_params` غير موجود	تغيير في دورة الـ callbacks	لا حاجة إليه
`ImageDataGenerator` محذوف	حُلّ من Keras 3	استخدم `keras.layers.RandomFlip` و`keras.layers.RandomRotation`
`keras.preprocessing.text` غير موجود	حُلّ من Keras 3	`keras.layers.TextVectorization`

10. أخطاء «الأداء الرهيب»

العرض	السبب	الحل
التدريب بطيء والـ GPU خامل	خط أنابيب البيانات هو عنق الزجاجة	`map` + `prefetch` متوازٍ + `cache`
كل حقبة تستغرق ثوانٍ	`batch_size` صغير جدًا	ارفعه إلى 64-256
أول حقبة بطيئة جدًا	تهيئة/تجميع	طبيعي في Keras 3
ذاكرة المعالج ممتلئة	بيانات محمّلة كلها	`tf.data.Dataset` مع `cache`
التعلّم غير مستقر	`lr` كبير	`ReduceLROnPlateau` أو `CosineDecay`

11. فحص سريع

قبل أن تبحث طويلًا، شغّل هذا:

PYTHON

```python
import numpy as np
import keras

print("keras:", keras.__version__)
print("backend:", keras.backend.backend())

x = np.random.rand(8, 4).astype("float32")
y = np.random.randint(0, 2, size=8)

model = keras.Sequential([
    keras.layers.Input((4,)),
    keras.layers.Dense(8, activation="relu"),
    keras.layers.Dense(2),
])
model.compile(optimizer="adam",

loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=[keras.metrics.SparseCategoricalAccuracy()])
model.fit(x, y, epochs=1, verbose=0)
print("shape:", model.predict(x, verbose=0).shape)
print("params:", model.count_params())
print("OK ✓")
```

إن نجح هذا، فالمشكلة في بياناتك أو بنية نموذجك – لا في البيئة.

ملحق F: المراجع والمصادر

ملحق F – المراجع والمصادر

> مصادر رُوجعت لصدقة 2026. الروابط التي لا تتطلّب حسابًا مُقيَّدًا قُرئت مباشرة؛ والمصادر التي تتطلّب تسجيلًا مُشار إليها بوضوح.

1. التوثيق الرسمي

المصدر	الرابط	لماذا تفتحه
توثيق Keras 3 الكامل	`https://keras.io/api/`	مرجع كل الأصناف والدوال
أدلة Keras (Guides)	`https://keras.io/guides/`	أمثلة رسمية مُختبَرة لكل موضوع
صفحة Keras 3.0	`https://keras.io/keras_3/`	شرح تغييرات الإصدار الثالث
مرجع Keras 3 على ReadTheDocs	`https://keras.io/api/`	بديل سريع
إصدارات Keras على GitHub	`https://github.com/keras-team/keras/releases`	سجلّ ما تغيّر في كل إصدار
شيفرة Keras المصدرية	`https://github.com/keras-team/keras`	حين تبحث عن سلوك غامض

2. التثبيت والواجهات الخلفية

المصدر	الرابط
PyPI – Keras	`https://pypi.org/project/keras/`
TensorFlow	`https://www.tensorflow.org/install`
JAX	`https://docs.jax.dev/en/latest/installation.html`
PyTorch	`https://pytorch.org/get-started/locally/`
OpenVINO	`https://docs.openvino.ai/`
KerasHub	`https://keras.io/keras_hub/`

3. المراجع العلمية الأساسية

المرجع	MLA	ماذا يشرح
Chollet, F. (2015)	*Deep Learning with Python*, Manning	الطريق إلى Keras وTensorFlow
Goodfellow, I., Bengio, Y. & Courville, A. (2016)	*Deep Learning*, MIT Press	الأساس النظري (الفصلان 3 و7)
Géron, A. (2022)	*Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed., O'Reilly	أقوى كتاب تطبيقي مرجعي
He, K. et al. (2015)	*Delving Deep into Rectifiers*, ICCV	تهيئة He سبب نجاح ReLU (الفصل 11)
Hochreiter, S. & Schmidhuber, J. (1997)	*Long Short-Term Memory*, Neural Computation	بنية LSTM (الفصل 25)
Vaswani, A. et al. (2017)	*Attention Is All You Need*, NeurIPS	بنية المحوّل (الفصل 29)
He, K. et al. (2016)	*Deep Residual Learning for Image Recognition*, CVPR	الوصلات المتبقّية (الفصل 21)
Dosovitskiy, A. et al. (2021)	*An Image Is Worth 16x16 Words*, ICLR	محوّل الرؤية (الفصل 23)
Kingma, D. & Welling, M. (2014)	*Auto-Encoding Variational Bayes*, ICLR	VAE (الفصل 31)
Goodfellow, I. et al. (2014)	*Generative Adversarial Nets*, NeurIPS	GAN (الفصل 32)

المرجع	MLA	ماذا يشرح
Ho, J. et al. (2020)	*Denoising Diffusion Probabilistic Models*, NeurIPS	الانتشار (الفصل 33)
Srivastava, N. et al. (2014)	*Dropout*, JMLR	Dropout (الفصل 13)
Lin et al. (2017)	*Batch Normalization*, ICML	تطبيع الدفعة (الفصل 13)
Loshchilov & Hutter (2019)	*Decoupled Weight Decay Regularization*, ICLR	لماذا AdamW (الفصل 9)
Devlin et al. (2019)	*BERT*, NAACL	BERT (الفصل 29)
Redmon et al. (2016)	*You Only Look Once*, CVPR	YOLO (الفصل 22)
Ronneberger et al. (2015)	*U-Net*, MICCAI	U-Net (الفصل 22)
Howard et al. (2019)	*Searching for MobileNetV3*, ICCV	EfficientNet/MobileNet (الفصل 21)

4. أدوات مفيدة

الأداة	الاستخدام	الرابط
TensorBoard	مراقبة التدريب	`pip install tensorboard`
Weights & Biases	تتبّع التجارب	`https://wandb.ai`
MLflow	إدارة النماذج	`https://mlflow.org`
ONNX Runtime	استدلال ONNX	`https://onnxruntime.ai`
Netron	معاينة نماذج ONNX	`https://netron.app`
Excalidraw	رسم المخططات	`https://excalidraw.com`
fast.ai	دروس عملية مجانية	`https://course.fast.ai`

5. مجتمعات

المجتمع	الرابط	الطبيعة
GitHub Discussions (Keras)	`https://github.com/keras-team/keras/discussions`	أسئلة وأجوبة رسمية
Keras Slack / Discord	عبر `https://keras.io`	نقاشات مباشرة
Stack Overflow – [keras]	`https://stackoverflow.com/questions/tagged/keras`	أسئلة مُتراكمة
r/MachineLearning	`https://reddit.com/r/MachineLearning`	نقاشات عامة
Papers with Code	`https://paperswithcode.com`	تنفيذ مرجعي لكل ورقة

6. مجموعات البيانات

المجموعة	الوصف	Keras API
MNIST	أرقام مكتوبة 28×28	`keras.datasets.mnist`
Fashion-MNIST	أزياء 28×28	`keras.datasets.fashion_mnist`
CIFAR-10	10 فئات 32×32	`keras.datasets.cifar10`
CIFAR-100	100 فئة	`keras.datasets.cifar100`
IMDB	مراجعات أفلام نصية	`keras.datasets.imdb`
Reuters	أخبار نصية	`keras.datasets.reuters`
California Housing	أسعار المنازل	`keras.datasets.california_housing`

> **ملاحظة:** بعض مجموعات `keras.datasets` القديمة (مثل `boston_housing`) أُزيلت أو وُضعت تحت إشعار إهمال لأن مصدرها لم يعد متاحًا قانونيًا. عند اختيار مجموعة تحقّق من مصدرها وحالتها.

7. كيف Staying محدثًا

1. **راجع سجلّ الإصدارات** في `github.com/keras-team/keras/releases` كل بضعة أشهر.
2. **تابع KerasHub** للنماذج الجاهزة الجديدة.
3. **اختبر الترقيات** على نسخة صغيرة من مشروعك قبل الترقية الكاملة.

4. **ثبّت الإصدارات** في بيئة الإنتاج، واقرأ «ما الجديد» قبل الترقية.
5. **تابع Issue Tracker** للكتبات التي تعتمد عليها.

8. اقتباسات مفيدة

> «الشبكة العصبية ليست معجزة؛ إنها ملفوفات خطّية متكرّرة مع دالة غير خطية بينها. كل ما تعلّمته في هذا الكتاب هو تفاصيل هذا الملفوف.»
>
> – خلاصة الفصل 7
>
> «المصفوفة التي لا تستطيع تفسيرها، لا تستطيع تصحيحها.»
>
> – مبدأ grads debugging في الفصل 39
>
> «النموذج الذي لا تستطيع قياسه، لا تستطيع تحسينه – تحسّن القياس قبل التحسين.»
>
> – الفصل 10

9. شكر وتقدير

شكر لكل من راجع المسودات، ولكل من سأل سؤالًا صعبًا في المنتدى: **الأسئلة الجيدة هي التي تصنع المراجع الجيدة.** وأهدي هذا العمل لكل من يقضي سهرات في debugger – لأن هذه السهرات هي التي علّمتني ما بين السطور.